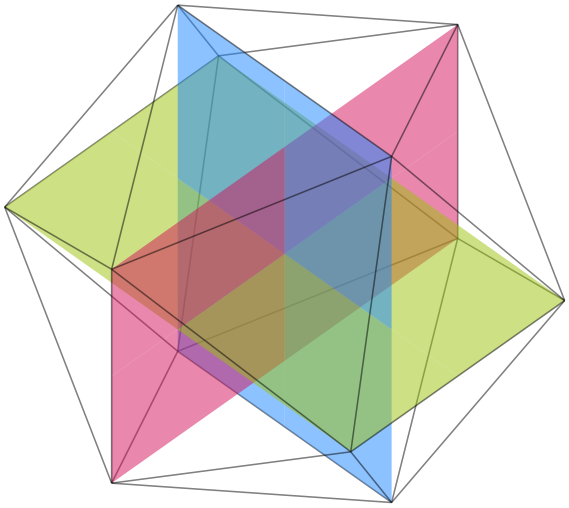
SYMMETRY

Am Anfang war die Symmetrie – In the beginning was symmetry!

Werner Heisenberg, *Der Teil und das Ganze*:
Gespräche im Umkreis der Atomphysik, 1969,
English translation, *Physics and Beyond*, 1971.

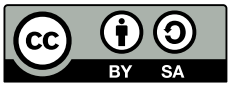


by

Marc Bezem
Ulrik Buchholtz
Pierre Cagne
Bjørn Ian Dundas
Daniel R. Grayson

Book version: `05e244f` (2026-09-16)

Copyright © 2025 by Marc Bezem, Ulrik Buchholtz, Pierre Cagne,
Bjørn Ian Dundas, and Daniel R. Grayson. All rights reserved.



This work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License. To view a copy of this license, visit:
http://creativecommons.org/licenses/by-sa/4.0/

This book is available at: https://unimath.github.io/SymmetryBook/book.pdf

To cite the book, the following BibTeX code may be useful:

```
@misc{Symmetry,
   title         = {Symmetry},
   author        = {Marc Bezem and Ulrik Buchholtz and Pierre Cagne
                    and Bjørn Ian Dundas and Daniel R. Grayson},
   date          = {2026-09-16},
   howpublished  = {\url{https://github.com/UniMath/SymmetryBook}},
   note          = {Commit: \texttt{05e244f}}
}
```

Short contents

Short contents · iii

Contents · v

1 *Introduction to the topic of this book* · 1

2 *An introduction to univalent mathematics* · 8

3 *The universal symmetry: the circle* · 64

4 *Groups, concretely* · 99

5 *Actions* · 122

6 *A categorical interlude* · 150

7 *Groups, abstractly* · 168

8 *Constructing groups* · 184

9 *Normal subgroups and quotients* · 197

10 *Finite groups* · 225

11 *Group presentations* · 232

12 *Abelian Groups* · 241

13 *Rings, fields and vector spaces* · 251

14 *Geometry and groups* · 265

15 *Galois theory* · 273

A *Historical remarks* · 277

B *Metamathematical remarks* · 278

Bibliography · 288

Glossary · 291

Index · 294

Contents

Short contents · iii

Contents · v

1 *Introduction to the topic of this book* · 1

2 *An introduction to univalent mathematics* · 8

2.1 What is a type? · 8
2.2 Types, elements, families, and functions · 9
2.3 Universes · 12
2.4 The type of natural numbers · 13
2.5 Identity types · 15
2.6 Product types · 19
2.7 Identifying elements in members of families of types · 20
2.8 Sum types · 22
2.9 Equivalences · 23
2.10 Identifying pairs · 26
2.11 Binary products · 28
2.12 More inductive types · 28
2.13 Univalence · 32
2.14 Heavy transport · 33
2.15 Propositions, sets and groupoids · 34
2.16 Propositional truncation and logic · 39
2.17 More on equivalences; surjections and injections · 41
2.18 Decidability, excluded middle and propositional resizing · 44
2.19 The replacement principle · 45
2.20 Predicates and subtypes · 46
2.21 Pointed types · 48
2.22 Operations that produce sets · 50
2.23 More on natural numbers · 54
2.24 The type of finite sets · 57
2.25 Type families and maps · 59
2.26 Higher truncations · 61
2.27 Higher structure: stuff, structure, and properties · 62

3 *The universal symmetry: the circle* · 64

3.1 The circle and its universal property · 64
3.2 The integers · 67
3.3 Set bundles · 68
3.4 The symmetries in the circle · 73
3.5 A reinterpretation of the circle · 75

3.6 Connected set bundles over the circle · 79
3.7 Interlude: combinatorics of permutations · 87
3.8 The mth root: set bundles over the components of Cyc · 88
3.9 Higher images · 92
3.10 Universal property of Cyc_n · 96
3.11 Getting our cycles in order · 98

4 Groups, concretely · 99
4.1 Brief overview of the chapter · 99
4.2 The type of groups · 100
4.3 Abstract groups · 106
4.4 Homomorphisms · 108
4.5 The sign homomorphism · 113
4.6 Bicycles · 116
4.7 Infinity groups (∞-groups) · 120

5 Actions · 122
5.1 Brief overview of the chapter · 122
5.2 Group actions (G-sets) · 122
5.3 Subgroups · 128
5.4 Invariant maps and orbits · 134
5.5 The classifying type is the type of torsors · 141
5.6 Any symmetry is a symmetry in Set · 145
5.7 The lemma that is not Burnside's · 147

6 A categorical interlude · 150
6.1 Brief overview of the chapter · 151
6.2 Categories · 151
6.3 Abstract notions and duality · 155
6.4 Functors and natural transformations · 156
6.5 Adjunctions · 159
6.6 The Yoneda Lemma · 161
6.7 Equivalences of categories · 162
6.8 Limits and colimits · 165
6.9 Monoidal categories · 167

7 Groups, abstractly · 168
7.1 Brief overview of the chapter · 168
7.2 Monoids and abstract groups · 168
7.3 Abstract homomorphisms · 171
7.4 Groups: from abstract to concrete and back · 172
7.5 Homomorphisms, from abstract to concrete and back · 177
7.6 Benefits from having the categorical equivalence · 180
7.7 Actions, from abstract to concrete and back · 181
7.8 Delooping groups and homomorphisms via the Rezk completion · 182
7.9 Heaps (†) · 182

8 Constructing groups · 184
8.1 Brief overview of the chapter · 184

8.2 Semidirect products · 184
8.3 Wreath products · 188
8.4 The pullback · 188
8.5 Pushouts of types · 190
8.6 Sums of groups · 190
8.7 Free groups · 194

9 *Normal subgroups and quotients* · 197
9.1 Brief overview of the chapter · 197
9.2 Epimorphisms, and an equivalent take on monomorphisms · 197
9.3 Images, kernels and cokernels · 200
9.4 The action on the set of subgroups · 210
9.5 Normal subgroups · 211
9.6 Intersecting with normal subgroups · 216
9.7 Automorphisms of groups · 217
9.8 The Weyl group · 220
9.9 The isomorphism theorems · 222
9.10 More about automorphisms · 222

10 *Finite groups* · 225
10.1 Brief overview of the chapter · 226
10.2 Lagrange's theorem, counting version · 226
10.3 Cauchy's theorem · 228
10.4 Sylow's Theorems · 229

11 *Group presentations* · 232
11.1 Brief overview of the chapter · 232
11.2 Graphs and Cayley graphs · 233
11.3 Examples · 236
11.4 Subgroups of free groups · 236
11.5 Intersecting subgroups · 239
11.6 Connections with automata (*) · 239

12 *Abelian Groups* · 241
12.1 Brief overview of the chapter · 241
12.2 Abelian groups · 241
12.3 Homomorphisms of abelian groups · 249
12.4 The category of abelian groups · 250
12.5 Direct sums and reduced wreath products · 250
12.6 Stabilization · 250

13 *Rings, fields and vector spaces* · 251
13.1 Rings, abstract and concrete · 251
13.2 vector spaces · 264
13.3 the general linear group as automorphism group · 264
13.4 determinants (†) · 264
13.5 examples: rationals, polynomials, adding a root, field extensions · 264

13.6 ordered fields, real-closed fields, pythagorean fields, euclidean fields · 264
13.7 complex fields, quadratically closed fields, algebraically closed fields · 264

14 Geometry and groups · 265
14.1 Inner product spaces · 265
14.2 Euclidean spaces · 266
14.3 Geometric objects · 268
14.4 The icosahedron · 269
14.5 Frieze patterns · 269
14.6 Incidence geometries and the Levi graph · 269
14.7 Affine geometry · 269
14.8 Inversive geometry (Möbius) · 271
14.9 Projective geometry · 271

15 Galois theory · 273
15.1 Covering spaces and field extensions · 273
15.2 Intermediate extensions and subgroups · 275
15.3 separable/normal/etc. · 276
15.4 fundamental theorem · 276

A Historical remarks · 277

B Metamathematical remarks · 278
B.1 Equality by definition · 279
B.2 The Limited Principle of Omniscience · 280
B.3 Topology · 281
B.4 Choice for finite sets (†) · 281

Bibliography · 288

Glossary · 291

Index · 294

1

Introduction to the topic of this book

ch:intro

> *Poincaré sagte gelegentlich, dass alle Mathematik eine Gruppengeschichte war. Ich erzählte ihm dann über dein Programm, das er nicht kannte.*
>
> *Poincaré was saying that all of mathematics was a tale about groups. I then told him about your program, which he didn't know about.*
>
> (Letter from Sophus Lie to Felix Klein, October 1882)

Since this book is called "Symmetry" it is reasonable to hope that by the time you've reached the end you'll have a clear idea of what symmetry means.

Ideally the answer should give a solid foundation for dealing with questions about symmetries. It should also equip you with language with which to talk about symmetries, making precise – but also reflecting faithfully – the intuition humans seem to be born with.

So, we should start by talking about how one intuitively can approach the subject while giving hints about how this intuition can be made into the solid, workable tool, which is the topic of this book.

What is symmetry?

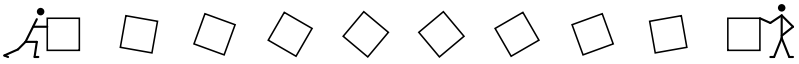
When we say that something is "symmetric" or possesses many "symmetries", we mean that the thing remains unchanged, even if we "do things to it." The best examples to begin with is if the something is some shape, for instance this square □. Rotating by 90 degrees doesn't change it, so we may say that "rotation by 90 degrees is a symmetry of □" Of course, rotating by 90 degrees will move individual points in □, but that is not of essence – the shape remains the same. However, the outcome of rotating by 360 degree or not at all is the same - even from the point of view of each individual point in □ – so it probably feels contrived to count rotations by 0 and 360 degrees as different rotations.

It feels reasonable to consider the rotations by 0°, 90°, 180°, and 270° to be all the (rotational) symmetries of □. Two thoughts may strike you:

(1) are these *all* the symmetries?

(2) "rotation" indicates a *motion*, through different squares, joining □ with itself via a "journey in the world of squares".

    The following cartoon animates a rotation of □ by 90°. The center of the square should be thought of as being in the same place all the time.



How is that reconcilable with a precise notion of symmetry?

The answer to the first question clearly depends on the context. For example, if we allow reflections the answer is "no". Each context has its own answer to what the symmetries of the square are.

Actually, the two questions should be seen as connected. If a symmetry of □ is like a round trip (loop) in the world (type) of squares, what symmetries are allowed is dependent on how big a "world of squares" we consider. Is it, for instance, big enough to contain a loop representing a reflection?

We argue that in order to pin down the symmetries of a thing (a "shape"), all you need to do is specify

(1) a type X (of things), and

(2) the particular thing x (in X).

It is (almost) that simple!

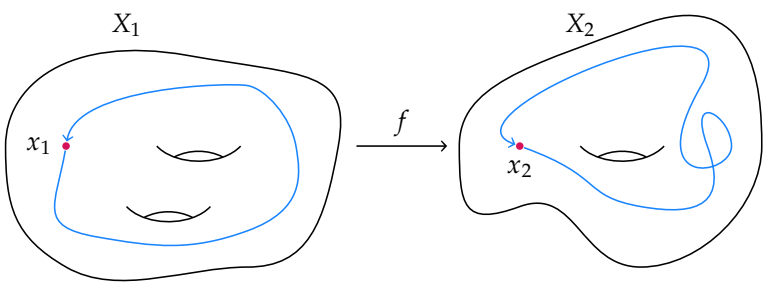
Note that this presupposes that our setup is strong enough to support the notion of a round trip.

From "things" to mathematical objects

Different setups have different advantages. The theory of sets is an absolutely wonderful setup, but supporting the notion of a round trip in sets requires at the very least developing fields like *mathematical analysis*, *topology* and *homotopy theory*, which (while fun and worthwhile in itself) is something of a detour.

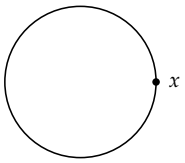
The setup we adopt, homotopy type theory, or univalent foundations, seems custom-built for supporting the notion of a round trip of a thing x in a type X. We get support for important operations on round trips of x: one can do such round trips after another (composition), one can go any round trip in the reversed direction (inverse), and there is always the trivial round trip of staying in place (unit). This provides round trips with a structure that is called a *group* in mathematics, satisfying all the properties that these operations ought to have.

In practice, one of the most important things is to be able to *compare* symmetries of "thing 1" and "thing 2". In our case this amounts to nothing but a function, $f : X_1 \to X_2$, that takes thing 1, x_1 in X_1, to thing 2, x_2 in X_2.

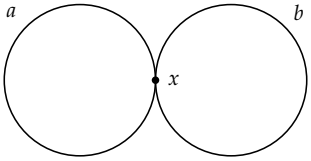


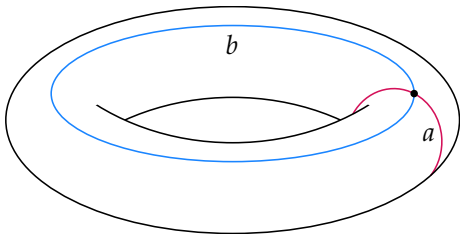
While such comparisons of symmetries are traditionally handled by something called a *group homomorphism* which is a function satisfying a rather long list of axioms, in our setup the only thing we need to know of the function is that it really does take thing 1 to thing 2 – everything else then follows naturally.

Some important examples have provocatively simple representations in this framework. For instance, consider the circle shown in the margin, with one designated point x on it. Since symmetries of x are interpreted as loops, you see that you have a loop for every integer – the number 7 can be represented by looping seven times counterclockwise. As we shall see, in our setup any loop in the circle is naturally identified with a unique integer (the *winding number* if you will). Everything you can wish to know about the structure of the *group of integers* is built-in in the circle.



Another example is the *free group of words in two letters a and b*. This is represented by the figure eight in the margin. In order to be able to distinguish the two circles we call them a and b, with the point x as the (only) point on both. The word ab^2a^{-1} is represented by looping around circles a and b respectively 1, 2 and -1 times in succession – notice that since the b^2 is in the middle it prevents the a and the a^{-1} from meeting and cancelling each other out. If you wanted the *abelian* group on the letters a and b (where a and b are allowed to move past each other), you should instead look at the torus:





Just why this last example works can remain a puzzle for now.

The importance of the ambient type X "of things"

In many situations, the type X "of things" can be more difficult to draw, or to define mathematically. For instance, what is the "type of all squares" which we discussed earlier, representing all rotational symmetries of □? You have perhaps already visualized it as the type of all squares in the plane, with □ being the shape the loop must start and stop in. This idea works well for the *oriented square* depicted in the margin. Note that the only reflective symmetry of the oriented square is reflection in the center – and the outcome is the same as a rotation by 180°. However, for □ we would get reflective symmetries that are not rotations. It is actually a little difficult to come up with a simple geometry of the plane that gives exactly the rotational symmetries of □. Later in the book, we will first pursue an algebraic approach, using that any rotational symmetry of □ can be reached by doing the 90°-rotation a few times, together with the fact that taking any loop four times reduces to not doing anything at all: they represent the *cyclic group of order four*.



A by-product of this line of thinking is the distinguished position of the circle. To express this it is convenient to give names to things: let •

(i.e., a dot) be the chosen base point in the circle and ↻ the loop winding once around the circle counterclockwise. Then a symmetry of a shape x_0 in X is uniquely given by the image of ↻ under a function $S^1 \to X$ taking • to x_0. So,

> the study of symmetries is the study of (pointed) functions between types of things, with the circle being the type that gives you access to individual symmetries.

This is similar to the idea of replacing membership in a set S by function from a one-point set 1 into S: a point s in S is uniquely given by the function $1 \to S$ taking the value s.

Just as you don't need much information about the one-point set to get this to work, you don't need much information about the circle to embark on a study of symmetries. Essentially you need to know of • and ↻, and that there is no "hidden relation" between the symmetries of •. Contrast this to the type of squares which has such a "hidden relation": where we identified a 360° rotation with doing nothing. This point of view has the benefit of being readily formalized while offering geometric intuition.

Symmetries have natural scopes

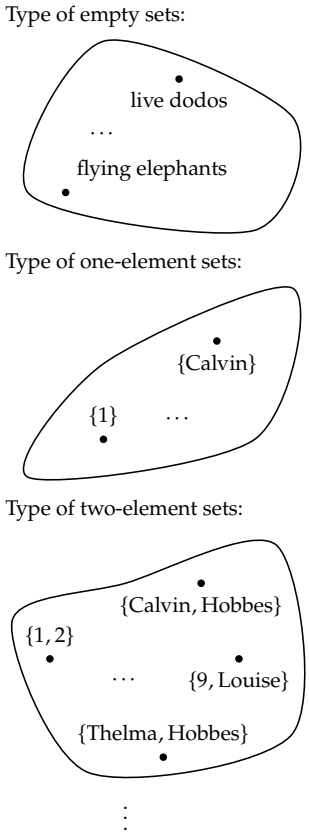
The natural scope of the symmetries of a thing x in a type X are the things in X that can be reached from x by a journey in X.

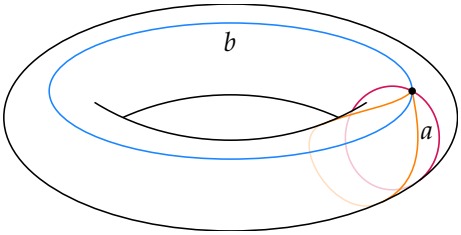
Let's make this precise with an example. In our setup, as a consequence of univalence, journeys from one set to another in the type of sets are uniquely given by one-to-one correspondences between these sets, commonly called *bijections*.

Now consider the set $\{1, 2, 3\}$. Then a symmetry of $\{1, 2, 3\}$ in the type of finite sets amounts to the same thing as a symmetry of $\{1, 2, 3\}$ in the type of sets with three elements: a symmetry of $\{1, 2, 3\}$ will not "pass through" sets that have, say, five elements. Think of the type of finite sets as being the disjoint union of all the types of sets with n-elements, where $n = 0, 1, 2, \ldots$: if a symmetry is a loop it should not be allowed to jump between the type of sets with three elements and the type of sets with five elements.

In fact, *any* type X can be naturally divided into "components": each element x_0 in X belongs to one and only one component, and the one x_0 belongs to we call $X_{(x_0)}$, and the symmetries of x_0 in X may be identified with the symmetries of x_0 in $X_{(x_0)}$. Hence from the perspective of symmetries of x_0 only the component containing it matters, and we confine our discussion to "connected" types of things, i.e., those having just one component.

The geometric intuition also points to the possibility of seemingly different symmetries being identified: when looping once around the circle it shouldn't matter "how" or "how fast" you do it. Consider the picture of the abelian group on two letters a and b from before, but now together with a more frivolous loop (in pink) homotopic to a:





You might think of a symmetries of x_0 as a rubber band confined to the circle and pinned to x_0. In the picture we've drawn such a rubber band (in orange) which can be deformed to a, and this deformation we consider as an *identification of the two symmetries*. In the language we adopt, this is hard-wired, and so our arguments are independent of any picture: pictures serve only as inspirations and are very helpful when trying to discover proofs.[1]

Our use of univalent foundations has several advantages. Roughly, univalence is the assertion that two types are "equivalent" if and only if there is a "path" (called an "identification") between them in the (large) "type of types". In group theory, two groups share exactly the same properties if there is an "isomorphism" between them (an invertible homomorphism), and with univalent foundation this is manifested by the isomorphism corresponding to a path between the groups in the type of groups. Hence we can use this path to transport any theorem about one group to the other: the two groups are "identified". The power of univalence is hard to overstate; it will simplify many proofs and make many statements accessible that otherwise would have been out of reach.

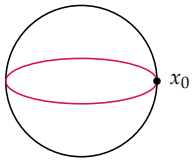
There are many kinds of symmetry and many ways of studying it. Euclidean plane geometry is the study of properties that are invariant under rigid motions of the plane. Other kinds of geometry arise by considering other notions of transformation. Univalent mathematics gives a new perspective on symmetries: Motions of the plane are forms of identifying the plane with itself in possibly non-trivial ways. It may also be useful to consider different presentations of planes (for instance as embedded in a common three-dimensional space) and different identifications between them. For instance, when drawing images in perspective we identify planes in the scene with the image plane, not in a rigid Euclidean way, but rather via a perspectivity (see Figure 1.1). This gives rise to projective geometry.

Does that mean that a plane from the point of view of Euclidean geometry is not the same as a plane from the point of view of projective or affine geometry? Yes. These are of different types, because they have different notions of identification, and thus they have different properties.

Here we follow Quine's dictum: No entity without identity! To know a type of objects is to know what it means to identify representatives of the type. The collection of self-identifications (self-transformations) of a given object form a *group*.

Group theory emerged from many different directions in the latter half of the 19th century. Lagrange initiated the study of the invariants under permutations of the roots of a polynomial equation $f(x) = 0$, which culminated in the celebrated work of Abel and Galois, proving

[1] There's a subtle point, which may be a source of confusion if brushed under the carpet: a priori there could be "several ways" in which two symmetries should be identified. For many purposes this poses no problem, but we want to present a theory that mirrors the classical theory faithfully, and so restrict our "types of things" where there aren't multiple ways of identifying symmetries. The technical term – when we get that far will be "pointed connected groupoids". This means disallowing types like the sphere:



There are fundamentally different ways of identifying the symmetry represented of x_0 by the equator with the trivial symmetry: when thought of as a rubber band the equator can contract either over the north or the south poles (or more complicated ways). There's something called "truncation" which can fix any type to one of the desired sort where identifications of symmetries are unique.

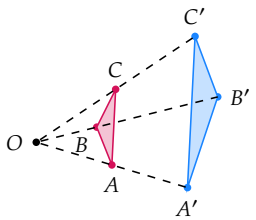


FIGURE 1.1: A perspectivity identifies the planes determined by the triangles ABC and $A'B'C'$ in a way that doesn't preserve Euclidean distances or angles.

fig:perspectivity

the unsolvability of general quintic (and higher degree) polynomials by radicals. In number theory, Gauss had made detailed studies of modular arithmetic, proving for instance that the group of units of $\mathbb{Z}/n\mathbb{Z}$ is cyclic precisely when n is 1, 2, 4, p^k or $2p^k$, where p is an odd prime and $k > 0$. Klein was bringing order to geometry by considering groups of transformation, while Lie was applying group theory in analysis to the study of differential equations.

Galois was the first to use the word "group" in a technical sense, speaking of collections of permutations closed under composition. He realized that the existence of a resolvent equation is equivalent to the existence of a normal subgroup of prime index in the group of the equation.

1.0.1 *Who is this book for*?

sec:who

At the outset the plan for this book was that it ought to cater for two very groups of readers. If you already have a classical first course in abstract group theory, this text has as its ambition that you should gain a new perspective on the material, *and at the same time* learn about homotopy type theory by seeing it applied to a field you are familiar with. However, at the outset, another audience seemed just as plausible to us: what if you're not well versed in abstract algebra, but open to learning about it from a type theoretic perspective? This might apply to a computer science student with aspirations towards the many applications of algebra.

The first audience may have become our predominant target as the book has progressed, partially because it probably is more sizable than the second since most students have been brain-washed to think only in terms of sets at the time they're ready for this book.

1.0.2 *Outline of the book*

sec:outline

TBD

All of mathematics is a tale, not about groups, but about ∞-groupoids. However, a lot of the action happens already with groups.

Glossary of coercions

Throughout this book we will use the following coercions to make the text more readable.

- If X is the pointed type (A, a), then $x : X$ means $x : A$.
- On hold, lacking context: If p and q are paths, then (p, q) means $(p, q)^{=}$.
- If e is a pair of a function and a proof, we also use e for the function.
- If e is an equivalence between types A and B, we use $\bar{e}$ for the identification of A and B induced by univalence.
- If $p : A \xrightarrow{=} B$ with A and B types, then we use $\tilde{p}$ for the canonical equivalence from A to B (also only as function).
- If X is $(A, a, \ldots)$ with $a : A$, then pt_X and even just pt mean a.
- In phrases like "a function of type $A \to B$" we may leave out "of type" and say "a function $A \to B$", for brevity. Likewise for other situations (equivalences, paths, etc.), as long as no ambiguity can occur.

How to read this book

A word of warning. We include a lot of figures to make it easier to follow the material. But like all mathematical writing, you'll get the most out of it, if you maintain a skeptical attitude: Do the pictures really accurately represent the formal constructions? Don't just believe us: Think about it!

The same goes for the proofs: When we say that something *clearly* follows, it should be *clear to you*. So clear, in fact, that you could go and convince a proof assistant, should you so desire.

Acknowledgement

The authors acknowledge the support of the Centre for Advanced Study (CAS) at the Norwegian Academy of Science and Letters in Oslo, Norway, which funded and hosted the research project Homotopy Type Theory and Univalent Foundations during the academic year 2018/19, as well as the CAS Alumni Fellowship, which financed several meetings and gatherings instrumental to getting the book closer to its final form.

2

An introduction to univalent mathematics

ch:univalent-mathematics
sec:what-is-a-type

2.1 *What is a type*?

In some computer programming languages, all variables are introduced along with a declaration of the type of thing they will refer to. Knowing the type of thing a variable refers to allows the computer to determine which expressions in the language are *grammatically well formed*[1], and hence valid. For example, if s is a string[2] and x is a real number, we may write $1/x$, but we may not write $1/s$.[3]

To enable the programmer to express such declarations, names are introduced to refer to the various types of things. For example, the name Bool may be used to declare that a variable is a Boolean value[4], Int may refer to 32-bit integers, and Real may refer to 64-bit floating point numbers[5].

Types occur in mathematics, too, and are used in the same way: all variables are introduced along with a declaration of the type of thing they will refer to. For example, one may say "consider a real number x", "consider a natural number n", "consider a point P of the plane", or "consider a line L of the plane". After that introduction, one may say that the *type* of n is *natural number* and that the *type* of P is *point of the plane*. Just as in a computer program, type declarations such as those are used to determine which mathematical statements are grammatically well formed. Thus one may write "P lies on L" or $1/x$, but not "L lies on P" nor $1/L$.[6]

Often ordinary English writing is good enough for such declarations in mathematics expositions, but, for convenience, mathematicians usually introduce symbolic names to refer to the various types of things under discussion. For example, the name $\mathbb{N}$ is usually used when declaring that a variable is a natural number, the name $\mathbb{Z}$ is usually used when declaring that a variable is an integer, and the name $\mathbb{R}$ is usually used when declaring that a variable is a real number. Ways are also given for constructing new type names from old ones: for example, the name $\mathbb{R} \times \mathbb{R}$ may be used when declaring that a variable is a point of the plane, for it conveys the information that a point of the plane is a pair of real numbers.

Once one becomes accustomed to the use of names such as $\mathbb{N}$ in mathematical writing and speaking, it is natural to take the next step and regard those names as denoting things that exist. Thus, we shall refer to $\mathbb{N}$ as the *type of all natural numbers*, and we will think of it as a mathematical object in its own right. Intuitively and informally, it is a collection whose members (or *elements*) are the natural numbers.

[1] The grammar of a programming language consists of all the language's rules. A statement or expression in a programming language is grammatically well formed if it follows all the rules.

[2] A *string* is a sequence of characters, such as "qwertyuiop".

[3] In a programming language, the well formed expression $1/x$ may produce a run-time error if x happens to have the value 0.

[4] A Boolean value is either *true* or *false*.

[5] An example of a *floating point number* is $.625 \times 2^{33}$ – the *mantissa* .625 and the *exponent* 33 are stored inside the floating point number. The "point", when the number is written in base 2 notation, is called "floating", because its position is easily changed by modifying the exponent.

[6] In mathematics there are no "run-time" errors; rather, it is legitimate to write the expression $1/x$ only if we already know that x is a non-zero real number.

Once we view the various types as existing as mathematical objects, they become worthy of study. The language of mathematics is thereby improved, and the scope of mathematics is broadened. For example, we can consider statements such as "$\mathbb{N}$ is infinite" and to try to prove it.

Historically, there was some hesitation[7] about introducing the collection of all natural numbers as a mathematical object, perhaps because if one were to attempt to build the collection from nothing by adding numbers to it one at a time, it would take an eternity to complete the assembly. We won't regard that as an obstacle.

[7] TO DO : Include some pointers to discussions of potential infinity and actual infinity, perhaps.

We have said that the types of things are used to determine whether mathematical statements are well formed. Therefore, if we expect "$\mathbb{N}$ is infinite" to be a well-formed statement, we'll have to know what type of thing $\mathbb{N}$ is, and we'll have to have a name for that type. Similarly, we'll have to know what type of thing that type is, and we'll have to have a name for it, and so on forever. Indeed, all of that is part of what will be presented in this chapter.

2.2 *Types, elements, families, and functions*

univalent_mathematics

In this section we build on the intuition imparted in the previous section.

In *univalent mathematics*,[8] types are used to classify all mathematical objects. Every mathematical object is an *element* (or a *member*) of some *type*. Before one can talk about an object of a certain type, one must introduce the type itself. There are enough ways to form new types from old ones to provide everything we need to write mathematics.

[8] The term "univalent" is a word coined by Vladimir Voevodsky, who introduced it to describe his principle that types that are *equivalent* in a certain sense can be identified with each other. The principle is stated precisely in **??** 2.13.2. As Voevodsky explained, the word comes from a Russian translation of a mathematics book, where the English mathematical term "faithful" was translated into Russian as the Russian word that sounds like "univalent". He also said "Indeed these foundations seem to be faithful to the way in which I think about mathematical objects in my head."

One expresses the declaration that an object a is an element of the *type* X by writing $a : X$.[9]

[9] The notation in mathematics based on *set theory* that corresponds (sort of) to this is $a \in X$.

Using that notation, each variable x is introduced along with a declaration of the form $x : X$, which declares that x will refer to something of type X, but provides no other information about x. The declared types of the variables are used to determine which statements of the theory are grammatically well formed.

After introducing a variable $x : X$, it may be possible to form an expression T representing a type, all of whose components have already been given a meaning. (Here the variable x is regarded also as having already been given a meaning, even though the only thing known about it is its type.) To clarify the dependence of T on x primarily, we may write $T(x)$ (or T_x) instead of T. Such an expression will be called a *family of types* parametrized by the variable x of type X. Such a family provides a variety of types, for, if a is any expression denoting an object of X, one may replace all occurrences of x by a in T, thereby obtaining a new expression representing a type, which may be regarded as a *member* and which may be denoted by $T(a)$.

Naturally, if the expression T doesn't actually involve the variable x, then the members of the family are all the same, and we'll refer to the family as a *constant family* of types.

Here's an example of a family of types: let T be the type of all natural numbers greater than 2. For any element n of T we let P_n be the type of n-sided polygons in the plane. It gives a family of types parametrized by the elements of T. One of the members of the family is the type P_5 of all pentagons in the plane.

A family of types may be parametrized by more than one variable. For example, after introducing a variable $x : X$ and a family of types T parametrized by x, we may introduce a variable $t : T$. Then it may be possible to form an expression S representing a type that involves the variables x and t. Such an expression will be called a family of types parametrized by x and t, and we may write $S(x, t)$ instead of S to emphasize the dependence on x and t. The same sort of thing works with more variables.

After introducing a variable $x : X$ and a family of types T, it may be possible to form an expression e of type T, all of whose components have already been given a meaning. Such an expression will also be called a *family of elements of* T parametrized by the elements of X, when we wish to focus on the dependence of e (and perhaps T) on the variable x. To clarify the dependence of e on x primarily, we may write $e(x)$ (or e_x) instead of e. Such a family provides a variety of elements of members of the family T, for, if a is any expression denoting an object of X, one may replace all occurrences of x by a in e and in T, thereby obtaining an element of $T(a)$, which may be regarded as a *member* of the family e and which will be denoted by $e(a)$.

Naturally, if the expressions e and T don't actually involve the variable x, then the members of the family are all the same, and we'll refer to the family as a *constant family* of elements.

Here's an example of a family of elements in a constant family of types: we let n be a natural number and consider the real number $\sqrt{n}$. It gives a family of real numbers parametrized by the natural numbers. (The family may also be called a *sequence* of real numbers). One of the members of the family is $\sqrt{11}$.

Here's an example of a family of elements in a (non-constant) family of types. As above, let T be the type of all natural numbers greater than 2 and let P_n be the type of n-sided polygons in the plane, for any $n : T$. Now consider the regular n-sided polygon p_n of radius 1 with a vertex on the positive x-axis, for any $n : T$. We see that $p_n : P_n$. One of the members of this family of *elements* p_n is the regular pentagon p_5 of radius 1 with a vertex on the positive x-axis. The pentagon p_5 is an element of the type P_5, which is a member of the family of *types* P_n $(n : T)$. In short, $5 : T$ and $p_5 : P_5$.

The type X containing the variable for a family of types or a family of elements is called the *parameter type* of the family.

Just as a family of types may depend on more than one variable, a family of elements may also depend on more than one variable.

Families of elements can be enclosed in mathematical objects called *functions* (or *maps*), as one might expect. Let e be a family of elements of a family of types T, both of which are parametrized by the elements x of X. We use the notation $x \mapsto e$ for the function that sends an element a of X to the element $e(a)$ of $T(a)$; the notation $x \mapsto e$ can be read as "x maps to e" or "x goes to e". (Recall that $e(a)$ is the expression that is obtained from e by replacing all occurrences of x in e by a.) If we name the function f, then that element of T will be denoted by $f(a)$. The *type* of the function $x \mapsto e$ is called a *product type* and will be denoted by $\prod_{x:X} T(x)$. If T is a constant family of types, then the type will also be called a *function type* and will be denoted by $X \to T$. Thus when we

write $f : X \to T$, we mean that f is an element of the type $X \to T$, and we are saying that f is a function from X to T. The type X may be called the *domain* of f, and the type T may be called the *codomain* of f.

Given an element t of T, we can consider the function $x \mapsto t$ mapping all elements of X to t. Such functions are called constant functions, and they occur so often that we introduce a special notation for them.

def:cst-map

Definition 2.2.1. Let X and T be types, and t an element of T. Define $\mathrm{cst}_t^X : X \to T$ to be $x \mapsto t$, the *constant function at* t. ⌟

An example of a function is the function $n \mapsto \sqrt{n}$ of type $\mathbb{N} \to \mathbb{R}$.

Another example of a function is the function $n \mapsto p_n$ of type $\prod_{n:\mathbb{N}} P_n$, where P_n is the type of polygons introduced above, and p_n is the polygon introduced above.

Another example of a function is the function $m \mapsto (n \mapsto m + n)$ of type $\mathbb{N} \to (\mathbb{N} \to \mathbb{N})$. It is a function that accepts a natural number as argument and returns a function as its value. The function returned is of type $\mathbb{N} \to \mathbb{N}$. It accepts a natural number as argument and returns a natural number as value.

The reader may wonder why the word "product" is used when speaking of product types. To motivate that, we consider a simple example informally. We take X to be a type with just two elements, b and c. We take $T(x)$ to be a family of types parametrized by the elements of X, with $T(b)$ being a type with 5 elements and $T(c)$ being a type with 11 elements. Then the various functions f of type $\prod_{x:X} T(x)$ are plausibly obtained by picking a suitable element for $f(b)$ from the 5 possibilities in $T(b)$ and by picking a suitable element for $f(c)$ from the 11 possibilities in $T(c)$. The number of ways to make both choices is 5×11, which is a *product* of two numbers. Thus $\prod_{x:X} T(x)$ is sort of like the product of $T(b)$ and $T(c)$, at least as far as counting is concerned.

The reader may wonder why we bother with functions at all: doesn't the expression e serve just as well as the function $x \mapsto e$, for all practical purposes? The answer is no. One reason is that the expression e doesn't inform the reader that the variable under consideration is x. Another reason is that we may want to use the variable x for elements of a different type later on: then $e(x)$ is no longer well formed. For example, imagine first writing this: "For a natural number n we consider the real number $\sqrt{n}$" and then writing this: "Now consider a triangle n in the plane." The result is that $\sqrt{n}$ is no longer usable, whereas the function $n \mapsto \sqrt{n}$ has enclosed the variable and the family into a single object and remains usable.[10]

Once a family e has been enclosed in the function $x \mapsto e$, the variable x is referred to as a *dummy variable* or as a *bound variable*.[11] This signifies that the name of the variable no longer matters, in other words, that $x \mapsto e(x)$ and $t \mapsto e(t)$ may regarded as identical. Moreover, the variable x that occurs inside the function $x \mapsto e$ is regarded as unrelated to variables x which may appear elsewhere in the discussion.

If the variable x in our notation $x \mapsto e(x)$ is a dummy variable, and its name doesn't matter, then we may consider the possibility of not specifying a variable at all. We introduce now a methodical way to do that, by replacing the occurrences of the variable x in the expression $e(x)$ by an *underscore*, yielding $e(_)$ as alternative notation for the function

[10] Students of trigonometry are already familiar with the concept of function, as something enclosed this way. The sine and cosine functions, sin and cos, are examples.

[11] Students of calculus are familiar with the concept of dummy variable and are accustomed to using identities such as $\int_a^b f(t)\,dt = \int_a^b f(x)\,dx$.

$x \mapsto e(x)$. For example, the notation $\sqrt{_}$ can serve as alternative notation for the function $n \mapsto \sqrt{n}$ introduced above, and $2 + _$ can serve as alternative notation for the function $n \mapsto 2 + n$ of type $\mathbb{N} \to \mathbb{N}$.

We have mentioned above the possibility of giving a name to a function. We expand on that now by introducing notation for making and for using *definitions*.

The notation $x :\equiv z$ will be an announcement that we are defining the expression x to be the expression z, all of whose components have already been given a meaning; in that case, we will say that x has been *defined* to be (or to mean) z. The forms allowed for the expression x will be made clear by the examples we give.

For example, after writing $n :\equiv 12$, we will say that n has been defined to be 12.

For another example, naming the function $x \mapsto e(x)$ as f (as we did above) can be done by writing $f :\equiv (x \mapsto e(x))$. Alternatively and more traditionally, we may write $f(x) :\equiv e(x)$. Both mean that f has been defined to be $x \mapsto e(x)$ and that, consequently, $f(a)$ has been defined to be $e(a)$, for any element a of X.

The notation $b \equiv c$ will denote the statement that the expressions b and c become the same thing if all the subexpressions within b or c are expanded according to their definitions, if any; in that case, we will say that b and c are *the same by definition*. For example, after writing $n :\equiv 12$ and $m :\equiv n$, we may say that $j + 12 \equiv j + m$ and that $m \times 11 \equiv 12 \times 11$.

Whenever two expressions are the same by definition, we may replace one with the other inside any other expression, because the expansion of definitions is regarded as trivial and transparent.

We proceed now to the promised example. Consider functions $f : X \to Y$ and $g : Y \to Z$. We define the *composite* function $g \circ f : X \to Z$ by setting $g \circ f :\equiv (a \mapsto g(f(a)))$. In other words, it is the function that sends an arbitrary element a of X to $g(f(a))$ in Z. (The expression $g \circ f$ may be read as "g circle f" or as "g composed with f".) The composite function $g \circ f$ may also be denoted simply by gf.

Now consider functions $f : X \to Y$, $g : Y \to Z$, and $h : Z \to W$. Then $(h \circ g) \circ f$ and $h \circ (g \circ f)$ are the same by definition, since applying the definitions within expands both expressions to $a \mapsto h(g(f(a)))$. In other words, we have established that $(h \circ g) \circ f \equiv h \circ (g \circ f)$. Thus, we may write $h \circ g \circ f$ for either expression, without danger of confusion.

One may define the identity function $\mathrm{id}_X : X \to X$ by setting $\mathrm{id}_X :\equiv (a \mapsto a)$. Application of definitions shows that $f \circ \mathrm{id}_X$ is the same by definition as $a \mapsto f(a)$, which, by a standard convention, which we adopt[12], is to be regarded as the same as f. In other words, we have established that $f \circ \mathrm{id}_X \equiv f$. A similar computation applies to $\mathrm{id}_Y \circ f$.

[12] The convention that $f \equiv (a \mapsto f(a))$ is referred to as the *η-rule* in the jargon of type theory.

In the following sections we will present various other elementary types and elementary ways to make new types from old ones.

page:idofetaf

2.3 *Universes*

sec:universes

In Section 2.2 we have introduced the objects known as *types*. They have *elements*, and the type an element belongs to determines the type of thing that it is. At various points in the sequel, it will be convenient for types also to be elements, for that will allow us, for example, to enclose families

of types in functions. To achieve this convenience, we introduce types that are *universes*. Some care is required, for the first temptation is to posit a single new type $\mathcal{U}$ called *the universe*, so that every type is realized as an element of $\mathcal{U}$. This universe would be "the type of all types", but introducing it would lead to an absurdity, for roughly the same reason that introduction of a "set of all sets" leads to the absurdity in traditional mathematics known as Russell's paradox.[13] Some later approaches to set theory included the notion of a *class*, with the collection of all sets being the primary example of a class. Classes are much like sets, and every set is a class, but not every class is a set. Then one may wonder what sort of thing the collection of all classes would be. Such musings are resolved in univalent mathematics as follows.

(1) There are some types called *universes*.

(2) If $\mathcal{U}$ is a universe, and $X : \mathcal{U}$ is an element of $\mathcal{U}$, then X is a type.

(3) If X is a type, then it appears as an element in some universe $\mathcal{U}$. Moreover, if X and Y are types, then there is a universe $\mathcal{U}$ containing both of them. This universe $\mathcal{U}$ also contains the type $X \to Y$ and similar types constructed from X and Y.

it:cumulative

(4) If $\mathcal{U}$ and $\mathcal{U}'$ are universes, $\mathcal{U} : \mathcal{U}'$, X is a type, and $X : \mathcal{U}$, then also $X : \mathcal{U}'$. (Thus we may regard $\mathcal{U}'$ as being *larger* than $\mathcal{U}$.)

(5) There is a particular universe $\mathcal{U}_0$, which we single out to serve as a repository for certain basic types to be introduced in the sequel. Moreover, $\mathcal{U}_0 : \mathcal{U}$ for every other universe $\mathcal{U}$, and thus $\mathcal{U}_0$ is the *smallest* universe.

It follows from the properties above that there are an infinite number of universes, for each one is an element of a larger one. For the sake of clarity, throughout this book, we use an infinite sequence of universes $\mathcal{U}_0 : \mathcal{U}_1 : \mathcal{U}_2 : \dots$.

Now suppose we have a type X and a family $T(x)$ of types parametrized by a variable x of type X. Choose a universe $\mathcal{U}$ with $T(x) : \mathcal{U}$. Then we can make a function of type $X \to \mathcal{U}$, namely $f :\equiv (x \mapsto T(x))$. Conversely, if f' is a function of type $X \to \mathcal{U}$, then we can make a family of types parametrized by x, namely $T' :\equiv f'(x)$. The flexibility offered by this correspondence between families of types in $\mathcal{U}$ and functions to $\mathcal{U}$ will often be used.

[13] In fact, type theory can trace its origins to Russell's paradox, announced in a 1902 letter to Frege as follows:

> There is just one point where I have encountered a difficulty. You state that a function too, can act as the indeterminate element. This I formerly believed, but now this view seems doubtful to me because of the following contradiction. Let w be the predicate: to be a predicate that cannot be predicated of itself. Can w be predicated of itself? From each answer its opposite follows. Therefore we must conclude that w is not a predicate. Likewise there is no class (as a totality) of those classes which, each taken as a totality, do not belong to themselves.

To which Frege replied:

> Incidentally, it seems to me that the expression "a predicate is predicated of itself" is not exact. A predicate is as a rule a first-level function, and this function requires an object as argument and cannot have itself as argument (subject).

Russell then quickly added *Appendix B* to his *Principles of Mathematics* (1903), in which he said that "it is the distinction of logical types that is the key to the whole mystery", where types are the *ranges of significance* of variables. For more on the history of type theory, see Coquand[14].

[14] Thierry Coquand. "Type Theory". In: *The Stanford Encyclopedia of Philosophy*. Ed. by Edward N. Zalta. Metaphysics Research Lab, Stanford University, 2018. URL: https://plato.stanford.edu/archives/fall2018/entries/type-theory/.

2.4 *The type of natural numbers*

sec:natural-numbers

Here are Peano's rules[15] for constructing the natural numbers in the form that is used in type theory.

P1 (P1) there is a type called $\mathbb{N}$ in the universe $\mathcal{U}_0$ (whose elements will be called *natural numbers*);

P2 (P2) there is an element of $\mathbb{N}$ called 0, called *zero*;

P3 (P3) if m is a natural number, then there is also a natural number $\mathrm{succ}(m)$, called the *successor* of m;

P4 (P4) suppose we are given:

[15] Giuseppe Peano. *Arithmetices principia: nova methodo*. See also https://github.com/mdnahas/Peano_Book/ for a parallel translation by Vincent Verheyen. Fratres Bocca, 1889. URL: https://books.google.com/books?id=z80GAAAAYAAJ.

a) a family of types $X(m)$ parametrized by a variable m of type $\mathbb{N}$;

b) an element a of $X(0)$; and

c) a family of functions $g_m : X(m) \to X(\mathrm{succ}(m))$.

Then from those data we are provided with a family of elements $f(m) : X(m)$, satisfying $f(0) \equiv a$ and $f(\mathrm{succ}(m)) \equiv g_m(f(m))$.

The first three rules present few problems for the reader. They provide us with the smallest natural number $0 : \mathbb{N}$, and we may introduce as many others as we like with the following definitions.

$$\begin{aligned} 1 &:\equiv \mathrm{succ}(0) \\ 2 &:\equiv \mathrm{succ}(1) \\ 3 &:\equiv \mathrm{succ}(2) \\ &\vdots \end{aligned}$$

You may recognize rule (P4) as "the principle of mathematical induction".[16] We will refer to it simply as "induction on $\mathbb{N}$".

You may also recognize the function f in (P4) as "defined by recursion". The point of the induction principle is that the type $X(m)$ of $f(m)$ may depend on m. An important special case is when $X(m)$ does not depend on m, that is, when $X(m) :\equiv Y$ for some type Y. In this non-dependent case we refer to the principle as "the recursion principle for $\mathbb{N}$". In other words, throughout this book, the difference between an induction principle and the corresponding recursion principle is that in the latter principle the type family is constant.

The resulting family f may be regarded as having been defined inductively by the two declarations $f(0) :\equiv a$ and $f(\mathrm{succ}(m)) :\equiv g_m(f(m))$, and indeed, we will often simply write such a pair of declarations as a shorthand way of applying rule (P4). The two declarations cover the two ways of introducing elements of $\mathbb{N}$ via the use of the two rules (P2) and (P3). (In terms of computer programming, those two declarations amount to the code for a recursive subroutine that can handle any incoming natural number.)

With that notation in hand, speaking informally, we may regard (P4) above as defining the family f by the following infinite sequence of definitions.

$$\begin{aligned} f(0) &:\equiv a \\ f(1) &:\equiv g_0(a) \\ f(2) &:\equiv g_1(g_0(a)) \\ f(3) &:\equiv g_2(g_1(g_0(a))) \\ &\vdots \end{aligned}$$

(The need for the rule (P4) arises from our inability to write down an infinite sequence of definitions in a finite amount of space, and from the need for $f(m)$ to be defined when m is a variable of type $\mathbb{N}$, and thus is not known to be equal to 0, nor to 1, nor to 2, etc.)

We may use induction on $\mathbb{N}$ to define of *iteration* of functions. Let Y be a type, and suppose we have a function $e : Y \to Y$. We define by

[16] Rule (P4) and our logical framework are stronger than in Peano's original formulation, and this allows us to omit some rules that Peano had to include: that different natural numbers have different successors; and that no number has 0 as its successor. Those omitted rules remain true in this formulation and can be proved from the other rules, after we have introduced the notion of equality in our logical framework.

induction on $\mathbb{N}$ the m-fold *iteration* $e^m : Y \to Y$ by setting $e^0 :\equiv \mathrm{id}_Y$ and $e^{\mathrm{succ}(m)} :\equiv e \circ e^m$. (Here we apply rule (P4) with the the type $Y \to Y$ as the family of types $X(m)$, the identity function id_Y for a, and the function $d \mapsto e \circ d$ for the family $g_m : (Y \to Y) \to (Y \to Y)$ of functions.)

We may now define addition of natural numbers by induction on $\mathbb{N}$. For natural numbers n and m we define $n + m : \mathbb{N}$ by induction on $\mathbb{N}$ with respect to the variable m by setting $n + 0 :\equiv n$ and $n + \mathrm{succ}(m) :\equiv \mathrm{succ}(n + m)$. (The reader should be able to extract the family $X(m)$, the element a, and the family of functions g_m from that pair of definitions.) Application of definitions shows, for example, that $2 + 2$ and 4 are the same by definition, and thus we may write $2 + 2 \equiv 4$, because both expressions reduce to $\mathrm{succ}(\mathrm{succ}(\mathrm{succ}(\mathrm{succ}(0))))$.

Similarly we define the product $m \cdot n : \mathbb{N}$ by induction on m by setting setting $0 \cdot n :\equiv 0$ and $\mathrm{succ}(m) \cdot n :\equiv (m \cdot n) + n$.

Alternatively (and equivalently) we may use iteration of functions to define addition and multiplication, by setting $n + m :\equiv \mathrm{succ}^m(n)$ and $m \cdot n :\equiv (i \mapsto i + n)^m(0)$.

Finally, we may define the factorial function $\mathrm{fact} : \mathbb{N} \to \mathbb{N}$ by induction on $\mathbb{N}$, setting $\mathrm{fact}(0) :\equiv 1$ and $\mathrm{fact}(\mathrm{succ}(m)) :\equiv \mathrm{succ}(m) \cdot \mathrm{fact}(m)$. (One can see that this definition applies rule (P4) with $X(m) :\equiv \mathbb{N}$, with 1 for a, and with the function $n \mapsto \mathrm{succ}(m) \cdot n$ for g_m.) Application of the definitions shows, for example, that $\mathrm{fact}(3) \equiv 6$, as the reader may verify.

2.5 *Identity types*

sec:identity-types

One of the most important types is the *identity type*, which implements a notion of equality. Identity types are formed of a type and two elements of that type; we shall have no need to compare elements of different types.

Here are the rules for constructing and using identity types.

rules-for-equality

E1 (E1) for any type X and for any elements a and b of it, there is an *identity type* $a \xrightarrow{=} b$; moreover, if X is an element of a universe $\mathcal{U}$, then so is $a \xrightarrow{=} b$.

When the type of a and b is not clear we may clarify it by writing $a \xrightarrow{=}_X b$.

E2 (E2) for any type X and for any element a of it, there is an element refl_a of type $a \xrightarrow{=} a$ (the name refl comes from the word "reflexivity")

E3 (E3) suppose we are given:

a) a type X and an element $a : X$;

b) a family of types $P(b, e, \dots)$ parametrized by a variable b of type X, a variable e of type $a \xrightarrow{=} b$, and perhaps some further variables; and

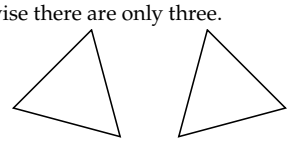
c) an element p of $P(a, \mathrm{refl}_a, \dots)$.

Then from those data we are provided with a family of elements $f(b, e, \dots) : P(b, e, \dots)$. Moreover, $f(a, \mathrm{refl}_a, \dots) \equiv p$.

We will refer to an element i of $a \xrightarrow{=} b$ as an *identification* of a with b. Since the word "identification" is a long one, we may also refer to i as a *path* from a to b – this has the advantage of incorporating the intuition that an identification may proceed gradually through intermediate steps.

The need to record, using the element i, the way we identify a with b may come as a surprise, since normally, in mathematics, one is accustomed to regarding a as either equal to b or not. However, this reflects a situation commonly encountered in geometry when *congruence* of geometric figures is considered. For example, in Euclidean space, two equilateral triangles of the same size are congruent in six (different) ways.[17] The chief novelty of univalent mathematics is that the basic logical notion of equality, as implemented by the identity types $a \xrightarrow{=} b$, is carefully engineered to accommodate notions of congruence and symmetry from diverse areas of mathematics, including geometry. Exposing that point of view in the context of geometry is the main point of this book.

[17] Six, since we allow reflections, otherwise there are only three.



In light of the analogy with geometry just introduced, we will refer to an element i of $a \xrightarrow{=} a$ as a *symmetry* of a. Think, for example, of a congruence of a triangle with itself. An example of a non-trivial symmetry will be seen in Exercise 2.13.3.

Consider the identity type $\mathrm{fact}(2) \xrightarrow{=} 2$, where fact denotes the factorial function defined in Section 2.4. Expansion of the definitions in $\mathrm{fact}(2) \xrightarrow{=} 2$ simplifies it to $\mathrm{succ}(\mathrm{succ}(0)) \xrightarrow{=} \mathrm{succ}(\mathrm{succ}(0))$, so we see from rule (E2) that $\mathrm{refl}_{\mathrm{succ}(\mathrm{succ}(0))}$ serves as an element of it.[18] We may also write either refl_2 or $\mathrm{refl}_{\mathrm{fact}(2)}$ for that element. A student might want a more detailed derivation that fact(2) may be identified with 2, but as a result of our convention above that definitions may be applied without changing anything, the application of definitions, including inductive definitions, is normally regarded as a trivial operation, and the details are usually omitted.

[18] We will see later that numbers only have trivial symmetries, so the possibility that there are other ways to identify fact(2) with 2 doesn't arise.

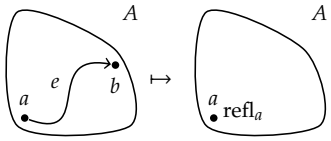
We will refer to rule (E3) as "induction for identity". To signal that we wish to apply it, we may announce that we argue *by* (*path*) *induction on* e, or simply *by path induction*.

The family f resulting from an application of rule (E3) may be regarded as having been completely defined by the single declaration $f(a, \mathrm{refl}_a) :\equiv p$, and indeed, we will often simply write such a declaration as a shorthand way of applying rule (E3). The rule says that to construct something from every identification e of a with something else, it suffices to consider the special case where the identification e is $\mathrm{refl}_a : a \xrightarrow{=} a$.[19]

[19] Notice that the single special case in such an induction corresponds to the single way of introducing elements of identity types via rule (E2), and compare that with (P4), which dealt with the two ways of introducing elements of $\mathbb{N}$.

Intuitively, the induction principle for identity amounts to saying that the element refl_a "generates" the system of types $a \xrightarrow{=} b$, as b ranges over elements of A.[20]

[20] We can also use a geometric intuition: when b "freely ranges" over elements of A, together with a path $e : a \xrightarrow{=} b$, while we keep the element a fixed, we can picture e as a piece of string winding through A, and the "freeness" of the pair (b, e) allows us to pull the string e, and b with it, until we have the constant path at a, refl_a.



Conversely, we can imagine b starting at a and e starting out as refl_a, and then think of b roaming throughout A, pulling the string e along with it, until it finds every path from a to some other element.

Equality relations are *symmetric*. For identity types we establish something similar, taking into account that the notion of equality implemented here keeps track of the way two things are identified, and there can be multiple ways. Given a type X and elements a and b of X, we have an identity type $a \xrightarrow{=} b$ of (zero or more) identifications of a with b. We also have an identity type $b \xrightarrow{=} a$ of identifications of b with a. Symmetry now takes the form of a function from type $a \xrightarrow{=} b$ to type $b \xrightarrow{=} a$, intuitively reversing any identification of a with b to give an identification of b with a. In order to produce an element of $b \xrightarrow{=} a$ from an element e of $a \xrightarrow{=} b$, for any b and e, we argue by induction on e. We let $P(b, e)$ be $b \xrightarrow{=} a$ for any b of type X and for any e of type $a \xrightarrow{=} b$, for use in rule (E3) above. Application of rule (E3) reduces us to the case where b is a and p is refl_a, and our task is now to produce an element of $a \xrightarrow{=} a$; we choose refl_a for

it.

Equality relations are also *transitive*. We proceed in a similar way as for symmetry. For each $a, b, c : X$ and for each $p : a \xrightarrow{=} b$ and for each $q : b \xrightarrow{=} c$ we want to produce an element of type $a \xrightarrow{=} c$. By induction on q we are reduced to the case where c is b and q is refl_b, and we are to produce an element of $a \xrightarrow{=} b$. The element p serves the purpose.

Now we state our symmetry result a little more formally.

def:eq-symm

Definition 2.5.1. For any type X and for any $a, b : X$, let

$$\mathrm{symm}_{a,b} : (a \xrightarrow{=} b) \to (b \xrightarrow{=} a)$$

be the function defined by induction by setting $\mathrm{symm}_{a,a}(\mathrm{refl}_a) :\equiv \mathrm{refl}_a$.

This operation on paths is called *path inverse*, and we may abbreviate $\mathrm{symm}_{a,b}(p)$ as p^{-1}. ⌟

Similarly, we formulate transitivity a little more formally, as follows.

def:eq-trans

Definition 2.5.2. For any type X and for any $a, b, c : X$, let

$$\mathrm{trans}_{a,b,c} : (a \xrightarrow{=} b) \to ((b \xrightarrow{=} c) \to (a \xrightarrow{=} c))$$

be the function defined by induction by setting $(\mathrm{trans}_{a,b,b}(p))(\mathrm{refl}_b) :\equiv p$.

This binary operation is called *path composition* or *path concatenation*, and we may abbreviate $(\mathrm{trans}_{a,b,c}(p))(q)$ as either $p * q$, or as $q \cdot p$, qp, or $q \circ p$. ⌟

The intuition that the path p summarizes a gradual change from a to b, and q summarizes a gradual change from b to c, leads to the intuition that $p * q$ progresses gradually from a to c by first changing a to b and then changing b to c; see Figure 2.1.

The notation $q \circ p$ for path composition, with p and q in reverse order, fits our intution particularly well when the paths are related to functions and the composition of the paths is related to the composition of the related functions in the same order, as happens, for example, in connection with *transport* (defined below in Definition 2.5.4) in Exercise 2.5.5.

The types of $\mathrm{symm}_{a,b}$ and $\mathrm{trans}_{a,b,c}$ express that $\xrightarrow{=}$ is symmetric and transitive. Another view of $\mathrm{symm}_{a,b}$ and $\mathrm{trans}_{a,b,c}$ is that they are operations on identifications, namely reversing an identification and concatenating two identifications. The results of various combinations of these operations can often be identified: we formulate some of these identifications in the following exercise.

xca:path-groupoid-laws

Exercise 2.5.3. Let X be a type and let $a, b, c, d : X$ be elements.

(1) For $p : a \xrightarrow{=} b$, construct an identification of type $p * \mathrm{refl}_b \xrightarrow{=} p$.

(2) For $p : a \xrightarrow{=} b$, construct an identification of type $\mathrm{refl}_a * p \xrightarrow{=} p$.

(3) For $p : a \xrightarrow{=} b$, $q : b \xrightarrow{=} c$, and $r : c \xrightarrow{=} d$, construct an identification of type $(p * q) * r \xrightarrow{=} p * (q * r)$.

(4) For $p : a \xrightarrow{=} b$, construct an identification of type $p^{-1} * p \xrightarrow{=} \mathrm{refl}_b$.

(5) For $p : a \xrightarrow{=} b$, construct an identification of type $p * p^{-1} \xrightarrow{=} \mathrm{refl}_a$.

(6) For $p : a \xrightarrow{=} b$, construct an identification of type $(p^{-1})^{-1} \xrightarrow{=} p$. ⌟

Given an element $p : a \xrightarrow{=} a$, we may use concatenation to define powers $p^n : a \xrightarrow{=} a$ by induction on $n : \mathbb{N}$; we set $p^0 :\equiv \mathrm{refl}_a$ and $p^{n+1} :\equiv p \cdot p^n$. Negative powers p^{-n} are defined as $(p^{-1})^n$.[21]

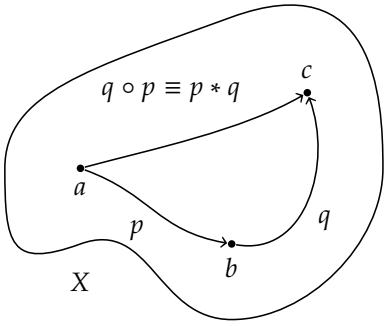


Figure 2.1: Composition (also called concatenation) of paths in X

fig:path-concatenation

[21] We haven't yet assigned a meaning to $-n$, but after we introduce the set of integers Z below in Definition 3.2.1, we'll be justified in writing p^z for any $z : \mathrm{Z}$. See also Example 2.12.9.

One frequent use of elements of identity types is in *substitution*, which is the logical principle that supports our intuition that when x can by identified with y, we may replace x by y in mathematical expressions at will. A wrinkle new to students will likely be that, in our logical framework where there may be various ways to identify x with y, one must specify the identification used in the substitution. Thus one may prefer to speak of using an identification to *transport* properties and data about x to properties and data about y.

Here is a geometric example: if x is a triangle of area 3 in the plane, and y is congruent to x, then y also has area 3.

Here is another example: if x is a right triangle in the plane, and y is congruent to x, then y is also a right triangle, and the congruence informs us which of the 3 angles of y is the right angle.

Now we introduce the notion more formally.

def: transport

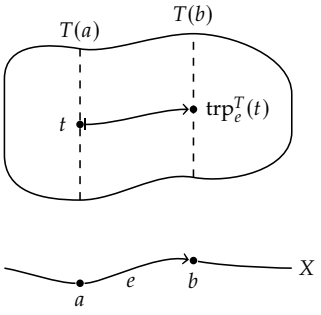
DEFINITION 2.5.4. Let X be a type, and let $T(x)$ be a family of types parametrized by a variable $x : X$ (as discussed in Section 2.2). Suppose $a, b : X$ and $e : a \xrightarrow{=} b$. Then we may construct a function of type $T(a) \to T(b)$. The function

$$\mathrm{trp}_e^T : T(a) \to T(b)$$

is defined by induction setting $\mathrm{trp}^T_{\mathrm{refl}_a} :\equiv \mathrm{id}_{T(a)}$. ⌟

The function thus defined may be called *the transport function in the type family T along the path e*, or, less verbosely, *transport*.[22] We may also simplify the notation to just trp_e. The transport functions behave as expected: we may construct an identification of type $\mathrm{trp}_{e' \circ e} \xrightarrow{=} \mathrm{trp}_{e'} \circ \mathrm{trp}_e$. In words: transport along the composition $e \circ e'$ can be identified with the composition of the two transport functions. This may be proved by induction in the following exercise.

[22] We sometimes picture this schematically as follows: We draw X as a (mostly horizontal) line, and we draw each type $T(x)$ as a vertical line lying over $x : X$. As x moves around in X, these lines can change shape, and taken all together they form a 2-dimensional blob lying over X. The transport functions map points between the vertical lines.



xca: trp-compose

EXERCISE 2.5.5. Let X be a type, and let $T(x)$ be a family of types parametrized by a variable $x : X$. Suppose we are given elements $a, b, c : X$, $e : a \xrightarrow{=} b$, and $e' : b \xrightarrow{=} c$. Construct an identification of type

$$\mathrm{trp}_{e' \circ e} \xrightarrow{=} \mathrm{trp}_{e'} \circ \mathrm{trp}_e.$$ ⌟

Yet another example of good behavior is given in the following exercise.

xca: trp-nondep

EXERCISE 2.5.6. Let X, Y be types. As discussed in Section 2.2, we may regard the expression Y as a constant family of types parametrized by a variable $x : X$. Produce an identification of type $\mathrm{trp}_p^Y \xrightarrow{=} \mathrm{id}_Y$, for any path $p : a \xrightarrow{=} b$. ⌟

In Section 2.15 below we will discuss what it means for a type to have at most one element. When the types $T(x)$ may have more than one element, we may regard an element of $T(x)$ as providing additional *structure* on x. In that case, we will refer to the transport function $\mathrm{trp}_e : T(a) \to T(b)$ as *transport of structure* from a to b.

Take, for example, $T(x) :\equiv (x \xrightarrow{=} x)$. Then trp_e is of type $(a \xrightarrow{=} a) \to (b \xrightarrow{=} b)$ and transports a symmetry of a to a symmetry of b.

By contrast, when the types $T(x)$ have at most one element, we may regard an element of $T(x)$ as providing a proof of a property of x. In that case, the transport function $\mathrm{trp}_e : T(a) \to T(b)$ provides a way to establish a claim about b from a claim about a, so we will refer to it as

substitution. In other words, elements that can be identified have the same properties.

2.6 *Product types*

sec:product-types

Functions and product types have been introduced in Section 2.2, where we have also explained how to create a function by enclosing a family of elements in one. In this section we treat functions and product types in more detail.

Recall that if X is a type and $Y(x)$ is a family of types parametrized by a variable x of type X, then there is a *product type*[23] $\prod_{x:X} Y(x)$ whose elements f are functions that provide elements $f(a)$ of type $Y(a)$, one for each $a : X$. We will refer to X as the *parameter type* of the product. By contrast, if Y happens to be a constant family of types, then $\prod_{x:X} Y$ will also be denoted by $X \to Y$, and it will also be called a *function type*.

[23] Also known as a *Pi-type*.

If X and $Y(x)$ are elements of a universe $\mathcal{U}$, then so is $\prod_{x:X} Y(x)$.

Functions preserve identity, and we will use this frequently later on. More precisely, functions induce maps on identity types, as the following definition makes precise.

def:ap

Definition 2.6.1. For all types X, Y, functions $f : X \to Y$ and elements $x, x' : X$, the function

$$\mathrm{ap}_{f,x,x'} : (x \xrightarrow{=} x') \to (f(x) \xrightarrow{=} f(x'))$$

is defined by induction by setting $\mathrm{ap}_{f,x,x}(\mathrm{refl}_x) :\equiv \mathrm{refl}_{f(x)}$. ⌟

The function $\mathrm{ap}_{f,x,x'}$, for any elements x and x' of X, is called an *application* of f to paths or to identifications, and this explains the choice of the symbol ap in the notation for it. It may also be called the function (or map) *induced* by f on identity types.

When x and x' are clear from the context, we may abbreviate $\mathrm{ap}_{f,x,x'}$ by writing ap_f instead. For convenience, we may abbreviate it even further, writing $f(p)$ for $\mathrm{ap}_f(p)$.

The following lemma shows that ap_f is compatible with composition.

lem:apcomp

Construction 2.6.2. *Given a function $f : X \to Y$, and elements $x, x', x'' : X$, and paths $p : x \xrightarrow{=} x'$ and $p' : x' \xrightarrow{=} x''$, we have an identification of type $\mathrm{ap}_f(p' \cdot p) \xrightarrow{=} \mathrm{ap}_f(p') \cdot \mathrm{ap}_f(p)$.*

Similarly, we have that ap_f is compatible with path inverse in that we have an identification of type $\mathrm{ap}_f(p^{-1}) \xrightarrow{=} (\mathrm{ap}_f(p))^{-1}$ for all $p : x \xrightarrow{=} x'$.

Finally, we have an identification of type $\mathrm{ap}_{\mathrm{id}}(p) \xrightarrow{=} p$ for all $p : x \xrightarrow{=} x'$.

Implementation of Construction 2.6.2. By induction on p and p', one reduces to producing an identification of type

$$\mathrm{ap}_f(\mathrm{refl}_x \cdot \mathrm{refl}_x) \xrightarrow{=} \mathrm{ap}_f(\mathrm{refl}_x) \cdot \mathrm{ap}_f(\mathrm{refl}_x).$$

Both $\mathrm{ap}_f(\mathrm{refl}_x \cdot \mathrm{refl}_x)$ and $\mathrm{ap}_f(\mathrm{refl}_x) \cdot \mathrm{ap}_f(\mathrm{refl}_x)$ are equal to $\mathrm{refl}_{f(x)}$ by definition, so the identification $\mathrm{refl}_{\mathrm{refl}_{f(x)}}$ has the desired type.

The other two parts of the construction are also easily done by induction on p. □

xca:trp-ap

Exercise 2.6.3. Let X be a type and $T(x)$ a family of types parametrized by a variable $x : X$. Furthermore, let A be a type, let $f : A \to X$ be a function,

let a and a' be elements of A, and let $p : a \xrightarrow{=} a'$ be a path. Verify that the two functions $\mathrm{trp}_p^{T\circ f}$ and $\mathrm{trp}^T_{\mathrm{ap}_f(p)}$ are of type $T(f(a)) \to T(f(a'))$. Then construct an identification between them, i.e., construct an element of type $\mathrm{trp}_p^{T\circ f} \xrightarrow{=} \mathrm{trp}^T_{\mathrm{ap}_f(p)}$. ⌟

If two functions f and g of type $\prod_{x:X} Y(x)$ can be identified, then their values can be identified, i.e., for every element x of X, we may produce an identification of type $f(x) \xrightarrow{=} g(x)$, which can be constructed by induction, as follows.

def:ptw

Definition 2.6.4. Let $f, g : \prod_{x:X} Y(x)$. Define the function

$$\mathrm{ptw}_{f,g} : (f \xrightarrow{=} g) \to \left(\prod_{x:X} f(x) \xrightarrow{=} g(x)\right),$$

by induction by setting $\mathrm{ptw}_{f,f}(\mathrm{refl}_f) :\equiv x \mapsto \mathrm{refl}_{f(x)}$. [24] ⌟

Conversely, given $f, g : \prod_{x:X} Y(x)$, from a basic axiom called *function extensionality*, postulated below in **??** 2.9.18, an identification $f \xrightarrow{=} g$ can be produced from a family of identifications of type $f(x) \xrightarrow{=} g(x)$ parametrized by a variable x of type X.

def:naturality-square

Definition 2.6.5. Let X, Y be types and $f, g : X \to Y$ functions. Given an element h of type $\prod_{x:X} f(x) \xrightarrow{=} g(x)$, elements x and x' of X, and a path $p : x \xrightarrow{=} x'$, we have two elements $h(x') \cdot \mathrm{ap}_f(p)$ and $\mathrm{ap}_g(p) \cdot h(x)$ of type $f(x) \xrightarrow{=} g(x')$. We construct an identification

$$\mathrm{ns}(h,p) : \left(h(x') \cdot \mathrm{ap}_f(p) \xrightarrow{=} \mathrm{ap}_g(p) \cdot h(x)\right),$$

between them by induction on p, by setting $\mathrm{ns}(h, \mathrm{refl}_x)$ to be an element of $h(x) \cdot \mathrm{refl}_{f(x)} \xrightarrow{=} h(x)$, which can be constructed by induction on $h(x)$, as in Exercise 2.5.3. We depict the type of $\mathrm{ns}(h,p)$ in Figure 2.2 and call $\mathrm{ns}(h,p)$ a *naturality square*. ⌟

[24] The notation ptw is chosen to remind the reader of the word "point-wise", because the identifications are provided just for each point x. An alternative approach goes by considering, for any $x : X$, the evaluation function $\mathrm{ev}_x : \left(\prod_{x:X} Y(x)\right) \to Y(x)$ defined by $\mathrm{ev}_x(f) :\equiv f(x)$. Then one could define $\mathrm{ptw}_{f,g}(p,x) :\equiv \mathrm{ap}_{\mathrm{ev}_x}(p)$. The functions provided by these two definitions are not equal by definition, but they can be identified, and one can easily be used in place of the other.

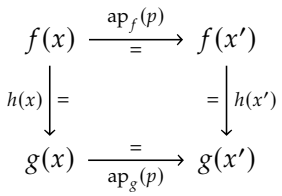


Figure 2.2: Illustration of Definition 2.6.5.

2.7 *Identifying elements in members of families of types*

If $Y(x)$ is a family of types parametrized by a variable x of type X, and a and a' are elements of type X, then after identifying a with a' it turns out that it is possible to "identify" an element of $Y(a)$ with an element of $Y(a')$, in a certain sense. That is the idea of the following definition.

def:pathsoverpaths

Definition 2.7.1. Suppose we are given a type X in a universe $\mathcal{U}$ and a family of types $Y(x)$, also in $\mathcal{U}$, parametrized by a variable x of type X. Given elements $a, a' : X$, $y : Y(a)$, and $y' : Y(a')$ and a path $p : a \xrightarrow{=} a'$, we define a new type $y \xrightarrow[p]{=} y'$ in $\mathcal{U}$ as follows. We proceed by induction on a' and p, which reduces us to the case where a' is a and p is refl_a, rendering y and y' of the same type $Y(a)$ in $\mathcal{U}$, allowing us to define $y \xrightarrow[\mathrm{refl}_a]{=} y'$ to be $y \xrightarrow{=} y'$, which is also in $\mathcal{U}$. ⌟

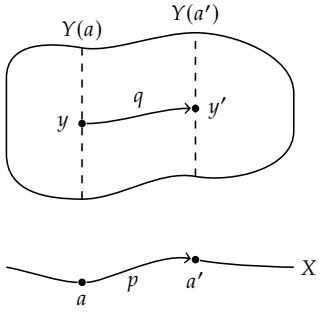
fig:dpath-sq

fig:dpath-pic

An element $q : y \xrightarrow[p]{=} y'$ is called an *identification* of y with y' *over* p, or a *path* from y to y' *over* p. Intuitively, we regard p as specifying a way for a to change gradually into a', and this provides a way for $Y(a)$ to change gradually into $Y(a')$; then q charts a way for y to change gradually into y' as $Y(a)$ changes gradually into $Y(a')$.[25]

Remark 2.7.2. Given a type Z, Definition 2.7.1 has a special case in which $Y(x) :\equiv Z$ for all $x : X$. Given elements $a, a' : X$, a path $p : a \xrightarrow{=} a'$ and

[25] We picture this as follows: the path from y to y' over p travels through the vertical lines representing the types $Y(x)$ as $x : X$ moves along the path p in X from a to a':



elements $z, z' : Z$, we can form both the type $z \xrightarrow[p]{=} z'$ and the identity type $z \xrightarrow{=} z'$. These types are readily identified by induction on p. ⌟

The following definition identifies the type of paths over p with a type of paths using transport along p.

def:pathover-crp

Definition 2.7.3. In the context of Definition 2.7.1, define by induction on p an identification $\mathrm{po}_p : \left(y \xrightarrow[p]{=} y'\right) \xrightarrow{=} \left(\mathrm{trp}^Y_p(y) \xrightarrow{=} y'\right)$ in $\mathcal{U}$, by setting $\mathrm{po}_{\mathrm{refl}_x} :\equiv \mathrm{refl}_{y \xrightarrow{=} y'}$. ⌟

Many of the operations on paths have counterparts for paths over paths. For example, we may define composition of paths over paths as follows.

def:pathovercomposition

Definition 2.7.4. Suppose we are given a type X and a family of types $Y(x)$ parametrized by the elements x of X. Suppose also that we have elements $x, x', x'' : X$, a path $p : x \xrightarrow{=} x'$, and a path $p' : x' \xrightarrow{=} x''$. Suppose further that we have elements $y : Y(x)$, $y' : Y(x')$, and $y'' : Y(x'')$, with paths $q : y \xrightarrow[p]{=} y'$ over p and $q' : y' \xrightarrow[p']{=} y''$ over p'. Then we define the *composite* path $(q' \mathbin{\underline{\circ}} q) : y \xrightarrow[p' \circ p]{=} y''$ over $p' \circ p$ as follows. First we apply path induction on p' to reduce to the case where x'' is x' and p' is $\mathrm{refl}_{x'}$. That also reduces the type $y' \xrightarrow[p']{=} y''$ to the identity type $y' \xrightarrow{=} y''$, so we may apply path induction on q' to reduce to the case where y'' is y' and q' is $\mathrm{refl}_{y'}$. Now observe that $p' \circ p$ is p, so q provides the element we need. ⌟

Similarly, one can define the inverse of a path over a path, writing $q^{-1} : y' \xrightarrow[p^{-1}]{=} y$ for the inverse of $q : y \xrightarrow[p]{=} y'$. For these operations on paths over paths we have identifications analogous to those for the operations on paths in Exercise 2.5.3, after some modification. For example, $g^{-1} \mathbin{\underline{\circ}} q$ of type $y \xrightarrow[p^{-1} \circ p]{=} y$ and refl_y of type $y \xrightarrow[\mathrm{refl}_x]{=} y$ cannot be directly used to form an identity type, since their types are not equal by definition. We will state these identifications when we need them.

xca:cp

Exercise 2.7.5. Try to state some of these identifications yourself. ⌟

The following construction shows how to handle application of a dependent function f to paths using the definition above.

def:apd

Definition 2.7.6. Suppose we are given a type X, a family of types $Y(x)$ parametrized by the elements x of X, and a function $f : \prod_x Y(x)$. Given elements $x, x' : X$ and a path $p : x \xrightarrow{=} x'$, we define

$$\mathrm{apd}_f(p) : f(x) \xrightarrow[p]{=} f(x')$$

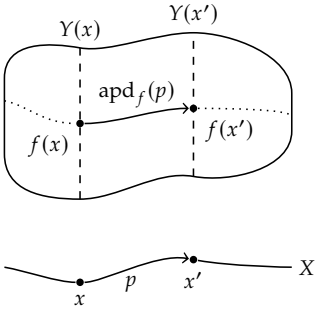
by induction on p, setting

$$\mathrm{apd}_f(\mathrm{refl}_x) :\equiv \mathrm{refl}_{f(x)}. \qquad ⌟$$

The function apd_f is called *dependent application* of f to paths.[26] For convenience, we may abbreviate $\mathrm{apd}_f(p)$ to $f(p)$, when there is no risk of confusion.

The following construction shows how functions of two variables may be applied to paths over paths.

[26] We picture f via its *graph* of the values $f(x)$ as x varies in X. The dependent application of f to p is then the piece of the graph that lies over p:



def:applfun2

Definition 2.7.7. Suppose we are given a type X, a family of types $Y(x)$ parametrized by the elements x of X, and a type Z. Suppose also we are given a function $g : \prod_{x:X}(Y(x) \to Z)$ of two variables. Given elements $x, x' : X$, $y : Y(x)$, and $y' : Y(x')$, a path $p : x \xrightarrow{=} x'$, and a path $q : y \xrightarrow[p]{=} y'$ over p, we may construct a path

$$\mathrm{apap}_g(p)(q) : g(x)(y) \xrightarrow{=} g(x')(y')$$

by induction on p and q, setting

$$\mathrm{apap}_g(\mathrm{refl}_x)(\mathrm{refl}_y) :\equiv \mathrm{refl}_{g(x)(y)}. \qquad \lrcorner$$

The function $p \mapsto q \mapsto \mathrm{apap}_g(p)(q)$ is called *application* of g to paths over paths. For convenience, we may abbreviate $\mathrm{apap}_g(p)(q)$ to $g(p)(q)$.

The following definition will be useful later.

def:applfun2comp

Definition 2.7.8. Suppose we are given a type X, a family of types $Y(x)$ parametrized by the elements x of X, and a type Z. Suppose also we are given a function $g : \prod_{x:X}(Y(x) \to Z)$ of two variables. Given an element $x : X$, elements $y, y' : Y(x)$, and an identification $q : y \xrightarrow{=} y'$, then we define an identification of type $\mathrm{apap}_g(\mathrm{refl}_x)(q) \xrightarrow{=} \mathrm{ap}_{g(x)}(q)$, by induction on q, thereby reducing to the case where y' is y and q is refl_y, rendering the two sides of the equation equal, by definition, to $\mathrm{refl}_{g(x)(y)}$. $\lrcorner$

2.8 *Sum types*

sec:sum-types

There are *sums* of types. By this we mean if X is a type and $Y(x)$ is a family of types parametrized by a variable x of type X, then there will be a type[27] $\sum_{x:X} Y(x)$ whose elements are all pairs (a, b), where $a : X$ and $b : Y(a)$. Since the type of b may depend on a we also call such a pair a *dependent* pair. We may refer to X as the *parameter type* of the sum.[28]

If X and $Y(x)$ are elements of a universe $\mathcal{U}$, then so is $\sum_{x:X} Y(x)$.

Proving something about (or constructing something from) every element of $\sum_{x:X} Y(x)$ is done by performing the construction on elements of the form (a, b), for every $a : X$ and $b : Y(a)$. Two important examples of such constructions are:

(1) *first projection*, $\mathrm{fst} : (\sum_{x:X} Y(x)) \to X$, $\mathrm{fst}(a, b) :\equiv a$;

(2) *second projection*, $\mathrm{snd}(a, b) : Y(a)$, $\mathrm{snd}(a, b) :\equiv b$.

it:second-projection

In (2), the type of snd is, in full, $\prod_{z:\sum_{x:X} Y(x)} Y(\mathrm{fst}(z))$.

rem:non-dependent-sums

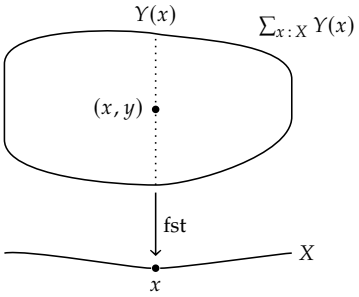
Remark 2.8.1. An important special case of sum types is when the type $Y(x)$ does not depend on $x : X$. In that case the sum type $\sum_{x:X} Y(x)$ is denoted as $X \times Y$ and called a *binary product type*, see Section 2.11. $\lrcorner$

rem:iterated-sums

Remark 2.8.2. One may consider sums of sums. For example, suppose X is a type, suppose $Y(x)$ is a family of types parametrized by a variable x of type X, and suppose $Z(x, y)$ is a family of types parametrized by variables $x : X$ and $y : Y(x)$. In this case, the *iterated sum* $\sum_{x:X} \sum_{y:Y(x)} Z(x, y)$ consists of pairs of the form $(x, (y, z))$. For simplicity, we introduce the notation $(x, y, z) :\equiv (x, (y, z))$, and refer to (x, y, z) as a *triple* or as a 3-*tuple*.

[27] Also known as a *Sigma-type*.

[28] We may denote $\sum_{x:X} Y(x)$ by $\mathrm{Tot}(Y)$ and also call it the *total type* of the family $Y(x)$. We can picture it, in the style of the pictures above, as the entire blob lying over X. (Each $Y(x)$ is a vertical line over $x : X$, and a point $y : Y(x)$ becomes a point (x, y) in the blob.)



Another example of an iterated sum is when $Z'(u)$ is a family of types parameterized by a variable u of type $\sum_{x:X} Y(x)$. Elements of the type $\sum_{u:\sum_{x:X} Y(x)} Z'(u)$ are triples $((x, y), z)$. We use the triple-notation also for this case.

That process can be repeated: suppose X_1 is a type, suppose $X_2(x_1)$ is a family of types parametrized by a variable x_1 of type X_1, suppose $X_3(x_1, x_2)$ is a family of types parametrized by variables $x_1 : X_1$ and $x_2 : X_2(x_1)$, and so on, up to a family $X_n(x_1, \dots, x_{n-1})$ of types. In this case, the *iterated sum*

$$\sum_{x_1 : X_1} \sum_{x_2 : X_2(x_1)} \cdots \sum_{x_{n-1} : X_{n-1}(x_1, \dots, x_{n-2})} X_n(x_1, \dots, x_{n-1})$$

consists of elements of the form $(x_1, (x_2, (\dots (x_{n-1}, x_n) \dots)))$; each such element is a pair whose second member is a pair, and so on, so we may refer to it as an *iterated pair*. For simplicity, we introduce the notation $(x_1, x_2, \dots, x_n)$ for such an iterated pair, and refer to it as an *n-tuple*. ⌟

2.9 *Equivalences*

sec:equivalence

Using a combination of sum, product, and identity types allows us to express important notions, as done in the following definitions.

The property that a type X has "exactly one element" may be made precise by saying that X has an element such that every other element is equal to it. This property is encoded in the following definition.

def:contractible

Definition 2.9.1. Given a type X, define a type $\mathrm{isContr}(X)$ by setting

$$\mathrm{isContr}(X) :\equiv \sum_{c : X} \prod_{x : X} (c \xrightarrow{=} x).$$ ⌟

If $(c, h) : \mathrm{isContr}(X)$, then c will be called the *center* of the the *contraction* h, and we call the type X *contractible*.

By path composition, one sees that any element $x : X$ can serve as the center of a contraction of a contractible type X.

The following lemma gives an important example of a contractible type.

Given a type X and an element a of X, the *singleton type* $\sum_{x : X} (a \xrightarrow{=} x)$ consists of pairs (x, i) with $i : a \xrightarrow{=} x$. The following lemma shows that a singleton type has exactly one element, justifying the name.

lem:thepathspaceiscontractible

Lemma 2.9.2. *For any type X and $a : X$, the singleton type $\sum_{x : X} (a \xrightarrow{=} x)$ is contractible.*

Proof. Take as center the pair (a, refl_a). We have to produce, for any element x of X and for any identification $i : a \xrightarrow{=} x$, an identification of type $(a, \mathrm{refl}_a) \xrightarrow{=} (x, i)$. This is done by path induction on i, which reduces us to producing an identification of type $(a, \mathrm{refl}_a) \xrightarrow{=} (a, \mathrm{refl}_a)$; reflexivity provides one, namely $\mathrm{refl}_{(a, \mathrm{refl}_a)}$. □

def:fiber

Definition 2.9.3. Given a function $f : X \to Y$ and an element $y : Y$, the *fiber* (or *preimage*) $f^{-1}(y)$ is encoded by defining

$$f^{-1}(y) :\equiv \sum_{x : X} (y \xrightarrow{=} f(x)).$$

In other words, an element of the fiber $f^{-1}(y)$ is a pair consisting of an element x of X and an identification of type $y \xrightarrow{=} f(x)$. A *fiber sequence* presents this data in the form of a sequence $f^{-1}(y) \xrightarrow{\mathrm{fst}} X \xrightarrow{f} Y$. ⌟

In set theory, a function $f : X \to Y$ is a bijection if and only if all preimages $f^{-1}(y)$ consist of exactly one element. We can also express this in type theory, in a definition due to Voevodsky, for types in general.

def:equivalence

Definition 2.9.4. A function $f : X \to Y$ is called an *equivalence* if $f^{-1}(y)$ is contractible for all $y : Y$. The condition is encoded by the type

$$\mathrm{isEquiv}(f) :\equiv \prod_{y:Y} \mathrm{isContr}(f^{-1}(y)). \qquad \lrcorner$$

We may say that X and Y are *equivalent* if we have an equivalence between them.

def:type-of-equivalences

Definition 2.9.5. We define the type $X \xrightarrow{\simeq} Y$ of equivalences from X to Y by the following definition.

$$(X \xrightarrow{\simeq} Y) :\equiv \sum_{f:X\to Y} \mathrm{isEquiv}(f). \qquad \lrcorner$$

Suppose $f : X \xrightarrow{\simeq} Y$ is an equivalence, and let $t(y) : \mathrm{isContr}(f^{-1}(y))$, for each $y : Y$, be the corresponding witness to contractibility of the fiber. Using t we can define an inverse function $g : Y \to X$ by setting $g(y) :\equiv \mathrm{fst}(\mathrm{fst}(t(y)))$. This can be seen as follows.

By unfolding all the definitions[29], we have an identification of type $f(g(y)) \xrightarrow{=} y$. Moreover, $(x, \mathrm{refl}_{f(x)})$ is an element of the fiber $f^{-1}(f(x))$, and $t(f(x))$ is a proof that this fiber is contractible. Hence the center of contraction $\mathrm{fst}(t(f(x))$ is identified with $(x, \mathrm{refl}_{f(x)})$, and so $g(f(x)) \equiv (\mathrm{fst}(\mathrm{fst}(t(f(x))))) \xrightarrow{=} x$.

We have shown that f and g are inverse functions. When it won't cause confusion with the notation for the fibers of f, we will write f^{-1} instead of g.

For any type X, the identity function id_X is an equivalence from X to X. To see that, observe that for every element a in X, $\mathrm{id}_X^{-1}(a)$ is a singleton type and hence is contractible. This observation, combined with the fact that $\mathrm{trp}^T_{\mathrm{refl}_x} \equiv \mathrm{id}_{T(x)}$, gives that the function trp^T_e from Definition 2.5.4 is an equivalence from $T(x)$ to $T(y)$, for all $e : x \xrightarrow{=} y$.

xca:equivalence-invers

Exercise 2.9.6. Make sure you understand the two applications of fst in the definition $f^{-1}(y) :\equiv \mathrm{fst}(\mathrm{fst}(t(y)))$ above. Show that f^{-1} is an equivalence from Y to X. Give a function $(X \xrightarrow{\simeq} Y) \to (Y \xrightarrow{\simeq} X)$. $\lrcorner$

xca:equivalence-comp

Exercise 2.9.7. Give a function $(X \xrightarrow{\simeq} Y) \to ((Y \xrightarrow{\simeq} Z) \to (X \xrightarrow{\simeq} Z))$. $\lrcorner$

xca:2-out-of-3

Exercise 2.9.8. Consider types A, B, and C, functions $f : A \to B$, $g : A \to C$ and $h : B \to C$, together with an element $e : h\,f \xrightarrow{=} g$. Prove that if two of the three functions are equivalences, then so is the third one. $\lrcorner$

The following lemma gives an equivalent characterization of equivalence that is sometimes easy to use.

lem:weq-iso

Construction 2.9.9. *Let X, Y be types. For each equivalence $f : X \to Y$, we have a function $g : Y \to X$ such that for all $x : X$ we have $g(f(x)) \xrightarrow{=} x$ and for all $y : Y$ we have $f(g(y)) \xrightarrow{=} y$. Conversely, if we have such a function g, then f is an equivalence.*

Implementation of Construction 2.9.9. Given an equivalence $f : X \to Y$ we can take $g :\equiv f^{-1}$. For the converse, see Chapter 4 of the HoTT Book,[30] or `isweq_iso`. □

[29] Note that $\mathrm{fst}(t(y)) : f^{-1}(y)$, so $\mathrm{fst}(\mathrm{fst}(t(y))) : X$ with $\mathrm{snd}(\mathrm{fst}(t(y))) : y \xrightarrow{=} f(\mathrm{fst}(\mathrm{fst}(t(y))))$.

[30] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study: `https://homotopytypetheory.org/book`, 2013.

We put Construction 2.9.9 immediately to good use.

lem:contract-away

LEMMA 2.9.10. *Let X be a type with element a, and let $B(x,i)$ be a type for all $x:X$ and $i:a \xrightarrow{=} x$. Define $f(x,i):B(x,i) \to B(a,\mathrm{refl}_a)$ by induction on i, setting $f(a,\mathrm{refl}_a,b) :\equiv b$ for all $b:B(a,\mathrm{refl}_a)$. Then f defines an equivalence*

$$f \;:\; \sum_{x:X} \sum_{i:a\xrightarrow{=}x} B(x,i) \quad\to\quad B(a,\mathrm{refl}_a).$$

Proof. We can also define $g:B(a,\mathrm{refl}_a) \to \sum_{x:X}\sum_{i:a\xrightarrow{=}x} B(x,i)$ mapping $b:B(a,\mathrm{refl}_a)$ to (a,refl_a,b). Clearly $f(g(b)) \xrightarrow{=} b$ for all $b:B(a,\mathrm{refl}_a)$. Moreover, $g(f(x,i,b)) \xrightarrow{=} (x,i,b)$ is clear by induction on i, for all $b:B(x,i)$. By Construction 2.9.9 it follows that f is an equivalence. □

The above lemma clearly reflects the contractibility of the singleton type $\sum_{x:X}(a \xrightarrow{=} x)$.[31] For this reason we call application of this lemma 'to contract away' the prefix $\sum_{x:X}\sum_{i:a\xrightarrow{=}x}$, in order to obtain a simpler type. It is often applied in the following simpler form.

[31] In fact, an alternative proof would go as follows: First, we use Construction 2.9.9 to construct an element of $\sum_{x:X}\sum_{y:Y(x)} Z(x,y) \xrightarrow{\simeq} \sum_{w:(\sum_{x:X} Y(x))} Z(\mathrm{fst}\,w, \mathrm{snd}\,w)$, i.e., the associativity of sum types, where X is a type, $Y(x)$ is a family of types depending on $x:X$, and $Z(x,y)$ is a family of types depending on $x:X$ and $y:Y(x)$. Then, we construct for any contractible type X and for any family of types $Y(x)$ depending on $x:X$, an equivalence between $\sum_{x:X} Y(x)$ and $Y(c)$, where c is the center of contraction of X.

ft:Sigma-assoc

cor:contract-away

COROLLARY 2.9.11. *With conditions as above, but with B not depending on i, the same f establishes an equivalence*

$$\sum_{x:X}((a \xrightarrow{=} x) \times B(x)) \quad\xrightarrow{\simeq}\quad B(a).$$

In the direction of further generality, we offer the following exercise.

xca:sum-equiv-base

EXERCISE 2.9.12. Suppose X, Y are types related by an equivalence $f:X \to Y$. Let $B(x)$ be a type parameterized by $x:X$. Construct an equivalence between $\sum_{x:X} B(x)$ and $\sum_{y:Y} B(f^{-1}(y))$. ⌟

The next exercise gives a dual to Corollary 2.9.11 that may be dubbed 'to substitute away'.

xca:substitute-away

EXERCISE 2.9.13. Let X be a type with element a, and let $B(x)$ be a type parameterized by $x:X$. Give an equivalence between $\prod_{x:X}((a \xrightarrow{=} x) \to B(x))$ and $B(a)$. ⌟

We proceed now to define the notion of equivalences in families.

def:fiberwise

DEFINITION 2.9.14. Let X be a type, and let $Y(x), Z(x)$ be families of types parametrized by $x:X$. A map f of type $\prod_{x:X}(Y(x) \to Z(x))$ can be viewed as a family of maps $f(x):Y(x) \to Z(x)$ and is called a map of families. The *totalization* of f is defined by $\mathrm{tot}(f)(x,y) :\equiv (x, f(x)(y))$. Using the denotation Tot(_) for the total type of a type family we thus have

$$\mathrm{tot}(f):\mathrm{Tot}(Y) \to \mathrm{Tot}(Z).$$ ⌟

lem:fiberwise

LEMMA 2.9.15. *Let conditions be as in Definition 2.9.14. If $f(x):Y(x) \to Z(x)$ is an equivalence for every $x:X$ (we say that f is an equivalence of families), then $\mathrm{tot}(f)$ is an equivalence.*

Proof. If $f(x):Y(x) \to Z(x)$ is an equivalence for all x in X, then the same is true of all $f(x)^{-1}:Z(x) \to Y(x)$. Then we have the totalization $\mathrm{tot}(x \mapsto f(x)^{-1})$, which can easily be proved to be an inverse of $\mathrm{tot}(f)$ (see the next exercise). Now apply Construction 2.9.9. □

xca:fiberwise

EXERCISE 2.9.16. Complete the details of the proof of Lemma 2.9.15. ⌟

The converse to Lemma 2.9.15 also holds.

lem:fiberwise-equiv-from-tot

LEMMA 2.9.17. *Continuing with the setup of Definition 2.9.14, if* $\mathrm{tot}(f)$ *is an equivalence, then* f *is a equivalence of families.*

For a proof see Theorem 4.7.7 of the HoTT Book[32].

Yet another application of the notion of equivalence is to postulate axioms.

def:funext

PRINCIPLE 2.9.18. The axiom of *function extensionality* postulates that the function $\mathrm{ptw}_{f,g} : (f \overset{=}{\to} g) \to \prod_{x:X} f(x) \overset{=}{\to} g(x)$ in Definition 2.6.4 is an equivalence. Formally, we postulate the existence of an element $\mathrm{funext} : \mathrm{isEquiv}(\mathrm{ptw}_{f,g})$. From that we can construct the corresponding inverse function

$$\mathrm{ptw}^{-1}_{f,g} : \Big(\prod_{x:X} f(x) \overset{=}{\to} g(x)\Big) \to (f \overset{=}{\to} g).$$

Thus two functions whose values can all be identified can themselves be identified. This supports the intuition that there is nothing more to a function than the values it sends its arguments to. ⌟

xca:Xequivtrue

EXERCISE 2.9.19. Let X be a type. Construct an equivalence of type $(\mathrm{True} \to X) \overset{\simeq}{\to} X$. ⌟

xca:Xequivtimes1

EXERCISE 2.9.20. Let X be a type, and regard True as a constant family of types over X. Construct an equivalence of type $(\sum_{x:X} \mathrm{True}) \overset{\simeq}{\to} X$. ⌟

xca:Sigma-distrib

EXERCISE 2.9.21. Let X and Y be types, and let $Z(y)$ be a type parameterized by $y : Y$. Construct an equivalence of type $\big(X \times \sum_{y:Y} Z(y)\big) \overset{\simeq}{\to} \sum_{y:Y}(X \times Z(y))$. ⌟

xca:Sigma-comm

EXERCISE 2.9.22. Let X and Y be types, and let $Z(x, y)$ be a type parameterized by $x : X$ and $y : Y$. Construct an equivalence of type $\big(\sum_{x:X}\sum_{y:Y} Z(x,y)\big) \overset{\simeq}{\to} \sum_{y:Y}\sum_{x:X} Z(x,y)$. ⌟

xca:fib-of-comp

EXERCISE 2.9.23. Let X, Y and Z be types. Given functions $f : X \to Y$ and $g : Y \to Z$, construct a family of equivalences of type $(gf)^{-1}(z) \overset{\simeq}{\to} \sum_{w:g^{-1}(z)} f^{-1}(\mathrm{fst}\, w)$ parameterized by $z : Z$. Hint: use Footnote 31. ⌟

xca:AC-in-TT

EXERCISE 2.9.24. Let X and Y be types, and let $Z(x, y)$ be a type parameterized by $x : X$ and $y : Y$. Construct an equivalence of type $\big(\prod_{x:X}\sum_{y:Y} Z(x,y)\big) \overset{\simeq}{\to} \sum_{f:X\to Y}\prod_{x:X} Z(x, f(x))$.[33] ⌟

xca:section-above-f

EXERCISE 2.9.25. Let X and Z be types, and let $Y(x)$ be a type parameterized by $x : X$. For any function $f : Z \to X$, construct an equivalence of type $\big(\prod_{z:Z} Y(f(z))\big) \overset{\simeq}{\to} \sum_{g:Z\to\sum_{x:X} Y(x)}(f \overset{=}{\to} \mathrm{fst} \circ g)$.[34] ⌟

xca:Sigma-curry

EXERCISE 2.9.26. Let X and Z be types, and let $Y(x)$ be a type parameterized by $x : X$. Construct an equivalence[35] of type $\big((\sum_{x:X} Y(x)) \to Z\big) \overset{\simeq}{\to} \prod_{x:X}(Y(x) \to Z)$. ⌟

2.10 *Identifying pairs*

sec:pairpaths

The identity type of two elements of $\sum_{x:X} Y(x)$ is inductively defined in Section 2.5, as for any other type, but one would like to express the identity type for pairs in terms of identifications in the constituent types. This would explain better what it means for two pairs to be identified. We start with a definition.

def:pairtopath

DEFINITION 2.10.1. Suppose we are given a type X and a family of types $Y(x)$ parametrized by the elements x of X. Consider the function

[32] Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*.

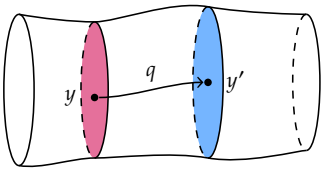
[33] This equivalence is sometimes called the type-theoretic axiom of choice; more prosaically, it expresses the distributivity of products (Π-types) over sums (Σ-types). We discuss the real axiom of choice in Section B.4.

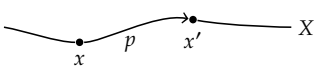
[34] The special case $Z \equiv X$, $f \equiv \mathrm{id}_X$ applies to any product type.

[35] This canonical equivalence is often called "currying", after Haskell B. Curry, and will be treated transparently, i.e., we will pass between $f(x, y)$ and $f(x)(y)$ without denoting it. Note that the equivalence goes between $(X \times Y) \to Z$ and $X \to (Y \to Z)$ in case $Y(x)$ is constant.

We picture paths between pairs much in the same way as paths over paths, cf. Footnote 25. Just as, to give a pair in the sum type $\sum_{x:X} Y(x)$, we need both the point x in the parameter type X as well as the point y in $Y(x)$, to give a path from (x, y) to (x', y'), we need both a path $p : x \overset{=}{\to} x'$ as well as a path $q : y \overset{=}{\underset{p}{\to}} y'$ over p.

Here's a similar picture, where we depict the types in the family as being 2-dimensional for a change.





$$\mathrm{pair} : \prod_{x:X}\Big(Y(x) \to \sum_{x':X} Y(x')\Big)$$

defined by

$$\mathrm{pair}(x)(y) :\equiv (x, y).$$

For any elements (x, y) and (x', y') of $\sum_{x:X} Y(x)$, we define the map

$$\Big(\sum_{p:x \xrightarrow{=} x'} y \xrightarrow[p]{=} y'\Big) \to \big((x, y) \xrightarrow{=} (x', y')\big)$$

by

$$(p, q) \mapsto \mathrm{apap}_{\mathrm{pair}}(p)(q).$$

(Refer to Definition 2.7.1 for the meaning of the type $y \xrightarrow[p]{=} y'$, and to Definition 2.7.7 for the definition of apap.) We introduce $\overline{(p, q)}$ as notation for $\mathrm{apap}_{\mathrm{pair}}(p)(q)$. ⌟

cor:iseq-pair=

Construction 2.10.2. *In the situation of Definition 2.10.1, if x' is x, so that we have $(y \xrightarrow[\mathrm{refl}_x]{=} y') \equiv (y \xrightarrow{=} y')$, then for any $q : y \xrightarrow{=} y'$, we can construct an identification of type*:

$$\overline{(\mathrm{refl}_x, q)} \xrightarrow{=} \mathrm{ap}_{\mathrm{pair}(x)}\, q$$

Implementation of Construction 2.10.2. By induction on q it suffices to establish an identification

$$\overline{(\mathrm{refl}_x, \mathrm{refl}_y)} \xrightarrow{=} \mathrm{ap}_{\mathrm{pair}(x)}(\mathrm{refl}_y),$$

both sides of which are equal to $\mathrm{refl}_{(x,y)}$ by definition. □

The following lemma gives the desired characterization of paths between pairs.

lem:iseq-pair=

Lemma 2.10.3. *Suppose we are given a type X and a family of types $Y(x)$ parametrized by the elements x of X. For any elements (x, y) and (x', y') of $\sum_{x:X} Y(x)$, the map defined in Definition 2.10.1 defined by*

$$(p, q) \mapsto \overline{(p, q)}$$

is an equivalence of type

$$\Big(\sum_{p:x \xrightarrow{=} x'} y \xrightarrow[p]{=} y'\Big) \xrightarrow{\simeq} \big((x, y) \xrightarrow{=} (x', y')\big).$$

Proof. Call the map Φ. A map the other way,

$$\Psi : \big((x, y) \xrightarrow{=} (x', y')\big) \to \sum_{p:x \xrightarrow{=} x'} y \xrightarrow[p]{=} y',$$

can be defined by induction, by setting

$$\Psi(\mathrm{refl}_{(x,y)}) :\equiv (\mathrm{refl}_x, \mathrm{refl}_y).$$

One proves, by induction on paths, the identifications $\Psi(\Phi(p, q)) \xrightarrow{=} (p, q)$ and $\Phi(\Psi(r)) \xrightarrow{=} r$, so Ψ and Φ are inverse functions. Applying Construction 2.9.9, we see that Φ and Ψ are inverse equivalences, thereby obtaining the desired result. □

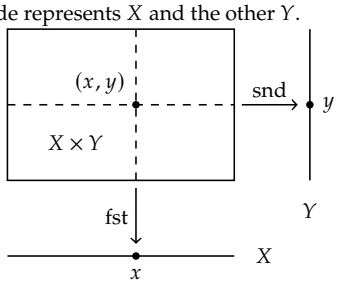
We often use $\mathrm{fst}(\overline{(p, q)}) \xrightarrow{=} p$ and $\mathrm{snd}(\overline{(p, q)}) \xrightarrow{=} q$, which follow by induction on p and q from the definitions of ap and $\overline{(_, _)}$. Similarly, $r \xrightarrow{=} \overline{(\mathrm{fst}(r), \mathrm{snd}(r))}$ by induction on r.

2.11 *Binary products*

sec:binprod-types

There is special case of sum types that deserves to be mentioned since it occurs quite often. Let X and Y be types, and consider the constant family of types $Y(x) :\equiv Y$. In other words, $Y(x)$ is a type that depends on an element x of X that happens to be Y for any such x. (Recall Exercise 2.5.6.) Then we can form the sum type $\sum_{x:X} Y(x)$ as above. Elements of this sum type are pairs (x, y) with x in X and y in $Y(x) \equiv Y$.[36] In this case the type of y doesn't depend on x, and in this special case the sum type is called the *binary product*, or *cartesian product* of the types X and Y, denoted by $X \times Y$.

[36] These *cartesian* products we illustrate as usual by rectangles where one side represents X and the other Y.



At first glance, it might seem odd that a sum is also a product, but exactly the same thing happens with numbers, for the sum $5 + 5 + 5$ is also referred to as the product 3×5. Indeed, that's one way to define 3×5.

Recall that we have seen something similar with the product type $\prod_{x:X} Y(x)$, which we let $X \to Z$ denote in the case where $Y(x)$ is a constant family of the form $Y(x) :\equiv Z$, for some type Z.

The type $X \times Y$ inherits the functions fst, snd from $\sum_{x:X} Y(x)$, with the same definitions $\mathrm{fst}(x, y) :\equiv x$ and $\mathrm{snd}(x, y) :\equiv y$. Their types can now be denoted in a simpler way as $\mathrm{fst} : (X \times Y) \to X$ and $\mathrm{snd} : (X \times Y) \to Y$, and they are called as before the first and the second projection, respectively.

Again, proving something about (or constructing something from) every element (a, b) of $X \times Y$ is simply done for all $a : X$ and $b : Y$.

There is an equivalence between $(a_1, b_1) \xrightarrow{=} (a_2, b_2)$ and $(a_1 \xrightarrow{=} a_2) \times (b_1 \xrightarrow{=} b_2)$. This follows from Lemma 2.10.3 together with Exercise 2.5.6.

If $f : X \to Y$ and $f' : X' \to Y'$, then we let $f \times f'$ denote the map of type $(X \times X') \to (Y \times Y')$ that sends (x, x') to $(f(x), f'(x'))$.

The following lemma follows from Lemma 2.10.3, combined with Definition 2.7.3 and Exercise 2.5.6.

lem:isEq-pair-bin=

Lemma 2.11.1. *Suppose we are given type X and Y. For any elements (x, y) and (x', y') of $X \times Y$, the map defined in Definition 2.10.1 defined by*

$$(p, q) \mapsto \overline{(p, q)}$$

is an equivalence of type

$$(x \xrightarrow{=} x') \times (y \xrightarrow{=} y') \xrightarrow{\simeq} \left((x, y) \xrightarrow{=} (x', y')\right).$$

xca:binary-prod-equiv

Exercise 2.11.2. Let X, Y be types in a universe $\mathcal{U}$, and consider the type family $T(z)$ in $\mathcal{U}$ depending on $z : \mathrm{Bool}$ defined by $T(\mathrm{no}) :\equiv X$ and $T(\mathrm{yes}) :\equiv Y$. Show that the function $(\prod_{b:\mathrm{Bool}} T(b)) \to X \times Y$ sending f to $(f(\mathrm{no}), f(\mathrm{yes}))$, is an equivalence. ⌟

xca:binary-prod-comm

Exercise 2.11.3. Let X and Y be types. Construct an equivalence of type $(X \times Y) \xrightarrow{\simeq} (Y \times X)$. ⌟

2.12 *More inductive types*

sec:inductive-types

There are other examples of types that are conveniently introduced in the same way as we have seen with the natural numbers and the identity types. A type presented in this style shares some common features: there are some ways to create new elements, and there is a way (called

induction) to prove something about every element of the type (or family of types). We will refer to such types as *inductive* types, and we present a few more of them in this section, including the finite types, and then we present some other constructions for making new types from old ones. For each of these constructions we explain the identity type for two elements of the newly constructed type in terms of identity types for elements of the constituent types.

2.12.1 *Finite types*

sec:finite-types

Firstly, there is the *empty* type in the universe $\mathcal{U}_0$, denoted by $\emptyset$ or by False. It is an inductive type, with no way to construct elements of it. The induction principle for $\emptyset$ says that to prove something about (or to construct something from) every element of $\emptyset$, it suffices to consider no special cases (!). Hence, every statement about an arbitrary element of $\emptyset$ can be proven. (This logical principle is traditionally called *ex falso* (*sequitur*) *quodlibet*.[37]) As an example, we may prove that any two elements x and y of $\emptyset$ are equal (i.e., construct an identification of type $x \xrightarrow{=} y$) by using induction on x. We may even prove by induction on $x : \emptyset$ that the elements 0 and succ(0) of $\mathbb{N}$ are equal (i.e., construct a function of type $\emptyset \to (0 \xrightarrow{=} \mathrm{succ}(0))$.

[37] From falsehood, anything follows. Also called the principle of explosion.

An element of $\emptyset$ will be called an *absurdity*. Of course, one expects that there are no real absurdities in mathematics, nor in any logical system (such as ours) that attempts to provide a language for mathematics, but it is important to have such a name so we can discuss the possibility, which might result inadvertently from the introduction of unwarranted assumptions. For example, to assert that a type T has no elements, it would be sensible to assert that an element of T would lead to an absurdity. Providing a function of type $T \to \emptyset$ is a convenient way to make that assertion.

Secondly, there will also be an inductive type called True in the universe $\mathcal{U}_0$ provided with a single element triv; (the name triv comes from the word "trivial"). Its induction principle states that, in order to prove something about (or to construct something from) every element of True, it suffices to consider the special case where the element is triv. As an example, we may construct, for any element $u : \mathrm{True}$, an identification of type $u \xrightarrow{=} \mathrm{triv}$; we use induction to reduce to the case where u is triv, and then $\mathrm{refl}_{\mathrm{triv}}$ provides the desired element. One may also construct, for any elements x and y of True, an identification of type $x \xrightarrow{=} y$ by using induction both on x and on y.

There is a function $X \to \mathrm{True}$, for any type X, namely: $a \mapsto \mathrm{triv}$. This corresponds, for propositions, to the statement that an implication holds if the conclusion is true.

xca:True-univ-prop

Exercise 2.12.2. Let X be a type. Define the function e of type $(\mathrm{True} \to X) \to X$ by $e(f) :\equiv f(\mathrm{triv})$. Prove that e is an equivalence. This is called *the universal property of* True. ⌟

Thirdly, there will be an inductive type called Bool in the universe $\mathcal{U}_0$, provided with two elements, yes and no. Its induction principle states that, in order to prove something about (or to construct something from) every element of Bool, it suffices to consider two cases: the special case where the element is yes and the special case where the element is no.

We may use substitution to construct an element of type (yes $\overset{=}{\to}$ no) $\to \emptyset$, expressing that the identification of yes with no leads to an absurdity. To do this, we introduce a family of types $P(b)$ in the universe $\mathcal{U}_0$ parametrized by a variable $b : \mathrm{Bool}$. We define $P(b)$ by induction on b by setting $P(\mathrm{yes}) :\equiv \mathrm{True}$ and $P(\mathrm{no}) :\equiv \mathrm{False}$. (The definition of $P(b)$ is motivated by the expectation that we will be able to construct an equivalence between $P(b)$ and yes $\overset{=}{\to} b$.) If there were an element $e : \mathrm{yes} \overset{=}{\to} \mathrm{no}$, we could substitute no for yes in $\mathrm{triv} : P(\mathrm{yes})$ to get an element of $P(\mathrm{no})$, which is absurd. Since e was arbitrary, we have defined a function (yes $\overset{=}{\to}$ no) $\to \emptyset$, as desired.

In the same way, we may use substitution to prove that it is absurd that successors of natural numbers are identical to 0, i.e., for any $n : \mathbb{N}$ that $(0 \overset{=}{\to} \mathrm{succ}(n)) \to \emptyset$. To do this, we introduce a family of types $P(i)$ in $\mathcal{U}_0$ parametrized by a variable $i : \mathbb{N}$. Define P recursively by specifying that $P(0) :\equiv \mathrm{True}$ and $P(\mathrm{succ}(m)) :\equiv \mathrm{False}$. (The definition of $P(i)$ is motivated by the expectation that we will be able to construct an equivalence between $P(i)$ and $0 \overset{=}{\to} i$.) If there were an element $e : 0 \overset{=}{\to} \mathrm{succ}(n)$, we could substitute $\mathrm{succ}(n)$ for 0 in $\mathrm{triv} : P(0)$ to get an element of $P(\mathrm{succ}(n))$, which is absurd. Since e was arbitrary, we have defined a function $(0 \overset{=}{\to} \mathrm{succ}(n)) \to \emptyset$, establishing the claim.

In a similar way we will in Section 2.24 define types $\mathbb{n}$ for any n in $\mathbb{N}$ such that $\mathbb{n}$ is a type (set) of n elements.

2.12.3 *Binary sums*

sec:binsum-types

For sum types of the form $\sum_{b:\mathrm{Bool}} T(b)$, with $T(b)$ a type depending on b in Bool, there is an equivalence with a simpler type.[38] After all, the type family $T(b)$ is fully determined by two types, namely by the types $T(\mathrm{no})$ and $T(\mathrm{yes})$. The elements of $\sum_{b:\mathrm{Bool}} T(b)$ are dependent pairs (no, x) with x in $T(\mathrm{no})$ and (yes, y) with y in $T(\mathrm{yes})$. The resulting type can be viewed as the *disjoint union* of $T(\mathrm{no})$ and $T(\mathrm{yes})$: from an element of $T(\mathrm{no})$ or an element of $T(\mathrm{yes})$ we can produce an element of $\sum_{b:\mathrm{Bool}} T(b)$.

These disjoint union types can be described more clearly in the following way. The *binary sum* of two types X and Y, denoted $X + Y$, is an inductive type with two constructors: $\mathrm{inl} : X \to X + Y$ and $\mathrm{inr} : Y \to X + Y$.[39] Proving a property of any element of $X + Y$ means proving that this property holds of any inl_x with $x : X$ and any inr_y with $y : Y$. In general, constructing a function f of type $\prod_{z:X+Y} T(z)$, where $T(z)$ is a type depending on z, is done by defining $f(\mathrm{inl}_x)$ for all x in X and $f(\mathrm{inr}_y)$ for all y in Y.

xca:binary-sum-equiv

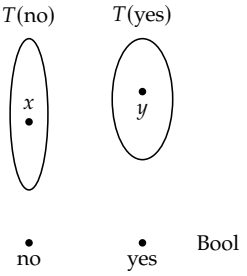
Exercise 2.12.4. Let X, Y be types in a universe $\mathcal{U}$, and consider the type family $T(z)$ in $\mathcal{U}$ depending on $z : \mathrm{Bool}$ defined by induction on z by $T(\mathrm{no}) :\equiv X$ and $T(\mathrm{yes}) :\equiv Y$. Show that the map $f : X + Y \to \sum_{b:\mathrm{Bool}} T(b)$, defined by $f(\mathrm{inl}_x) :\equiv (\mathrm{no}, x)$ and $f(\mathrm{inr}_y) :\equiv (\mathrm{yes}, y)$, is an equivalence. ⌟

Identification of two elements a and b in $X + Y$ is only possible if they are constructed with the same constructor. Thus $\mathrm{inl}_x \overset{=}{\to} \mathrm{inr}_y$ is always empty, and there are equivalences of type $(\mathrm{inl}_x \overset{=}{\to} \mathrm{inl}_{x'}) \overset{\simeq}{\to} (x \overset{=}{\to} x')$ and $(\mathrm{inr}_y \overset{=}{\to} \mathrm{inr}_{y'}) \overset{\simeq}{\to} (y \overset{=}{\to} y')$.

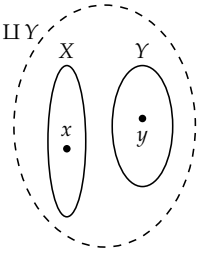
xca:binary-sum-id

Exercise 2.12.5. Prove these statements using Exercise 2.12.4, Lemma 2.10.3, and a characterization of the identity types of Bool. ⌟

[38] In a case like this, we can thicken up the lines denoting $T(\mathrm{no})$ and $T(\mathrm{yes})$ in our picture, if we like:



[39] Beware that in a picture, the same point may refer either to x in X or to inl_x in the sum $X + Y$:



xca:bin-sum-univ-prop

Exercise 2.12.6. Let X, Y, Z be types. Define a function e from $(X \amalg Y) \to Z$ to $(X \to Z) \times (Y \to Z)$ by precomposition with the constructors. Prove that e is an equivalence. This is called *the universal property of the binary sum*. ⌟

Exercise 2.12.7. Let X be a type. Construct an equivalence of type $(X \amalg \emptyset) \xrightarrow{\simeq} X$. ⌟

sec:unary-sum-types

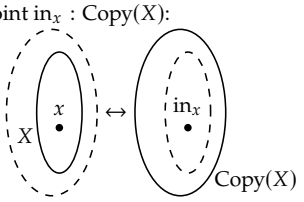
2.12.8 *Unary sums*

Sometimes it is useful to be able to make a copy of a type X: A new type that behaves just like X, though it is not equal to X by definition. The *unary sum* or *wrapped copy* of X is an inductive type $\mathrm{Copy}(X)$ with a single constructor, $\mathrm{in} : X \to \mathrm{Copy}(X)$.[40] Constructing a function $f : \prod_{z:\mathrm{Copy}(X)} T(z)$, where $T(z)$ is a type depending on $z : \mathrm{Copy}(X)$, is done by defining $f(\mathrm{in}_x)$ for all $x : X$. Taking $T(z)$ to be the constant family at X, we get a function, $\mathrm{out} : \mathrm{Copy}(X) \to X$, called the *destructor*, with $\mathrm{out}(\mathrm{in}_x) :\equiv x$ for $x : X$, and the induction principle implies that $\mathrm{in}_{\mathrm{out}(z)} \xrightarrow{=} z$ for all $z : \mathrm{Copy}(X)$, so there is an equivalence of type $\mathrm{Copy}(X) \xrightarrow{\simeq} X$, as expected. It follows that there are equivalences of type $(\mathrm{in}_x \xrightarrow{=} \mathrm{in}_{x'}) \xrightarrow{\simeq} (x \xrightarrow{=} x')$ and $(\mathrm{out}(z) \xrightarrow{=} \mathrm{out}(z')) \xrightarrow{\simeq} (z \xrightarrow{=} z')$.

Note that we can make several copies of X that are not equal to each other by definition, for instance, by picking different names for the constructor. We write $\mathrm{Copy}_{\mathrm{con}}(X)$ for a copy of X whose constructor is

$$\mathrm{con} : X \to \mathrm{Copy}_{\mathrm{con}}(X).$$

[40] A point $x : X$ corresponds to the point $\mathrm{in}_x : \mathrm{Copy}(X)$:



Note that $\mathrm{Copy}(X)$ can alternatively be defined as $\sum_{z:\mathrm{True}} X$.

exa:nnn

Example 2.12.9. Here's an example to illustrate why it can be useful to make such a wrapped type: We introduced the natural numbers $\mathbb{N}$ in Section 2.4. Suppose we want a type consisting of negations of natural numbers, $\{\ldots, -2, -1, 0\}$, perhaps as an intermediate step towards building the set of integers $\{\ldots, -2, -1, 0, 1, 2, \ldots\}$.[41] Of course, the type $\mathbb{N}$ itself would do, but then we would need to pay extra attention to whether $n : \mathbb{N}$ is supposed to represent n as an integer or its negation. So instead we take the wrapped copy $\mathbb{N}^- :\equiv \mathrm{Copy}_(\mathbb{N})$, with constructor $- : \mathbb{N} \to \mathbb{N}^-$. We will also write $- : \mathbb{N}^- \to \mathbb{N}$ for the destructor, inductively defined by $-(-n) :\equiv n$. (The ambiguity will always be resolved by the types.) In fact, the constructor and the destructor are each other's inverse since we also have $-(-(-n)) \equiv -n$, and so by induction $-(-m) = m$ for all $m : \mathbb{N}^-$. By Construction 2.9.9 we get that they are equivalences. ⌟

[41] We implement this in Definition 3.2.1.

sec:lists

2.12.10 *Lists*

One other very common inductive type is that of *lists* over a given type X. Intuitively, a list of elements of type X is a sequence $x_1 x_2 \ldots x_n$ of elements of X, which is possibly the *empty list*, denoted ε. That is, we allow $n = 0$. The number of elements n is called the *length* of the list.

More formally, we have the following:

def:lists

Definition 2.12.11. For any type X, let X^* be the type of lists of elements of X.[42] This is the inductive type with constructors $\varepsilon : X^*$ (*the empty list*) and *concatenation*[43] of type $X \to X^* \to X^*$, taking an element $x : X$ and a list ℓ to the extended list $x\ell$ consisting of x followed by the elements of ℓ. In the extended list $x\ell$, x is called the *head* and ℓ is called the *tail*. ⌟

[42] In other places, the type of lists is denoted $\mathrm{List}\, X$ or $[X]$.

[43] This constructor doesn't have a name for us, since all it does is juxtapose its two arguments. It is often called "cons", as it is a kind of prototypical constructor, since many other kinds of data can be represented in terms of lists. This is the basis of Lisp: the *list* processing programming language.

As an inductive type, X^* comes with a induction principle: Constructing a function f of type $\prod_{\ell : X^*} T(\ell)$, where $T(\ell)$ is a type depending on $\ell : X^*$, may be done by giving:

(1) an element $t_\varepsilon : T(\varepsilon)$; and

(2) a family of functions $g_{x,\ell} : T(\ell) \to T(x\ell)$.

The resulting function satisfies $f(\varepsilon) \equiv t_\varepsilon$ and $f(x\ell) \equiv g_{x,\ell}(f(\ell))$. That is, we can produce a function on all lists by specifying how to handle the empty list and how to reduce the case of an extended list to that of its tail.

For example, we can define a function $\mathsf{len} : X^* \to \mathbb{N}$, giving the length of a list, satisfying $\mathsf{len}(\varepsilon) \equiv 0$ and $\mathsf{len}(x\ell) \equiv \mathsf{succ}(\mathsf{len}(\ell))$.

xca:list-contr

Exercise 2.12.12. Prove that $\mathsf{len} : X^* \to \mathbb{N}$ is an equivalence whenever X is contractible. ⌟

Note that there are no general functions producing the head and tail of an arbitrary list, since the empty list has neither head nor tail. However, we can use binary sums with the one-element type True to define

$$\mathsf{hd} : X^* \to X + \mathsf{True}, \quad \mathsf{tl} : X^* \to X^* + \mathsf{True}$$

satisfying $\mathsf{hd}(\varepsilon) \equiv \mathsf{inr}_{\mathsf{triv}}$, $\mathsf{hd}(x\ell) \equiv \mathsf{inl}_x$, $\mathsf{tl}(\varepsilon) \equiv \mathsf{inr}_{\mathsf{triv}}$, and $\mathsf{tl}(x\ell) \equiv \mathsf{inl}_\ell$.

xca:reverse

Exercise 2.12.13. Define a function of type $X^* \to X^* \to X^*$ that concatenates two lists. (Hint: Use induction on the first argument.)

Use this to define a function $\mathsf{rev} : X^* \to X^*$ that reverses a list. ⌟

xca:concat-assoc

Exercise 2.12.14. Construct an identification of type $\ell_1(\ell_2\ell_3) \xrightarrow{=} (\ell_1\ell_2)\ell_3$ for any $\ell_1, \ell_2, \ell_3 : X^*$. (Hint: Use induction on ℓ_1.) ⌟

We shall see in Theorem 2.22.2 below that X^* is a set whenever X is. Exercise 2.12.14 shows that concatenation of lists is associative, so we don't need to use parentheses to indicate grouping within a list. It also justifies denoting both the binary constructor and concatenation as juxtaposition, with no separate symbol or name.

2.13 *Univalence*

sec:univax

The univalence axiom, to be presented in this section, greatly enhances our ability to produce identifications between the two types and to use the resulting identifications to transport (in the sense of Definition 2.5.4) properties and structure between the types. It asserts that if $\mathcal{U}$ is a universe, and X and Y are types in $\mathcal{U}$, then a specific function, mapping identifications between X and Y to equivalences between X and Y, is an equivalence.

We now define the function that the univalence axiom postulates to be an equivalence.

def:idtoeq

Definition 2.13.1. For types X and Y in a universe $\mathcal{U}$ and a path $p : X \xrightarrow{=} Y$, transport along p in the type family $\mathsf{id}_{\mathcal{U}}$ is a function from X to Y. We recall the definition by path induction from Definition 2.5.4, setting $\mathsf{trp}^{\mathsf{id}_{\mathcal{U}}}_{\mathsf{refl}_X} :\equiv \mathsf{id}_X$. As observed in Section 2.9, transport functions are equivalences, so that the result is a function

$$(p \mapsto \mathsf{trp}^{\mathsf{id}_{\mathcal{U}}}_p) : (X \xrightarrow{=} Y) \to (X \xrightarrow{\simeq} Y). \qquad ⌟$$

We may write $\mathrm{trp}_p^{\mathrm{id}_{\mathcal{U}}}$ more briefly as $\tilde{p}$, which we also use to denote the corresponding function of type $X \to Y$, instead of $X \xrightarrow{\simeq} Y$.

We are ready to state the univalence axiom.

def:univalence

Principle 2.13.2 (Univalence Axiom). Let $\mathcal{U}$ be a universe. Voevodsky's *univalence axiom* for $\mathcal{U}$ postulates that $p \mapsto \tilde{p}$ is an equivalence of type $(X \xrightarrow{=} Y) \to (X \xrightarrow{\simeq} Y)$, for all $X, Y : \mathcal{U}$. Formally, we postulate the existence of a family of elements

$$\mathrm{ua}_{X,Y} : \mathrm{isEquiv}((p : X \xrightarrow{=} Y) \mapsto \mathrm{trp}_p^{\mathrm{id}_{\mathcal{U}}})$$

parameterized by $X, Y : \mathcal{U}$. ⌟

For an equivalence $f : X \xrightarrow{\simeq} Y$, we will adopt the notation $\bar{f} : X \xrightarrow{=} Y$ to denote $(p \mapsto \tilde{p})^{-1}(f)$, the result of applying the inverse function of $(p \mapsto \tilde{p})$, given by $\mathrm{ua}_{X,Y}$, to f. Thus there are identifications of type $\bar{\tilde{p}} \xrightarrow{=} p$ and $\tilde{\bar{f}} \xrightarrow{=} f$. There are also identifications of type $\overline{\mathrm{id}_X} \xrightarrow{=} \mathrm{refl}_X$ and $\overline{g\, f} \xrightarrow{=} \bar{g}\, \bar{f}$ if $g : Y \xrightarrow{\simeq} Z$.

xca:C2

Exercise 2.13.3. Prove that $\mathrm{Bool} \xrightarrow{=} \mathrm{Bool}$ has exactly two elements, $\mathrm{refl}_{\mathrm{Bool}}$ and swap (where swap is given by univalence from the equivalence $\mathrm{Bool} \to \mathrm{Bool}$ interchanging (swapping) the two elements of Bool), and that $\mathrm{swap} \cdot \mathrm{swap} \xrightarrow{=} \mathrm{refl}_{\mathrm{Bool}}$. ⌟

2.14 *Heavy transport*

sec:heavy-transport

In this section we collect useful results on transport in type families that are defined by a type constructor applied to families of types. Typical examples of such 'structured' type families are $Y(x) \to Z(x)$ and $x \xrightarrow{=} x$ parametrized by $x : X$.

def:function-type-families

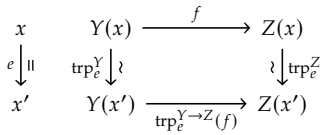
Definition 2.14.1. Let X be a type, and let $Y(x)$ and $Z(x)$ be families of types parametrized by a variable $x : X$. Define $Y \to Z$ to be the type family with $(Y \to Z)(x) :\equiv Y(x) \to Z(x)$. ⌟

Recall from Definition 2.9.14 that an element $f : \prod_{x:X}(Y \to Z)(x)$ is called a map of families, and f is called an equivalence of families, if $f(x) : Y(x) \to Z(x)$ is an equivalence for all $x : X$.

lem:trp-in-function-type

Construction 2.14.2. *Let X be a type, and let $Y(x)$ and $Z(x)$ be types for every $x : X$. Then we have for every $x, x' : X$, $e : x \xrightarrow{=} x'$, $f : Y(x) \to Z(x)$, and $y' : Y(x')$ (see the diagram in the margin)*:

$$\mathrm{trp}_e^{Y \to Z}(f)(y') \xrightarrow{=} \mathrm{trp}_e^{Z}\left(f\left(\mathrm{trp}_{e^{-1}}^{Y}(y')\right)\right).$$

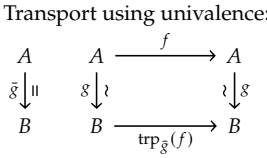


Implementation of Construction 2.14.2. By induction on $e : x \xrightarrow{=} x'$. For $e \equiv \mathrm{refl}_x$, we have $e^{-1} \equiv \mathrm{refl}_x$, and all transports are identity functions of appropriate type. □

An important special case of the above lemma is with $\mathcal{U}$ as parameter type and type families $Y :\equiv Z :\equiv \mathrm{id}_{\mathcal{U}}$. Then $Y \to Z$ is $X \mapsto (X \to X)$. Now, if $A, B : \mathcal{U}$ and $e : A \xrightarrow{=} B$ comes from an equivalence $g : A \xrightarrow{\simeq} B$ by applying the univalence axiom, then the above construction combined with function extensionality yields that for any $f : A \to A$ (see the diagram in the margin)

$$\mathrm{trp}_{\bar{g}}^{X \mapsto (X \to X)}(f) \xrightarrow{=} g \circ f \circ g^{-1}.$$

Transport using univalence:



The following construction is implemented by induction on $e : x \xrightarrow{=} x'$.

lem:trp-in-fx=fgx

CONSTRUCTION 2.14.3. *Let X, Y be types, $f, g : X \to Y$ functions, and let $Z(x) :\equiv (f(x) \xrightarrow{=} g(x))$ for every $x : X$. Then for all x, x' in X, $e : x \xrightarrow{=} x'$, and $i : f(x) \xrightarrow{=} g(x)$ we have*:

$$\mathrm{trp}^Z_e(i) \xrightarrow{=} \mathrm{ap}_g(e) \cdot i \cdot \mathrm{ap}_f(e)^{-1}.$$

xca:trp-in-a/x=b/x

EXERCISE 2.14.4. Implement Construction 2.14.3 in the following special cases, where $Y \equiv X$ and a, b are elements of X:

trp-in-a=x

trp-in-x=a

trp-in-x=x

(1) $\mathrm{trp}^{x \mapsto a \xrightarrow{=} b}_e(i) \xrightarrow{=} i$;

(2) $\mathrm{trp}^{x \mapsto a \xrightarrow{=} x}_e(i) \xrightarrow{=} e \cdot i$;

(3) $\mathrm{trp}^{x \mapsto x \xrightarrow{=} b}_e(i) \xrightarrow{=} i \cdot e^{-1}$;

(4) $\mathrm{trp}^{x \mapsto x \xrightarrow{=} x}_e(i) \xrightarrow{=} e \cdot i \cdot e^{-1}$ (also called *conjugation*). ⌟

There is also a dependent version of Construction 2.14.3, which is again proved by induction on e.[44]

lem:trp-in-fx=Ygx

CONSTRUCTION 2.14.5. *Let $X, Y(x)$ be types and $f(x), g(x) : Y(x)$ for all $x : X$. Let $Z(x) :\equiv (f(x) \xrightarrow{=} g(x))$, with the identification in $Y(x)$, for every $x : X$. Then for all x, x' in X, $e : x \xrightarrow{=} x'$, and $i : f(x) \xrightarrow{=} g(x)$ we have*:

$$\mathrm{trp}^Z_e(i) \xrightarrow{=} \mathrm{po}_e\left(\mathrm{apd}_g(e)\right) \cdot \mathrm{ap}_{\mathrm{trp}^Y_e}(i) \cdot \mathrm{po}_e\left(\mathrm{apd}_f(e)\right)^{-1}.$$

The following construction will be used later in the book.

def:Dan's-lemma

DEFINITION 2.14.6. Let $X, Y(x)$ be types and $f(x) : Y(x)$ for all $x : X$. Given elements $x, x' : X$ and a path $p : x \xrightarrow{=} x'$, we define an equivalence $(f(x) \xrightarrow[e]{=} f(x')) \xrightarrow{\simeq} (f(x) \xrightarrow{=} f(x))$. We do this by inducion on p, using Definition 2.7.1, thereby reducing to the case $(f(x) \xrightarrow{=} f(x)) \xrightarrow{\simeq} (f(x) \xrightarrow{=} f(x))$, which we solve in the canonical way as before. ⌟

xca:trp-in-y=_(x)

EXERCISE 2.14.7. Let X and Y be types with elements $x : X$ and $y : Y$. Let $f, g : X \to Y$ be functions and $e : f \xrightarrow{=} g$ and identification. Define by induction on e and for any $p : y \xrightarrow{=} f(x)$ an identification $\mathrm{trptw}(e, p)$, called *pointwise transport*, of type $\mathrm{trp}^{h \mapsto (y \xrightarrow{=} h(x))}_e(p) \xrightarrow{=} \mathrm{ptw}(e)(x) \cdot p$. ⌟

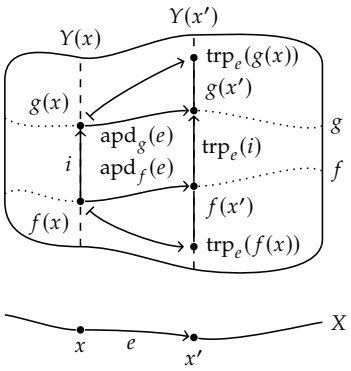
2.15 *Propositions, sets and groupoids*

sec:props-sets-gpds

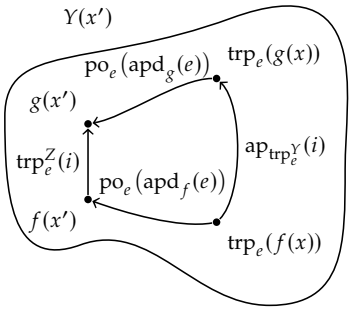
Let P be a type. The property that P has at most one element may be expressed by saying that any two elements are equal. Hence it is encoded by $\prod_{a,b:P}(a \xrightarrow{=} b)$. We shall call a type P with that property a *proposition*, and its elements will be called *proofs* of P. We will use them for doing logic in type theory. The reason for doing so is that the most relevant thing about a logical proposition is whether it has a proof or not. It is therefore reasonable to require for any type representing a logical proposition that all its members are equal.

Suppose P is a proposition. Then English phrases such as "P holds", "we know P", and "we have shown P", will all mean that we have an element of P. We will not use such phrases for types that are not propositions, nor will we discuss knowing P conditionally with a phrase such as "whether P". Similarly, if "Q" is the English phrase for a statement encoded by the proposition P, then the English phrases "Q

[44] We picture this in two stages. First, we show the fiberwise situation as follows:



Here, there's not room to show all that's going on in the fiber $Y(x')$, so we illustrate that as follows:



holds", "we know Q", and "we have shown Q", will all mean that we have an element of P.

Typically, mathematical properties expressed in English as *adjectives* will be encoded by types that are propositions, for in English speech, when you assert that a certain adjective holds, you are simply asserting it, and not providing further information. Examples: the number 6 is *even*; the number 7 is *prime*; the number 28 is *perfect*; consider a *regular* pentagon; consider an *isosceles* triangle.

Sometimes adjectives are used in mathematics, not to refer to properties of an object, but to modify the meaning of a noun, producing a different noun phrase denoting a different mathematical concept. For example, a *directed* graph is a graph, each of whose edges is given a bit of additional information: a direction in which it points. Other examples: *differentiable* manifold; *bipartite* graph; *vector* space; *oriented* manifold.

Let X be a type. If for any $x : X$ and any $y : X$ the identity type $x \xrightarrow{=} y$ is a proposition, then we shall say that X is a *set*. The reason for doing so is that the most relevant thing about a set is which elements it has; distinct identifications of equal elements are not relevant. Alternatively, we shall say that X is a 0-*type*.[45]

[45] Sets are thought to consist of points. Points are entities of dimension 0, which explains why the count starts here. One of the contributions of Vladimir Voevodsky is the extension of the hierarchy downwards, with the notion of proposition, including logic in the same hierarchy. Some authors therefore call propositions (-1)-*types*, and they call contractible types (-2)-*types*.

The following definition introduces notational alternatives commonly used in mathematics.

def:neg-eq-ne

Definition 2.15.1. Let P be a proposition as defined above. We define the *negation* of P by setting $\neg P :\equiv (P \to \emptyset)$.

Let A be a *set*, as defined above, and let a and b be elements of A. We write $a = b$ as alternative notation for the type $a \xrightarrow{=} b$. Formally, we define it as follows.

$$(a = b) :\equiv (a \xrightarrow{=} b)$$

The type $a = b$ is called an *equation*. When it has an element, we say that a and b are *equal*. In line with this definition we also define the type $(a \neq b) :\equiv \neg(a = b)$; an element of it asserts that the elements a and b of the set A are not equal. ⌟

Equations are propositions, so we can speak of them being true or false, and we may use them after the words *if*, *since*, *whether*, and *because* in a sentence. In set theory, everything is a set and all equations $a = b$ are propositions; our definition of $a = b$ is designed to make the transition from set theory to type theory minimally disconcerting.

(Good motivation for the form of the equal sign in the notation $a = b$ is provided by a remark made by Robert Recorde in 1557 in the *Whetstone of Witte*[46]: "And to avoid the tedious repetition of these words *is equal to*, I will set, as I do often in work use, a pair of parallels, or twin lines of one length, thus: =, because no two things can be more equal."[47] In fact, the remark of Recorde presages the approach described in this book, for although those two little lines are congruent, they were not considered to be equal traditionally, since they are in different places, whereas they may be considered to be equal in the presence of univalence, which converts congruences to identifications.)

[46] Robert Recorde and John Kingston. *The whetstone of witte: whiche is the seconde parte of Arithmetike, containyng thextraction of rootes, the cossike practise, with the rule of equation, and the woorkes of surde nombers*. Imprynted at London: By Ihon Kyngstone, 1557. URL: https://archive.org/details/TheWhetstoneOfWitte.

[47] And to auoide the tediouſe repetition of theſe woordeſ : if equalle to : I will ſette aſ I doe often in woorke vſe, a paire of paralleleſ, or Gemowe lineſ of one lengthe, thuſ: ═══ , bicauſe noe .2. thyngeſ, can be moare equalle.

Let X be a type. If for any $x : X$ and any $y : X$ the identity type $x \xrightarrow{=} y$ is a set, then we shall say that X is a *groupoid*, also called a 1-*type*.

The pattern continues. If for any $n : \mathbb{N}$, any $x : X$, and any $y : X$ the identity type $x \xrightarrow{=} y$ is an *n-type*, then we shall say that X is an *$(n+1)$-type*. If X is an n-type, we also say that X is *n-truncated*.

We prove that every proposition is a set, from which it follows by induction that every n-type is an *$(n+1)$-type*.

lem:prop-is-set

LEMMA 2.15.2. *Every type that is a proposition is also a set.*

Proof. Let X be a type and let $f : \prod_{a,b:X}(a \xrightarrow{=} b)$. Let $a, b, c : X$ and let $P(x)$ be the type $a \xrightarrow{=} x$ depending on $x : X$. Then $f(a,b) : P(b)$ and $f(a,c) : P(c)$. By path induction we construct for all $q : b \xrightarrow{=} c$ an identification of type $q \cdot f(a,b) \xrightarrow{=} f(a,c)$. For this it suffices to observe that $\mathrm{refl}_b \cdot f(a,b)$ and $f(a,b)$ are equal by definition. Since a is arbitrary, it follows that any $q : b \xrightarrow{=} c$ can be identified with $f(b,c) \cdot f(b,b)^{-1}$, which doesn't depend on q. Hence X is a set. □

A more interesting example of a set is Bool.

lem:isset-bool

LEMMA 2.15.3. Bool *is a set.*

Proof. The following elegant, self-contained proof is due to Simon Huber. For proving $p \xrightarrow{=} q$ for all $b, b' : \mathrm{Bool}$ and $p, q : b \xrightarrow{=} b'$, it suffices (by induction on q) to show $p \xrightarrow{=} \mathrm{refl}_b$ for all $b : \mathrm{Bool}$ and $p : b \xrightarrow{=} b$. To this end, define by induction on $b, b' : \mathrm{Bool}$, a type $C(b, b', p)$ for all $p : b \xrightarrow{=} b'$, by setting $C(\mathrm{yes}, \mathrm{yes}, p) :\equiv (p \xrightarrow{=} \mathrm{refl}_{\mathrm{yes}})$, $C(\mathrm{no}, \mathrm{no}, p) :\equiv (p \xrightarrow{=} \mathrm{refl}_{\mathrm{no}})$, and arbitrary in the other two cases. By induction on b one proves that $C(b, b, p) \xrightarrow{=} (p \xrightarrow{=} \mathrm{refl}_b)$ for all p. Hence it suffices to prove $C(b, b', p)$ for all $b, b' : \mathrm{Bool}$ and $p : b \xrightarrow{=} b'$. By induction on p this reduces to $C(b, b, \mathrm{refl}_b)$, which is immediate by induction on $b : \mathrm{Bool}$. □

We now collect a number of useful results on propositions.

lem:prop-utils

prop-utils-false-true

prop-utils-codom

prop-utils-pi

prop-utils-times

prop-utils-sum

prop-utils-lem

LEMMA 2.15.4. *Let A be a type, and let P and Q propositions. Let $R(a)$ be a proposition depending on $a : A$. Then we have*:

(1) False *and* True *are propositions*;

(2) $A \to P$ *is a proposition*;

(3) $\prod_{a:A} R(a)$ *is a proposition*;

(4) $P \times Q$ *is a proposition*;

(5) *if A is a proposition, then $\sum_{a:A} R(a)$ is a proposition*;

(6) $P \amalg \neg P$ *is a proposition.*

Proof. (1): If $p, q : \mathrm{False}$, then $p \xrightarrow{=} q$ holds by induction for False. If $p, q : \mathrm{True}$, then $p \xrightarrow{=} q$ is proved by double induction, which reduces the proof to observing that $\mathrm{refl}_{\mathrm{triv}} : \mathrm{triv} \xrightarrow{=} \mathrm{triv}$.

(2): If $p, q : A \to P$, then $p \xrightarrow{=} q$ is proved by first observing that p and q are functions which, by function extensionality, can be identified if they have equal values $p(x) = q(x)$ in P for all x in A. This is actually the case since P is a proposition.

(3): If $p, q : \prod_{a:A} R(a)$ one can use the same argument as for $A \to P$ but now with *dependent* functions p, q.

(4): If $(p_1, q_1), (p_2, q_2) : P \times Q$, then $(p_1, q_1) \xrightarrow{=} (p_2, q_2)$ is proved componentwise. Alternatively, we may regard this case as a special case of (5).

(5): Given $(a_1, r_1), (a_2, r_2) : \sum_a R(a)$, we must establish that $(a_1, r_1) \xrightarrow{=} (a_2, r_2)$. Combining the map in Definition 2.10.1 with the identity type in Definition 2.7.3 yields a map $\left(\sum_{u : a_1 = a_2} \mathrm{trp}^Y_u(r_1) = r_2\right) \to ((a_1, r_1) \xrightarrow{=} (a_2, r_2))$, so it suffices to construct an element in the source of the map. Since A is a proposition, we may find $u : a_1 = a_2$. Since $R(a_2)$ is a proposition, we may find $v : \mathrm{trp}^Y_u(r_1) = r_2$. The pair (u, v) is what we wanted to find.

(6): If $p, q : P \amalg \neg P$, then we can distinguish four cases based on inl/inr, see Section 2.8. In two cases we have both P and $\neg P$ and we are done. In the other two, either $p \equiv \mathrm{inl}_{p'}$ and $q \equiv \mathrm{inl}_{q'}$ with $p', q' : P$, or $p \equiv \mathrm{inr}_{p'}$ and $q \equiv \mathrm{inr}_{q'}$ with $p', q' : \neg P$. In both these cases we are done since P and $\neg P$ are propositions. □

Several remarks can be made here. First, the lemma supports the use of False and True as truth values, and the use of $\to, \prod, \times$ for implication, universal quantification, and conjunction, respectively. Since False is a proposition, it follows by (2) above that $A \to \emptyset$ is a proposition for any type A. As noted before, (2) is a special case of (3).

Notably absent in the lemma above are disjunction and existential quantification. This has a simple reason: True ∐ True has two distinct elements $\mathrm{inl}_{\mathrm{triv}}$ and $\mathrm{inr}_{\mathrm{triv}}$, an is therefore *not* a proposition. Similarly, $\sum_{n : \mathbb{N}} \mathrm{True}$ has infinitely many distinct elements (n, triv) and is not a proposition. We will explain in Section 2.16 how to work with disjunction and existential quantification for propositions.

The lemma above has a generalization from propositions to n-types which we state without proving. (The proof goes by induction on n, with the lemma above serving as the base case where n is -1.)

lem:level-n-utils

level-n-utils-codom

level-n-utils-pi

level-n-utils-times

level-n-utils-sum

Lemma 2.15.5. *Let A be a type, and let X and Y be n-types. Let $Z(a)$ be an n-type depending on $a : A$. Then we have*:

(1) *$A \to X$ is an n-type;*

(2) *$\prod_{a : A} Z(a)$ is an n-type;*

(3) *$X \times Y$ is an n-type.*

(4) *if A is an n-type, then $\sum_{a : A} Z(a)$ is an n-type;*

We formalize the definitions from the start of this section.

def:isSet

Definition 2.15.6.

$$\begin{aligned}
\mathrm{isProp}(P) &:\equiv \prod_{p,q : P} (p \xrightarrow{=} q) \\
\mathrm{isSet}(S) &:\equiv \prod_{x,y : S} \mathrm{isProp}(x \xrightarrow{=} y) \equiv \prod_{x,y : S} \prod_{p,q : (x \xrightarrow{=} y)} (p \xrightarrow{=} q) \\
\mathrm{isGrpd}(G) &:\equiv \prod_{g,h : G} \mathrm{isSet}(g \xrightarrow{=} h) \equiv \ldots
\end{aligned}$$ ⌟

lem:isX-is-prop

Lemma 2.15.7. *For any type A, the following types are propositions*:

(1) $\mathrm{isContr}(A)$;

(2) $\mathrm{isProp}(A)$;

(3) $\mathrm{isSet}(A)$;

(4) isGrpd(A);

(5) *the type that encodes whether A is an n-type, for $n \geq 0$.*

Consistent with that, we will use identifiers starting with "is" only for names of types that are propositions. Examples are isSet(A) and isGrpd(A), and also isEquiv(f).

Proof. Recall that isContr(A) is $\sum_{a:A} \prod_{y:A} (a \xrightarrow{=} y)$. Let (a, f) and (b, g) be elements of the type isContr(A). By Definition 2.10.1, to give an element of $(a, f) \xrightarrow{=} (b, g)$ it suffices to give an $e : a \xrightarrow{=} b$ and an $e' : f \xrightarrow[e]{=} g$. For e we can take $f(b)$; for e' it suffices by Definition 2.7.3 to give an $e'' : \mathrm{trp}_e\, f \xrightarrow{=} g$. Clearly, A is a proposition and hence a set by Lemma 2.15.2. Hence the type of g is a proposition by Lemma 2.15.4(3), which gives us e''.

We leave the other cases as exercises. □

xca:isX-is-prop

Exercise 2.15.8. Make sure you understand that isProp(P) is a proposition, using the same lemmas as for isContr(A). Show that isSet(S), isGrpd(G) and isEquiv(f) are propositions. ⌟

The following exercise shows that the inductive definition of n-types can indeed start with n as -2, where we have the contractible types.

xca:prop-contractible=

Exercise 2.15.9. Given a type P, show that P is a proposition if and only if $p \xrightarrow{=} q$ is contractible, for any $p, q : P$. ⌟

rem:diagram

Remark 2.15.10. We now present the notion of a *diagram*. A diagram is a graph whose vertices are types and whose edges are functions or identifications. This means the edges have a direction. Usually there is one vertex with only outgoing edges, called the *source*, and one with only incoming edges, called the *sink*. Here is an example.

$$\begin{array}{ccc} X & \xrightarrow{f} & Y \\ \downarrow{\scriptstyle p} & & \downarrow{\scriptstyle q} \\ S & \xrightarrow{g} & T \end{array}$$

The information conveyed by this diagram to the reader is that X, Y, S, and T are types, and that f, g, p, and q are functions; moreover, f is of type $X \to Y$, g is of type $S \to T$, p is of type $X \to S$, and q is of type $Y \to T$. The source is the left upper vertex X and the sink is the lower right vertex T.

Observe that we can travel through the diagram from X to T by following first the arrow labeled f and then the arrow labelled q. Consequently, the composite function $q \circ f$ is of type $X \to T$.

There is another route from X to T : we could follow first the arrow labeled p and then the arrow labelled g. Consequently, the composite function $g \circ p$ is also of type $X \to T$.

We say that a diagram *is commutative by definition* if, whenever there are two routes from one vertex to another, the corresponding composite functions are equal by definition. For example, in the diagram above, the condition would be that $g \circ p \equiv q \circ f$.

When the sink of the diagram is a set, then equality of functions into the sink is a proposition, and we may consider whether two such functions are equal. In that case, we say that a diagram *is commutative* if, whenever

there are two routes from one vertex to another, the corresponding composite functions are equal. For example, in the diagram above, the condition would be that $g \circ p = q \circ f$.

In general, a diagram is a visual way to represent identity types. For example, if in the above diagram the type T is not a set, then the diagram represents the identity type $g \circ p \xrightarrow{=} q \circ f$. To give an element of such an identity type (which may or may not be possible) is called to *fill* the diagram, and such an element is then called a *filler* of the diagram.

There are other sorts of diagrams. For example, identifications may be composed, and thus we may have a diagram of identifications between elements of the same type. For example, suppose W is a type, suppose that x, y, s, and t are elements of W, and consider the following diagram.

$$\begin{array}{ccc} x & \xrightarrow[=]{f} & y \\ \| & & \| \\ \downarrow{\scriptstyle p} & & \downarrow{\scriptstyle q} \\ s & \xrightarrow[=]{g} & t \end{array}$$

It indicates that f is of type $x \xrightarrow{=} y$, g is of type $s \xrightarrow{=} t$, p is of type $x \xrightarrow{=} s$, and q is of type $y \xrightarrow{=} t$. We may also consider whether such a diagram is commutative by definition, or, in the case where all the identity types are sets, is commutative. Such diagrams are again a visual way to represent identity types.[48] For example, the above diagram represents the identity type $g \circ p \xrightarrow{=} q \circ f$, and we also speak of *filling* diagrams of identifications. For a concrete example of filling such a diagram, see the naturality square in Definition 2.6.5. ⌟

[48] When diagrams get more complicated, the information they convey is not always sufficient to find out which identity type(s) they represent. In such cases additional information will be provided.

2.16 *Propositional truncation and logic*

sec:prop-trunc

As explained in Section 2.15, the type formers $\to, \prod, \times$ can be used with types that are propositions for the logical operations of implication, universal quantification, and conjunction, respectively. Moreover, True and False can be used as truth values, and $\neg$ can be used for negation. We have also seen that $\amalg$ and Σ can lead to types that are not propositions, even though the constituents are propositions. This means we are still lacking disjunction ($P \vee Q$) and existence ($\exists_{x:X} P(x)$) from the standard repertoire of logic, as well as the notion of *non-emptiness* of a type. In this section we explain how to implement these three notions.

To motivate the construction that follows, consider non-emptiness of a type T. In order to be in a position to encode the mathematical assertion expressed by the English phrase "T is non-empty", we will need a proposition P. The proposition P will have to be constructed somehow from T. Any element of T should somehow give rise to an element of P, but, since all elements of propositions are equal to each other, all elements of P arising from elements of T should somehow be made to equal each other. Finally, any proposition Q that is a consequence of having an element of T should also be a consequence of P.

We define now an operation called propositional truncation,[49] that enforces that all elements of a type become equal.

[49] The name "truncation" is slightly misleading since it suggests leaving something out, whereas the correct intuition is one of adding identifications so everything becomes equal.

def:prop-trunc

Definition 2.16.1. Let T be a type. The *propositional truncation* of T is the type $\|T\|$ defined by the following constructors:

(1) an *element* constructor $|t| : \|T\|$ for all $t : T$;

(2) an *identification* constructor providing an identification of type $x \xrightarrow{=} y$ for all $x, y : \|T\|$.

The identification constructor ensures that $\|T\|$ is a proposition. The induction principle states that, for any family of propositions $P(x)$ parametrized by a variable $x : \|T\|$, in order to prove $\prod_{x:\|T\|} P(x)$, it suffices to prove $\prod_{t:T} P(|t|)$. In other words, in order to define a function $f : \prod_{x:\|T\|} P(x)$, it suffices to give a function $g : \prod_{t:T} P(|t|)$. Moreover, the function f will satisfy $f(|t|) \equiv g(t)$ for all $t : T$. ⌟

Consider the special case where the family $P(x)$ is constant. We see that any function $g : T \to P$ to a proposition P yields a (unique) function $f : \|T\| \to P$ satisfying $f(|t|) \equiv g(t)$ for all $t : T$.[50] A useful consequence of this recursion principle is that, for any proposition P, precomposition with $|_|$ is an equivalence of type

$$(\|T\| \to P) \quad \xrightarrow{\simeq} \quad (T \to P).$$

This is called *the universal property of propositional truncation*.

[50] Given $t, t' : T$, we have an identification of type $|t| \xrightarrow{=} |t'|$. The existence of the function g implies that we have an identification of type $g(|t|) \xrightarrow{=} g(|t'|)$, and hence an identification of type $f(t) \xrightarrow{=} f(t')$. Thus a necessary condition for the existence of g is the existence of identifications of type $f(t) \xrightarrow{=} f(t')$. That justifies the the hypothesis that P is proposition.

def:non-empty

Definition 2.16.2. Let T be a type. We call T *non-empty* if we have an element of $\|T\|$.[51] ⌟

[51] We may alternatively say that T is *inhabited*, in order to avoid confusion with the concept of T *not being empty*, which would be represented by the proposition $\neg(T \xrightarrow{=} \emptyset)$, which is equivalent to $\neg\neg T$.

When we view propositional truncation as an operation on types, the type of $\|_\|$ is $\mathcal{U} \to \mathcal{U}$. However, that view does not take into account that $\|T\|$ is a proposition. It is more informative to pack this information into the codomain of the operation and let $\|_\|$ have the type $\mathcal{U} \to \sum_{X:\mathcal{U}} \mathrm{isProp}(X)$. The type $\sum_{X:\mathcal{U}} \mathrm{isProp}(X)$ is also denoted as $\mathrm{Prop}_{\mathcal{U}}$ and even as Prop. See Example 2.20.6 for more information.

Now that propositional truncation is available, we are ready to define logical disjunction and existence.

Definition 2.16.3. Given propositions P and Q, define their *disjunction* by $(P \vee Q) :\equiv \|P \amalg Q\|$. It expresses the property that P is true or Q is true. ⌟

Definition 2.16.4. Given a type X and a family $P(x)$ of propositions parametrized by a variable x of type X, define a proposition that encodes the property that there exists a member of the family for which the property is true by $(\exists_{x:X} P(x)) :\equiv \|\sum_{x:X} P(x)\|$. It expresses the property that there *exists* an element $x : X$ for which the property $P(x)$ is true; the element x is not given explicitly. ⌟

The following logical quantifier could have been defined earlier, since it doesn't use propositional truncation. We present it now, for completeness.

Definition 2.16.5. Given a type X and a family $P(x)$ of propositions parametrized by a variable x of type X, define a proposition that encodes the property that there exists a *unique* member of the family for which the property is true by the proposition $(\exists!_{x:X} P(x)) :\equiv \mathrm{isContr}(\sum_{x:X} P(x))$. ⌟

Exercise 2.16.6. Given $x : \|T\|$, prove that $\exists_{t:T}(x = |t|)$. ⌟

xca:prop-trivia-1

Exercise 2.16.7. Suppose P is a proposition. Produce an equivalence of type $P \xrightarrow{\simeq} \|P\|$. ⌟

The exercise above us to easily convert elements of type $\|P\|$ to elements of type P when P is a proposition.

def:connected

Definition 2.16.8. Let A be a type. For any element a of A, the type $A_{(a)} :\equiv \sum_{x:A} \|a \xrightarrow{=} x\|$ is called the *connected component* of a in A.[52] We say that elements x, y of A are *in the same component* of A if $\|x \xrightarrow{=} y\|$. The type A is called *connected*[53] if it is non-empty with all elements in the same component. Formally, this property is encoded by the following proposition.

$$\mathrm{isConn}(A) :\equiv \|A\| \times \prod_{x,y:A} \|x \xrightarrow{=} y\|. \qquad \lrcorner$$

Note that the empty type $\emptyset$ is *not* connected.

One can view being connected as a weak form of being contractible – without direct access to a center and to identifications of elements.

xca:component-connected

Exercise 2.16.9. Show that the component of a in A is connected. Show that elements in the same component have the same *propositional* properties, that is, for any $P : A \to \mathrm{Prop}$, $P(x) \xrightarrow{=} P(y)$ for any $x, y : A$ with $\|x = y\|$. $\lrcorner$

xca:connected-trivia

Exercise 2.16.10. Let A be a connected type, and suppose that $a \xrightarrow{=} a$ is a proposition for every $a : A$. Show that A is contractible. Show that any connected set is contractible. $\lrcorner$

xca:connected-trivia-1

Exercise 2.16.11. Let A be a type and $B : A \to \mathcal{U}$ a type family. Show that $\sum_{x:A} B(x)$ is connected when A is connected and $B(x)$ is connected for any $x : A$. $\lrcorner$

xca:connected-trivia-2

Exercise 2.16.12. New: Let A be a type and $B : A \to \mathcal{U}$ a type family. Show: if $\sum_{a:A} B(a)$ is connected, then so is $\sum_{a:A} \|B(a)\|$. $\lrcorner$

In the following definition we introduce the adverb *merely*, which serves as a quicker way to say *the propositional truncation of* in English speech.

def:merely

Definition 2.16.13. What we mean by *merely* constructing an element of a type T is constructing an element of $\|T\|$. $\lrcorner$

For example, a type is non-empty if it *merely has an element*, and a type is connected if any two elements can be *merely identified* with each other.

2.17 *More on equivalences; surjections and injections*

sec:more-on-equivalences

In this section we collect a number of useful results on equivalences.

Consider the function $f : \mathbb{1} \to \mathbb{2}$ that is constant 0. The fibers of f at 0 and 1 are $\sum_{x:\mathbb{1}} 0 \xrightarrow{=} 0$ and $\sum_{x:\mathbb{1}} 1 \xrightarrow{=} 0$, respectively. The latter fiber is not contractible: having an element of it would mean having an element of $1 \xrightarrow{=} 0$, which would in turn lead to an element in False (using a similar reasoning as in Section 2.12.1). Hence f is not an equivalence. Observe that both fibers are propositions, that is, contain at most one element.

As a function between sets f is an injection (one-to-one), but not a surjection. We need these important concepts for types in general. We define them as close as possible to their usual meaning in set theory: a function from A to B is surjective if the preimage of any $b : B$ is non-empty, and injective if such preimages contain at most one element. This motivates the following definitions.

[52] In Section 2.20 we will define the notion of subtype. It will turn out that $A_{(a)}$ is a subtype of A.

[53] In Exercise 2.22.5 below we will define the *set of connected components* of a type.

def:surjection

Definition 2.17.1. A function $f : A \to B$ is a *surjection*, or is *surjective*, if for all $b : B$ there exists an $a : A$ such that $b \overset{=}{\to} f(a)$, that is, $\exists_{a:A}(b \overset{=}{\to} f(a))$.[54] ⌟

def:injection

Definition 2.17.2. A function $f : A \to B$ is an *injection*, or is *injective*, if $f^{-1}(b)$ is a proposition for all $b : B$. The property of being an injection is encoded by the type $\mathrm{isInj}(f) :\equiv \prod_{b:B} \mathrm{isProp}(f^{-1}(b))$. ⌟

xca:inj-sets

Exercise 2.17.3. Let A be a type and B a set and $f : A \to B$ a function. Give maps in both directions between the proposition $\mathrm{isInj}(f)$ and the type $\prod_{a,a':A}((f(a) \overset{=}{\to} f(a')) \to (a \overset{=}{\to} a'))$. Give a function $f : \mathrm{True} \to \mathrm{Set}$ that is not injective. ⌟

lem:inj+surj

Lemma 2.17.4. *For all types A, B, a function $f : A \to B$ is an equivalence if and only if f is an injection and a surjection.*

Proof. If $f : A \to B$ is an equivalence, then all fibers are contractible, so f is both an injection and a surjection. Conversely, if f is both injective and surjective, we show that $f^{-1}(b)$ is contractible, for each $b : B$. Being contractible is a proposition, so by Definition 2.16.1 we can drop the truncation in $\|\sum_{a:A} b \overset{=}{\to} f(a)\|$. Now apply injectivity.[55] □

[54] A function $f : A \to B$ is a *split surjection* if for all $b : B$ we have an $a : A$ with $b \overset{=}{\to} f(a)$, in other words, we have a function of type $\prod_{b:B} \sum_{a:A}(b \overset{=}{\to} f(a))$. This is equivalent to saying we have a function $g : B \to A$ and an identification $p : f \circ g \overset{=}{\to} \mathrm{id}_B$ (such a g is called a *section* of f).

[55] This argument applies generally: Any non-empty proposition is contractible.

If the types A and B in the above lemma are *sets*, then we call equivalences between A and B also *bijections*.

cor:inj+connected

Corollary 2.17.5. *Let A, B be types such that A is non-empty and B is connected. Then any injection $f : A \to B$ is an equivalence.*

Proof. By Lemma 2.17.4 it suffices to show that f is surjective. This is a proposition, so by Definition 2.16.1 and $\|A\|$ we may assume $a : A$, so $f(a) : B$. By $\prod_{x,y:B}\|x \overset{=}{\to} y\|$ we now get that all preimages under f are non-empty. □

lem:whenisahasespaceconnected

Lemma 2.17.6. *Let $f : X \to Y$ be a surjective map from a connected type X. Then Y is connected too.*

Proof. For any map $f : X \to Y$ between arbitrary types, if $y, y' : Y$ and we are given $x, x' : X$, $p : y \overset{=}{\to} f(x)$, $p' : y' \overset{=}{\to} f(x')$ and $q : x \overset{=}{\to} x'$, then we have a path between y and y' given by the composite

$$y \xrightarrow[p]{=} f(x) \xrightarrow[f(q)]{=} f(x') \xrightarrow[p'^{-1}]{=} y'.$$

Now the lemma follows by eliminating the propositional truncations in the assumptions, using that the conclusion is a proposition. □

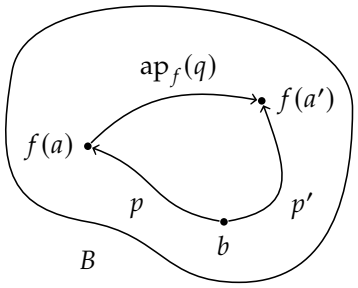
con:fib-vs-path

Construction 2.17.7. *For every $f : A \to B$, $b : B$, and $z, z' : f^{-1}(b)$, there is an equivalence*

eq:fib_vs_path

$$(z \overset{=}{\to} z') \overset{\simeq}{\to} \mathrm{ap}_f^{-1}(\mathrm{snd}\, z' \cdot \mathrm{snd}\, z^{-1}). \tag{2.17.1}$$

Implementation of Construction 2.17.7. We can construct this equivalence for $z \equiv (a, p)$ and $z' \equiv (a', p')$, where $a, a' : A$, $p : b \overset{=}{\to} f(a)$ and $p' : b \overset{=}{\to} f(a')$, as the composition



$$\begin{aligned}(z \xrightarrow{=} z') &\equiv \big((a,p) \xrightarrow{=} (a',p')\big)\\ &\xrightarrow{\simeq} \sum_{q\,:\,a \xrightarrow{=} a'} p \xrightarrow[q]{=} p'\\ &\xrightarrow{\simeq} \sum_{q\,:\,a \xrightarrow{=} a'} \mathrm{ap}_f(q) \cdot p \xrightarrow{=} p'\\ &\xrightarrow{\simeq} \sum_{q\,:\,a \xrightarrow{=} a'} p' \cdot p^{-1} \xrightarrow{=} \mathrm{ap}_f(q)\\ &\equiv \mathrm{ap}_f^{-1}(p' \cdot p^{-1}).\end{aligned}$$

The second equivalence relies on Definition 2.7.3 and Construction 2.14.3. □

lem:inj-ap

Lemma 2.17.8. *A function $f : A \to B$ is an injection if and only if each induced function $\mathrm{ap}_f : (a \xrightarrow{=} a') \to (f(a) \xrightarrow{=} f(a'))$ is an equivalence, for all $a, a' : A$.*[56]

[56] *Warning*: If A and B are sets, then each ap_f is an equivalence if and only if all implications $(f(a) \xrightarrow{=} f(a')) \to (a \xrightarrow{=} a')$ hold, but this is in general not sufficient.

Proof. It follows directly from (2.17.1) that if ap_f is an equivalence, then $f^{-1}(b)$ is a proposition, as all its identity types (fibers of ap_f) are contractible.

On the other hand, if we fix $a, a' : A$ and $p : f(a) \xrightarrow{=} f(a')$, then (2.17.1) applied to $b :\equiv f(a)$, $z :\equiv (a, \mathrm{refl}_{f(a)})$ and $z' :\equiv (a', p)$, gives $\mathrm{ap}_f^{-1}(p) \xrightarrow{\simeq} (z \xrightarrow{=} z')$, which shows that if each $f^{-1}(b)$ is a proposition, then ap_f is an equivalence. □

cor:fib-vs-path
set-fib-vs-path
set-fib-vs-path-point
conn-fib-vs-path
conn-fib-vs-path-point

Corollary 2.17.9. *Let A and B be types and let $f : A \to B$ be a function. Then we have*:

(1) *All fibers of f are $(n+1)$-types if and only if all fibers of each map induced by f on identity types are n-types*;

(2) *If A is connected and $a : A$, then all fibers of f are $(n+1)$-types if and only if all fibers of $\mathrm{ap}_f : (a \xrightarrow{=} a) \to (f(a) \xrightarrow{=} f(a))$ are n-types*;

(3) *If A and B are connected, then f is an equivalence if and only if each map induced by f on identity types is an equivalence*;

(4) *If A and B are connected and $a : A$, then f is an equivalence if and only if $\mathrm{ap}_f : (a \xrightarrow{=} a) \to (f(a) \xrightarrow{=} f(a))$ is an equivalence.*

Proof. (1) When n is -2 this is Lemma 2.17.8 and the proof for $n \geq -1$ is similar. (2) By (1) and Exercise 2.16.9. (3) By Lemma 2.17.8 and Corollary 2.17.5. (4) By (3) and Exercise 2.16.9. □

xca:sum-equivalences

Exercise 2.17.10. Let $A, B : \mathcal{U}$, $F : A \to \mathcal{U}$ and $G : B \to \mathcal{U}$, and $f : A \xrightarrow{\simeq} B$ and $g : \prod_{a:A}(F(a) \xrightarrow{\simeq} G(f(a)))$. Give an equivalence from $\sum_{a:A} F(a)$ to $\sum_{b:B} G(b)$. (An important special case is $F \equiv G \circ f$.) ⌟

Another application of propositional truncation is the notion of image.

def:prop-image

Definition 2.17.11. Let A, B be types and let $f : A \to B$. We define the *image* of f as

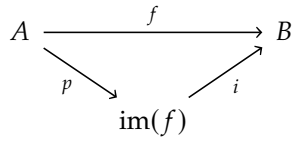
$$\mathrm{im}(f) :\equiv \sum_{y:B} \exists_{x:A} (y \xrightarrow{=} f(x)).$$ ⌟

Note that $(\exists_{x:A}(y \overset{=}{\to} f(x))) \equiv \|f^{-1}(y)\|$, the propositional truncation of the fiber. For this reason, $\mathrm{im}(f)$ is called the *propositional* image. Later we will meet other notions of image, based on other truncation operations.

xca:unique-fact-image

Exercise 2.17.12. Show that the image of $f : A \to B$ induces a factorization $f \overset{=}{\to} i \circ p$, visualized by the following diagram[57]

[57] This diagram actually commutes by definition.



where p is surjective and i is injective. Show that the following type of image factorizations of $f : A \to B$ is contractible:

$$\sum_{C:\mathcal{U}} \sum_{g:A\to C} \sum_{h:C\to B} ((f \overset{=}{\to} h \circ g) \times \mathrm{isSurj}(g) \times \mathrm{isInj}(h)).$$ ⌟

Thus the image factorization of f is a 6-tuple. For convenience we may simplify and speak of the "image factorization $f \overset{=}{\to} h \circ g$." Here C is implicit in the types of g and h. The particular identification of f with $h \circ g$ follows from the context, as do the proofs that g is an injection and h is a surjection.

xca:prop-image-is-set

Exercise 2.17.13. Let A be a type and B as set and $f : A \to B$. Show that $\mathrm{im}(f)$ is a set. ⌟

xca:all-prop-image

Exercise 2.17.14. Let $f : A \to B$ for A and B types, and let $P(b)$ be a proposition depending on $b : B$. Show that $\prod_{z:\mathrm{im}(f)} P(\mathrm{fst}(z))$ if and only if $\prod_{a:A} P(f(a))$. ⌟

2.18 *Decidability, excluded middle and propositional resizing*

sec:decidability

Recall from Lemma 2.15.4(6) that $P \amalg \neg P$ is a proposition whenever P is a proposition.

def:decidability

Definition 2.18.1. A proposition P is called *decidable* if $P \amalg \neg P$ holds. ⌟

In traditional mathematics, it is usually assumed that every proposition is decidable. This is expressed by the following principle, commonly abbreviated LEM.

pri:lem

Principle 2.18.2 (Law of Excluded Middle). For every proposition P, the proposition $P \amalg \neg P$ holds. ⌟

(The "middle" ground excluded by this principle is the possibility that there is a proposition that is neither true nor false.)

Type theory is born in a constructivist tradition which aims at developing as much mathematics as possible without assuming the Law of Excluded Middle.[58] Following this idea, we will explicitly state whenever we are assuming the Law of Excluded Middle.

[58] Besides any philosophical reasons, there are several pragmatic reasons for developing constructive mathematics. One is that proofs in constructive mathematics can be executed as programs, and another is that the results also hold in non-standard models, for instance a model where every type has a topological structure, and all constructions are continuous. See also Footnote 14.

xca:lem-prop

Exercise 2.18.3. Show that the Law of Excluded Middle is equivalent to asserting that the map $(\mathrm{yes} = _) : \mathrm{Bool} \to \mathrm{Prop}$ is an equivalence. ⌟

A useful consequence of the Law of Excluded Middle is the principle of "proof by contradiction": to prove a proposition P, assume its negation $\neg P$ and derive a contradiction. Without the Law of Excluded Middle, this proves only the double negation of P, that is $\neg\neg P$. However, with the Law of Excluded Middle, one can derive P from the latter: indeed, according to the Law of Excluded Middle, either P or $\neg P$ holds; but $\neg P$ leads to a contradiction by hypothesis, making P hold necessarily.

xca:dne-lem

Exercise 2.18.4. Show that, conversely, LEM follows from the principle of *double-negation elimination*: For every proposition P, if $\neg\neg P$, then P holds. ⌟

Remark 2.18.5. We will later encounter a weaker version of the Law of Excluded Middle, called the Limited Principle of Omniscience (**??** 3.6.22), which is often enough.[59] ⌟

Sometimes we make use of the following, which is another consequence of the Law of Excluded Middle:

pri:prop-resizing

Principle 2.18.6 (Propositional Resizing). For any pair of nested universes $\mathcal{U} : \mathcal{U}'$, the map $(P \mapsto P) : \mathrm{Prop}_{\mathcal{U}} \to \mathrm{Prop}_{\mathcal{U}'}$ is an equivalence.[61] ⌟

xca:lem-prop-sizing

Exercise 2.18.7. Show that if the Law of Excluded Middle holds, then Propositional Resizing holds. ⌟

2.19 *The replacement principle*

sec:replacement

In this section we fix a universe $\mathcal{U}$. We think of types $A : \mathcal{U}$ as *small* compared to arbitrary types, which are then *large* in comparison.[62] Often we run into types that are not in $\mathcal{U}$ (small) directly, but are nevertheless equivalent to types in $\mathcal{U}$.

def:ess-loc-small

Definition 2.19.1. We say that a type A is *essentially $\mathcal{U}$-small* if we have a type $X : \mathcal{U}$ and an equivalence $A \xrightarrow{\sim} X$. And A is *locally $\mathcal{U}$-small* if all its identity types are essentially $\mathcal{U}$-small. ⌟

Note that $\sum_{X:\mathcal{U}}(A \xrightarrow{\sim} X)$, the type expressing that A is essentially $\mathcal{U}$-small, is a proposition by the univalence axiom for $\mathcal{U}$. Of course, any $A : \mathcal{U}$ is essentially $\mathcal{U}$-small, and any essentially $\mathcal{U}$-small type is locally $\mathcal{U}$-small.

To show that a type is locally $\mathcal{U}$-small we have to give a reflexive relation $\mathrm{Eq}_A : A \to A \to \mathcal{U}$ that induces, by path induction, a family of equivalences $(x \xrightarrow{=} y) \xrightarrow{\sim} \mathrm{Eq}_A(x, y)$.

Exercise 2.19.2. Show that $\mathcal{U}$ is locally $\mathcal{U}$-small, and investigate the closure properties of essentially and locally $\mathcal{U}$-small types. (For instance, show that if $A : \mathcal{U}$ and $B(x)$ is a family of locally $\mathcal{U}$-small types parametrized by $x : A$, then $\prod_{x:A} B(x)$ is locally $\mathcal{U}$-small.) ⌟

Remark 2.19.3. Note that propositional resizing (**??** 2.18.6) equivalently says that any proposition is essentially $\mathcal{U}$-small, where we may take $\mathcal{U}$ to be the smallest universe $\mathcal{U}_0$. When we assume this, we get that any set is locally $\mathcal{U}_0$-small. ⌟

We will make use of the following principle (recall the definition of the image, Definition 2.17.11).

pri:replacement

Principle 2.19.4 (Replacement). For any map $f : A \to B$ from an essentially $\mathcal{U}$-small type A to a locally $\mathcal{U}$-small type B, the image $\mathrm{im}(f)$ is essentially $\mathcal{U}$-small. ⌟

This is reminiscent of the replacement principle of set theory which states that for a large (class-sized) function with domain a small set and codomain the class V of all small sets, the image is again a small set. This follows from our replacement principle, assuming propositional resizing, or the even stronger principle of the excluded middle.

The replacement principle can be proved using the join construction of the image, cf. Rijke[63], which uses as an assumption that the universes are closed under pushouts.[64]

[59] As the naming indicates, we can think of the Law of Excluded Middle itself as an omniscience principle, telling us for every proposition P, whether P is true or false. It was this interpretation of the Law of Excluded Middle that led Brouwer to reject it in his 1908 paper on *De onbetrouwbaarheid der logische principes*.[60]

[60] Mark van Atten and Göran Sundholm. "L.E.J. Brouwer's 'Unreliability of the Logical Principles A New Translation, with an Introduction". In: *History and Philosophy of Logic* 38.1 (2017), pp. 24–47. DOI: 10.1080/01445340.2016.1210986. arXiv: 1511.01113.

[61] The map $P \mapsto P$ is welltyped by *cumulativity* of the universes, that is, by point (4) of Section 2.3. Note that the map is not the identity function due to its type.

[62] The terminology *small/large* is also known from set theory, where classes are large collections, and sets are small collections.

[63] Egbert Rijke. *The join construction*. 2017. arXiv: 1701.07538.

[64] Pushouts are certain higher inductive types that suffice to construct all the higher inductive types that we need, but we don't actually need them in this book.

xca:comp-loc-small-ess-small

Exercise 2.19.5. Show that the replacement principle implies that for any locally $\mathcal{U}$-small type A, and any element $a : A$, the connected component $A_{(a)}$ is essentially $\mathcal{U}$-small. ⌟

Another consequence is that the type of finite sets, which we'll define below in Definition 2.24.5, is essentially small.

2.20 *Predicates and subtypes*

sec:subtype

In this section, we give two (equivalent) definitions of the notion of a subtype of a given type T. The first definition is based on the notion of a predicate on T. A predicate tells, or 'predicates', whether an element of T belongs to the subtype. The second definition is based on the notion of injection, defined in Definition 2.17.2.

def:predicate

Definition 2.20.1. Let T be a type and let $P(t) : \mathrm{Prop}$[65] be a family of propositions parametrized by a variable $t : T$. Then we call P a *predicate* on T.[66] If $P(t)$ is a decidable proposition for any $t : T$, then we say that P is a *decidable predicate* on T. ⌟

[65] Recall that Prop abbreviates $\mathrm{Prop}_{\mathcal{U}} \equiv \sum_{T:\mathcal{U}} \mathrm{isProp}(T)$.

[66] Note that giving a predicate on T is equivalent to giving a map $Q : T \to \mathrm{Prop}_{\mathcal{U}}$ for a suitable universe $\mathcal{U}$, and we often say that Q itself is the predicate. We leave $\mathcal{U}$ implicit.

Given a type T and a function $f : T \to \mathrm{Bool}$, Lemma 2.15.3 yields that $f(t) = \mathrm{yes}$ is a proposition, and we can form the predicate $P(t) :\equiv (f(t) = \mathrm{yes})$. Then $P : T \to \mathrm{Prop}$ is a decidable predicate by Exercise 2.20.2. However, not every predicate can be given through a $f : T \to \mathrm{Bool}$, since Prop and Bool are only equivalent if LEM holds (Exercise 2.18.3).

In the special case that $P : T \to \mathcal{U}$ is a decidable predicate we can define $\chi_P : T \to \mathrm{Bool}$ by induction (actually, only case distinction) on $d(t) : P(t) + \neg P(t)$, setting $\chi_P(t) = \mathrm{yes}$ if $d(t) \equiv \mathrm{inl}_$ and $\chi_P(t) = \mathrm{no}$ if $d(t) \equiv \mathrm{inr}_$. In this way, decidable predicates on a type T correspond to their characteristic functions $T \to \mathrm{Bool}$.

xca:decidability

Exercise 2.20.2. Show that $f(t) = \mathrm{yes}$ is a decidable predicate on T, for any type T and function $f : T \to \mathrm{Bool}$. Show that $(P \xrightarrow{\simeq} \mathrm{True}) + (P \xrightarrow{\simeq} \mathrm{False})$ holds for every decidable proposition P. ⌟

def:subtype

Definition 2.20.3. Let T be a type. The type of *subtypes* of T, denoted by $\mathrm{Sub}(T)$, is defined by

$$\mathrm{Sub}(T) :\equiv (T \to \mathrm{Prop}).$$

Given a predicate P on T, we define $T_P :\equiv \sum_{t:T} P(t)$ to be the *underlying type* of the subtype of T characterized by P. ⌟

The following lemma states that identity types in a subtype[67] are equivalent to those in the type itself.

[67] The phrase 'subtype' is often used for 'underlying type of the subtype'. See Footnote 68 for when it is important to be precise.

lem:subtype-eq-=

Lemma 2.20.4. *Let T be a type and $P : T \to \mathrm{Prop}$ a predicate on T. Recall the underlying type $T_P \equiv \sum_{t:T} P(t)$, and consider the projection map* fst *from T_P to T. Then $\mathrm{ap}_{\mathrm{fst}} : ((x_1, p_1) \xrightarrow{=} (x_2, p_2)) \to (x_1 \xrightarrow{=} x_2)$ is an equivalence, for any elements (x_1, p_1) and (x_2, p_2) of T_P.*

Proof. Corollary 2.9.11 gives that $\mathrm{fst}^{-1}(t) \simeq P(t)$ for all $t : T$, so that fst is an injection. Now apply Lemma 2.17.8. □

rem:subtype-convention

Remark 2.20.5. A very convenient consequence of Lemma 2.20.4 is that we can afford not to distinguish carefully between elements (t, p) of the subtype T_P and elements t of type T for which the proposition $P(t)$ holds. We will hence often silently coerce from T_P to T via the first projection,

and if $t : T$ is such that $P(t)$ holds, we'll write $t : T_P$ to mean any pair (t, p) where $p : P(t)$, since when $P(t)$ holds, the type $P(t)$ is contractible. ⌟

def:Prop-Set

Example 2.20.6. The type of types that are propositions and the type of types that are sets are defined as:

$$\mathrm{Prop}_{\mathcal{U}} :\equiv \sum_{X:\mathcal{U}} \mathrm{isProp}(X) \quad \text{and} \quad \mathrm{Set}_{\mathcal{U}} :\equiv \sum_{X:\mathcal{U}} \mathrm{isSet}(X).$$

Both $\mathrm{Prop}_{\mathcal{U}}$ and $\mathrm{Set}_{\mathcal{U}}$ are subtypes of $\mathcal{U}$, and both are types in a universe one higher than $\mathcal{U}$. We just write Prop and Set when we don't care about the precise universe $\mathcal{U}$.

Following the convention in Remark 2.20.5, when we have a type A for which we know that it is a proposition (or a set), we simply write $A : \mathrm{Prop}$ (or $A : \mathrm{Set}$). ⌟

lem:Prop-in-Set

Lemma 2.20.7. *The proposition* isSet(Prop) *holds, that is,* Prop *is a set.*

Proof. We show that $P \xrightarrow{=} Q$ is a proposition for all propositions P and Q. By univalence, $P \xrightarrow{=} Q$ is equivalent to $(P \xrightarrow{\simeq} Q) \equiv \sum_{f:P\to Q} \mathrm{isEquiv}(f)$. The latter is a proposition by Lemma 2.15.4(2)(5), using that $\mathrm{isEquiv}(f)$ is a proposition. □

ft:subtypes-subsets

Since Prop is a set, Sub(T) is also a set, for any type T.[68]

Exercise 2.20.8. Let T and X be types, $f : X \to T$ a function, and $P : T \to \mathrm{Prop}$ a predicate. Show that $\prod_{x:X} P(f(x))$ holds if and only if the following type[69] is contractible:

xca:subtype-univ-prop

$$\sum_{g : X \to \sum_{t:T} P(t)} f \xrightarrow{=} \mathrm{fst} \circ g.$$

ft:subtype-univ-prop ⌟

We call the result in Exercise 2.20.8 the *universal property of subtypes*.[70]

A pair like (T_P, fst) in Lemma 2.20.4 is actually an example of the second approach to subtypes, which we will explain now.

def:injtype

Definition 2.20.9. A *injection into a type* T is a type S together with an injection $f : S \to T$. The type S is called the *underlying type* of the injection into T.[71] Selecting a universe $\mathcal{U}$ as a repository for such types S allows us to introduce the type of injections into T in $\mathcal{U}$ as follows.

$$\mathrm{Inj}^{\mathcal{U}}(T) :\equiv \sum_{S:\mathcal{U}} \sum_{f:S\to T} \mathrm{isInj}(f).$$

When no confusion can arise, we simply write $\mathrm{Inj}(T)$ for $\mathrm{Inj}^{\mathcal{U}}(T)$. ⌟

lem:Sub(T)=Inj(T)

Lemma 2.20.10. *The function mapping any subtype* P *of* T *to the injection* $\mathrm{fst} : T_P \to T$ *defines an equivalence from* Sub(T) *to* Inj(T)*, for any type* T*. The inverse equivalence maps any injection* $i : S \to T$ *to the subtype* $(t \mapsto i^{-1}(t))$ *of* T*.*

Proof. We postpone the proof till Construction 2.25.6 (2), where this and similar results are obtained by a general method. If you just can't wait, do Exercise 2.20.11. □

As a consequence, $\mathrm{Inj}(T)$ is a set since Sub(T) is, for any type T.

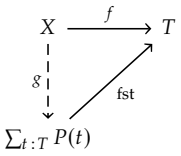
xca:Sub(T)=Inj(T)

Exercise 2.20.11. Prove Lemma 2.20.10. Hints: For the round trip starting with P, use function extensionality and Corollary 2.9.11. For the round trip starting with (S, i), show that the function g in Footnote 69 is an equivalence in case $X \equiv S$, $f \equiv i$ is an injection, and $P \equiv i^{-1}(_)$. ⌟

ft:incl-vs-inj

[68] Caution: When identifying 'subtypes', it should be clear whether they are considered as elements of Sub(T) or as underlying types of subtypes, i.e., as elements of some universe $\mathcal{U}$. The identity types $P =_{\mathrm{Sub}(T)} Q$ and $T_P \xrightarrow{=}_{\mathcal{U}} T_Q$ are in general not equivalent!

[69]



[70] In set theory, if $S \subseteq T$ and $f : X \to T$ is such that $f(x)$ is in S for all x in X, then $x \mapsto f(x)$ is the unique function $g : X \to S$ such that $f = i \circ g$, where i is the inclusion map of S in T.

[71] Instead of using this tedious phrase, we will simply call S a 'subtype' of T, if the injection is clear from the context. The cautioning Footnote 68 applies here as well.

[72] The full notation as an element of Inj(T) would be $(\{\, t : T \mid P(t) \,\}, \mathrm{fst}, p)$, with p witnessing that fst is an injection. In traditional set theory one would call fst the inclusion of the subset, which is unique for each subset. In contrast, there can be *many* pairs (X, i), with $i : X \to T$ an injection, defining the same subset of T. If in set theory one would define subsets through such pairs, one would have to solve a size issue and define an equivalence relation such that equivalent pairs define the same subset. In type theory, however, we have universes, and the identity type of $\mathrm{Inj}^{\mathcal{U}}(T)$ identifies precisely the triples defining the same subset.

Such considerations also apply to subtypes, and later to subgroups in Definition 5.3.13.

Lemma 2.20.4 has other important consequences:

cor:subtype-same-level

Corollary 2.20.12. *For any $n \geq -1$, if T is a n-type, then T_P is also a n-type.*

In particular, if T is a set, then T_P is again a set; we then call T_P a *subset* of T and we may denote it by $\{\, t : T \mid P(t) \,\}$.[72]

xca:subsets-inclusion

Exercise 2.20.13. Let T be a set. Define the relation $\subseteq : (\mathrm{Sub}(T) \times \mathrm{Sub}(T)) \to \mathrm{Prop}$ by $(P_0 \subseteq P_1) :\equiv \prod_{t:T}(P_0(t) \to P_1(t))$.[73] Prove that the relation $\subseteq$ is a partial order[74] with a least and a greatest element (even if T is the empty type). ⌟

def:decidable-set

Definition 2.20.14. A type A is called a *decidable set* if the identity type $x \xrightarrow{=} y$ is a decidable proposition for all $x, y : A$. ⌟

Note the slight subtlety of this definition together with Definition 2.18.1: Any proposition has decidable identity types (since each instance is contractible) and is thus a *decidable set*, even though it may not be *decidable as a proposition*.

The way we phrased this definition implies that A is a set. The following celebrated and useful theorem states that this is unnecessary.

thm:hedberg

Theorem 2.20.15 (Hedberg). *Any type A for which we have a function of type $\prod_{x,y:A}\big((x \xrightarrow{=} y) \amalg \neg(x \xrightarrow{=} y)\big)$ is a decidable set.*

For a proof see Theorem 7.2.5 of the HoTT Book[75].

[73] Recall that we can move without notice between $(\mathrm{Sub}(T) \times \mathrm{Sub}(T)) \to \mathrm{Prop}$ and $\mathrm{Sub}(T) \to \mathrm{Sub}(T) \to \mathrm{Prop}$.

[74] Recall that an *partial order* on a set S is a relation R that is (1) *reflexive*: $R(x, x)$, (2) *transitive*: $R(x, y) \to R(y, z) \to R(x, z)$, and (3) *antisymmetric*: $R(x, y) \to R(y, x) \to x = y$. See also Example 6.2.9.

[75] Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*.

2.21 *Pointed types*

sec:pointedtypes

Sometimes we need to equip types with additional structure that cannot be expressed by a proposition such as $\mathrm{isProp}(X)$ and $\mathrm{isSet}(X)$ above. Therefore the following is *not* a subtype of $\mathcal{U}$.

def:pointedtypes

Definition 2.21.1. A *pointed type* is a pair (A, a) where A is a type and a is an element of A. The *type of pointed types* is

$$\mathcal{U}_* :\equiv \sum_{A:\mathcal{U}} A.$$

Given a type A we let A_+ be the pointed type you get by adding a default element: $A_+ :\equiv (A \amalg \mathrm{True}, \mathrm{inr}_{\mathrm{triv}})$. Given a pointed type $X \equiv (A, a)$, the *underlying type* is $X_\div :\equiv A$,[76] and the *base point* is $\mathrm{pt}_X :\equiv a$, so that $X \equiv (X_\div, \mathrm{pt}_X)$.

Let $X :\equiv (A, a)$ and $Y :\equiv (B, b)$ be pointed types. Define the map $\mathrm{ev}_a : (A \to B) \to B$ by $\mathrm{ev}_a(f) :\equiv f(a)$. Then the fiber of ev_a at b is the type $\mathrm{ev}_a^{-1} \equiv \sum_{f:A\to B}(b \xrightarrow{=} f(a))$. The latter type is also called the type of *pointed functions* from X to Y and denoted by $X \to_* Y$. In the notation above,

$$(X \to_* Y) \equiv \sum_{f : X_\div \to Y_\div} (\mathrm{pt}_Y \xrightarrow{=} f(\mathrm{pt}_X)).$$

Given a pointed function $g \equiv (f, p)$, the *underlying function* is $g_\div :\equiv f$, and the *pointing path* is $g_{\mathrm{pt}} :\equiv p$, so that $g \equiv (g_\div, g_{\mathrm{pt}})$.

If Z is also a pointed type, and we have pointed functions $f : X \to_* Y$ and $g : Y \to_* Z$, then their composition $gf : X \to_* Z$ is defined as the pair $(g_\div f_\div, g_\div(f_{\mathrm{pt}})g_{\mathrm{pt}})$, as illustrated below.[77]

[76] The *obelus* $\div$ is sometimes used to denote division, but is also used to for subtraction, especially in Northern Europe. This inspired our use, considering its "adjoint" relationship to + detailed in Exercise 2.21.4.

[77] In particular, $(gf)_\div \equiv g_\div f_\div$.

$$\begin{array}{ccc} \mathrm{pt}_Y & \xrightarrow[=]{f_{\mathrm{pt}}} & f_\div(\mathrm{pt}_X) \\ \big\downarrow{\scriptstyle g_\div} & & \big\downarrow{\scriptstyle g_\div} \\ \mathrm{pt}_Z & \xrightarrow[=]{g_{\mathrm{pt}}} g_\div(\mathrm{pt}_Y) & \xrightarrow[=]{g_\div(f_{\mathrm{pt}})} g_\div(f_\div(\mathrm{pt}_X)) \end{array}$$

We may also use the notation $g \circ f$ for the composition. ⌟

def:pointedidentity

Definition 2.21.2. If $X \equiv (A, a)$ is a pointed type, then we define the *pointed identity map* $\mathrm{id}_X : X \to_* X$ by setting $\mathrm{id}_X :\equiv (\mathrm{id}_A, \mathrm{refl}_a)$. ⌟

rem:coerceawaypt

Remark 2.21.3. If X is a pointed type, then $X_\div$ is a type, but X itself is *not* a type. It is therefore unambiguous, and quite convenient, to write $x : X$ for $x : X_\div$, and $X \to \mathcal{U}$ for $X_\div \to \mathcal{U}$. Likewise, we can write $f : X \to Y$ for $f_\div : X_\div \to Y_\div$. In that case we still write $f_{\mathrm{pt}} : \mathrm{pt}_X \xrightarrow{=} f(\mathrm{pt}_Y)$ for the witness of pointedness. ⌟

xca:plusforgetadjoint

Exercise 2.21.4. If A is a type and B is a pointed type, give an equivalence from $A \to B_\div$ to $A_+ \to_* B$. ⌟

xca:freemaps

Exercise 2.21.5. Let A be a pointed type and B a type. Give an equivalence from $\sum_{b:B}(A \to_* (B, b))$ to $(A_\div \to B)$. ⌟

Since $\mathcal{U}_*$ and $X \to_* Y$ are sum types, the results on identifying pairs in Section 2.10 apply to pointed types and pointed maps as well.

def:pointedequiv

Definition 2.21.6. Given pointed types X and Y, we define the type of pointed equivalences from X to Y as:

$$X \xrightarrow{\simeq}_* Y :\equiv \sum_{f : X \to_* Y} \mathrm{isEquiv}(f_\div)$$ ⌟

xca:pointedequiv

Exercise 2.21.7. From an identification of pointed types $p : X \xrightarrow{=} Y$, construct an identification of the underlying types, $p_\div : X_\div \xrightarrow{=} Y_\div$, as well as an identification $q : \mathrm{pt}_Y \xrightarrow{=} \tilde{p}_\div(\mathrm{pt}_X)$. Together, this gives a map of type

$$(X \xrightarrow{=} Y) \to (X \xrightarrow{\simeq}_* Y).$$

Show that this map is an equivalence. ⌟

The following result gives a useful characterization of identity types of pointed maps, extending **??** 2.9.18.

con:identity-ptd-maps

Construction 2.21.8. *Let X and Y be pointed types and $f, g : X \to_* Y$ pointed maps from X to Y. Then we have an equivalence* ptw_* *of type*

$$(f \xrightarrow{=} g) \xrightarrow{\simeq} \sum_{h : \prod_{x:X}(f_\div(x) \xrightarrow{=} g_\div(x))} ((h(\mathrm{pt}_X) \cdot f_{\mathrm{pt}}) \xrightarrow{=} g_{\mathrm{pt}}).$$

Implementation of Construction 2.21.8. Define the type family T by $T(k) :\equiv (\mathrm{pt}_Y \xrightarrow{=} k(\mathrm{pt}_X))$ for any $k : X \to Y$. The equivalence ptw_* is the composite of the following chain of known equivalences:

$$\begin{aligned}
(f \xrightarrow{=} g) &\xrightarrow{\simeq} \sum_{e : (f_\div \xrightarrow{=} g_\div)} (f_{\mathrm{pt}} \xrightarrow[e]{=} g_{\mathrm{pt}}) \quad \text{by Lemma 2.10.3} \\
&\xrightarrow{\simeq} \sum_{e : (f_\div \xrightarrow{=} g_\div)} (\mathrm{trp}^T_e(f_{\mathrm{pt}}) \xrightarrow{=} g_{\mathrm{pt}}) \quad \text{by Definition 2.7.3} \\
&\xrightarrow{\simeq} \sum_{h : \prod_{x:X}(f_\div(x) \xrightarrow{=} g_\div(x))} (\mathrm{trp}^T_{\mathrm{ptw}^{-1}(h)}(f_{\mathrm{pt}}) \xrightarrow{=} g_{\mathrm{pt}}) \quad \text{by Exercise 2.9.12} \\
&\xrightarrow{\simeq} \sum_{h : \prod_{x:X}(f_\div(x) \xrightarrow{=} g_\div(x))} ((\mathrm{ptw}(\mathrm{ptw}^{-1}(h)))(\mathrm{pt}_X) \cdot f_{\mathrm{pt}}) \xrightarrow{=} g_{\mathrm{pt}}) \quad (*) \\
&\xrightarrow{\simeq} \sum_{h : \prod_{x:X}(f_\div(x) \xrightarrow{=} g_\div(x))} ((h(\mathrm{pt}_X) \cdot f_{\mathrm{pt}}) \xrightarrow{=} g_{\mathrm{pt}}) \quad (**).
\end{aligned}$$

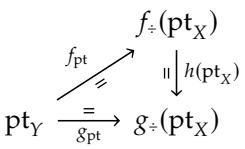


Figure 2.3: Transport in T.

Here (*) uses pointwise transport from Exercise 2.14.7,

$$\mathrm{trptw}(\mathrm{ptw}^{-1}(h), f_{\mathrm{pt}}) : \mathrm{trp}^{T}_{\mathrm{ptw}^{-1}(h)}(f_{\mathrm{pt}}) \xrightarrow{=} ((\mathrm{ptw}(\mathrm{ptw}^{-1}(h)))(\mathrm{pt}_X) \cdot f_{\mathrm{pt}}),$$

and (**) uses that ptw is an equivalence. □

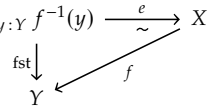
xca:identity-ptd-maps

xca:filler-ptd-sum-of-fibers

Exercise 2.21.9. Implement the function ptw_* by induction on $f \xrightarrow{=} g$. (You would then still have to prove that it is an equivalence.) ⌟

Exercise 2.21.10. New: Given pointed types X, Y, and a pointed map $f : X \to_* Y$, recall Lemma 2.25.2 for the equivalence $e : (\sum_{y:Y} f^{-1}(y)) \xrightarrow{\simeq} X$. With $\sum_{y:Y} f^{-1}(y)$ pointed at $(\mathrm{pt}_Y, (\mathrm{pt}_X, f_{\mathrm{pt}}))$ we point e by reflexivity. We also have $\mathrm{fst} : (\sum_{y:Y} f^{-1}(y)) \xrightarrow{\simeq} Y$ pointed by reflexivity. Identify fst and fe as pointed maps.[78] ⌟

[78] This identification is a filler of the following diagram of pointed maps:



xca:ptd-equiv-inv

Exercise 2.21.11. New: Given pointed types X, Y, and a pointed equivalence $f : X \xrightarrow{\simeq}_* Y$, we define $f^{-1} : Y \xrightarrow{\simeq}_* X$ as the (unpointed) inverse equivalence of f, pointed by the data included in f, i.e.,

$$(f^{-1})_{\mathrm{pt}} :\equiv \left(\mathrm{ptw}(h)(\mathrm{pt}_X) \cdot f^{-1}(f_{\mathrm{pt}})\right)^{-1} : \left(\mathrm{pt}_X \xrightarrow{=} f^{-1}(\mathrm{pt}_Y)\right),$$

where $h : f^{-1}f \xrightarrow{=} \mathrm{id}_X$. Give identifications of $f^{-1}f$ with $(\mathrm{id}_X, \mathrm{refl}_{\mathrm{pt}_X})$ and of ff^{-1} with $(\mathrm{id}_Y, \mathrm{refl}_{\mathrm{pt}_Y})$. Hint: See Theorem 4.2.3 of the HoTT Book.[79] ⌟

[79] Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*.

2.22 *Operations that produce sets*

sec:operations-on-sets

lem:Set-is-groupoid

The following lemma holds for n-types in general, but we only need it for propostions and sets.

Lemma 2.22.1. *Let X and Y be types.*

(1) *If X and Y are propositions, then so are $X \xrightarrow{\simeq} Y$ and $X \xrightarrow{=} Y$. In other words,* Prop *is a set.*

(2) *If X and Y are sets, then so are $X \xrightarrow{\simeq} Y$ and $X \xrightarrow{=} Y$. In other words,* Set *is a groupoid.*

Proof. By univalence, $X \xrightarrow{=} Y$ and $X \xrightarrow{\simeq} Y$ are equivalent, whereas the latter is equal by definition to $\sum_{f:X\to Y} \mathrm{isEquiv}(f)$. If X and Y are propositions (sets), then by Lemma 2.15.5 also $X \to Y$ is a proposition (set). Moreover, $\mathrm{isEquiv}(f)$ is a proposition by Lemma 2.15.7. Now the lemma follows by Corollary 2.20.12. □

One may wonder whether $\mathbb{N}$ as defined in Section 2.12 is a set. The answer is yes, but it is harder to prove than one would think. In fact we have the following theorem.

thm:isset-inductive-types

Theorem 2.22.2. *All inductive types in Section 2.12 are sets if all constituent types are sets.*[80]

Proof. We only do the case of lists X^*, for a set X, and leave the other cases to the reader (cf. Exercise 2.22.3). We have to give identifications of type $p \xrightarrow{=} q$ for all $\ell, \ell' : X^*$ and $p, q : \ell = \ell'$. By induction on q it suffices to give identifications of type $p \xrightarrow{=} \mathrm{refl}_\ell$ for all $p : \ell \xrightarrow{=} \ell$. Note that this cannot simply be done by induction on p. Instead we first give an inversion principle for identifications in X^* as follows. Define a type $T(\ell, \ell', p)$ for $\ell, \ell' : X^*$ and $p : \ell \xrightarrow{=} \ell'$ by induction on ℓ and ℓ':[82]

[80] Our proof follows the same idea due to Simon Huber that we used in the case of Bool in Lemma 2.15.3.

A variation can be used to give a complete characterization of the identity types of inductive types. See the HoTT book, e.g., Section 2.13 for details on the *encode–decode method*.[81]

For lists, this gives equivalences

$$(\varepsilon \xrightarrow{=} \varepsilon) \xrightarrow{\simeq} \mathrm{True}$$
$$(x\ell \xrightarrow{=} x'\ell') \xrightarrow{\simeq} (x \xrightarrow{=} x') \times (\ell \xrightarrow{=} \ell')$$
$$(\varepsilon \xrightarrow{=} x\ell) \xrightarrow{\simeq} (x\ell \xrightarrow{=} \varepsilon) \xrightarrow{\simeq} \mathrm{False}$$

from which we can deduce more generally that X^* is an n-type, when X is an n-type and $n \geq 0$. Corollary 2.24.9 below gives a different proof of this.

[81] Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*.

[82] Recall that the function apap from Definition 2.7.7 gives the action on paths for a function taking two arguments. Here it takes a path in X, $q : x = x'$, and a path in X^*, $r : \ell \xrightarrow{=} \ell'$, to a path between the concatenations, $\mathrm{apap}(q)(r) : x\ell \xrightarrow{=} x'\ell'$. On reflexivities it satisfies $\mathrm{apap}(\mathrm{refl}_x)(\mathrm{refl}_\ell) \equiv \mathrm{refl}_{x\ell}$.

$$T(\varepsilon, \varepsilon, p) :\equiv (p \xrightarrow{=} \mathrm{refl}_\varepsilon)$$
$$T(x\ell, x'\ell', p) :\equiv \sum_{q : x = x'} \sum_{r : \ell \xrightarrow{=} \ell'} (p \xrightarrow{=} \mathrm{apap}(q)(r))$$

For the other cases the choice is immaterial, say $T(\varepsilon, x\ell, p) :\equiv T(x\ell, \varepsilon, p) :\equiv \emptyset$. Next we give elements of type $T(\ell, \ell', p)$ for all ℓ, ℓ', and p by induction on p, reducing to $T(\ell, \ell, \mathrm{refl}_\ell)$ for all $\ell : X^*$, which we deal with by case distinction on the list ℓ. For ε we use $\mathrm{refl}_{\mathrm{refl}_\varepsilon}$, and for the case $x\ell$ we use the triple $(\mathrm{refl}_x, \mathrm{refl}_\ell, \mathrm{refl}_{\mathrm{refl}_{x\ell}})$, noting that $\mathrm{apap}(\mathrm{refl}_x)(\mathrm{refl}_\ell) \equiv \mathrm{refl}_{x\ell}$.

We can now give identifications of type $p \xrightarrow{=} \mathrm{refl}_\ell$ for all $p : \ell \xrightarrow{=} \ell$ by list induction on ℓ. For ε we use the element of $T(\varepsilon, \varepsilon, p)$ constructed above. For the case $x\ell$, the element of $T(x\ell, x\ell, p)$ constructed above yields a triple (q, r, s) with $q : x = x$, $r : \ell \xrightarrow{=} \ell$ and $s : p \xrightarrow{=} \mathrm{apap}(q)(r)$. Since X is a set, we have $q = \mathrm{refl}_x$ (as already indicated by the ordinary equals signs), and by induction hypothesis we have an identification $e : r \xrightarrow{=} \mathrm{refl}_\ell$. We get the desired identification by concatenating s and $\mathrm{apap}_{\mathrm{apap}}(!)(e)$:

$$p \xrightarrow{=} \mathrm{apap}(q)(r) \xrightarrow{=} \mathrm{apap}(\mathrm{refl}_x)(\mathrm{refl}_\ell) \equiv \mathrm{refl}_{x\ell}. \qquad \square$$

Exercise 2.22.3. Show that $X \amalg Y$ is a set if X and Y are sets. ⌟

xca:set-sum

Recall that propositional truncation is turning any type into a proposition by adding identifications of any two elements. Likewise, there is a operation turning any type into a set by adding (higher) identifications of any two identifications of any two elements. The latter operation is called set truncation. It is yet another example of a higher-inductive type.

def:set-truncation

Definition 2.22.4. Let T be a type. The *set truncation* of T is a type $\|T\|_0$ defined by the following constructors:

(1) an *element* $|t|_0 : \|T\|_0$ for all $t : T$;

(2) a *identification* $p \xrightarrow{=} q$ for all $x, y : \|T\|_0$ and $p, q : x \xrightarrow{=} y$.

The (unnamed) second constructor ensures that $\|T\|_0$ is a set. The induction principle states that, for any family of sets $S(x)$ defined for each $x : \|T\|_0$, in order to define a function $f : \prod_{x : \|T\|_0} S(x)$, it suffices to give a function $g : \prod_{t : T} S(|t|_0)$. Computationally, we get $f(|t|_0) \equiv g(t)$ for all $t : T$. ⌟

In the non-dependent case we get that for any set S and any function $g : T \to S$ there is a (unique) function $f : \|T\|_0 \to S$ satisfying $f(|t|_0) \equiv g(t)$ for all $t : T$.[83] A consequence of this recursion principle is that, for any set S, precomposition with $|_|_0$ is an equivalence

$$(\|T\|_0 \to S) \quad \to \quad (T \to S).$$

This is called *the universal property of set truncation*.[85]

xca:sum-of-conn-components

Exercise 2.22.5. Let A be a type. Define for every element $z : \|A\|_0$ the connected component corresponding to z, $A_{(z)}$, a subtype of A, such that for $a : A$, you recover the notion from Definition 2.16.8: $A_{(|a|_0)} \equiv A_{(a)}$.[86]

Prove that the set truncation map $|_|_0 : A \to \|A\|_0$ in this way exhibits A as the sum of its connected components, parametrized by $\|A\|_0$:

$$A \xrightarrow{\simeq} \sum_{z : \|A\|_0} A_{(z)}.$$

[83] Lemma 7.3.12[84] gives an equivalence from $|t|_0 = |t'|_0$ to $\|t \xrightarrow{=} t'\|$ for all $t, t' : T$.

[84] Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*.

[85] More generally, there are operations turning any type into an n-type, satisfying a similar universal property as propositional truncation and set truncation. We denote these operations by $\|_\|_n$ with corresponding constructor $|_|_n$. Propositional truncation $\|_\|$ can thus also be denoted as $\|_\|_{-1}$. Sometimes it is convenient to consider contractible types as -2-types, with constant truncation operator $\|T\|_{-2} :\equiv \mathrm{True}$ and constructor $|t|_{-2} :\equiv \mathrm{triv}$.

[86] *Hint*: Use maps $\|a \xrightarrow{=} _\| : A \to \mathrm{Prop}$ and the fact that the universe of propositions is a set.

Prove that A is connected iff $\|A\|_0$ is contractible. ⌟

Exercise 2.22.6. New: Let A be a type and $P : \|A\|_0 \to \mathrm{Prop}$ a subset of its set truncation. Give an equivalence from $\|\sum_{a:A} P(|a|_0)\|_0$ to $\sum_{b:\|A\|_0} P(b)$. ⌟

xca:subset-of-set-trunc

2.22.7 *Weakly constant maps*

The universal property of the propositional truncation, Definition 2.16.1, only applies directly to construct elements of *propositions* (that is, to prove them). Here we discuss how we can construct elements of *sets*.

Definition 2.22.8. A map $f : A \to B$ is *weakly constant* if $f(x) \mathrel{=\!\!\!\to} f(x')$ for all $x, x' : A$. ⌟

This is in contrast to a *constant* map, which can be identified with one of the form $x \mapsto b$ for some $b : B$. Any constant map is indeed weakly constant. Note also that when B is a set, weak constancy of $f : A \to B$ is a proposition.

Theorem 2.22.9. *If $f : A \to B$ is a weakly constant map, and B is a set, then there is an induced map $g : \|A\| \to B$ such that $g(|x|) \equiv f(x)$ for all $x : A$.*

thm:wconstant-elim

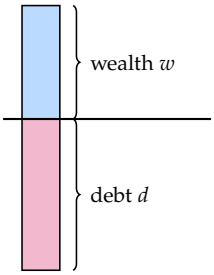
Proof. Consider the image factorization (Exercise 2.17.12) $A \xrightarrow{p} \mathrm{im}(f) \xrightarrow{i} B$ of f, where $p(x) :\equiv (f(x), |(x, \mathrm{refl}_{f(x)})|)$ and $i(y, _) :\equiv y$.

The key point is that $\mathrm{im}(f)$ is a proposition because f is weakly constant. First note that $\mathrm{im}(f)$ is a set by Exercise 2.17.13. Let $(y_1, z_1), (y_2, z_2) : \mathrm{im}(f)$. We have to prove $(y_1, z_1) = (y_2, z_2)$, which is a proposition. Hence we may hypothesize (by truncation induction on z_i) that we have $x_1, x_2 : A$ with $y_i = f(x_i)$ for $i = 1, 2$. Hence we get $y_1 = f(x_1) = f(x_2) = y_2$ and therefore $(y_1, z_1) = (y_2, z_2)$.

Thus, by the universal property of the truncation, we get $g' : \|A\| \to \mathrm{im}(f)$ such that $g'(|x|) \equiv p(x) \equiv (f(x), |(x, \mathrm{refl}_{f(x)})|)$. Composing with i we get $g :\equiv i \circ g' : \|A\| \to B$ with $g(|x|) :\equiv f(x)$, as desired. □

2.22.10 *Set quotients*

As an example, we first present an abstraction of the possible economical situations of a person as a quotient. Net worth can be defined as wealth minus debt. Let's assume wealth w and debt d are natural numbers. The debt can be greater than the wealth, yielding a negative net worth, but at this point in our book we do not have negative numbers at our disposal. However, we do have the binary product, and the pair $(w, d) : (\mathbb{N} \times \mathbb{N})$ also completely determines the net worth. However, (w, d) contains more information than necessary for the net worth: $(\mathrm{succ}(w), \mathrm{succ}(d))$, for example, determines the same net worth as (w, d), and $(\mathrm{succ}(w), \mathrm{succ}(d)) \neq (w, d)$. Put differently, the type $\mathbb{N} \times \mathbb{N}$ does not capture the notion of net worth, since its identity types don't capture equality of net worth.



Clearly, we need a different type to capture the notion of net worth. Of course, we want a type construction that works not only for the special case of net worth, but also in similar situations. Common to such situations is that we have a type A and an equivalence relation[87] $R : A \to A \to \mathrm{Prop}$. In the example of net worth, we have $A :\equiv (\mathbb{N} \times \mathbb{N})$, and the equivalence relation is $R((w_1, d_1), (w_2, d_2)) :\equiv (w_1 + d_2 = w_2 + d_1)$,

[87] Recall that an *equivalence relation* is one that is (1) *reflexive*: $R(x, x)$, (2) *symmetric*: $R(x, y) \to R(y, x)$, and (3) *transitive*: $R(x, y) \to R(y, z) \to R(x, z)$.

precisely capturing equality of net worth, $w_1 - d_1 = w_2 - d_2$, without actually using subtraction and negative numbers.

What we need is a new type, which is like A, but with R as equality. Note that the latter requires that the new type is a set. The quotient set A/R that we will define and study in this section fulfills these requirements. In the special case of $A :\equiv (\mathbb{N} \times \mathbb{N})$, and $R((w_1, d_1), (w_2, d_2)) :\equiv (w_1 + d_2 = w_2 + d_1)$, the type A/R could in fact be used as a type of integers, cf. Section 3.2 and see Exercise 2.22.15.

def:quotient-set

Definition 2.22.11. Given a type A and an equivalence relation $R : A \to A \to \mathrm{Prop}$, we define the *quotient set*[88] A/R as the image of the map $R : A \to (A \to \mathrm{Prop})$. Indeed, A/R is a set, since Prop is a set, and so are $A \to \mathrm{Prop}$ and the image $\sum_{P:A\to\mathrm{Prop}} \exists_{a:A}(P = R(a))$ of R. For $a : A$ we define $[a] :\equiv (R(a), |(a, \mathrm{refl}_{R(a)})|)$ in A/R; $[a]$ is called the *equivalence predicate of* a.[89] ⌟

Any element of the image of R is merely an equivalence predicate: a predicate P on A for which there exists $a : A$ such that $P(x)$ holds if and only if $R(a, x)$ holds.

In the following proofs we frequently use Exercise 2.17.14.

lem:equiv-class-prop

Lemma 2.22.12. *For any equivalence predicate $P : A/R$ and $a : A$, P and $[a]$ are equal if and only if $P(a)$ holds.*

Proof. Assume P and $[a]$ are equal. Then $P(x)$ iff $R(a, x)$ for all $x : A$. Now take $x :\equiv a$ and use reflexivity $R(a, a)$ to conclude $P(a)$.

Conversely, assume $P(a)$, and let $x : A$ be given. To prove the proposition $P(x) = R(a, x)$ we may assume that $P \equiv [b]$ for some $b : A$. Then $P(x) \equiv R(b, x)$, and we need to show $R(b, x) = R(a, x)$. This follows from $P(a) \equiv R(b, a)$ using symmetry and transitivity. □

The following theorem gives two important properties of the set quotient, the second is commonly called the universal property.

thm:quotient-property

Theorem 2.22.13. *We have $[x] = [x']$ if and only if $R(x, x')$ for all $x, x' : A$. Also, let B be a* set *and $f : A \to B$ a function such that $f(x) = f(x')$ for all $x, x' : A$ such that $R(x, x')$. Then the type $\sum_{g:A/R\to B}(f = g \circ [_])$ is contractible.*[90]

We will construct the center of contraction $\bar{f} : A/R \to B$ such that $\bar{f}([x]) \equiv f(x)$ for all $x : A$.

Proof. For the first part we use Lemma 2.22.12 applied to $P_x :\equiv [x]$ and x'.

Now let B be a set and let $f : A \to B$ a function satisfying $f(x) = f(x')$ for all $x, x' : A$ such that $R(x, x')$. We first define the center of contraction $\bar{f} : A/R \to B$. Let $z \equiv (P, p) : A/R$. To define $\bar{f}(z)$ in B, we note that $f \circ \mathrm{fst}$ is a weakly constant map of type $\sum_{x:A}(P = [x]) \to B$. By Theorem 2.22.9 we get a map $g : \exists_{x:A}(P = [x]) \to B$ and we put $\bar{f}(z) :\equiv g(p)$.

We check the equality by definition: As an element of A/R, equivalence predicate $[x]$ is accompanied by the witness $p \equiv |(x, \mathrm{refl}_{[x]})| : \exists_{y:A}([x] = [y])$. By Theorem 2.22.9, this is mapped by g, by definition, to $(f \circ \mathrm{fst})(x, \mathrm{refl}_{[x]}) \equiv f(x)$, as desired.

Now, if g, h satisfy $g \circ [_] = f = h \circ [_]$, then for any $z : A/R$, the type $g(z) = h(z)$ is a proposition since B is a set, so we may assume $z \equiv [x]$ for some $x : A$. Then $g([x]) = f(x) = h([x])$, as desired. □

[88] We may wonder about the universe level of A/R, assuming $A : \mathcal{U}$ and $R : A \to A \to \mathrm{Prop}_{\mathcal{U}}$. By the Replacement **??** 2.19.4, A/R is essentially $\mathcal{U}$-small, since $A \to \mathrm{Prop}_{\mathcal{U}}$ is locally $\mathcal{U}$-small. Alternatively, we could use Propositional Resizing **??** 2.18.6 to push the values of R into a lower universe.

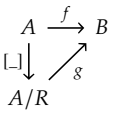
[89] In set theory, A would be a set and the equivalence relation R would be a subset of $A \times A$, satisfying the conditions in Footnote 87. Equivalence *classes* would be subsets of A.

Our definition may look different, but is actually a natural generalization of the definition in set theory to type theory. First, we let A be an arbitrary type. Note that $R \mapsto (z \mapsto R(\mathrm{fst}(z))(\mathrm{snd}(z))$ is an equivalence from $A \to (A \to \mathrm{Prop})$ to $(A \times A) \to \mathrm{Prop}$). So, indeed the equivalence relation R corresponds to a subtype of $A \times A$.

Note further that $\mathrm{fst}([a]) \equiv R(a)$ and that $\mathrm{snd}([a])$ certifies the (obvious) fact that $R(a)$ is in the image of R. For each $a : A$, the predicate $R(a) : A \to \mathrm{Prop}$ is a subtype of A. Therefore we call $[a]$ the *equivalence predicate* (instead of *class*) of a, which is true *for* a since $R(a)(a)$, that is, by reflexivity.

We will use $[a]$ and $R(a)$ interchangeably.

[90] In a diagram:



xca:A/True-prop-trunc

xca:ints-as-quotient

Exercise 2.22.14. Give an equivalence $A/R \to \|A\|$ when $R(x,y) :\equiv \mathrm{True}$ for all $x, y : A$.[91] ⌟

Exercise 2.22.15. Let $A :\equiv (\mathbb{N} \times \mathbb{N})$ and $R : A \to A \to \mathrm{Prop}$ defined by $R((w_1,d_1),(w_2,d_2)) :\equiv (w_1 + d_2 = w_2 + d_1)$. Let $Z :\equiv \{(w,d) \mid (d = 0) \vee (w = 0 \wedge d \neq 0)\}$. Construct an equivalence $f : A/R \to Z$ such that for all $(w,d,p) : Z$ we have $f([(w,d)]) = (w,d)$. ⌟

It is also possible to postulate[92] the quotient set as a higher inductive type.

def:quotient-as-HIT

Definition 2.22.16. Let A be a type and $R : A \to A \to \mathrm{Prop}$ an equivalence relation. Define the quotient A/R to be type with the following constructors:

(1) a constructor s of type $\mathrm{isSet}(A/R)$ ensuring that A/R is a set;

(2) an element constructor $[x] : A/R$ for all $x : A$;

(3) a constructor providing a proof $r(x,y,p)$ of $[x] = [y]$ for all $x, y : A$ and $p : R(x,y)$.

Let $B(z)$ be a set for every element $z : A/R$. The induction principle for A/R states that, in order to define an element of $B(z)$ for every $z : A/R$, it suffices to give elements $b_x : B([x])$ for every $x : A$ together with a proof of the proposition $b_x \xrightarrow[r(x,y,p)]{=} b_y$ for all $x, y : A$ and $p : R(x,y)$. The function f thus defined satisfies $f([x]) \equiv b_x$ for all $x : A$. ⌟

xca:quotients-equivalence

Exercise 2.22.17. Give an equivalence between A/R as defined in Definition 2.22.11 and A/R as defined in Definition 2.22.16. ⌟

rem:set-trunc-as-quotient

Remark 2.22.18. We can use set quotients to give an alternative definition of the set truncation $\|A\|_0$ of a type A. Consider the relation $R : A \to A \to \mathrm{Prop}$ given by $R(x,y) :\equiv \|x \xrightarrow{=} y\|$. This is easily seen to be an equivalence relation, using refl, symm and trans from Section 2.5. Hence we get a quotient set A/R that satisfies $(|x|_0 = |y|_0) \simeq \|x \xrightarrow{=} y\|$, for all elements x and y of A, where we write $|_|_0$ for the equivalence predicates. Furthermore, Theorem 2.22.13 implies that A/R satisfies the recursion principle of Definition 2.22.4: If S is a set, and $g : A \to S$ is any function, then $g(x) = g(y)$ holds whenever $\|x \xrightarrow{=} y\|$ by the induction principle of the propositional truncation, and hence we get a function $f : A/R \to S$ satisfying $f(|x|_0) \equiv g(x)$ for all $x : A$, as desired.[94] ⌟

xca:map-induces-quotient

Exercise 2.22.19. Let A and B be types and $f : A \to B$ a function. Consider the *equivalence relation on A induced by f* given by[95]

$$=_f :\equiv (a \mapsto (a' \mapsto \|f(a) \xrightarrow{=} f(a')\|)) : A \to (A \to \mathrm{Prop}).$$

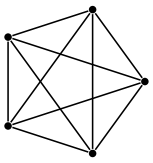
Let the quotient set $A/{=_f}$ be defined as in Definition 2.22.11. Show that the map $(|_|_0 \circ p) : A \to \|\mathrm{im}(f)\|_0$, with p from Exercise 2.17.12, induces a unique equivalence $e_f : A/{=_f} \to \|\mathrm{im}(f)\|_0$ with $|_|_0 \circ p = e_f \circ [_]$. ⌟

2.23 *More on natural numbers*

sec:more-on-N

A useful function $\mathbb{N} \to \mathbb{N}$ is the *predecessor* pred defined by $\mathrm{pred}(0) :\equiv 0$ and $\mathrm{pred}(\mathrm{succ}(n)) :\equiv n$. Elementary properties of addition, multiplication and predecessor can be proved in type theory in the usual way. We freely use them, sometimes even in definitions, leaving most of the proofs/constructions to the reader.

[91] If A is a finite set, we can picture this relation as a complete symmetric graph, i.e., with an edge between every pair of nodes, like this:



Convince yourself that a general equivalence relation on a finite set looks like a union of such complete graphs.

[92] *The method of 'postulating' what we want has many advantages; they are the same as the advantages of theft over honest toil.* Russell[93]

[93] Bertrand Russell. *Introduction to mathematical philosophy*. 2nd Ed. Dover Publications, Inc., New York, 1993, pp. viii+208.

[94] Expanding the definitions, this means that we can take the 0-truncation $\|A\|_0$ of $A : \mathcal{U}$ to be the $\mathcal{U}$-small image of the (-1)-truncated identity relation $A \to (A \to \mathrm{Prop}_{\mathcal{U}})$. Similarly, we can recursively construct the $(n+1)$-truncation by taking the $\mathcal{U}$-small image of the n-truncated identity relation $A \to (A \to \sum_{X:\mathcal{U}} \mathrm{is}n\mathrm{Type})$.

[95] If f is injective, then ap_f is an equivalence by Lemma 2.17.8, so that $A/{=_f}$ is essentially given by Remark 2.22.18.

def:orderonN

DEFINITION 2.23.1. Let $n, m : \mathbb{N}$. We say that m is less than or equal to n, and write $m \leq n$, if there is a $k : \mathbb{N}$ such that $k + m = n$. Such a k is unique, and if it is not 0, we say that m is less than n, denoted by $m < n$. Both $m \leq n$ and $m < n$ are propositions for all $n, m : \mathbb{N}$. ⌟

xca:try-your-luck-N

EXERCISE 2.23.2. Try your luck in type theory proving any of the following. The successor function satisfies $(\mathrm{succ}(n) = \mathrm{succ}(m)) \simeq (n = m)$. The functions $+$ and $\cdot$ are commutative and associative, $\cdot$ distributes over $+$. The relations $\leq$ and $<$ are transitive and preserved under $+$; $\leq$ also under $\cdot$. We have $(m \leq n) \simeq ((m < n) \amalg (m = n))$ (so $\leq$ is reflexive). Furthermore, $((m \leq n) \times (n \leq m)) \simeq (m = n)$, and $\neg((m < n) \times (n < m))$ (so $<$ is irreflexive). ⌟

We can prove the following lemma by double induction.

lem:dec-eq-order-N

LEMMA 2.23.3. *The relations $=$, $\leq$ and $<$ on $\mathbb{N}$ are decidable.*

By Hedberg's Theorem 2.20.15, we get an alternate proof that $\mathbb{N}$ is a set.

We will now prove an important property of $\mathbb{N}$, called the *least number principle for decidable, non-empty subsets of* $\mathbb{N}$. We give some more details of the proof, since they illustrate an aspect of type theory that has not been very prominent up to now, namely the close connection between proving and computing.

def:Wellordered

CONSTRUCTION 2.23.4. *Let $P(n)$ be a proposition for all natural numbers n. Define the type $P_{\min}(n)$ expressing that n is the smallest natural number such that $P(n)$:*

$$P_{\min}(n) :\equiv P(n) \times \prod_{m:\mathbb{N}} (P(m) \to n \leq m)$$

Then we seek a function

eqn:min

$$\min(P) : \prod_{n:\mathbb{N}} (P(n) \amalg \neg P(n)) \to \exists_{n:\mathbb{N}} P(n) \to \sum_{n:\mathbb{N}} P_{\min}(n), \tag{2.23.1}$$

computing a minimal witness for P from evidence that P is decidable and that a witness exists.

Implementation of Construction 2.23.4. First note that $P_{\min}(n)$ is a proposition, and that all n such that $P_{\min}(n)$ are equal. Therefore the type $\sum_{n:\mathbb{N}} P_{\min}(n)$ is also a proposition.

Given a function $d(n) : P(n) \amalg \neg P(n)$ deciding $P(n)$ for each $n : \mathbb{N}$, we define a function $\mu_P : \mathbb{N} \to \mathbb{N}$ which, given input n, searches for a $k < n$ such that $P(k)$. If such a k exists, μ_P returns the least such k, otherwise $\mu_P(n) = n$. This is a standard procedure that we will call *bounded search*. The function μ_P is defined by induction, setting $\mu_P(0) :\equiv 0$ and $\mu_P(\mathrm{succ}(n)) :\equiv \mu_P(n)$ if $\mu_P(n) < n$. Otherwise, we set $\mu_P(\mathrm{succ}(n)) :\equiv n$ if $P(n)$, and $\mu_P(\mathrm{succ}(n)) :\equiv \mathrm{succ}(n)$ otherwise, using $d(n)$ to decide, that is, by induction on $d(n) : P(n) \amalg \neg P(n)$. By design, μ_P 'remembers' where it has found the least k (if so). We are now done with the computational part and the rest is a correctness proof.

By induction on $n : \mathbb{N}$ and $d(n) : P(n) \amalg \neg P(n)$ we show

$$\mu_P(n) \leq n \quad \text{and} \quad \mu_P(n) < n \to P(\mu_P(n)).$$

The base case where $n :\equiv 0$ is easy. For the induction step, review the computation of $\mu_P(\mathrm{succ}(n))$. If $\mu_P(\mathrm{succ}(n)) = \mu_P(n)$ since $\mu_P(n) < n$,

then we are done by the induction hypothesis. Otherwise, either $\mu_P(\mathrm{succ}(n)) = n$ and $P(n)$, or $\mu_P(\mathrm{succ}(n)) = \mathrm{succ}(n)$. In both cases we are done.

Also by induction on $n : \mathbb{N}$ and $d(n) : P(n) \amalg \neg P(n)$ we show

$$P(m) \to \mu_P(n) \leq m, \text{ for all } m \text{ in } \mathbb{N}.$$

The base case $n :\equiv 0$ holds since $\mu_P(0) = 0$. For the induction step, assume $P(m) \to \mu_P(n) \leq m$ for all m (IH). Let $m : \mathbb{N}$ and assume $P(m)$. We have to prove $\mu_P(\mathrm{succ}(n)) \leq m$. If $\mu_P(\mathrm{succ}(n)) = \mu_P(n)$ we are done by IH. Otherwise we have $\mu_P(n) = n$ and $\mu_P(\mathrm{succ}(n)) = \mathrm{succ}(n)$ and $\neg P(n)$. Then $\mu_P(n) \leq m$ by IH, and $n \neq m$, so $\mu_P(\mathrm{succ}(n)) \leq m$.

By contraposition we get from the previous result

$$\mu_P(n) = n \to \neg P(m), \text{ for all } m < n.$$

Note that there may not be any n such that $P(n)$; the best we can do is to prove

$$P(n) \to P_{\min}(\mu_P(\mathrm{succ}(n)))$$

by combining previous results. Assume $P(n)$. Then $\mu_P(\mathrm{succ}(n)) \leq n < \mathrm{succ}(n)$, so that $P(\mu_P(\mathrm{succ}(n)))$. Moreover, $P(m) \to \mu_P(\mathrm{succ}(n)) \leq m$ for all m in $\mathbb{N}$. Hence $P_{\min}(\mu_P(\mathrm{succ}(n)))$.

Since $\sum_{n:\mathbb{N}} P_{\min}(n)$ is a proposition, we obtain the required function by the induction principle for propositional truncation, Definition 2.16.1:

$$\min(P) : \prod_{n:\mathbb{N}} (P(n) \amalg \neg P(n)) \to \left\| \sum_{n:\mathbb{N}} P(n) \right\| \to \sum_{n:\mathbb{N}} P_{\min}(n). \qquad \square$$

rem:computations-can-decide

Remark 2.23.5. In the interest of readability, we do not always make the use of witnesses of decidability in computations explicit. A typical example is the case distinction on $\mu_P(n) < n$ in Construction 2.23.4 above. This remark applies to all sets and decidable relations on them. We shall immediately put this convention to good use in the proof of a form of the so-called *Pigeonhole Principle* (PHP). ⌟

lem:PHP

Lemma 2.23.6. *For all $N : \mathbb{N}$ and $f : \mathbb{N} \to \mathbb{N}$ such that $f(n) < N$ for all $n < N + 1$, there exist $m < n < N + 1$ such that $f(n) = f(m)$.*

Proof. By induction on N. In the base case $N = 0$ there is nothing to do. For the induction case $N + 1$, assume the lemma proved for N (induction hypothesis, IH, for all f). Let f be such that $f(n) < N + 1$ for all $n < N + 2$. The idea of the proof is to search for an $n < N + 1$ such that $P(n) :\equiv (f(n) = N)$, by computing $\mu_P(N + 1)$ as in Construction 2.23.4. If $\mu_P(N + 1) = N + 1$, that is, $f(n) < N$ for all $n < N + 1$, then we are done by IH. Assume $\mu_P(N + 1) < N + 1$, so $f(\mu_P(N + 1)) = N$. If also $f(N + 1) = N$ then we are done. If $f(N + 1) < N$, then we define g by $g(n) = f(N + 1)$ if $f(n) = N$, and $g(n) = f(n)$ otherwise. Then IH applies to g, and we get $m < n < N + 1$ with $g(n) = g(m)$. If $f(n) = f(m)$ we are of course done. Otherwise, $f(n)$, $f(m)$ cannot both be smaller than N, as $g(n) = g(m)$. In both remaining cases, $f(n) = g(n) = g(m) = f(N + 1)$ and $f(N + 1) = g(n) = g(m) = f(m)$, we are done. $\square$

We can now rule out the existence of equivalences between finite sets of different size.

cor:Fin-n-injective

Corollary 2.23.7. *If $m < n$, then $(\sum_{k:\mathbb{N}} k < m) \neq (\sum_{k:\mathbb{N}} k < n)$.*

Another application of Construction 2.23.4 is a short proof of Euclidean division.

lem:euclid-div

Lemma 2.23.8. *For all $n, m : \mathbb{N}$ with $m > 0$ there exist unique $q, r : \mathbb{N}$ such that $r < m$ and $n = qm + r$.*

Proof. Define $P(k) :\equiv (n \leq km)$. Since $m > 0$ we have $P(n)$. Now set $k :\equiv \mu_P(n)$ as in Construction 2.23.4. If $n = km$ and we set $q :\equiv k$ and $r :\equiv 0$. If $n < km$, then $k > 0$ and we set $q :\equiv k - 1$. By minimality we have $qm < n < km$ and hence $n = qm + r$ for some $r < m$. □

2.24 *The type of finite sets*

sec:typeFin

Recall from Section 2.12.1 the types False, True and Bool containing zero, one and two elements, respectively. We now define generally the type of n elements for any $n : \mathbb{N}$.

def:finiteset

Definition 2.24.1. For any type X define $\mathrm{succ}(X) :\equiv X \amalg \mathrm{True}$. Define inductively the type family $\mathrm{Fin}(n)$, for each $n : \mathbb{N}$, by setting $\mathrm{Fin}(0) :\equiv \emptyset$ and $\mathrm{Fin}(\mathrm{succ}(n)) :\equiv \mathrm{succ}(\mathrm{Fin}(n))$. The type $\mathrm{Fin}(n)$ is called the type with n elements, and we denote its elements by $0, 1, \ldots, n-1$ rather than by the corresponding expressions using inl and inr.

We also define as abbreviation $\mathbb{n} :\equiv \mathrm{Fin}(n)$ for a natural number n, so $\mathbb{0} :\equiv \mathrm{Fin}(0)$, $\mathbb{1} :\equiv \mathrm{Fin}(1)$, $\mathbb{2} :\equiv \mathrm{Fin}(2)$, etc. ⌟

xca:finite-types

Exercise 2.24.2.

(1) Denote in full the elements of $\mathbb{0}$, $\mathbb{1}$, and $\mathbb{2}$.

(2) Construct an equivalence in $\mathbb{1} \xrightarrow{\simeq}$ True and one in $\mathbb{2} \xrightarrow{\simeq}$ Bool.

(3) Construct equivalences in $\mathbb{n} \xrightarrow{\simeq} \sum_{k:\mathbb{N}} k < n$ for all $n : \mathbb{N}$.

(4) Show that $m = n$ if we are given an element of type $\mathbb{m} \xrightarrow{=} \mathbb{n}$. ⌟

def:is-finite

Definition 2.24.3. Given a type X, we define the proposition

$$\mathrm{isFinSet}(X) :\equiv \exists_{n:\mathbb{N}} (X \xrightarrow{=} \mathbb{n})$$

to express that X *is a finite set*.[96] ⌟

lem:maxonefinitetype

Lemma 2.24.4. *For all types X we have*:

(1) $\sum_{n:\mathbb{N}} \|X \xrightarrow{=} \mathbb{n}\|$ *is a proposition*;[97]

(2) $\sum_{n:\mathbb{N}} \|X \xrightarrow{=} \mathbb{n}\|$ *if and only if* $\mathrm{isFinSet}(X)$.

Proof.

(1) Assume $(n, p), (m, q) : \sum_{n:\mathbb{N}} \|X \xrightarrow{=} \mathbb{n}\|$. Then $q \circ p^{-1} : \mathbb{n} \xrightarrow{=} \mathbb{m}$, so $n = m$ by Exercise 2.24.2. By Lemma 2.10.3, Definition 2.7.3 and the fact that the type of q is a proposition, it follows that $(n, p) = (m, q)$.

(2) Functions in both directions are easily defined by using the recursion principle of propositional truncation, see after Definition 2.16.1. □

def:groupoidFin

Definition 2.24.5. The *groupoid of finite sets* is defined by[98]

$$\mathrm{FinSet} :\equiv \sum_{S:\mathrm{Set}} \mathrm{isFinSet}(S).$$

[96] When moving beyond sets, there are two different ways in which a type can be finite: an *additive* way and a *multiplicative* way, but it would take us too far afield to define these notions here.

[97] In other words, an $n : \mathbb{N}$ such that $\|X \xrightarrow{=} \mathbb{n}\|$ is unique if it exists.

[98] Here it doesn't matter whether we take the sum over Set or over $\mathcal{U}$, since any finite set is a set. Hence we also have $\mathrm{FinSet}_n \equiv \mathrm{Set}_{(\mathbb{n})} \xrightarrow{=} \mathrm{FinSet}_{(\mathbb{n})} \xrightarrow{=} \mathcal{U}_{(\mathbb{n})}$.

For $n : \mathbb{N}$, the *groupoid of sets of cardinality n* is defined by

$$\mathrm{FinSet}_n :\equiv \sum_{S:\mathrm{Set}} \|\mathbb{m} \xrightarrow{\simeq} S\|.$$

Lemma 2.24.4 yields a function $\# : \mathrm{FinSet} \to \mathbb{N}$ such that $\#(S)$ is the cardinality of the finite set S.[99] ⌟

[99] $\#(S) = n$ is also phrased as: *S is in the same component in* Set *as* $\mathbb{m}$, or *S has cardinality n*, or *S is an n-element set*.

Observe that we have identifications in $\mathrm{FinSet}_0 \xrightarrow{\simeq} \mathrm{FinSet}_1 \xrightarrow{\simeq} \mathbb{1}$, and in FinSet $\xrightarrow{\simeq} \sum_{n:\mathbb{N}} \mathrm{FinSet}_n$ by Lemma 2.24.4. Also, FinSet is the image of the map $\mathrm{Fin} : \mathbb{N} \to \mathcal{U}$ from Definition 2.24.1, and is hence essentially $\mathcal{U}$-small (for any universe $\mathcal{U}$), by **??** 2.19.4, Item (P1) in Section 2.4, and our assumption that $\mathcal{U}_0$ is the smallest universe.

Exercise 2.24.6. Show that every finite set is a decidable set. ⌟

xca:finsets-decidable

We have already seen several examples of 2-element sets: Bool, $\mathbb{2}$, $\mathbb{1} \amalg \mathbb{1}$ that can easily be identified. Which one to use depends on the context and is a matter of convenience. Later we will also use $\{\pm 1\}$. In contrast to these concrete examples, one cannot identify[100] an *arbitrary* 2-element set with any of these. The following exercise makes this precise, and gives a useful and surprising case of a 2-element set that actually *can* be identified with $\mathbb{2}$.

[100] Any 2-element set is by definition *merely* identified with $\mathbb{2}$, but the problem is that we cannot "name" the elements, not even one of them. Having a name for one of the elements would be sufficient, since then the "other" element is uniquely determined.

Exercise 2.24.7. Show that $T \xrightarrow{\simeq} T$ is a 2-element set for every 2-element set T. Using univalence, show that $\neg \prod_{T:\mathrm{FinSet}_2}(T \xrightarrow{\simeq} \mathbb{2})$. In spite of the above, give an element of $\prod_{T:\mathrm{FinSet}_2}((T \xrightarrow{\simeq} T) \xrightarrow{\simeq} \mathbb{2})$.

Finally, give an element of $\prod_{T:\mathrm{FinSet}_2}(T \xrightarrow{\simeq} (\mathbb{2} \xrightarrow{\simeq} T))$. ⌟

xca:2-element-sets

Exercise 2.24.8. Recall the definition of lists of elements of a type X, X^*, from Definition 2.12.11. Construct an equivalence

$$\mathrm{lookup} : X^* \to \sum_{n:\mathbb{N}} (\mathrm{Fin}\, n \to X)$$

that sends a list $\ell \equiv x_1 x_2 \dots x_n$ to the pair (n, x) of its length n and the function x that maps an element i of $\mathrm{Fin}(n)$ to the element $x_i : X$. ⌟

xca:tabulation

Using this, we get a generalization of Theorem 2.22.2:

Corollary 2.24.9. *For any $n \geq 0$, if X is an n-type, then so is X^*.*[101]

[101] We need $n \geq 0$, since for a contractible (-2-type) X we get an equivalence $X^* \xrightarrow{\simeq} \mathbb{N}$ by Exercise 2.12.12, and $\mathbb{N}$ is not contractible.

Proof. Combine Lemma 2.15.5 with the fact that $\mathbb{N}$ is a set. □

cor:lists-truncated

Remark 2.24.10. A subset of a finite set is not necessarily finite itself: Let p be a proposition. Then p is also a set. If p is a finite set, then we have $\#(p) : \mathbb{N}$, and we can prove that p holds if and only if $\#(p) = 1$. Since equality in $\mathbb{N}$ is decidable, this would mean that we can decide p. Conversely we have that p is a finite set if p is decidable: If $p \amalg \neg p$, then $p = \mathbb{1}$ in case p and $p = \mathbb{0}$ in case $\neg p$.

It now follows from Exercise 2.24.12 below that every decidable predicate on a finite set S defines a finite subset of S. ⌟

rem:subset-of-fin-set

Exercise 2.24.11. Let X be a finite set and $P : X \to \mathrm{Prop}$ a decidable predicate. Show that $\exists_{x:X} P(x)$ and $\prod_{x:X} P(x)$ are decidable. Hint: since the goals are propositions, you may assume an identification of X with a standard n-element set. Use induction on n, being careful about the induction hypothesis. ⌟

xca:dec-quant-finset

Exercise 2.24.12. Let X be a finite set and $F : X \to \mathrm{FinSet}$ a family of finite sets. Show that the sum type $\sum_{x:X} F(x)$ is a finite set.

xca:fin-sum-of-finsets

Let Y be a finite set and assume we have an equivalence $e(x) : F(x) \xrightarrow{\simeq} Y$ for every $x : X$. Then show that $\#(\sum_{x:X} F(x)) = \#(X) \times \#(Y)$.

For any map $f : X \to \mathbb{N}$, define the arithmetical sum $(\sum_{x:X} f(x)) : \mathbb{N}$. ⌟

xca:dec-quot-finite-set

Exercise 2.24.13. Let X be a finite set and $R : X \to X \to \mathrm{Prop}$ a decidable equivalence relation. Show that the quotient X/R is a finite set. ⌟

2.25 *Type families and maps*

sec:typefam

There is a natural equivalence between maps into a type A and type families parametrized by A. The key idea is that the fibers of a map form a type family. We will elaborate this idea and some variations.

lem:fst-fiber(a)=B(a)

Lemma 2.25.1. *Let $A : \mathcal{U}$ and $B : A \to \mathcal{U}$. Recall the projection function* $\mathrm{fst} : (\sum_{a:A} B(a)) \to A$. *The function $e_a : B(a) \to \mathrm{fst}^{-1}(a)$ defined by $e_a(b) :\equiv ((a, b), \mathrm{refl}_a)$ is an equivalence, for all $a : A$.*

Proof. Note that $\mathrm{fst}(x, b) \equiv x$ and that $a \xrightarrow{=} x$ does not depend on b. Hence $\mathrm{fst}^{-1}(a) \xrightarrow{\simeq} \sum_{x:A}(B(x) \times (a \xrightarrow{=} x))$ via rearranging brackets. Applying Corollary 2.9.11 leads indeed to the equivalence e_a. □

lem:sum-of-fibers

Lemma 2.25.2. *Let $A, B : \mathcal{U}$ and $f : A \to B$. Then $e : \left(\sum_{b:B} f^{-1}(b)\right) \to A$ defined by $e(b, a, p) :\equiv a$ is an equivalence.*

Proof. The function e is the composite of three equivalences

$$\left(\sum_{b:B}\sum_{a:A}(b \xrightarrow{=} f(a))\right) \xrightarrow{\simeq} \left(\sum_{a:A}\sum_{b:B}(b \xrightarrow{=} f(a))\right) \xrightarrow{\simeq} \left(\sum_{a:A} \mathrm{True}\right) \xrightarrow{\simeq} A,$$

where the first one interchanges the first two arguments, the second one contracts away the inner sumtype (using Lemma 2.9.2), and the third one is fst (using Exercise 2.9.20). □

If f in Lemma 2.25.2 is an injection, then $(\sum_{b:B} f^{-1}(b), \mathrm{fst})$ corresponds to a subtype of B, and hence A is a n-type if B is a n-type by Corollary 2.20.12.

lem:typefamiliesandfibrations

Lemma 2.25.3. *Let $A : \mathcal{U}$ be a type.*[102] *Then*

$$\mathrm{preim} : \sum_{B:\mathcal{U}} (B \to A) \quad \to \quad (A \to \mathcal{U})$$

given by $\mathrm{preim}(B, f)(a) :\equiv f^{-1}(a)$ *is an equivalence. The inverse equivalence is given by sending $C : A \to \mathcal{U}$ to* $(\sum_{a:A} C(a), \mathrm{fst})$.

[102] Note that we need A to be in the same universe as the one we're taking type families in.

Proof. We apply Construction 2.9.9, and verify the two conditions. Let $C : A \to \mathcal{U}$. We have to identify C with $\mathrm{preim}(\sum_{a:A} C(a), \mathrm{fst})$. As $\mathrm{preim}(\sum_{a:A} C(a), \mathrm{fst})(a) \equiv \mathrm{fst}^{-1}(a)$, it suffices by function extensionality to identify the latter fiber with $C(a)$, for all $a : A$. This follows directly from Lemma 2.25.1 and the univalence axiom.

Let $f : B \to A$. We have to identify $(\sum_{a:A} f^{-1}(a), \mathrm{fst})$ with (B, f). Using the univalence axiom, we get an identification $\bar{e} : \sum_{a:A} f^{-1}(a) \xrightarrow{=} B$, where e is the equivalence from Lemma 2.25.2. Using Lemma 2.10.3, it remains to give an element of the type $\mathrm{fst} \underset{\bar{e}}{\xrightarrow{=}} f$.

As an auxiliary step we note that for any $p : X \xrightarrow{=} Y$ and $g : X \to A$, $h : Y \to A$, the type $g \underset{p}{\xrightarrow{=}} h$ of paths over p can be identified with the

type $g \overset{=}{\to} h \circ \tilde{p}$, since the two types are equal by definition for $p \equiv \mathrm{refl}_X$. Applying this here means that we must give an identification of fst with $f \circ \tilde{e}$. Hence it suffices to identify fst and $f \circ e$, which follows by function extensionality from the definition of e in Lemma 2.25.2. □

The above result can be generalized to situations with more properties and/or structure. Examples are to be found in Construction 2.25.6 below. We prepare by the following exercises.

xca:sum-base-path

Exercise 2.25.4. Let X and Y be types, $p : Y \overset{=}{\to} X$ an identification, and $T : X \to \mathcal{U}$ a type family. Construct an equivalence of type $\sum_{x:X} T(x) \overset{\simeq}{\to} \sum_{y:Y} T(\tilde{p}(y))$. ⌟

xca:sum-families

Exercise 2.25.5. Let $S : \mathcal{U} \to \mathcal{U}$ and let X be a type. Construct an equivalence of type $(X \to \sum_{Y:\mathcal{U}} S(Y)) \overset{\simeq}{\to} \sum_{F:X\to\mathcal{U}} \prod_{x:X} S(F(x))$. ⌟

lem:Prop-Set-pointed-families
cnstr:families+structure
lem:Prop-families
lem:Set-families
cnstr:families-points

Construction 2.25.6. *Let A be a type and $S : \mathcal{U} \to \mathcal{U}$. Then we have equivalences of the following types*:

(1) $(A \to \sum_{B:\mathcal{U}} S(B)) \overset{\simeq}{\to} \sum_{B:\mathcal{U}} \sum_{f:B\to A} \prod_{a:A} S(f^{-1}(a))$.

(2) $(A \to \mathrm{Prop}_{\mathcal{U}}) \overset{\simeq}{\to} \sum_{B:\mathcal{U}} \sum_{f:B\to A} \prod_{a:A} \mathrm{isProp}(f^{-1}(a))$;

(3) $(A \to \mathrm{Set}_{\mathcal{U}}) \overset{\simeq}{\to} \sum_{B:\mathcal{U}} \sum_{f:B\to A} \prod_{a:A} \mathrm{isSet}(f^{-1}(a))$;

(4) $(A \to \mathcal{U}_*) \overset{\simeq}{\to} \sum_{B:\mathcal{U}} \sum_{f:B\to A} \prod_{a:A} f^{-1}(a)$.

Implementation of Construction 2.25.6. (1) In view of Exercise 2.25.5, and rearranging sums on the right, it suffices to construct an equivalence of type $(\sum_{F:A\to\mathcal{U}} \prod_{a:A} S(F(a))) \overset{\simeq}{\to} \sum_{(B,f):\sum_{B:\mathcal{U}}(B\to A)} \prod_{a:A} S(f^{-1}(a))$. Now we can apply the equivalence constructed in Exercise 2.25.4 with p the path induced by the equivalence preim from Lemma 2.25.3. Indeed, for $T(F) :\equiv \prod_{a:A} S(F(a))$ we have $T(\mathrm{preim}(B, f)) \equiv \prod_{a:A} S(f^{-1}(a))$.

For (2), use (1) with $S :\equiv \mathrm{isProp}$.

For (3), use (1) with $S :\equiv \mathrm{isSet}$.

For (4), use (1) with $S :\equiv \mathrm{id}_{\mathcal{U}}$. □

Since Prop is a set, by Lemma 2.20.7, we obtain the following corollary of Construction 2.25.6(2).

cor:Inj(T)-is-set

Corollary 2.25.7. *Subtypes as in Definition 2.20.9 correspond to predicates by taking fibers, and* $\mathrm{Inj}(T)$ *is a set, for any type T.*

Exercise 2.25.8. For any pair of nested universes $\mathcal{U} : \mathcal{U}'$, let $S : \mathcal{U}' \to \mathcal{U}'$ be the predicate that determines the essentially $\mathcal{U}$-small $\mathcal{U}'$-types,

$$S(A) :\equiv \sum_{X:\mathcal{U}} A \overset{\simeq}{\to} X,$$

as in Definition 2.19.1. Show that projection to the $\mathcal{U}$-type defines an equivalence of type $\left(\sum_{A:\mathcal{U}'} S(A)\right) \overset{\simeq}{\to} \mathcal{U}$, and whence construct an equivalence of type

$$(A \to \mathcal{U}) \overset{\simeq}{\to} \sum_{B:\mathcal{U}'} \sum_{f:B\to A} \prod_{a:A} S(f^{-1}(a)),$$

between families of $\mathcal{U}$-small types parametrized by A and maps to A in $\mathcal{U}'$ with essentially $\mathcal{U}$-small fibers, for any $A : \mathcal{U}'$. ⌟

2.26 *Higher truncations*

sec:higher-truncations

We've seen the propositional truncation in Section 2.16 and the set truncation in Section 2.22. As mentioned in Remark 2.22.18, it's possible to define the latter in terms of the former by considering the propositional truncation of the identity types of a type A. In this section we want to generalize this to higher truncation levels and show how we can inductively define all the n-truncation operations using propositional truncation combined with the replacement principle, **??** 2.19.4, which is used to stay within a given universe.

def:join-construction-of-truncation

Construction 2.26.1. *For any integer $n \geq -1$ we have an* n-truncation operation $\|_\|_n : \mathcal{U} \to \mathcal{U}$, *along with* unit maps $|_|_n : A \to \|A\|_n$, *satisfying the following universal property.*

For any n-type B, precomposition with $|_|_n$ induces an equivalence:

$$\big(\|A\|_n \to B\big) \xrightarrow{\simeq} (A \to B).$$

Implementation of Construction 2.26.1. We proceed by induction. For $n \equiv -1$, we have this from the higher inductive type definition, Definition 2.16.1, with element constructor $|_| : A \to \|A\|$.

To go from n to $n+1$, we fix a type $A : \mathcal{U}$ and consider the n-truncated identity type family

$$I_n : A \to \left(A \to \sum_{X:\mathcal{U}} \mathrm{is}n\mathrm{Type}(X)\right), \quad x \mapsto (y \mapsto \|x \xrightarrow{=} y\|_n).$$

Let $\|A\|_{n+1} :\equiv \mathrm{im}(I_n)$ be the image of I_n, Definition 2.17.11, and let $|_|_{n+1} : A \to \|A\|_{n+1}$ be the map from the domain of I_n to its image, $x \mapsto (I_n(x), |(x, \mathrm{refl}_{I_n(x)})|)$ with $I_n(x) \equiv \|x \xrightarrow{=} _\|_n$ as defined above.

Since the type of n-types is an $(n+1)$-type, $\|A\|_{n+1}$ is an $(n+1)$-type by Lemma 2.15.5. We also note that the map

eq:trunc-path-eq

$$\|x \xrightarrow{=} y\|_n \xrightarrow{\simeq} (|x|_{n+1} \xrightarrow{=} |y|_{n+1}), \tag{2.26.1}$$

induced by the universal property of n-truncation, is an equivalence. Indeed, the right-hand side is equivalent to

$$\prod_{z:A} \big(\|x \xrightarrow{=} z\|_n \xrightarrow{\simeq} \|y \xrightarrow{=} z\|_n\big),$$

and we get an inverse by going backwards along this equivalence at $|\mathrm{refl}_y|_n : \|y \xrightarrow{=} y\|_n$.

To prove the universal property, let B be any $(n+1)$-type and $g : A \to B$ any map.

It suffices to show that for any $z : \|A\|_{n+1}$, there is a contractible type of extensions $\sum_{y:B}((_ \mapsto y) \xrightarrow{=}_{(|_|_{n+1}^{-1}(z)) \to B} (g \circ \mathrm{fst}))$, visualized by

$$\begin{array}{ccc} |_|_{n+1}^{-1}(z) & & \\ \downarrow & \searrow^{g \circ \mathrm{fst}} & \\ \mathbb{1} & \dashrightarrow & B, \end{array}$$

since then there's a contractible type of extensions of g to all of $\|A\|_{n+1}$. Since this is a proposition and $|_|_{n+1}$ is surjective, it suffices to prove this for z of the form $|x|_{n+1}$ with $x : A$. We need to show that the type

$$\prod_{x:A} \sum_{y:B} \prod_{x':A} \big((|x|_{n+1} \xrightarrow{=} |x'|_{n+1}) \to (y \xrightarrow{=} g(x'))\big)$$

is contractible. By the equivalence above, we can rewrite this, first as

$$\prod_{x:A}\sum_{y:B}\prod_{x':A}\left(\|x \xrightarrow{=} x'\|_n \to (y \xrightarrow{=} g(x'))\right),$$

and then, since $y \xrightarrow{=} g(x')$ is an n-type, as

$$\prod_{x:A}\sum_{y:B}\prod_{x':A}\left((x \xrightarrow{=} x') \to (y \xrightarrow{=} g(x'))\right).$$

Now we can contract away x' and the identification $x \xrightarrow{=} x'$, so we're left with

$$\prod_{x:A}\sum_{y:B}(y \xrightarrow{=} g(x')),$$

which is indeed contractible.

Finally, we need to re-size $\|A\|_{n+1}$ to fit in the universe $\mathcal{U}$ that A came from. By (2.26.1), its identity types are essentially $\mathcal{U}$-small by induction hypothesis, so again since $|_|_{n+1}$ is a surjection from the $\mathcal{U}$-small type A, the replacement principle, **??** 2.19.4, implies that $\|A\|_{n+1}$ is essentially $\mathcal{U}$-small. □

This construction is due to Rijke[103], see also the presentation in his book[104].

[103] Rijke, *The join construction*.

[104] Egbert Rijke. *Introduction to Homotopy Type Theory*. Forthcoming book with CUP. Version from 06/02/22. 2022.

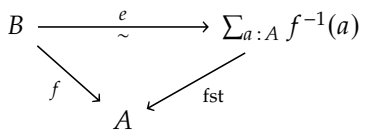
2.27 *Higher structure: stuff, structure, and properties*

sec:stuff-struct-prop

Recall from Lemma 2.25.2 that any map $f : B \to A$ can be described as "projecting away" its fibers, by using the equivalence e:

eq:forget-fibers

(2.27.1)



We say that f *forgets* these fibers. If A and B are groupoids, these fibers are themselves groupoids, but it can happen that they are sets, propositions, or even contractible. Accordingly, we say that:

- f *forgets at most structure* if all the fibers are sets;
- f *forgets at most properties* if all the fibers are propositions;
- f *forgets nothing* if all the fibers are contractible.

Here, the structure and properties in question are *on* a or *of* a, respectively, as captured by the fibers at a, for each $a : A$. Of course, a map forgets properties if and only if it's an injection, and it forgets nothing if and only if it's an equivalence.

Going in the other direction, we say that:

- f *forgets at most n-structure* if all the fibers are n-truncated. If $n \geq 1$, this is therefore a kind of *higher structure*.[106]

Thus, an element of a groupoid is 1-structure (this is sometimes informally called *stuff*), while an element of a set is a structure, or 0-structure, while an proof of a proposition is a property, or (-1)-structure.

Looking at (2.27.1) another way, we see that to give an element b of B lying over a given element $a : A$ amounts to specifying an element of

[105] John C. Baez and Michael Shulman. "Lectures on n-categories and cohomology". In: *Towards higher categories*. Vol. 152. IMA Vol. Math. Appl. Springer, New York, 2010, pp. 1–68. DOI: 10.1007/978-1-4419-1524-5_1. arXiv: math/0608420.

The precise formalization of the intuitive notions of "stuff", "structure", and "properties" was worked out in terms of category theory in *UseNet* discussions between John Baez, Toby Bartels, and James Dolan on `sci.physics.research` in 1998. It was clear that the simplest description was in terms of homotopy types, and hence it's even simpler in type theory. See also Baez and Shulman[105] for further discussion.

[106] We're updating the terminology slightly: In the above references, n-structure is referred to as *n-stuff*, but nowadays the term *higher structure* is more common, so we have renamed n-stuff into *n-structure*.

$f^{-1}(a)$, so we say that the elements of B are elements of A *with extra n-structure*, if the fibers $f^{-1}(a)$ are n-truncated.

Refining the usual image and image factorization from Definition 2.17.11 and Exercise 2.17.12, using Lemma 2.25.2, we can factor $f : B \to A$ through first its 0-*image* and then its usual (-1)-image as follows:[107]

$$B \xrightarrow{=} \sum_{a:A} f^{-1}(a) \to \sum_{a:A} \|f^{-1}(a)\|_0 \to \sum_{a:A} \|f^{-1}(a)\|_{-1} \to A$$

Here, the first map *forgets pure higher structure*, the second map *forgets pure structure*, while the last forgets at most properties (this is the inclusion of the usual image). Of course, each of these maps may happen to forget nothing at all. Saying that the second map forgets *pure* structure indicates that not only are the fibers sets, they are *nonempty* sets, so the structure in question exists, at least. Note also that the fibers of the first map are connected, which indicates that what is forgotten at this step, if anything, is pure higher structure.

[107] Using the general n-truncation from Section 2.26, we can define the n-image in a similar way and prove that the n-image factorization is unique. See Section 3.9 for the details. Since the unit type $\mathbb{1}$ is the unique (-2)-type, we have $\|X\|_{-2} \xrightarrow{=} \mathbb{1}$ for any type X.

Example 2.27.1. Let us look at some examples:

exa:stuff-struct-prop

- The first projection $\mathrm{fst} : \mathrm{FinSet} \times \mathrm{FinSet} \to \mathrm{FinSet}$ forgets 1-structure (stuff), namely the second set in the pair.
- The first projection $\mathrm{fst} : \sum_{A:\mathrm{FinSet}} A \to \mathrm{FinSet}$ from the type of pointed finite sets to the type of finite sets forgets structure, namely the structure of a chosen point.
- The inclusion of the type of sets with cardinality n, FinSet_n, into the type of all finite sets, FinSet, forgets properties, namely the property "having cardinality n". ⌟

Exercise 2.27.2. Analyze more examples of maps between groupoids in terms of "what is forgotten". ⌟

xca:stuff-struct-prop

xca:0im-to-1im

Exercise 2.27.3. Let $|_|' : \|f^{-1}(a)\|_0 \to \|f^{-1}(a)\|$ be the map defined by the induction principle in Definition 2.22.4 from $|_| : f^{-1}(a) \to \|f^{-1}(a)\|$. In the refined image factorization above, the map for the second arrow maps any pair (a, x) with $x : \|f^{-1}(a)\|_0$ to the pair $(a, |x|')$. For any $p : \|f^{-1}(a)\|$, give an equivalence from the fiber of the latter map at (a, p) to $\|f^{-1}(a)\|_0$. What is forgotten by this map, and what is remembered? ⌟

3
The universal symmetry: the circle

cha:circle

An effective principle in mathematics is that when you want to study a certain phenomenon you should search for a single type that captures this phenomenon. Here are two examples:[1]

it:one-out

(1) The contractible type $\mathbb{1}$ has the property that given any type A a function $\mathbb{1} \to A$ provides exactly the same information as picking an element in A. For, an equivalence from A to $\mathbb{1} \to A$ is provided by the function $a \mapsto (x \mapsto a)$, see Exercise 2.9.19.

it:prop-in

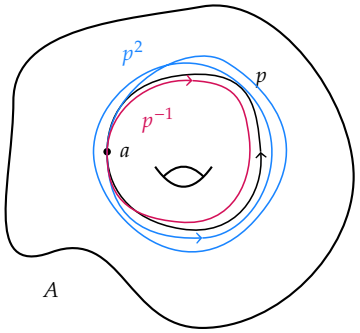
(2) The type Prop of propositions has the property that given any type A a function $A \to \mathrm{Prop}$ provides exactly the same information as picking a subtype of A, see Definition 2.20.3 and Lemma 2.20.10.

We are interested in symmetries, and so we should search for a type X which is so that given *any* type A the type of functions $X \to A$ (or $A \to X$, but that's not what we're going to do) picks out exactly the symmetries in A. We will soon see that there is such a type: the circle[2] which is built *exactly* so that this "universality with respect to symmetries" holds. It may be surprising to see how little it takes to define it; especially in hindsight when we eventually discover some of the many uses of the circle.

A symmetry in A is an identification $p : a \xrightarrow{=} a$ for some $a : A$. Now, we can take any iteration of p (composing p with itself a number of times), and we can consider the inverse p^{-1} and *its* iterations. So, by giving one symmetry we give at the same time a lot of symmetries. For a particular $p : a \xrightarrow{=} a$ it may be that some of the iterations can be identified (in their type $a \xrightarrow{=} a$). For instance, it may be that there is an identification of type $p^2 \xrightarrow{=} p^0$ (as in Exercise 2.13.3). Even more dramatically: if there is an identification of type $p \xrightarrow{=} \mathrm{refl}_a$, then *all* the iterations of p can be identified with each other. However, in general we must be prepared that all the iterations p_n of p (for n positive, 0 and negative) are distinct. Hence, the circle must have a distinct symmetry for every integer. We would have enjoyed defining the integers this way, but being that ideological would be somewhat inefficient. Hence we give a more hands-on approach and define the circle and the integers separately. Thereafter we prove that the type of symmetries in the circle is equivalent to the set of integers.

[1] Notice that these have arrows pointing in different directions: In (1) we're mapping *out* of $\mathbb{1}$, while in (2) we're mapping *in* to Prop.

[2] We call this type the "circle" because it has many properties which are analogues, in our context, of properties of the topological circle $\{(x, y) \in \mathbb{R}^2 \mid x^2 + y^2 = 1\}$. See Section B.3 for a discussion of the relationship between topological spaces and types. In the later chapters on geometry we'll return to "real" geometrical circles.



3.1 *The circle and its universal property*

sec:S1

Propositional truncation from Section 2.16 was the first *higher inductive type*, that is, an inductive type with constructors both for elements and for

identifications, we introduced. The circle is another example of a higher inductive type, see Chapter 6 of the HoTT book[3] for more information.

[3] Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*.

def:circle

DEFINITION 3.1.1. The circle is a type $\mathrm{S}^1 : \mathcal{U}$ with an element (constructor) $\bullet : \mathrm{S}^1$ and an identification (constructor) $\circlearrowleft : \bullet \xrightarrow{=} \bullet$. For convenience and clarity the (higher) induction principle for S^1 is explained by first stating a recursion principle for S^1.

Let A be a type. In order to define a function $f : \mathrm{S}^1 \to A$, it suffices to give an element a of A together with an identification l of type $a \xrightarrow{=} a$. The function f defined by this data satisfies $f(\bullet) \equiv a$ and the recursion principle provides an identification of type $\mathrm{ap}_f(\circlearrowleft) = l$.

Let $A(x)$ be a family of types parametrized by the variable $x : \mathrm{S}^1$. The induction principle of S^1 states that, in order to define a family of elements of $A(x)$ parametrized by the variable $x : \mathrm{S}^1$, it suffices to give an element a of $A(\bullet)$ together with an identification l of type $a \xrightarrow[\circlearrowleft]{=} a$, see Figure 3.1. The function $f : \prod_{x:\mathrm{S}^1} A(x)$ defined by this data satisfies $f(\bullet) \equiv a$ and the induction principle provides an identification of type $\mathrm{apd}_f(\circlearrowleft) \xrightarrow{=} l$. ⌟

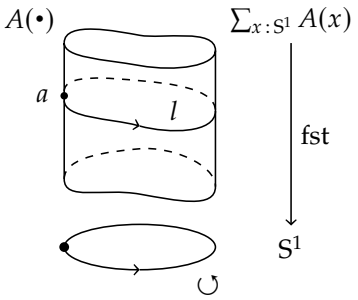


FIGURE 3.1: The induction principle of S^1.

fig:circle-induction

Giving a as above is referred to as 'the base case', and giving l as 'the loop case'. Given this input data to define a function f will often be abbreviated by writing $f(\bullet) :\equiv a$ and $f(\circlearrowleft) := l$. Notice the use of $:=$ in the second definition, instead of $:\equiv$. That signifies that $f(\circlearrowleft)$ and l are not equal by definition, but rather, that an identification is given between them, i.e., an element of type $f(\circlearrowleft) \xrightarrow{=} l$ is given, or an element of $\mathrm{apd}_f(\circlearrowleft) \xrightarrow{=} l$ is given, in the dependent case.

The following result states that any function from the circle exactly picks out an element and a symmetry of that element. This is a "universal property" of the circle.

lem:freeloopspace

THEOREM 3.1.2. *For all types A, the evaluation function*

$$\mathrm{ev}_A : (\mathrm{S}^1 \to A) \to \sum_{a:A} (a \xrightarrow{=} a) \text{ defined by } \mathrm{ev}_A(g) :\equiv (g(\bullet), g(\circlearrowleft))$$

is an equivalence, with inverse ve_A defined by the recursion principle of the circle.

Proof. Fix $A : \mathcal{U}$. We apply Construction 2.9.9. For all $a : A$ and $l : a \xrightarrow{=} a$ we may construct an identification of type $\mathrm{ev}(\mathrm{ve}(a, l)) \xrightarrow{=} (a, l)$ by the recursion principle. It remains to construct identifications of type $\mathrm{ve}(\mathrm{ev}(f)) \xrightarrow{=} f$ for all $f : \mathrm{S}^1 \to A$. Such constructions are provided by the following more general result. Given $f, g : \mathrm{S}^1 \to A$, $p : f(\bullet) \xrightarrow{=} g(\bullet)$, and $q : f(\circlearrowleft) \xrightarrow{=} p^{-1} \cdot g(\circlearrowleft) \cdot p$, we construct an identification of type $f \xrightarrow{=} g$, as follows. It suffices, by function extensionality, to construct an element of type $P(x) :\equiv (f(x) \xrightarrow{=} g(x))$ for a variable $x : \mathrm{S}^1$. This we do by circle induction. For the base case we take p. The loop case reduces to constructing an identification of type $\mathrm{trp}^P_{\circlearrowleft}(p) \xrightarrow{=} p$, by Definition 2.7.3. By Construction 2.14.3 we have an identification of type $\mathrm{trp}^P_{\circlearrowleft}(p) \xrightarrow{=} g(\circlearrowleft) \cdot p \cdot f(\circlearrowleft)^{-1}$. Using q we construct an identification of type $g(\circlearrowleft) \xrightarrow{=} p \cdot f(\circlearrowleft) \cdot p^{-1}$. Hence we may construct an identification of type $\mathrm{trp}^P_{\circlearrowleft}(p) \xrightarrow{=} p$, by an easy calculation. Now apply Lemma 2.10.3, and we have constructed a function of type $(\mathrm{ev}(f) \xrightarrow{=} \mathrm{ev}(g)) \to (f \xrightarrow{=} g)$.

Now we get an identification of type $\mathrm{ve}(\mathrm{ev}(f)) \xrightarrow{=} f$, for we have an identification of type $\mathrm{ev}(\mathrm{ve}(\mathrm{ev}(f))) \xrightarrow{=} (f(\bullet), f(\circlearrowleft))$, and $(f(\bullet), f(\circlearrowleft)) \equiv \mathrm{ev}(f)$, with $p :\equiv \mathrm{refl}_{f(\bullet)}$ and q coming from the induction principle. □

cor:circle-loopspace

Corollary 3.1.3. *For any $a : A$, the function*

$$\mathrm{ev}^a_A : ((\mathrm{S}^1, \bullet) \to_* (A, a)) \to (a \xrightarrow{=} a)$$

sending (g, p) to $p^{-1} \cdot g(\circlearrowleft) \cdot p$ is an equivalence.

Proof.[4] Consider the following diagram (see Remark 2.15.10):

$$\begin{array}{ccc} (\mathrm{S}^1 \to A) & \xrightarrow{g \mapsto (g(\bullet), g, \mathrm{refl}_{g(\bullet)})} & \sum_{a:A} ((\mathrm{S}^1, \bullet) \to_* (A, a)) \\ & {\scriptstyle \mathrm{ev}_A} \searrow \quad \swarrow {\scriptstyle \mathrm{tot}(\mathrm{ev}^-_A)} & \\ & \sum_{a:A} (a \xrightarrow{=} a), & \end{array}$$

where the top map is an equivalence by Corollary 2.9.11, and the left map is an equivalence by Theorem 3.1.2. This diagram represents the identity type $\mathrm{ev}_A \xrightarrow{=} (g \mapsto (g(\bullet), \mathrm{refl}^{-1}_{g(\bullet)} \cdot g(\circlearrowleft) \cdot \mathrm{refl}_{g(\bullet)}))$. An identification of this type is provided by function extensionality and Exercise 2.5.3. The result now follows from Lemma 2.9.17. □

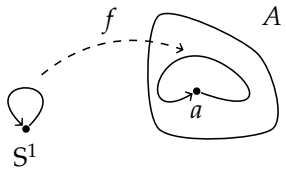
[4] This can also be done directly: The inverse to ev^a_A sends $l : a \xrightarrow{=} a$ to $(\mathrm{ve}_A(a, l), \mathrm{refl}_a)$. Try to verify this!

rem:dep-univ-prop-circle

Remark 3.1.4. By almost the same argument as for Theorem 3.1.2 one can obtain the dependent universal property of the circle. Given a type family $A : \mathrm{S}^1 \to \mathcal{U}$, the dependent evaluation function, which also maps g to $(g(\bullet), g(\circlearrowleft))$ but has type $(\prod_{x:\mathrm{S}^1} A(x)) \to \sum_{a:A(\bullet)} (a \xrightarrow[\circlearrowleft]{=} a)$, is an equivalence. (Compare the latter type to the type of ev_A in Theorem 3.1.2 and see Figure 3.1.) ⌟

Remark 3.1.5. A function $f : \mathrm{S}^1 \to A$ is often called a *loop* in A, the picture being that f throws $\circlearrowleft : \bullet \xrightarrow{=} \bullet$ as a lasso in the type A.

Using the equivalence in Corollary 3.1.3 and univalence, $a \xrightarrow{=} a$ is identified with the pointed functions from the circle, which allows for a very graphic interpretation of the symmetries of a in A: they are traced out by a function f from the circle and can be seen as loops in the type A starting and ending at a![5] ⌟



[5] This is of course how we have been picturing loops the whole time.

lem:circleisconnected

Lemma 3.1.6. *The circle is connected.*

Proof. We show $\|\bullet \xrightarrow{=} z\|$ for all $z : \mathrm{S}^1$ by circle induction as in Definition 3.1.1. For the base case we take $|\mathrm{refl}_\bullet| : \|\bullet \xrightarrow{=} \bullet\|$. The loop case is immediate as $\|\bullet \xrightarrow{=} \bullet\|$ is a proposition. □

In the proof above, the propositional truncation coming from the definition of connectedness is essential. If this truncation were removed we wouldn't know what to do in the induction step (actually, having an element of type $\prod_{z:\mathrm{S}^1} (\bullet \xrightarrow{=} z)$ contradicts the univalence axiom). This said, the family $R : \mathrm{S}^1 \to \mathcal{U}$ with $R(z) :\equiv (\bullet \xrightarrow{=} z)$ is extremely important for other purposes. In Example 3.3.9, we will call R the "universal set bundle" of the circle, and it is the key tool in proving that the type of symmetries in the circle is a set that can be identified with the set of integers. Recall that we use the phrase "symmetries *in* the circle" to refer to the elements of $\bullet \xrightarrow{=} \bullet$,[6] whereas we use the phrase "symmetries *of* the

[6] Here we are using "the circle" to mean the *pointed* type $(\mathrm{S}^1, \bullet)$. But it also turns out that the type $\bullet \xrightarrow{=} \bullet$ is equivalent to the type $x \xrightarrow{=} x$, for any $x : \mathrm{S}^1$.

circle" to refer to the elements of $S^1 \xrightarrow{=}_{\mathcal{U}} S^1$. The latter type is equivalent to $S^1 \amalg S^1$, as follows from Exercise 3.4.11 and Exercise 3.4.12.

In order to proceed, we should properly define the set of integers and explore the concept of set bundles.

3.2 *The integers*

sec:integers

def:zet

ft:many-integers

We define the type of integers in one of the many possible ways.[7]

Definition 3.2.1. Let Z be the higher inductive type with the following three constructors:

(1) $\iota_+ : \mathbb{N} \to \mathsf{Z}$ for the nonnegative numbers, $0, 1, \dots$

(2) $\iota_- : \mathbb{N}^- \to \mathsf{Z}$ for the nonpositive numbers, $-0, -1, \dots$

(3) $\mathrm{zeq} : \iota_-(-0) = \iota_+(0)$.

Because we used the copy $\mathbb{N}^-$ for the nonpositive numbers from Example 2.12.9, we can leave out the constructor symbols $\iota_\pm$ when the type is clear from context. Thus we have $\dots, -2, -1, -0, 0, 1, 2, \dots : \mathsf{Z}$ and $\mathrm{zeq} : -0 \xrightarrow{=}_{\mathsf{Z}} 0$.

The type Z comes with an induction principle: Let $T(z)$ be a family of types parametrized by $z : \mathsf{Z}$. In order to construct an element $f(z)$ of $T(z)$ for all $z : \mathsf{Z}$, it suffices to give functions g and h such that $g(n) : T(\iota_+(n))$ and $h(n) : T(\iota_-(m))$ for all $n : \mathbb{N}$, $m : \mathbb{N}^-$, together with $q : h(-0) \xrightarrow[\mathrm{zeq}]{=} g(0)$. Here g and h can be defined by induction on $n : \mathbb{N}$, $m : \mathbb{N}^-$.[8]

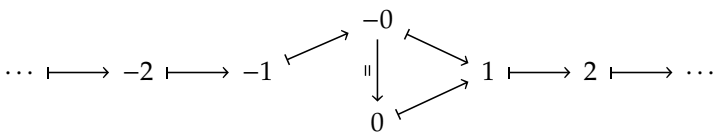
The resulting function $f : \prod_{z:\mathsf{Z}} T(z)$ satisfies $f(n) \equiv g(n)$ and $f(-n) \equiv h(-n)$ for $n : \mathbb{N}$, and there is an (unnamed) element of $\mathrm{apd}_f(\mathrm{zeq}) = q$. ⌟

Like the type $\mathbb{N}$, the type Z is a set with decidable equality and ordering relations.

One well known self-equivalence is *negation*, $- : \mathsf{Z} \to \mathsf{Z}$, inductively defined by setting $-\iota_+(n) :\equiv \iota_-(-n)$, $-\iota_-(m) :\equiv \iota_+(-m)$, $\mathrm{ap}_-(\mathrm{zeq}) := \mathrm{zeq}^{-1}$.[9] Negation is its own inverse.

The *successor* function $\mathrm{s} : \mathsf{Z} \to \mathsf{Z}$ is likewise defined inductively, setting $\mathrm{s}(n) :\equiv \mathrm{succ}(n)$, $\mathrm{s}(-0) :\equiv 1$, $\mathrm{s}(-\mathrm{succ}(n)) :\equiv -n$, and $\mathrm{ap}_{\mathrm{s}}(\mathrm{zeq}) := \mathrm{refl}_1$.

The successor function s is an equivalence. It is instructive to depict iterating s in both directions as a doubly infinite sequence containing all integers:



The inverse s^{-1} of s is called the *predecessor* function. We recall the n-fold iteration s^n defined earlier; the n-fold iteration of s^{-1} will be denoted by s^{-n}. Since $\mathrm{s}^0 \equiv \mathrm{id} \equiv \mathrm{s}^{-0}$, this defines the iteration s^z for all $z : \mathsf{Z}$.[10]

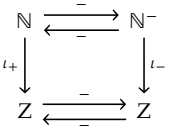
Addition of integers is now defined by iteration: $z + y :\equiv \mathrm{s}^y(z)$. This extends + on the ι_+-image of $\mathbb{N}$, see Exercise 3.2.2. From addition and $- : \mathsf{Z} \to \mathsf{Z}$ one can define a *subtraction* function setting $z - y :\equiv z + (-y)$. Since addition and subtraction are mutually inverse, the function $w \mapsto z + w$ is an equivalence, and we may iterate it to define *multiplication*: $zy :\equiv (w \mapsto z + w)^y(0)$.

[7] Here are some of these alternatives:

- As the copy of $\mathbb{N}$ where $2n$ means n and $2n + 1$ means $-n - 1$, for $n : \mathbb{N}$.
- As the sum $\mathbb{N} \amalg \mathbb{N}$, where inl_n means $-n - 1$ and inr_n means n.
- As the sum $\mathbb{N} \amalg \mathbb{1} \amalg \mathbb{N}$, where from the left copy of $\mathbb{N}$ we get $-n - 1$, from the center $0 : \mathbb{1}$ we get 0, and from the right copy of $\mathbb{N}$ we get $n + 1$, for $n : \mathbb{N}$.
- As the quotient of $\mathbb{N} \times \mathbb{N}$ under the equivalence relation $(n, m) \sim (n', m')$ defined by $n + m' = n' + m$, where (n, m) represents $n - m$.
- As the subset of $\mathbb{N} \times \mathbb{N}$ consisting of those (n, m) with $n = 0 \vee m = 0$ (picking canonical representatives for the above equivalence relation).
- As the loops $\bullet \xrightarrow{=} \bullet$ in the circle.

[8] Of course, giving h is the same as giving $h' : \prod_{n:\mathbb{N}} T(-n)$.

[9] Here we included the constructor symbols for clarity, but the definition allows us to use the negation symbol unadorned, because the following diagram is commutative by definition:



[10] In the same way, we can define the iteration $f^z : X \to X$ for any *equivalence* $f : X \to X$.

xca:addition-on-Z-and-N

EXERCISE 3.2.2. Show that $\iota_+(n + m) = \iota_+(n) + \iota_+(m)$ and $\iota_+(nm) = \iota_+(n)\iota_+(m)$ for all $n, m : \mathbb{N}$. ⌟

The ordering relations $<$ and $\leq$ on Z are easily defined and shown to extend those on $\mathbb{N}$.

Recall the induction principle for Z in Definition 3.2.1 above. Instead of defining g and h explicitly, we will often give $f(0)$ directly, and define g' and h' such that $g'(z) : T(z) \to T(z + 1)$ for all $z : \mathrm{Z}$ with $z \geq 0$, and $h'(z) : T(z) \to T(z - 1)$ for all $z : \mathrm{Z}$ with $z \leq 0$. The function f thus defined satisfies $f(-0) \equiv f(0)$, $f(z + 1) \equiv g'(z, f(z))$ for all $z \geq 0$, and $f(z - 1) \equiv h'(z, f(z))$ for all $z \leq 0$.

xca:commutative-add-Z

EXERCISE 3.2.3. Show that $x + y = y + x$ and $xy = yx$ for all $x, y : \mathrm{Z}$. ⌟

3.3 *Set bundles*

sec:covering

As mentioned earlier, it is possible to define the integers as the type $\bullet \xrightarrow{=} \bullet$ of symmetries in the circle. Our investigation of $\bullet \xrightarrow{=} \bullet$ will use the concept of set bundles. Since we are going to return to this concept several times, we take the time for a fuller treatment before we continue with proving the equivalence of $\bullet \xrightarrow{=} \bullet$ and Z.

def:covering

DEFINITION 3.3.1. A *set bundle* over a type B is a map $f : A \to B$ such that for each $b : B$ the preimage (fiber) $f^{-1}(b)$ is a set. We say that a set bundle $f : A \to B$ over B is

- *connected* if A is connected,
- *finite* if all preimages are finite sets,
- *decidable* if all preimages are decidable sets.

If A and B are pointed types, a *pointed* set bundle is a pointed map $f : A \to_* B$ such that, when forgetting the points, $f_\div : A_\div \to B_\div$ is a set bundle. Here it suffices that A is a pointed type.[11]

We do not require the preimages of $f_\div$ to be pointed types. ⌟

With a formula, given a type B, the type of set bundles over B is

$$\mathrm{SetBundle}(B) :\equiv \sum_{A:\mathcal{U}} \sum_{f:A\to B} \prod_{b:B} \mathrm{isSet}(f^{-1}(b)),$$

with variations according to the flavor.

Recall the equivalence in Construction 2.25.6(3) between the type $B \to \mathrm{Set}$ of families of sets parametrized by elements of B, and the type of set bundles over B given above. We shall frequently use this equivalence, even without explicit mention.

lem:setbundle-is-groupoid

LEMMA 3.3.2. *For any type B,* $\mathrm{SetBundle}(B)$ *is a groupoid.*

Proof. By Lemma 2.22.1 we have that Set is a groupoid, and hence $B \to \mathrm{Set}$ is a groupoid by Lemma 2.15.5(1). □

Moreover, by Corollary 2.20.12, all variations of set bundles in Definition 3.3.1 defined by a predicate are groupoids as well. This does not apply *pointed* set bundles: a point is extra structure, not just a property.

We should notice that the notion of a set bundle is just one step up from the notion of an injection (a map such that all the preimages are propositions – following the logic, injections perhaps ought to be called

[11] Given a pointed type (A, a), a type B and a map $f : A \to B$, $(f, \mathrm{refl}_{f(a)}) : (A, a) \to_* (B, f(a))$ is a pointed map. Indeed, the forgetful map $\left(\sum_{b:B}((A, a) \to_* (B, b))\right) \to (A \to B)$ is an equivalence by Corollary 2.9.11.

"proposition bundles"). The formulation we give is not the only one and for some purposes a formulation based on $B \to \mathrm{Set}$ is more convenient.

xca:covering-utils

Exercise 3.3.3. Let A, B and C be types. Show:

(1) The (unique) map of type $A \to \mathbb{1}$ is a set bundle iff A is a set;

(2) For any $b : B$, the map $x \mapsto b$ from $\mathbb{1}$ to B is a set bundle iff $b \xrightarrow{=} b$ is a set;

(3) If $f : A \to B$ and $g : B \to C$ are set bundles, then gf is a set bundle.

it:left-cancel-cover

(4) If $f : A \to B$ and $g : B \to C$, and g and gf are set bundles, then f is a set bundle. Hint: apply Corollary 2.17.9 to $\mathrm{ap}_g : (b \xrightarrow{=} f(a)) \to (g(b) \xrightarrow{=} g(f(a)))$.

(5) If A is connected, $a \xrightarrow{=}_A a$ is connected for some $a : A$, B is a groupoid, and $f : A \to B$ is a set bundle, then A is contractible. Hint: use Corollary 2.17.9 and Exercise 2.16.10.

it:cover-n-type

(6) If $f : A \to B$ is a set bundle and B is an n-type with $n \geq 0$, then A is also an n-type. ⌟

Figure 3.2 visualizes two examples of set bundles over the circle. Consider the picture on the left first. If we let b be the element on the circle marked at the bottom left hand side, then the preimage $f^{-1}(b)$ is marked by the the two dots in A straight above b, so that in this case each preimage contains two points (i.e., each preimage can be merely identified with Bool). However, A is not the constant family, like A' depicted on the right, since we have a string of identifications $A' :\equiv \sum_{z:\mathrm{S}^1} \mathrm{Bool} \xrightarrow{=} (\mathrm{S}^1 \times \mathrm{Bool}) \xrightarrow{=} (\mathrm{S}^1 + \mathrm{S}^1)$, and the latter type is not connected. Obviously something way more fascinating is going on.

fig:covering

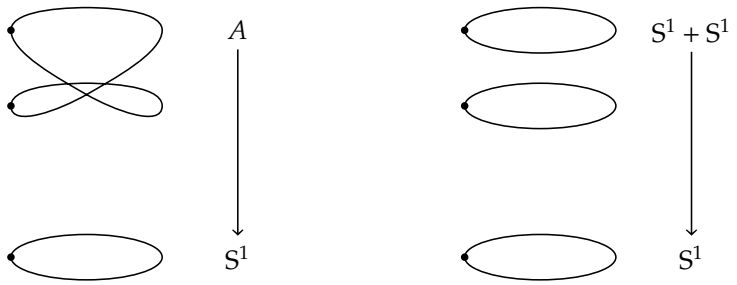


Figure 3.2: A visualization of two set bundles over the circle

xca:twoS1coverings

Exercise 3.3.4. In this exercise you are asked to elaborate the difference between A and A' above. Let $c_{\mathrm{Bool}} :\equiv (z \mapsto \mathrm{Bool}) : \mathrm{S}^1 \to \mathrm{Set}$.

(1) This part is about A'. Show that $\sum_{z:\mathrm{S}^1} \mathrm{Bool}$ is not connected. Give an element of the type $c_{\mathrm{Bool}} \xrightarrow{=} \mathrm{ve}_{\mathrm{Set}}(\mathrm{Bool}, \mathrm{refl}_{\mathrm{Bool}})$.

(2) One can define A by $A :\equiv \sum_{z:\mathrm{S}^1} \mathrm{ve}_{\mathrm{Set}}(\mathrm{Bool}, \mathrm{swap})$. Show that A is connected. Give an element of type $(c_{\mathrm{Bool}} \xrightarrow{=} \mathrm{ve}_{\mathrm{Set}}(\mathrm{Bool}, \mathrm{swap})) \to \mathrm{False}$. Hint: use Exercise 2.13.3 and Theorem 3.1.2. ⌟

Remark 3.3.5. It *is* possible to misunderstand what a "connected set bundle" is: the other interpretation "all the preimages are connected" would simply give us an equivalence (since connected sets are contractible), and this is *not* what is intended. (Equivalences are set bundles, but not necessarily connected set bundles and connected set bundles are not necessarily equivalences.)

Likewise for the other qualifications; for instance, in a "finite set bundle" $f : A \to B$, all fibers are finite sets, but the type A is usually *not* a finite set.

We trust the reader to keep our definitions in mind and not the other interpretations. ⌟

Remark 3.3.6. Set bundles are closely related to a concept from topology called "covering spaces" (or any variant of this concept, including Galois theory) and from algebra as locally constant sheaves (of sets). Either way, the concept is useful because it singles out the (sub)symmetries. ⌟

In this chapter, we focus on set bundles over the circle. We start by refining the notion of diagram introduced in Remark 2.15.10.

rem:subtype-diagram

Remark 3.3.7. Consider the left diagram below, where i_1, i_2 are injections constituting S as a subtype of X and T as a subtype of Y, respectively, in the sense of Definition 2.20.9.[12] This diagram represents the identity type $f \circ i_1 \xrightarrow{=} i_2 \circ g$. Since i_2 is an injection, the type $\sum_{g : S \to T}(f \circ i_1 \xrightarrow{=} i_2 \circ g)$ is a proposition,[13] which may or may not be true. So, when is it true?

$$\begin{array}{ccc} X & \xrightarrow{f} & Y \\ {\scriptstyle i_1}\uparrow & & \uparrow{\scriptstyle i_2} \\ S & \xrightarrow{g} & T \end{array} \qquad\qquad \begin{array}{ccc} X & \xrightarrow{f} & Y \\ {\scriptstyle \mathrm{fst}}\uparrow & & \uparrow{\scriptstyle \mathrm{fst}} \\ X_P & \overset{g}{\dashrightarrow} & Y_Q \end{array}$$

In the right diagram we depict the case in which S is given by a predicate $P : X \to \mathrm{Prop}$ and T by a predicate $Q : Y \to \mathrm{Prop}$, with the injections being first projections. We can now apply the universal property of subtypes Exercise 2.20.8 to Y_Q with $(f \circ \mathrm{fst}) : X_P \to Y$ and get that the three propositions $\prod_{z : X_P} Q(f(\mathrm{fst}(z)))$ and $\prod_{x : X}(P(x) \to Q(f(x)))$ and $\sum_{g : X_P \to Y_Q}(f \circ \mathrm{fst} \xrightarrow{=} \mathrm{fst} \circ g)$ are logically equivalent. If these propositions hold, we say that f *respects the subtypes* and we may call the diagram a *subtype diagram*.

If q is a proof of $\prod_{x : X}(P(x) \to Q(f(x)))$, then we can uniquely define the function $g : X_P \to Y_Q$, the one labelling the dashed arrow in the right diagram above, by $(x, p) \mapsto (f(x), q(x, p))$. The functions $f \circ \mathrm{fst}$ and $\mathrm{fst} \circ g$ are identified by reflexivity. We may call g the *function induced by f on subtypes*, and will also denote it by f.

Now consider the special case in which $f : X \to Y$ is an equivalence. If the inverse of f also respects the subtypes, that is, if $\prod_{y : Y}(Q(y) \to P(f^{-1}(y)))$, then the function that is induced by f on the subtypes is also an equivalence. Moreover, the functions induced by f and f^{-1} are then each other's inverses. ⌟

thm:coveringsofS1preims

Theorem 3.3.8. *In the diagram below, the equivalence f in the second row is* preim *from Lemma 2.25.3, and the equivalence g in the same row is defined by $g(S) :\equiv (S(\bullet), \mathrm{trp}^{\mathrm{id}_{\mathcal{U}}}_{S(\circlearrowleft)})$ (Theorem 3.1.2 applied with $A :\equiv \mathcal{U}$, and Definition 2.13.1). Along the vertical arrows we have maps that forget the property that constitutes its domain as a subtype of the codomain, all modest variations of the first projection.*

The statement of the theorem is now that the diagram below is the composite of subtype diagrams (see Remark 3.3.7) in which the induced functions f and g in the third row are equivalences as well.

[12] To stress that a function is an injection we may decorate the $\to$ in its type with a hook: $\hookrightarrow$.

[13] Consider $i_2^{-1}(f(i_1(s)))$ for all $s : S$, and then use Exercise 2.9.24.

$$\begin{array}{ccccc}
 & & & & \sum_{X:\mathcal{U}}(X \to X) \\
 & & & & \uparrow \\
\left(\sum_{A:\mathcal{U}}(A \to \mathrm{S}^1)\right) & \xrightarrow[\sim]{f} & \left(\mathrm{S}^1 \to \mathcal{U}\right) & \xrightarrow[\sim]{g} & \sum_{X:\mathcal{U}}(X \xrightarrow{\simeq} X) \\
\uparrow & & \uparrow & & \uparrow \\
\mathrm{SetBundle}(\mathrm{S}^1) & \xrightarrow[f]{\sim} & \left(\mathrm{S}^1 \to \mathrm{Set}\right) & \xrightarrow[g]{\sim} & \sum_{X:\mathrm{Set}}(X \xrightarrow{\simeq} X)
\end{array}$$

Proof. We prove first that preim respects the subtypes. Let $A : \mathcal{U}$ and $h : \mathrm{S}^1 \to A$ such that (A, h) is a set bundle. This means that $h^{-1}(a)$ is a set, for any $a : A$. Since $\mathrm{preim}(A, h)(a) \equiv h^{-1}(a)$, we immediately get that $\mathrm{preim}(A, h) : \mathrm{S}^1 \to \mathrm{Set}$. In order to prove that preim^{-1} also respects the subtypes one simply reverses this argument.

Next we prove that g respects the subtypes. Let $S : \mathrm{S}^1 \to \mathcal{U}$ be such that $S(z)$ is a set for all $z : \mathrm{S}^1$. This means in particular that $S(\bullet)$ is a set. Since $g(S) \equiv (S(\bullet), \mathrm{trp}_{S(\circlearrowleft)})$, we are done. In order to prove that g^{-1} also respects the subtypes we reason as follows. Let $X : \mathcal{U}$ and $h : X \xrightarrow{\simeq} X$ be given. Assume that X is a set. We have $g^{-1}(X, h) \equiv \mathrm{ve}_{\mathcal{U}}(X, \bar{h})$, see Theorem 3.1.2 and **??** 2.13.2. Now, since $X \equiv \mathrm{ve}_{\mathcal{U}}(X, \bar{h})(\bullet)$ is a set and S^1 is connected, we have $g^{-1}(X, h) : \mathrm{S}^1 \to \mathrm{Set}$ and we are done.

Note that the left subtype diagram is fully general: it also holds when we replace S^1 by any type B. This is not true for the right subtype diagram. □

In slogan form: A set bundle over the circle is a set with a permutation of its elements. The fiber over $\bullet : \mathrm{S}^1$ gives the set, and transporting along $\circlearrowleft$ gives the permutation.

Example 3.3.9. A simple yet important example of a set bundle over a groupoid B with an element b_0 is given by the family of identity types $\mathbb{P}_{b_0}(b) :\equiv (b_0 \xrightarrow{=} b)$ parameterized by $b : B$. These identity types are indeed sets since B is a groupoid. The alternative form of this (pointed) set bundle is the map $\mathrm{fst} : \sum_{b:B}(b_0 \xrightarrow{=} b) \to B$, the domain canonically pointed at $(b_0, \mathrm{refl}_{b_0})$, and with refl_{b_0} as the pointing path of fst.

In the above example, the reader may have noticed that, by Lemma 2.9.2, $\sum_{b:B}(b_0 \xrightarrow{=} b)$ is contractible. Hence yet another form of this set bundle is the constant map $\mathrm{cst}_{b_0} : \mathbb{1} \to B$, also with pointing path refl_{b_0}. What is special about these examples is captured by the following definition and ensuing lemma. ⌟

def:universalcover

Definition 3.3.10. Let A and B be pointed types and $f : A \to_* B$ a pointed set bundle. We call f *universal* if for every pointed set bundle $g : C \to_* B$ there is a unique $h : A \to_* C$ with $f \xrightarrow{=} gh$, that is, if the following type is contractible:

def:univ-cover

$$\sum_{h : A \to_* C} f \xrightarrow{=}_{A \to_* B} gh. \qquad ⌟$$

In the above definition we get that $h : A \to_* C$ is a set bundle as well by Exercise 3.3.3(4). The examples preceding Definition 3.3.10 are indeed universal set bundles according to the following lemma.

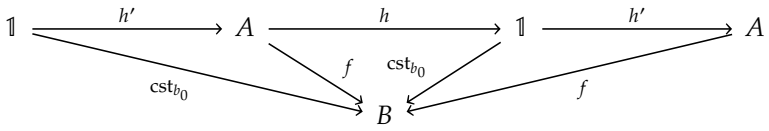
lem:univ-cover-of-groupoid

Lemma 3.3.11. *Let (A, a_0) be a pointed type, (B, b_0) a pointed groupoid, and $f : (A, a_0) \to_* (B, b_0)$ a pointed set bundle. Then f is universal if and only if A is contractible.*

Proof. Let conditions be as above and assume A is contractible. Let (C, c_0) be a pointed type and $g : (C, c_0) \to_* (B, b_0)$ a set bundle. Let $f_0 : b_0 \xrightarrow{=} f(a_0)$ and $g_0 : b_0 \xrightarrow{=} g(c_0)$ be the respective pointing paths. Define $h : (A, a_0) \to_* (C, c_0)$ by $a \mapsto c_0$ with pointing path refl_{c_0}. Clearly $g_0 f_0^{-1} : f(a_0) \xrightarrow{=} g(h(a_0)) \equiv g(c_0)$, which yields an identification of $f(a)$ and $g(h(a))$ for all $a : A$ as A is contractible. Apply now function extensionality **??** 2.9.18 to get an identification of type $f \xrightarrow{=}_{A\to B} gh$. The pointing path of gh is also $g_0 : b_0 \xrightarrow{=} g(c_0)$. We get an identification of type $(f, f_0) \xrightarrow{=}_{A\to_* B} (gh, g_0)$ since $g_0 = (g_0 f_0^{-1}) f_0$. The type $(A, a_0) \to_* (C, c_0)$ is contractible since A is contractible, yielding that h is unique.

Draw the triangle!

For the other direction of the lemma we use a reasoning pattern that is typical for universality. Assume that f is universal. As shown above, $\mathrm{cst}_{b_0} : \mathbb{1} \to_* (B, b_0)$ is also universal. Hence we have maps h and h' and identifications of all identity types represented in the following diagram, simplified by ignoring the points:



Using the universality of f, we can identify $h'h$ with id_A. Using the universality of cst_{b_0}, we can identify hh' with $\mathrm{id}_{\mathbb{1}}$. Now Construction 2.9.9 yields an equivalence between $\mathbb{1}$ and A, implying that A is contractible. □

A particularly important example of a pointed set bundle is the following.

def:RtoS1

Definition 3.3.12. Recall the set of integers Z from Definition 3.2.1, with its successor function $\mathrm{s} : \mathrm{Z} \xrightarrow{\simeq} \mathrm{Z}$ being an equivalence. The set bundle $R : \mathrm{S}^1 \to \mathcal{U}$ is defined by the recursion principle of the circle from Definition 3.1.1 by putting $R(\bullet) :\equiv \mathrm{Z}$ and $R(\circlearrowleft) := \bar{\mathrm{s}}$. This is indeed a set bundle since S^1 is connected, so that $R(x)$ is a set for all $x : \mathrm{S}^1$. We also write $R : \mathrm{S}^1 \to \mathrm{Set}$. Recall $\mathrm{Tot}(R) \equiv \sum_{z:\mathrm{S}^1} R(z)$ and point $\mathrm{Tot}(R)$ at $(\bullet, 0)$. Now define

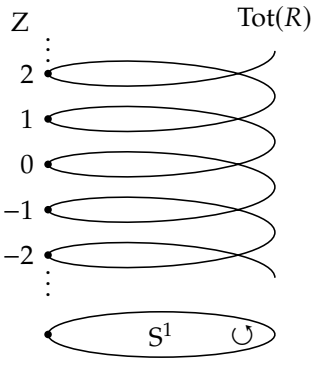
$$\mathrm{exp} :\equiv \mathrm{fst} : \mathrm{Tot}(R) \to \mathrm{S}^1, \text{ with pointing path } \mathrm{refl}_{\bullet}.$$

We call exp the *exponential set bundle over the circle*. ⌟

rem:exptorreal

Remark 3.3.13. The reason for the name "exponential" comes from the following visualization. If x is a real number, then the complex exponentiation $\mathrm{e}^{2\pi \mathrm{i} x} = \cos(2\pi x) + \mathrm{i}\sin(2\pi x)$ has absolute value 1 and so defines a continuous function to the unit circle $\{(x, y) : \mathbb{R}^2 \mid x^2 + y^2 = 1\}$, where we have identified $\mathbb{R}^2$ with the complex numbers. Choosing any point z on the unit circle, we see that the preimage of z under the exponential function is a shifted copy of the integers inside the reals.[14]

This connection between the integers and the unit circle is precisely captured in a form that we can take further by studying the set bundle $\mathrm{exp} : \mathrm{Tot}(R) \to \mathrm{S}^1$. ⌟



In the next section we will see that the exponential set bundle of the circle is in fact universal. We'll continue the general study of set bundles in Section 5.2 and indeed throughout the book. For now, we'll focus our attention on the circle and set bundles over it.

[14] *homotopy types* have to wait until Appendix B.3

3.4 *The symmetries in the circle*

sec:symcirc

With the set Z of integers *defined* as in Section 3.2, we will now construct an equivalence between Z and the type $\bullet \xrightarrow{=}_{S^1} \bullet$, and that under this equivalence $0 : Z$ corresponds to $\mathrm{refl}_\bullet : \bullet \xrightarrow{=} \bullet$, and 1 to $\circlearrowleft$, and -1 to $\circlearrowleft^{-1}$. More generally, the successor $s : Z \to Z$ corresponds to composition with $\circlearrowleft$, while the predecessor s^{-1} corresponds to composition with $\circlearrowleft^{-1}$.

It follows directly that *addition* of integers corresponds to *composition* of loops.

The first step is to identify the exponential set bundle in Definition 3.3.12 with the universal set bundle in Example 3.3.9, i.e., identify the type family

$$R : S^1 \to \mathcal{U}, \qquad R(\bullet) :\equiv Z,\ R(\circlearrowleft) := \bar{s}$$

with the family

$$\mathbb{P}_\bullet : S^1 \to \mathcal{U}, \qquad \mathbb{P}_\bullet(z) :\equiv (\bullet \xrightarrow{=} z).$$

What does it mean to identify the families $\mathbb{P}_\bullet$ and R? Type families are a special case of functions. Function extensionality reduces the question to the pointwise identification of $\mathbb{P}_\bullet$ and R as functions. Using univalence, it suffices to give an equivalence from $\mathbb{P}_\bullet(z)$ to $R(z)$ for every $z : S^1$, that is, recalling Definition 2.14.1, giving a (fiberwise) equivalence $f : \mathbb{P}_\bullet \to R$. We will use Construction 2.9.9, so will also define $g : R \to \mathbb{P}_\bullet$.

rem:univalence-transparent

Remark 3.4.1. We recall Construction 2.14.2 defining how transport behaves in families of function types. Given a type A and two type families $P, Q : A \to \mathcal{U}$, transport along $p : a \xrightarrow{=} a'$ of $h : P(a) \to Q(a)$ can be identified with the function $\mathrm{trp}^Q_p \circ h \circ \mathrm{trp}^P_{p^{-1}}$ of type $P(a') \to Q(a')$. As a simplification we could use the notation $\tilde{_}$ introduced after **??** 2.13.2 for the transport functions. However, we now take the further step of allowing univalence to be completely transparent, that is, leaving out both $\tilde{_}$ and $\bar{_}$ when no confusion can occur. Here this means that the picture for the transport of h becomes:

$$\begin{array}{ccc} a & & P(a) \xrightarrow{\quad h \quad} Q(a) \\ \| \downarrow p & & \wr \downarrow P(p) \qquad\qquad \wr \downarrow Q(p) \\ a' & & P(a') \xrightarrow{Q(p)\, h\, P(p)^{-1}} Q(a'). \end{array}$$

In, for example, the definition of the exponential set bundle R above, this means that we may denote $R(\circlearrowleft)$ as s instead of $\bar{s}$, and may write $R(\circlearrowleft)(0) = 1$. ⌟

If A is S^1, then the induction principle for the circle says that giving an $h(z) : P(z) \to Q(z)$ for all $z : S^1$ is the same as specifying an element $h(\bullet) : P(\bullet) \to Q(\bullet)$ and, using Definition 2.7.3 and Remark 3.4.1, an identification $h(\circlearrowleft) : Q(\circlearrowleft)\, h(\bullet)\, P(\circlearrowleft)^{-1} \xrightarrow{=} h(\bullet)$, see the following diagram:

$$\begin{array}{ccc} P(\bullet) & \xrightarrow{h(\bullet)} & Q(\bullet) \\ \wr \downarrow P(\circlearrowleft) & & \wr \downarrow Q(\circlearrowleft) \\ P(\bullet) & \xrightarrow{h(\bullet)} & Q(\bullet). \end{array}$$

If P, Q are families of sets, then $h(\circlearrowleft)$ is a proof that this diagram commutes.

We now define $f : \mathbb{P}_\bullet \to R$ and $g : R \to \mathbb{P}_\bullet$ that will turn out to give inverse equivalences between $\mathbb{P}_\bullet(z)$ and $R(z)$, for each $z : S^1$.

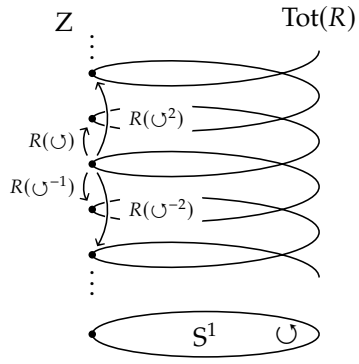


Figure 3.3: Transport in the family R

fig:transportalongloop

def:fptoR

Definition 3.4.2. The function $f : \prod_{z:S^1}(\mathbb{P}_\bullet(z) \to R(z))$ is defined by $f(z)(p) :\equiv R(p)(0)$. ⌟

In Figure 3.3, the function $f(\bullet)(p)$ above has been visualised for $p \equiv \circlearrowleft^n$, $n \equiv -2, -1, 0, 1, 2$.

lem:windingnumber

Lemma 3.4.3. *For f as in Definition 3.4.2 we have $f(\bullet)(\circlearrowleft^n) = n$ for all $n : \mathbb{Z}$.*

Proof. First consider positive $n : \mathbb{N}$ and apply induction. In the base case $n = 0$ we have $f(\bullet)(\circlearrowleft^0) \equiv f(\mathrm{refl}_\bullet) \equiv \mathrm{trp}^R_{\mathrm{refl}_\bullet}(0) \equiv 0$. For $n \equiv \mathrm{s}(m)$ with $m : \mathbb{N}$ we have

$$\begin{aligned}
f(\bullet)(\overset{\mathrm{s}(m)}{\circlearrowleft}) &\equiv R(\overset{\mathrm{s}(m)}{\circlearrowleft})(0) \\
&= R(\circlearrowleft \overset{m}{\circlearrowleft})(0) \\
&= R(\circlearrowleft)(R(\overset{m}{\circlearrowleft})(0)) && \text{since ap preserves composition} \\
&\equiv R(\circlearrowleft)(f(\bullet)(\overset{m}{\circlearrowleft})) \\
&= \mathrm{s}(f(\bullet)(\overset{m}{\circlearrowleft})) = \mathrm{s}(m) && \text{by the induction hypothesis.}
\end{aligned}$$

This completes the induction step for positive n. For negative n the proof is similar. □

In the definition of the second map, take into account that $R(\bullet) \equiv \mathbb{Z}$ and $\mathbb{P}_\bullet(\bullet) \equiv (\bullet \xrightarrow{=} \bullet)$.

def:gktop

Definition 3.4.4. The function $g : \prod_{z:S^1}(R(z) \to \mathbb{P}_\bullet(z))$ is defined by circle induction. We first define

$$g(\bullet) :\equiv \left(n \mapsto \overset{n}{\circlearrowleft}\right) : \mathbb{Z} \to (\bullet \xrightarrow{=} \bullet).$$

Then, using Remark 3.4.1, the type $g(\circlearrowleft)$ should be

$$\mathbb{P}_\bullet(\circlearrowleft)\, g(\bullet)\, R(\circlearrowleft)^{-1} \xrightarrow{=} g(\bullet).$$

By definition, $R(\circlearrowleft)$ is s. Using Exercise 2.14.4(2) we can identify $\mathbb{P}_\bullet(\circlearrowleft)$ with composition with $\circlearrowleft$. The element $g(\circlearrowleft)$ is obtained by function extensionality and a simple calculation, using the identification of $\circlearrowleft\, \circlearrowleft^{n-1}$ and $\circlearrowleft^n$ for any $n : \mathbb{Z}$. ⌟

The type of $g(\circlearrowleft)$ can be expressed by this diagram:

$$\begin{array}{ccc}
\mathbb{Z} & \xrightarrow{n \mapsto \circlearrowleft^n} & (\bullet \xrightarrow{=} \bullet) \\
\wr \downarrow \mathrm{s} & & \wr \downarrow p \mapsto \circlearrowleft \cdot p \\
\mathbb{Z} & \xrightarrow{n \mapsto \circlearrowleft^n} & (\bullet \xrightarrow{=} \bullet).
\end{array}$$

lem:univsexp

Theorem 3.4.5. *For every $z : S^1$, the functions $f(z)$ defined in Definition 3.4.2 and $g(z)$ in Definition 3.4.4 are inverse equivalences between $\mathbb{P}_\bullet(z)$ and $R(z)$.*

Proof. We apply Construction 2.9.9 and verify the two conditions. First, we need to give elements $H(z, p) : g(z)(f(z)(p)) \xrightarrow{=} p$ for all $z : S^1$ and $p : \mathbb{P}_\bullet(z) \equiv (\bullet \xrightarrow{=} z)$. By induction on $p : \bullet \xrightarrow{=} z$ it suffices to set $H(\bullet, \mathrm{refl}_\bullet) :\equiv \mathrm{refl}_{\mathrm{refl}_\bullet}$ since $g(\bullet)(f(\bullet)(\mathrm{refl}_\bullet)) \equiv g(\bullet)(0) \equiv \mathrm{refl}_\bullet$.

Secondly, we need to give elements $G(z)(n) : f(z)(g(z)(n)) = n$ for all $z : S^1$ and $n : R(z)$. By circle induction it suffices to define $G(\bullet)$ and $G(\circlearrowleft)$, but the type of $G(\bullet)$ is a proposition (as $\mathbb{Z}$ is a set), so the information for $G(\circlearrowleft)$ is redundant. Hence, it suffices to show that $f(\bullet)(g(\bullet)(n)) \equiv f(\bullet)(\circlearrowleft^n) = n$ for all $n : \mathbb{Z}$. This follows from Lemma 3.4.3. □

cor:Sigroupoid

Corollary 3.4.6. *The circle S^1 is a groupoid, and the function*

$$\circlearrowleft^- : \mathbb{Z} \to (\bullet \xrightarrow{=}_{S^1} \bullet)$$

sending n to $\circlearrowleft^n$ is an equivalence.

Proof. For any $z : S^1$, the type $\mathbb{P}_\bullet(z) \equiv (\bullet \xrightarrow{=}_{S^1} z)$ is a set since $R(z)$ is a set and $f(z) : \mathbb{P}_\bullet(z) \to R(z)$ an equivalence. Since the circle is connected and being a set is a proposition, it follows that $y \xrightarrow{=}_{S^1} z$ is a set, for any $y, z : S^1$. Hence S^1 is a groupoid. By Definition 3.4.4, $\circlearrowleft^- \equiv g(\bullet)$ is an equivalence. □

Recall the definition of universal set bundle from Definition 3.3.10. Now that we know that the circle is a groupoid we can harvest the following results.

cor:univ-covers-S1

Corollary 3.4.7. *The set bundle $\mathbb{P}_\bullet$ from Example 3.3.9 is universal. The exponential set bundle* exp *from Definition 3.3.12 is universal.*

Proof. By Lemma 3.3.11 and Theorem 3.4.5. □

def:windingnumber

Definition 3.4.8. The inverse equivalence $f(\bullet)$ of $g(\bullet) \equiv \circlearrowleft^- \equiv (n \mapsto \circlearrowleft^n)$ is called the *winding number function* $\mathrm{wdg} : (\bullet \xrightarrow{=} \bullet) \xrightarrow{\simeq} \mathbb{Z}$. ⌟

The following lemma is a simple example of a technique called *delooping*, which we will further elaborate in Section 7.5.

lem:S1-delooping

Lemma 3.4.9. *Let A be a connected type and $a : A$ an element. Assume we have an equivalence $e : (\bullet \xrightarrow{=} \bullet) \to (a \xrightarrow{=} a)$ of symmetries such that $e(\mathrm{refl}_\bullet) \xrightarrow{=} \mathrm{refl}_a$ and $e(p \cdot q) \xrightarrow{=} e(p) \cdot e(q)$, for all $p, q : (\bullet \xrightarrow{=} \bullet)$. Then $\check{e} : S^1 \to A$ defined by circle recursion by setting $\check{e}(\bullet) :\equiv a$ and $\check{e}(\circlearrowleft) := e(\circlearrowleft)$ is an equivalence.*

Proof. We have $\mathrm{ap}_{\check{e}} \xrightarrow{=} e$ since they produce equal values when applied to $\circlearrowleft^n$, for all $n : \mathbb{Z}$. Now use that A and S^1 are connected and apply Corollary 2.17.9(3). □

xca:general-winding

Exercise 3.4.10. Generalizing Definition 3.4.8, of winding numbers, use circle induction to define, for any point $x : S^1$ of the circle an equivalence, $\mathrm{wdg}_x : (x \xrightarrow{=} x) \xrightarrow{\simeq} \mathbb{Z}$. (You'll need commutativity of addition in $\mathbb{Z}$.) Conclude from Lemma 3.4.9 that we have equivalences $f_x : S^1 \xrightarrow{\simeq} S^1$ with $f_x(\bullet) \equiv x$, for each $x : S^1$.[15] ⌟

xca:S1=S1-components

Exercise 3.4.11. Let $-\mathrm{id}_{S^1} : S^1 \to S^1$ be defined by $-\mathrm{id}_{S^1}(\bullet) :\equiv \bullet$ and $-\mathrm{id}_{S^1}(\circlearrowleft) := \circlearrowleft^{-1}$. Show the $-\mathrm{id}_{S^1}$ and id_{S^1} are not in the same component of $S^1 \to S^1$. Prove the following proposition:

$$\prod_{t : S^1 \simeq S^1} \|\mathrm{id}_{S^1} \xrightarrow{=} t\| \amalg \|-\mathrm{id}_{S^1} \xrightarrow{=} t\|. \qquad ⌟$$

xca:(S1-~>S1)_(f)-equ-S1

Exercise 3.4.12. For any $f : S^1 \to S^1$, give an equivalence from S^1 to $(S^1 \to S^1)_{(f)}$, that is, from S^1 to the component of $S^1 \to S^1$ at f. Hint: use Lemma 3.4.9. ⌟

We note in passing that combining the above two exercises yields an equivalence from $(S^1 \xrightarrow{=} S^1)$ to $(S^1 \amalg S^1)$, that is, a characterization of the symmetries *of* the circle (in constrast to the title of this Section 3.4).

3.5 *A reinterpretation of the circle*

sec:S1isC

In this section we return to the equivalences in Theorem 3.3.8. We'll use these to get a different perspective on the circle, which highlights it as a type classifying very simple symmetries, namely sets with permutations. We have already seen one example in Definition 3.3.12, namely the set $\mathbb{Z}$ of integers together with the successor $\mathrm{s} : \mathbb{Z} \xrightarrow{\simeq} \mathbb{Z}$, defining the exponential

[15] If we think of the circle as represented by the unit length complex numbers, then $f_x(y)$ corresponds to the usual product xy. Alternatively, if we think of points on the circle as representing rotations of the unit circle in $\mathbb{R}^2$, then $f_x(y)$ corresponds to the composition of the rotations by x and y.

set bundle exp. By Corollary 3.4.7, exp and its friends $\mathbb{P}_\bullet : S^1 \to \mathrm{Set}$ and $\mathrm{cst}_\bullet : \mathbb{1} \to S^1$ are appearances of the universal set bundle over the circle.

The importance of exp will become apparent when we eventually explain that *the circle is equivalent to the connected component of* (Z, s) *in the type* $\sum_{X:\mathcal{U}}(X \to X)$.[16]

Recall from Theorem 3.3.8 the equivalence

$$gf : \mathrm{SetBundle}(S^1) \xrightarrow{\simeq} \sum_{X:\mathrm{Set}} (X \xrightarrow{\simeq} X).$$

When restricting to corresponding connected components, we get equivalences between these. So to understand the components of $\mathrm{SetBundle}(S^1)$ it suffices to understand the components of $\sum_{X:\mathrm{Set}}(X \xrightarrow{\simeq} X)$, which correspond to components of $\sum_{X:\mathcal{U}}(X \to X)$ at pairs (X, t), where X is a set with a permutation t.[17]

We are particularly interested in understanding the symmetries in these components, so before we prove that the circle is equivalent to the component containing (Z, s), let us investigate the equalities in the type $\sum_{X:\mathcal{U}}(X \to X)$ a bit further.

Define the type family D by $D(X) :\equiv (X \to X)$ for all $X : \mathcal{U}$. Recall that, given $X, Y : \mathcal{U}$ and $t : X \to X$ and $u : Y \to Y$, Lemma 2.10.3 and Definition 2.7.3 give an equivalence between the identity type $(X, t) \xrightarrow{=} (Y, u)$ and type of pairs consisting of a $p : X \xrightarrow{=} Y$ and an identification of type $\mathrm{trp}^D_p(t) \xrightarrow{=} u$. The transport on the left is precisely the special case described after Construction 2.14.2 (see diagram in the margin), so that the latter identity type type is equivalent to $\tilde{p} \circ t \circ \tilde{p}^{-1} \xrightarrow{=} u$. If $p \equiv \bar{e}$ for an equivalence $e : X \xrightarrow{\simeq} Y$, this is equivalent to $e \circ t \xrightarrow{=} u \circ e$, or $et \xrightarrow{=} ue$ for short. In total, we have an equivalence between the identity type $(X, t) \xrightarrow{=} (Y, u)$ and the sum type (see diagram in the margin)

$$\sum_{e : X \xrightarrow{\simeq} Y} et \xrightarrow{=}_{X \to Y} ue.$$

These types are sets whenever X and Y are, and then we may write $et = ue$.

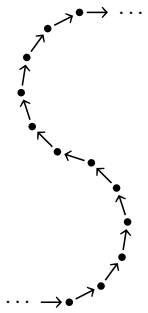
In particular, given a set X with a permutation t, we have an equivalence from $(Z, s) \xrightarrow{=} (X, t)$ to $\sum_{e : Z \xrightarrow{\simeq} X} e\,s = te$. See Figure 3.4 for an illustration. This equivalence is transparent in the sense that we never denote it. For example, any power s^n of s itself gives a symmetry $(s^n, p) : (Z, s) \xrightarrow{=} (Z, s)$, where p is a proof of $s^n\,s = s\,s^n$.

rem:bang

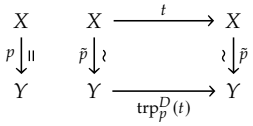
Remark 3.5.1. The type $s^n\,s = s\,s^n$ in the paragraph above is a proposition. Since all elements of a proposition are equal, it is often not necessary to name such elements explicitly. If the proposition in question is clear from the context, we may use ! as a default name of its elements. Be warned that different occurrences of ! may refer to elements of different propositions. In cases where the element of a proposition is not of interest (beyond its mere existence), we may even just ignore it. For example, again in the paragraph above, we may ignore p and consider s^n as a symmetry of (Z, s). (Note that we did already coerce the *function* s^n to the *equivalence*.) ⌟

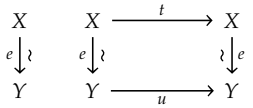
The following property jumps out at us when we contemplate Figure 3.4: the equivalence e is uniquely determined by the element $e(0) : X$. More precisely:

[16] The elements of this connected component can be thought of as *infinite cycles*: sets X with a successor function $t : X \to X$ such that (X, t) can be merely identified with (Z, s). That is, (X, t) looks exactly like (Z, s), but we don't know which element of X is "zero":



[17] Given a set X with a permutation t, we may coerce and view (X, t) as an element of $\sum_{X:\mathcal{U}}(X \to X)$. Then, for any (Y, u) in the same connected component of $\sum_{X:\mathcal{U}}(X \to X)$ as (X, t), we have that Y also is a set and u also a permutation of Y.





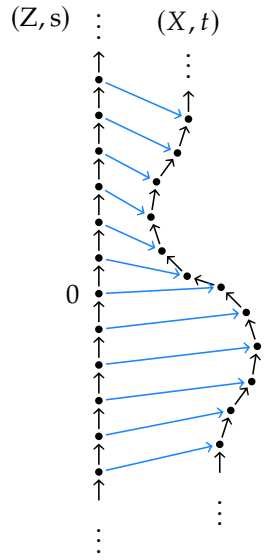


Figure 3.4: An identification of two infinite cycles. The equivalence $e : Z \xrightarrow{\simeq} X$ is marked in blue.

fig:Zs=Xt

lem:IdCiszet

Lemma 3.5.2. *For every (X, t) in the component of $\sum_{X:\mathcal{U}}(X \to X)$ containing (Z, s), the function*

$$\mathrm{ev}_0 : \big((Z, s) \xrightarrow{=} (X, t)\big) \to X \quad \textit{defined by} \quad \mathrm{ev}_0(e, !) :\equiv e(0)$$

is an equivalence.

Proof. We'll prove that every fiber of ev_0 is contractible. Given $x_0 : X$ we must determine a unique equivalence $e : Z \to X$ such that $es = te$ and $e(0) = x_0$. Induction on $n : Z$ (positive and negative n separately) shows that for such an e, we have $e(n) = t^n(x_0)$ for all $n : Z$. It remains to prove that $n \mapsto t^n(x_0)$ is an equivalence, for every $x_0 : Z$. Since we are proving a proposition, and we are assuming (X, t) is in the component of (Z, s), it suffices to prove it for $(X, t) \equiv (Z, s)$. Clearly, for any $x_0, n : Z$, we have $s^n(x_0) = n + x_0$, and the map $n \mapsto n + x_0$ is an equivalence, with inverse $n \mapsto n - x_0$. □

In particular, $\mathrm{ev}_0 : ((Z, s) \xrightarrow{=} (Z, s)) \to Z$ is an equivalence, mapping s^n to n for all $n : Z$. Cf. $\mathrm{wdg} : (\bullet \xrightarrow{=} \bullet) \to Z$ from Definition 3.4.8.

def:SItoC

Definition 3.5.3. Let InfCyc be the component of $\sum_{X:\mathcal{U}}(X \to X)$ containing (Z, s). Elements of InfCyc are called *infinite cycles*.[18]

Define by circle induction

$$c : S^1 \to \mathrm{InfCyc} \quad \text{setting} \quad c(\bullet) :\equiv (Z, s)$$

and $c(\circlearrowleft) : c(\bullet) \xrightarrow{=} c(\bullet)$ given by the *predecessor* equivalence $s^{-1} : (Z \to Z)$ and the trivial proof of the proposition $s^{-1}\, s = s\, s^{-1}$. ⌟

[18] See also Definition 3.6.3 below for general cycles.

As explained in Remark 3.5.1, we often leave out the propositional data pertaining to InfCyc (and other subtypes) from the notation.

The main result of this section is Theorem 3.5.6 below, stating that the function c from Definition 3.5.3 is an equivalence. Since it's such a crucial result, we are going to give two proofs. Each proof illuminates a different aspect and gives methods that will be used later.

For the first, we return to the equivalences of Theorem 3.3.8. As said above, these restrict to equivalences between corresponding components. In particular, $\mathrm{ev}_{\mathcal{U}} : (S^1 \to \mathcal{U}) \xrightarrow{\simeq} \sum_{X:\mathcal{U}}(X \xrightarrow{\simeq} X)$ maps the type family $\mathbb{P}_\bullet$ to the pair $(\bullet \xrightarrow{=} \bullet, \circlearrowleft \cdot _)$, which can be identified with (Z, s) through Corollary 3.4.6. Hence, $\mathrm{ev}_{\mathcal{U}}$ restricts to an equivalence between the connected component of $\mathbb{P}_\bullet$ in $S^1 \to \mathcal{U}$ and the connected component of (Z, s) in $\sum_{X:\mathcal{U}}(X \xrightarrow{\simeq} X)$.

Recall the constant maps $\mathrm{cst}_z : (\mathbb{1} \to S^1)$ for $z : S^1$. The equivalence preim maps $\mathrm{cst}_\bullet$ to $(x : S^1) \mapsto \sum_{_:\mathbb{1}}(x \xrightarrow{=} \bullet)$ which can be identified with $\mathbb{P}_\bullet$. Now consider the following diagram:

eq:setbundle-SC-univ-comp

$$\begin{array}{ccccc} & & S^1 & & \\ & {\scriptstyle(\mathbb{1},\mathrm{cst}__)}\swarrow & \downarrow{\scriptstyle \mathbb{P}__} & \searrow{\scriptstyle c} & \\ \mathrm{SetBundle}(S^1)_{(\mathbb{1},\mathrm{cst}_\bullet)} & \xrightarrow[\mathrm{preim}]{\sim} & (S^1 \to \mathcal{U})_{(\mathbb{P}_\bullet)} & \xrightarrow[\mathrm{ev}_{\mathcal{U}}]{\sim} & \mathrm{InfCyc} \end{array} \tag{3.5.1}$$

Both the left and the right triangle represent identity types. We have an identification for the left triangle because the fiber $\sum_{_:\mathbb{1}}(x \xrightarrow{=} z)$ of cst_z at $x : S^1$ can be identified with $\mathbb{P}_z(x) \equiv (z \xrightarrow{=} x)$, for any $z : S^1$. For the right triangle we apply circle induction to construct an element of

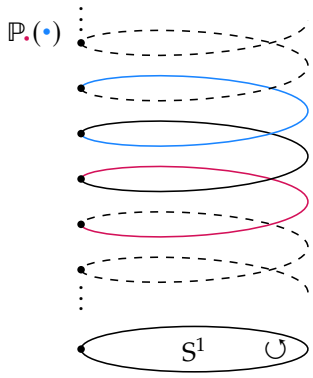


Figure 3.5: For the fiber of the universal set bundle, $\mathbb{P}_\bullet(\bullet) \equiv (\bullet = \bullet)$, we *increase* the winding number when we transport the endpoint (in blue) along $\circlearrowleft$, and we *decrease* it when we transport the starting point (in red) in the same way.

fig:plus-minus-one

$\prod_{z:S^1} c(z) \xrightarrow{=} \mathrm{ev}_{\mathcal{U}}(\mathbb{P}_z)$. The base case $z \equiv \bullet$ is exactly the abovementioned application of Corollary 3.4.6. For the loop case we observe that the following diagram commutes:

$$\begin{array}{ccc} Z & \xrightarrow{\circlearrowleft^-} & (\bullet \xrightarrow{=} \bullet) \\ \Big\downarrow{\scriptstyle s^{-1}} & & \Big\downarrow{\scriptstyle _\cdot\circlearrowleft^{-1}} \\ Z & \xrightarrow{\circlearrowleft^-} & (\bullet \xrightarrow{=} \bullet). \end{array}$$

Note that to transport in the family $\mathbb{P}_-(\bullet) \equiv (_ \xrightarrow{=} \bullet)$, we use Exercise 2.14.4(3), and *that* is why we picked the predecessor equivalence in Definition 3.5.3. This is also illustrated in Figure 3.5.[19]

With (3.5.1) in hand, we see that c is an equivalence if and only if either of the two other downward maps are.[20]

We now show that the map $(\mathbb{1}, \mathrm{cst}_)$ on the left is an equivalence. Since the codomain is connected, it suffices to show that the fiber at $(\mathbb{1}, \mathrm{cst}_\bullet)$ is contractible. This fiber is the sum type $\sum_{z:S^1}((\mathbb{1}, \mathrm{cst}_\bullet) \xrightarrow{=} (\mathbb{1}, \mathrm{cst}_z))$, where the identity type is by Lemma 2.10.3 equivalent to pairs of an equivalence $e : \mathbb{1} \to \mathbb{1}$ and elements of the identity type represented by the triangle

$$\begin{array}{ccc} \mathbb{1} & \xrightarrow{e} & \mathbb{1} \\ & {\scriptstyle \mathrm{cst}_\bullet} \searrow \quad \swarrow {\scriptstyle \mathrm{cst}_z} & \\ & S^1. & \end{array}$$

Since $\mathbb{1}$ is contractible, this just amounts to the identity type $\bullet \xrightarrow{=} z$, and $\sum_{z:S^1}(\bullet \xrightarrow{=} z)$ is indeed contractible.

xca:TYoneda

Exercise 3.5.4. This exercise is about results that go by the name "the type-theoretic Yoneda Lemma." See the book by Riehl[21] for the Yoneda lemma in category theory.

Let X be a type and $F : X \to \mathcal{U}$ a function. Use transport to give an equivalence e_x from the type $F(x)$ to the type $\prod_{y:X}((x \xrightarrow{=} y) \to F(y))$, for any $x : X$. The functions $(x \xrightarrow{=} y) \to F(y)$ thus obtained need not be equivalences, but sometimes they are.

Next, for $X : \mathcal{U}$, show that the map sending $x : X$ to $(y \mapsto (x \xrightarrow{=} y)) : X \to \mathcal{U}$ is an injection. Hint: use Lemma 2.17.8 and e_x above, for suitable F. In order to appreciate this hint, you can also directly prove that all fibers of the map are propositions. ⌟

We now give the second, more direct, proof that c is an equivalence. For this we use the following lemma, which is of independent interest.

lem:conn-eq-f-ap-f-x

Lemma 3.5.5. *Let X and Y be connected types, x an element of X, and f a function from X to Y. Then f is an equivalence if and only if* $\mathrm{ap}_f : (x \xrightarrow{=} x) \to (f(x) \xrightarrow{=} f(x))$ *is an equivalence.*

Proof. Using Corollary 2.17.9(3) it suffices to show that each map induced by f on identity types is an equivalence if and only if the specific map $\mathrm{ap}_f : (x \xrightarrow{=} x) \to (f(x) \xrightarrow{=} f(x))$ is an equivalence. Being an equivalence is a proposition, so the result follows in two easy steps from X being connected, using Exercise 2.16.9. □

thm:S1bysymmetries

Theorem 3.5.6. *The function $c : S^1 \to \mathrm{InfCyc}$ from Definition 3.5.3 is an equivalence.*

[19] Another option would have been to choose the opposite equivalence $Z \xrightarrow{=} \mathbb{P}_\bullet(\bullet)$, sending n to $\circlearrowleft^{-n}$, in the base case. The point is: You can move the minus sign around, but it has to pop up somewhere.

[20] At this point we could conclude with an appeal to Exercise 3.5.4, yielding that $\mathbb{P}_-$ is an equivalence.

[21] Emily Riehl. *Category Theory in Context*. Aurora: Modern Math Originals. Dover Publications, 2016. URL: https://math.jhu.edu/~eriehl/context/.

Proof. In view of Lemma 3.5.5 we only need to show that $\mathrm{ap}_c : (\bullet \xrightarrow{=} \bullet) \to ((\mathsf{Z}, \mathrm{s}) \xrightarrow{=} (\mathsf{Z}, \mathrm{s}))$ is an equivalence. Note that both the domain and the co-domain of ap_c have been identified with Z. Consider the following diagram in which we compose c with the equivalences from Corollary 3.4.6 and Lemma 3.5.2:

$$\mathsf{Z} \xrightarrow{\circlearrowleft^-} (\bullet \xrightarrow{=} \bullet) \xrightarrow{\mathrm{ap}_c} \big((\mathsf{Z}, \mathrm{s}) \xrightarrow{=} (\mathsf{Z}, \mathrm{s})\big) \xrightarrow{\mathrm{ev}_0} \mathsf{Z}$$

For c to be an equivalence, it suffices to show that the composition is an equivalence from Z to itself. By definition, $\mathrm{ap}_c(\circlearrowleft)$ is the identification corresponding to s^{-1}, sending 0 to -1, and by induction on $n : \mathsf{Z}$ it follows that $\mathrm{ev}_0(\mathrm{ap}_c(\circlearrowleft^n)) = \mathrm{s}^{-n}(0) = -n$. And the map $n \mapsto -n$ is indeed an equivalence. □

3.6 *Connected set bundles over the circle*

sec:conS1

Let A be a type and $f : A \to \mathsf{S}^1$ a function. By Corollary 2.17.9(1), f is a set bundle over S^1 if and only if each map induced by f on identity types is injective. Assume that $f : A \to \mathsf{S}^1$ is a set bundle with A connected. Let a_0 be an element of A. By Exercise 2.16.9 the condition that *each* ap_f is injective can be relaxed to $\mathrm{ap}_f : (a_0 \xrightarrow{=} a_0) \to (f(a_0) \xrightarrow{=} f(a_0))$ being injective. Now look at the following diagram, with wdg the winding number function from Exercise 3.4.10 and $\circlearrowleft^-$ from Corollary 3.4.6:

By Exercise 3.3.3(6) A is a groupoid.

eq:subgroup-covering-1

$$(a_0 \xrightarrow{=}_A a_0) \xhookrightarrow[\mathrm{ap}_f]{} (f(a_0) \xrightarrow{=}_{\mathsf{S}^1} f(a_0)) \xrightarrow[\mathrm{wdg}_{f(a_0)}]{\sim} \mathsf{Z} \xrightarrow[\circlearrowleft^-]{\sim} (\bullet \xrightarrow{=} \bullet) \tag{3.6.1}$$

Define the composite $g_f :\equiv \mathrm{wdg}_{f(a_0)} \circ \mathrm{ap}_f$, and consider its image, which is a subset of the integers. Clearly, g_f is an injection, so that its fibers are propositions, and the image is the subset $\sum_{n:\mathsf{Z}} g_f^{-1}(n)$. Obviously, a classification of connected set bundles over the circle also classifies certain subsets of Z, or, equivalently, certain subsets of symmetries of $\bullet$. Such subsets of Z are closed under addition and negation, and those of $(\bullet \xrightarrow{=} \bullet)$ are closed under concatenation and inverses, since ap_f, wdg and $\circlearrowleft^-$ are compatible with these operations. Using language to be introduced in Chapter 9, we actually "classify the subgroups of the integers".

Since A is connected, the proposition $g_f^{-1}(n)$ does not depend on the choice of a_0, so the subset only depends on f.

For subgroups in general, in Chapter 9, the setbundle f is pointed, and has a pointing path $p : \mathrm{pt}_B \xrightarrow{=} f(\mathrm{pt}_A)$. Then ap_f is composed with $p^{-1} _ p$, conjugation. See also Definition 4.4.3.

Recall that set bundles over the circle are equivalent to sets with permutations. Which sets with permutations (X, t) correspond to connected set bundles? It is not so surprising that the answer has to do with whether any two points $x, x' : X$ can be connected by applying t some number of times.

def:connected-by-t

Definition 3.6.1. Let X be a set with a permutation t. Elements $x, x' : X$ such that $x' = t^n(x)$ for some $n : \mathsf{Z}$ are said to be *connected* by t, denoted $x \sim x'$ whenever t is clear from the context. The relation $\sim$ is an equivalence relation. (Exercise: Check this.) ⌟

Recall that the iteration t^n makes sense for all integers n since t is an equivalence.

Recall Figure 3.2. We now have all the tools to analyze the difference between the left and the right picture in full generality.

con:cycset-connS1cover

Construction 3.6.2. *Let X be a set with a permutation t, defining the equivalence relation $\sim$ as in Definition 3.6.1. The set bundle over the circle corresponding to (X, t) in Theorem 3.3.8 is the pair $(\sum_{z:\mathsf{S}^1} E(z), \mathrm{fst})$ where*

$E :\equiv \mathrm{ve}_{\mathcal{U}}(X, \bar{t}) : \mathbb{S}^1 \to \mathcal{U}$, with ve *defined by circle induction in Theorem 3.1.2. Then we have a bijection between $\|\sum_{z:\mathbb{S}^1} E(z)\|_0$ and the quotient $X/\!\sim$ as defined in Definition 2.22.11.*

Implementation of Construction 3.6.2. Abbreviate $\sum_{z:\mathbb{S}^1} E(z)$ by A. We define a map $g : \|A\|_0 \to X/\!\sim$, from the set of components of A to the quotient set of X using the universal property of set truncation (Definition 2.22.4), pair induction, and circle induction. To define $g_0 : \prod_{z:\mathbb{S}^1}(E(z) \to X/\!\sim)$, we put $g_0(\bullet) :\equiv [_] : X \to X/\!\sim$ and need $g_0(\circlearrowleft) : g_0(\bullet) \xrightarrow[\circlearrowleft]{=} g_0(\bullet)$, equivalent to $g_0(\bullet) \xrightarrow{=}_{X \to X/\sim} g_0(\bullet)t$. The latter we get by function extensionality and Theorem 2.22.13, since $x \sim t(x)$ for any $x : X$.

The inverse of $h : (X/\!\sim) \to \|A\|_0$ of g is defined as the extension of $h_0 : X \to \|A\|_0$ with $h_0(x) :\equiv |(\bullet, x)|_0$. We just need to check that $h_0(x) = h_0(x')$, or equivalently, $\|(\bullet, x) \xrightarrow{=}_A (\bullet, x')\|$, whenever $x \sim x'$. Since this is a proposition, if $x' = t^n(x)$ with $n : \mathbb{Z}$, we may use induction on n (positive and negative) together with the paths, $\overline{(\circlearrowleft, \mathrm{refl}_{t(x)})} : (\bullet, x) \xrightarrow{=}_A (\bullet, t(x))$, to conclude.

It's easy to check that g and h are mutually inverse. □

In Figure 3.6 we see the set bundle corresponding to the set $\{1, 2, 3, 4, 5\}$ with the permutation $1 \mapsto 2 \mapsto 3 \mapsto 1, 4 \mapsto 5 \mapsto 4$. There are two components, showing that the permutation splits into two cycles.

def:Cyc

Definition 3.6.3. Let Cyc be the subtype of $\sum_{X:\mathcal{U}}(X \to X)$ of those pairs (X, t) where X is a *nonempty* set with an *equivalence* t and any $x, x' : X$ are connected by t. Expressed in a formula:

$$\mathrm{Cyc} :\equiv \sum_{X:\mathrm{Set}} \sum_{t:X \xrightarrow{\simeq} X} \left(\|X\| \times \prod_{x,x':X} \exists_{n:\mathbb{Z}} (x' = t^n(x))\right).$$

Elements of Cyc are called *cycles*.[22] ⌟

thm:cycset-comS1cover

Corollary 3.6.4. *Under the equivalence described in Construction 3.6.2, connected set bundles over the circle correspond to cycles.*

Proof. We use the notations of the implementation of Construction 3.6.2. If A is connected, then $\|A\|_0$ is contractible and hence also $X/\!\sim$ is contractible, so (X, t) is a cycle.

Conversely, if (X, t) is a cycle, then $X/\!\sim$ is contractible and hence also $\|A\|_0$ is contractible, so A is connected. □

We already know some connected set bundles over the circle, namely the universal set bundle, which is also represented by the constant map $\mathrm{cst}_\bullet : \mathbb{1} \to \mathbb{S}^1$, and which we showed is equal to the exponential set bundle, which in turn corresponds to the infinite cycle $(\mathbb{Z}, \mathrm{s})$ consisting of the set of integers $\mathbb{Z}$ with the successor permutation. Another example is the left one of the two examples given in Figure 3.2.

We now introduce the remaining set bundles over the circle, first as functions to the circle, then as families of sets. Eventually we'll show – assuming a weak form of the Law of the Excluded Middle – that these (with the universal set bundle) are all the decidable connected set bundles over the circle.

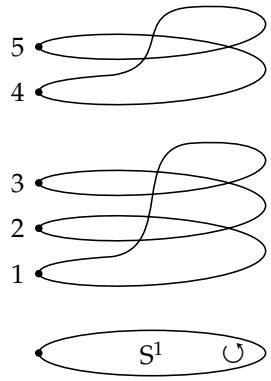


Figure 3.6: A set bundle with two components.

fig:two-comp-S1-cover

[22] Our cycles are a special case of what is elsewhere called *cyclically ordered sets*, and they are closely related to the *cyclic sets* of Connes[23].

[23] Alain Connes. "Cohomologie cyclique et foncteurs Ext^n". In: *C. R. Acad. Sci. Paris Sér. I Math.* 296.23 (1983), pp. 953–958.

def:dgm-map

Definition 3.6.5. For $m : \mathbb{N}$ positive, define the *degree m function* by circle induction

$$\delta_m : S^1 \to S^1, \text{ setting } \delta_m(\bullet) :\equiv \bullet \text{ and } \delta_m(\circlearrowleft) := \circlearrowleft^m. \qquad \lrcorner$$

On loops, the degree m function is the map $(_)^m : (\bullet \xrightarrow{=} \bullet) \to (\bullet \xrightarrow{=} \bullet)$, which is indeed an injection for positive m, so δ_m is a set bundle corresponding to the subset of $(\bullet \xrightarrow{=} \bullet)$ consisting of $\circlearrowleft^{mn} : \bullet \xrightarrow{=} \bullet$ for all $n : \mathbb{Z}$.

Note that $(_)^0 : (\bullet \xrightarrow{=} \bullet) \to (\bullet \xrightarrow{=} \bullet)$ is constant and hence not injective.

As a subset of $\mathbb{Z}$, this is simply all multiples of m.

In Section 3.4 we gained a lot of insight into the universal set bundle, $\mathrm{cst.} : \mathbb{1} \to S^1$, by constructing an equivalence with the exponential set bundle, see Theorem 3.4.5. In this section, we'll learn more about the degree m map, $\delta_m : S^1 \to S^1$, by constructing an equivalence with another concrete family.

Fix a positive number $m : \mathbb{N}$. Recall the finite set $\mathbb{m}$ from Definition 2.24.1 with elements denoted $0, 1, \ldots, m-1$, as well as the equivalence of type $\mathbb{m} \xrightarrow{\simeq} \sum_{k:\mathbb{N}} k < m$ from Exercise 2.24.2. Hence we may define a successor map $\mathrm{s} : \mathbb{m} \to \mathbb{m}$ by

$$\mathrm{s}(k) :\equiv \begin{cases} k+1 & \text{if } k < m-1, \\ 0 & \text{if } k = m-1. \end{cases}$$

Exercise 3.6.6. Show that $\mathrm{s} : \mathbb{m} \to \mathbb{m}$ is an equivalence by defining an explicit inverse. $\lrcorner$

Thus, $(\mathbb{m}, \mathrm{s})$ is another key example of a cycle called the *standard finite m-element cycle*. As seen in Theorem 3.3.8, any cycle corresponds to a set bundle over S^1. Just as the set bundle R in Definition 3.3.12 corresponds to the standard infinite cycle $(\mathbb{Z}, \mathrm{s})$, we will now define the set bundle R_m corresponding to standard finite m-element cycle.

def:RmtoS1

Definition 3.6.7. Fix $m : \mathbb{N}$ positive. Define the (m-fold) set bundle $R_m : S^1 \to \mathrm{Set}$ by $R_m(\bullet) :\equiv \mathbb{m}$ and $R_m(\circlearrowleft) := \bar{\mathrm{s}}$. Recall $\mathrm{Tot}(R_m) \equiv \sum_{z:S^1} R_m(z)$ and point $\mathrm{Tot}(R_m)$ at $(\bullet, 0)$. Now define

$$\mathrm{pow}_m :\equiv \mathrm{fst} : \mathrm{Tot}(R_m) \to S^1 \text{ with pointing path } \mathrm{refl}_\bullet.$$

We call pow_m the *m^{th} power bundle of the circle*.

$\lrcorner$

rem:finite-cyclic-subgroups

Remark 3.6.8. The analogue of our degree m function is the mth power of complex numbers restricted to the unit circle, mapping z to z^m if $|z| = 1$. If we parameterize the unit circle by the angle $\theta : \mathbb{R}$ (defined up to multiples of 2π), so $z = e^{\theta \mathrm{i}}$, then $z^m = e^{m\theta \mathrm{i}}$. Figure 3.7 illustrates the mth power bundle over the circle. Choosing any point z on the unit circle, we see that the preimage of z under the mth power map is a shifted copy of the m different m^{th} roots of unity inside the unit circle. $\lrcorner$

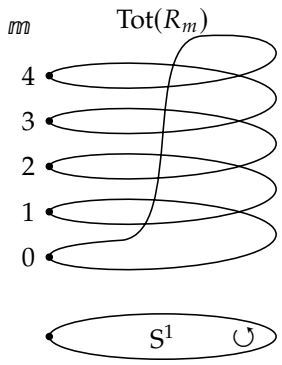


Figure 3.7: The mth power bundle for $m = 5$.

To identify δ_m and pow_m as set bundles over S^1, it suffices to define an equivalence $\psi_m : \mathrm{Tot}(R_m) \to S^1$ and an identification α_m of the identity type $\delta_m \psi_m \xrightarrow{=} \mathrm{pow}_m$ represented by the triangle below.

$$\begin{array}{ccc} \mathrm{Tot}(R_m) & \xrightarrow{\psi_m} & S^1 \\ & {\scriptstyle \mathrm{pow}_m} \searrow \quad \swarrow {\scriptstyle \delta_m} & \\ & S^1 & \end{array}$$

To see how to define ψ_m and α_m, we draw in Figure 3.8 the type $\mathrm{Tot}(R_m)$ unrolled into a "clock", with marks $0, 1, \dots, m-1$ (the mark k is the element $(\bullet, k) : \mathrm{Tot}(R_m)$), and arcs following the successor permutation of $\mathbb{m}$. We denote these arcs by $a_k :\equiv \overline{(\circlearrowleft, \mathrm{refl}_{\mathrm{s}(k)})} : (\bullet, k) \xrightarrow{=} (\bullet, \mathrm{s}(k))$. The mth power map (which is just the first projection) sends each mark to $\bullet : \mathrm{S}^1$ and each arc to $\circlearrowleft$.

This is indicated in blue on the inside of the clock. To define ψ_m, we must send all the marks to $\bullet : \mathrm{S}^1$ and all arcs to refl$\bullet$, except one, which goes to $\circlearrowleft$. This is indicated in red on the outside of the clock.

Construction 3.6.9. *For each positive integer m, there is an equivalence $\psi_m : \mathrm{Tot}(R_m) \to \mathrm{S}^1$ and an element $\alpha_m : \delta_m \psi_m \xrightarrow{=} \mathrm{pow}_m$.*

Implementation of Construction 3.6.9. Since $\mathrm{Tot}(R_m) \equiv \sum_{z:\mathrm{S}^1} R_m(z)$, to define ψ_m we first split the argument into a pair (z, k). In order to facilitate circle induction we consider ψ_m as an element of the type $\prod_{z:\mathrm{S}^1}(R_m(z) \to \mathrm{S}^1)$. We define $\psi_m(z) : R_m(z) \to \mathrm{S}^1$ by circle induction on z. The base case is $\psi_m(\bullet) :\equiv \mathrm{cst}_\bullet : \mathbb{m} \to \mathrm{S}^1$, the constant function at $\bullet$ (recall $R_m(\bullet) \equiv \mathbb{m}$). Since transport in a function type is by conjugation (Construction 2.14.2), and the codomain type is constant, we need to give an identification $\psi_m(\circlearrowleft)$ of type $\psi_m(\bullet) \xrightarrow{=}_{\mathbb{m}\to\mathrm{S}^1} \psi_m(\bullet) R_m(\circlearrowleft)$. We construct $\psi_m(\circlearrowleft)$ using function extensionality, by giving an element in $\mathbb{m} \to (\bullet \xrightarrow{=} \bullet)$. Since ψ_m needs to send all arcs, except the last, in $\mathrm{Tot}(R_m)$ to reflexivity, we map k to refl$\bullet$ for $k < m-1$, and we map $m-1$ to $\circlearrowleft$.

The inverse of ψ_m maps $\bullet$ to $(\bullet, 0)$, i.e., the mark at 0, and $\circlearrowleft$ to $a_{m-1} \cdots a_0$, i.e., the product of all the arcs around the circle. We leave it as an exercise to prove that this really defines an inverse to ψ_m.

We likewise use function extensionality and pair and circle induction to define α, reducing the problem to giving (with a slight abuse of notation) $\alpha_m(\bullet, k) : \mathrm{pow}_m(\bullet, k) \xrightarrow{=} \delta_m(\psi_m(\bullet, k))$ together with elements $\alpha_m(\circlearrowleft, k)$ witnessing that the two composites agree in the square

$$\begin{array}{ccc} \mathrm{pow}_m(\bullet, k) & \xrightarrow[=]{\alpha_m(\bullet,k)} & \delta_m(\psi_m(\bullet, k)) \\ {\scriptstyle \mathrm{pow}_m(a_k)}\downarrow{\scriptstyle =} & & {\scriptstyle =}\downarrow{\scriptstyle \delta_m(\psi_m(a_k))} \\ \mathrm{pow}_m(\bullet, \mathrm{s}(k)) & \xrightarrow[=]{\alpha_m(\bullet,\mathrm{s}(k))} & \delta_m(\psi_m(\bullet, \mathrm{s}(k))). \end{array}$$

In Figure 3.9 we show these m squares with the left and right hand sides simplified according to the definitions.

We see that we can pick $\alpha_m(\bullet, k) :\equiv \circlearrowleft^{-k}$, and then we can take for $\alpha_m(\circlearrowleft, k)$ the trivial proofs that $\mathrm{refl}_\bullet \circlearrowleft^{-k} = \circlearrowleft^{-(k+1)} \circlearrowleft$, for $k < m-1$, and $\circlearrowleft^m \circlearrowleft^{-(m-1)} = \circlearrowleft^{-0} \circlearrowleft$, for $k = m-1$. □

In Figure 3.10, which is an adaptation of Figure 3.8, we illustrate the last part of the above construction in the case $m = 5$.

The labels on the inner arcs show $\mathrm{pow}_5(\circlearrowleft, k)$, on the outer arcs $\delta_5\psi_5(\circlearrowleft, k)$, and on the radiuses $\alpha_m(\bullet, k)$. The proofs $\alpha_m(\circlearrowleft, k)$ prove the commutativity of the five squares in circular arrangement.

Corollary 3.6.10. *The degree m map $\delta_m : \mathrm{S}^1 \to \mathrm{S}^1$ is a connected set bundle for each positive integer m, and all the preimages $\delta_m^{-1}(z)$, $z : \mathrm{S}^1$, are m-element finite sets.*

We get an explicit equivalence $\mathbb{m} \simeq \delta_m^{-1}(\bullet)$ from ψ_m and α_m: send k to $(\bullet, \circlearrowleft^{-k})$, using the following exercise.

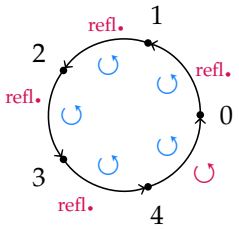


Figure 3.8: Unrolling $\mathrm{Tot}(R_5)$ as a "clock". (Here we're going around in a counterclockwise fashion as mathematicians are wont to do.)

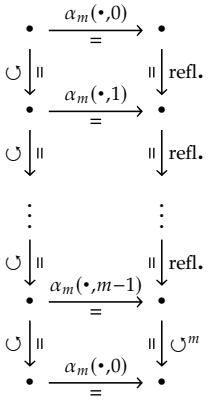


Figure 3.9: The simplified types of the squares $\alpha_m(\circlearrowleft, k)$.

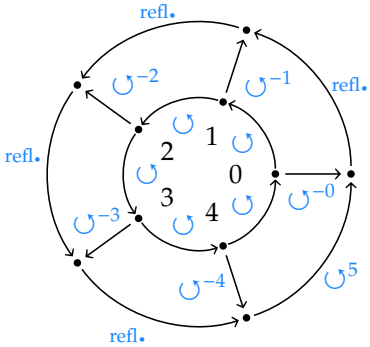
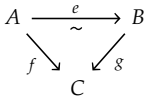


Figure 3.10: The proof for the case $m = 5$ around the clock.

con:psi-alpha-m
fig:m-power-clock
fig:psi-alpha-m-help
cor:dgm-connov
fig:psi-alpha-5-clock

xca:preim-eq

EXERCISE 3.6.11. Let A, B, C be types and $f : A \to C$, $g : B \to C$ functions. Assume moreover we have an equivalence $e : A \to B$, an element $h : \prod_{x:A} f(x) \xrightarrow{=} g(e(x))$, and an element $c : C$. Show that $(a, p) \mapsto (e(a), h(a)p)$ defines an equivalence $f^{-1}(c) \to g^{-1}(c)$. ⌟



Recall that our goal is to understand the *type* of connected set bundles over the circle. Since the type of set bundles is equivalent to $S^1 \to \mathrm{Set}$, and Set is a groupoid (Lemma 2.22.1), Lemma 2.15.5(1) gives that the type of set bundles over the circle is a groupoid. We will pin this groupoid down by first analyzing the sets of identifications in it.

To do this, we generalize Lemma 3.5.2 to other kinds of cycles. However, since we're dealing with multiple components, it'll be useful to have a set labeling the components first.

def:subgroup-set-of-cycle

DEFINITION 3.6.12. For any cycle (X, t), let $H_t :\equiv \{ n : \mathbb{Z} \mid t^n = \mathrm{id} \} : \mathrm{Sub}(\mathbb{Z})$. ⌟

Thus, H_t is the subset of Z determined by the predicate $t^n = \mathrm{id}$ for $n : \mathbb{Z}$. Recall that $\mathrm{Sub}(\mathbb{Z}) \equiv (\mathbb{Z} \to \mathrm{Prop})$ is a set.

lem:cycle-order-point-ap

LEMMA 3.6.13. *Let (A, f) be a connected set bundle over the circle with corresponding cycle (X, t) according to Corollary 3.6.4. For any $x : X$ we have $H_t = \{ n : \mathbb{Z} \mid t^n(x) = x \}$, and for any $a : A$, we have that H_t also equals the image of the composite*

eq:order-composite

$$(a \xrightarrow{=}_A a) \xrightarrow{\mathrm{ap}_f} (f(a) \xrightarrow{=}_{S^1} f(a)) \xrightarrow{\simeq} \mathbb{Z}, \tag{3.6.2}$$

where the second map is the winding number function from Exercise 3.4.10.

Proof. We may suppose that the set bundle (A, f) over the circle has the form $(\sum_{z:S^1} E(z), \mathrm{fst})$, where $E \equiv \mathrm{ve}_{\mathcal{U}}(X, \bar{t}) : S^1 \to \mathcal{U}$ is the family corresponding to the cycle (X, t). To prove the proposition in the lemma quantifying over A, i.e., over $z : S^1$ and $x : E(z)$, it suffices to consider the case $z \equiv \bullet$ and $x : X$, since the circle is connected.

For any point $x : X$, corresponding to the point $a :\equiv (\bullet, x) : A$, the type $(a \xrightarrow{=}_A a)$ is equivalent to $\sum_{n:\mathbb{Z}} t^n(x) = x$ in such a way that the composite function (3.6.2) corresponds to the first projection. Hence the image of (3.6.2) is precisely $\{ n : \mathbb{Z} \mid t^n(x) = x \}$.

It remains to show that $\{ n : \mathbb{Z} \mid t^n(x) = x \} \subseteq H_t$ (the other inclusion being clear). So assume $t^n(x) = x$. Then if $x' : X$ is any other point, to prove the proposition $t^n(x') = x'$, we may assume we have $k : \mathbb{Z}$ with $x' = t^k(x)$. Then $t^n(x') = t^{n+k}(x) = t^k(x) = x'$, as desired. □

lem:idCycle

LEMMA 3.6.14. *Let (X, t) and (Y, u) be cycles. The following propositions are equivalent*:

nr:TruncCtYu

(1) $\|(X, t) \xrightarrow{=} (Y, u)\|$;

nr:HSubZYu

(2) $H_t =_{\mathrm{Sub}(\mathbb{Z})} H_u$;

nr:eXtuPtd

(3) *For all $x_0 : X$, $y_0 : Y$, the type $\sum_{e:(X,t)\xrightarrow{=}(Y,u)} e(x_0) = y_0$ is contractible.*

Proof. Proving (2) from (1) is easy, since (2) is a proposition.

Assume (2), i.e., for any $x : X$, $y : Y$ and $n : \mathbb{Z}$, $t^n(x) = x$ if and only if $u^n(y) = y$. In order to prove (3), let $x_0 : X$ and $y_0 : Y$. We must determine a unique equivalence $e : X \to Y$ such that $et = ue$ and $e(x_0) = y_0$.

A necessary condition that e has to fulfill is the following. For any $x : X$ and $n : \mathbb{Z}$ with $x = t^n(x_0)$, we must have

$$e(x) = e(t^n(x_0)) = u^n(e(x_0)) = u^n(y_0).$$

This shows uniqueness of e once its existence has been established. For showing existence, we use that for any $x : X$ there exists an $n : \mathbb{Z}$ with $x = t^n(x_0)$ and that $u^n(y_0)$ is independent of such n. Technically, to use the proposition $\exists_{n:\mathbb{Z}}(x = t^n(x_0))$ to construct $e(x) : Y$, we prove instead that the type $P_x :\equiv \sum_{y:Y} \prod_{n:\mathbb{Z}}((x = t^n(x_0)) \to (y = u^n(y_0)))$ is contractible, and define $e(x)$ to be its center. Note that P_x is a subtype of Y (the product part is a proposition since Y is a set).

Let $x : X$. Since being contractible is a proposition we may assume a $m : \mathbb{Z}$ with $x = t^m(x_0)$. As center of P_x we take $y = u^m(y_0)$. We need to show, for any $n : \mathbb{Z}$, that $x = t^m(x_0) = t^n(x_0)$ implies $u^m(y_0) = u^n(y_0)$. But this follows from our starting assumption, since the former is equivalent to $t^{m-n}(x_0) = x_0$ and the latter to $u^{m-n}(y_0) = y_0$. Note that we also get $e(x_0) = y_0$ as the center of P_{x_0}. We still need to show that any two y, y' in P_x are equal. But this is clear, since $x = t^m(x_0)$, so $y = u^m(y_0) = y'$. It's easy to prove the proposition that this e is indeed an equivalence, so this is left to the reader.

Finally, we prove that (1) follows from (3). This is almost immediate: since (1) is a proposition we may assume $x_0 : X$ and $y_0 : Y$ and use the center of contraction. □

The following corollary of Lemma 3.6.14 (3) generalizes Lemma 3.5.2.

cor:commCycles

Corollary 3.6.15. *Let $(X, t), (Y, u) : \mathrm{Cyc}$ and let $x_0 : X$. If any of (1)–(3) in Lemma 3.6.14 is true, then the function*

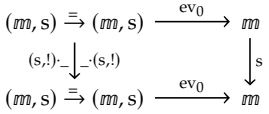
$$\mathrm{ev}_0 : \big((X, t) \xrightarrow{=} (Y, u)\big) \to Y \quad \textit{defined by} \quad \mathrm{ev}_0(e, !) :\equiv e(x_0)$$

is an equivalence.

As a second consequence, we get the following for the type of loops at the standard m-cycle.

cor:id-m-cycle

Corollary 3.6.16. *For cycles (m, s), evaluation at $0 : m$ gives an equivalence $\big((m, \mathrm{s}) \xrightarrow{=} (m, \mathrm{s})\big) \xrightarrow{\simeq} m$ with $\mathrm{ev}_0(\mathrm{refl}_{(m,\mathrm{s})}) = 0$, and composition with the identification $(\mathrm{s}, !) : (m, \mathrm{s}) \xrightarrow{=} (m, \mathrm{s})$ corresponds to the operation $\mathrm{s} : m \to m$, that is, the diagram in the margin commutes.*



Remark 3.6.17. In Corollary 3.6.16, the equivalence $\mathrm{s} : m \to m$ is not. uniquely determined by the stated property. Its inverse would give the same result for any m (even for $\mathbb{Z}$). In fact there are as many as there are positive integers less than m that are relatively prime to m. This behavior has number theoretic consequences and origins and will be investigated further when we have the proper machinery to put it to good use. ⌟

sec:generatorsofmodulararithmetic1

And as a third consequence, we get a more concrete description of the set of components of Cyc, and hence, by Corollary 3.6.4, of the type of connected set bundles over the circle.

cor:set-trunc-cyc

Corollary 3.6.18. *Let $H_{_} : \mathrm{Cyc} \to \mathrm{Sub}(\mathbb{Z})$ be the map sending (X, t) to H_t. Then the image of $H_{_}$ is equal to the the subset of $\mathrm{Sub}(\mathbb{Z})$ consisting of those $H \subseteq \mathbb{Z}$ that contain 0 and are closed under addition and negation.*[24]

[24] By $H \subseteq \mathbb{Z}$ being closed under addition and negation, we simply mean that if z, z' are in H, then so are $z + z'$ and $-z$.

Proof. Recall $\mathrm{im}(H_) \equiv \sum_{H:\mathrm{Sub}(\mathbb{Z})} \exists_{(X,t):\mathrm{Cyc}}(H = H_t)$. Let $H : \mathrm{Sub}(\mathbb{Z})$. We have to prove that $\exists_{(X,t):\mathrm{Cyc}}(H = H_t)$ if and only if $H \subseteq \mathbb{Z}$ contains 0 and is closed under addition and negation. Assume $\exists_{(X,t):\mathrm{Cyc}}(H = H_t)$. Since we have to prove a proposition, we may assume we have a cycle (X, t) with $H = H_t$. Now the required properties of H follow immediately from the definition of $H_t \equiv \{n : \mathbb{Z} \mid t^n = \mathrm{id}\}$.

Conversely, suppose $H \subseteq \mathbb{Z}$ contains 0 and is closed under addition and negation. Define the relation $\sim_H$ on $\mathbb{Z}$ by setting $z \sim_H z'$ if and only if the difference $z - z'$ is in H. This is an equivalence relation: it is reflexive since H contains 0, transitive since H is closed under addition, and symmetric since H is closed under negation. So let $X :\equiv \mathbb{Z}/\sim_H$, and define $t([z]) :\equiv [\mathrm{s}(z)]$ for $z : \mathbb{Z}$. This is well defined, since $z \sim_H z'$ holds if and only if $\mathrm{s}(z) \sim_H \mathrm{s}(z')$. It is clear that (X, t) is a cycle with $H_t = H$. □

xca:map-of-cycles

Exercise 3.6.19. Let (X, t) and (Y, u) be cycles, and $f : X \to Y$ a map such that $uf = ft$. Show: (i) $H_t \subseteq H_u$; (ii) f is surjective; (iii) if $H_u \subseteq H_t$ then f is also injective. ⌟

The components of Cyc will pop up many times from now on, so we make the following definitions to make it easier to talk about them.

def:Order

Definition 3.6.20. The type of *orders* is defined to be $\mathrm{Order} :\equiv \|\mathrm{Cyc}\|_0$. We say that the infinite cycle $(\mathbb{Z}, \mathrm{s})$ has *infinite order*, and the standard m-cycle $(\mathbb{m}, \mathrm{s})$ has *finite order* m, for positive $m : \mathbb{N}$.

We write $\mathrm{ord} :\equiv |_|_0 : \mathrm{Cyc} \to \mathrm{Order}$ for the map from cycles to their orders, and we write $\mathrm{ord}(t) :\equiv \mathrm{ord}(X, t)$ for short.

We say that the order $d :\equiv \mathrm{ord}(X, t)$ *divides* the order $k :\equiv \mathrm{ord}(Y, u)$, written $d|k$, for cycles $(X, t), (Y, u)$, if $H_u \subseteq H_t$. ⌟

Note that we're still being cavalier with universe levels. Really, we should write $\mathrm{SetBundle}(S^1)_{\mathcal{U}}$, $\mathrm{Cyc}_{\mathcal{U}}$, $\mathrm{Order}_{\mathcal{U}}$, etc., to indicate from which universe $\mathcal{U}$ we draw the types involved. We trust that the reader can fill these in if desired.

We have a canonical injection $\mathbb{N} \hookrightarrow \mathrm{Order}$, mapping 0 to the infinite order and each positive n to the finite order n. The orders in the image are called *principal*, and we don't make any notational distinction between a natural number d and the corresponding principal order. As a subset of $\mathbb{Z}$ according to Corollary 3.6.18, a principal order is simply $d\mathbb{Z}$, so we see that the divisibility relation on orders extends that on natural numbers.

From the proof of Corollary 3.6.18 we get a map $\mathrm{Order} \to \mathrm{Cyc}$, mapping the corresponding subset $H : \mathrm{Sub}(\mathbb{Z})$, containing 0 and closed under addition and negation, to the cycle $(\mathbb{Z}/\sim_H, \mathrm{s})$. This generalizes the definition of the standard cycles from principal orders to all orders.

def:standard-cycle

Definition 3.6.21. Given an order $d : \mathrm{Order}$, we call $(\mathbb{Z}/\sim_d, \mathrm{s})$ the *standard cycle of order* d, where $\sim_d$ is the equivalence relation with $z \sim_d z'$ if and only if $t^{z-z'} = \mathrm{id}$ for a cycle (X, t) of order d, and $\mathrm{s}([z]) :\equiv [z + 1]$. ⌟

Note that $\sim_d$ doesn't depend on the chosen cycle.

The description in Corollary 3.6.18 is still not as concrete as we'd like. Is it true that any order is principal, in other words, that every cycle has either infinite order or finite order m for some positive $m : \mathbb{N}$? Most other textbooks will tell you that the answer is yes, but the proof is unfortunately not constructive. It makes sense first to restrict to decidable set bundles/cycles.[25] Even so, we need one further non-constructive assumption, namely:

LPO

Principle 3.6.22 (Limited Principle of Omniscience). For any given function $P : \mathbb{N} \to 2$, either there is a smallest number $n_0 : \mathbb{N}$ such that $P(n_0) = 1$, or P is a constant function with value 0. ⌟

[25] This rules out certain pathological cycles, such as the subset $\{(e^{2\pi\alpha i})^n : \mathbb{C} \mid n : \mathbb{Z}\}$, with a suitable equivalence, e.g., incrementing the exponent. Here $\alpha : \mathbb{R}$ is an unknown real number, of which we don't know whether it is rational or not.

The Limited Principle of Omniscience is weaker than the Law of Excluded Middle **??** 2.18.2, as we prove in the following lemma.[26]

LEMMA 3.6.23. *The Law of Excluded Middle implies the Limited Principle of Omniscience.*

Proof. Let $P : \mathbb{N} \to 2$. By the Law of Excluded Middle, either P is constant 0, or there exists some $n : \mathbb{N}$ such that $P(n) = 1$. But in that case we may apply Construction 2.23.4 to conclude that there is a smallest $n_0 : \mathbb{N}$ such that $P(n_0) = 1$. □

xca:not-not-lpop

EXERCISE 3.6.24. Without using LEM or LPO, show that $(Q(P) \to \text{False}) \to \text{False}$ holds for every function $P : \mathbb{N} \to 2$, where $Q(P)$ is the proposition obtained by applying the Limited Principle of Omniscience to the function P. ⌟

As for the Law of Excluded Middle, we are free to assume the Limited Principle of Omniscience or not, and we will be explicit about where we will use it. The Limited Principle of Omniscience makes it possible to prove that the canonical map $\mathbb{N} \to \text{Order}^{\text{dec}}$ (the codomain being the subtype of Order given by decidable cycles), is an equivalence. We will elaborate this equivalence in the next paragraphs.

We already know from Corollary 3.6.18 that the map is an injection, and a cycle (X, t) has infinite order if and only if $H_t = \{0\}$,[27] and it has finite order m if and only if $H_t = m\mathbb{Z}$, for positive $m : \mathbb{N}$.

Fix now a decidable cycle (X, t), and consider the corresponding subset $H :\equiv H_t \equiv \{\, n : \mathbb{Z} \mid t^n = \text{id} \,\}$. This is a decidable subset, since $t^n = \text{id}$ is a proposition, and n is in H if and only if $t^n(x) = x$ for some/all $x : X$ (recall that X is non-empty).

Apply the Limited Principle of Omniscience (**??** 3.6.22) to the function $P : \mathbb{N} \to 2$ defined by $P(n) = 1$ if $n + 1$ is in H, and $P(n) = 0$ otherwise. If $P(n)$ is constant 0, then $H = \{0\}$, so (X, t) has infinite order. (As a set bundle, it is then equivalent to the universal set bundle.)

Otherwise, if n_0 is the smallest natural number with $m :\equiv n_0 + 1$ in H, then we claim $H = m\mathbb{Z}$, from which it follows that (X, t) has order m.

Clearly, $m\mathbb{Z} \subseteq H$, since if $t^m = \text{id}$, then also $t^{nm} = \text{id}$. And if $t^q = \text{id}$, then by Euclidean division of integers, cf. Lemma 2.23.8, there exist $k : \mathbb{Z}$ and $r : \mathbb{N}$ with $r < m$ so that $q = km + r$. Now, the number r is in H, since $t^r = t^{q-km} = \text{id}$, and is less than the minimal positive value m in H, and so we must conclude that $r = 0$. In other words, q is a multiple km, as desired.

We summarize these results in the following lemma.

lem:componentsofcoversofS1

LEMMA 3.6.25. *The Limited Principle of Omniscience* (**??** 3.6.22) *implies that the type of connected decidable set bundles over the circle is the sum of the component containing the universal set bundle and for each positive integer m, the component containing the m-fold set bundle.*

rem:fliptheCircle

REMARK 3.6.26. The reader may wonder how the "orientation reversing" map $r : S^1 \to S^1$ given by $r(\bullet) :\equiv \bullet$ and $r(\circlearrowleft) := \circlearrowleft^{-1}$ fits into the picture.[28] As connected decidable set bundles, we have $(S^1, r) \xrightarrow{=} (S^1, \text{id})$, since r is

[26] It is also the case that the Limited Principle of Omniscience does not imply the Law of Excluded Middle, because a model that satisfies the Limited Principle of Omniscience but not the Law of Excluded Middle can be built using sheaves over the real line $\mathbb{R}$.

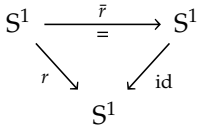
Nevertheless, the Limited Principle of Omniscience is not constructive, for otherwise we could simply decide the truth of every open problem in mathematics that can (equivalently) be expressed by a function $P : \mathbb{N} \to 2$ being constant with value 0. This type of argument was first given by Brouwer.

Here we give an example based on the famous Goldbach conjecture, which states that every even integer greater than 2 is the sum of two primes. Using that the latter two primes are necessarily smaller than the even integer itself, it is possible to (equivalently) express the truth of the Goldbach conjecture by a function $P : \mathbb{N} \to 2$ being constantly 0. Now assume we have a proof t of the Limited Principle of Omniscience in type theory, not using any axioms. Then $t(P)$ is an element of the sum type $L \amalg R$, where R expresses that the function P is constantly 0, and L implies the negation of R. By the computational properties of type theory one can compute the *canonical form* of $t(P)$, which is either inr_r for some element $r : R$, or inl_l for some element $l : L$. If $t(P) \equiv \text{inr}_r$ the Goldbach conjecture is true, and if $t(P) \equiv \text{inl}_l$ the Goldbach conjecture is false. Thus the Goldbach conjecture would be solved, and therefore it is unlikely that t exists. In the appendix, B.2, we give a longer but decisive argument against the constructivity of the Limited Principle of Omniscience.

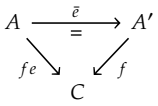
[27] This is why it's natural to associate to $0 : \mathbb{N}$ the infinite order.

[28] As an operation on infinite cycles, see Definition 3.5.3, $crc^{-1} : \text{InfCyc} \to \text{InfCyc}$ maps (X, t) to (X, t^{-1}), flipping the arrows.

an equivalence:



This is a special case of the general case of an equivalence $e : A \to A'$ depicted in the diagram in the margin, implying $(A, fe, !) \xrightarrow{=} (A', f, !)$. The point is that the degree m and degree $-m$ maps give the same *bundles* (by composing with r), while as *maps* they are different. ⌟



3.7 *Interlude*: *combinatorics of permutations*

sec:permutations

In this section, we take a break from analyzing set bundles in order to look more closely at permutations themselves, in particular permutations of finite sets. In Figure 3.11 we depict the same permutation as in Figure 3.6, but "unfolded".

It will be useful to have a more concise notation for permutations. The permutation σ will be denoted (1 2 3)(4 5). The two groups of parentheses indicate the two cycles, and the order within a group indicates the cyclic order. Since the starting point in a cycle doesn't matter, we could also have written, e.g., (3 1 2)(5 4).

In general, if $a_1, a_2, \ldots, a_k$ are pairwise distinct elements of a decidable set A, then we write $(a_1\ a_2\ \cdots\ a_k)$ for the permutation of A that maps a_1 to $a_2, \ldots, a_k$ to a_1, and leaves any other elements untouched. Such a permutation is called a *cyclic permutation* or, somewhat confusingly, a *cycle*. If we want to specify the length, we call it a *k-cycle*. A 2-cycle is also called a *transposition*.

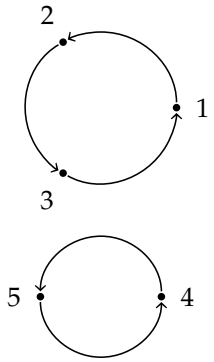


Figure 3.11: A permutation σ with two cycles.

fig:cycle-decomposition

Remark 3.7.1. Any cycle (X, t) in the sense of Definition 3.6.3 (i.e., a cyclically ordered set) gives rise to a permutation t of X consisting of a single cycle. If X is an n-element set and $x_0 : X$, then we can write this permutation in cycle notation as $(x_0\ t(x_0)\ \cdots\ t^{n-1}(x_0))$.

rem:cycle-vs-cycle

Any permutation t on a set X corresponds via Theorem 3.3.8 to a set bundle over S^1, $p : A \to S^1$. Writing A as a sum of its connected components, we express this set bundle as a sum of connected set bundles, but these correspond to cycles by Corollary 3.6.4. Note that cyclic permutations can move at most finitely many elements, and cannot give, e.g., the infinite cycle $(\mathbb{Z}, \mathrm{s})$. Moreover, to define the cycle (X, t) from, e.g., the transposition $(x\ x')$ requires that the set X is decidable. ⌟

Definition 3.7.2. Let A be a set with a permutation σ. If $\sigma(a) = a$, we say that a is a *fixed point* of σ. If $\sigma(a) \neq a$, we say that a is *moved* by σ. The *support* of σ is the subset of A consisting of the elements that are moved by σ. ⌟

def:support-permutation

Note that if A is decidable, then we can decide whether an element is moved or is a fixed point.

Exercise 3.7.3. Let A be a decidable set with two permutations σ, τ. Show that if σ, τ have disjoint supports, then they *commute* in the sense that $\sigma\tau = \tau\sigma$.[29] ⌟

[29] Thus, disjoint cycles commute, so when we express a permutation on a finite set as a product of disjoint cycles, the order doesn't matter.

Exercise 3.7.4. Prove that a k-cycle permutation of a decidable set A can be written as a composition of $k - 1$ transpositions by verifying the

xca:perm-prod-transpositions

identity

$$(a_1\ a_2\ \cdots\ a_k) = (a_1\ a_k)(a_1\ a_{k-1})\cdots(a_1\ a_2).$$ ⌟

COROLLARY 3.7.5. *Any permutation of a finite set can be expressed as a composition of transpositions.*

To show this, first write the permutation as a composition of cyclic permutations, then apply Exercise 3.7.4 to each cycle.[30]

xca:factorial

EXERCISE 3.7.6. Show that there are $n!$ permutations of a finite set of cardinality n, where $n! :\equiv \mathrm{fact}(n)$ is the usual notation for the factorial function.

Hint: One way (not the only one) is to construct bijections $\mathrm{Aut}(\mathbb{0}) \xrightarrow{\simeq} \mathbb{1}$ and

eq:type-factorial

$$\mathrm{Aut}(A \amalg \mathbb{1}) \xrightarrow{\simeq} (A \amalg \mathbb{1}) \times \mathrm{Aut}(A) \tag{3.7.1}$$

for all finite sets A.[31] ⌟

EXERCISE 3.7.7. Let A be a finite set of cardinality n and assume $0 \leq k \leq n$. Show that the number of k-element subsets of A is given by the binomial coefficient[33]

$$\binom{n}{k} :\equiv \frac{n!}{k!(n-k)!}.$$

Find a formula for the number of k-cycle permutations of A using factorials and/or binomial coefficients. ⌟

3.8 *The mth root: set bundles over the components of* Cyc

sec:mthroot

Let's first give names to some important components of Cyc that we have met in previous sections, e.g., in Lemma 3.6.25.

def:Cyc-components

DEFINITION 3.8.1. Define $\mathrm{Cyc}_0 :\equiv \mathrm{Cyc}_{(\mathbb{Z},\mathrm{s})}$. For each positive $m : \mathbb{N}$, define $\mathrm{Cyc}_m :\equiv \mathrm{Cyc}_{(m,\mathrm{s})}$. We call Cyc_0 and Cyc_m the *type of infinite cycles* and *type of m-cycles*, respectively. ⌟

Recall the equivalence $c : \mathrm{S}^1 \xrightarrow{\simeq} \mathrm{Cyc}_0$ of Definition 3.5.3 between the circle and the type of infinite cycles. In this section, we reinterpret the degree m function δ_m as a map of infinite cycles. In fact δ_m makes sense as a map on all cycles, and we'll use it to begin the classification of the connected set bundles over Cyc_n, for positive integers n. That's why it's instructive to rephrase connected set bundles over S^1 in terms of cycles, even though they could just be transported along the identification $\bar{c} : \mathrm{S}^1 \xrightarrow{=} \mathrm{Cyc}_0$ corresponding to c.

Before we do the degree m maps, let's note that the universal set bundle over Cyc_0 is represented by the constant function $\mathrm{cst}_{\mathrm{pt}_0} : \mathbb{1} \to \mathrm{Cyc}_0$, sending the unique element of $\mathbb{1}$ to $\mathrm{pt}_0 :\equiv (\mathbb{Z}, \mathrm{s}) : \mathrm{Cyc}_0$, the standard infinite cycle.[34]

For the rest of this section, we fix some positive $m : \mathbb{N}$. We now give a description of the m-fold set bundle over the circle in terms of cycles.

We proceed as follows. First we present the answer, a set bundle we call $\rho_m : \mathrm{Cyc}_0 \to \mathrm{Cyc}_0$, and then we prove that $\delta_m : \mathrm{S}^1 \to \mathrm{S}^1$ and $\rho_m : \mathrm{Cyc}_0 \to \mathrm{Cyc}_0$ correspond to each other (and to $\mathrm{pow}_m : \mathrm{Tot}(R_m) \to \mathrm{S}^1$) under the equivalence $c : \mathrm{S}^1 \xrightarrow{\simeq} \mathrm{Cyc}_0$.

What should we require of $\rho_m(X,t)$ for $(X,t) : \mathrm{Cyc}_0$? Well, $\delta_m : \mathrm{S}^1 \to \mathrm{S}^1$ sends $\bullet$ to $\bullet$ and $\circlearrowleft$ to $\circlearrowleft^m$; only the $\circlearrowleft^k$ where k is a multiple of k is in

[30] This representation is not unique, as for example $(1\ 2) = (2\ 3)(1\ 3)(2\ 3)$ as permutations of $\{1,2,3\}$. However, in Corollary 4.5.11 below, we'll show that the *parity* (odd/even) of the number of transitions is invariant.

[31] In fact, the bijection (3.7.1) can be constructed for any decidable set. Escardó[32] constructed more generally, for any type X, an equivalence $\mathrm{Aut}(X \amalg \mathbb{1}) \xrightarrow{\simeq} (X \amalg \mathbb{1})' \times \mathrm{Aut}(X)$, where

$$Y' :\equiv \sum_{y:Y} \prod_{z:Y} ((y \xrightarrow{=} z) \amalg ((y \xrightarrow{=} z) \to \emptyset)).$$

By a local version of Hedberg's Theorem 2.20.15, Y' is a subtype of Y.

[32] Martín Escardó. *UF-Factorial*. Agda formalization. 2019. URL: https://www.cs.bham.ac.uk/~mhe/TypeTopology/UF-Factorial.html.

[33] Binomial coefficients are familiar from Pascal's triangle,

$$\begin{array}{ccccccccc} & & & & 1 & & & & \\ & & & 1 & & 1 & & & \\ & & 1 & & 2 & & 1 & & \\ & 1 & & 3 & & 3 & & 1 & \\ 1 & & 4 & & 6 & & 4 & & 1 \\ 1 & & 5 & & 10 & & 10 & & 5 & & 1 \\ & & & & \vdots & & & & \end{array}$$

where each number is the sum of the two above, e.g., $\binom{4}{2} = 6$.

The forgetful map from Cyc_0 to InfCyc is an equivalence. Therefore we consider Cyc_0 and InfCyc as definitionally equal.

[34] In light of Lemma 3.5.2 we see that the fiber of this universal set bundle over $(X,t) : \mathrm{Cyc}_0$ is (equivalent to) X itself – that's certainly a universal set associated to the infinite cycle (X,t)!

the image of δ_m. So we have to find an infinite cycle (Y, u) with "u^m corresponding to t". We achieve this by "stretching" X: Let Y be m copies of X and let u jump idly from one copy to another except every m^{th} time when u also is allowed to use t. This is illustrated in Figure 3.12 with the shift by t being vertical and the movement from copy to copy going around a circle.

con:root

Construction 3.8.2. *For any type X and $t : X \to X$, we define the mth root*

$$\sqrt[m]{t} : (\mathbb{m} \times X) \to (\mathbb{m} \times X).$$

Implementation of Construction 3.8.2. We set

$$\sqrt[m]{t}(k, x) :\equiv \begin{cases} (k+1, x) & \text{for } k < m-1 \text{ and} \\ (0, t(x)) & \text{for } k = m-1. \end{cases}$$ □

Only one m^{th} of the time does $\sqrt[m]{t}$ use $t : X \to X$, the rest of the time it applies the successor in $\mathbb{m}$. Indeed, iterating $\sqrt[m]{t}$ we get an identification of type $(\sqrt[m]{t})^m(k, x) \xrightarrow{=} (k, t(x))$; hence the term "$m^{\text{th}}$ root" is apt.

def:root

Definition 3.8.3. The *formal m^{th} root function* is defined by:

$$\rho_m : \sum_{X:\mathcal{U}} (X \to X) \to \sum_{X:\mathcal{U}} (X \to X), \qquad \rho_m(X, t) :\equiv (\mathbb{m} \times X,\ \sqrt[m]{t}).$$ ⌟

We use ρ for "root" to denote this incarnation of the degree m function.

lem:root-pres-equiv

Lemma 3.8.4. *If $t : X \to X$ is an equivalence, then so is $\sqrt[m]{t} : (\mathbb{m} \times X) \to (\mathbb{m} \times X)$.*

Proof. Let $t : X \to X$ be an equivalence. We prove that the fibers of $\sqrt[m]{t}$ are contractible.

For the fiber at $(0, x)$ we note, using Lemma 2.10.3, that identifications in $(0, x) \xrightarrow{=} (\sqrt[m]{t})(\ell, y)$ consist of pairs of proofs of $\ell = m - 1$ and identifications in $x \xrightarrow{=} t(y)$. Both $\sum_{\ell:\mathbb{m}} \ell = m - 1$ and $t^{-1}(x)$ are contractible, and so $(\sqrt[m]{t})^{-1}(0, x)$ is contractible.

For the fiber at (k, x) with $k : \mathbb{m}$ not 0, identifications in $(k, x) \xrightarrow{=} (\sqrt[m]{t})(\ell, y)$ consist of pairs of proofs of $\ell + 1 = k$ and identifications in $x \xrightarrow{=} y$, so $(\sqrt[m]{t})^{-1}(k, x)$ is contractible since both $\sum_{\ell:\mathbb{m}} \ell + 1 = k$ and $\sum_{y:X} x \xrightarrow{=} y$ are. □

Lemma 3.8.5. *Let $X : \mathcal{U}$ and $t : X \to X$. If (X, t) is a cycle, then so is $\rho_m(X, t)$.*

Proof. Clearly, $\mathbb{m} \times X$ is a nonempty set if X is. We already know $\sqrt[m]{t}$ is an equivalence if t is. For connectedness, let $(k, x), (k', x') : (\mathbb{m} \times X)$. We need to show the proposition that there exists $n : \mathbb{Z}$ with $(k', x') = (\sqrt[m]{t})^n(k, x)$. Let $n : \mathbb{Z}$ be such that $x' = t^n(x)$. Then $(\sqrt[m]{t})^{nm}(k, x) = (k, t^n(x)) = (k, x')$, so if $k = k'$ we're done. Assume $k < k'$. Then $(\sqrt[m]{t})^{k'-k}(k, x') = (k', x')$, so $(\sqrt[m]{t})^{nm+k'-k}(k, x) = (k', x')$, as desired. The case $k > k'$ is similar. □

The question now arises: how does ρ_m act on the components of Cyc, and what can we say about the preimages $\rho_m^{-1}(X, t)$ for an arbitrary cycle (X, t)?

The first part is easy, since the product of $\mathbb{m}$ with an n-element set is an mn-element set.

lem:deg-m-on-Cyc

Lemma 3.8.6. *The degree m function restricts to give pointed maps*

$$\rho_m : \mathrm{Cyc}_n \to_* \mathrm{Cyc}_{mn} \quad \textit{and} \quad \rho_m : \mathrm{Cyc}_0 \to_* \mathrm{Cyc}_0.$$

$\sqrt[m]{t} : (\mathbb{m} \times X) \to (\mathbb{m} \times X)$

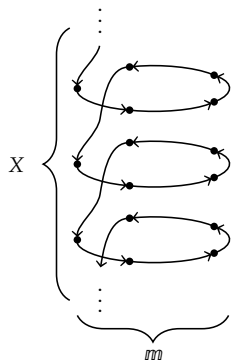


Figure 3.12: The mth root $\sqrt[m]{t}$ of a function $t : X \to X$, here illustrated in the case $m = 5$.

fig:root

Of course, it's also quite easy to write down an inverse of $\sqrt[m]{t}$ given an inverse of t.

In terms of iterated addition, we have $\varphi(k, r) = (z \mapsto z + m)^r(k)$.

Proof. Recall Definition 3.8.1. The components Cyc_k are pointed by $\mathrm{pt}_0 :\equiv (\mathrm{Z}, \mathrm{s})$ if $k = 0$, and $\mathrm{pt}_k :\equiv (\mathbb{k}, \mathrm{s})$ else. Note that the function $\varphi : (\mathbb{m} \times \mathrm{Z}) \to \mathrm{Z}$ given by $\varphi(k, r) :\equiv k + mr$ is an equivalence, with inverse given by Euclidean division by m. Moreover, we have $\varphi \sqrt[m]{\mathrm{s}} = \mathrm{s}\, \varphi$, since

$$\varphi\left(\sqrt[m]{\mathrm{s}}(k, r)\right) = k + 1 + mr = \mathrm{s}(\varphi(k, r)) \quad \text{for all } (k, r) : \mathbb{m} \times \mathrm{Z}.$$

This shows that φ gives an identification of infinite cycles $(\mathbb{m} \times \mathrm{Z}, \sqrt[m]{\mathrm{s}}) \xrightarrow{=} (\mathrm{Z}, \mathrm{s})$, and hence the mth root construction maps the component Cyc_0 to itself.

Analogously, we can restrict φ to an equivalence $\mathbb{m} \times \mathbb{n} \xrightarrow{\simeq} \sum_{k:\mathbb{N}} (k < mn)$, and get an identification of cycles $\rho_m(\mathrm{pt}_n) \xrightarrow{=} \mathrm{pt}_{mn}$, showing that ρ_m maps the component Cyc_n to the component Cyc_{mn}. □

We now analyze how ρ_m acts on paths. Let $(\bar{e}, !) : (X, t) \xrightarrow{=} (X', t')$. Since ρ_m maps first components X to $\mathbb{m} \times X$, we get that the first projection of $\mathrm{ap}_{\rho_m}(\bar{e}, !)$ is $\mathrm{id} \times e : (\mathbb{m} \times X) \xrightarrow{=} (\mathbb{m} \times X')$. We are particularly interested in the case of the loops, that is, $(\bar{e}, !) : (X, t) \xrightarrow{=} (X, t)$. We calculate $(\mathrm{id} \times e)(k, x) = (k, e(x))$, which by the property of the mth root is equal to $(\sqrt[m]{e})^m(k, x)$. In particular, if we take $e :\equiv t^{-1}$, then we get $(\mathrm{id} \times t^{-1}) = (\sqrt[m]{t^{-1}})^m$, which means that $\mathrm{ap}_{\rho_m}(\bar{t}^{-1}, !)$ is indeed the mth power of a generating loop at the image cycle $\rho_m(X, t)$. In particular, this holds for the standard infinite cycle $(\mathrm{Z}, \mathrm{s}) : \mathrm{Cyc}_0$ and the standard n-cycle $(\mathbb{n}, \mathrm{s}) : \mathrm{Cyc}_n$.

Why does $\rho_m : \mathrm{Cyc}_0 \to \mathrm{Cyc}_0$ correspond to the m-fold set bundle we defined in Definition 3.6.5? Recall the equivalence $c : \mathrm{S}^1 \to C$ from Definition 3.5.3. For the two m-fold set bundles to correspond under this equivalence we need an element in the identity type represented by

$$\begin{array}{ccc} \mathrm{S}^1 & \xrightarrow{c} & \mathrm{Cyc}_0 \\ {\scriptstyle \delta_m}\downarrow & & \downarrow{\scriptstyle \rho_m} \\ \mathrm{S}^1 & \xrightarrow{c} & \mathrm{Cyc}_0. \end{array}$$

That is, we need an element in $\rho_m c \xrightarrow{=}_{\mathrm{S}^1 \to \mathrm{Cyc}_0} c\, \delta_m$. Under the equivalence

$$\mathrm{ev}_{\mathrm{Cyc}_0} : (\mathrm{S}^1 \to \mathrm{Cyc}_0) \xrightarrow{\simeq} \sum_{(X,t):\mathrm{Cyc}_0} \left((X, t) = (X, t)\right)$$

of Theorem 3.1.2, the composite $c\, \delta_m$ is given by $\left((\mathrm{Z}, \mathrm{s}), \mathrm{s}^{-m}\right)$ and the composite $\rho_m c$ is given by $\left((\mathbb{m} \times \mathrm{Z}, \sqrt[m]{\mathrm{s}}), \mathrm{id} \times \mathrm{s}^{-1}\right)$: we must produce an element in

$$\left((\mathbb{m} \times \mathrm{Z}, \sqrt[m]{\mathrm{s}}), \mathrm{id} \times \mathrm{s}^{-1}\right) \xrightarrow{=} \left((\mathrm{Z}, \mathrm{s}), \mathrm{s}^{-m}\right).$$

Consider the equivalence $\varphi : (\mathbb{m} \times \mathrm{Z}) \xrightarrow{\simeq} \mathrm{Z}$ with $\varphi(k, n) :\equiv k + mn$ also used in Lemma 3.8.6. discussed above. Transport of $\sqrt[m]{\mathrm{s}}$ along φ is exactly s, i.e., $\varphi \sqrt[m]{\mathrm{s}} = \mathrm{s}\, \varphi$.[35] Likewise, transport of $\mathrm{id} \times \mathrm{s}^{-1}$ along φ is s^{-m}, so that φ lifts to an element in $\left((\mathbb{m} \times \mathrm{Z}, \sqrt[m]{\mathrm{s}}), \mathrm{id} \times \mathrm{s}^{-1}\right) \xrightarrow{=} \left((\mathrm{Z}, \mathrm{s}), \mathrm{s}^{-m}\right)$.

[35] Note that we formulate this in such a way that we don't need to talk about the inverse of φ. Of course, the inverse of φ maps $z : \mathrm{Z}$ to the remainder and the integer quotient of z under Euclidean division by m, cf. Lemma 2.23.8.

Exercise 3.8.7. Extend the above construction to an identification of type $\rho_m c \xrightarrow{=}_{\mathrm{S}^1 \to_* \mathrm{Cyc}_0} c\, \delta_m$ in case all these maps are taken to be pointed. ⌟

So we know that the fiber of ρ_m at an infinite cycle (X, t) is an m-element set. In fact, we will identify this set as $X/m :\equiv X/{\sim_m}$ where $\sim_m$ is the equivalence relation that identifies points that are a distance mr

xca:pointed-maps-circle

apart, for some $r : \mathbb{Z}$. Formally, let $x \sim_m x'$ if and only if $\exists_{r:\mathbb{Z}}(x' = t^{mr}(x))$. (Such an r is unique if it exists.) Indeed, the fiber is

$$\sum_{(Y,u):\mathrm{Cyc}_0} \left((X,t) \xrightarrow{\equiv} (\mathbb{m} \times Y, \sqrt[m]{u})\right).$$

We sketch an equivalence from X/m to $\rho_m^{-1}(X,t)$. See Construction 3.8.11 below for a careful proof of a more general statement for arbitrary cycles, not only infinite ones. Let Y be an equivalence class of X/m, taken as a set. One should think of Y as a set $\{\ldots, t^{-2m}(x), t^{-m}(x), x, t^m(x), t^{2m}(x), \ldots\}$ for some $x : X$. Then (Y, t^m) is an infinite cycle and we can construct a natural[36] identification $i : (X,t) \xrightarrow{\equiv} (\mathbb{m} \times Y, \sqrt[m]{t^m})$, so that $(Y, t^m, i) : \rho_m^{-1}(X,t)$. The map $Y \mapsto (Y, t^m, i)$ is the intended equivalence.

[36] The map defined by $e(k,x) :\equiv t^k(x)$ is an equivalence from $\mathbb{m} \times Y$ to X such that $te = e\,\sqrt[m]{t^m}$.

The reader will no doubt have noticed that X/m is a *finite cycle*. We'll return to the significance of this below in Section 3.9.

Our next step is to identify the fiber of ρ_m over a general cycle (X,t). Classically, the remaining cases are those of finite n-cycles, but it's illuminating to be a bit more general. Note that the equivalence relation $\sim_m$ defined above for an infinite cycle makes sense for all cycles.

lem:sum-cycle-point-contr

Lemma 3.8.8. *For any order d : Order, the type $\sum_{(X,t):\mathrm{Cyc}_d} X$ is contractible, where Cyc_d denotes the component of* Cyc *consisting of cycles of order d.*

Proof. First we note that the goal is a proposition. Clearly, for any cycle (Y,u), the singleton type $\sum_{(X,t):\mathrm{Cyc}_{(Y,u)}}((X,t) \xrightarrow{\equiv} (Y,u))$ is contractible. Using Lemma 3.6.14 and Corollary 3.6.15, it follows that $\sum_{(X,t):\mathrm{Cyc}_{(Y,u)}} X$ is contractible. Now the lemma follows by set truncation elimination. □

lem:m-root-id

Lemma 3.8.9. *For any cycle (X,t), if $(\sqrt[m]{t})^n = \mathrm{id}_{\mathbb{m}\times X}$, then m divides n, i.e., $n = mq$ for some $q : \mathbb{Z}$, and $t^q = \mathrm{id}_X$. In other words, m divides the order of $\sqrt[m]{t}$.*

This follows simply by looking at the first component, where $\sqrt[m]{t}$ acts as the successor operation on $\mathbb{m}$. See Definition 3.6.20 for the order.

We're almost ready to identify the fiber of ρ_m at a cycle (X,t). Let's explore first the problem of finding an identification of (X,t) with $\rho_m(Y,u) \equiv (\mathbb{m} \times Y, \sqrt[m]{u})$ for a given cycle (Y,u). By Lemma 3.6.14, a necessary condition for such an identification is $H_t =_{\mathrm{Sub}(\mathbb{Z})} H_{\sqrt[m]{u}}$. Recall from Definition 3.6.12 that $H_t \equiv \{\, n : \mathbb{Z} \mid t^n = \mathrm{id}_X \,\}$ and $H_{\sqrt[m]{u}} \equiv \{\, n : \mathbb{Z} \mid (\sqrt[m]{u})^n = \mathrm{id}_{\mathbb{m}\times Y} \,\}$. We know from Lemma 3.8.9 that m divides the order of $\sqrt[m]{u}$, so the fiber $\rho_m^{-1}(X,t)$ is nonempty only if m divides the order of t.

A key ingredient for the converse is the following.

lem:X-mod-m-chosen

Lemma 3.8.10. *Let $(X,t) : \mathrm{Cyc}$ be a cycle with order divisible by m and let x_0 be an element of X. Then the map $f : \mathbb{m} \to X/m$, $f(k) :\equiv [t^k(x_0)]$ is an equivalence.*

Proof. Fix an equivalence class $V : X/m$ and consider its preimage under f, $f^{-1}(V) \equiv \sum_{k:\mathbb{m}}(V = [t^k(x_0)])$. The contractibility of this type is a proposition, so we may choose $x : X$ with $V = [x]$. Then $(V = [t^k(x_0)]) \simeq ([x] = [t^k(x_0)]) \simeq (x \sim_m t^k(x_0))$. So we need to show that $\sum_{k:\mathbb{m}}(x \sim_m t^k(x_0))$ is contractible. More simply, we need to show that there is a unique k with $x \sim_m t^k(x_0)$. Since (X,t) is a cycle, we may further choose $n : \mathbb{Z}$ with $x = t^n(x_0)$. By Euclidean division, write $n = qm + r$

Here we take V not as a set, but as an element of the set $X \to \mathrm{Prop}$. See the discussion after Lemma 2.20.4 for the distinction.

with $q : \mathbb{Z}$, $r : \mathbb{m}$. Then $x = t^n(x_0) \sim_m t^r(x_0)$, so we have our center. Let $k : \mathbb{m}$ also satisfy $x \sim_m t^k(x_0)$. We need to show the proposition $k = r$. But $t^{r-k}(x_0) \sim_m x_0$, so we may take $q' : \mathbb{Z}$ with $t^{q'm+r-k}(x_0) = x_0$. Since m divides the order of t, this implies $r = k$, as desired. □

Now we have all the pieces needed to prove the main result.

thm:fiber-cdg

Construction 3.8.11. *For* any *cycle* (X, t)*, we have an equivalence between* $\rho_m^{-1}(X, t)$ *and* $P \times X/m$*, where* P *expresses that* m *divides the order of* t*, formally* $P :\equiv (H_t \subseteq m\mathbb{Z})$ *(see Definition 3.6.20).*[37]

[37] When X is a decidable set then LPO allows for a simpler formulation: Then P is either false, in which case $\rho_m^{-1}(X, t)$ is empty, or true, in which case the preimage is X/m.

Implementation of Construction 3.8.11. We'll use Construction 2.9.9, and we first define the function

$$g : \rho_m^{-1}(X, t) \to P \times X/m,$$

by mapping (Y, u) and an identification of cycles $e : (X, t) \xrightarrow{\simeq} (\mathbb{m} \times Y, \sqrt[m]{u})$ to the proof of P from Lemma 3.8.9 and the class $V_e :\equiv [e^{-1}(0, y)] : X/m$, for any $y : Y$.[38] As a subset of X, $V_e = \{\, x : X \mid \mathrm{fst}(e(x)) = 0 \,\}$.

[38] Note that this doesn't depend on y, so that Theorem 2.22.9 applies: $(0, y)$ and $(0, y')$ are a distance mr apart, and e^{-1} preserves this distance.

In the other direction, to define the function

$$h : P \times X/m \to \rho_m^{-1}(X, t),$$

fix an equivalence class V of X/m, and assume that m divides the order of t. As in the discussion after Exercise 3.8.7, we take V as the set of elements in X that lie in the class V. Then (V, t^m) is a cycle.[39] We also need an identification $(X, t) \xrightarrow{\simeq} \rho_m(V, t^m) \equiv (\mathbb{m} \times V, \sqrt[m]{t^m})$. This we define via a map $e' : \mathbb{m} \times V \to X$, $e'(k, v) :\equiv t^k(v)$, which preserves cycle structure: $te' = e'\sqrt[m]{t^m}$. The map e' is an equivalence if $H_t \subseteq H_{\sqrt[m]{t^m}}$, by Exercise 3.6.19. So let $n : \mathbb{Z}$, and assume that $t^n = \mathrm{id}_X$. Then P implies that we may write $n = qm$ for some $q : \mathbb{Z}$, so

[39] Indeed t^m is properly restricted to V: If x lies in V, then so does $t^m(x)$.

$$(\sqrt[m]{t^m})^n = (\sqrt[m]{t^m})^{qm} = (\mathrm{id}_{\mathbb{m}} \times t^m)^q = (\mathrm{id}_{\mathbb{m}} \times t^{mq}) = (\mathrm{id}_{\mathbb{m}} \times \mathrm{id}_V) = \mathrm{id}_{\mathbb{m} \times V}.$$

Straight from these definitions, we see that $g \circ h = \mathrm{id}$. We leave to the reader to check that $h \circ g = \mathrm{id}$. □

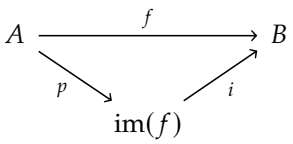
3.9 *Higher images*

sec:higher-images

In this section we take a quick break from characterizing the connected set bundles of Cyc_n for finite orders n in order to make good on our earlier promise to say something about the fact that each fiber of ρ_m carries a cycle structure. This involves the notion of 0-image of a map, but we might as well introduce the general notion of n-image while we're at it.

Recall from Definition 2.17.11 the propositional image $\sum_{y:B} \|f^{-1}(y)\|$ of a map $f : A \to B$. The propositional image can be generalized using n-truncation instead of propositional truncation.

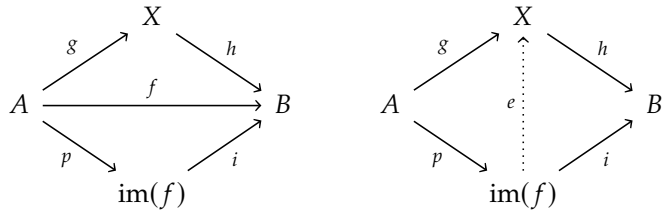
Recall furthermore the image factorization from Exercise 2.17.12:



Here p is surjective and i is injective, and any such factorization is equivalent to this one. Both surjectivity (Definition 2.17.1) and injectivity (Definition 2.17.2) rely on the notion of a proposition: all fibers of p are nonempty and all fibers of i are propositions.

The uniqueness of the factorization $f \overset{=}{\to} ip$ can be visualized in terms of the two diagonals of the diamond below: for any surjection $g : A \to X$ and injection $h : X \to B$ with $f \overset{=}{\to} hg$, one can construct a (unique) equivalence e with $g \overset{=}{\to} ep$ and $i \overset{=}{\to} he$.

(3.9.1) eqn:image-univ-prop



The existence of a unique equivalence e as above is called the *universal property of the propositional image*. Uniqueness of e above also follows from the following two exercises.

Recall that $i_$ and $_p$ are compact denotations of the maps $h \mapsto ih$ and $h \mapsto hp$, respectively.

xca:cancel-injection

Exercise 3.9.1. Let A, B, X be types and $i : A \to B$ an injection. Let $i_ : (X \to A) \to (X \to B)$ be postcomposition with i. Show that $\mathrm{ap}_{i_} : (f \overset{=}{\to} g) \to (if \overset{=}{\to} ig)$ is an equivalence, for any $f, g : X \to A$. ⌟

xca:cancel-surjection

Exercise 3.9.2. Let A, B, Y be types and $p : A \to B$ be a surjection. Let $_p : (B \to Y) \to (A \to Y)$ be precomposition with p. Show that $\mathrm{ap}_{_p} : (f \overset{=}{\to} g) \to (fp \overset{=}{\to} gp)$ is an equivalence, for any $f, g : B \to Y$. ⌟

We will now define higher images and generalize the notions of injection and surjection such that a similar universal property of higher images can be proved.

def:n-image

Definition 3.9.3. Let A, B be types and let $f : A \to B$. We define the *n-image* of f as

$$\mathrm{im}_n(f) :\equiv \sum_{b:B} \|f^{-1}(b)\|_n. \qquad ⌟$$

Observe that $\mathrm{im}_{-1}(f) \equiv \mathrm{im}(f)$.

If A and B are sets, then the fibers of $f : A \to B$ are sets as well. Hence $\mathrm{im}_0(f)$ amounts to the set of pairs (b, a) such that $b = f(a)$, that is, the inverse of the relation that is commonly known as the graph of f.

def:n-connected

Definition 3.9.4. A type A is called *n-connected* if its truncation $\|A\|_n$ is contractible. A function $f : A \to B$ is called *n-connected* if the fiber $f^{-1}(b)$ is n-connected, for each $b : B$. ⌟

Thus, any type is (-2)-connected, since its (-2)-truncation is contractible. Moreover, the (-1)-connected types are precisely the nonempty ones, and the 0-connected types are those we have called connected in Definition 2.16.8.

def:n-truncated

Definition 3.9.5. A function $f : A \to B$ is called *n-truncated* if the fiber $f^{-1}(b)$ is an n-type, for each $b : B$. ⌟

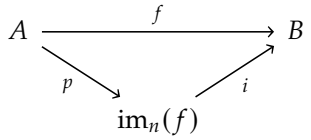
One may verify now that the (-1)-connected functions are the surjections, and the (-1)-truncated functions are the injections.

It is instructive to explore the special case of $n = -2$. Every map $f : A \to B$ is trivially (-2)-connected. Moreover, $\mathrm{fst} : \mathrm{im}_{-2}(f) \to B$ is an equivalence and (-2)-truncated maps are precisely equivalences. Thus the factorization of $f : A \to B$ through its (-2)-image can go through B and be identified with $f \overset{=}{\to} \mathrm{id}_B f$.

There is a factorization $f \overset{=}{\to} ip$ of a map $f : A \to B$ through its n-image, where p is defined by setting $p(a) :\equiv (f(a), |(a, \mathrm{refl}_{f(a)})|_n)$, and where i is defined by setting $i :\equiv \mathrm{fst}$, as in the following diagram.

eqn:n-image-factorization

(3.9.2)



The map i is n-truncated, because, for any $b : B$, the fiber $i^{-1}(b)$ is equivalent to $\|f^{-1}(b)\|_n$. Furthermore, by Lemma 2.25.2 and the following lemma, p is n-connected.[40]

lem:trunc-n-connected

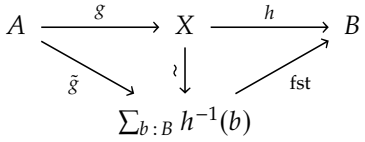
Lemma 3.9.6. *For every type A, the constructor $|_|_n : A \to \|A\|_n$ is n-connected.*

Proof. We have to prove that the n-truncation of each fiber of $|_|_n$ is contractible. We start by defining a function $c : \prod_{x:\|A\|_n} \||x|_n^{-1}\|_n$ producing the centers. Since c takes values in n-types, we can define c by n-truncation elimination by setting $c(|a|_n) :\equiv |(a, \mathrm{refl}_{|a|_n})|_n$.

The next step is to construct an element of $\prod_{x:\|A\|_n} \prod_{y:\||x|_n^{-1}\|_n} (c(x) \xrightarrow{=} y)$. Since the identity $c(x) \xrightarrow{=} y$ is an $(n-1)$-type, it suffices to give an element of $\prod_{x:\|A\|_n} \prod_{z:|x|_n^{-1}} (c(x) \xrightarrow{=} |z|_n)$. Since fibers are sum types, it suffices to give an element of $\prod_{x:\|A\|_n} \prod_{a:A} \prod_{p:x \xrightarrow{=} |a|_n} (c(x) \xrightarrow{=} |(a,p)|_n)$. After swapping the first two products, the identity reduces by path induction to $c(|a|_n) \xrightarrow{=} |(a, \mathrm{refl}_{|a|_n})|_n$, for which we can use the reflexivity path. □

con:fibcomp=fibfib

Construction 3.9.7. *Let $g : A \to X$ and $h : X \to B$, and let $\tilde{g} : A \to \sum_{b:B} h^{-1}(b)$ be the composition of g with the canonical equivalence $X \to \sum_{b:B} h^{-1}(b)$ from Lemma 2.25.2. Thus $\tilde{g}(a) \equiv (h(g(a)), g(a), \mathrm{refl}_{h(g(a))})$ for each $a : A$, and the situation is visualized in the following diagram*:



Then we have equivalences $e(b) : (hg)^{-1}(b) \xrightarrow{\simeq} \sum_{y:h^{-1}(b)} \tilde{g}^{-1}(b, y)$ for all $b : B$.

Implementation of Construction 3.9.7. Let, for each $b : B$, $e(b)$ map any pair $(a, p) : (hg)^{-1}(b)$ to $((g(a), p), (a, q))$. Here q is of type $(b, g(a), p) \xrightarrow{=} (h(g(a)), g(a), \mathrm{refl}_{h(g(a))})$ and is given componentwise by $p : b \xrightarrow{=} h(g(a))$, $\mathrm{refl}_{g(a)}$, and by the easy path over p from p to $\mathrm{refl}_{h(g(a))}$ in the identity type family $_ \xrightarrow{=} h(g(a))$. This construction uses Definition 2.10.1, Definition 2.7.3, and Exercise 2.14.4(3). □

xca:fibcomp=fibfib

Exercise 3.9.8. Complete the details of Construction 3.9.7. In particular, prove that e is a fiberwise equivalence. Alternatively, construct your own e by using Corollary 2.9.11 (twice!). ⌟

xca:trunc-sum-type+fam

Exercise 3.9.9. Let X be a type and let $Y(x)$ be a type for all $x : X$. Construct an equivalence between $\|\sum_{x:X} Y(x)\|_n$ and $\|\sum_{x:X} \|Y(x)\|_n\|_n$. ⌟

We shall show in Theorem 3.9.11 that the n-image factorization of $f : A \to B$ in Equation (3.9.2) is unique. This result is called the *universal property of the n-image*. We start by defining a convenient abbreviation.

def:type-of-factorizations

Definition 3.9.10. Let X and Y be types. For any $f : X \to Y$ we define the type $\mathrm{Fact}(f)$ of *factorizations of f* as follows:

$$\mathrm{Fact}(f) :\equiv \sum_{Z:\mathcal{U}} \sum_{g:X\to Z} \sum_{h:Z\to Y} f \xrightarrow{=} h \circ g \qquad ⌟$$

[40] For $T : B \to \mathcal{U}$ and p' a map sending $(b, t) : \sum_{b:B} T(b)$ to $(b, |t|_n)$, the fiber of p' at any $(b, t') : \sum_{b:B} \|T(b)\|_n$ is equivalent to the fiber of $|_|_n$ at t', by Lemma 2.9.10.

As a figure of speech we may speak of "a factorization $f \xrightarrow{=} h \circ g$." Here Z is implicit in the types of g and h. The particular identification of f with $h \circ g$ follows from the context.

thm:n-im-univ-prop

THEOREM 3.9.11. *Let $f : A \to B$ and $n \geq -2$. Then the following type of* factorizations of f through its n-image *is contractible*:

$$\sum_{(C,g,h,r) : \mathrm{Fact}(f)} (\mathrm{is}n\mathrm{Conn}(g) \times \mathrm{is}n\mathrm{Trunc}(h)).$$

Proof. As the center of contraction we take $(\mathrm{im}_n(f), p, i, \mathrm{refl}_f, !, !)$, with p and i as in Equation (3.9.2). We can use $\mathrm{refl}_f : f \xrightarrow{=} ip$ since $\mathrm{fst}(p(a)) \equiv f(a)$ for all $a : A$.

Let X be a type and assume we are given an n-connected function $g : A \to X$ and an n-truncated function $h : X \to B$ and an identification $y : f \xrightarrow{=} hg$. Our task is then to construct a equivalence $e : \mathrm{im}_n(f) \to X$ and to give identifications represented by the left and the right triangle in Figure 3.13, that is, of types $g \xrightarrow{=} ep$ and $i \xrightarrow{=} he$. Then the factorization $(X, g, h, s, !, !)$ can be identified with the center of contraction by standard transport lemmas.

To simplify these constructions, we are going to replace g and h by projection maps. In view of Construction 3.9.7, we may assume without loss of generality that $X \equiv \sum_{b:B} P(b)$ for some family of n-types $P(b)$, and $h \equiv \mathrm{fst}$.

By Lemma 2.25.2 we may also assume without loss of generality that $A \equiv \sum_{b:B} \sum_{y:P(b)} Q(b, y)$, where $Q(b, y) :\equiv g^{-1}(b, y)$ are the fibers of g, which are all n-connected by assumption. Define $R(b) :\equiv \sum_{y:P(b)} Q(b, y)$ for all $b : B$. With $A \equiv \sum_{b:B} R(b)$, the function g takes the form of the projection map $(b, y, q) \mapsto (b, y)$, as shown in Figure 3.14. Using $s : f \xrightarrow{=} hg$ we get an identification of f with the first projection, and an equivalence between its n-image and $\sum_{b:B} \|R(b)\|_n$. The n-connected map p then takes the form $(b, y, q) \mapsto (b, |(y, q)|_n)$ as shown in Figure 3.14.

Since $(\sum_{b:B} \mathbb{1}) \xrightarrow{=} B$ via fst, each type in Figure 3.14 can be considered to be the sum of a type family parametrized by $b : B$. For constructing the equivalence e that makes Figure 3.14 commute, it suffices to construct for each $b : B$ the equivalence e_b such that Figure 3.15 commutes. Then we obtain e as desired by summing over B, that is, by putting $e(b, z) :\equiv (b, e_b(z))$ for all $b : B$ and $z : \|R(b)\|_n$.

Now let $b : B$. We have $\|R(b)\|_n \equiv \|\sum_{y:P(b)} Q(b, y)\|_n$. Since $P(b)$ is an n-type by assumption, we have the canonical equivalence of type $\|\sum_{y:P(b)} Q(b, y)\|_n \to \sum_{y:P(b)} \|Q(b, y)\|_n$ defined by mapping $|(y, q)|_n$ to $(y, |q|_n)$ (cf. Exercise 3.9.9). Since $Q(y, b)$ is n-connected for each $y : P(b)$ by assumption, so that $\|Q(b, y)\|_n$ is contractible, we also have the canonical equivalence $\mathrm{fst} : \sum_{y:P(b)} \|Q(b, y)\|_n \to P(b)$. The composite of these two equivalences is e_b.

Finally, the identifications of type $g \xrightarrow{=} ep$ and $i \xrightarrow{=} he$, represented by the left and right triangle in Figure 3.14, can also be constructed pointwise for every $b : B$, that is, in Figure 3.15. Indeed, we have $e_b|_|_n \equiv \mathrm{fst}$ for the left triangle, and the right triangle commutes trivially. □

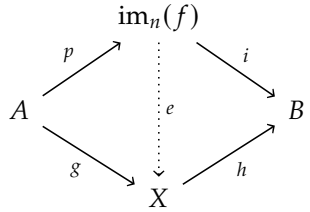


FIGURE 3.13: Visualization of task to construct e.

fig:n-image-univ-prop

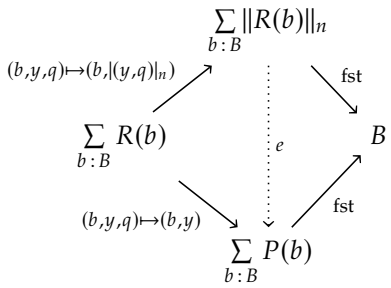


FIGURE 3.14: Visualization of task to construct e, reinterpreted.

fig:n-im-univ-prop-sumB

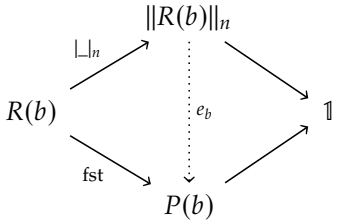


FIGURE 3.15: Taking summands for $b : B$ in Figure 3.14.

fig:n-im-univ-prop-b:B

xca:im_preserves_conn

EXERCISE 3.9.12. Let A be a connected type, B a type, and $f : A \to B$ a function. Then all n-images of f ($n \geq -1$) are connected. ⌟

For the rest of this section, fix some natural number $m > 0$. As for our promised application, we consider the fibers of the mth root map ρ_m. On infinite cycles, this is equivalent to the degree m map of the circle by Exercise 3.8.7, so we have a map $_/m : \mathrm{Cyc}_0 \to \mathrm{Set}$,

which we identify with the family $R_m : S^1 \to \mathrm{Set}$ (Definition 3.6.7) by precomposing with the equivalence $c : S^1 \to \mathrm{Cyc}_0$ from Theorem 3.5.6. For every infinite cycle (X, t), the set X/m has m elements, and the (-1)-image is readily identified with FinSet_m, the groupoid of m-element sets (Definition 2.24.5). But what is the 0-image? The following theorem identifies the 0-image of X/m with Cyc_m.

thm:image-Z-to-Cm

THEOREM 3.9.13. *The 0-image factorization of the map* $_/m : \mathrm{Cyc}_0 \to \mathrm{Set}$ *consists of the type* Cyc_m *and maps* $q : \mathrm{Cyc}_0 \to \mathrm{Cyc}_m$ *and* $r : \mathrm{Cyc}_m \to \mathrm{Set}$. *The map* q *sends any infinite cycle* (X, t) *to the* m*-cycle* $(X/m, \bar{t})$, *where* $\bar{t} : X/m \to X/m$ *maps* $[x]$ *to* $[t(x)]$. *The map* $r : \mathrm{Cyc}_m \to \mathrm{Set}$ *sends any* m*-cycle to its underlying set, so that indeed* $_/m \equiv rq$.

Proof. We need to check that q is 0-connected and that r is 0-truncated.

The latter is direct, since the preimage of r at any set S can be identified as a subset of the set of functions $S \to S$.

To show that q is 0-connected, it suffices to consider the fiber at the standard m-cycle $(\mathbb{m}, \mathrm{s})$. We'll show that this fiber is equivalent to Cyc_0 itself, which is indeed 0-connected. The mediating map is induced by our old friend ρ_m. Indeed, define $\varphi : \mathrm{Cyc}_0 \to q^{-1}(\mathbb{m}, \mathrm{s})$ by $\varphi(X, t) :\equiv (\rho_m(X, t), e^{-1})$, where $e : (\mathbb{m} \times X)/m \xrightarrow{\simeq} \mathbb{m}$ maps $[(k, x)]$ to k.[41] As inverse of φ, define ψ by $\psi((Y, u), e') :\equiv (e'(0), u^m)$, for all $(Y, u) : \mathrm{Cyc}_0$ and $e' : \mathbb{m} \xrightarrow{\simeq} Y/m$. □

[41] To see that e is well defined, keep in mind that $\mathbb{m} \times X$ is equipped with permutation $\sqrt[m]{t}$ by ρ_m. Also, e preserves cycle structure.

EXERCISE 3.9.14. Complete the details of the proof above. ⌟

The theorem and its proof in fact generalize to cycles of all orders.

xca:image-Cmd-to-Cm

EXERCISE 3.9.15. Let d be any order and let $\rho_m : \mathrm{Cyc}_d \to \mathrm{Cyc}_{md}$ be the restriction of the mth root map to Cyc_d. Define $_/m : \mathrm{Cyc}_{md} \to \mathrm{Set}$ as the family of fibers of ρ_m. Show that the 0-image factorization of $_/m$ goes via Cyc_m by lifting $_/m$ to $q : \mathrm{Cyc}_{md} \to \mathrm{Cyc}_m$. In particular, show that the preimage of q at the standard m-cycle can be identified with Cyc_d. ⌟

3.10 *Universal property of* Cyc_n

sec:universal-property-cyc-n

Fix a natural number $n > 0$ and recall the definition of Cyc_n from Definition 3.8.1. This section is devoted to showing that maps out of Cyc_n into a groupoid A are given by the choice of a point together with a symmetry of order n: any map $\mathrm{Cyc}_n \to A$ is uniquely determined by a point $a : A$ together with a symmetry $\sigma : a \xrightarrow{=} a$ such that $\mathrm{refl}_a = \sigma^n$.[42]

[42] Notice that this is a less general result than Theorem 3.1.2, the universal property of the circle, where we don't need to assume that A is a groupoid. That's why $n > 0$ in this section.

Recall that Cyc_n contains the point $\mathrm{pt}_n :\equiv (\mathbb{m}, \mathrm{s})$, i.e., the standard n-cycle. This point has a symmetry $\sigma_n :\equiv (\mathrm{s}^{-1}, !)$ whose second projection is a proof that $\mathrm{s}\,\mathrm{s}^{-1} = \mathrm{s}^{-1}\,\mathrm{s}$. Recall also from Corollary 3.6.16 that all symmetries of pt_n are of the form σ_n^i for $i = 0, \ldots, n-1$.

Given a groupoid A, and a map $f : \mathrm{Cyc}_n \to A$, one can consider $f(\mathrm{pt}_n) : A$ and $\mathrm{ap}_f(\sigma_n) : f(\mathrm{pt}_n) \xrightarrow{=} f(\mathrm{pt}_n)$. Proofs of the equation $\mathrm{refl}_{\mathrm{pt}_n} = \sigma_n^n$ in the set $\mathrm{pt}_n \xrightarrow{=} \mathrm{pt}_n$ are mapped by ap_f to proofs of $\mathrm{refl}_{f(\mathrm{pt}_n)} = \mathrm{ap}_f(\sigma_n)^n$. Hence, the following map is well defined:

$$\mathrm{ev}_{n,A} : (\mathrm{Cyc}_n \to A) \to \sum_{a:A} \sum_{\sigma : a \xrightarrow{=} a} \mathrm{refl}_a = \sigma^n, \quad f \mapsto (f(\mathrm{pt}_n), \mathrm{ap}_f(\sigma_n), !).$$

prop:imp-cyc1-intro-groupoids

THEOREM 3.10.1. *For any groupoid* A, *the map* $\mathrm{ev}_{n,A}$ *above is an equivalence.*

Proof. Let $a : A$ and $\sigma : a \xrightarrow{=} a$ be such that $\mathrm{refl}_a = \sigma^n$ holds. We want to prove that the fiber

$$\sum_{f : \mathrm{Cyc}_n \to A} (a, \sigma, !) \xrightarrow{=} \mathrm{ev}_{n,A}(f)$$

is contractible. Hence we first need to construct a function $f : \mathrm{Cyc}_n \to A$ together with an identification $p : a \xrightarrow{=} f(\mathrm{pt}_n)$ such that $\mathrm{ap}_f(\sigma_n)p = p\sigma$, see the diagram in the margin.

$$\begin{array}{ccc} a & \xrightarrow[=]{p} & f(\mathrm{pt}_n) \\ {\scriptstyle\sigma}\downarrow{\scriptstyle =} & & {\scriptstyle =}\downarrow{\scriptstyle \mathrm{ap}_f(\sigma_n)} \\ a & \xrightarrow[p]{=} & f(\mathrm{pt}_n). \end{array}$$

In order to do so, we will construct a function $f : \mathrm{Cyc}_n \to A$ together with a family of functions $\hat{p}_x : (\mathrm{pt}_n \xrightarrow{=} x) \to (a \xrightarrow{=} f(x))$, parametrized by $x : \mathrm{Cyc}_n$, satisfying $\hat{p}_x(\tau\sigma_n) = \hat{p}_x(\tau)\sigma$ for all $\tau : \mathrm{pt}_n \xrightarrow{=} x$. By setting $p \equiv \hat{p}_{\mathrm{pt}_n}(\mathrm{refl}_{\mathrm{pt}_n})$, we will then have succeeded.

Let's explain why the above indeed suffices. First, a simple path induction on $\alpha : x \xrightarrow{=} x'$ shows that $\mathrm{ap}_f(\alpha)\hat{p}_x(_) = \hat{p}_{x'}(\alpha_)$. On the other hand, instantiating the condition on $\hat{p}$ with $x :\equiv \mathrm{pt}_n$ proves that $\hat{p}_{\mathrm{pt}_n}(\tau\sigma_n) = \hat{p}_{\mathrm{pt}_n}(\tau)\sigma$ for all $\tau : \mathrm{pt}_n \xrightarrow{=} \mathrm{pt}_n$. This leads to the chain of equations:

$$\begin{aligned} \mathrm{ap}_f(\sigma_n)p &\equiv \mathrm{ap}_f(\sigma_n)\hat{p}_{\mathrm{pt}_n}(\mathrm{refl}_{\mathrm{pt}_n}) = \hat{p}_{\mathrm{pt}_n}(\sigma_n \mathrm{refl}_{\mathrm{pt}_n}) \\ &= \hat{p}_{\mathrm{pt}_n}(\mathrm{refl}_{\mathrm{pt}_n}\sigma_n) = \hat{p}_{\mathrm{pt}_n}(\mathrm{refl}_{\mathrm{pt}_n})\sigma \equiv p\sigma \end{aligned}$$

This shows that $\hat{p}$ suffices.

It remains to construct the promised f and $\hat{p}$. For each $x : \mathrm{Cyc}_n$, consider the type (with the product part visualized in the margin)

$$T(x) :\equiv \sum_{b : A} \sum_{\pi : (\mathrm{pt}_n \xrightarrow{=} x) \to (a \xrightarrow{=} b)} \prod_{\tau : \mathrm{pt}_n \xrightarrow{=} x} \pi(\tau\sigma_n) =_{(a \xrightarrow{=} b)} \pi(\tau)\sigma.$$

$$\begin{array}{ccc} (\mathrm{pt}_n \xrightarrow{=} x) & \xrightarrow{-\sigma_n} & (\mathrm{pt}_n \xrightarrow{=} x) \\ {\scriptstyle\pi}\downarrow & & \downarrow{\scriptstyle\pi} \\ (a \xrightarrow{=} b) & \xrightarrow[-\sigma]{} & (a \xrightarrow{=} b) \end{array}$$

We claim that $T(x)$ is contractible for each $x : \mathrm{Cyc}_n$. We then get $f(x)$, $\hat{p}_x$ as well as the proof that $\hat{p}_x$ has the desired property as the three components of the center of contraction, respectively.

To prove that $T(x)$ is contractible for all x in the connected type Cyc_n, it is enough to prove it for $x \equiv \mathrm{pt}_n$. First, the equivalence $i \mapsto \sigma_n^i$ of type $\mathbb{m} \xrightarrow{\simeq} (\mathrm{pt}_n = \mathrm{pt}_n)$ induces an equivalence of type

$$T(\mathrm{pt}_n) \xrightarrow{\simeq} \sum_{b : A} \sum_{\pi : \mathbb{m} \to (a \xrightarrow{=} b)} \prod_{k : \mathbb{m}} \pi(s(k)) =_{(a \xrightarrow{=} b)} \pi(k)\sigma.$$

Now, note that any $\pi : \mathbb{m} \to (a \xrightarrow{=} b)$ such that $\pi(s(k)) = \pi(k)\sigma$ for all $k : \mathbb{m}$ is entirely determined by $\pi(0)$, as then $\pi(i) = \pi(0)\sigma^i$ for all $i : \mathbb{m}$. Moreover, any path q in $a \xrightarrow{=} b$ defines a function $\pi_q : i \mapsto q\sigma^i$ which satisfies $\pi_q(0) = q$ and $\pi_q(s(k)) = \pi_q(k)\sigma$ for all $k : \mathbb{m}$. Thus, evaluation at 0 is an equivalence

$$\mathrm{ev}_0 : \left(\sum_{\pi : \mathbb{m} \to (a \xrightarrow{=} b)} \prod_{k : \mathbb{m}} \pi(s(k)) =_{(a \xrightarrow{=} b)} \pi(k)\sigma \right) \xrightarrow{\simeq} (a \xrightarrow{=} b), \quad \mathrm{ev}_0(\pi, !) :\equiv \pi(0).$$

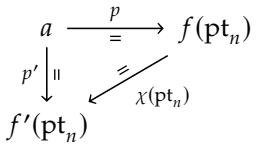
The equivalence ev_0 induces an equivalence of type

$$T(\mathrm{pt}_n) \xrightarrow{\simeq} \left(\sum_{b : A} a \xrightarrow{=} b \right)$$

and hence $T(\mathrm{pt}_n)$ is contractible. This completes the construction of the center of contraction of the fiber $\mathrm{ev}_{n,A}^{-1}(a, \sigma, !)$.

The construction of f is really a special case of the delooping of the abstract group homomorphism $\sigma_n^i \mapsto \sigma^i$ in Section 7.5.

Finally, we prove that the fiber $\mathrm{ev}^{-1}_{n,A}(a,\sigma,!)$ is a proposition. Let $(f,p,!)$ and $(f',p',!)$ be two elements of the fiber. We want to identify them. From the proofs in their third components we infer $p\sigma p^{-1} = \mathrm{ap}_f(\sigma_n)$ and $p'\sigma p'^{-1} = \mathrm{ap}_{f'}(\sigma_n)$, respectively. Define the family of sets $U(x) :\equiv (f(x) \xrightarrow{=} f'(x))$ parametrized by $x : \mathrm{Cyc}_n$. It suffices to find a $\chi : \prod_{x:\mathrm{Cyc}_n} U(x)$ such that the diagram in the margin commutes.



The element $\tau :\equiv p'p^{-1} : U(\mathrm{pt}_n)$ is peculiar in that $\mathrm{trp}^U_q(\tau) = \tau$ for all $q : \mathrm{pt}_n \xrightarrow{=} \mathrm{pt}_n$. Indeed, we use once again that symmetries of pt_n in Cyc_n are of the form σ^i_n and we calculate:

$$\mathrm{trp}^U_{\sigma^i_n}(\tau) = \mathrm{ap}_{f'}(\sigma^i_n)\cdot\tau\cdot\mathrm{ap}_f(\sigma^i_n)^{-1} = p'\sigma^i p'^{-1}\tau p\sigma^{-i}p^{-1} = p'p^{-1}$$

Now it is easy to prove that the following type is contractible:

$$V(x) :\equiv \sum_{\alpha:U(x)} \prod_{r:\mathrm{pt}_n \xrightarrow{=} x} \alpha = \mathrm{trp}^U_r(\tau)$$

To do so, we use the connectedness of Cyc_n and verify the contractibility of $V(\mathrm{pt}_n)$. Clearly, $(\tau,!)$ is a center of contraction by the peculiarity of τ. Also, if α and β are elements of $V(\mathrm{pt}_n)$, then $\alpha = \beta$ by taking $r \equiv \mathrm{refl}_{pt_n}$. Now χ is defined as the function mapping x to the center of contraction of $V(x)$, so that $\chi(\mathrm{pt}_n) = \tau$ as we wanted. □

The construction of χ is really an ad hoc version of the following fact: for any G-set X, the type of fixed points of X is equivalent to the type of sections of $\sum_{z:BG} X(z) \to BG$. If we move this section forward, one can rewrite it as such. TODO

As a direct corollary, we can classify the connected set bundles over Cyc_n for finite orders n. Indeed, the corresponding families $S : \mathrm{Cyc}_n \to \mathrm{Set}$ are precisely those cycles (X,t) with $t^n = \mathrm{id}$, i.e., whose order divides n. If we restrict to decidable connected set bundles, equivalently, decidable cycles, these are the usual finite cycles with order dividing n.

3.11 *Getting our cycles in order*

sec:cycles-order

TODO: Exposition and figures

Exercise 3.11.1. Prove that if $(X,t),(Y,u)$ are cycles, $x_0 : X$, then the type of maps $f : (X,t) \to (Y,u)$ is equivalent to $P \times Y$, where $P :\equiv (\mathrm{ord}(u) \mid \mathrm{ord}(t)) \equiv (H_t \subseteq H_u)$. ⌟

Thus, an order p divides an order q if and only if there is a map of cycles from a cycle of order q to a cycle of order p.

Theorem 3.11.2. *The partially ordered set* $(\mathrm{Order}, |)$ *is a lattice with least element the finite order* 1 *and greatest element the infinite order, represented by the number* 0*, and meets and joins given by "gcd" and "lcm", respectively.*

subgroups of C_n: C_k where $k|n$, connected set bundles of Cyc_n.

3.11.3 *More TODO*

- Classify connected set bundles over Cyc_n.
- Universal property of Cyc_n among groupoids.
- Bijective proof of $mn = \mathrm{lcm}(m,n)\times\gcd(m,n)$ via the product of cycles. Chinese remainder stuff.
- Somehow sneak in totatives and automorphisms of cyclic groups?

4
Groups, concretely

ch.groups

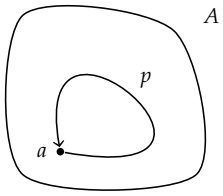
An identity type is not just any type: in the previous sections we have seen that the identity type $a \xrightarrow{=}_A a$ reflects the "symmetries" of an element a in a type A.[1] Symmetries have special properties. For instance, you can rotate a square by 90°, and you can reverse that motion by rotating it by −90°. Symmetries can also be composed, and this composition respects certain rules that hold in all examples. One way to study the concept of "symmetries" would be to isolate the common rules for all our examples, and to show, conversely, that anything satisfying these rules actually *is* an example.

With inspiration of geometric and algebraic origins, it became clear to mathematicians at the end of the 19th century that the properties of such symmetries could be codified by saying that they form an abstract *group*. In Section 2.5 we saw that equality is "reflexive, symmetric and transitive" – implemented by operations refl_a, $\mathsf{symm}_{a,b}$ and $\mathsf{trans}_{a,b,c}$, and an abstract group is just a set with such operations satisfying appropriate rules.

We attack the issue more concretely: instead of focusing on the abstract properties, we bring the type exhibiting the symmetries to the fore. This type is called the *classifying type* of the group. The axioms for an abstract group follow from the rules for identity types, without us needing to impose them. We will show in Chapter 7 that the two approaches give the same end result.

In this chapter we lay the foundations and provide some basic examples of groups.

[1] Since the symmetries $p : a \xrightarrow{=}_A a$ are paths that start and end at the point $a : A$, we also call them *loops* at a, or *automorphisms* of a.



4.1 *Brief overview of the chapter*

In Section 4.2 we give the formal definition of a group along with some basic examples. In Section 4.3 we expand on the properties of a group and compare these with those of an abstract group. In Section 4.4 we explain how groups map to each other through "homomorphisms" (which to us are simply given by pointed maps), and what this entails for the identity types: the preservation of the abstract group properties. As an important example, we study the sign homomorphism in Section 4.5, which also provides us with the alternating groups.

In most of our exposition we make the blanket assumption that the identity type in question is a set, but in Section 4.7 we briefly discuss ∞-groups, where this assumption is dropped.

4.2 *The type of groups*

sec:typegroup

In order to motivate the formal definition of a group we revisit some types that we have seen in earlier chapters, paying special attention to the symmetries in these types.

ex:base=base

Example 4.2.1. We defined the circle S^1 in Definition 3.1.1 by declaring that it has a point $\bullet$ and an identification ("symmetry") $\circlearrowleft : \bullet \xrightarrow{=} \bullet$. In Corollary 3.4.6 we proved that $\bullet \xrightarrow{=} \bullet$ is equivalent to the set $\mathbb{Z}$ (of integers), where $n \in \mathbb{Z}$ corresponds to the n-fold composition of $\circlearrowleft$ with itself (which works for both positive and negative n). We can think of this as describing the symmetries of $\bullet$ as follows. We have one "generating symmetry" $\circlearrowleft$, and this symmetry can be composed with itself any number of times, giving a symmetry for each integer. Composition of symmetries here corresponds to addition of integers.

The circle is an efficient packaging of the "group" of integers, for the declaration of $\bullet$ and $\circlearrowleft$ not only gives the *set* $\mathbb{Z}$ of integers, but also the addition operation. ⌟

Example 4.2.2. Recall the finite set $2 : \mathrm{FinSet}_2$ from Definition 2.24.1, containing two elements. According to Exercise 2.13.3, the identity type $2 \xrightarrow{=} 2$ has exactly two distinct elements, refl_2 and swap, and doing swap twice yields refl_2. We see that these are all the symmetries of a two point set you'd expect to have: you can let everything stay in place (refl_2); or you can swap the two elements (swap). If you swap twice, the result leaves everything in place. The pointed type FinSet_2 (of "finite sets with two elements"), with 2 as the base point, is our embodiment of these symmetries, i.e., they are the elements of $2 \xrightarrow{=} 2$.

Observe that, by the induction principle of S^1, there is an interesting function $S^1 \to \mathrm{FinSet}_2$, sending $\bullet : S^1$ to $2 : \mathrm{FinSet}_2$ and $\circlearrowleft$ to swap. We saw this already in Figure 3.2. ⌟

Note that the types S^1 and FinSet_2 in the examples above are groupoids. For an arbitrary type A and an element $a : A$, the symmetries of a in A form an ∞-group, cf. Section 4.7 below. However, in elementary texts it is customary to restrict the notion of a group to the case when $a \xrightarrow{=}_A a$ is a *set*, as we will do, starting in Section 4.3. This makes things considerably easier: if are we given two elements $g, h : a \xrightarrow{=}_A a$, then the identity type $g \xrightarrow{=} h$ is a proposition (and we can simply write $g = h$). That is, g can be equal to h in at most one way, and questions relating to uniqueness of identification will never present a problem.

The examples of groups that Klein and Lie were interested in often had more structure on the set $a \xrightarrow{=}_A a$, for instance a topology or a smooth structure. For such a group it makes sense to look at smooth maps from the real numbers to $a \xrightarrow{=}_A a$, or to talk about a convergent sequence of symmetries of a. [2] See Appendix A for a brief summary of the history of groups.

rem:heap-preview

Remark 4.2.3. The reader may wonder about the status of the identity type $a \xrightarrow{=}_A a'$ where $a, a' : A$ are different elements. One problem is of course that if $p, q : a \xrightarrow{=}_A a'$, there is no obvious way of composing p and q to get another element in $a \xrightarrow{=}_A a'$. Another problem is that $a \xrightarrow{=}_A a'$ does not have a distinguished element, such as $\mathrm{refl}_a : a \xrightarrow{=}_A a$.[3] Given an $f : a \xrightarrow{=}_A a'$ we can use transport along f to compare $a \xrightarrow{=}_A a'$ with

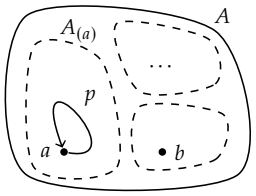
[2] Such groups give rise to ∞-groups by converting continuous (or smooth) symmetries of a in A parametrized by the continuous (or smooth) real interval, into identifications, as described already in Footnote 14 in Chapter 2. Then also smooth or continuous paths in $a \xrightarrow{=}_A a$ turn into identifications of symmetries. See also Section B.3.

[3] The type $a \xrightarrow{=}_A a'$ does have an interesting *ternary* composition, mapping p, q, r to $pq^{-1}r$. A set with this kind of operation is called a *heap*, and we'll explore heaps further in Section 7.9.

$a \xrightarrow{=}_A a$ (much as affine planes can be compared with the standard plane or a finite dimensional real vector space is isomorphic to some Euclidean space), but absent the existence and choice of such an f the identity types $a \xrightarrow{=}_A a'$ and $a \xrightarrow{=}_A a$ are different animals. We will return to this example in Section 7.9. ⌟

rem:whypointedconngpoid

Remark 4.2.4. As a consequence of Lemma 2.20.4, the inclusion of the component $A_{(a)} :\equiv \sum_{x:A} \| a \xrightarrow{=} x \|$ into A (i.e., the first projection) induces an equivalence of identity types from $(a, !) \xrightarrow{=}_{A_{(a)}} (a, !)$ to $a \xrightarrow{=}_A a$. This means that, when considering the loop type $a \xrightarrow{=}_A a$, "only the elements $x : A$ with x merely equal to a are relevant". To avoid irrelevant extra components, we should consider only *connected* types A (cf. Definition 2.16.8).



Also, our preference for $a \xrightarrow{=}_A a$ to be a *set* indicates that we should consider only the connected types A that are *groupoids*. ⌟

def:pt-conn-groupoid

Definition 4.2.5. The type of *pointed, connected groupoids* is the type

$$\mathcal{U}_*^{=1} :\equiv \sum_{A:\mathcal{U}} (A \times \mathrm{isConn}(A) \times \mathrm{isGrpd}(A)). \qquad ⌟$$

The meaning of the superscript "= 1" can be explained as follows: We also define

$$\mathcal{U}^{\leq 1} :\equiv \text{Groupoid} :\equiv \sum_{A:\mathcal{U}} \mathrm{isGrpd}(A)$$

to emphasize that groupoids are 1-types; the type of connected types is defined as follows.

$$\mathcal{U}^{>0} :\equiv \sum_{A:\mathcal{U}} \mathrm{isConn}(A)$$

Similar notations with a subscript "$*$" indicate pointed types.

xca:defgroup

Exercise 4.2.6. Given a type A and an element $a : A$, show that A is connected if and only if the proposition $\prod_{x:A} \| a \xrightarrow{=}_A x \|$ holds. Show furthermore that A is a groupoid if and only if the type $a \xrightarrow{=}_A a$ is a set. Conclude by showing that the type $\mathcal{U}_*^{=1}$ is equivalent to the type

$$\sum_{A:\mathcal{U}} \sum_{a:A} \left(\left(\prod_{x:A} \| a \xrightarrow{=}_A x \| \right) \times \mathrm{isSet}(a \xrightarrow{=}_A a) \right). \qquad ⌟$$

Remark 4.2.7. We shall refer to a pointed connected groupoid (A, a, p, q) simply by the pointed type $X :\equiv (A, a)$. There is no essential ambiguity in this, for the types $\mathrm{isConn}(A)$ and $\mathrm{isGrpd}(A)$ are propositions (Lemma 2.15.4 and Lemma 2.15.7), and so the witnesses p and q are unique. ⌟

We are now ready to define the type of groups.

def:typegroup

Definition 4.2.8. The *type of groups* is a wrapped copy (see Section 2.12.8) of the type of pointed connected groupoids $\mathcal{U}_*^{=1}$,

$$\mathrm{Group} :\equiv \mathrm{Copy}_{\underline{\Omega}}(\mathcal{U}_*^{=1}),$$

with constructor $\underline{\Omega} : \mathcal{U}_*^{=1} \to \mathrm{Group}$. [4] A *group* is an element of Group. ⌟

def:classifying-type

Definition 4.2.9. We write $\mathrm{B} : \mathrm{Group} \to \mathcal{U}_*^{=1}$ for the destructor associated with $\mathrm{Copy}_{\underline{\Omega}}(\mathcal{U}_*^{=1})$. For $G : \mathrm{Group}$, we call BG the *classifying type* of G.[5] Moreover, the elements of BG will be referred to as the *shapes of* G, and we define the *designated shape of* G by setting $\mathrm{sh}_G :\equiv \mathrm{pt}_{BG}$, i.e., the designated shape of G is the base point of its classifying type, see Definition 2.21.1. ⌟

def:looptype

Definition 4.2.10. Given a pointed type $X \equiv (A, a)$, we define $\Omega X :\equiv (a \xrightarrow{=}_A a)$, i.e., the type of the symmetries of $a : A$. The type ΩX is pointed at refl_a. ⌟

def:group-symmetries

Definition 4.2.11. Let G be a group. We regard every group as a group of symmetries, and thus we refer to the elements of ΩBG as the *symmetries in* G; they are the symmetries of the designated shape sh_G of G. We adopt the notation

$$UG :\equiv \Omega BG$$

[4] The reader may ask why we use $\underline{\Omega}$, which only makes a wrapped copy of each $(A, s, p, q) : \mathcal{U}_*^{=1}$. The answer is that flatly defining groups as their classifying types would be confusing. Using $\underline{\Omega}$ we avoid awkward terminology such as ' 'the group of the integers is the circle". The symbol $\underline{\Omega}$ is inspired by Ω in Definition 4.2.10, which in Section 4.3 will be used to recover the traditional concept of a group. Recall also the example of the negated natural numbers $\mathbb{N}^-$ from Section 2.12.8: Its elements are $-n$ for $n : \mathbb{N}$ to remind us how to think about them. And the same applies to Group: Its elements are $\underline{\Omega} X$ for $X : \mathcal{U}_*^{=1}$ to remind us how to think about them.

[5] As a notational convention we always write the "B" so that it sits next to and matches the shape of its operand. You see immediately the typographical reason behind this convention: The italic letters B, G get along nicely, while the roman B would clash with its italic friend G if we wrote BG instead.

for the type of symmetries in G; it is a set.[6] (Notice the careful distinction above between the phrases "*symmetries in*" and "*symmetries of*".) ⌟

[6] Taking the symmetries in a group thus defines a map $\mathrm{U} : \mathrm{Group} \to \mathrm{Set}$, with $\underline{\Omega} X \mapsto \Omega X$. Just as with "B", we write the "U" so that it matches the shape of its operand.

def:finite-group

Definition 4.2.12. A group G is a *finite group* if the set UG is finite. For any finite group G we denote the number of symmetries in G by $\#(G) :\equiv \#(UG)$, also called the *cardinality* of G. ⌟

rem:aut

Remark 4.2.13. As noted in Section 2.12.8, the constructor and destructor pair forms an equivalence $\mathrm{Group} \simeq \mathcal{U}_*^{=1}$. The type $\mathcal{U}_*^{=1}$ is a subtype of $\mathcal{U}_*$, so once you know that a pointed type X is a connected groupoid, you also know that X is the classifying type of a group, namely $G :\equiv \underline{\Omega} X$.

Note that the equivalence also entails that identifications (of groups) of type $G \xrightarrow{=} H$ are equivalent to identifications (of pointed types) of type $BG \xrightarrow{=} BH$. ⌟

rem:BG-convention

Remark 4.2.14. Defining a function $f : \prod_{G:\mathrm{Group}} T(G)$, where $T(G)$ is a type parametrized by $G : \mathrm{Group}$, amounts to defining $f(G)$ for $G \equiv \underline{\Omega} X$, where X is a pointed connected groupoid, namely the classifying type BG.[7] ⌟

[7] If you are bothered by the convention to write the classifying type of G in *italic* like a variable, you can either think of BG as a locally defined variable denoting the classifying type that is defined whenever a variable G of type Group is introduced, or you can imagine that whenever such a G is introduced (with the goal of making a construction or proving a proposition), we silently apply the induction principle to reveal a wrapped variable $BG : \mathcal{U}_*^{=1}$.

Frequently we want to consider the symmetries $\Omega(A, a)$ of some element a in some groupoid A, so we introduce the following definition.

def:automorphism-group

Definition 4.2.15. For a groupoid A with a specified point a, we define the *automorphism group* of $a : A$ by

$$\mathrm{Aut}_A(a) :\equiv \underline{\Omega}(A_{(a)}, (a, !)),$$

i.e., $\mathrm{Aut}_A(a)$ is the group with classifying type $\mathrm{BAut}_A(a) \equiv (A_{(a)}, (a, !))$, the connected component of A containing a, pointed at a. ⌟

rem:symmetriesofnonconnectedgroupoids

Remark 4.2.16. If A is connected, then $\mathrm{fst} : A_{(a)} \to A$ is an equivalence between the pointed types $(A_{(a)}, (a, !))$ and (A, a), pointed by refl_a. Consequently, for any $G \equiv \underline{\Omega}(A, a) : \mathrm{Group}$, we have an identification of type $G \xrightarrow{=} \mathrm{Aut}_A(a)$.

In other words, for any $G \equiv \underline{\Omega} BG$, we have an identification $G \xrightarrow{=} \mathrm{Aut}_{BG}(\mathrm{sh}_G)$, of G with the automorphism group of the designated shape $\mathrm{sh}_G : BG$. ⌟

4.2.17 *First examples*

sec:firstgroupexamples
ex:circlegroup

Example 4.2.18. The circle S^1, which we defined in Definition 3.1.1, is a connected groupoid (Lemma 3.1.6, Corollary 3.4.6) and is pointed at $\bullet$. The identity type $\bullet \xrightarrow{=}_{\mathrm{S}^1} \bullet$ is equivalent to the set of integers Z and composition corresponds to addition. This justifies our definition of the *group of integers* as

$$\mathbb{Z} :\equiv \underline{\Omega}(\mathrm{S}^1, \bullet).$$

In other words, the classifying type of $\mathbb{Z}$ is $\mathrm{B}\mathbb{Z} :\equiv \mathrm{S}^1$, pointed at $\bullet$. Recall from Remark 4.2.16 that there is then a canonical identification of type $\mathbb{Z} \xrightarrow{=} \mathrm{Aut}_{\mathrm{S}^1}(\bullet)$. It is noteworthy that along the way we gave several versions of the circle, each of which has its own merits. For example, the type of infinite cycles in Definition 3.5.3 and Theorem 3.5.6,

$$\mathrm{InfCyc} \equiv \sum_{X:\mathcal{U}} \sum_{t:X\to X} \|(\mathrm{Z}, \mathrm{s}) \xrightarrow{=} (X, t)\|. \qquad ⌟$$

xca:groups

Exercise 4.2.19. Use various results from Chapter 3 to construct two different identifications of type $\mathbb{Z} \xrightarrow{=} \mathrm{Aut}_{\mathrm{Cyc}}(\mathrm{Z}, \mathrm{s})$. ⌟

ex:groups

Example 4.2.20. Apart from the circle, there are some important groups that come almost for free: namely the automorphisms of specific elements in the groupoid Set, and even one in the groupoid Prop.

ex:trivgroup

(1) Recall that True, and hence True $\xrightarrow{=}$ True, is contractible. Hence $\mathrm{Aut}_{\mathrm{Prop}}(\mathrm{True})$ is a group called the *trivial group*, denoted by $\mathbb{1}$. In fact, for any proposition P we can also identify the trivial group with $\mathrm{Aut}_{\mathrm{Prop}}(P)$, see Exercise 4.2.21. Unlike Prop, the type True is connected, so we can also identify the trivial group with $\underline{\Omega}(\mathrm{True}, \mathrm{triv})$, or with $\underline{\Omega}(C, c)$ for any contractible type C and element $c : C$, or with $\mathrm{Aut}_S(x)$ for any set S and element $x : S$.[8]

ex:permgroup

(2) If $n : \mathbb{N}$, then the *permutation group of n letters* (also known as the *symmetric group of degree n*) is

$$\Sigma_n :\equiv \mathrm{Aut}_{\mathrm{Set}}(\mathbb{m}).$$

The classifying type is thus $\mathrm{B}\Sigma_n \equiv (\mathrm{FinSet}_n, \mathbb{m})$, where $\mathrm{FinSet}_n \equiv \mathrm{Set}_{(\mathbb{m})}$ is the groupoid of sets of cardinality n (cf. 2.24.5).

Again, we can also identify the group Σ_n with $\mathrm{Aut}_{\mathrm{FinSet}}(\mathbb{m})$ (by Exercise 4.2.21), with $\mathrm{Aut}_{\mathrm{FinSet}_n}(\mathbb{m})$ (by Remark 4.2.16), or even with $\mathrm{Aut}_{\mathcal{U}}(\mathbb{m})$ (by stretching the definition of Aut, using that $\mathcal{U}_{(\mathbb{m})}$ is a connected groupoid, see Remark 4.7.5).

ex:genpermgroup

(3) More generally, if S is a set, is there a pointed connected groupoid (A, a) so that $a \xrightarrow{=}_A a$ models all the "permutations" $S \xrightarrow{=}_{\mathrm{Set}} S$ of S? Again, the only thing wrong with the groupoid Set of sets is that Set is not connected. The *group of permutations of S* is defined to be

$$\Sigma_S :\equiv \mathrm{Aut}_{\mathrm{Set}}(S),$$

with classifying type $\mathrm{B}\Sigma_S \equiv (\mathrm{Set}_{(S)}, S)$. ⌟

xca:group-example-details

Exercise 4.2.21. Show that $\mathrm{Aut}_{\mathrm{Prop}}(P)$ is a trivial group for any proposition P. Verify that Σ_0, Σ_1, and Σ_{False} are all trivial groups. Using Definition 2.24.1, give identifications of type $\mathrm{Aut}_{\mathrm{FinSet}}(\mathbb{m}) \xrightarrow{=} \Sigma_n$ for $n : \mathbb{N}$. Also, give an identification of type $\mathrm{Aut}_{\mathrm{Set}}(\mathbb{N}) \xrightarrow{=} \mathrm{Aut}_{\mathrm{Set}}(\mathrm{Z})$. ⌟

ex:cyclicgroups

Example 4.2.22. In Corollary 3.6.16 we studied the symmetries of the standard m-cycle $(\mathbb{m}, \mathrm{s})$ for m a positive integer, and showed that there were m different such symmetries. Moreover, we showed that these symmetries can be identified with the elements $0, 1, \ldots, m-1$ of $\mathbb{m}$ (according to the image of 0), and under this correspondence composition of symmetries correspond to addition modulo m, with 0 the identity. Note that all of these can be obtained from 1 under addition. With Cyc, Cyc_m from Definition 3.6.3, 3.8.1, the *cyclic group of order m* is thus defined to be

$$\mathrm{C}_m :\equiv \mathrm{Aut}_{\mathrm{Cyc}}(\mathbb{m}, \mathrm{s}),$$

with classifying type $\mathrm{BC}_m \equiv (\mathrm{Cyc}_m, (\mathbb{m}, \mathrm{s}))$.[9]

By using univalence on the equivalences of Theorem 3.3.8, we get a chain of identifications

$$\begin{array}{ccc} \mathrm{C}_m & \xrightarrow{=} & \mathrm{Aut}_{\sum_{X:\mathrm{Set}}(X\to X)}(\mathbb{m}, \mathrm{s}) \\ & & \downarrow{\scriptstyle \|} \\ & & \mathrm{Aut}_{\mathrm{SetBundle}(\mathrm{S}^1)}(\mathrm{S}^1, \delta_m) \xrightarrow{=} \mathrm{Aut}_{\mathrm{S}^1\to\mathrm{Set}}(R_m), \end{array}$$

[8] This note is for those who worry about size issues – a theme we usually ignore in our exposition. Recall from Section 2.3 the chain of universes $\mathcal{U}_0 : \mathcal{U}_1 : \mathcal{U}_2 : \ldots$ such that for each i all types in $\mathcal{U}_i$ are also in $\mathcal{U}_j$ for all $j > i$. Let $\mathrm{Prop}_0 :\equiv \sum_{P:\mathcal{U}_0} \mathrm{isProp}(P)$ be the type of propositions in $\mathcal{U}_0$. Then $\mathrm{True} : \mathrm{Prop}_0$ and $\mathrm{Prop}_0 : \mathcal{U}_1$ (because the sum is taken over $\mathcal{U}_0$). In order to accommodate the trivial group $\mathrm{Aut}_{\mathrm{Prop}_0}(\mathrm{True})$, the universe "$\mathcal{U}$" appearing as a subscript of the first Σ-type in Definition 4.2.5, reappearing later in Definition 4.2.8 of the type of groups, needs to be at least as big as $\mathcal{U}_1$. If $\mathcal{U}$ is taken to be $\mathcal{U}_1$, then the type Group of groups will not be in $\mathcal{U}_1$, but in the bigger universe $\mathcal{U}_2$. Exercise 4.2.35 below asks you to verify that Group is a (large) groupoid. If we then choose some group $G : \mathrm{Group}$ and look at its group of automorphisms, $\mathrm{Aut}_{\mathrm{Group}}(G)$, this will be an element of Group only if the universe $\mathcal{U}$ in the definition of Group is at least as big as $\mathcal{U}_2$. Clearly, this doesn't stop and so we also need an ascending chain of types of groups:

$$\mathrm{Group}_i :\equiv \mathrm{Copy}_{\underline{\Omega}}\left((\mathcal{U}_i)_*^{=1}\right) : \mathcal{U}_{i+1}.$$

Any group we encounter will be an element of Group_i for i large enough. As a matter of fact, the trivial group $\mathrm{Aut}_{\mathrm{True}}(\mathrm{triv})$ is an element of Group_0. The Replacement **??** 2.19.4 often allows us to conclude that a group G belongs to Group_0. This is the case for Σ_S, for $S : \mathrm{Set}_0$, and for $\mathrm{Aut}_{\mathrm{Group}}(G)$, for $G : \mathrm{Group}_0$, as we invite the reader to check. (Hint: use Exercise 2.19.5.) However, even with this principle there are groups that only belong to Group_i for $i > 0$ large enough.

Issues concerning universes are nontrivial and important, but in this text we have chosen to focus on other matters.

[9] Note that the cyclic group of order 1 is the trivial group, the cyclic group of order 2 is equivalent to the symmetric group Σ_2: there is exactly one nontrivial symmetry f and f^2 is the identity. When $m > 2$ the cyclic group of order m is a group that does not appear elsewhere in our current list. In particular, the cyclic group of order m has only m different symmetries, whereas we will see that the group of permutations Σ_m has $m! = 1 \cdot 2 \cdot \cdots \cdot m$ symmetries.

where $\delta_m : S^1 \to S^1$ is the degree m map, and $R_m : S^1 \to \mathrm{Set}$ is the mth power bundle from Definition 3.6.7.

For reasons that will become clear later (Definition 9.5.8), we introduce another name for the cyclic group of order m, corresponding to the last step above, namely,

$$\mathbb{Z}/m\mathbb{Z} :\equiv \mathrm{Aut}_{S^1 \to \mathrm{Set}}(R_m). \qquad \lrcorner$$

ex:Cn

Example 4.2.23. There are other (beside the symmetries of the m-cycle and of the m-fold set bundle) ways of obtaining the cyclic group of order m, which occasionally are more convenient. The prime other interpretation comes from thinking about the symmetries of the m-cycle in a slightly different way. We can picture the m-cycle as consisting of m points on a circle, e.g., as the set of mth roots of unity in the complex plane, as shown in Figure 4.1.

Any cyclic permutation is in particular a permutation of the m-element set underlying the cycle. This manifests itself as the projection map $\mathrm{pr} : \mathrm{Cyc}_m \to \mathrm{FinSet}_m : ((X, t), !) \mapsto (X, !)$,[10] equivalently, using the notation introduced above, $\mathrm{pr} : \mathrm{BC}_m \to \mathrm{B}\Sigma_m$, where the group $\Sigma_m \equiv \mathrm{Aut}_{\mathrm{Set}}(\mathbb{m})$ is that of *all* permutations of the set $\mathbb{m}$. This projection map, whose fiber at $X : \mathrm{B}\Sigma_m$ can be identified with the set $\sum_{t : X \to X} \|(X, t) \xrightarrow{\sim} (\mathbb{m}, \mathrm{s})\|$, captures C_m as a "subgroup" of the permutations, namely the cyclic ones, corresponding to the fact that the shapes of C_m (i.e., the elements of BC_m) are those of Σ_m together with the extra structure of the "cyclic ordering" determined by t.

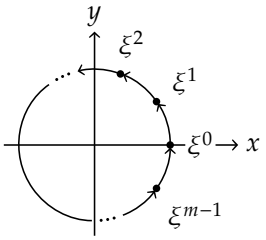


Figure 4.1: The m-cycle as the mth roots of unity. (Here $\xi = e^{2\pi i/m}$ is a primitive m^{th} root.)

fig:m-cycle-roots

[10] In the terminology of Section 2.27, this map forgets the cycle structure on the underlying set.

But how do we capture the other aspect of C_m, mentioned in Example 4.2.22, that all the cyclic permutations can be obtained by a single generating one? When thinking of the mth roots of unity as in Figure 4.1, we can take complex multiplication by ξ to be the generating symmetry.

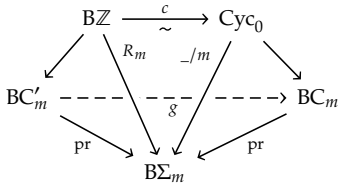
The key insight is provided by the function $R_m : S^1 \to \mathrm{FinSet}_m$ from Definition 3.6.7, with $R_m(\bullet) :\equiv \mathbb{m}$ and $R_m(\circlearrowleft) :\equiv \mathrm{s}$, picking out exactly the cyclic permutation $\mathrm{s} : \mathbb{m} \xrightarrow{\sim} \mathbb{m}$ (and its iterates) among all permutations. Using our new notation, we can also write this as

$$R_m : \mathrm{B}\mathbb{Z} \to \mathrm{B}\Sigma_m.$$

Set truncation (Definition 2.22.4) provides us with a tool for capturing only the symmetries in FinSet_m hit by R_m: the (in language to come) subgroup of the permutation group generated by the cyclic permutation s is the group

$$\mathrm{C}'_m :\equiv \underline{\Omega}(\mathrm{BC}'_m, \mathrm{sh}_{\mathrm{C}'_m}),$$

where $\mathrm{BC}'_m :\equiv \sum_{X : \mathrm{FinSet}_m} \|R_m^{-1}(X)\|_0$ and $\mathrm{sh}_{\mathrm{C}'_m} :\equiv (\mathbb{m}, |(\bullet, \mathrm{refl}_{\mathbb{m}})|_0)$. That is, BC'_m is the 0-image of R_m in the sense of Section 3.9, and is in particular a pointed connected groupoid. Since we have a factorization of R_m as the equivalence $c : S^1 \xrightarrow{\sim} \mathrm{Cyc}_0$ followed by the map $_/m : \mathrm{Cyc}_0 \to \mathrm{B}\Sigma_m$, and since Cyc_m is the 0-image of the latter by Theorem 3.9.13, we get a uniquely induced pointed equivalence $g : \mathrm{BC}'_m \xrightarrow{\sim}_* \mathrm{BC}_m$.[11] This identifies the set $\|R_m^{-1}(X)\|_0$ with the set of cycle structures on the m-element set X. $\lrcorner$



[11] More precisely, but using language not yet established: C_m is both isomorphic to $\mathbb{Z}/m\mathbb{Z}$, the "quotient group" (cf. Definition 9.5.8) of $\mathbb{Z}$ by the "kernel" (cf. Definition 9.3.2) induced by R_m, and to C'_m, which is the corresponding "image" (cf. Section 9.3.13). This pattern will later be captured in Theorem 9.9.2.

Exercise 4.2.24. Show that the set truncation of $R_2^{-1}(2)$ is contractible. This reflects that C_2 and Σ_2 can be identified.[12] $\lrcorner$

xca:CC2isSG2

[12] We will later see that $\mathrm{C}_2 \xrightarrow{\sim}_{\mathrm{Group}} \Sigma_2$ is contractible.

xca:HmLoopCm

Exercise 4.2.25. Elaborate the symmetries of $\mathrm{sh}_{C'_m} \equiv (\mathbb{m}, |(\bullet, \mathrm{refl}_{\mathbb{m}})|_0)$ in BC'_m and show that they are indeed the permutations of $\mathbb{m}$ than can be generated by $R_m(\circlearrowleft)$, that is, by s. ⌟

ex:productofgroups

Example 4.2.26. If you have two groups G and H, their *product* $G \times H$ is given by taking the product of their classifying types:[13]

$$G \times H :\equiv \underline{\Omega}(BG \times BH)$$

[13] Note that $\mathrm{B}(G \times H) \equiv BG \times BH$ is pointed at $\mathrm{sh}_{G\times H} \equiv (\mathrm{sh}_G, \mathrm{sh}_H)$.

For instance, $\Sigma_2 \times \Sigma_2$ is called the *Klein four-group* or *Vierergruppe*, because it has four symmetries. In general, Lemma 2.11.1 gives an identification $\mathrm{U}(G \times H) \xrightarrow{\simeq} UG \times UH$. ⌟

xca:klein-not-cyclic

Exercise 4.2.27. Show that we cannot identify C_4 and $\Sigma_2 \times \Sigma_2$, i.e., the Klein four-group is not a cyclic group. ⌟

ex:bigproductofgroups

Example 4.2.28. If S is an n-element finite set, $n : \mathbb{N}$, and $G : S \to \mathrm{Group}$ is an S-indexed family of groups, then we can likewise form the *product* of the family, by taking the product of the classifying types:

$$\prod_{s:S} G(s) :\equiv \underline{\Omega}\left(\prod_{s:S} BG(s), s \mapsto \mathrm{sh}_{G(s)}\right)$$

Function Extensionality, **??** 2.9.18, says that that the function ptw of Definition 2.6.4 gives an equivalence:

$$\mathrm{ptw} : \mathrm{U}\left(\prod_{s:S} G(s)\right) \xrightarrow{\simeq} \prod_{s:S} UG(s)$$ ⌟

ircabigproductfamily

Exercise 4.2.29. (1) Show that a finite product of connected groupoids is again connected, so that the above definition makes sense.[14]

[14] For infinite products, we can either use the Axiom of Choice, **??** B.4.1, or take the connected component of base point, $s \mapsto \mathrm{sh}_{G(s)}$.

(2) Show that when S is identified with a standard 2-element set such as Bool, then the product of an S-indexed family of groups reduces to the binary product of Example 4.2.26. ⌟

Remark 4.2.30. In Lemma 4.3.3 we will see that the identity type of a group satisfies a list of laws justifying the name "group" and we will later show in Lemma 7.4.7 that groups are uniquely characterized by these laws. ⌟

Some groups have the property that the order you compose the symmetries is immaterial. The prime example is the group of integers $\mathbb{Z} \equiv \underline{\Omega}(\mathrm{S}^1, \bullet)$. Any symmetry is of the form $\circlearrowleft^n$ for some integer n, and if $\circlearrowleft^m$ is also a symmetry, then $\circlearrowleft^n \circlearrowleft^m = \circlearrowleft^{n+m} = \circlearrowleft^{m+n} = \circlearrowleft^m \circlearrowleft^n$.

Such cases are important enough to have their own name:

def:abgp

Definition 4.2.31. A group G is *abelian* if all symmetries commute, in the sense that the proposition

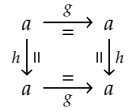
$$\mathrm{isAb}(G) :\equiv \prod_{g,h:UG} gh = hg$$

is true. In other words, the type of abelian groups is

$$\mathrm{AbGroup} :\equiv \sum_{G:\mathrm{Group}} \mathrm{isAb}(G).$$ ⌟

exer:first examples

Exercise 4.2.32. Show that symmetric group Σ_2 is abelian, but that Σ_3 is not. Show that if G and H are abelian groups, then so is their product $G \times H$. ⌟

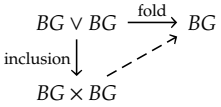


We can visualize symmetries g and h commuting with each other in a group $A \equiv \underline{\Omega}(A, a)$ by the picture in the margin; going from (upper left hand corner) a to (lower right hand corner) a by either composition gives the same result.

rem:whatAREabeliangroups

REMARK 4.2.33. Abelian groups have the amazing property that their classifying types are themselves identity types (of certain 2-types). This can be used to give a very important characterization of what it means to be abelian. We will return to this point in Section 12.2.

Alternatively, the reference to underlying symmetries in the definition of abelian groups is avoidable using the "one point union" of pointed types $X \vee Y$ of Definition 8.6.1. (It is the sum of X and Y where the base points are identified.). Exercise 8.6.6 offers the alternative definition that a group G is abelian if and only if the "fold" map $BG \vee BG \to_* BG$ (where both summands are mapped by the identity) factors through the inclusion $BG \vee BG \to_* BG \times BG$ (where inl_x is mapped to (x, sh_G) and inr_x to (sh_G, x)). The latter turns out to be a proposition equivalent to $\mathrm{isAb}(G)$. ⌟



EXERCISE 4.2.34. Let $\underline{\Omega}(A, a)$: Group and let b be an arbitrary element of A. Prove that the groups $\underline{\Omega}(A, a)$ and $\underline{\Omega}(A, b)$ are merely identical, in the sense that the proposition $\|\underline{\Omega}(A, a) \xrightarrow{=} \underline{\Omega}(A, b)\|$ is true. Similarly for ∞-groups in Section 4.7 when you get that far. ⌟

xca:typegroupisgroupoid

EXERCISE 4.2.35. Given two groups G and H. Prove that $G \xrightarrow{=} H$ is a set. Prove that the type of groups is a groupoid. This means that, given a group G, the component of Group, containing (and pointed at) G, is again a group, $\mathrm{Aut}_{\mathrm{Group}}(G)$, which we will call more simply the *group* $\mathrm{Aut}(G)$ *of automorphisms* of G, or the *automorphism group* of G. ⌟

We'll see more examples of groups in Sections 4.5 and 4.6 and indeed throughout the rest of the book.

4.3 *Abstract groups*

sec:identity-type-as-abstract

Studying the identity type leads one to the definition of what an abstract group should be. We fix a type A and an element $a : A$ for the rest of the section, and we focus on the identity type $a \xrightarrow{=} a$. We make the following observations about its elements and operations on them.

(1) There is an element $\mathrm{refl}_a : a \xrightarrow{=} a$. (See page 15, rule (E2).) We set $e :\equiv \mathrm{refl}_a$ as notation for the time being.

(2) For $g : a \xrightarrow{=} a$, the inverse $g^{-1} : a \xrightarrow{=} a$ was defined in Definition 2.5.1. Because it was defined by path induction, this inverse operation satisfies $e^{-1} \equiv e$.

(3) For $g, h : a \xrightarrow{=} a$, the product $h \cdot g : a \xrightarrow{=} a$ was defined in Definition 2.5.2. Because it was defined by path induction, this product operation satisfies $e \cdot g \equiv g$.

For any elements $g, g_1, g_2, g_3 : a \xrightarrow{=} a$, we consider the following four identity types:

(1) *the right unit law*: $g \xrightarrow{=} g \cdot e$,

(2) *the left unit law*: $g \xrightarrow{=} e \cdot g$,

it:right-unit

it:left-unit

(3) *the associativity law*: $g_1 \cdot (g_2 \cdot g_3) \xrightarrow{=} (g_1 \cdot g_2) \cdot g_3$,

(4) *the law of inverses*: $g \cdot g^{-1} \xrightarrow{=} e$.

it:associativity
it:inverse

In Exercise 2.5.3, the reader has constructed explicit elements of these identity types. If $a \xrightarrow{=} a$ is a set, then the identity types above are all propositions. Then, in line with the convention adopted in Section 2.15, we could simply say that Exercise 2.5.3 establishes that the equations hold. That motivates the following definition, in which we introduce a new set S to play the role of $a \xrightarrow{=} a$. We introduce a new element $e : S$ to play the role of refl_a, a new multiplication operation, and a new inverse operation. The original type A and its element a play no further role.[15]

[15] In Section 4.7 we will come back to A and a and consider the case in which A is an arbitrary connected type and $a : A$. Then $a \xrightarrow{=} a$ need not be a set.

Definition 4.3.1. An *abstract group* consists of the following data.

def:abstractgroup
struc:under-set
struc:unit
struc:mult-op

(1) A set S, called the *underlying set*.

(2) An element $e : S$, called the *unit* or the *neutral element*.

(3) A function $S \to S \to S$, called *multiplication*, taking two elements $g_1, g_2 : S$ to their *product*, denoted by $g_1 \cdot g_2 : S$.

Moreover, the following equations should hold, for all $g, g_1, g_2, g_3 : S$.

axiom:unit-laws
axiom:ass-law

(a) $g \cdot e = g$ and $e \cdot g = g$ (the *unit laws*)

(b) $g_1 \cdot (g_2 \cdot g_3) = (g_1 \cdot g_2) \cdot g_3$ (the *associativity law*)

struc:inv-op

(4) A function $S \to S$, the *inverse operation*, taking an element $g : S$ to its *inverse* g^{-1}.

Moreover, the following equation should hold, for all $g : S$.

axiom:inv-law

(c) $g \cdot g^{-1} = e$ (the *law of inverses*) ⌟

Remark 4.3.2. Strictly speaking, the proofs of the various equations are part of the data defining an abstract group, too. But, since the equations are propositions, the proofs are unique, and by the convention introduced in Remark 2.20.5, we can afford to omit them, when no confusion can occur. Moreover, one need not worry whether one gets a different group if the equations are given different proofs, because proofs of propositions are unique. ⌟

Taking into account the introductory comments we have made above, we may state the following lemma.

lem:idtypesgiveabstractgroups

Lemma 4.3.3. *If G is a group, then the set $UG \equiv (\mathrm{sh}_G \xrightarrow{=} \mathrm{sh}_G)$ of symmetries in G (see Definition 4.2.11), together with $e :\equiv \mathrm{refl}_{\mathrm{sh}_G}$, $g^{-1} :\equiv \mathrm{symm}_{\mathrm{sh}_G,\mathrm{sh}_G}\, g$ and $h \cdot g :\equiv \mathrm{trans}_{\mathrm{sh}_G,\mathrm{sh}_G,\mathrm{sh}_G}(g)(h)$, define an abstract group.*

Proof. The type UG is a set, because BG is a groupoid. Exercise 2.5.3 shows that all the relevant equations hold, as required. □

def:absG

Definition 4.3.4. Given a group G, the abstract group of Lemma 4.3.3, $\mathrm{abs}(G)$, is called the *abstract group associated to G*. ⌟

Lemma 4.3.3 implies that all examples of groups, such as those in Section 4.2.17, can easily be turned into examples of abstract groups. The following exercise provides a different source of examples.

xca:abstract-group-of-maps

EXERCISE 4.3.5. Let $\mathcal{G}$ be an abstract group with underlying set S and neutral element e. Let X be a set. Show that the set $X \to S$ of functions from X to S, together with the "pointwise operations" inherited from $\mathcal{G}$,

$$\begin{aligned} \mathrm{cst}_e &\equiv (x \mapsto e) && \text{(neutral element)} \\ f \cdot g &:\equiv (x \mapsto f(x) \cdot g(x)) && \text{(multiplication)}, \end{aligned}$$

forms an abstract group, which furthermore is abelian if and only if $\mathcal{G}$ is. ⌟

We leave the study of abstract groups for now; in Chapter 7 we'll show that the $G \mapsto \mathrm{abs}(G)$ construction furnishes an equivalence from the type of groups to the type of abstract groups, and we'll correlate concepts and constructions on groups to corresponding ones for abstract groups.

4.4 *Homomorphisms*

sec:homomorphisms
rem:homom-eqs

REMARK 4.4.1. Let G and H be groups, and suppose we have a pointed function $k : BG \to_* BH$. Suppose also, for simplicity (and without loss of generality), that $\mathrm{pt}_{BH} \equiv k(\mathrm{pt}_{BG})$ and $k_{\mathrm{pt}} \equiv \mathrm{refl}_{\mathrm{pt}_{BH}}$. Applying Definition 2.6.1 yields a function $f :\equiv \mathrm{ap}_k : UG \to UH$, which satisfies the following identities:

$$\begin{aligned} f(\mathrm{refl}_{\mathrm{pt}_{BG}}) &= \mathrm{refl}_{\mathrm{pt}_{BH}}, \\ f(g^{-1}) &= (f(g))^{-1} && \text{for any } g : UG, \\ f(g' \cdot g) &= f(g') \cdot f(g) && \text{for any } g, g' : UG. \end{aligned}$$

The first one is true by definition, the others follow from Construction 2.6.2. These three identities assert that the function ap_k *preserves*, in a certain sense, the operations provided by Lemma 4.3.3 that make up the abstract groups $\mathrm{abs}(G)$ and $\mathrm{abs}(H)$. In the traditional study of abstract groups, these three identities play an important role and entitle one to call the function f a *homomorphism of abstract groups*. ⌟

A slight generalization of the discussion above will be to suppose that we have a general pointed map with an arbitrary pointing path $k_{\mathrm{pt}} : \mathrm{pt}_{BH} \xrightarrow{=} k(\mathrm{pt}_{BG})$, not necessarily given by reflexivity. Indeed, that works out, thereby motivating the following definition.

def:grouphomomorph.sm

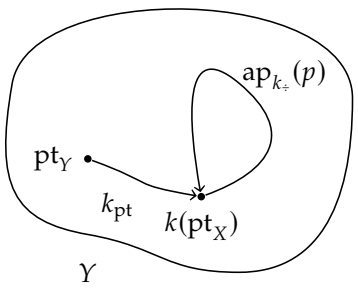
DEFINITION 4.4.2. The type of *group homomorphisms* from $G : \mathrm{Group}$ to $H : \mathrm{Group}$ is defined to be

$$\mathrm{Hom}(G, H) :\equiv \mathrm{Copy}_{\underline{\Omega}}(BG \to_* BH),$$

i.e., it is a wrapped copy of the type of pointed maps of classifying spaces with constructor $\underline{\Omega} : (BG \to_* BH) \to \mathrm{Hom}(G, H)$. We again write $\mathrm{B} : \mathrm{Hom}(G, H) \to (BG \to_* BH)$ for the destructor, and we call Bf the *classifying map* of the homomorphism f.[16] ⌟

We would like to understand explicitly the effect of a general homomorphism f from G to H on the underlying symmetries UG, UH, again without assuming that pointing path of Bf is given by reflexivity. So we should first study how pointed maps affect loops:

[16] When it is clear from context that a homomorphism is intended, we may write $f : G \to H$.



[17] Recall Definition 2.6.1 for ap, and that we may abbreviate $\mathrm{ap}_f(p)$ by $f(p)$.

def:loops-map

Definition 4.4.3. Given pointed types X and Y and a pointed function $k : X \to_* Y$ (as defined in Definition 2.21.1), we define a function $\Omega k : \Omega X \to \Omega Y$ by setting[17]

$$\Omega k(p) :\equiv k_{\mathrm{pt}}^{-1} \cdot \mathrm{ap}_{k_\div}(p) \cdot k_{\mathrm{pt}}, \quad \text{for all } p : \mathrm{pt}_X \xrightarrow{=} \mathrm{pt}_X.$$

Later we will need that Ωk is a pointed map and define $(\Omega k)_{\mathrm{pt}} : \mathrm{refl}_{\mathrm{pt}_Y} \xrightarrow{=} k_{\mathrm{pt}}^{-1} \cdot k_{\mathrm{pt}}$ by the inverse law of path algebra, Exercise 2.5.3. ⌟

rem:loops-map

Remark 4.4.4. If $k : X \to_* Y$ has the reflexivity path $\mathrm{refl}_{\mathrm{pt}_Y}$ as its pointing path, then we have an identification $\Omega k \xrightarrow{=} \mathrm{ap}_{k_\div}$. ⌟

def:USym-hom

Definition 4.4.5. Given groups G and H and a homomorphism f from G to H, we define the function $Uf : UG \to UH$ by setting $Uf :\equiv \Omega Bf$. In other words, the homomorphism $\underline{\Omega} Bf$ induces ΩBf as the map on underlying symmetries. ⌟

lem:grouphomomaxioms

Lemma 4.4.6. *Given groups G and H and a homomorphism $f : \mathrm{Hom}(G, H)$, the function $Uf : UG \to UH$ defined above satisfies the following identities*:

gp-homo-unit

$$(Uf)(\mathrm{refl}_{\mathrm{pt}_{BG}}) = \mathrm{refl}_{\mathrm{pt}_{BH}}, \tag{4.4.1}$$

gp-homo-comp

$$(Uf)(g^{-1}) = ((Uf)(g))^{-1} \qquad \text{for any } g : UG, \tag{4.4.2}$$

gp-homo-inv

$$(Uf)(g' \cdot g) = (Uf)(g') \cdot (Uf)(g) \qquad \text{for any } g, g' : UG. \tag{4.4.3}$$

Proof. We write $f \equiv (f_\div, p)$, where $p : \mathrm{pt}_{BH} \xrightarrow{=} f_\div(\mathrm{pt}_{BG})$. By induction on p, which is allowed since pt_{BH} is arbitrary, we reduce to the case where $\mathrm{pt}_{BH} \equiv f_\div(\mathrm{pt}_{BG})$ and $p \equiv \mathrm{refl}_{\mathrm{pt}_{BH}}$. We finish by applying Remark 4.4.1 and 4.4.4. □

def:groupisomorphism

Definition 4.4.7. A homomorphism $f : G \to H$ is an *isomorphism* if its classifying map Bf is an equivalence. We let $\mathrm{Iso}(G, H)$ be the subset of isomorphisms in $\mathrm{Hom}(G, H)$.[18] ⌟

[18] Both $\mathrm{Iso}(G, H)$ and $\mathrm{Hom}(G, H)$ are *sets*, using Lemma 4.4.12 below.

def:identity-group-homomorphism

Definition 4.4.8. If G is a group, then we use Definition 2.21.2 to define the *identity homomorphism* $\mathrm{id}_G : G \to G$ by setting $\mathrm{id}_G :\equiv \underline{\Omega}(\mathrm{id}_{BG})$. The identity homomorphism is an isomorphism. ⌟

remark:groupsasunivalentype

Remark 4.4.9. From Exercise 2.21.7, we have an equivalence

$$(G \xrightarrow{=}_{\mathrm{Group}} H) \xrightarrow{\simeq} \mathrm{Iso}(G, H)$$

between the identity type of the groups G and H and the set of isomorphisms. We use the convention introduced in Remark 3.4.1 also here. That is, we allow ourselves to also write $p : \mathrm{Iso}(G, H)$ for the isomorphism corresponding to an identification $p : G \xrightarrow{=} H$, and $Bp : BG \xrightarrow{\simeq}_* BH$ for the corresponding pointed equivalence of classifying types. Conversely, given an isomorphism $f : \mathrm{Iso}(G, H)$, we may denote the corresponding path also as $f : G \xrightarrow{=} H$. ⌟

def:group-homomorphism-composition

Definition 4.4.10. If G, G', and G'' are groups, and $f : G \to G'$ and $f' : G' \to G''$ are homomorphisms, then we use the definition of composition of pointed functions in Definition 2.21.1 to define the *composite homomorphism* $f' \circ f : G \to G''$ by setting $f' \circ f :\equiv \underline{\Omega}(Bf' \circ Bf)$. ⌟

Recall from Section 2.21, that when there is little danger of confusion, we may drop the subscript "$\div$" when talking about the unpointed structure.

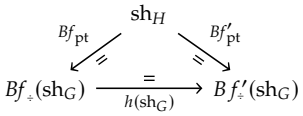
[19] We use the same notational convention regarding "B" applied to homomorphisms as we do for groups.

rem:Bf-convention

REMARK 4.4.11. To construct a function $\varphi : \prod_{f : \mathrm{Hom}(G,H)} T(f)$, where $T(f)$ is a family of types parametrized by $f : \mathrm{Hom}(G,H)$, it suffices to consider the case $f \equiv \underline{\Omega} Bf$.[19] ⌟

Identifications of homomorphisms $f \xrightarrow{=}_{\mathrm{Hom}(G,H)} f'$ are equivalent to identifications of pointed maps $Bf \xrightarrow{=}_{BG \to_* BH} Bf'$; the latter are (by Construction 2.21.8 and the fact that BH is a groupoid) given by identifications of (unpointed) maps $h : Bf_{\div} \xrightarrow{=} Bf'_{\div}$ such that

$$h(\mathrm{sh}_G) Bf_{\mathrm{pt}} = Bf'_{\mathrm{pt}}.$$



We will later show that if G and H are groups, then $\mathrm{Hom}(G,H)$ is equivalent to the *set* of "abstract group homomorphisms" from $\mathrm{abs}(G)$ to $\mathrm{abs}(H)$ (see Lemma 7.5.1), but it is instructive to give a direct proof of the following.

lem:hom-is-set

LEMMA 4.4.12. *The type of homomorphisms* $\mathrm{Hom}(G,H)$ *is a set for all groups* G, H.

Proof. Given homomorphisms $f, f' : \mathrm{Hom}(G,H)$, we use the equivalence just described,

$$(f \xrightarrow{=} f') \xrightarrow{\simeq} \sum_{h : Bf_{\div} \xrightarrow{=} Bf'_{\div}} h(\mathrm{sh}_G) Bf_{\mathrm{pt}} = Bf'_{\mathrm{pt}}.$$

Thus our goal is to prove that any two elements $(h, !), (j, !)$ of the right-hand side can be identified. By function extensionality, the type $h \xrightarrow{=} j$ is equivalent to the proposition $\prod_{t : BG_{\div}} h(t) = j(t)$. So now we can use connectedness of $BG_{\div}$, and only check the equality on the point sh_G. By assumption,

$$h(\mathrm{sh}_G) = Bf'_{\mathrm{pt}} Bf_{\mathrm{pt}}^{-1} = j(\mathrm{sh}_G).$$

This concludes the proof that $f \xrightarrow{=} f'$ is a proposition, or in other words that $\mathrm{Hom}(G,H)$ is a set.[20] □

ft:ptd-decr-h-1-ef

ex:groups-morphisms

EXAMPLE 4.4.13.

(1) Consider two sets S and T. Recall from Example 4.2.20 that $\mathrm{Set}_{(S)} \equiv \sum_{X : \mathrm{Set}} \| S \xrightarrow{=} X \|$ is the component of the groupoid Set containing S, and when pointed at S represents the permutation group Σ_S. The map $_ \amalg T : \mathrm{Set}_{(S)} \to \mathrm{Set}_{(S \amalg T)}$ sending X to $X \amalg T$ induces a group homomorphism $\Sigma_S \to \Sigma_{S \amalg T}$, pointed by the path $\mathrm{refl}_{S \amalg T} : S \amalg T \xrightarrow{=} (_ \amalg T)(S)$. Thought of as symmetries, this says that if you have a symmetry of S, then we get a symmetry of $S \amalg T$ (which doesn't do anything to T).

Likewise, we have a map $_ \times T : \mathrm{Set}_{(S)} \to \mathrm{Set}_{(S \times T)}$ sending X to $X \times T$, inducing a group homomorphism $\Sigma_S \to \Sigma_{S \times T}$, pointed by the path $\mathrm{refl}_{S \times T} : S \times T \xrightarrow{=} (_ \times T)(S)$. Thought of as symmetries, this says that if you have a symmetry of S, then we get a symmetry of $S \times T$ (which doesn't do anything to the second component of pairs in $S \times T$).

In particular, we get homomorphisms of symmetric groups $\Sigma_m \to \Sigma_{m+n}$ and $\Sigma_m \to \Sigma_{mn}$, induced by identifications $\mathrm{Fin}(m+n) \xrightarrow{=} \mathrm{Fin}(m) \amalg \mathrm{Fin}(n)$ and $\mathrm{Fin}(mn) \xrightarrow{=} \mathrm{Fin}(m) \times \mathrm{Fin}(n)$.[21]

[20] The same argument shows that the type $X \to_* Y$ is a set whenever X is connected and Y is a groupoid. A more general fact is that $X \to_* Y$ is an n-type whenever X is $(k-1)$-connected and Y is $(n+k)$-truncated, for all $k \geq 0$ and $n \geq -1$.

[21] The latter identification is somewhat arbitrary, but let's say it's defined using the lexicographic ordering on the product.

(2) Let G be a group. Since there is a unique map from BG to $\mathbb{1}$ (uniquely pointed by the reflexivity path of the unique element of $\mathbb{1}$), we get a unique homomorphism from G to the trivial group. Likewise, there is a unique morphism from the trivial group to G, sending the unique element of $\mathbb{1}$ to sh_G, and pointed by $\mathrm{refl}_{\mathrm{sh}_G}$; the uniqueness follows from Lemma 2.9.10, cf. Lemma 3.3.11.

(3) If G and H are groups, the projections $BG \leftarrow BG \times BH \rightarrow BH$ and inclusions $BG \rightarrow BG \times BH \leftarrow BH$ (e.g., the inclusion $BG \rightarrow BG \times BH$ is given by $z \mapsto (z, \mathrm{sh}_H)$) give rise to group homomorphisms between $G \times H$ and G and H, namely projections $G \leftarrow G \times H \rightarrow H$ and inclusions $G \rightarrow G \times H \leftarrow H$.

i:mod_m

(4) In Example 4.2.22 we gave an example of an isomorphism, namely one from the cyclic group C_m to $\mathbb{Z}/m\mathbb{Z}$, and in Example 4.2.23 we looked at $R_m : \mathrm{B}\mathbb{Z} \rightarrow_* \mathrm{B}\Sigma_m$, which, pointed by $\mathrm{refl}_{\mathbb{m}}$, classifies a homomorphism $\mathbb{Z} \rightarrow \Sigma_m$. This homomorphism factors through C_m (equivalently through $\mathbb{Z}/m\mathbb{Z}$) via a homomorphism $\mathrm{mod}_m : \mathbb{Z} \rightarrow \mathrm{C}_m$ classified by $\mathrm{Bmod}_m : \mathrm{B}\mathbb{Z} \rightarrow_* \mathrm{BC}_m$ sending $\bullet$ to $(\mathbb{m}, \mathrm{s})$ and $\circlearrowleft$ to s,[22] and pointed by reflexivity. The homomorphism $\mathrm{C}_m \rightarrow \Sigma_m$ forgets the cycle structure. ⌟

[22] The function s here is the successor function modulo m.

Remark 4.4.14. In the examples above, we insisted on writing the path pointing a group homomorphism, even when this path was a reflexivity path. We now adopt the convention that there is no need to specify the path in this case.[23] Thus, given a map $f : A \rightarrow B$ between connected groupoids and $a : A$, the group homomorphism $\mathrm{Aut}_A(a) \rightarrow \mathrm{Aut}_B(f(a))$ defined by $(f, \mathrm{refl}_{f(a)})$ will simply be referred to as f.

[23] Or more generally, whenever the pointing path is clear from context.

However, it is important to understand that different homomorphisms can have the same underlying unpointed function.[24] Consider, for example, the group Σ_3, whose classifying space is $\mathrm{B}\Sigma_3 :\equiv (\mathrm{FinSet}_3, \mathfrak{Z})$, and the symmetry $\tau : \mathrm{U}\Sigma_3$ that is defined (through univalence) by

$$0 \mapsto 1, \quad 1 \mapsto 0, \quad 2 \mapsto 2, \qquad \text{i.e., } \tau \text{ is the transposition } (0\ 1).$$

[24] Later, in Theorem 9.10.2, we'll examine this phenomenon in more detail.

Then the function $\mathrm{id} : \mathrm{FinSet}_3 \rightarrow \mathrm{FinSet}_3$ gives rise to two elements of $\mathrm{Hom}(\Sigma_3, \Sigma_3)$: the first one is $(\mathrm{id}, \mathrm{refl}_3)$, which is simply denoted id_{Σ_3}; the second one is (id, τ), which we will denote $\tilde{\tau}$ temporarily. Let us prove $\mathrm{id}_{\Sigma_3} \neq \tilde{\tau}$, that is, we suppose $\mathrm{id}_{\Sigma_3} = \tilde{\tau}$ and derive a contradiction. By Definition 4.4.3 we get $\sigma = \Omega(\mathrm{id}_{\Sigma_3})(\sigma) = \Omega(\tilde{\tau})(\sigma) = \tau^{-1}\sigma\tau$ for all $\sigma : \mathrm{U}\Sigma_3$, so τ commutes with every other element of $\mathrm{U}\Sigma_3$. This fails for the transposition $\sigma :\equiv (1\ 2)$, since $\sigma\tau(0) = 2$ while $\tau\sigma(0) = 1$. (See also Exercise 4.2.32.) ⌟

def:loops-compose

Construction 4.4.15. *For pointed types X, Y, Z and pointed maps $f : X \rightarrow_* Y$ and $g : Y \rightarrow_* Z$, we get an identification of type*

$$\Omega(g \circ f) \xrightarrow{=}_{(\Omega X \rightarrow \Omega Z)} \Omega(g) \circ \Omega(f).$$

Implementation of Construction 4.4.15. Let x denote the base point of X. By induction on f_{pt} and on g_{pt}, we reduce to the case where $f_{\mathrm{pt}} \equiv \mathrm{refl}_{f(x)}$ and $g_{\mathrm{pt}} \equiv \mathrm{refl}_{g(f(x))}$, and it suffices to identify $\mathrm{ap}_{g \circ f}$ with $\mathrm{ap}_g \circ \mathrm{ap}_f$. By **??** 2.9.18, it suffices to identify $\mathrm{ap}_{g \circ f}(p)$ with $\mathrm{ap}_g(\mathrm{ap}_f(p))$ for each $p : \Omega X$. For that purpose, it suffices to even identify $\mathrm{ap}_{g \circ f}(p)$ with $\mathrm{ap}_g(\mathrm{ap}_f(p))$ for any $x' : X$ and any $p : x \xrightarrow{=} x'$. Then by induction on p, it suffices to

give an identification $\mathrm{ap}_{g\circ f}(\mathrm{refl}_x) \xrightarrow{=} \mathrm{ap}_g(\mathrm{ap}_f(\mathrm{refl}_x))$, and that can be done by reflexivity, by observing that both sides are equal, by definition, to $\mathrm{refl}_{g(f(x))}$. □

cor:USym-compose

Corollary 4.4.16. *For composable group homomorphisms $\varphi : \mathrm{Hom}(G, H)$, $\psi : \mathrm{Hom}(H, K)$, we get an identification $\mathrm{U}(\psi \circ \varphi) = \mathrm{U}\psi \circ \mathrm{U}\varphi$.*

The following example expresses that $\mathbb{Z}$ is a "free group with one generator".

ex:Zinitial

Example 4.4.17. Chapter 3 was all about the circle S^1 and its role as a "universal symmetry" and how it related to the integers. In our current language, $\mathbb{Z} \equiv \underline{\Omega}(\mathrm{S}^1, \bullet)$ and much[25] of the universality of S^1 is found in the following observation. If G is a group, then Corollary 3.1.3 yields an equivalence of sets

[25] Not all: BG is a groupoid and not an arbitrary type, cf. Section 4.7.

$$\mathrm{ev}_{BG} : \big((\mathrm{S}^1, \bullet) \to_* BG\big) \xrightarrow{\simeq} UG, \quad \mathrm{ev}_{BG}(f_\div, f_{\mathrm{pt}}) :\equiv \Omega(f_\div, f_{\mathrm{pt}})(\circlearrowleft).$$

The domain of this equivalence classifies $\mathrm{Hom}(\mathbb{Z}, G)$. Hence, ev_{BG} provides a way to identify $\mathrm{Hom}(\mathbb{Z}, G)$ with the underlying set UG. Like in Theorem 3.1.2, the inverse of ev_{BG} is denoted ve_{BG} and satisfies $\mathrm{ve}_{BG}(g)(\bullet) \equiv \mathrm{sh}_G$ and $\mathrm{ve}_{BG}(g)(\circlearrowleft) = g$. Moreover, $\mathrm{ve}_{BG}(g)$ is pointed by $\mathrm{refl}_{\mathrm{sh}_G}$. ⌟

The following lemma states the "naturality" of ev_{BG} in the previous example.

lem:Znatural

Lemma 4.4.18. *Let G and H be groups and $f : \mathrm{Hom}(G, H)$. Then the following diagram commutes,*

$$\begin{array}{ccc} \mathrm{Hom}(\mathbb{Z}, G) & \xrightarrow[\sim]{\mathrm{ev}} & UG \\ {\scriptstyle f\circ_}\downarrow & & \downarrow{\scriptstyle Uf} \\ \mathrm{Hom}(\mathbb{Z}, H) & \xrightarrow[\sim]{\mathrm{ev}} & UH, \end{array}$$

where the horizontal maps evaluate the classifying maps of the homomorphisms from $\mathbb{Z}$ at the loop $\circlearrowleft : \mathrm{U}\mathbb{Z}$.

Proof. Let $k : \mathrm{Hom}(\mathbb{Z}, G)$, giving $Uk : \mathrm{U}\mathbb{Z} \to UG$. Going across horizontally and then down, k is mapped first to $Uk(\circlearrowleft)$, and then to $Uf(Uk(\circlearrowleft))$. Going the other way takes k to $\mathrm{U}(f \circ k)(\circlearrowleft)$, which is equal to $Uf(Uk(\circlearrowleft))$ by Corollary 4.4.16. □

xca:BGtotype

Exercise 4.4.19. Let G be a group and A a groupoid. Use the definitions and Exercise 2.21.5 to construct equivalences between the types:[26]

[26] We'll return to these in more detail in Section 5.2.27.

(1) $BG_\div \to A$

(2) $\sum_{a:A} \sum_{f : BG_\div \to A} a \xrightarrow{=} f(\mathrm{sh}_G)$

(3) $\sum_{a:A} (BG \to_* (A, a))$

(4) $\sum_{a:A} \mathrm{Hom}(G, \mathrm{Aut}_A(a))$ ⌟

rem:first-abs-hom

Remark 4.4.20. The definition of group homomorphism in Definition 4.4.2 should be contrasted with the usual – and somewhat more cumbersome – notion of a group homomorphism $f : \mathcal{G} \to \mathcal{H}$ of abstract groups where we must ask of a function of the underlying sets that it in addition preserves the neutral element, multiplication, and inverse operation. In

our setup this is simply true, as we saw in Lemma 4.4.6. In terms of the abstract groups determined by G and H, we can write these equations as

$$\begin{aligned} Uf(e_G) &= e_H \\ Uf(g \cdot_G g') &= Uf(g) \cdot_H Uf(g') && \text{for all } g, g' : UG, \\ Uf(g^{-1}) &= (Uf(g))^{-1} && \text{for all } g : UG. \end{aligned}$$

We come back to abstract homomorphisms in Section 7.3. ⌟

exa:conj-concrete

Example 4.4.21. In this example we analyse what happens when we move the shape of a group along a path in the classifying type. This path can in particular be a loop at the shape. More precisely, let G be a group, y an element of BG, and p a path of type $\mathrm{sh}_G \xrightarrow{=} y$. Then $(\mathrm{id}_{BG}, p^{-1})$ is a pointed equivalence of type $BG \xrightarrow{\simeq}_* (BG_{\div}, y)$ and hence induces an isomorphism from G to $\underline{\Omega}(BG_{\div}, y)$.[27] By Remark 4.4.9 we then get an identification of these groups. Moreover, by path induction on p, the equivalence $\mathrm{U}(\underline{\Omega}(\mathrm{id}_{BG}, p^{-1})) \equiv \Omega(\mathrm{id}_{BG}, p^{-1})$ of type $(\mathrm{sh}_G \xrightarrow{=} \mathrm{sh}_G) \xrightarrow{\simeq} (y \xrightarrow{=} y)$[28] can be identified with the map $g \mapsto pgp^{-1}$. This map is called *conjugation*.[29] In Exercise 7.2.10 we come back to the special case in which $y \equiv \mathrm{sh}_G$. ⌟

The above example motivates and justifies the following definition of a homomorphism from a group to its *inner* automorphisms, that is, automorphisms that come from conjugation. Such automorphisms will further be discussed in Section 9.7. Recall that $\mathrm{BAut}(G)$ is the connected component of G in the type Group, pointed at G.

def:inner-autos

Definition 4.4.22. Let G be a group. Define the homomorphism $\mathrm{inn} : G \to \mathrm{Aut}(G)$ by setting

$$\mathrm{Binn} : BG \to_* \mathrm{BAut}(G), \quad y \mapsto \underline{\Omega}(BG_{\div}, y),$$

where the path pointing Binn is $p_{\mathrm{inn}} :\equiv \mathrm{refl}_G : G \xrightarrow{=} \mathrm{Binn}(\mathrm{sh}_G)$. Note that p_{inn} is well defined since $\mathrm{Binn}(\mathrm{sh}_G) \equiv G$. Notice furthermore that the codomain of Binn is correct: since BG is connected, the proposition $\|G \xrightarrow{=} \underline{\Omega}(BG_{\div}, y)\|$ holds for all $y : BG$, by the argument in Example 4.4.21. ⌟

4.5 *The sign homomorphism*

sec:sign-homomorphism

In this section we're going to define the very important *sign homomorphism* $\mathrm{sgn} : \Sigma_n \to \Sigma_2$, defined for $n \geq 2$.[30] To do this, we need to assign to every n-element set A a 2-element set $\mathrm{Bsgn}(A)$.

We get this 2-element set as a quotient of the set of all possible ways of choosing an element from each 2-element subset of A, where two different such choices are deemed the same if they differ in an *even* number of pairs. Since choosing an element from a 2-element set is equivalent to ordering it (e.g., chosen element first), we can also talk about ways of ordering all possible 2-element subsets of A, or equivalently, ways of directing the complete graph on A. Figure 4.2 shows all 8 ways of directing the complete graph on a 3-element set divided into the 2 resulting equivalence classes.

To see that this really defines an equivalence relation, it helps to generalize a bit. Thus, fix a finite set E, and let $P : E \to \mathrm{B}\Sigma_2$ be a family of 2-element sets with parameter type E.

[27] One may wonder why p^{-1} in $(\mathrm{id}_{BG}, p^{-1})$. The reason is our convention for the direction of the pointing path of a pointed map.

[28] Note that $\mathrm{U}(\underline{\Omega}(BG_{\div}, y)) \equiv \Omega(BG_{\div}, y) \equiv (y \xrightarrow{=} y)$.

[29] We have seen similar maps, e.g., all the way back in Exercise 2.14.4(4).

[30] The approach we take here is similar to that of Mangel and Rijke[31].

[31] Éléonore Mangel and Egbert Rijke. *Delooping the sign homomorphism in univalent mathematics*. 2023. arXiv: 2301.10011 [math.GR].

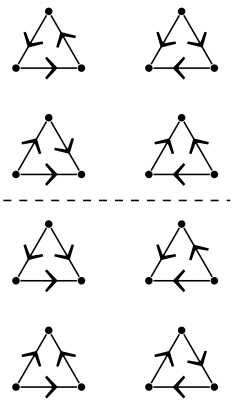


Figure 4.2: The two equivalence classes of directions of the complete graph on a 3-element set.

fig:sign-orderings-3

Definition 4.5.1. The parity relation $\sim$ on $\prod_{e:E} P(e)$ relates functions that disagree in an even number of points. That is, $f \sim g$ holds if and only if the subset $\{\, e : E \mid f(e) \neq g(e) \,\}$ has an even number of elements.[32] ⌟

[32] This makes sense because any 2-element set is decidable, and a subset of a finite set specified by a decidable predicate is itself a finite set. We may apply the usual set-theoretic operators, such as union and set difference, to these subsets. Note also that the parity relation is itself decidable.

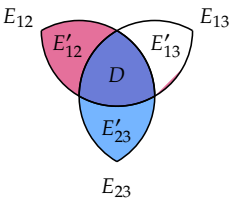
lem:parityequiv

Lemma 4.5.2. *The parity relation $\sim$ is an equivalence relation on the set $\prod_{e:E} P(e)$, and the quotient is a 2-element set if E is nonempty, otherwise it is a 1-element set.*

Proof. The $\sim$ relation is clearly symmetric, and it is reflexive, since the empty set has an even number of elements. To show transitivity, let $f_1, f_2, f_3 : \prod_{e:E} P(e)$. We can partition E according to whether the f_i agree or disagree:

$$E_{ij} :\equiv \{\, e : E \mid f_i(e) = f_j(e) \,\}, \quad F_{ij} :\equiv \{\, e : E \mid f_i(e) \neq f_j(e) \,\}.$$

By transitivity of equality, $E_{ij} \cap E_{jk} \subseteq E_{ik}$, for all i, j, k. Hence, the Venn diagram of these sets has the simplified form shown in the margin, where we set



$$D :\equiv \{\, e : E \mid f_1(e) = f_2(e) = f_3(e) \,\}, \quad E'_{ij} :\equiv E_{ij} \setminus D.$$

Here we also use that $E_{12} \cup E_{23} \cup E_{13} = E$ (as subsets of E), since of the three function values at any e in E, two must agree.

We now find $F_{12} = E'_{13} \cup E'_{23}$ (disjoint union), and similarly for F_{13} and F_{23}. Taking cardinalities, we get

$$\#(F_{12}) + \#(F_{13}) + \#(F_{23}) = 2\big(\#(E'_{12}) + \#(E'_{13}) + \#(E'_{23})\big),$$

so if two of the F_{ij}'s have an even number of elements, then so does the third. We also see that at least one of the F_{ij}'s has even cardinality, so the quotient has at most 2 elements.

Clearly, if E is empty, then $\prod_{e:E} P(e)$ is contractible, so the quotient is contractible. Assume now that E is nonempty. To show the proposition that the quotient is a 2-element set, we may assume that E is the n-element set $\{1, \ldots, n\}$ (since $n > 0$), and (by induction on n) that each set $P(e)$ is $\{\pm 1\}$ (our favorite 2-element set for the moment). Then any function is equivalent to either the all $+1$-function or the function that is -1 at 1 and $+1$ otherwise, according to how many times it takes the value -1. □

Recall from Example 4.2.28 that we can form the product of any (finite) family of groups. In particular, if we take the constant family at G, indexed by a finite set S, we get a power G^S, with classifying type BG^S and underlying set of symmetries UG^S.[33]

[33] See Exercise 4.2.29(1).

def:mu_E

Definition 4.5.3. Given a finite set E, we define a homomorphism $\mu_E : \mathrm{Hom}(\Sigma_2^E, \Sigma_2)$ by deciding whether E is nonempty, and proceeding accordingly:

If E is nonempty, we use the construction $P \mapsto \big(\prod_{e:E} P(e)\big)/\!\sim$ from above, pointed by the identification indicated in the proof of Lemma 4.5.2, i.e., identifying the class of the all $+1$-function with $+1$ in $\{\pm 1\}$.

If E is empty, then $\mathrm{B}\Sigma_2^E$ is contractible, so Σ_2^E is the trivial group and we take the corresponding unique definition of μ_E. ⌟

Exercise 4.5.4. From Exercise 4.2.29(1) we know that Function Extensionality identifies the set of symmetries in Σ_2^E with $\{\pm 1\}^E$. Show that under

[34] Note that this works even when E is empty, since the product of an empty collection of numbers is $+1$.

this identification, $\mathrm{U}\mu_E$ maps a function $s : E \to \{\pm 1\}$ to the product of its values.[34] ⌟

def:sign-ordering

Definition 4.5.5. A *local ordering* of a finite set A is an element of the set $\prod_{e : E(A)} P(e)$, where $E(A)$ is the set of 2-element subsets of A, and $P : E(A) \to \mathrm{B}\Sigma_2$ maps a 2-element subset to the underlying 2-element set.

A *sign ordering*[35] of a finite set A is an element of $\left(\prod_{e : E(A)} P(e)\right)/\!\sim$, i.e., the quotient of the set of local orderings modulo the parity relation. ⌟

[35] This term is used in analogy with total and cyclic orderings, even though it's harder to visualize as an ordering. It seems to have first been used by Kuperberg[36].

def:sgn

Definition 4.5.6. The *sign homomorphism* $\mathrm{sgn} : \mathrm{Hom}(\Sigma_n, \Sigma_2)$ is defined via the pointed map $\mathrm{Bsgn} : \mathrm{B}\Sigma_n \to_* \mathrm{B}\Sigma_2$, where $\mathrm{Bsgn}(A) :\equiv \mathrm{B}\mu_{E(A)}(P)$, with P as in Definition 4.5.5 and $\mu_{E(A)}$ as in Definition 4.5.3. We make Bsgn pointed using the total ordering $0 < 1 < \cdots < n-1$ on the standard n-element set, $\mathbb{m} \equiv \mathrm{sh}_{\Sigma_n}$, to identify each 2-element subset with the standard 2-element set, and using the pointedness of $\mathrm{B}\mu$. ⌟

[36] Greg Kuperberg. "Noninvolutory Hopf algebras and 3-manifold invariants". In: *Duke Math. J.* 84.1 (1996), pp. 83–129. DOI: 10.1215/S0012-7094-96-08403-3.

Not only does the notion of a sign ordering allow us to define the sign homomorphism, we also get a new family of examples of groups:[37]

[37] We'll study this construction more generally later in Section 9.3: in these terms A_n is the *kernel* of the sign homomorphism.

def:alternating-groups

Definition 4.5.7. For any $n : \mathbb{N}$, we define the *alternating group of degree n* to be

$$\mathrm{A}_n :\equiv \Omega\Big(\sum_{A\,:\,\mathrm{B}\Sigma_n} \mathrm{Bsgn}(A),\ \big(\mathbb{m}, \mathrm{Bsgn}_{\mathrm{pt}}(\mathrm{pt}_2)\big)\Big),$$

i.e., the shapes of A_n are *sign ordered n-element sets*, and the designated shape is $\mathbb{m}$ with the sign ordering coming from the usual total ordering.

The symmetries in A_n are called *even permutations*. ⌟

xca:isos_A3_C3

Exercise 4.5.8. Give two isomorphisms from A_3 to C_3. ⌟

Something interesting happens when we consider permutations on other shapes in $\mathrm{B}\Sigma_n$, i.e., arbitrary n-element sets A. The same map, Bsgn, can be considered as a map $\mathrm{BAut}(A) \to \mathrm{B}\Sigma_2$, but we can cannot make this pointed uniformly in A.[38] However, the self-identifications of a 2-element set T, $(T \xrightarrow{\simeq} T)$, *can* be identified with $\{\pm 1\}$,[39] according to whether it transposes the elements of T, or not. Hence, we can define the sign of any permutation of a finite set:

[38] Why not? A construction $p : \prod_{A\,:\,\mathrm{B}\Sigma_n} (\mathrm{Bsgn}_{\div}(A) \xrightarrow{\simeq} \mathrm{sh}_{\Sigma_2})$ would amount to an identification of Bsgn with the constant map.

[39] See Exercise 2.24.7. In this section, we identify $\mathrm{U}\Sigma_2$ with the set $\{\pm 1\}$, which has a compatible abstract group structure given by multiplication.

def:sgn-permutation

Definition 4.5.9. Let A be a finite set, and let σ be a permutation of A. If the cardinality of A is 0 or 1, then the *sign* of σ is +1. Otherwise, the *sign* of σ is ± 1 according to whether $\mathrm{Bsgn}_{\div}(\sigma)$ swaps the elements of the 2-element set $\mathrm{Bsgn}_{\div}(A)$, or not. We write $\mathrm{sgn}(\sigma) : \{\pm 1\}$ for the sign of σ, and call σ *even* if $\mathrm{sgn}(\sigma) = 1$, and *odd* otherwise. ⌟

For permutations of the standard n-element set, this is the same as the value $\mathrm{Usgn}(\sigma) : \mathrm{U}\Sigma_2$. Note that sgn defines an abstract homomorphism from $\mathrm{Aut}(A)$ to Σ_2 for each A, since it does so for $A \equiv \mathrm{sh}_{\Sigma_n}$. Even better, this abstract homomorphism comes from a concrete one $\mathrm{sgn}^A : \mathrm{Hom}(\mathrm{Aut}(A), \Sigma_2)$ for each finite set A. Indeed, since $T \xrightarrow{\simeq} U$ is a 2-element set for any 2-element sets T and U, we can consider the map $\mathrm{Bsgn}^A_{\div} : \mathrm{BAut}(A) \to \mathrm{B}\Sigma_2$ that maps $B : \mathrm{BAut}(A)$ to $(\mathrm{Bsgn}_{\div}(A) \xrightarrow{\simeq} \mathrm{Bsgn}_{\div}(B))$. The identification of $\mathrm{Bsgn}^A_{\div}(A)$ with $\{\pm 1\}$ mentioned above makes $\mathrm{Bsgn}^A_{\div}$ into a pointed map $\mathrm{Bsgn}^A : \mathrm{BAut}(A) \to_* \mathrm{B}\Sigma_2$, i.e., it defines an homomorphism $\mathrm{sgn}^A : \mathrm{Hom}(\mathrm{Aut}(A), \Sigma_2)$, as announced.[40]

[40] This is an instance of a more general construction, called *delooping* (see Section 7.5). The formula for $\mathrm{Bsgn}^A_{\div}$ here is very simple since Σ_2 is a fairly simple group.

it:sgn-cycle

Lemma 4.5.10. (1) *The sign of a transposition is* -1.

(2) *The sign of a k-cycle is* $(-1)^{k-1}$.

(3) *The identity permutation can only be expressed as a product of an even number of transpositions.*

Proof. For (1), it suffices to consider the transposition (1 2) of a standard n-element set $\{1, 2, \dots, n\}$. Relative to the standard local ordering $(1 < 2, 1 < 3, \dots, 1 < n, 2 < 3, \dots, n-1 < n)$, the transposition only changes the ordering $1 < 2$ to $2 < 1$, thus differing at exactly one place.

Now (2) follows via Exercise 3.7.4.

For (3), assume $\mathrm{id}_A = (a_1\ b_1) \cdots (a_k\ b_k)$, and take the sign of both sides. Since sgn is a homomorphism, we get $+1 = (-1)^k$, so k is even. □

Corollary 4.5.11. *If a permutation σ is expressed as a product of transpositions in two ways,*

$$\sigma = (a_1\ b_1) \cdots (a_m\ b_m) = (c_1\ d_1) \cdots (c_n\ d_n),$$

then the parity of m equals that of n, and we have $\mathrm{sgn}(\sigma) = (-1)^m = (-1)^n$.

Exercise 4.5.12. Here's a different way of finding the sign of a permutation of the standard n-element set $\mathbb{n}$ (or of any totally ordered n-element set – but these are all uniquely identified with $\mathbb{n}$).

For $\sigma : \mathbb{n} \xrightarrow{\simeq} \mathbb{n}$, we call an ordered pair of elements i, j with $i < j$ but $\sigma(i) > \sigma(j)$ an *inversion*. If we represent σ graphically as in Figure 4.3, then inversions are crossings of the edges $(i, \sigma(i))$ and $(j, \sigma(j))$. Show that $\mathrm{sgn}(\sigma) = (-1)^{\mathrm{inv}(\sigma)}$, where $\mathrm{inv}(\sigma)$ is the number of inversions. ⌟

Remark 4.5.13. The two graphical representations Figures 3.11 and 4.3 each have their uses: In the former, the cycle decomposition is immediately visible, while permutations are easily composed using the latter style. Note that the number of inversions depend on the linear ordering, whereas the sign itself does not.

We also remark that when we compose permutations in the latter style, we don't immediately see the number of crossings/inversions, but we can imagine "pulling the strings taut", whereby the parity of the number of crossings (and thus the sign) is preserved, as seen in Figure 4.4. ⌟

Exercise 4.5.14. Recall from Exercise 3.7.6 that there are $n!$ permutations in Σ_n. Show that there are $n!/2$ even permutations for $n \geq 2$. ⌟

4.6 *Bicycles*

In Definition 3.6.3 we introduced the type of cycles: pairs (X, t) of a nonempty set X and a bijection $t : X \xrightarrow{\simeq} X$ such that any two elements $x, x' : X$ can be connected in the sense that we have (under a propositional truncation) a way to get from x to x' by repeated application of t and its inverse. These gave rise to the group of integers $\mathbb{Z}$ via the infinite cycle (Z, s) in Example 4.2.18 and the cyclic groups of finite order C_m via the finite cycles $(\mathbb{m}, \mathrm{s})$ in Example 4.2.22.

To give many more concrete examples of groups, we now focus on sets with *two* bijections, a and b, such that any two elements x, x' can be connected by repeated application of a and b and their inverses, such as the ones depicted in Figures 4.5 and 4.6, where we use the colors amaranth and bluebell to indicate the actions of a and b, respectively. We call these bicycles the *infinite dihedral* and the *quaternion* bicycle, respectively, for reasons that will become clear later.

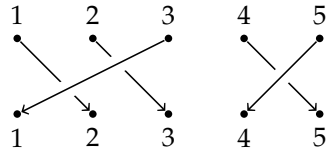


Figure 4.3: A different representation of the permutation σ from Figure 3.11.

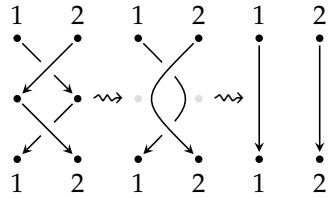


Figure 4.4: The composition (1 2)(1 2) = id_2 illustrated in the style of Figure 4.3, with first two, then no crossings.

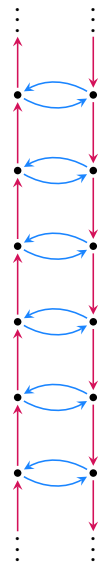


Figure 4.5: The infinite dihedral bicycle.

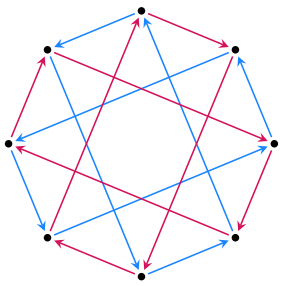


Figure 4.6: The quaternion bicycle.

To capture the idea of "connectedness" for bicycles, we note that it may be necessary to alternate the application of the two equivalences (and their inverses) an arbitrary number of times. One convenient way of formalizing this is via lists of elements of $\mathbb{Z} \amalg \mathbb{Z}$, where the left/right elements indicate a power of a/b, respectively. Given a type X with two self-equivalences $a, b : X \xrightarrow{\simeq} X$, we define the *meaning* $[\![\ell]\!] : X \xrightarrow{\simeq} X$ of such a list ℓ by induction, cf. Section 2.12.10:

$$\begin{aligned} [\![\varepsilon]\!] &:\equiv \mathrm{id}_X \\ [\![\mathrm{inl}_n\, \ell]\!] &:\equiv a^n \circ [\![\ell]\!] \\ [\![\mathrm{inr}_n\, \ell]\!] &:\equiv b^n \circ [\![\ell]\!] \end{aligned}$$

For example, we have $[\![\mathrm{inl}_3\, \mathrm{inr}_{-2}\, \mathrm{inl}_{-1}\, \mathrm{inr}_1]\!] = a^3 b^{-2} a^{-1} b$. With this in place, we can define the type of bicycles as follows:

def:bicycle

Definition 4.6.1. Let Bicyc be the subtype of $\sum_{X:\mathcal{U}} (X \to X) \times (X \to X)$ of those pairs (X, a, b) where X is a *nonempty* set with two *self-equivalences* a and b, such that any $x, x' : X$ are connected by a and b. Expressed in a formula:

$$\mathrm{Bicyc} :\equiv \sum_{X:\mathrm{Set}} \sum_{a:X \xrightarrow{\simeq} X} \sum_{b:X \xrightarrow{\simeq} X} \left(\|X\| \times \prod_{x,x':X} \exists_{\ell:(\mathbb{Z} \amalg \mathbb{Z})^*} (x' = [\![\ell]\!](x)) \right).$$

Elements of Bicyc are called *bicycles*. ⌟

rem:F2-first-look

Remark 4.6.2. In Section 8.7 we shall see that just like cycles are equivalently described as connected set bundles over the circle S^1, the bicycles are the connected set bundles over the type $S^1 \vee S^1$: two circles with their base points linked together. This type can also be constructed in analogy with S^1 as a higher inductive type with three constructors: a base point, $\bullet$, and *two* loops, $\circlearrowleft_1$ and $\circlearrowleft_2$, as depicted in Figure 4.7.

We shall also generalize to an arbitrary set S of self-equivalences, and the "S-fold cycles" will be the connected set bundles over the classifying type BF_S of the "free group" on S many generators. We postpone this, since it requires some machinery to show that BF_S is a groupoid. All in good time; first we need to learn to ride our bicycles![41] ⌟



Figure 4.7: The type $S^1 \vee S^1$ is a point with two loops attached.

fig:first-view-BF2

With the definition of bicycles in place, we can define the infinite dihedral and quaternion groups as automorphism groups:

def:Dinfty-Q

Definition 4.6.3. Letting $(\mathbb{Z} \amalg \mathbb{Z}, a, b)$ be the *standard infinite dihedral bicycle*, with

$$\begin{aligned} a(\mathrm{inl}_n) &:\equiv \mathrm{inl}_{n+1}, & a(\mathrm{inr}_n) &:\equiv \mathrm{inr}_{n-1}, \\ b(\mathrm{inl}_n) &:\equiv \mathrm{inr}_n, & b(\mathrm{inr}_n) &:\equiv \mathrm{inl}_n, \end{aligned}$$

we define the *infinite dihedral group* to be $D_\infty :\equiv \mathrm{Aut}_{\mathrm{Bicyc}}(\mathbb{Z} \amalg \mathbb{Z}, a, b)$.[44]

Similarly, letting $(\mathbb{8}, a, b)$ be the *standard quaternion bicycle*, with

$$a(k) :\equiv \begin{cases} k+1, & \text{if } k \text{ is even,} \\ k+3, & \text{if } k \text{ is odd} \end{cases}$$

$$b(k) :\equiv \begin{cases} k-1, & \text{if } k \text{ is even,} \\ k-3, & \text{if } k \text{ is odd} \end{cases}$$

(all operations modulo 8), we define the *quaternion group* to be $Q_8 :\equiv \mathrm{Aut}_{\mathrm{Bicyc}}(\mathbb{8}, a, b)$. ⌟

[41] Like "cycle", our use of "bicycle" is idiosyncratic. But just like cycles give rise to cyclic groups, bicycles give rise to a generalization of the notion of bicyclic groups, see Douglas[42,43].

[42] Jesse Douglas. "On finite groups with two independent generators. I–IV". in: *Proc. Nat. Acad. Sci. U.S.A.* 37 (1951), pp. 604–610, 677–691, 749–760, 808–813. DOI: 10.1073/pnas.37.9.604. DOI: 10.1073/pnas.37.10.677. DOI: 10.1073/pnas.37.11.749. DOI: 10.1073/pnas.37.12.808.

[43] Jesse Douglas. "On the supersolvability of bicyclic groups". In: *Proc. Nat. Acad. Sci. U.S.A.* 47 (1961), pp. 1493–1495. DOI: 10.1073/pnas.47.9.1493.

[44] We'll define more dihedral groups, and gain a new perspective on D_∞, in Section 8.2.

Now let us investigate the identifications of bicycles: If (X, a, b) and (X', a', b') are elements of $\sum_{X:\mathcal{U}}(X \to X) \times (X \to X)$, then univalence, together with Definition 2.7.3, Lemma 2.10.3, and Construction 2.14.2, gives an equivalence

$$\big((X, a, b) \xrightarrow{=} (X', a', b')\big) \xrightarrow{\simeq} \sum_{e : X \xrightarrow{\simeq} X'} (ea \xrightarrow{=} a'e) \times (eb \xrightarrow{=} b'e),$$

to a type whose three components we can visualize as:

$$\begin{array}{ccc} X & X & X \\ {\scriptstyle e}\downarrow\wr & {\scriptstyle e}\downarrow\wr & \wr\downarrow{\scriptstyle e} \\ X' & X' & X' \end{array} \qquad \begin{array}{ccc} & {\scriptstyle a} & \\ X & \longrightarrow & X \\ & {\scriptstyle a'} & \\ X' & \longrightarrow & X' \end{array} \qquad \begin{array}{ccc} & {\scriptstyle b} & \\ X & \longrightarrow & X \\ & {\scriptstyle b'} & \\ X' & \longrightarrow & X' \end{array}$$

If X and X' are sets, then this is the subtype of $X \to X'$ consisting of equivalences e satisfying $ea = a'e$ and $eb = b'e$. This means that the symmetries of a bicycle (X, a, b) are given by those self-equivalences $e : X \xrightarrow{\simeq} X$ that *commute* with both a and b in the sense that $ae = ae$ and $be = eb$.

We now see the added complexity of going from cycles to bicycles: For a (uni)cycle (X, t), any power t^n of t will commute with t itself, but for a bicycle (X, a, b), we need not have $ab = ba$. Indeed, neither of the bicycles in Figures 4.5 and 4.6 satisfies this. And there are many bicycles whose *only* symmetry is the identity, e.g., the one in Figure 4.8, or has fewer symmetries than desired, as in Figure 4.9.

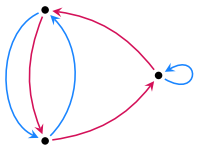


FIGURE 4.8: An "abnormal" bicycle with only the identity symmetry.

fig:abnormal-bicycle

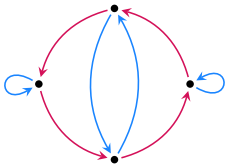


FIGURE 4.9: Another "abnormal" bicycle: It has four elements, but only two symmetries.

fig:somewhat-abnormal-bi

However, all is not lost! Since all elements are connected by two self-equivalences, we still have that any identification $(X, a, b) \xrightarrow{=} (X', a', b')$ is determined by the image of any given element $x : X$, giving a weakening of Corollary 3.6.15 for cycles.

lem:evisinj-bicycle

LEMMA 4.6.4. *Given bicycles (X, a, b) and (X', a', b'), for any $x_0 : X$, we have that the evaluation map*

$$\mathrm{ev}_{x_0} : \big((X, a, b) \xrightarrow{=} (X', a', b')\big) \to X', \qquad \mathrm{ev}_{x_0}(e) :\equiv e(x_0)$$

is injective.

Proof. Fix $x' : X'$. It suffices to show that there is at most one equivalence $e : X \xrightarrow{\simeq} X'$ satisfying $ea = a'e$, $eb = b'e$, and $e(x_0) = x'$. It follows by list induction on $\ell : (\mathsf{Z} \amalg \mathsf{Z})^*$ that $e[\![\ell]\!] = [\![\ell]\!]'e$, where $[\![_]\!]$ and $[\![_]\!]'$ use the respective pairs of self-equivalences, (a, b) and (a', b').

Now by connectivity, for every $x : X$ there exists a list ℓ with $x = [\![\ell]\!](x_0)$. Since we're proving a proposition (the uniqueness of the value of $e(x)$), we may assume we have such a list. But then $e(x) = e([\![\ell]\!](x_0)) = [\![\ell]\!]'e(x_0) = [\![\ell]\!]'x'$ is independent of e, as desired. □

This tells us what's special about the infinite dihedral and the quaternion bicycles: they are *normal*.[45]

def:normal-bicycle

DEFINITION 4.6.5. A bicycle (X, a, b) is *normal* if the evaluation map

$$\mathrm{ev}_x : \big((X, a, b) \xrightarrow{=} (X, a, b)\big) \to X, \qquad \mathrm{ev}_x(e) :\equiv e(x)$$

is an equivalence for all $x : X$. ⌟

In other words, a normal bicycle has the maximum possible amount of symmetry, in that any element is just like any other.

[45] What follows is a special case of a more general story that resumes in Definition 5.2.25 (for actions) and will be the focus of Chapter 9 on normal subgroups.

xca:normal-bicycle-equiv

Exercise 4.6.6. Show that if the evaluation map is an equivalence for some $x : X$, then its an equivalence for all $x : X$. ⌟

In other words, for a normal bicycle (X, a, b) there is a unique symmetry (i.e., permutation of X commuting with a and b) mapping any x to x' for any $x, x' : X$.

def:cbid-bicycle

Definition 4.6.7. Given a normal bicycle (X, a, b) with elements $x, x' : X$, let ${}_{x'}\square_x : (X, a, b) \xrightarrow{=} (X, a, b)$ be the symmetry that sends x to x'. ⌟

It follows that ${}_x\square_x = \mathrm{id}_X$ and ${}_{x''}\square_{x'} \circ {}_{x'}\square_x = {}_{x''}\square_x$. We also have that the inverse of $\mathrm{ev}_x : \big((X, a, b) \xrightarrow{=} (X, a, b)\big) \to X$ maps x' to ${}_{x'}\square_x$.

In Section 3.6 we used the subset $H_t \equiv \{\, n : \mathbb{Z} \mid t^n = \mathrm{id} \,\}$ of $\mathbb{Z}$ to study a cycle (X, t). There, we get the equal subsets $\{\, n : \mathbb{Z} \mid t^n(x) = x \,\}$ no matter which $x : X$ we pick.[46] For a bicycle (X, a, b), however, the relationship between the subsets

$$H_x :\equiv \{\, \ell : (\mathbb{Z} \amalg \mathbb{Z})^* \mid [\![\ell]\!](x) = x \,\}$$

for varying $x : X$ is exactly what determines normality. We leave this as an exercise now, as we'll return to normality in greater generality later, especially in Chapter 9.

Exercise 4.6.8. Show that a bicycle (X, a, b) is normal if and only if $H_x = H_y$ for all $x, y : X$. ⌟

Exercise 4.6.9. Show that any *commuting* bicycle (X, a, b), i.e., one satisfying $ab = ba$, is normal. Then show that the map

$$\mathrm{Cyc} \times \mathrm{Cyc} \to \mathrm{Bicyc}, \qquad ((X, t), (Y, u)) \mapsto (X \times Y, t \times \mathrm{id}_Y, \mathrm{id}_X \times u)$$

induces an equivalence onto the subtype of commuting bicycles.[47] ⌟

Assume now that we are given a normal bicycle (X, a, b) with a chosen element $x_0 : X$. We get a surjective map $[\![_]\!](x_0) : (\mathbb{Z} \amalg \mathbb{Z})^* \to X$, which induces an equivalence relation on $(\mathbb{Z} \amalg \mathbb{Z})^*$.

Exercise 4.6.10. Check that two lists $\ell, \ell' : (\mathbb{Z} \amalg \mathbb{Z})^*$ are equivalent if and only if $[\![\ell]\!] = [\![\ell']\!]$. ⌟

rem:bicycle-list-concat

Remark 4.6.11. Let us consider how list concatenation behaves with respect to the induced symmetries of (X, a, b). Note that if a symmetry maps x to x', then it also maps $[\![\ell]\!](x)$ to $[\![\ell]\!](x')$, since symmetries commute with a, b, and hence with $[\![\ell]\!]$. That is, ${}_{x'}\square_x = {}_{[\![\ell]\!](x')}\square_{[\![\ell]\!](x)}$. Then we can use $[\![\ell']\!][\![\ell]\!] = [\![\ell'\ell]\!]$ to calculate:

$$[\![\ell]\!](x_0)\square_{x_0} \circ [\![\ell']\!](x_0)\square_{x_0} = [\![\ell'\ell]\!](x_0)\square_{[\![\ell']\!](x_0)} \circ [\![\ell']\!](x_0)\square_{x_0} = [\![\ell'\ell]\!](x_0)\square_{x_0}$$
$$x_0\square_{[\![\ell]\!](x_0)} \circ x_0\square_{[\![\ell']\!](x_0)} = x_0\square_{[\![\ell]\!](x_0)} \circ [\![\ell]\!](x_0)\square_{[\![\ell\ell']\!](x_0)} = x_0\square_{[\![\ell\ell']\!](x_0)}$$

This is the punchline: To get concatenation of lists to correspond to composition of symmetries, we need to go backwards to the symmetry that takes us to x_0 from $[\![\ell]\!](x_0)$, rather than the other way round. ⌟

rem:inf-dihedral-frieze

Remark 4.6.12. The reader may have noticed that the symmetries of the infinite dihedral bicycle in Figure 4.5 can be realized as geometric symmetries of our picture of it, namely vertical translations and 180° rotations. In fact, our figure has the same symmetries as the frieze pattern of Figure 4.10. In Figure 4.11 we superimpose the bicycle on the frieze. We also fix an element x_0, which allows us to name all the elements via

[46] This is because all cycles (X, t) are normal in the general sense of Corollary 3.6.15.

[47] For example, the Klein four-group from Example 4.2.26 is equivalent to the automorphism group of the commuting bicycle:



Figure 4.10: A frieze pattern with infinite dihedral symmetry.

fig:first-frieze

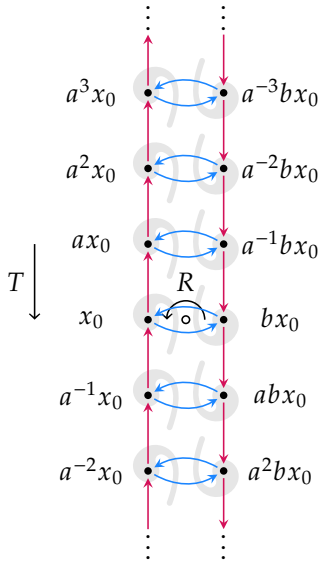


Figure 4.11: The frieze in Figure 4.10 with the infinite dihedral bicycle of Figure 4.5 superimposed.

fig:first-frieze-bicycle

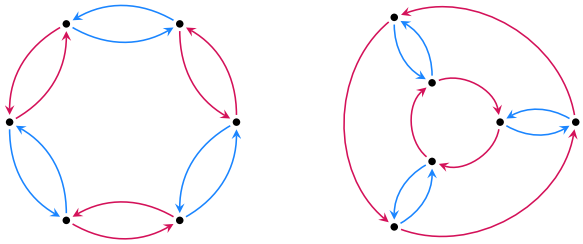
applications of a and b. Finally, we indicate two generating geometric transformations: T, a downwards translation, and R, a 180° rotation around the midpoint between x_0 and bx_0 (the white circle). In other words, $T = {}_{x_0}\square_{ax_0} = {}_{a^{-1}x_0}\square_{x_0}$ and $R = {}_{x_0}\square_{bx_0} = {}_{b^{-1}x_0}\square_{x_0}$. Notice that R can map elements quite far geometrically, for instance, $R(a^n x_0) = a^n b x_0$. In general, we have

$$T(\llbracket \ell \rrbracket(x_0)) = {}_{\llbracket \ell \rrbracket(a^{-1}x_0)}\square_{\llbracket \ell \rrbracket(x_0)}(\llbracket \ell \rrbracket(x_0)) = \llbracket \ell \,\mathrm{inl}_{-1} \rrbracket(x_0),$$
$$R(\llbracket \ell \rrbracket(x_0)) = {}_{\llbracket \ell \rrbracket(b^{-1}x_0)}\square_{\llbracket \ell \rrbracket(x_0)}(\llbracket \ell \rrbracket(x_0)) = \llbracket \ell \,\mathrm{inr}_{-1} \rrbracket(x_0),$$

so T/R amounts to appending $\mathrm{inl}_{-1}/\mathrm{inr}_{-1}$ to the *end* of the list, respectively, that names a given point. Conversely, if we named the points by applying T and R (and inverses) to x_0, then it would be the geometrically local operations a and b that would correspond to inserting T^{-1} and R^{-1} at the end. For example, $a(T^{-2}R(x_0)) = T^{-2}RT^{-1}(x_0)$. In fact, we already saw one manifestation of this in Figure 3.5 back in Section 3.5, and we'll return to this phenomenon several times throughout the book. We'll discuss friezes and other geometrical objects in more detail in Chapter 14. ⌟

Exercise 4.6.13. Construct an identification between the infinite dihedral bicycle (X, a, b) and its geometric cousin (X, T, R), where T and R are as in Figure 4.11. ⌟

Exercise 4.6.14. Two (normal) bicycles may represent the same group even though they belong to two different components of Bicyc: Construct an isomorphism between the automorphism groups of the bicycles below:



Then construct an isomorphism between either of these automorphism groups and the symmetric group Σ_3. ⌟

4.7 *Infinity groups* (∞-*groups*)

sec:infty-gps

Disregarding the requirement that the classifying type of a group G is a groupoid (so that UG is a set) we get the simpler notion of ∞-groups:

def:infty-gps

Definition 4.7.1. The type of ∞-groups is

$$\infty\mathrm{Group} :\equiv \mathrm{Copy}(\mathcal{U}_*^{>0}), \quad \text{where} \quad \mathcal{U}_*^{>0} :\equiv \sum_{A:\mathcal{U}} A \times \mathrm{isConn}(A)$$

is the type of pointed, connected types.

As for groups, we have the constructor $\underline{\Omega} : \mathcal{U}_*^{>0} \to \infty\mathrm{Group}$ and the destructor $\mathrm{B} : \infty\mathrm{Group} \to \mathcal{U}_*^{>0}$. ⌟

rem:pointedtypes

Remark 4.7.2. Just as "group" is a synonym for "pointed, connected groupoid" (wrapped with $\underline{\Omega}$), "∞-group" is a synonym for "pointed, connected type" (wrapped with $\underline{\Omega}$). As for pointed, connected groupoids,

we suppress the propositional information from the notation, and write (A, a) instead of $(A, a, !)$ for an pointed, connected type. ⌟

def:classifyingspace

Definition 4.7.3. Given $G : \infty\text{Group}$, the underlying pointed type $BG : \mathcal{U}_*$ is called the *classifying type* of G and $\text{sh}_G :\equiv \text{pt}_{BG}$ is called the *designated shape*. ⌟

Definition 4.7.4. For any type A with a specified point a, we define the *automorphism ∞-group* of $a : A$ by

$$\text{Aut}_A(a) :\equiv \underline{\Omega}(A_{(a)}, (a, !)),$$

i.e., $\text{Aut}_A(a)$ is the ∞-group with classifying type $\text{BAut}_A(a) \equiv (A_{(a)}, (a, !))$, the connected component of A containing a, pointed at a. ⌟

rem:autinfgp

Remark 4.7.5. It can certainly happen that the connected component of A containing a is groupoid, even though A itself is not a groupoid. For example, consider a type universe $\mathcal{U}$ and a *set* $S : \mathcal{U}$. Then $\mathcal{U}_{(S)}$ is a groupoid, and the automorphism ∞-group $\text{Aut}_{\mathcal{U}}(S)$ is an ordinary group.

Because we have an inclusion ${\mathcal{U}_*}^{=1} \hookrightarrow {\mathcal{U}_*}^{>0}$, we get a corresponding injection $\text{Group} \hookrightarrow \infty\text{Group}$. ⌟

Definition 4.7.6. A homomorphism of ∞-groups is a pointed function of classifying types, i.e., given two ∞-groups G and H,we define

$$\text{Hom}(G, H) :\equiv \text{Copy}(BG \to_* BH).$$

Given $f \equiv \underline{\Omega}Bf : \text{Hom}(G, H)$, we call $Bf : BG \to_* BH$ the *classifying map* of f. ⌟

5
Actions

ch:actions

Historically, groups have appeared because they can "act" on a set (or more general objects), that is to say, they collect some of the symmetries of the set. This is a point of view that we will return to many times and we give the basic theory in Section 5.2. This section should remind the reader of the material in Chapter 3, where we dealt with the special case of the group of integers. More generally, connected set bundles now reappear in the guise of "transitive G-sets", and these are intimately related to the set of subgroups of a group. These also generalize the bicycles of Section 4.6, from which we lift the notion of "normality".

A central notion in this chapter is that of "G-torsor", introduced in Section 5.2. The main result about G-torsors is that their type can be identified with the classifying type of the group G. In other words, the group G can be completely recovered from its torsors. This idea will be essential in Chapter 7 where we show that our definition of concrete groups is equivalent to the abstract version taught in most algebra classes.

5.1 *Brief overview of the chapter*

After setting things up in Section 5.2, and studying subgroups in Section 5.3, we introduce the important operations of taking *invariant maps* and *orbits* of an action in Section 5.4. The fundamental equivalence between the classifying type BG of a group G and the type of G-torsors is constructed in Section 5.5. In Section 5.6 we apply G-torsors to prove Cayley's Theorem for our groups, and in Section 5.7 we begin the study of the combinatorics of group actions. This allows us to count, for instance, how many ways there are of "coloring" objects acted on by groups, and it lays the groundwork for the combinatorics of finite groups we'll be looking at in Chapter 10.

5.2 *Group actions* (G-*sets*)

sec:gsets

One of the most important features of groups: they can *act* on things (giving rise to manifestations of symmetries)! For example, in the circle S^1, the loop $\circlearrowleft$ is a non-trivial transformation of the base point $\bullet$. This can *act* on other mathematical entities: Take for example the 2-fold set bundle of S^1 whose fiber at $\bullet$ is 2. The symmetry $\circlearrowleft$ acts on its two elements by swapping them. The following definition decribes that phenomenon for arbitrary groups.

def:Gset

Definition 5.2.1. For G a group, a *G-set* is a function[1]

[1] Recall that, to make things more readable, we write $BG \to \mathrm{Set}$ instead of the more correct $BG_\circ \to \mathrm{Set}$.

$$X : BG \to \mathrm{Set},$$

and $X(\mathrm{sh}_G)$ is referred to as the *underlying set*. If $p : x \xrightarrow{=} y$ in BG, then the transport function $\mathrm{trp}_X(p) : X(x) \xrightarrow{\simeq} X(y)$ is also denoted by $X(p)$. We denote $X(p)(a)$ by $p \cdot_X a$. The operation $\cdot_X$ is called the *group action* of X. When X is clear from the context we may leave out the subscript X.[2] In particular, if $g : UG$, then $X(g)$ is a permutation of the underlying set $X(\mathrm{sh}_G)$ of X.

The type of G-sets is

$$G\text{-Set} :\equiv (BG \to \mathrm{Set}). \qquad \lrcorner$$

def:trivGset

Example 5.2.2. If G is a group and X is a set, then $\mathrm{triv}_G X$ defined by

$$\mathrm{triv}_G X(z) :\equiv X, \quad \text{for all } z : BG,$$

is a G-set. Examples of this sort (regardless of X) are called *trivial G-sets*. $\lrcorner$

rem:G-set-vs-set-bundle

Remark 5.2.3. The reader may have noticed that the type of G-sets is equivalent to the type of set bundles over BG. The reason we have allowed ourselves two names is that our focus is different: for a G-set $X : BG \to \mathrm{Set}$ we focus on the sets $X(z)$, whereas when talking about set bundles the first projection $\sum_{z:BG} X(z) \to BG$ takes center stage. Each focus has its advantages. $\lrcorner$

def:principaltorsor

Example 5.2.4. If G is a group, then

$$\mathbb{P}_{\mathrm{sh}_G} : BG \to \mathrm{Set}, \qquad \mathbb{P}_{\mathrm{sh}_G}(z) :\equiv (\mathrm{sh}_G \xrightarrow{=} z)$$

is a G-set called the *principal G-torsor*.[3] We've seen this family before in the guise of (preimages of) the "universal set bundle" of Example 3.3.9.

There is nothing sacred about starting the identification $\mathrm{sh}_G \xrightarrow{=} z$ at sh_G. Define more generally

eq:pathsp

$$\mathbb{P}_{_} : BG \to G\text{-Set}, \qquad \mathbb{P}_y :\equiv (z \mapsto (y \xrightarrow{=} z)), \tag{5.2.1}$$

Applying $\mathbb{P}_{_}$ to a path $q : y \xrightarrow{=} y'$ induces an equivalence from $\mathbb{P}_y$ to $\mathbb{P}_{y'}$ that sends $p : y \xrightarrow{=} z$ to $pq^{-1} : y' \xrightarrow{=} z$. As a matter of fact, Theorem 5.5.7 will identify BG with the type of G-torsors via the map $\mathbb{P}_{_}$, using the full transport structure of the identity type $\mathbb{P}_y(z) \equiv (y \xrightarrow{=} z)$.

Note that the underlying set of $\mathbb{P}_{\mathrm{sh}_G}$ is

$$\mathbb{P}_{\mathrm{sh}_G}(\mathrm{sh}_G) \equiv (\mathrm{sh}_G \xrightarrow{=} \mathrm{sh}_G) \equiv UG,$$

the underlying symmetries of G. $\lrcorner$

def:adjointrep

Example 5.2.5. If we vary both ends of the identifications simultaneously, we get another G-set: If G is a group, then

$$\mathrm{Ad}_G : BG \to \mathrm{Set}, \qquad \mathrm{Ad}_G(z) :\equiv (z \xrightarrow{=} z)$$

is a G-set called the *adjoint G-set*.[4] By the induction principle for the circle, the type

fr:adjoint-transport

$$\sum_{z:BG} \mathrm{Ad}_G(z) \equiv \sum_{z:BG} (z \xrightarrow{=} z)$$

is equivalent to the type of (unpointed!) maps $S^1 \to BG$, known in other contexts as the *free loop space* of BG, an apt name given that it is

[2] The reader might worry that this notation conflicts with that of path composition. The type of the operation $\cdot : (x \xrightarrow{=} y) \to X(x) \to X(y)$ helps identify which is which. However, see Example 5.2.4 for a special case where both conflates (i.e., where $\cdot_X$ is indeed path composition).

Much of what follows will work equally well for ∞-groups; if G is (a group or) an infinity group, a *G-type* is a function $X : BG \to \mathcal{U}$, with *underlying type* $X(\mathrm{sh}_G)$. This is an *action in* $\mathcal{U}$, and more generally, an action of G on an element of type A is a function $X : BG \to A$, see Section 5.2.27 below.

[3] The term "G-torsor" will reappear several times and will mean nothing but a G-set in the component of $\mathbb{P}_{\mathrm{sh}_G}$ – a "twisted" version of $\mathbb{P}_{\mathrm{sh}_G}$.

[4] Note that Ad_G also makes sense for ∞-groups. With the name "adjoint" we conform to usual terminology. The action of Ad_G works as conjugation: if $p : y \xrightarrow{=} z$, then $\mathrm{Ad}_G(p) : (y \xrightarrow{=} y) \xrightarrow{\simeq} (z \xrightarrow{=} z)$ is given by:

$$\mathrm{Ad}_G(p)(q) \xrightarrow{=} pqp^{-1} \text{ in } z \xrightarrow{=} z.$$

The picture

$$\begin{array}{ccc} y & \xrightarrow{p} & z \\ {\scriptstyle q}\downarrow{\scriptstyle =} & & {\scriptstyle =}\downarrow{\scriptstyle \mathrm{Ad}_G(p)(q)} \\ y & \xrightarrow[p]{=} & z \end{array}$$

is a mnemonic device illustrating that it couldn't have been different, and should be contrasted with the picture for $\mathbb{P}_{\mathrm{sh}_G}(p) : (\mathrm{sh}_G \xrightarrow{=} y) \xrightarrow{\simeq} (\mathrm{sh}_G \xrightarrow{=} z)$:

$$\begin{array}{ccc} \mathrm{sh}_G & \xrightarrow{\mathrm{refl}_{\mathrm{sh}_G}} & \mathrm{sh}_G \\ {\scriptstyle q}\downarrow{\scriptstyle =} & & {\scriptstyle =}\downarrow{\scriptstyle \mathbb{P}_{\mathrm{sh}_G}(p)(q)} \\ y & \xrightarrow[p]{=} & z. \end{array}$$

the type of "all symmetries in BG." Under that equivalence, the first projection $\sum_{z:BG} \mathrm{Ad}_G(z) \to BG$ correspond to the evaluation function $(S^1 \to BG) \to BG$ at the base point $\bullet$. ⌟

ex:HomHasGset

Example 5.2.6. Let G and H be groups. Recall that the type $\mathrm{Hom}(H, G)$ of homomorphisms from H to G is a set (Lemma 4.4.12). We will define group actions on $\mathrm{Hom}(H, G)$ by moving the shapes of G and H as in Example 4.4.21: Reusing the notation $\mathrm{Hom}(H, G)$, define for any $x : BH$ and $y : BG$

$$\mathrm{Hom}(H, G)(x, y) :\equiv \mathrm{Hom}(\underline{\Omega}(BH_{\div}, x), \underline{\Omega}(BG_{\div}, y)).$$

Alternatively, by Definition 2.21.1 and Definition 4.4.2, we have

$$\mathrm{Hom}(H, G)(x, y) \equiv \mathrm{Copy}_{\underline{\Omega}}\left(\sum_{f : BH_{\div} \to BG_{\div}} (y \xrightarrow{=} f(x))\right).$$

The type $\mathrm{Hom}(H, G)$ may be considered to be a $(H \times G)$-set:

$$\mathrm{Hom}(H, G) : (BH \times BG) \to \mathrm{Set},$$

and we shall be particularly interested in the restriction to G, giving a G-set for which we again reuse the notation:

$$\mathrm{Hom}(H, G)(y) :\equiv \mathrm{Hom}(H, G)(\mathrm{sh}_H, y).$$ ⌟

xca:HomZGvsAdG

Exercise 5.2.7. Provide an identification between the G-sets Ad_G and $\mathrm{Hom}(\mathbb{Z}, G)$ of Examples 5.2.5 and 5.2.6.[5] ⌟

def:map-of-Gsets

Definition 5.2.8. If G is a group and X, Y are G-sets,[6] then a *map from X to Y* is an element of the set

$$\mathrm{Hom}_G(X, Y) :\equiv \prod_{z:BG} (X(z) \to Y(z)).$$

When f is such a map, we may write f_z for $f(z)$.[7] ⌟

xca:equivariant-map-totalization

Exercise 5.2.9. Show that $\mathrm{Hom}_G(X, Y)$ is equivalent to the type of functions from the totalization of X to that of Y that commute to the projections, i.e., $\sum_{f : \mathrm{Tot}(X) \to \mathrm{Tot}(Y)} \mathrm{fst} \xrightarrow{=} \mathrm{fst} \circ f$. ⌟

rem:map-of-Gsets

Remark 5.2.10. Given G-sets X, Y and a map f from X to Y, we have $f_w(g \cdot_X x) = g \cdot_Y f_z(x)$ for all $z, w : BG$, $x : X(z)$, $g : z \xrightarrow{=} w$. In other words, the diagram on the right commutes:

$$\begin{array}{ccc} z & X(z) & \xrightarrow{f_z} & Y(z) \\ {\scriptstyle g}\downarrow{\scriptstyle =} & {\scriptstyle g\cdot_X -}\downarrow & & \downarrow{\scriptstyle g\cdot_Y -} \\ w & X(w) & \xrightarrow[f_w]{} & Y(w) \end{array}$$

An important special case is when Y is the trivial G-set $\mathrm{triv}_G\mathrm{Prop}$ that is constantly the set Prop: Given a map P from X to $\mathrm{triv}_G\mathrm{Prop}$, we have $P_w(g \cdot x)$ if and only if $P_z(x)$ for all $z, w : BG$, $x : X(z)$, $g : z \xrightarrow{=} w$. This applies to the following definition. ⌟

def:Gsubset

Definition 5.2.11. A *G-subset* of a G-set X is a map from X to the G-set $\mathrm{triv}_G\mathrm{Prop}$. The type of all such maps is denoted[8]

ft:SubTotX

$$\mathrm{Sub}_G(X) :\equiv \mathrm{Hom}_G(X, \mathrm{triv}_G\mathrm{Prop}) \equiv \prod_{z:BG} \mathrm{Sub}(X(z)).$$

[5] Hint: This is similar to Example 4.4.17: identify $\mathrm{Hom}(\mathbb{Z}, G)(y)$ with $\sum_{z:BG} \sum_{p:z \xrightarrow{=} z} (y \xrightarrow{=} z)$ and use Lemma 2.9.10.

[6] This definition generalizes to ∞-groups and G-types.

[7] Such a map f is called a map of families in Definition 2.9.14.

[8] Recall Definition 2.20.3: $\mathrm{Sub}(T) \equiv (T \to \mathrm{Prop})$.

[9] The type $\mathrm{Sub}_G(X)$ can be uncurried (Exercise 2.9.26) as $\mathrm{Tot}(X) \to \mathrm{Prop}$, the type of subtypes of $\mathrm{Tot}(X) \equiv \sum_{z:BG} X(x)$ (Definition 2.20.3).

Similarly to Corollary 2.20.12, $\mathrm{Sub}_G(X)$ is a set.[9] If P is a G-subset of X, then the *underlying G-set of P*, denoted by X_P, is defined by

$$X_P(z) :\equiv \sum_{x : X(z)} P(z)(x), \quad \text{for all } z : BG. \qquad \lrcorner$$

xca:SubGX-closedSubXshG

Exercise 5.2.12. Show that evaluation at sh_G is an equivalence from $\mathrm{Sub}_G(X)$ to

$$\sum_{Q\,:\,\mathrm{Sub}(X(\mathrm{sh}_G))} \prod_{x\,:\,X(\mathrm{sh}_G)} \Big(Q(x) \to \prod_{g\,:\,UG} Q(g \cdot x)\Big).$$

The latter type is the type of all subsets of $X(\mathrm{sh}_G)$ that are closed under the group action. $\lrcorner$

The following exercise wil be used in the subsequent remark.

xca:prd-conn-to-comp

Exercise 5.2.13. Let (A, a) and (B, b) be pointed types and let A be connected. Give an equivalence from $(A, a) \to_* (B, b)$ to $(A, a) \to_* (B_{(b)}, (b, !))$. $\lrcorner$

rem:GSet-SetBGG

Remark 5.2.14. A G-set X is often presented by focusing on the underlying set $X(\mathrm{sh}_G)$ and providing it with a structure relating it to G determining the entire function $X : BG \to \mathrm{Set}$. More precisely, since BG is connected, using Exercise 5.2.13, we have the following chain of easy equivalences:

$$\begin{aligned}
G\text{-Set} &\equiv (BG_\div \to \mathrm{Set}) \\
&\xrightarrow{\simeq} \sum_{S\,:\,\mathrm{Set}} \sum_{X\,:\,(BG_\div \to \mathrm{Set})} (S \xrightarrow{\equiv} X(\mathrm{sh}_G)) \\
&\equiv \sum_{S\,:\,\mathrm{Set}} (BG \to_* (\mathrm{Set}, S)) \\
&\xrightarrow{\simeq} \sum_{S\,:\,\mathrm{Set}} (BG \to_* (\mathrm{Set}_{(S)}, (S, !))) \\
&\xrightarrow{\simeq} \sum_{S\,:\,\mathrm{Set}} \mathrm{Hom}(G, \Sigma_S)
\end{aligned}$$

Hence a G-set X can, without loss of information, be considered as a set $X(\mathrm{sh}_G)$ and a homomorphism from G to the permutation group of $X(\mathrm{sh}_G)$. Conversely, a set S together with a homomorphism f from G to Σ_S can be described as a G-set with underlying set S and whose group action $(g \cdot _)$ for $g : UG$ is $f(g)$.[10] $\lrcorner$

[10] The G-set associated to the homomorphism f derives from the "delooping" of f, and it will be formally defined in 7.5.

def:Gaction

Definition 5.2.15. If G is a group and S is a set, then an *action* of G on S is a homomorphism from G to the permutation group of Σ_S of S. $\lrcorner$

By the construction in Remark 5.2.14 we identify G-sets and sets with an action of G on a set.

xca:Ad-triv-abelian

Exercise 5.2.16. Prove that a group G is abelian if and only if the G-sets Ad_G and $\mathrm{triv}_G(UG)$ are identical. $\lrcorner$

xca:Ad-princ-trivial

Exercise 5.2.17. Prove that a group G is the trivial group if and only if the G-sets Ad_G and $\mathbb{P}_{\mathrm{sh}_G}$ are identical. $\lrcorner$

def:finite-G-set

Definition 5.2.18. Let G be a group and $X : BG \to \mathrm{Set}$ a G-set. We say X is *finite* if the underlying set $X(\mathrm{sh}_G)$ is finite.[11] For any finite G-set X we denote the number of elements in $X(\mathrm{sh}_G)$ by $\#(X)$, also called the *cardinality* of X. $\lrcorner$

[11] Since BG is connected, $X(\mathrm{sh}_G)$ is finite if and only if $X(z)$ is finite for any $z : BG$. Similarly, for any $n : \mathbb{N}$, $X(\mathrm{sh}_G)$ is an n-element set if and only if $X(z)$ is such for any $z : BG$.

sec:transitiveGsets

5.2.19 *Transitive G-sets*

We saw in Chapter 3 that *connected set bundles* play a special role: In the case of the circle, classifying the group of integers $\mathbb{Z}$, they correspond to cycles (Corollary 3.6.4).

We hinted there that they are connected to subgroups, so we now study them over a general group G. When seen as G-sets, connected set bundles over BG are called transitive G-sets:

def:transitiveGset

Definition 5.2.20. A G-set $X : BG \to \mathrm{Set}$ is *transitive* if the proposition[12]

$$\mathrm{isTrans}(X) :\equiv \exists_{x : X(\mathrm{sh}_G)} \prod_{y : X(\mathrm{sh}_G)} \exists_{g : UG} (x = g \cdot y)$$

holds. ⌟

[12] This is a natural translation in the concrete world of the usual definition of transitivity in ordinary group theory: An action of an abstract group of a set $\mathcal{X}$ is transitive when there exists x in $\mathcal{X}$ such that for all y in $\mathcal{X}$ there exists an element g of the group such that $x = g \cdot y$.

Remark 5.2.21. In other words, X is transitive if and only if there exists some $x : X(\mathrm{sh}_G)$ such that the map $_ \cdot x : UG \to X(\mathrm{sh}_G)$ is surjective.

Note also that by connectedness (cf. Exercise 2.16.9) it is equivalent to demand this over all $z : BG$:

eq:Gset-trans-gen

$$\prod_{z : BG} \exists_{x : X(z)} \prod_{y : X(z)} \exists_{g : z \xrightarrow{=} z} (x = g \cdot y). \tag{5.2.2}$$

Yet another equivalent way of expressing that X is transitive is to say that $X(\mathrm{sh}_G)$ is nonempty and for any $x, y : X(\mathrm{sh}_G)$ there exists some $g : UG$ with $x = g \cdot y$. Note that the empty G-set is not transitive. ⌟

lem:conistrans

Lemma 5.2.22. *A G-set is transitive if and only if the associated set bundle is connected* (*see Definition 3.3.1*).

Proof. Consider a G-set $X : BG \to \mathrm{Set}$ and its associated set bundle $\mathrm{fst} : \mathrm{Tot}(X) \to BG$. Recall that $\mathrm{Tot}(X) :\equiv \sum_{w : BG} X(w)$. Now, $\mathrm{Tot}(X)$ is connected if and only if it is nonempty and for all $z : BG$ and $x : X(z)$ and all $w : BG$ and $y : X(w)$ there exists $g : z \xrightarrow{=} w$ such that $y = g \cdot x$. Since BG is connected, this is equivalent to asserting that $\mathrm{Tot}(X)$ is nonempty and for all $x, y : X(\mathrm{sh}_G)$ there exists $g : UG$ such that $y = g \cdot x$. □

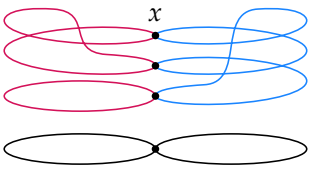


Figure 5.1: An $\underline{\Omega}(S^1 \vee S^1)$-set X for which ev_x is not surjective. At the bottom the type $S^1 \vee S^1$ is visualized as two circles with a common base point. Note that the underlying set of X with the red and the blue permutation is a bicycle in the sense of Definition 4.6.1.

The next lemma is an analog of Corollary 3.6.15 (for cycles), and a generalization of Lemma 4.6.4 (for bicycles). The action in Figure 5.1 corresponds to the bicycle back in Figure 4.8 (and reproduced in Figure 5.2) illustrates what can go wrong.

lem:evisinjwhentransitive

Lemma 5.2.23. *Let $X, Y : BG \to \mathrm{Set}$ be G-sets. Let $z : BG$ and $x : X(z)$. If X is transitive, then the evaluation map*

$$\mathrm{ev}_x : \mathrm{Hom}_G(X, Y) \to Y(z), \qquad \mathrm{ev}_x(f) :\equiv f_z(x)$$

is injective.[13]

Proof. We show that for any $y : Y(z)$, there is at most one $f : \mathrm{Hom}_G(X, Y)$ such that $f_z(x) = y$. Let $f, f' : \mathrm{Hom}_G(X, Y)$ such that $f_z(x) = y = f'_z(x)$. Let $w : BG$ and $x' : X(w)$. It suffices to show that $f_w(x') = f'_w(x')$. Since the latter is a proposition, we may assume (by the transitivity of X, using Lemma 5.2.22) that we have a $g : z \xrightarrow{=} w$ such that $g \cdot_X x = x'$. Using Remark 5.2.10, we have

$$f_w(x') = f_w(g \cdot_X x) = g \cdot_Y f_z(x) = g \cdot_Y f'_z(x) = f'_w(g \cdot_X x) = f'_w(x'). \quad \square$$

fig:not-normal

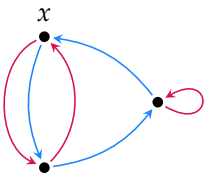


Figure 5.2: Alternative representation of the $\underline{\Omega}(S^1 \vee S^1)$-set X from Figure 5.1, using colors and arrows to represent which parts lies over which circle in which orientation.

[13] Recall that for type families $X, Y : T \to \mathcal{U}$, and $f : \prod_{z:T}(X(z) \to Y(z))$, we may write $f_z : (X(z) \to Y(z))$ (instead of the more correct $f(z)$) for its evaluation at $z : T$.

fig:not-normal-graph

In Section 9.5, in our study of *normal* subgroups, we will see exactly when we also get surjectivity of the evaluation map.

Via function extensionality, the identity type $X \xrightarrow{=} Y$, for G-sets X, Y is a subtype of the type $\mathrm{Hom}_G(X, Y)$. Hence we also have that evaluation at some $x : X(z)$, for any given $z : BG$, is an injection

$$\mathrm{ev}_x : (X \xrightarrow{=}_{G\text{-Set}} Y) \to Y(z).$$

xca:not-normal

Exercise 5.2.24. Reverse engineer the $\underline{\Omega}(S^1 \vee S^1)$-set X in Figures 5.1 and 5.2. Show that $X \xrightarrow{=} X$ is contractible. Conclude that ev_x, while injective, is not surjective. (Hint: the induction principle for $S^1 \vee S^1$ is a generalization of the induction principle for the circle to two loops.) ⌟

We can now generalize the definition of normal bicycle from Definition 4.6.5 to transitive G-sets:

def:normal-action

Definition 5.2.25. A transitive G-set $X : BG \to \mathrm{Set}$ is *normal* if the evaluation map

$$\mathrm{ev}_x : \left(X \xrightarrow{=}_{G\text{-Set}} X\right) \to X(z), \qquad \mathrm{ev}_x(e) :\equiv e_z(x)$$

is an equivalence for all $z : BG$ and $x : X(z)$. ⌟

xca:normal-action-equiv

Exercise 5.2.26. Show that if the evaluation map is an equivalence for some pair of a $z : BG$ and a $x : X(z)$, then it is an equivalence for all such pairs. (This generalizes Exercise 4.6.6.) ⌟

5.2.27 *Actions in a type*

sec:actions

Oftentimes it is interesting not to have an action on a set, but on an element in any given type (not necessarily the type of sets). For instance, a group can act on another, giving rise to the notion of the semidirect product in Section 8.2. We will return to these more general types of actions many times.

def:action

Definition 5.2.28. If G is any group[14] and A is any type, then we define an *action of G in A* as a function

$$X : BG \to A.$$

[14] Even an ∞-group in the sense of Section 4.7.

The particular "object of type A being acted on" is $X(\mathrm{sh}_G) : A$,

Fixing $a : A$ as the underlying object, we define an *action of G on a* to be a homomorphism from G to $\mathrm{Aut}_A(a)$. ⌟

This generalizes our earlier definition of G-sets $X : BG \to \mathrm{Set}$ from Definition 5.2.1, and harmonizes with Remark 5.2.14, relating G-sets and actions of G on a set. Indeed, we identify an action of G in A with a pair of an underlying object $a : A$ and an action of G on a:

$$(BG \to A) \xrightarrow{\simeq} \sum_{a : A} \mathrm{Hom}(G, \mathrm{Aut}_A(a))$$

This equivalence, hinted at in Exercise 4.4.19, maps an action $X : BG \to A$ to the pair consisting of $a :\equiv X(\mathrm{sh}_G)$ and the homomorphism classified by the pointed map from BG to the pointed component $A_{(a)}$ given by X.

std-action

Definition 5.2.29. The *standard action* of G on its designated shape sh_G is obtained by taking $A :\equiv BG$ and $X :\equiv \mathrm{id}_{BG}$. ⌟

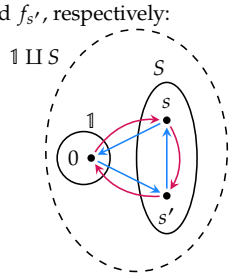
ex:S2-acts-on-C3

Example 5.2.30. The symmetric group Σ_2 acts on the cyclic group C_3 as follows. Given a 2-element set S consider the type $\sum_{X:\mathrm{Set}}(S \to (X \to X))$ of pairs (X, f) of a set X and a "pair" of functions $f_s : X \to X$ (one for each $s : S$). In this type we have the element $(\mathbb{1} \amalg S, f)$, consisting of the 3-element set $\mathbb{1} \amalg S$ and the function $f : S \to ((\mathbb{1} \amalg S) \to (\mathbb{1} \amalg S))$ defined by

$$\begin{aligned} f_s(\mathrm{inl}_0) &:\equiv \mathrm{inr}_s, \\ f_s(\mathrm{inr}_s) &:\equiv \mathrm{inr}_{\mathrm{swap}(s)}, \\ f_s(\mathrm{inr}_{\mathrm{swap}(s)}) &:\equiv \mathrm{inl}_0. \end{aligned}$$

Then $G(S) :\equiv \mathrm{Aut}_{\sum_{X:\mathrm{Set}} S \to X \to X}(\mathbb{1} \amalg S, f)$ defines an action $B\Sigma_2 \to \mathrm{Group}$.[15] Furthermore, we identify $G(\mathrm{Bool})$ with BC_3 by mapping a shape (X, f) in $BG(\mathrm{Bool})$ to the 3-cycle (X, f_{yes}) and identifying the 3-cycle $(\mathbb{1} \amalg \mathrm{Bool}, f_{\mathrm{yes}})$, for the f defined above, with the standard 3-cycle $(\mathbb{3}, \mathrm{s})$, correlating inl_0 with $0 : \mathbb{3}$. ⌟

[15] If S is $\{s, s'\}$, then we can picture the designated shape as follows, where the blue and red arrows denote f_s and $f_{s'}$, respectively:



xca:AutC3

Exercise 5.2.31. Show that action of Σ_2 on C_3 from Example 5.2.30 gives an identification $\Sigma_2 \xrightarrow{\simeq} \mathrm{Aut}(C_3)$. ⌟

Example 5.2.32. By composing constructions we can build new actions starting from simple building blocks. For example, the standard action of symmetric group Σ_n is to permute the elements of the standard n-element set $\mathbb{n}$. Composing with the projection $B\Sigma_n \to \mathrm{Set}$, we get the corresponding standard Σ_n-set.[16] Composing further with the operation $_ \to \mathrm{Bool} : \mathrm{Set} \to \mathrm{Set}$, mapping any set S to the set $S \to \mathrm{Bool}$, we get the action of Σ_n on the set of decidable subsets of $\mathbb{n}$. ⌟

[16] Check that this action is transitive for $n > 0$.

5.3 *Subgroups*

sec:subgroups

In our discussion of the group $\mathbb{Z} :\equiv \mathrm{Aut}_{S^1}(\bullet)$ of integers in Chapter 3 we discovered that some of the symmetries of $\bullet$ were picked out by the degree m function $\delta_m : S^1 \to S^1$ (for some particular natural number $m > 0$, see Definition 3.6.5). At the level of the set $U\mathbb{Z} :\equiv (\bullet \xrightarrow{\simeq} \bullet)$, the symmetries picked out are all the iterates (positive or negative or even zero-fold) of $\circlearrowleft^m$. The important thing is that we can compose or invert any of the iterates of $\circlearrowleft^m$ and get new symmetries of the same sort (because of distributivity $mn_1 + mn_2 = m(n_1 + n_2)$).

The case of $m = 0$ is special. The iterates of $\circlearrowleft^0$, i.e., of $\mathrm{refl}_\bullet$, can also be composed and inverted, never to give something else than $\circlearrowleft^0$ itself. We can pick out the single symmetry $\circlearrowleft^0$ by the constant map $\mathrm{cst}_\bullet : \mathbb{1} \to S^1$.

In every case, while we do not get all symmetries of $\bullet$ (unless $m = 1$), we get a set of these that behave, under the group laws of $\mathbb{Z}$, as the set of symmetries of some shape. This phenomenon is captured by the notion of *subgroup*.

Formally speaking, we can sum up the situation for $\mathbb{Z}$ as follows. Both δ_m and $\mathrm{cst}_\bullet$ can trivially be pointed to make them into classifying maps of homomorphisms that are injections on the respective sets of symmetries. Using Corollary 3.6.10, each δ_m is a pointed connected set bundle over the circle, and $\mathrm{cst}_\bullet$ is even the universal set bundle by Lemma 3.3.11. Finally, Lemma 5.2.22 lets us view these as transitive $\mathbb{Z}$-sets with a chosen element in their underlying sets. We adopt this last point of view to genereralize the notion to any group G.

5.3.1 *Subgroups through G-sets*

def:set-of-subgroups

DEFINITION 5.3.2. For any group G, define the type of *subgroups of* G as

$$\mathrm{Sub}(G) :\equiv \sum_{X : BG \to \mathrm{Set}} X(\mathrm{sh}_G) \times \mathrm{isTrans}(X). \qquad \lrcorner$$

xca:group-XX

EXERCISE 5.3.3. Show that, for any transitive G-set X, the type $\sum_{z:BG} X(z)$ above is a connected groupoid. Hint: use Lemma 5.2.22. $\lrcorner$

The reader could be surprised that the definition of a subgroup of a group G is **not** that of a group with some relationship to G. Let us thus unfold a bit the definition above. Given a subgroup[17] $(X, \mathrm{pt}) : \mathrm{Sub}(G)$ in the sense above, we can consider the total type $\mathrm{Tot}(X) :\equiv \sum_{z:BG} X(z)$ of the transitive G-set X. Together with the first projection, it makes a connected set bundle over BG and its fiber (at sh_G) contains a particular point, namely $(\mathrm{sh}_G, \mathrm{pt})$. Exercise 5.3.3 above tells us that this choice of point makes a group. The first projection then becomes (the classifying map of) a homomorphism from that group to G that embeds[18] the symmetries of $(\mathrm{sh}_G, \mathrm{pt})$ in $\mathrm{Tot}(X)$ into these of sh_G in BG. This explain the following definition:

[17] To lighten the notation, we leave out the proof that X is transitive. (Otherwise, we would write $(X, \mathrm{pt}, !) : \mathrm{Sub}(G)$.) In Remark 5.3.24 below we'll set out further notational conveniences regarding subgroups.

[18] More will be said on that "embedding" in Section 5.3.12.

def:underlying-group-of-subgroup

DEFINITION 5.3.4. Let G be a group. The *underlying group* of a subgroup $(X, \mathrm{pt}) : \mathrm{Sub}(G)$ is

$$\underline{\Omega}\left(\mathrm{Tot}(X), (\mathrm{sh}_G, \mathrm{pt})\right). \qquad \lrcorner$$

ex:Rm0-subgroup

EXAMPLE 5.3.5. Recall from Definition 3.6.7 the $\mathbb{Z}$-set $R_m : S^1 \to \mathrm{Set}$ defined by $R_m(\bullet) :\equiv m$ and $R_m(\circlearrowleft) := \bar{s}$. Here $m > 0$ so that we can point R_m by $0 : R_m(\bullet)$.[19] Transitivity of R_m is obvious. Which symmetries $p : \bullet \xrightarrow{=} \bullet$ are picked out by R_m, that keep the point $0 : R_m(\bullet)$ in place? Those that satisfy $R_m(p)(0) = 0$, i.e., $p = \circlearrowleft^{mk}$ for some integer k. Given α_m in Construction 3.6.9, it should not come as a surprise that these are precisely the symmetries picked out by δ_m.

[19] Any element of m would do.

The case of $m = 0$ connects to another old friend: the $\mathbb{Z}$-set $R : S^1 \to \mathrm{Set}$ defined by $R(\bullet) :\equiv Z$ and $R(\circlearrowleft) := \bar{s}$, see Definition 3.3.12. Again we point by $0 : R(\bullet)$ and transitivity of R is obvious. The only symmetry that keeps 0 in place is $\mathrm{refl}_\bullet$, since $R(\circlearrowleft^k)(0) = s^k(0) = k = 0$ if and only if $k = 0$. Again, no surprise in view of the results in Section 3.4 identifying R as the universal set bundle over S^1. $\lrcorner$

The following result is analogous to the fact that $\mathrm{Sub}(T)$ is a set for any type T, see Definition 2.20.3. It captures that the essence of picking out symmetries (or picking out elements of a type), is a predicate, like $R_m(p)(0) = 0$ in Example 5.3.5.

lem:SubGisset

LEMMA 5.3.6. *For any group G, the type $\mathrm{Sub}(G)$ of subgroups of G is a set.*

Proof. Let G be a group, and let (X, pt) and (X', pt') be subgroups of G. Since X is transitive we can use the remark following Lemma 5.2.23 that $\mathrm{ev}_{\mathrm{pt}} : (X \xrightarrow{=} X') \to X'(\mathrm{sh}_G)$ is injective. In particular, the preimage of pt' is thus a proposition. Now, we have a chain of equivalences:

$$\begin{aligned}
\left((X, \mathrm{pt}) \xrightarrow{=} (X', \mathrm{pt}')\right) &\simeq \sum_{p : X \xrightarrow{=} X'} \mathrm{pt}' = \mathrm{trp}_p\, \mathrm{pt} \\
&\simeq \sum_{p : X \xrightarrow{=} X'} \mathrm{pt}' = \mathrm{ev}_{\mathrm{pt}}(p) \\
&\equiv \mathrm{ev}_{\mathrm{pt}}{}^{-1}(\mathrm{pt}'). \qquad \square
\end{aligned}$$

exa:fix1subSGn

EXAMPLE 5.3.7. Consider the symmetric group Σ_n from Example 4.2.20(2), for some $n > 0$. The Σ_n-set $X : \mathrm{B}\Sigma_n \to \mathrm{Set}$ given by $X(A, !) :\equiv A$ for any $A : \mathrm{Set}$ together with $! : \|\mathbb{m} \xrightarrow{=} A\|$ is obviously transitive (through univalence). For any $k : \mathbb{m}$, we can point X by $k : X(\mathrm{sh}_{\Sigma_n}) \equiv \mathbb{m}$.[20] Thus we have $(X, k) : \mathrm{Sub}(\Sigma_n)$. The symmetries that are picked out are those $\pi : \mathbb{m} \xrightarrow{=} \mathbb{m}$ that satisfy $(\pi \cdot_X k) = k$. In other words, π keeps k in place and can be any permutation of the other elements of $\mathbb{m}$. From the next Exercise 5.3.8 we get that the underlying group of each (X, k) is isomorphic to Σ_{n-1}. ⌟

[20] The choice of the point does matter for the symmetries that are picked out.

xca:n-is-ptd-n+1

EXERCISE 5.3.8. Give an equivalence from the type of n-element sets to the type of pointed $(n+1)$-element sets. Hint: use Exercise 2.24.6. ⌟

xca:A-is-A-1+1

EXERCISE 5.3.9. For any set A with decidable equality, give an equivalence from A to $\sum_{B:\mathcal{U}}(A \xrightarrow{=} (B + \mathbb{1}))$. ⌟

exa:C3subC6

EXAMPLE 5.3.10. Recall from Example 4.2.22 the definition $\mathrm{C}_6 :\equiv \mathrm{Aut}_{\mathrm{Cyc}}(\mathbb{6}, \mathrm{s})$ of the cyclic group of order 6. This group can be visualized as the rotational symmetries of a regular hexagon, i.e., the rotations by $2\pi \cdot m/6$, where $m = 0, 1, 2, 3, 4, 5$. The symmetries of the regular triangle (rotations by $2\pi \cdot m/3$, where $m = 0, 1, 2$) can also be viewed as symmetries of the hexagon, see Figure 5.3. Thus there is a subgroup of C_6 which, as a group, is isomorphic to C_3, and which we now construct.

In order to obtain C_3 as (the underlying group of) a subgroup of C_6 we can define $F : \mathrm{C}_6 \to \mathrm{Set}$ by $F(X, t) :\equiv X/2$ for all $(X, t) : \mathrm{BC}_6$, where $X/2$ (defined in Section 3.8) is the quotient of X modulo identifying elements that are an even power of t away from each other. Clearly, F is a transitive G-set. On symmetries, F maps $\pi : (\mathbb{6}, \mathrm{s}) \xrightarrow{=} (\mathbb{6}, \mathrm{s})$ to $([k] \mapsto [\pi(k)]) : \mathbb{6}/2 \xrightarrow{=} \mathbb{6}/2$.[21] Thus defines a subgroup $(F, [0]) : \mathrm{Sub}(\mathrm{C}_6)$. The symmetries it picks out are those π satisfying $F(\pi)([0]) = [0]$ and are the even powers of s.[22] The underlying group of $(F, [0])$ is

$$\underline{\Omega}\left(\sum_{(X,t):\mathrm{Cyc}_6} X/2, ((\mathbb{6}, \mathrm{s}), [0])\right).$$

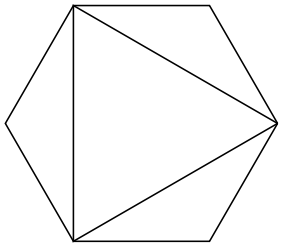


FIGURE 5.3: Geometrical shapes illustrating C_3 as subgroup of C_6.

fig:C3inC6

[21] The function $[k] \mapsto [\pi(k)]$ is well-defined since permutations that commute with s preserve distances.

[22] In view of Corollary 3.6.16, these symmetries can be visualized by the vertices of the regular triangle, see Figure 5.3. The same is true for the symmetries picked out by $F(\pi)([1]) = [1]$. Both $F(\pi)([0]) = [1]$ and $F(\pi)([1]) = [0]$ give the other inscribed regular triangle.

Using $\rho_2 : \mathrm{BC}_3 \to_* \mathrm{BC}_6$ from Lemma 3.8.6, the equivalence between $X/2$ and $\rho_2^{-1}(X, t)$ from Construction 3.8.11, and the equivalence from Lemma 2.25.2, we get an equivalence between the underlying group of $(F, [0])$ and C_3. ⌟

def:decidable-subgroup

DEFINITION 5.3.11. A subgroup (X, pt) of a group G is called *decidable* when $X(\mathrm{sh}_G)$ has decidable equality. ⌟

The subgroup of C_6 explained above is a decidable subgroup. There are other decidable subgroups of C_6, and they are accounted for simply by the various factorizations of the number 6.

5.3.12 *Subgroups as monomorphisms*

sec:subgroups-mono

For many purposes it is useful to have an alternative point-of-view on subgroups. We thus give here a second, equivalent definition of a subgroup. Going back to the motivating examples δ_m and cst**.** from the introduction of this chapter, recall that both $\mathrm{U}\delta_m$ and $\mathrm{U}\,\mathrm{cst}$**.** are injective: they "pick out" (in a nice controlled way) some symmetries of $\bullet$ in S^1. Also recall from Corollary 2.17.9(2) that Uf is injective iff Bf is a set bundle, for any homomorphism f.

def:typeofsmono

DEFINITION 5.3.13. Let G and H be groups. We say that a homomorphism $i : \mathrm{Hom}(H, G)$ is a *monomorphism*, denoted $\mathrm{isMono}(i)$, if $Ui : UH \to UG$ is injective (i.e., all preimages of Ui are propositions).

The *type of monomorphisms into* G[23] is

$$\mathrm{Mono}(G) :\equiv \sum_{H : \mathrm{Group}} \sum_{i : \mathrm{Hom}(H,G)} \mathrm{isMono}(i).$$

We call H the *underlying group* of any monomorphism[24] (H, i) into G. A monomorphism (H, i) into G is:

(1) *trivial* if H is the trivial group;[25]

(2) *proper* if i is not an isomorphism. ⌟

ex:SGn1nSGn+1

EXAMPLE 5.3.14. We will present the subgroups from Example 5.3.7 with monomorphisms. For each $n : \mathbb{N}$, consider the homomorphism $i_n : \Sigma_n \to \Sigma_{n+1}$ of permutation groups with Bi_n sending $A : \mathrm{B}\Sigma_n :\equiv \mathrm{FinSet}_n$ to $A \amalg \mathrm{True} : \mathrm{B}\Sigma_{n+1}$. As pointing path we take the reflexivity path. This is a monomorphism since $Ui_n : \mathrm{U}\,\Sigma_n \to \mathrm{U}\,\Sigma_{n+1}$ is an injection, extending any permutation π of $\mathbb{m}$ to a permutation of $\mathbb{m} + \mathbb{1}$ by adding the last element as a fixed point.

In the picture in the margin we have taken $n = 2$. The elements of $\mathbb{2}$ are named 1 and 2; those of $\mathbb{3}$ are named 1, 2 and 3. The symmetry through the dotted line is the non-identity symmetry in $\mathrm{U}\,\Sigma_3$ picked out by Ui_2.

Two other subgroups can be obtained by varying the pointing path of i_3. These pointing paths are induced by the permutations of $\mathbb{3}$. In Exercise 5.3.15 you are asked to elaborate each case.

There are other subgroups of Σ_3, see Exercise 5.3.16 for example. ⌟

xca:SG2subSG3

EXERCISE 5.3.15. Calculate $\mathrm{im}(Ui_3)$ for each pointing path $\pi : \mathbb{3} \xrightarrow{=} \mathbb{3}$. ⌟

xca:C3subSG3

EXERCISE 5.3.16. Define monomorphisms $j, j' : \mathrm{C}_3 \to \Sigma_3$ such that $Uj \neq Uj'$ while (C_3, j) and (C_3, j') can be identified in $\mathrm{Mono}(\Sigma_3)$. ⌟

ex:prod1nc1smono

EXAMPLE 5.3.17. If G and H are groups, then $i_G : G \to G \times H$, classified by $Bi_G : BG \to_* BG \times BH$ with $Bi_G(z) :\equiv (z, \mathrm{sh}_H)$, pointed by reflexivity, is a monomorphism: Ui_G maps $g : UG$ to $(g, \mathrm{refl}_{\mathrm{sh}_H})$ and is obviously injective.[26] We call i_G the *first inclusion* and we have a similar *second inclusion* $i_H : H \to G \times H$. ⌟

lem:SubG=MonoG

LEMMA 5.3.18. *Let G be a group. The map $F : \mathrm{Sub}(G) \to \mathrm{Mono}(G)$ sending the subgroup $(X, \mathrm{pt}) : \mathrm{Sub}(G)$ to the element*

$$\big(\underline{\Omega}\big(\mathrm{Tot}(X), (\mathrm{sh}_G, \mathrm{pt})\big), \underline{\Omega}(\mathrm{fst})\big) : \mathrm{Mono}(G),$$

where $\mathrm{fst} : \big(\mathrm{Tot}(X), (\mathrm{sh}_G, \mathrm{pt})\big) \to_* BG$ *is pointed by reflexivity, is an equivalence.*

Proof. Given a monomorphism $i : \mathrm{Hom}(H, G)$, write $(Bi_\div, Bi_{\mathrm{pt}})$ for the two components of $Bi : BH \to_* BG$. Then, since i is a monomorphism, $Bi_\div$ is a set bundle whose domain $BH_\div$ is connected, and thus the preimage function $Bi_\div^{-1} : BG \to \mathrm{Set}$ is a transitive G-set. Moreover, the pair $(\mathrm{sh}_H, Bi_{\mathrm{pt}})$ is an element of $Bi_\div^{-1}(\mathrm{sh}_G) :\equiv \sum_{x : BH}(\mathrm{sh}_G \xrightarrow{=} Bi_\div(x))$. Hence, we can define a map E as follows:

$$E : \mathrm{Mono}(G) \to \mathrm{Sub}(G), \qquad (H, i) \mapsto E(H, i) :\equiv (Bi_\div^{-1}, (\mathrm{sh}_H, Bi_{\mathrm{pt}})),$$

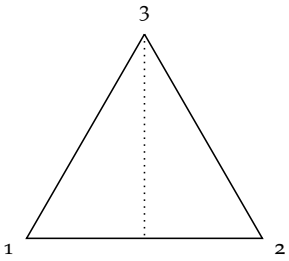
[23] The similarity of this type with the type of subtypes $\mathrm{Inj}(T) :\equiv \sum_{S : \mathcal{U}} \sum_{f : S \to T} \mathrm{isInj}(f)$ in Definition 2.20.9 is not coincidental, and the remarks made there in Footnote 72 apply here as well.

In particular, the identity type of $\mathrm{Mono}(G)$ identifies precisely the triples that define the same subset of UG, namely when their homomorphisms differ by precomposition by an identification of their underlying groups.

[24] As for subgroups, we abuse notation and leave out the propositional component, that is the element $! : \mathrm{isMono}(i)$.

[25] This amounts to Bi being the universal set bundle over BG, see Definition 3.3.10

That $i : \Sigma_2 \to \Sigma_3$ is a monomorphism can visualized as follows: if Σ_3 represent all symmetries of an equilateral triangle in the plane (with vertices 1, 2, 3), then i is represented by the inclusion of the symmetries leaving 3 fixed; i.e., reflection through the line marked with dots in the picture.



[26] Alternatively, the preimage of (z, t) by Bi_G is $\sum_{z' : BG}(z \xrightarrow{=} z') \times (t \xrightarrow{=} \mathrm{sh}_H)$ which contracts away to the set $t \xrightarrow{=} \mathrm{sh}_H$. Hence Bi_G is a set bundle.

We claim that E is an inverse equivalence for F. The details are left to the reader: see Exercise 5.3.20 below. □

exa:EforSG3

Example 5.3.19. In this example we explain how the equivalence between $\mathrm{Mono}(G)$ and $\mathrm{Sub}(G)$ works in the special case $G \equiv \Sigma_3$ and with two versions of the same subgroup.

Recall $(\Sigma_2, i_2) : \mathrm{Mono}(\Sigma_3)$ with $i_2 : \Sigma_2 \to_* \Sigma_3 : B \mapsto (B \amalg \mathrm{True})$ from Example 5.3.14. The preimage function Bi_2^{-1} maps any $A : \mathrm{B}\Sigma_3$ to $\sum_{B : \mathrm{B}\Sigma_2} (A \xrightarrow{=} (B \amalg \mathrm{True}))$. In particular we have $(2, \mathrm{refl}_3) : Bi_2^{-1}(3)$ (recall that i_2 is pointed by reflexivity).

We have $E(\Sigma_2, i_2) \equiv (Bi_2^{-1}, (2, \mathrm{refl}_3))$. Applying F to the result, we get $\left(\underline{\Omega}\left(\sum_{A : \mathrm{B}\Sigma_3} Bi_2^{-1}(A)\right), \underline{\Omega}(\mathrm{fst})\right)$. Using Lemma 2.25.2 one sees that, indeed, the latter monomorphism can be identified with (Σ_2, i_3).

Why do we say that $(X, 3) : \mathrm{Sub}(\Sigma_3)$ from Example 5.3.7 defines the "same subgroup" as $(\Sigma_2, i_2) : \mathrm{Mono}(\Sigma_3)$ from Example 5.3.14? The reason is that they pick out the same symmetries in Σ_3, as argued in these examples. Moreover, $(X, 3)$ and $E(\Sigma_2, i_2)$ can be identified in $\mathrm{Sub}(\Sigma_3)$.[27] Indeed, note that $X(A, !) \equiv A$ and $Bi_2^{-1}(A, !) \equiv \sum_{B : \mathrm{B}\Sigma_2} (A \xrightarrow{=} (B \amalg \mathrm{True}))$: Apply Exercise 5.3.9 and verify that the points correspond. Lemma 5.3.22 below offers a general result of this kind. ⌟

[27] Equivalently, $F(X, 3)$ and (Σ_2, i_2) can be identified in $\mathrm{Mono}(\Sigma_3)$.

xca:SubG=MonoG

Exercise 5.3.20. Complete the details of the proof of Lemma 5.3.18 above using Corollary 2.17.9(2), Lemma 2.25.2, Lemma 5.2.22. ⌟

Since $\mathrm{Sub}(G)$ is a set by Lemma 5.3.6, Lemma 5.3.18 allows us to conclude:

lem:set-of-subgroups

Corollary 5.3.21. *Let G be a group. Then $\mathrm{Mono}(G)$ is a set.*

The following lemma states that the equivalences in Lemma 5.3.18 preserve the subsets of symmetries that are picked out.

lem:F-preserves-symms

Lemma 5.3.22. *Let G be a group and $g : UG$ a symmetry. Recall the equivalence F from Lemma 5.3.18. For all $(X, \mathrm{pt}) : \mathrm{Sub}(G)$ and $(H, i) : \mathrm{Mono}(G)$ such that $(H, i) = F(X, \mathrm{pt})$, we have $g \cdot_X \mathrm{pt} = \mathrm{pt}$ in X_{sh_G} if and only if g is in the image[28] of Ui.*

Proof. Let $(X, \mathrm{pt}) : \mathrm{Sub}(G)$. It suffices to prove the lemma for $(H, i) \equiv F(X, \mathrm{pt}) : \mathrm{Mono}(G)$. This means $BH \equiv (\mathrm{Tot}(X), (\mathrm{sh}_G, \mathrm{pt}))$ and $Bi \equiv \mathrm{fst}$. We have to prove: $g \cdot_X \mathrm{pt} = \mathrm{pt}$ iff there exists an $h : (\mathrm{sh}_G, \mathrm{pt}) \xrightarrow{=} (\mathrm{sh}_G, \mathrm{pt})$ such that $g = Ui(h) \equiv \Omega\,\mathrm{fst}(h)$.

If $g \cdot_X \mathrm{pt} = \mathrm{pt}$, then we can simply take h to the symmetry defined by the pair $(g, \mathrm{refl}_{\mathrm{pt}})$.

For the converse, assume there exists an $h : (\mathrm{sh}_G, \mathrm{pt}) \xrightarrow{=} (\mathrm{sh}_G, \mathrm{pt})$ such that $g = Ui(h)$. Then h is defined by a pair of the form (g, p) for some $p : \mathrm{pt} \xrightarrow[g]{=} \mathrm{pt}$.[29] But $\mathrm{trp}^X_p\,\mathrm{pt}$ is precisely $X(g)(\mathrm{pt})$ (i.e., $g \cdot_X \mathrm{pt}$). □

[28] I.e., there exists $h : UH$ such that $g = Ui(h)$ in UG.

[29] This path p is in fact equal to $\mathrm{refl}_{\mathrm{pt}}$ since X_{sh_G} is a set.

Which of the equivalent sets $\mathrm{Mono}(G)$ and $\mathrm{Sub}(G)$ is allowed to be called "the set of subgroups of G" is, of course, a choice. It could easily have been the other way around and we informally refer to elements in either sets as "subgroups" and use the given equivalence E as needed.

An argument for our choice can be as follows. In set-based mathematics one has two options for defining "subgroup": either as a certain subset (uniquely given by its characteristic function to Prop) or as an equivalence class of injections (taking care of size issues since the class of monomorphisms will not form a small set). The former is the usual choice and is the one we model here with $\mathrm{Sub}(G)$, whereas the other corresponds to $\mathrm{Mono}(G)$. I strongly disagree with this note: $\mathrm{Mono}(G)$ is akin to the collection of injections, not the set of equivalence classes of those; and our $\mathrm{Sub}(G)$ is certainly not akin to the subset definition since it is equivalent to $\mathrm{Mono}(G)$ (and thus have the same "size").

Through the equivalence E we can translate the concepts in Definition 5.3.13 to subgroups in $\mathrm{Sub}(G)$. First, observe that the underlying groups of an element in $\mathrm{Mono}(G)$ and of its image under E in $\mathrm{Sub}(G)$ can be identified.

def:triv-proper-Mono

Definition 5.3.23. We say that a subgroup $(X, \mathrm{pt}) : \mathrm{Sub}(G)$ is:

(1) *trivial* if the underlying group $\underline{\Omega}(\mathrm{Tot}(X), (\mathrm{sh}_G, \mathrm{pt}))$ is trivial, i.e., when $\mathrm{Tot}(X)$ is contractible;

(2) *proper* if $X(\mathrm{sh}_G)$ is not contractible. ⌟

rem:notationsubgroup

REMARK 5.3.24. A note on classical notation is in order. If (X, pt) is a subgroup corresponding to a monomorphism (H, i) into a group G, tradition would permit us to relax the burden of notation and we could write "a subgroup $i : H \subseteq \leq G$", or, if we didn't need the name of $i : \mathrm{Hom}(H, G)$, simply "a subgroup $H \subseteq \leq G$" or "a subgroup H of G". ⌟

ex:prodincl1sGset

EXAMPLE 5.3.25. We saw in Example 5.3.17 that the first inclusion $i_1 : G \to G \times G'$ is a monomorphism. The corresponding $G \times G'$-set is the composite of the second projection $\mathrm{proj}_2 : BG_\div \times BG'_\div \to BG'_\div$ followed by the principal G'-torsor $\mathbb{P}_{\mathrm{sh}_{G'}} : BG' \to \mathrm{Set} : z' \mapsto (\mathrm{sh}_{G'} \xrightarrow{=} z')$ of Example 5.2.4.

Indeed, let us compute the image through F of the latter. First,

$$\begin{aligned} \mathrm{Tot}(\mathbb{P}_{\mathrm{sh}_{G'}} \circ \mathrm{proj}_2) &\equiv \sum_{(z,z') : BG_\div \times BG'_\div} (\mathrm{sh}_{G'} \xrightarrow{=} z') \\ &\xrightarrow{\simeq} BG_\div \times \sum_{z' : BG'_\div} (\mathrm{sh}_{G'} \xrightarrow{=} z') \\ &\xrightarrow{\simeq} BG_\div \end{aligned}$$

Notice that this equivalence ϕ can easily be pointed by $\mathrm{refl}_{\mathrm{sh}_G}$ and that, as pointed maps, we have an identification of type $\mathrm{fst} \xrightarrow{=} i_1 \circ \phi$. In other words, ϕ gives rise to an equality of pointed set bundles over $BG \times BG'$. This, in turn, gives rise to the equality $(BG, i_1) = F(\mathbb{P}_{\mathrm{sh}_{G'}} \circ \mathrm{proj}_2, \mathrm{refl}_{\mathrm{sh}'_G})$ (where F is the equivalence defined in Lemma 5.3.18) in $\mathrm{Mono}(G \times G')$, as promised. ⌟

5.3.26 *The Lagrange construction*

In this section we give a general version of Lagrange's Theorem. It serves as a basis for more traditional versions, such as the counting version in Exercise 5.3.29 below.

con:lagrange

CONSTRUCTION 5.3.27. *Let G be a group. For every subgroup $(X, \mathrm{pt}) : \mathrm{Sub}(G)$ of G, with underlying group called H, we have a function L_H of type*

$$\Big(\prod_{x : X(\mathrm{sh}_G)} \sum_{g : UG} g \cdot_X x = \mathrm{pt} \Big) \to \big(UG \xrightarrow{=} (X(\mathrm{sh}_G) \times UH) \big).$$

Implementation of Construction 5.3.27. Define the map $[_] : UG \to X(\mathrm{sh}_G)$ by $[g] :\equiv g \cdot_X \mathrm{pt}$ for all $g : UG$. Then Lemma 2.25.2 yields an equivalence from UG to the sum of fibers $\sum_{x : X(\mathrm{sh}_G)} [x]^{-1}$. For every $x : X(\mathrm{sh}_G)$, the fiber $[x]^{-1}$ of $[_]$ at x is $\sum_{g : UG} (x = g \cdot_X \mathrm{pt})$, and the latter subset of UG is equivalent to the set $(\mathrm{sh}_G, \mathrm{pt}) \xrightarrow{=} (\mathrm{sh}_G, x)$. So we get an equivalence from UG to $\sum_{x : X(\mathrm{sh}_G)} ((\mathrm{sh}_G, \mathrm{pt}) \xrightarrow{=} (\mathrm{sh}_G, x))$. We are done if we can replace this irritating little last x with pt, since $UH :\equiv ((\mathrm{sh}_G, \mathrm{pt}) \xrightarrow{=} (\mathrm{sh}_G, \mathrm{pt}))$. We use the premiss $\prod_{x : X(\mathrm{sh}_G)} \sum_{g : UG} (g \cdot_X x = \mathrm{pt})$. Applying Exercise 2.9.24 to this premiss, we obtain a function $g : X(\mathrm{sh}_G) \to UG$ such that $g(x) \cdot_X x = \mathrm{pt}$ for all $x : X(\mathrm{sh}_G)$. In other words, $(g(x), !)$ is a path of type $(\mathrm{sh}_G, x) \xrightarrow{=} (\mathrm{sh}_G, \mathrm{pt})$, and hence postconcatenation[30] gives the desired equivalence between $(\mathrm{sh}_G, \mathrm{pt}) \xrightarrow{=} (\mathrm{sh}_G, x)$ and $(\mathrm{sh}_G, \mathrm{pt}) \xrightarrow{=} (\mathrm{sh}_G, \mathrm{pt})$. Thus we get in total an equivalence between UG and $X(\mathrm{sh}_G) \times UH$, and we define $L_H(g)$ to be that equivalence.[31] □

[30] Preconcatenation with the inverse gives an equivalence between $(\mathrm{sh}_G, \mathrm{pt}) \xrightarrow{=} (\mathrm{sh}_G, x)$ and $(\mathrm{sh}_G, x) \xrightarrow{=} (\mathrm{sh}_G, x)$, leading to the equivalence L'_H in Construction 5.3.28.

[31] This construction also works for ∞-groups acting on types. However, the premiss may be harder to fulfill in such general cases.

A minor modification of the above implementation, indicated in Footnote 30 gives Construction 5.3.28, which is sometimes more convenient, e.g., in the proof of Lemma 5.7.2.

cor:lagrange-dep-sum

CONSTRUCTION 5.3.28. *Let conditions be as in Construction 5.3.27. Then, for each* $f : \prod_{x:X(\mathrm{sh}_G)} \sum_{g:UG}(g \cdot_X x = \mathrm{pt})$, *we have an equivalence* $L'_H(f)$ *between* UG *and* $\sum_{x:X(\mathrm{sh}_G)}((\mathrm{sh}_G, x) \xrightarrow{=} (\mathrm{sh}_G, x))$.

xca:lagrange

EXERCISE 5.3.29. The goal of this exercise is to state and prove the traditional formulation of Lagrange's Theorem. Let G be a finite group and $(X, \mathrm{pt}) : \mathrm{Sub}(G)$ a subgroup, whose underlying group we call H. Assume that X is a finite G-set. Show that H is finite and that $\#(G) = \#(X) \times \#(H)$. ⌟

xca:lagrange-Z-action-fm

EXERCISE 5.3.30. The goal of this exercise is to illustrate that Construction 5.3.27 also can be applied to infinite groups. Recall the group of integers $\mathbb{Z} \equiv \Omega(S^1, \bullet)$ and the $\mathbb{Z}$-set $R_m : S^1 \to \mathrm{Set}$ from Definition 3.6.7, defined by $R_m(\bullet) :\equiv m$ and $R_m(\circlearrowleft) := \mathrm{s}$, for $m > 0$. Let H_m be the underlying group of $(R_m, 0, !)$. Identify $U\mathbb{Z}$ with $m \times UH_m$. ⌟

xca:lagrange-if-subgr-not-normal

EXERCISE 5.3.31. Recall the example from Figure 5.1. The group here is $G :\equiv \Omega(S^1 \vee S^1)$, the free group on two generators (see 4.6.2). Consider the G-set X constructed by induction as follows: the base point $\bullet$ is mapped to $\mathbb{3} = \{0, 1, 2\}$; the first loop $\circlearrowleft_1$ is mapped to the cyclic successor $s : \mathbb{3} \to \mathbb{3}$; the second loop $\circlearrowleft_2$ is mapped to the transposition that swaps 0 and 1 and leaves 2 untouched. Our subgroup of interest here is $(X, 0) : \mathrm{Sub}(G)$. Find a "choice map" $f : \prod_{x:\mathbb{3}} \sum_{g:UG}(g \cdot x = 0)$ so that Lagrange's construction apply.[32] ⌟

[32] We shall later see that the underlying group of $(X, 0)$ is a free group as well, see **??**.

5.4 *Invariant maps and orbits*

sec:fixpts-orbits

We now return to some important constructions involving G-sets for a group G. Some of these make equally good sense for *G-types* for an ∞-group G, in which case we add a footnote to this effect.

We are particularly interested in what happens when a G-set is not transitive, that is, does not satisfy the requirement of Definition 5.2.20. In Chapter 3, under the name of set bundles over the circle, we have already seen examples of transitive and non-transitive $\mathbb{Z}$-sets: In Figure 3.2 the left picture exhibits a transitive one, and the right picture a non-transitive one. Also, Figure 3.6 shows a non-transitive $\mathbb{Z}$-set, whereas the mth power bundle over the circle in Figure 3.7 is a transitive $\mathbb{Z}$-set. Lemma 5.2.22 gives a good explanation of these pictures: A G-set is transitive if and only if the associated set bundle is connected. In other words, if a G-set $X : BG \to \mathrm{Set}$ is transitive, then the group action connects[33] any two elements in the total type $\mathrm{Tot}(X) \equiv \sum_{x:BG} X(z)$. If X is not transitive, then the latter total type falls apart in different components. Since these components are themselves connected, the choice of an element of them gives rise to a subgroup of G in the sense of Definition 5.3.2.

[33] In the sense that $\|(z, x) \xrightarrow{=} (w, y)\|$ if and only if there exists a $g : z \xrightarrow{=} w$ such that $g \cdot_X x = y$.

def:actiontype

DEFINITION 5.4.1. (1) Let G be a group and $X : BG \to \mathrm{Set}$ a G-set,[34] then the *action type* of X, denoted[35]

$$X_{hG} :\equiv \sum_{z:BG} X(z),$$

is the total type of X, see Section 2.8.

[34] This definition can be generalized to ∞-groups G and G-types X.

[35] The superscripts and subscripts are decorated with "hG", following a convention in homotopy theory. The action type is sometimes denoted $X /\!\!/ G$.

(2) The type of *invariant maps*[36] is

$$X^{hG} :\equiv \prod_{z:BG} X(z).$$

(3) The *set of orbits* is the subset of $\mathrm{Sub}_G(X)$ consisting of all G-subsets P of X such that the underlying G-subset X_P is transitive:[37]

$$X/G :\equiv \sum_{P\,:\,\mathrm{Sub}_G(X)} \mathrm{isTrans}(X_P). \qquad \lrcorner$$

[36] Invariant maps are dependent functions f and the reason for the new name in this context is that $f(z) = g \cdot_X f(z)$ for any $z : BG$ and $g : z \xrightarrow{=} z$. Cf. Lemma 5.4.20. Note that there need not be any invariant maps: the type of invariant maps of $\mathbb{P}_{\mathrm{sh}_{\mathbb{Z}}}$, i.e., $\prod_{z:S^1} \bullet \xrightarrow{=} z$, is empty. Using Theorem 3.4.5, Figure 3.3 explains why: the successor function has no fixed point.

[37] See Definition 5.2.20.

Remark 5.4.2. Note that, by Definition 2.7.3 and Definition 2.10.1, we get an equivalence that allows for a more practical form of identifications in the action type:

$$\big((z,x) \xrightarrow{=}_{X_{hG}} (w,y)\big) \xrightarrow{\simeq} \sum_{g\,:\,z\xrightarrow{=}w} g \cdot x = y,$$

which also goes for their (often used) propositional truncations. $\lrcorner$

These various definitions are closely related. For example, we will soon identify the set of orbits with the set truncation of the action type (see Lemma 5.4.5).

We have seen many instances of action types before: When G-sets are considered as set bundles $f : A \to BG$, they are the domains A. Recall for example Figure 3.6, showing an action of $\mathbb{Z}$ on $\{1,2,3,4,5\}$ with no invariant maps and an action type equivalent to a sum of two circles. In Figure 5.4, we show a similar $\mathbb{Z}$-set, with underlying set $\{0,1,2,3,4,5\}$, three orbits, and 5 corresponding to the only invariant map.[38]

In Figure 5.4 we have highlighted one single component of the action type in blue (i.e., corresponding to an element of the set of orbits), and we see that it contains a subset of the underlying set, the three red elements $\{0,1,2\}$. Such a set is what is traditionally called an orbit. This connection is emphasized in Corollary 5.4.6.

[38] Sending $\bullet$ to 5 and $\circlearrowleft$ to refl_5.

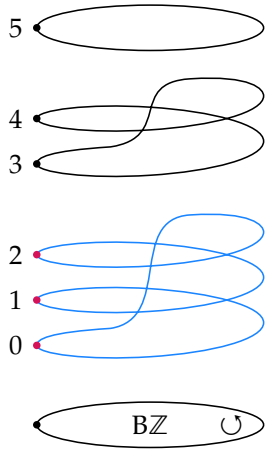


Figure 5.4: A $\mathbb{Z}$-set with three orbits and one invariant map.

Definition 5.4.3. Let G be a group and $X : BG \to \mathrm{Set}$ a G-set. We define the map $[_]_0$ from the action type X_{hG} of X to $\mathrm{Sub}_G(X)$, the set of G-subsets of X, as follows. For any $u : X_{hG}$, let $[u]_0$ be the G-subset of X that sends $z : BG$ to

$$(x \mapsto \|u \xrightarrow{=} (z,x)\|) : X(z) \to \mathrm{Prop}. \qquad \lrcorner$$

Recall from Remark 5.4.2 the equivalence of $\|(z,x) \xrightarrow{=} (w,y)\|$ and $\exists_{g\,:\,z\xrightarrow{=}w}(g \cdot x = y)$. The next lemma follows easily from the properties of $\|u \xrightarrow{=} (z,x)\|$.

Lemma 5.4.4. *Let G be a group and $X : BG \to \mathrm{Set}$ a G-set. For every $u : X_{hG}$, the underlying G-set*[39] *of $[u]_0$, mapping z to $\sum_{x\,:\,X(z)}\|u \xrightarrow{=} (z,x)\|$, is transitive. Hence $[_]_0$ is a map from X_{hG} to X/G.*

In view of the above lemma, we call $[u]_0$ *the orbit through* u. The following lemma implies that the set of orbits can be identified with the set truncation of the action type.

[39] See Definition 5.2.11.

Lemma 5.4.5. *Let G be a group and $X : BG \to \mathrm{Set}$ a G-set.*[40] *Then the map $[_]_0 : X_{hG} \to X/G$ is surjective. Moreover, we have a (unique) identification of $(X/G, [_]_0)$ and $(\|X_{hG}\|_0, |_|_0)$ in the type $\sum_{S\,:\,\mathrm{Set}}(X_{hG} \to S)$.*

[40] This lemma can be generalized to ∞-groups G and G-types X.

rem:path-in-action-type
def:orbit-map
lem:[]0-maps-to-X/G
lem:X/G=setTrunc_hG
fig:ZZ-set-orbits

Proof. Consider an orbit $O : X/G$, i.e., O is a G-subtype of X such that X_O is transitive. We have to show that there exists a $u : X_{hG}$ such that $O = [u]_0$. By the connectivity of BG it suffices to show $O(\mathrm{sh}_G) =_{X(\mathrm{sh}_G)\to\mathrm{Prop}} [u]_0(\mathrm{sh}_G)$ for some u. Transitivity of X_O means that there exists an $x : X(\mathrm{sh}_G)$ that belongs to the subset $O(\mathrm{sh}_G)$ and for all $y : X(\mathrm{sh}_G)$ in that subset $O(\mathrm{sh}_G)$ there exists a $g : UG$ such that $g \cdot x = y$, i.e., $[(\mathrm{sh}_G, x)]_0(\mathrm{sh}_G)(y)$. Since we are proving a proposition (namely, that the preimage of $[_]_0$ at O is non-empty), we can assume an actual $x : X(\mathrm{sh}_G)$ with the property above, and then we take $u :\equiv (\mathrm{sh}_G, x)$ and have to show $O(\mathrm{sh}_G)$ is logically equivalent to $[u]_0(\mathrm{sh}_G)$. But this follows directly from the observation made just above the lemma (see also Remark 5.4.9 below).

The second part of the lemma follows from Remark 2.22.18. □

Another way to state the above lemma is that the map $[_]_0 : X_{hG} \to X/G$ factors as the composite of $|_|_0$ followed by a unique equivalence: $X_{hG} \to \|X_{hG}\|_0 \xrightarrow{\simeq} X/G$.

cor:orbit-equiv

Corollary 5.4.6. *Define the map* $[_] : X(\mathrm{sh}_G) \to X/G$ *by* $[x] :\equiv [(\mathrm{sh}_G, x)]_0$. *Then* $[x] = [y]$ *is equivalent to* $\exists_{g:UG}(g \cdot x = y)$. *Moreover,* $[_]$ *is surjective and factors (uniquely) by an equivalence through the quotient set of* $X(\mathrm{sh}_G)$ *modulo the equivalence relation* $\exists_{g:UG}(g \cdot x = y)$, *cf. Exercise 2.22.19.*

In view of this corollary, we call $[x]$ *the orbit through* x.

Proof. In the proof of surjectivity in Lemma 5.4.5 we used $u :\equiv (\mathrm{sh}_G, x)$ to get $O = [u]_0$, so $[_]$ is surjective. The last statement follows since both propositions are equivalent to $\|(\mathrm{sh}_G, x) \xrightarrow{=} (\mathrm{sh}_G, y)\|$. □

rem:Sub$_G$=Sub(X/G)

Remark 5.4.7. Let G be a group and $X : BG \to \mathrm{Set}$ a G-set. We have the following chain of definitions and equivalences:

$$\begin{aligned}
\mathrm{Sub}_G(X) &\equiv \prod_{z:BG} (X(z) \to \mathrm{Prop}) \\
&\xrightarrow{\simeq} \big(X_{hG} \to \mathrm{Prop}\big) \quad \text{(by Footnote 9 and Exercise 2.9.26)} \\
&\xrightarrow{\simeq} \big(\|X_{hG}\|_0 \to \mathrm{Prop}\big) \quad \text{(since Prop is a set)} \\
&\xrightarrow{\simeq} \big(X/G \to \mathrm{Prop}\big) \quad \text{(by Lemma 5.4.5)} \\
&\equiv \mathrm{Sub}(X/G).
\end{aligned}$$

⌟

Exercise 5.4.8. Show: X/G is contractible if and only if X is transitive. ⌟

xca:transX-justorbit
rem:equivalents-of-[x]-[y]

Remark 5.4.9. Given a group G, a G-set X and $x, y : X(\mathrm{sh}_G)$, the following propositions are all equivalent and we may pass from one to another without mention:

- $[x] =_{X/G} [y]$;
- $[x](\mathrm{sh}_G) =_{X(\mathrm{sh}_G)\to\mathrm{Prop}} [y](\mathrm{sh}_G)$;
- $\exists_{g:UG}(g \cdot x = y)$;
- $\|(\mathrm{sh}_G, x) \xrightarrow{=} (\mathrm{sh}_G, y)\|$;
- $[x](\mathrm{sh}_G, y)$;
- $[y](\mathrm{sh}_G, x)$.

As functions of x and y, all of the above define the equivalence relation on $X(\mathrm{sh}_G)$ induced by the surjection $[_]$. ⌟

Thus, both the underlying set $X(\mathrm{sh}_G)$ and the action type X_{hG} have equivalence relations (induced by the surjections $[_]$ and $[_]_0$, respectively) with quotient set X/G.[41] We can write $X(\mathrm{sh}_G)$ and X_{hG} as sums of the respective fibers, which we will elaborate in the next paragraphs.

Let $O : X/G$ be an orbit and consider $[O]_0^{-1} \equiv \sum_{u : X_{hG}} (O = [u]_0)$. Note that the underlying G-set of O, i.e., $X_O \equiv (z : BG \mapsto \sum_{y : X(z)} O(z)(y))$, is transitive. It follows that $O(u)$ holds[42] if and only if $O = [u]_0$, for all $u : X_{hG}$.[43] Therefore, the fiber $[O]_0^{-1}$ is equivalent to the action type $(X_O)_{hG} \equiv \sum_{z : BG} X_O(z)$.

After the previous paragraph, the elaboration of $[O]^{-1} \equiv \sum_{x : X(\mathrm{sh}_G)} (O = [x])$ is easy. Recall that $[x] \equiv [(\mathrm{sh}_G, x)]_0$, so that the fiber $[O]^{-1}$ is equivalent to the underlying set of X_O, i.e., $X_O(\mathrm{sh}_G) \equiv \sum_{x : X(\mathrm{sh}_G)} O(\mathrm{sh}_G)(x)$ via identity on first components. We depict the situation in the diagramin the margin. Note how the role of X in Footnote 41 is taken over by X_O.

def:orbit-stabilizer

Definition 5.4.10. Let G be a group, $X : BG \to \mathrm{Set}$ a G-set, and $x : X(\mathrm{sh}_G)$ an element.[44]

(1) Define the group $G_x :\equiv \mathrm{Aut}_{X_{hG}}(\mathrm{sh}_G, x)$. Clearly, $\mathrm{fst} : BG_x \to BG$ is a set bundle: each fiber at $z : BG$ is a subset of $X(z)$. It is pointed trivially. Hence $(G_x, \underline{\Omega}(\mathrm{fst})) : \mathrm{Mono}(G)$ is a monomorphism into G. We call the subgroup G_x of G the *stabilizer* (*sub*)*group* at x. We shall write $i_x : \mathrm{Hom}(G_x, G)$ for $\underline{\Omega}(\mathrm{fst})$.

Notice that for $(X, \mathrm{pt}) : \mathrm{Sub}(G)$, the group G_{pt} is the underlying group of the subgroup (X, pt), and $(G_{\mathrm{pt}}, i_{\mathrm{pt}})$ is the corresponding monomorphism into G.

(2) Define $G \cdot x :\equiv \{\, y : X(\mathrm{sh}_G) \mid [x] =_{X/G} [y] \,\}$ to be the *underlying set of the orbit through* x.[45] ⌟

rem:orbit_fibs

Remark 5.4.11. In the above definition, the underlying G-set of the orbit $[x]$, that is

$$X_{[x]} \equiv (z : BG) \mapsto \sum_{y : X(\mathrm{sh}_G)} \|(\mathrm{sh}_G, x) \xrightarrow{=} (z, y)\|$$

plays an important double role: On one hand its action type $(X_{[x]})_{hG}$, pointed at (sh_G, x), is the classifying type BG_x of the stabilizer group G_x. On the other hand it is a transitive G-set whose underlying set $\sum_{y : X(\mathrm{sh}_G)} \|(\mathrm{sh}_G, x) \xrightarrow{=} (\mathrm{sh}_G, y)\|$ is precisely $G \cdot x$. In particular, $G \cdot x$ and $(BG_x)_\div$ depends only on the class $O \equiv [x]$, since they can be identified with $X_O(\mathrm{sh}_G)$ and $(X_O)_{hG}$, respectively. The situation is depicted in Figure 5.5.

Note furthermore that the base point of BG_x depends on the choice of x. ⌟

Exercise 5.4.12. Let G be a group and $X : BG \to \mathrm{Set}$ a G-set. Show: if $[x] = [y]$, then $\|G_x \xrightarrow{=} G_y\|$, for any $x, y : X(\mathrm{sh}_G)$. ⌟

xca:[x]=[y]-implies-||Gx=Gy||

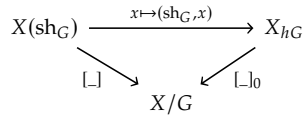
rem:subgrp-is-stabsubgr

Remark 5.4.13. The element of $\mathrm{Sub}(G)$ associated with the stabilizer subgroup $(G_x, i_x) : \mathrm{Mono}(G)$ of x is the transitive G-set $X_{[x]}$, pointed by x as element of the subset $X_{[x]}(\mathrm{sh}_G)$ of $X(\mathrm{sh}_G)$. If X is transitive to begin with, then all orbits are equal (Exercise 5.4.8) and the stabilizer subgroup of x simplifies to $(X, x) : \mathrm{Sub}(G)$. ⌟

The following lemma states that the orbits of a G-set X sum up to its underlying set, with the sum taken over the set of orbits X/G.

fig:fibs-of-[x]

[41] This also justifies the notation X/G. We have a diagram of surjective maps:



[42] Notice the abuse here where $O(u)$ stands for $O(z)(x)$ where $u \equiv (z, x)$.

[43] We use the first step of Remark 5.4.7. If $O(u)$ and $O(v)$, then $\|u \xrightarrow{=} v\|$ by the transitivity of X_O. The rest is obvious.

[44] This definition can be generalized to ∞-groups G and G-types X.

[45] This is short for the underlying set of the underlying G-set of the orbit $[x]$ of X.

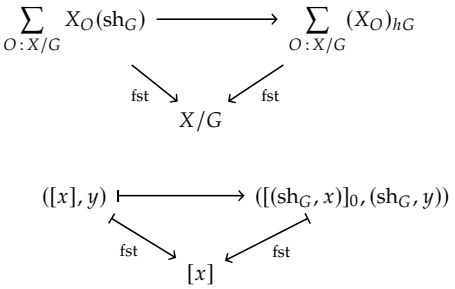


Figure 5.5: Along the horizontal arrow, the second component $y : G \cdot x$ is mapped to $(\mathrm{sh}_G, y) : BG_x$.

lem:splitting into orbits

LEMMA 5.4.14. *The inclusions of the orbits form an equivalence*

$$(O, x) \mapsto x \quad : \quad \Big(\sum_{O : X/G} [O]^{-1}\Big) \xrightarrow{\simeq} X(\mathrm{sh}_G).$$

Proof. Recall that $[O]^{-1} \equiv \sum_{x : X(\mathrm{sh}_G)}(O = [x])$, and then abstract away the O using Lemma 2.9.10.[46] □

[46] In fact, for every set A and every equivalence relation on A, the equivalence classes sum up to A.

There are two possible extreme cases for G_x that are important:

def:fixed-free

DEFINITION 5.4.15. Let G be a group, X a G-set and $x : X(\mathrm{sh}_G)$ an element of the underlying set.[47] We say that

(1) x is *fixed* if i_x is an isomorphism (so "G_x is all of G"), and

(2) x is *free* if G_x is trivial.

We say that X itself is *free* if each $x : X(\mathrm{sh}_G)$ is free. ⌟

[47] This definition can be generalized to ∞-groups G and G-types X.

exa:fixed-free-neither

EXAMPLE 5.4.16. Let G be a group. For every set S, every element $s : S$ is fixed under the trivial G-set $\mathrm{triv}_G S$, since the group action is the identity function. In contrast, every element $g : UG$ is free under the G-set $\mathbb{P}_{\mathrm{sh}_G} \equiv (\mathrm{sh}_G \xrightarrow{=} _)$, as $\sum_{z : BG} \mathbb{P}_{\mathrm{sh}_G}(z)$ is contractible. For an example with more variation, see Example 5.7.1 upto Table 5.1. Find the fixed elements, the free elements and those that are neither fixed nor free. ⌟

xca:fixed-free-neither

EXERCISE 5.4.17. Make sure you understand Example 5.4.16 by elaborating:

- BG_s (for $s : S$) in the case of $\mathrm{triv}_G S$,
- BG_g (for $g : G$) in the case of $\mathbb{P}_{\mathrm{sh}_G}$,
- BG_f for each $f : 4 \to 2$ in the case of Example 5.7.1, see Table 5.1. ⌟

lem:free-pt-char

LEMMA 5.4.18. *Let G be a group and X a G-set. Then we have for all $x : X(\mathrm{sh}_G)$ that x is free if and only if the (surjective) map $(_ \cdot x) : UG \to (G \cdot x)$ is injective (and hence a bijection).*

Proof. Consider two elements of the orbit, say $g \cdot x, g' \cdot x$ for $g, g' : UG$. We have $g \cdot x = g' \cdot x$ if and only if $x = g^{-1}g' \cdot x$ if and only if $g^{-1}g'$ lies in UG_x. Hence the map $(_ \cdot x)$ is injective iff UG_x is contractible. Now use Exercise 2.16.10 yielding that that G_x is trivial iff UG_x is contractible. □

lem:X_hG-set-iff-Xfree

LEMMA 5.4.19. *Let G be a group and $X : BG \to \mathrm{Set}$ a G-set.*[48] *Then the following propositions are equivalent*:

[48] This lemma can be generalized to ∞-groups G and G-types X.

(1) *The action type X_{hG} is a set*;

(2) *The map $[_]_0 : X_{hG} \to X/G$ from Definition 5.4.3 is an equivalence*;

(3) *The G-set X is free.*

Proof. We prove the relevant implications in circular order.

(1) Assume X_{hG} is a set. The map $[_]_0 : X_{hG} \to X/G$ is surjective by Lemma 5.4.5, so it suffices to show it is injective. Since X_{hG} is a set, it suffices to show that $[u]_0 = [v]_0$ implies $u = v$, for all $u, v : X_{hG}$. This follows immediately from the definition of $[_]_0$, as the propositional truncation plays no role.

(2) Assume $[_]_0 : X_{hG} \to X/G$ is an equivalence. Then X_{hG} is a set since X/G is a set, and hence all components of X_{hG} are contractible. It follows that all stabilizer groups G_x are trivial, and hence X is free.

(3) Assume X is free. Then all stabilizer groups G_x are trivial, so all identity types $(\mathrm{sh}_G, x) \xrightarrow{=} (\mathrm{sh}_G, x)$ are contractible. Since BG is connected we get that $u \xrightarrow{=} u$ is contractible for all $u : X_{hG}$. Hence X_{hG} is a set.[49] □

[49] A type T is a set if and only if all identity types $t \xrightarrow{=}_T t$ are contractible.

lem:fixed-char

Lemma 5.4.20. *Given a group G and a G-set X, an element $x : X(\mathrm{sh}_G)$ is fixed if and only if the orbit $G \cdot x$ is contractible, i.e., $x = g \cdot x$ for all $g : UG$.*[50]

[50] This lemma can be generalized to ∞-groups G and G-types X. In that case $UG :\equiv \Omega BG$ is the underlying *type* of G.

Proof. The orbit $G \cdot x$ of x is the fiber of $Bi_x : BG_x \to_* BG$ at sh_G. Since BG is connected, this is contractible if and only if all fibers of Bi_x are contractible, i.e., Bi_x is an equivalence, which in turn is equivalent to i_x being an isomorphism. □

When $X : BG \to \mathrm{Set}$ is a G-set for an ordinary group G, the subset

$$\{\, x : X(\mathrm{sh}_G) \mid x \text{ is fixed} \,\}$$

is closely related to the type X^{hG} of invariant maps. If we evaluate an invariant map $f : \prod_{z:BG} X(z)$ at sh_G we do indeed land in this subset: Letting $x :\equiv f(\mathrm{sh}_G)$, and taking the dependent action on paths, $\mathrm{apd}_f(g) : x \xrightarrow[g]{=} x$, we can use Definition 2.7.3 to conclude $\mathrm{trp}^X_g(x) \equiv g \cdot x = x$, for all $g : UG$. The following lemma states that, conversely, each fixed x uniquely determines an invariant map.

lem:fixpts-are-fixed
it:ev-is-inj
it:ev-is-eq-on-inv

Lemma 5.4.21. *Let G be a group and X a G-set.Then the evaluation map $\mathrm{ev} : X^{hG} \to X(\mathrm{sh}_G)$ at sh_G, defined by $\mathrm{ev}(f) :\equiv f(\mathrm{sh}_G)$,*

(1) *is injective, and*

(2) *has image $\{\, x : X(\mathrm{sh}_G) \mid x \text{ is fixed} \,\}$.*

Proof. Let $f, g : X^{hG}$ be invariant maps such that $f(\mathrm{sh}_G) = g(\mathrm{sh}_G)$. Since BG is connected, this inplies $f(z) = g(z)$ for all $z : BG$. By extensionality for dependent functions, we get $f = g$. This thus proves (1).

For (2), assume that $x : X(\mathrm{sh}_G)$ is fixed, so that $Bi_x \equiv \mathrm{fst} : BG_x \to BG$ is an equivalence. This means that $\mathrm{fst}^{-1}(z)$ is contractible for all $z : BG$. Spelling out $\mathrm{fst}^{-1}(z)$, and using Lemma 2.9.10, we get an identification of $\mathrm{fst}^{-1}(z)$ with $\sum_{y:X(z)} \|(\mathrm{sh}_G, x) \xrightarrow{=} (z, y)\|$. Projecting on the first component of each center of contraction gives a dependent map $f : X^{hG}$ such that $\|(\mathrm{sh}_G, x) \xrightarrow{=} (\mathrm{sh}_G, f(\mathrm{sh}_G))\|$, from which the proposition $x = f(\mathrm{sh}_G)$ follows since x is fixed. □

xca:Gset-A->B

Exercise 5.4.22. Let G be the group $\Sigma_2 \times \Sigma_2$ and X the G-set mapping any pair (A, B) of 2-element sets to the set $A \to B$. Elaborate the action of G on $X(\mathrm{sh}_G)$ and determine the set of orbits and the set of invariant maps. You can do the same exercise for the following easier cases first: the G-set that is constant 2×2, and the Σ_2-sets $X(_, 2)$ and $X(2, _)$. ⌟

xca:HomGsets-ev

Exercise 5.4.23. Let G be a group and let X and Y be G-sets. Show that evaluation at sh_G defines an equivalence beween $\mathrm{Hom}_G(X, Y)$ and the subset of $X(\mathrm{sh}_G) \to Y(\mathrm{sh}_G)$ consisting of the fixed elements for the G-set $z \mapsto (X(z) \to Y(z))$. ⌟

sec:orbit-stabilizer-theorem

5.4.24 *The Orbit–Stabilizer theorem*

Consider a group G, a G-set X and an element $x : X(\mathrm{sh}_G)$, and recall Definition 5.4.10. The classifying type of the stabilizer group G_x is the component of $X_{hG} \equiv \sum_{z:BG} X(z)$ pointed by the shape (sh_G, x). The first projection of a symmetry of (sh_G, x) is a symmetry of sh_G, and the second projection is a proof of a proposition. This suggest the following simple way for G_x to act on the symmetries of sh_G, by just ignoring the second projection:

def:Gx-action-on-G

Definition 5.4.25. Let G be a group, X a G-set and $x : X(\mathrm{sh}_G)$ an element of the underlying set. Recall that the stabilizer group G_x comes with a monomorphism $i_x : \mathrm{Hom}(G_x, G)$. Define the G_x-set $\tilde{G}_x : BG_x \to \mathcal{U}$ as the restriction of $\mathbb{P}_{\mathrm{sh}_G}$ along i_x, i.e., $\tilde{G}_x :\equiv \mathbb{P}_{\mathrm{sh}_G} \circ Bi_x$.[51] ⌟

[51] Spelled out: for all $(z, y) : X_{hG}$ in the same component as (sh_G, x), $\tilde{G}_x(z, y) \equiv (\mathrm{sh}_G \xrightarrow{=} z)$. In Definition 5.5.9 we will give a general definition of restrictions of G-sets along homomorphisms in $\mathrm{Hom}(G, H)$.

The underlying set of $\tilde{G}_x$ is UG. The group action of $\tilde{G}_x$ is explored in the following exercise.

xca:Gx-action-on-G

Exercise 5.4.26. Let $s : (\mathrm{sh}_G, x, !) \xrightarrow{=} (z, y, !)$ be a path in BG_x with first component s_1, and let $g : UG$. Show that $s \cdot_{\tilde{G}_x} g = s_1 g$, i.e., the group action of $\tilde{G}_x$ is path composition. ⌟

The following exercise prepares for the subsequent Orbit–Stabilizer Theorem.

xca:fixed-free-neither-action-types

Exercise 5.4.27. Elaborate the action type of $\tilde{G}_x$ from Definition 5.4.25 in each of the cases of Exercise 5.4.17, that is, elaborate

- $(\tilde{G}_s)_{hG_s}$ in the case of $\mathrm{triv}_G S$,
- $(\tilde{G}_g)_{hG_g}$ in the case of $\mathbb{P}_{\mathrm{sh}_G}$,
- $(\tilde{G}_f)_{hG_f}$ for each $f : 4 \to 2$, in the case of Example 5.7.1, Table 5.1.

Compare your findings with $G \cdot s$, $G \cdot g$, and each $G \cdot f$, respectively. ⌟

The action type of $\tilde{G}_x$ can be identified with the underlying set of the orbit through x under X. This is achieved by a chain of easy equivalences, spelled out in the following construction.

con:orbit-stabilizer

Construction 5.4.28 (Orbit–Stabilizer theorem). *Let G be a group, X a G-set X, $x : X(\mathrm{sh}_G)$ an element of the underlying set of X.*[52] *Recall the G_x-set $\tilde{G}_x$ from Definition 5.4.25. Then we have an equivalence from the action type $(\tilde{G}_x)_{hG_x}$ to the underlying set $(G \cdot_X x)$ of the orbit through x.*

[52] This construction can be generalized to ∞-groups G and G-types X.

Implementation of Construction 5.4.28. The desired equivalence is the composition of elementary equivalences for sums and products, followed by contracting away the variable z:

$$\begin{aligned}(\tilde{G}_x)_{hG_x} &\equiv \sum_{u:BG_x} \tilde{G}_x(u)\\ &\simeq \sum_{z:BG} \sum_{y:X(z)} \|(\mathrm{sh}_G, x) \xrightarrow{=} (z, y)\| \times (\mathrm{sh}_G \xrightarrow{=} z)\\ &\simeq \sum_{y:X(\mathrm{sh}_G)} [x] =_{X/G} [y] \quad \equiv \quad (G \cdot_X x).\end{aligned}$$

□

The above construction has some interesting consequences. One is that $(\tilde{G}_x)_{hG_x}$ is a set[53], so that Lemma 5.4.19 applies:

[53] Note that Exercise 5.4.26 already implies that $(\tilde{G}_x)_{hG_x}$ is a set: given $(\mathrm{sh}_G, x, !, g)$ and $(z, y, !, g')$, there can be at most one $s : \mathrm{sh}_G \xrightarrow{=} z$ such that $s \cdot_{\tilde{G}_x} g = sg = g'$.

cor:action-subgrp-free

Corollary 5.4.29. *The G_x-set $\tilde{G}_x$ is free.*

ft:action-type-tildeGx-set

We further obtain that the underlying set of the orbit of $\tilde{G}_x$ through g can be identified with the underlying set of G_x.

lem:cosets-Gx.g

COROLLARY 5.4.30. *For any $g : UG$, the map $(_ \cdot_{\tilde{G}_x} g)$ is an equivalence from UG_x to $(G_x \cdot_{\tilde{G}_x} g)$.*

Proof. This follows directly from Lemma 5.4.18, applied to G_x and $\tilde{G}_x$, using that $\tilde{G}_x$ is free. □

In the case of a subgroup of G, we have the following result.[54]

[54] In ordinary group theory, a subgroup H of a group G always acts on G by multiplication: this is the G_{pt}-set $\tilde{G}_{\mathrm{pt}}$ in our presentation. The set of orbits for this action is the set G/H of cosets: in our presentation, this is $\tilde{G}_{\mathrm{pt}}/G_{\mathrm{pt}}$. Each element of G/H is of the form Hg for some $g \in G$ and is in bijection with H: such an element is $G_{\mathrm{pt}} \cdot g$ is our presentation, and the bijectivity is Corollary 5.4.30. The group G acts transitively on the set of cosets G/H: this is the G-set X in our presentation.

con:preLagrange

CONSTRUCTION 5.4.31. *Let G be a group and let $(X, \mathrm{pt}) : \mathrm{Sub}(G)$ be a subgroup of G as defined in Definition 5.3.2. Then we have an equivalence $[_]_{\mathrm{pt}}$ from the underlying set $X(\mathrm{sh}_G)$ of X to $\tilde{G}_{\mathrm{pt}}/G_{\mathrm{pt}}$, the set of orbits of $\tilde{G}_{\mathrm{pt}}$.*

Implementation of Construction 5.4.31. The function $[_]_{\mathrm{pt}}$ is the composition of three equivalences. Since X is transitive, $\mathrm{fst} : (G \cdot_X \mathrm{pt}) \to X(\mathrm{sh}_G)$ is an equivalence. The Orbit–Stabilizer Construction 5.4.28 gives us an equivalence o from $(G \cdot_X \mathrm{pt})$ to $(\tilde{G}_{\mathrm{pt}})_{hG_{\mathrm{pt}}}$. Since the latter type is a set, the function $[_]_0$ from Lemma 5.4.5 is an equivalence from $(\tilde{G}_{\mathrm{pt}})_{hG_{\mathrm{pt}}}$ to $\tilde{G}_{\mathrm{pt}}/G_{\mathrm{pt}}$. Now define $[x]_{\mathrm{pt}} :\equiv [o(\mathrm{fst}^{-1}(x))]_0$ for any $x : X(\mathrm{sh}_G)$. □

The reader may notice that the last two results contain some of the ingredients of the traditional formulation of Lagrange's Theorem: the group G, the subgroup G_{pt}, the orbits (cosets) $(G_{\mathrm{pt}} \cdot_{\tilde{G}_{\mathrm{pt}}} g)$ and the set of orbits $\tilde{G}_{\mathrm{pt}}/G_{\mathrm{pt}}$. It is in fact possible to obtain the counting version of Lagrange's Theorem, Exercise 5.3.29, from the above results:

xca:lagrange2

EXERCISE 5.4.32. Let G be a finite group and $(X, \mathrm{pt}) : \mathrm{Sub}(G)$ a subgroup, whose underlying group we call H. Assume that X is a finite G-set. Show that $\#(G) = \#(X) \times \#(H)$ using Lemma 5.4.14, Corollary 5.4.30 and Construction 5.4.31, instead of Construction 5.3.27. ⌟

5.5 *The classifying type is the type of torsors*

sec:torsors

Recall the definition of the principal G-torsor $\mathbb{P}_{\mathrm{sh}_G} \equiv (\mathrm{sh}_G \xrightarrow{=} _)$ from Example 5.2.4. In this section we elaborate the concept of *torsor* and give one example of its use. In Section 7.4 we'll use abstract torsors to prove that the type of groups and the type of abstract groups are equivalent by classifying abstract groups via their pointed connected groupoid of abstract torsors. In the end we want to prove that the type of torsors of $\mathrm{abs}(G)$ is equivalent to BG. To do so we will need an intermediate equivalent type, namely that of torsors for the concrete group G. We will then prove Theorem 5.5.7, establishing that showing that BG is equivalent to the type of G-torsors. In Section 7.4 we'll then only have to show that abtract and concrete torsors are equivalent.

def:Gtorsor

DEFINITION 5.5.1. Given a group G, the type of *G-torsors*[55] is

$$\mathrm{Torsor}_G :\equiv \sum_{X : G\text{-Set}} \|\mathbb{P}_{\mathrm{sh}_G} \xrightarrow{=} X\|,$$

where $\mathbb{P}_{\mathrm{sh}_G} \equiv (\mathrm{sh}_G \xrightarrow{=} _)$ is the principal G-torsor of Example 5.2.4. ⌟

[55] This works equally well with ∞-groups: G-torsors are in that case G-types in the component of the principal torsor $\mathbb{P}_{\mathrm{sh}_G} : BG \to \mathcal{U}$. There is no conflict with the case when the ∞-group G is actually a group since then any G-type in the component of the principal G-torsor will be a G-set.

xca:torsor-free-transitive

EXERCISE 5.5.2. Show that a G-set is a G-torsor if and only if it is free and transitive. ⌟

REMARK 5.5.3. For G a group, the type of G-torsors is just another name for the component of the type of set bundles over BG containing the universal set bundle.

Observe that for a group G, Torsor_G is a connected groupoid[56] and so – by specifying the base point $\mathbb{P}_{\mathrm{sh}_G}$ – it classifies a group. Guess which one![57]. ⌟

def:BG2TorsG

DEFINITION 5.5.4. Recall[59] that the G-set $\mathbb{P}_y : BG \to \mathrm{Set}$ is defined for any $y : BG$ by $\mathbb{P}_y(z) \equiv (y \xrightarrow{=} z)$. Note that $\mathbb{P}_y$ is a G-torsor since its action type $(\mathbb{P}_y)_{hG}$ is contractible, so we can define

$$\mathbb{P}_{_} : BG \to_* (\mathrm{Torsor}_G, \mathbb{P}_{\mathrm{sh}_G}) : y \mapsto P_y,$$

pointed by reflexivity.[60] If G is not clear from the context, we may choose to write $\mathbb{P}^G_{_}$ instead of $\mathbb{P}_{_}$. ⌟

rem:pathsptransport

REMARK 5.5.5. We will use several variants of $\mathbb{P}_{_}$, in combination with some of the conventions introduced back in Chapter 2. In this remark, to avoid confusion, we explain these variants.

First, we also use $\mathbb{P}_{_}$ to denote its induced action on paths: for $y, z : BG$ we have

$$\mathbb{P}_{_} : (y \xrightarrow{=} z) \to (\mathbb{P}_y \xrightarrow{=} \mathbb{P}_z),$$

defined by path induction as in Definition 2.6.1.

Then, as $\mathbb{P}_y \xrightarrow{=} \mathbb{P}_z$ is an identity between families of types, function extensionality (**??** 2.9.18) applies. For $q : y \xrightarrow{=} z$, we may also use $\mathbb{P}_q$ to denote the corresponding function of type $\prod_{x:BG}(\mathbb{P}_y(x) \xrightarrow{=} \mathbb{P}_z(x))$.

Finally, as $\mathbb{P}_y(x)$ and $\mathbb{P}_z(x)$ are types, univalence (**??** 2.13.2) applies. Therefore we may use $\mathbb{P}_q(x)$ to denote the corresponding equivalence, i.e., transport in the type family $\mathbb{P}_{_}(x)$, sending $p : \mathbb{P}_y(x) \equiv (y \xrightarrow{=} x)$ to $pq^{-1} : \mathbb{P}_z(x) \equiv (z \xrightarrow{=} x)$.[61] ⌟

For connoisseurs of category theory, the following lemma is a corollary of a *type-theoretic Yoneda lemma*,[62] and the proof is Exercise 3.5.4.[63]

lem:pathsptransportiseq

LEMMA 5.5.6. *Let G be a group. For all $y, z : BG$ the induced map of identity types*

$$\mathbb{P}_{_} : (y \xrightarrow{=} z) \to (\mathbb{P}_y \xrightarrow{=} \mathbb{P}_z)$$

is an equivalence.

The following theorem justifies the title of this section, stating that the classifying type of a group is the type of its torsors.

lem:BGbytorsor

THEOREM 5.5.7. *Let G be a group. Then the function $\mathbb{P}^G_{_} : BG \to \mathrm{Torsor}_G$ from Definition 5.5.4 is an equivalence.*[64]

Proof. Since both Torsor_G and BG are pointed and connected, it suffices by Corollary 2.17.9(4) to show that $\mathbb{P}^G_{_} : (\mathrm{sh}_G \xrightarrow{=} \mathrm{sh}_G) \to (\mathbb{P}_{\mathrm{sh}_G} \xrightarrow{=} \mathbb{P}_{\mathrm{sh}_G})$ is an equivalence. This follows directly from Lemma 5.5.6. □

5.5.8 *Homomorphisms and torsors*

sec:homotor

In view of the equivalence $\mathbb{P}^G_{_}$ between BG and $(\mathrm{Torsor}_G, \mathbb{P}_{\mathrm{sh}_G})$ of Theorem 5.5.7 one might ask what a group homomorphism $f : \mathrm{Hom}(G, H)$ translates to on the level of torsors. Off-hand, the answer is the round-trip $(\mathbb{P}^H_{_})Bf(\mathbb{P}^G_{_})^{-1}$, but we can be more concrete than that. We do know that for $z : BG$ the G-torsor $\mathbb{P}^G_z$ should be sent to $\mathbb{P}^H_{Bf(z)}$, but how do we express this for an arbitrary G-torsor?

[56] Admittedly in a higher universe, but we can use the Replacement **??** 2.19.4 to see that Torsor_G is equivalent to a type in the same universe as G – even before we have Theorem 5.5.7 showing we can take BG.

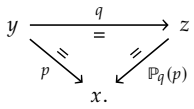
[57] By the way, the name "torsor" is a translation from the French *torseur*, introduced by Giraud,[58] who related them to "twisting" operations on bundles. Since BG is equivalent to the type of G-torsors, we can also think of shapes $t : BG$ as giving rise to "twists". Indeed, for a G-set X, we can think of $X(x)$ as a "twisted" version of the underlying set, $X(\mathrm{sh}_G)$.

[58] Jean Giraud. *Cohomologie non abélienne*. Die Grundlehren der mathematischen Wissenschaften, Band 179. Springer-Verlag, Berlin-New York, 1971, pp. ix+467.

[59] See Example 5.2.4 and Equation (5.2.1).

[60] That is, we have classified a homomorphism from G to $\mathrm{Aut}_{G\text{-}\mathrm{Set}}(\mathbb{P}_{\mathrm{sh}_G})$. It'll turn out to be an isomorphism.

[61] In a commutative diagram,



[62] This is more than an analogy: using Theorem 6.7.15 and Theorem 6.6.2, we obtain almost the same result. The only difference lies in that our $\mathbb{P}_{\mathrm{sh}_G}$ here translates to a *covariant* representable instead of a contravariant one.

[63] It is also possible to prove the lemma directly by an application of Construction 2.9.9: Take as inverse equivalence the map Q mapping any $f : \mathbb{P}_y \xrightarrow{=} \mathbb{P}_z$ to $Q(f) :\equiv (f_y(\mathrm{refl}_y))^{-1} : (y \xrightarrow{=} z)$.

[64] A similar results holds for ∞-groups.

def:restrictandinduce

DEFINITION 5.5.9. Let $f : \mathrm{Hom}(G, H)$ be a group homomorphism. If $Y : BH \to \mathrm{Set}$ is an H-set, then the *restriction* f^*Y of Y to G is the G-set given by precomposition[65]

$$f^*Y :\equiv (Y \circ Bf) : BG \to \mathrm{Set}.$$

If $X : BG \to \mathrm{Set}$ is a G-set, we define the *induced H-set* $f_!X : BH \to \mathrm{Set}$ by setting, for $w : BH$,[66]

$$f_!X(w) :\equiv \left\| \sum_{z:BG} \left((Bf(z) \xrightarrow{=} w) \times X(z) \right) \right\|_0 .$$

fc:f_!X(w)-orbitset

The following exercise shows that the set-truncation in the definition of $f_!$ above really makes a difference.[67]

xca:why-setTrunc-f_!

EXERCISE 5.5.10. Find groups G, H, a homomorphism $f : \mathrm{Hom}(G, H)$ and a G-set X such that $(w : BH) \mapsto \sum_{z:BG} \left((Bf(z) \xrightarrow{=} w) \times X(z) \right)$ is an H-type that is not an H-set. ⌟

xca:adjunction-_!-_^*

EXERCISE 5.5.11. Give an equivalence from $f_!X$ to $X \circ Bf^{-1}$ (i.e., $(f^{-1})^* X$) if f is an isomorphism. Give an equivalence between the types $\mathrm{Hom}_H(f_!X, Y)$ and $\mathrm{Hom}_G(X, f^*Y)$, for all G-sets X and H-sets Y. ⌟

rem:^*-_!-as-(pre)image

REMARK 5.5.12. The purpose of this remark is to explain how $f_!X$ and f^*Y may be viewed as a certain kind of image and preimage, respectively. In Definition 2.17.11 we defined the (propositional) image of a function $f : A \to B$, and in Section 3.9 the higher images. In these definitions we used the whole domain A of f. In Definition 2.9.3 we defined the preimage, or fiber, of f, for any element b of the codomain B.

It is natural to generalize both image and preimage of a function to subtypes of the domain and codomain. Let A and B be types, $f : A \to B$ a function, and consider the types of subtypes $\mathrm{Sub}(A) \equiv (A \to \mathrm{Prop})$ and $\mathrm{Sub}(B) \equiv (B \to \mathrm{Prop})$. Given a subtype $Y : \mathrm{Sub}(B)$ of B, consider $f^*Y \equiv Y \circ f$. Then f^*Y is subtype of A consisting of precisely those $a : A$ for which $f(a)$ is in Y, in other words, the preimage of Y under f.

Now let $X : \mathrm{Sub}(A)$ be a subtype of A and consider the subtype of B defined by the predicate

$$\exists_f X(b) :\equiv \left\| \sum_{a:A} \left((f(a) \xrightarrow{=} b) \times X(a) \right) \right\| \equiv \exists_{a:A} \left((f(a) \xrightarrow{=} b) \times X(a) \right).$$

This is $f_!X$ with propositional truncation instead of set truncation, and it holds precisely for all $b : B$ for which there exists an a in X with $f(a) \xrightarrow{=} b$, in other words, the image of X under f. These operations are illustrated in Figure 5.6 where we view f as a projection, so the fibers are vertical.

Note that the above makes little sense for $A \equiv BG$ and $B \equiv BH$, since predicates on connected types are constant. However, the intuition carries over to Set valued functions X and Y and a set truncated $f_!X$, analogously to higher images defined in Section 3.9. ⌟

rem:coinduced-Hset

REMARK 5.5.13. Dually to $f_!X(w)$ in Definition 5.5.9, there is also a *coinduced H-set* $f_* : BH \to \mathrm{Set}$ given by

$$f_*X(w) :\equiv \prod_{z:BG} \left((Bf(z) \xrightarrow{=} w) \to X(z) \right).$$

Note that this always lands in sets since X does.[68] ⌟

[65] Example: $\tilde{G}_x$ from Definition 5.4.25 can be written as $i_x^* \mathbb{P}_{\mathrm{sh}_G}$, i.e., as the restriction of the principal G-torsor to the stabilizer group G_x using $i_x : \mathrm{Hom}(G_x, G)$.

[66] Note that the set $f_!X(w)$ can also be identified as the set of orbits of the G-set $(z : BG) \mapsto (Bf(z) \xrightarrow{=} w) \times X(z)$, whose underlying set is equivalent to $\mathbb{P}_{\mathrm{sh}_H}(w) \times X(\mathrm{sh}_G)$.

[67] This situation is common in algebra and is often referred to by saying that some construction, in this case the untruncated definiens of $f_!X$, is not "exact".

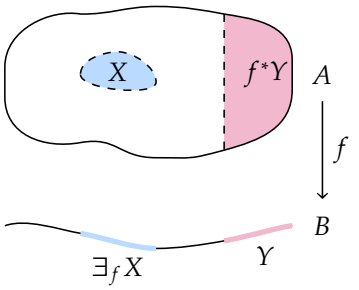


FIGURE 5.6: Image ($\exists_f$) and preimage (f^*) of subsets for a map $f : A \to B$.

fig:im-preim-subsets

[68] The type $f_*X(w)$ can also be identified as the set of invariant maps of the G-set $\mathbb{P}_{Bf}(w) \to X$, where $(\mathbb{P}_{Bf}(w) \to X)(z) :\equiv (Bf(z) \xrightarrow{=} w) \to X(z)$ for $z : BG$.

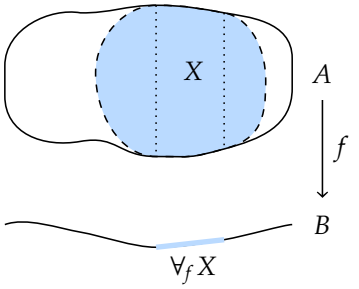


FIGURE 5.7: Coinduced image of a subset for a map $f : A \to B$.

fig:coinduced-image

rem:coinduced-subset

REMARK 5.5.14. As in Remark 5.5.12, we can also consider f_* as an operation on subsets $f_* : \mathrm{Sub}(A) \to \mathrm{Sub}(B)$ for $f : A \to B$. The coinduced subset $\forall_f X :\equiv f_* X$ is less familiar than the image; it contains those $b : B$ for which the entire preimage $f^{-1}(b)$ is contained in X, as illustrated in Figure 5.7. ⌟

xca:adjunction-^*-_*

EXERCISE 5.5.15. Give an equivalence between the types $\mathrm{Hom}_G(f^*Y, X)$ and $\mathrm{Hom}_H(Y, f_*X)$, for all G-sets X and H-sets Y. ⌟

When X is the G-torsor $\mathbb{P}^G_x$, for some $x : BG$, the contraction (recall Lemma 2.9.10) of $\sum_{z:BG}(x \xrightarrow{=} z)$ induces an equivalence η_w of type

$$f_! \mathbb{P}^G_x(w) \equiv \left\| \sum_{z:BG} \left((Bf(z) \xrightarrow{=} w) \times (x \xrightarrow{=} z) \right) \right\|_0 \xrightarrow{\simeq} (Bf(x) \xrightarrow{=} w) \equiv \mathbb{P}^H_{Bf(x)}(w).$$

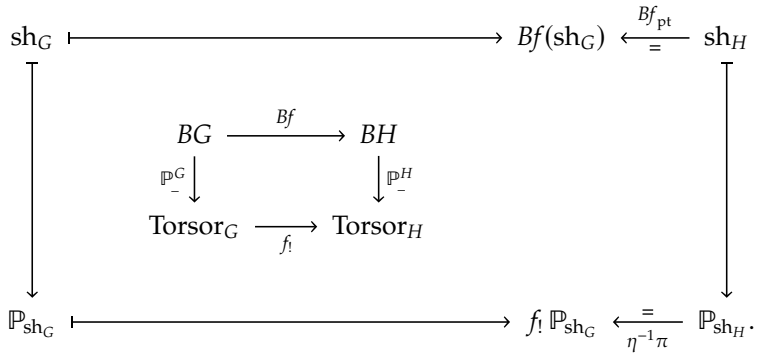
Taking $x \equiv \mathrm{sh}_G$, we get a path $\eta : f_! \mathbb{P}_{\mathrm{sh}_G} \xrightarrow{=} \mathbb{P}^H_{Bf(\mathrm{sh}_G)}$. We also have the path $Bf_{\mathrm{pt}} : \mathrm{sh}_H \xrightarrow{=} Bf(\mathrm{sh}_G)$, so that the action of $\mathbb{P}^H_{_}$ gives us a path $\pi : \mathbb{P}_{\mathrm{sh}_H} \equiv \mathbb{P}^H_{\mathrm{sh}_H} \xrightarrow{=} \mathbb{P}^H_{Bf(\mathrm{sh}_G)}$. Combining we get $\eta^{-1}\pi : \mathbb{P}_{\mathrm{sh}_H} \xrightarrow{=} f_! \mathbb{P}_{\mathrm{sh}_G}$.

If X is a G-set such that $\| \mathbb{P}_{\mathrm{sh}_G} \xrightarrow{=} X \|$, then $f_! X$ is an H-set such that $\| \mathbb{P}_{\mathrm{sh}_H} \xrightarrow{=} f_! X \|$, so that $f_! : \mathrm{Torsor}_G \to_* \mathrm{Torsor}_H$, pointed by $\eta^{-1}\pi$.

Summing up, we have implemented the following:

con:inducedtorsor

CONSTRUCTION 5.5.16. *Let $f : \mathrm{Hom}(G, H)$ be a group homomorphism. Then f induces a pointed map $f_! : \mathrm{Torsor}_G \to_* \mathrm{Torsor}_H$, and we have a path of type $f_! \mathbb{P}^G_{_} \xrightarrow{=} \mathbb{P}^H_{_} Bf \equiv f^* \mathbb{P}^H_{_}$, all represented by the following diagram*:



We will have the occasion to use the following result which essentially says that if $f : \mathrm{Hom}(G, H)$ is such that Uf is surjective,[69] then f^* embeds the type of H-sets as some of the components of the type of G-sets.

lem:epifullyfaithful

LEMMA 5.5.17. *Let G and H be groups and let $f : \mathrm{Hom}(G, H)$ be such that Uf is surjective. Then the map f^* from Definition 5.5.9 is an injection.*

Proof. By Lemma 2.17.8 it suffices to prove that, for all H-sets X and Y, the induced map $f^* : (X \xrightarrow{=} Y) \to (f^*X \xrightarrow{=} f^*Y)$ is an equivalence. By function extensionality, the domain of the induced f^* is equivalent to the type Z^{hH} of invariant maps of the H-set Z defined by $Z(w) :\equiv (X(w) \xrightarrow{=} Y(w))$ for all $w : BH$. By Lemma 5.4.21, Z^{hH} is equivalent to the subset of $Z(\mathrm{sh}_H)$ consisting of the fixed elements, i.e., those $p : Z(\mathrm{sh}_H)$ such that $p \cdot X(h) = Y(h) \cdot p$ for all $h : UH$.[70] Likewise, for the codomain we get equivalence to $(f^*Z)^{hG}$ and to the subset of $(f^*Z)(\mathrm{sh}_G)$ consisting of the fixed elements, i.e., those $q : Z(Bf(\mathrm{sh}_G))$ such that $q \cdot X(Bf(g)) = Y(Bf(g)) \cdot q$ for all $g : UG$. It remains to show that the latter two subsets are equivalent. The path Bf_{pt} induces an equivalence $Z(Bf_{\mathrm{pt}}) : Z(\mathrm{sh}_H) \xrightarrow{\simeq} Z(Bf(\mathrm{sh}_G))$ mapping $p : (X(\mathrm{sh}_H) \xrightarrow{=} Y(\mathrm{sh}_H))$ to $Y(Bf_{\mathrm{pt}}) \cdot p \cdot X(Bf_{\mathrm{pt}})^{-1}$. Figure 5.8

[69] In Section 9.2 we will prove such an f to be an *epimorphism*, just as we called f a *monomorphism* in Definition 5.3.13 when Uf is injective. The notions of monomorphism and epimorphism will be generalized in Definitions 6.3.7 and 6.3.8, respectively.

[70] Equivalently: $p(h \cdot_X x) =_{Y(\mathrm{sh}_H)} h \cdot_Y p(x)$ for all $x : X(\mathrm{sh}_H)$ and $h : UH$.

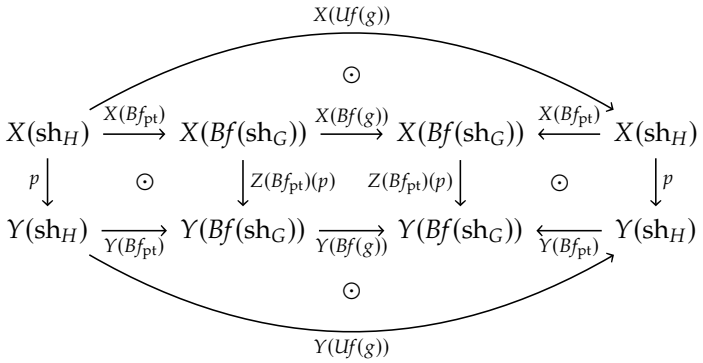


Figure 5.8: All arrows are paths, so invertible. The squares marked with ⊙ commute by the definition of Uf and of Z, plus easy path algebra. Hence the middle square commutes if and only if the outer one (the contour) does.

fig:epifullyfaithful

shows that the propositions $Z(Bf_{\mathrm{pt}})(p) \cdot X(Bf(g)) = Y(Bf(g)) \cdot Z(Bf_{\mathrm{pt}})(p)$ and $p \cdot X(Uf(g)) = Y(Uf(g)) \cdot p$, for any $g : UG$ are equivalent. In particular, if Uf is surjective we can use Exercise 2.17.14 and get that $Z(Bf_{\mathrm{pt}})$ is an equivalence of the respective subsets of fixed elements. □

5.6 *Any symmetry is a symmetry in* Set

sec:groupssubperm

For abstract groups there is a result, attributed to Cayley, which is often stated as "any group is a permutation group". In our parlance this translates to "any symmetry is a symmetry in Set". The aim of this section is to give a precise formulation of the latter and prove it, using what we learned in Section 5.5.

Let G be a group. Recall from Theorem 5.5.7 the equivalence

$$\mathbb{P}^G_{_} : BG \xrightarrow{\simeq} \mathrm{Torsor}_G : z \mapsto (y \mapsto (z \xrightarrow{=} y)).$$

Fixing the second argument y to sh_G, we obtain a map $\mathbb{P}^G_{_}(\mathrm{sh}_G) : BG \to \mathrm{Set}$. And since $\mathbb{P}^G_{\mathrm{sh}_G}(\mathrm{sh}_G) \equiv UG$, we can point $\mathbb{P}^G_{_}(\mathrm{sh}_G)$ with reflexivity to make it a pointed function $BG \to_* \mathrm{B}\Sigma_{UG}$, i.e., it classifies a homomorphism from G to the permutation group $\Sigma_{UG} \equiv \mathrm{Aut}_{\mathrm{Set}}(UG)$, denoted by[71]

$$\rho_G : \mathrm{Hom}(G, \Sigma_{UG}).$$

[71] The letter ρ commemorates the word "regular"

Theorem 5.6.1 (Cayley). *For any group G, ρ_G is a monomorphism.*[72]

thm:allgpsarepermutationgps

Proof. In view of Definition 5.3.13 we need to show that $\mathrm{B}\rho_G \equiv \mathbb{P}^G_{_}(\mathrm{sh}_G) : BG \to \mathrm{B}\Sigma_{UG}$ is a set bundle. Note first that $\mathbb{P}^G_{_}(\mathrm{sh}_G)$ can be factorized, as a pointed map, as:[73]

$$\begin{array}{ccc} BG & \xrightarrow[\sim]{\mathbb{P}^G_{_}} & (\mathrm{Torsor}_G, \mathbb{P}_{\mathrm{sh}_G}) \\ & \searrow^{\mathrm{B}\rho_G} & \downarrow{\scriptstyle \mathrm{ev}_{\mathrm{sh}_G}} \\ & & \mathrm{B}\Sigma_{UG} \end{array}$$

Here, $\mathrm{ev}_{\mathrm{sh}_G} : \mathrm{Torsor}_G \to \mathrm{Set}_{(UG)}$ is the evaluation map defined by $\mathrm{ev}_{\mathrm{sh}_G}(X) :\equiv X(\mathrm{sh}_G)$ and pointed by reflexivity.

[72] By Definition 5.3.13, ρ_G is a monomorphism means that the induced map $\mathrm{U}\rho_G$ from the symmetries of sh_G in BG_* to the symmetries of UG in Set is an injection, i.e., "any symmetry is a symmetry in Set".

[73] All the maps involved are pointed maps between groupoids, so the commutativity of this triangle is a proposition.

Since $\mathbb{P}^G_-$ is an equivalence, we are done if we show that, for any $S : \mathrm{Set}_{(UG)}$, the fiber $\mathrm{ev}^{-1}_{\mathrm{sh}_G}(S)$ is a set. This fiber is by definition $\sum_{X : \mathrm{Torsor}_G}(S \xrightarrow{=} X(\mathrm{sh}_G))$, which is a subtype of $\sum_{X : BG \to \mathrm{Set}}(S \xrightarrow{=} X(\mathrm{sh}_G))$. The latter is the type of pointed maps from BG to (Set, S) and hence a set by Lemma 4.4.12 (more speicifcally Footnote 20, since BG is connected and Set is a groupoid). Therefore the fiber $\mathrm{ev}^{-1}_{\mathrm{sh}_G}(S)$ a set. □

Remark 5.6.2. Note that there is a more elementary proof of the result above, that doesn't rely on Theorem 5.5.7 (a very strong result!). All we need to show is that $\mathrm{B}\rho_G : BG \to \mathrm{B}\Sigma_{UG}$ is a set bundle. But, for any set $X : \mathrm{B}\Sigma_{UG}$, the fiber of $\mathrm{B}\rho_G$ at X is

$$\mathrm{B}\rho_G^{-1}(X) \equiv \sum_{z : BG} X \xrightarrow{=} (z \xrightarrow{=} \mathrm{sh}_G).$$

Take two elements (z_1, ϕ_1) and (z_2, ϕ_2) of this fiber. Then an identification of type $(z_1, \phi_1) \xrightarrow{=} (z_2, \phi_2)$ is given by a path $p : z_1 \xrightarrow{=} z_2$ such that the following triangle commutes:

$$\begin{array}{ccc} X & \xrightarrow{\phi_1} & (z_1 \xrightarrow{=} \mathrm{sh}_G) \\ & \searrow^{\phi_2} & \downarrow{\scriptstyle -\cdot p^{-1}} \\ & & (z_2 \xrightarrow{=} \mathrm{sh}_G) \end{array}$$

The goal is to prove that $(z_1, \phi_1) \xrightarrow{=} (z_2, \phi_2)$ is a proposition, and so we can use the connectedness of BG and suppose that $z_1 \equiv \mathrm{sh}_G$. In this case, the commutative triangle above fully determines p: indeed, looking at the image of $\mathrm{refl}_{\mathrm{sh}_G}$ (in the upper right corner), we get $p = \phi_2(\phi_1^{-1}(\mathrm{refl}_{\mathrm{sh}_G}))^{-1}$.

However, the proof presented with the theorem's statement is conceptually more interesting, and offers a preview into the Yoneda Lemma (see Theorem 6.6.2). ⌟

Note that the Theorem 5.6.1 yields that (G, ρ_G) is a monomorphism into Σ_{UG}. In other words, G is a subgroup of Σ_{UG}. Note that the induced map on symmetries is:

$$\mathrm{U}\rho_G : \mathrm{U}G \to (\mathrm{U}G \xrightarrow{=} \mathrm{U}G) : g \mapsto (x \mapsto xg^{-1})$$

The reader may ask for a more informative underlying group of this subgroup than G itself. To this end we apply E from Lemma 5.3.18 to (G, ρ_G) to obtain a subgroup in the sense of Definition 5.3.2. The underlying group of $E(G, \rho_G)$ has classifying type $\sum_{X : \mathrm{B}\Sigma_{UG}} \mathrm{B}\rho_G^{-1}(X)$ and shape $(UG, (\mathrm{sh}_G, \mathrm{id}_{UG}))$. As illustrated in the following diagram, the symmetries of this shape are precisely those permutations π of UG for with there exist a $g : UG$ such that π is right multiplication by g^{-1}.[74]

$$\begin{array}{ccccc} UG & \xrightarrow[\sim]{\mathrm{id}} & (\mathrm{sh}_G & = & \mathrm{sh}_G) \\ {\scriptstyle\pi}\downarrow{\scriptstyle\wr} & & {\scriptstyle =}\vdots{\scriptstyle g}\downarrow & & \wr\downarrow{\scriptstyle -\cdot g^{-1}} \\ UG & \xrightarrow[\mathrm{id}]{\sim} & (\mathrm{sh}_G & = & \mathrm{sh}_G) \end{array}$$

Remark 5.6.3. In many cases, the set UG used in Theorem 5.6.1 is larger than necessary for obtaining the symmetries in G as symmetries of a set. A case in point is the group Σ_3, where the symmetries *are* already symmetries of a set, namely of the set $\mathfrak{3}$. However, $\mathrm{U}\Sigma_3 \equiv (\mathfrak{3} \xrightarrow{=} \mathfrak{3})$ is a

[74] Those who have read Caley's theorem in ordinary abstract group theory might find this computation suprising. Indeed, traditionally, Cayley's Theorem selects the permutations that are left multiplications by some g. We could have arranged for $\mathrm{U}\rho_G$ to select the same permutations here (by chosing $\mathrm{B}\rho_G$ to be $\mathbb{P}_{\mathrm{sh}_G}$ instead), but then the triangle of the proof does not commute anymore: to recover the commutativity, one must point $\mathrm{ev}_{\mathrm{sh}_G}$ by the non-trivial path $_^{-1} : UG \xrightarrow{=} UG$.

rem:Cayleyversize

6-element set. Let's take a closer look at where and how this happens in the proof.

As we noticed in the proof of Theorem 5.6.1, the map $\mathbb{P}^G_{_}(\mathrm{sh}_G) : BG \to_* \mathrm{Set}_{(UG)}$ classifying the monomorphism ρ_G is decomposed as an equivalence $\mathbb{P}^G_{_}$ (pointed by reflexivity) followed by the evaluation map $\mathrm{ev}_{\mathrm{sh}_G}$ (also pointed by reflexivity). This is depicted in the following diagram, where the second line shows the induced maps on the symmetries.

$$BG \xrightarrow[\sim]{\mathbb{P}_{_}} (\mathrm{Torsor}_G, \mathbb{P}_{\mathrm{sh}_G}) \xrightarrow{\mathrm{ev}_{\mathrm{sh}_G}} \mathrm{Set}_{(UG)}$$

$$UG \xrightarrow[\sim]{\mathbb{P}_{_}} (\mathbb{P}_{\mathrm{sh}_G} \xrightarrow{=} \mathbb{P}_{\mathrm{sh}_G}) \xrightarrow{\mathrm{ev}_{\mathrm{sh}_G}} (UG \xrightarrow{=} UG)$$

Let PP be the G-set given by $\mathrm{PP}(z) :\equiv (\mathbb{P}_{\mathrm{sh}_G}(z) \xrightarrow{=} \mathbb{P}_{\mathrm{sh}_G}(z))$ for all $z : BG$. By function extensionality, $\mathbb{P}_{\mathrm{sh}_G} \xrightarrow{=} \mathbb{P}_{\mathrm{sh}_G}$ is equivalent to $\prod_{z:BG} \mathrm{PP}(z)$, the type of invariant maps of PP. By Lemma 5.4.21(1), an invariant map h, and hence the corresponding symmetries of $\mathbb{P}_{\mathrm{sh}_G}$, is uniquely determined by its value h_{sh_G} at sh_G.[75] The underlying set of PP being $\mathrm{PP}(\mathrm{sh}_G) \equiv (UG \xrightarrow{=} UG)$, Lemma 5.4.21(2) characterizes exactly the invariant maps of PP as corresponding via $\mathrm{ev}_{\mathrm{sh}_G}$ with elements of $UG \xrightarrow{=} UG$ fixed by the action of PP. How does PP act on UG? That's for you to determine in Exercise 5.6.4. In other words, $\mathrm{ev}_{\mathrm{sh}_G}$ forgets about the extra structure of PP and sends invariant maps of PP to fixed permutations of UG. For example, in the case of Σ_3, the fixed permutations are the 6 elements of $\mathrm{U}\Sigma_3 \xrightarrow{=} \mathrm{U}\Sigma_3$ given by $_ \circ \sigma^{-1}$ for each of the 6 possible $\sigma : 3 \xrightarrow{=} 3$. ⌟

[75] This is an alternative way to understand that $\mathrm{ev}_{\mathrm{sh}_G}$, and hence $\mathbb{P}^G_{_}$, classifies a monomorphism.

xca:PP-fixed-permutations

Exercise 5.6.4. Let conditions be as in Remark 5.6.3. By analyzing transport in the type family $\mathbb{P}_{\mathrm{sh}_G}$, show that a permutation π of UG is fixed if and only if $\pi(gg') = g\pi(g')$ for all $g, g' : UG$. Show that the fixed permutations of UG form an abstract group and that evaluation of such a permutation at $\mathrm{refl}_{\mathrm{sh}_G}$ yields an abstract isomorphism from this group to $\mathrm{abs}(G)$. ⌟

Should we add a little remark on the other natural $\mathrm{B}\rho_G$ possible, namely $\mathbb{P}_{\mathrm{sh}_G}$? (It is equal to the current one if we point it with $_^{-1}$.)

5.7 *The lemma that is not Burnside's*

thm:burnsides-lemma
exa:prep-burnside

Example 5.7.1. Since the lemma to come is about counting orbits and elements of orbits, we start by elaborating an example. Recall from Example 4.2.22 the cyclic group $\mathrm{C}_4 \equiv \mathrm{Aut}_{\mathrm{Cyc}}(4, s)$, where Cyc is defined in Definition 3.6.3 as the type of cycles, i.e., pairs (X, t) of a set X and a permutation $t : X \xrightarrow{\simeq} X$ such that any two points of X are some t-steps apart. Let $X : \mathrm{BC}_4 \to \mathrm{Set}$ be the C_4-set mapping any $(A, f) : \mathrm{BC}_4$ to $A \to 2$. Then the underlying set of X is $4 \to 2$, i.e., binary sequences of length 4. The group action induced by X cyclically rotates such sequences, by 0, 1, 2 or 3 positions.[76]

[76] Use Corollary 3.6.16, univalence, and Construction 2.14.2.

By Corollary 5.4.6, the set of orbits X/C_4 is equivalent to the quotient of $4 \to 2$ induced by $[_] : (4 \to 2) \to X/\mathrm{C}_4$. As also stated by that lemma, the equivalence class of any $x : 4 \to 2$ consists precisely of all cyclic rotations of x. Clearly, 0000 and 1111 have singleton equivalence classes. The equivalence class of 0001 (resp. 0111) consists of all four

binary sequences with exactly one 1 (resp. 0). Before you start thinking that swapping 0's and 1's gives a new equivalence class, consider 0101 that forms an equivalence class together with 1010. Finally, 0011 forms an equivalence class together with 1001, 1100 and 0110. Thus we have distributed all 16 sequences over six orbits, as in the left column of Table 5.1.

orbit	stabilizers
0000	0, 1, 2, 3
1111	0, 1, 2, 3
0001, 0010, 0100, 1000	0
0111, 1011, 1101, 1110	0
0101, 1010	0, 2
1100, 0110, 0011, 1001	0

TABLE 5.1: Underlying sets of orbits and the stabilizers of their elements.

In the right column of Table 5.1, we have given in each row the respective stabilizing symmetries in BC_4. Exercise 5.4.12 tells us that it doesn't matter too much which element in the orbit one chooses.[77] For the cardinality $\#((\mathrm{C}_4)_x)$ of the finite stabilizer groups, the particular x one chooses within in each orbit is irrelevant, but may vary from orbit to orbit. Now we can observe something interesting: the product $\#(\mathrm{C}_4 \cdot x) \times \#((\mathrm{C}_4)_x)$ (i.e., in each row, the number of elements on the left times that on the right) is equal to $\#(\mathrm{C}_4) = 4$, for each x in the underlying set of X. This follows from Lagrange's Theorem, in particular Exercise 5.3.29, applied with $G \equiv \mathrm{C}_4$ and taking for X the underlying C_4-set of $[x]$, which is transitive.

[77] Here it matters even less since C_4 is abelian.

Another observation in Table 5.1 is that, since there are six orbits and the orbits induce a disjoint partition of $4 \to 2$, there are in total 24 pairs (g, x) with $g \cdot x = x$. This insight leads to the following lemma. ⌟

lem:burnside

LEMMA 5.7.2. *Let G be a finite group and let $X : BG \to \mathrm{Set}$ be a finite G-set. For any $g : UG$, define the set $X^g :\equiv \{\, x : X(\mathrm{sh}_G) \mid g \cdot x = x \,\}$ of points fixed by g. Then each X^g, the sum type $\sum_{g:UG} X^g$, and the set of orbits X/G are finite sets, and we have*

eq:burnside

$$\#\Big(\sum_{g:UG} X^g\Big) = \#(X/G) \times \#(G). \tag{5.7.1}$$

Proof. We first need to make sure that the sets involved are finite. Finite sets are decidable sets, see Exercise 2.24.6. Hence each X^g is a finite set, as it is a decidable subset of $X(\mathrm{sh}_G)$, see Remark 2.24.10.

Finiteness of $\sum_{g:UG} X^g$ follows from Exercise 2.24.12. Regarding the set of orbits, note that Corollary 5.4.6 yield that X/G is equivalent to the quotient of $X(\mathrm{sh}_G)$ modulo the equivalence relation $\exists_{g:UG}\, x = g \cdot y$. The latter proposition is decidable by Exercise 2.24.11. Now apply Exercise 2.24.13.

Since the main statement Equation (5.7.1) of the lemma is a proposition, we may assume that, for both $X(\mathrm{sh}_G)$ and UG, we have an equivalence to a standard finite set. Rearranging sums and writing $X(\mathrm{sh}_G)$ as the sum of fibers of $[_] : X(\mathrm{sh}_G) \to X/G$ gives equivalences:

$$\sum_{g:UG} X^g \equiv \sum_{g:UG} \sum_{x:X(\mathrm{sh}_G)} (g \cdot x = x) \xrightarrow{\simeq} \sum_{x:X(\mathrm{sh}_G)} \sum_{g:UG} (g \cdot x = x) \xrightarrow{\simeq}$$
$$\sum_{x:X(\mathrm{sh}_G)} UG_x \xrightarrow{\simeq} \sum_{O:X/G} \sum_{x:X(\mathrm{sh}_G)} \big((O = [x]) \times UG_x\big) \xrightarrow{\simeq} \sum_{O:X/G} \sum_{x:X_O(\mathrm{sh}_G)} UG_x$$

In the last step we have used that $O = [x]$ is equivalent to $O(\mathrm{sh}_G, x)$, which means that x is in the underlying set $X_O(\mathrm{sh}_G)$ of the orbit O, see Definition 5.4.1 and Definition 5.2.11.

Note that the last type in the chain above reflects how we counted in Table 5.1: for every orbit, and every element in the underlying set of that orbit, we counted the stabilizers of that element.

We aim to apply the Lagrange construction with subgroups defined by X_O and $x_O : X_O(\mathrm{sh}_G)$, for any orbit $O : X/G$. These points x_O can

be obtained as the 'least' $x : X(\mathrm{sh}_G)$ such that $O = [x]$, where 'least' means: corresponding to the smallest number under the equivalence of $X(\mathrm{sh}_G)$ with a standard finite set. We also have to give functions $f_O : \prod_{y : X_O(\mathrm{sh}_G)} \sum_{g : UG} g \cdot_{X_O} y = x_O$, for every $O : X/G$. Such functions are obtained by using the transitivity of X_O in combination with the equivalence between UG and a standard finite set: we can simply take the 'least' $g : UG$ such that $g \cdot_{X_O} y = x_O$. Applying Construction 5.3.28, we get an equivalence between UG and $\sum_{x : X_O(\mathrm{sh}_G)} UG_x$. We conclude that $\#(\sum_{g : UG} X^g) = \#(X/G) \times \#(G)$, using Exercise 2.24.12. □

As a first application of Burnside's Lemma, we note the following number-theoretic consequence, which falls out when we consider the analog of Example 5.7.1 for the case of C_p acting on base-n sequences of length p.

THEOREM 5.7.3 (Fermat's Little Theorem). *For any prime p and natural number n, we have $p \mid n^p - n$.*

Proof. Consider the C_p-set $X : \mathrm{BC}_p \to \mathrm{Set}$ defined by

$$X(S, t) :\equiv (S \to \mathbb{m}),$$

for any p-cycle (S, t). The underlying set is the type of functions $\mathbb{p} \to \mathbb{m}$, which is finite of cardinality n^p.

For any $f : \mathbb{p} \to \mathbb{m}$, we can apply Exercise 5.3.29 to the stabilizer subgroup $(\mathrm{C}_p)_f$. But since $p = \#(\mathrm{C}_p)$ is prime, $(\mathrm{C}_p)_f$ can only be trivial or all of C_p. The latter happens when f is one of the n constant functions. Otherwise,[78] select k such that $f(k) \neq f(0)$, and surely $s^k \cdot f \neq f$: then s^k is not in UC_p[79] the only option is that $(\mathrm{C}_p)_f$ is trivial. In other words, X^g is all of $X(\mathbb{p}, s)$ for $g = \mathrm{refl}_{(\mathbb{p},s)}$, and only the constant functions for anyof the other $p - 1$ symmetries in UC_p.

Now apply Burnside's Lemma 5.7.2. We get:

$$n^p + n(p-1) = \#\Big(\sum_{g : \mathrm{UC}_p} X^g\Big) = \#(X/\mathrm{C}_p) \times \#(\mathrm{C}_p) = \#(X/\mathrm{C}_p) \times p,$$

and so we conclude that p divides $n^p - n$. □

[78] The set $\mathbb{p} \to \mathbb{m}$ is finite, hence decidable, so there is no risk to make a case-by-base analysis.

[79] More rigorously, s^k is not in the image of Ui_f where $i_f : (\mathrm{C}_p)_f \to \mathrm{C}_p$ is the monomorphism defining the stablizer.

6
A categorical interlude

ch:cats

We have seen that many types carry a notion of morphism between its elements:

- We have functions $f : A \to B$ between types A and B in a universe $\mathcal{U}$ (Section 2.2),
- We have identifications $p : x \xrightarrow{=} y$ between elements x, y of any type A (Section 2.5),
- We have pointed functions $f : A \to_* B$ between pointed types A and B in $\mathcal{U}_*$ (Definition 2.21.1),
- We have fiberwise maps $f : \prod_{x:X}(A(x) \to B(x))$ between families $A, B : X \to \mathcal{U}$ (Definition 2.9.14),
- We have homomorphisms $f : \mathrm{Hom}(G, H)$ between groups $G, H : \mathrm{Group}$ (Definition 4.4.2),
- We have maps of cycles $f : (X, t) \to (Y, u)$ and of bicycles $(X, a, b) \to (X', a', b')$,
- We have maps of G-sets $f : \mathrm{Hom}_G(X, Y)$ for $X, Y : BG \to \mathrm{Set}$ (Definition 5.2.8; a special case of fiberwise maps).

In all those cases, we have notions of identity morphism and composition of morphisms. We have also seen that some maps between types are paired with maps on morphisms, for example, taking underlying symmetries in groups, $\mathrm{U} : \mathrm{Group} \to \mathrm{Set}$ (Definition 4.2.11), comes with a corresponding operation of taking the underlying map of symmetries of a group homomorphism,

$$\mathrm{U} : \mathrm{Hom}(G, H) \to (UG \to UH)$$

(Definition 4.4.5) satisfying $\mathrm{U}(\mathrm{id}_G) = \mathrm{id}_{UG}$ and $\mathrm{U}(\psi \circ \varphi) = \mathrm{U}\psi \circ \mathrm{U}\varphi$ (Corollary 4.4.16).

It's very useful to develop some abstractions for types equipped with a notion of morphism and maps equipped with maps of morphisms like this. These give the notions of (wild) *categories* and *functors*, respectively, and *category theory* is the study of these structures.

Here we give a brief primer[1] on category in order to systematize what we've done so far, and prepare for the main result of the next chapter, Chapter 7, which is to give an *equivalence of categories* between the categories of concrete and abstract groups.

This chapter introduces some useful terminology that we'll use in the rest of the book. It can probably be skipped at a first reading, and only consulted as needed.

[1] The topic is of course too vast to cover in detail here, so we refer to the literature for more details. Category theory in univalent foundations is also treated in Chapter 10 of the HoTT book[2] (based on Ahrens, Kapulkin, and Shulman[3]), while Awodey[4] and Riehl[5] give traditional expositions, and Mac Lane[6] gives a comprehensive treatment.

[3] Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*.

[4] Benedikt Ahrens, Krzysztof Kapulkin, and Michael Shulman. "Univalent categories and the Rezk completion". In: *Math. Structures Comput. Sci.* 25.5 (2015), pp. 1010–1039. ISSN: 0960-1295. DOI: 10.1017/S0960129514000486.

[5] Steve Awodey. *Category theory*. Second. Vol. 52. Oxford Logic Guides. Oxford University Press, Oxford, 2010, pp. xvi+311. ISBN: 978-0-19-923718-0.

[6] Riehl, *Category Theory in Context*.

[7] Saunders Mac Lane. *Categories for the working mathematician*. Second. Vol. 5. Graduate Texts in Mathematics. Springer-Verlag, New York, 1998, pp. xii+314. ISBN: 0-387-98403-8.

6.1 *Brief overview of the chapter*

In Section 6.2 we define the kinds of categories we need, along with many examples (including the above). Then we discuss various abstract notions in categories (terminal and initial objects, monomorphisms and epimorphisms) and remark on the importance of *duality* in Section 6.3. In Section 6.4 we cover functors and natural transformations, and in Section 6.5 we treat adjunctions; we have already seen some examples of adjunctions, for example in Exercises 5.5.11 and 5.5.15.

We have also seen an incarnation of the Yoneda lemma in Exercise 3.5.4; in Section 6.6 we treat the O.G. version. This is leveraged both in Section 7.6, where we cover equivalences of categories and the relationship with univalence, and in Section 6.8, where we treat general limits and colimits.

We end with a brief introduction to monoidal categories in Section 6.9, as we'll see in Chapter 12 that abelian groups form an example.

6.2 *Categories*

sec:categories

As mentioned above, many types come equipped with notion of *morphism* or *arrow* between its elements which is more general than identification or *isomorphism*. For instance we have type of functions $A \to B$ for $A, B : \mathcal{U}$ and the type of pointed functions $A \to_* B$ for $A, B : \mathcal{U}_*$. There are identity morphisms and composition of morphisms, and this motivates the following definition:

def:wild-cat
struc:cat-ob
struc:cat-mor
struc:cat-id
struc:cat-comp
struc:cat-unit-laws
struct:cat-assoc

Definition 6.2.1. A *wild precategory*[8] consists of the following data:

(1) A type Ob, called the *type of objects*.

(2) For each pair of objects $A, B : Ob$, a type of *morphisms* $\hom(A, B)$. These are also known as *arrows*, and written $A \to B$ when there's no danger of confusion. If $f : A \to B$ is such an arrow, then we say that the *domain* of f is A and the *codomain* of f is B.

(3) For each object $A : Ob$, an *identity arrow* $\mathrm{id}_A : A \to A$.

(4) For each pair of arrows $f : A \to B$ and $g : B \to C$, a *composite arrow* $g \circ f : A \to C$.[9]

(5) For each arrow $f : A \to B$, a pair of identifications

$$\lambda : \mathrm{id}_B \circ f \xrightarrow{=} f, \quad \rho : f \circ \mathrm{id}_A \xrightarrow{=} f.$$

(6) For each triple of arrows $f : A \to B$, $g : B \to C$, and $h : C \to D$, an identification

$$\alpha : h \circ (g \circ f) \xrightarrow{=} (h \circ g) \circ f.$$

If $\mathcal{C} \equiv (Ob, \hom, \mathrm{id}, \lambda, \rho, \alpha)$ is a wild precategory, then we write $A, B : \mathcal{C}$ instead of $A, B : Ob$ to indicate that A, B are elements of the underlying type of objects of $\mathcal{C}$. We also write $\mathrm{Ob}(\mathcal{C})$ for this type. We may write $f, g : A \to_{\mathcal{C}} B$ to emphasize where the arrows f and g live, if needed, and sometimes $\hom_{\mathcal{C}}(A, B)$ or $\mathcal{C}(A, B)$, instead of $\hom(A, B)$. ⌟

rem:wild

Remark 6.2.2 (On the adjective "wild"). With this definition, we readily equip the universes of types $\mathcal{U}$ and of pointed types $\mathcal{U}_*$ with a structure

[8] See below for remarks on the terminology. Adding further properties to the data given here eventually recovers the notion of a category *simpliciter*, see Definition 6.2.6.

[9] To be fully explicit, the composition operation has type

$$\prod_{A,B,C : Ob} (B \to C) \to (A \to B) \to (A \to C),$$

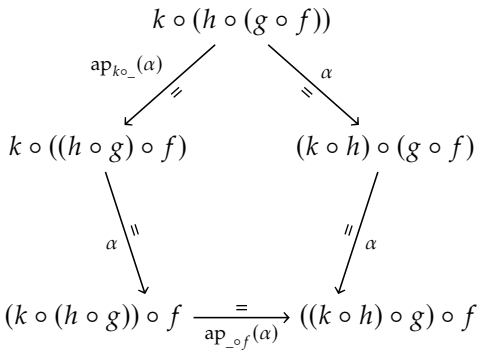
and we might denote it $g \circ_{A,B,C} f$. Since the objects A, B, and C can often be inferred, we leave them out, lest the notation becomes too heavy. A similar remark goes for the other operations.

of wild precategories. In the former case, we can use reflexivities for λ, ρ, and α. In the latter, we leave their definition as an exercise.

We use the adjective "wild" to highlight a deficiency of this definition as it stands: We haven't specified any further laws for the identifications λ, ρ, and α. For example, it would be sensible to require an identification of λ and ρ at an identity: $\mathrm{id}_A \circ \mathrm{id}_A \xrightarrow{=} \mathrm{id}_A$, as well as a filler for the pentagonal diagram of α's coming from four composable arrows:

eq:pentagon

(6.2.1)



Of course, we would on occasion then need a filler for a three dimensional diagram of pentagons for five composable arrows, etc., etc., *ad infinitum*.[10] The corresponding structure is known to connoisseurs as an $(\infty, 1)$-precategory. It is an open problem as we write this whether this notion can be defined in our type theory. Types and pointed types should furnish examples. ⌟

However, when the types of morphisms $\hom(A, B)$ are *sets*, then the types of λ, ρ, and α are propositions, so any coherence conditions are automatically fulfilled. This motivates the following definition.

def:precategory

Definition 6.2.3. A *precategory* is a wild precategory $\mathcal{C}$ in which the types $A \to_{\mathcal{C}} B$ are sets, for all objects $A, B : \mathcal{C}$. ⌟

Most (wild) precategories we shall meet satisfy a further condition that makes them better behaved than arbitrary precategories: a *univalence* condition. In fact, for the wild precategory of types and functions, this condition is exactly the Univalence Axiom (**??** 2.13.2)!

In order to define this, we need the notion corresponding to equivalence in an general wild precategory.

def:iso-in-cat

Definition 6.2.4. A morphism $f : A \to B$ in a wild precategory $\mathcal{C}$ is an *isomorphism* if have $g, h : B \to A$ and identifications $\sigma : \mathrm{id}_B \xrightarrow{=} f \circ g$ and $\rho : \mathrm{id}_A \xrightarrow{=} h \circ f$. This condition is encoded by the type

$$\mathrm{isIso}(f) :\equiv \left(\sum_{g : B \to A} f \circ g \xrightarrow{=} \mathrm{id}_B \right) \times \left(\sum_{h : B \to A} h \circ f \xrightarrow{=} \mathrm{id}_A \right)$$

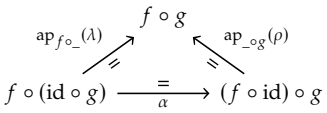
If f is an isomorphism, we also say that f is *invertible*.

We define the type $A \xrightarrow{\cong} B$ of *isomorphisms* from A to B in $\mathcal{C}$ ($A \xrightarrow{\cong}_{\mathcal{C}} B$ if needed) by the following definition.

$$(A \xrightarrow{\cong} B) :\equiv \sum_{f : A \to B} \mathrm{isIso}(f). \qquad ⌟$$

The type $\mathrm{isIso}(f)$ is indeed a proposition,[11] and every identity arrow is an isomorphism. We write f^{-1} for the inverse of an isomorphism f.

[10] There is a hierarchy of notions, A_n-precategories, for $n \geq 2$, with coherence conditions involving up to n composable arrows. The wild precategories lie between A_2 and A_3 in this hierarchy. Besides the mentioned identification of λ and ρ for two identities, we'd also require fillers for diagrams like



as well as coherences when one or both of f and g are identities.

[11] This follows just as for functions: If f is an isomorphism, then each factor in the product is contractible, as, e.g., $\sum_{g : B \to A} f \circ g \xrightarrow{=} \mathrm{id}_B$ is the fiber of $f \circ _ : (B \to A) \to (B \to B)$ at id_B, and all functions $f \circ _$ and $_ \circ f$ are equivalences of types using the data that makes f an isomorphism along with λ, ρ, and α.

def:univalent-cat

Definition 6.2.5. A wild precategory $\mathcal{C}$ is *univalent* if for all objects $A, B : \mathrm{Ob}(\mathcal{C})$, the function

$$\mathrm{idtoiso}_{A,B} : (A \xrightarrow{=}_{\mathrm{Ob}(\mathcal{C})} B) \to (A \xrightarrow{\cong}_{\mathcal{C}} B)$$

defined by path induction sending refl_A to id_A, is an equivalence. ⌟

def:category

Definition 6.2.6. A *wild category* is a univalent wild precategory, and a *category* is a univalent precategory. ⌟

Now we are ready to restate the examples mentioned in the introduction to the chapter. In each case we leave it to the reader to supply most of the data.

- The *wild category of types* (in universe $\mathcal{U}$) has $\mathcal{U}$ as its type of objects, and the function type $A \to B$ as its type of arrows for $A, B : \mathcal{U}$. It is univalent by the Univalence Axiom.
- The *wild category of pointed types* (in a universe $\mathcal{U}$) has $\mathcal{U}_*$ as its type of objects, and the type of pointed maps $A \to_* B$ as its type of arrows.
- The *wild category of families* over a given type X has $X \to \mathcal{U}$ as its type of objects, and the type of fiberwise maps $(X \to_B Y) :\equiv \prod_{b:B} X(b) \to Y(b)$ as its type of arrows.
- The *category of sets* has Set as its type of objects, and the function type $A \to B$ as its type of arrows. It's a category since each type $A \to B$ is a set, and it's univalent.
- The *category of groups* has Group as its type of objects, and the homomorphism type $(G \to H) :\equiv \mathrm{Hom}(G, H)$ as its type of arrows.
- The *category of G-sets*, for a group G, has G-Set as its type of objects, and the type of morphisms of G-sets $\mathrm{Hom}_G(X, Y)$ (Definition 5.2.8) as its type of arrows.

Note that the category of sets sits inside the wild category of types. This situation is quite typical is a rich source of further examples, captured by the following definition.

def:full-subcat

Definition 6.2.7. If $\mathcal{C}$ is a wild precategory, and $P : \mathrm{Ob}(\mathcal{C}) \to \mathrm{Prop}$ is a predicate on the objects, then we get a new wild precategory $\mathcal{C}_P$ with $\mathrm{Ob}(\mathcal{C}_P) :\equiv \mathrm{Ob}(\mathcal{C})_P$, using the notation of Definition 2.20.3, inheriting all the rest of the structure of $\mathcal{C}$. It is non-wild/univalent, if $\mathcal{C}$ is, and is called the *full* (*wide*) *sub*(*pre*)*category* of $\mathcal{C}$ on the predicate P. ⌟

By the univalence condition, in a category each identity type $A \xrightarrow{=} B$ is equivalent to the set $A \xrightarrow{\cong} B$, and is hence a set. Thus we get the following.

lem:obj-gpd

Lemma 6.2.8. *The type of objects of a category form a groupoid.*

And important special case is when each arrow type $A \to B$ is a *proposition*.

ex:poset

Example 6.2.9. A *preorder* is precategory in which every arrow type is a proposition. In this case, the types of λ, ρ, and α are contractible, so the data of a preorder reduces to just a type P and a binary relation, typically written $\leq : P \to P \to \mathrm{Prop}$, that is reflexive, $x \leq x$ (via the identities) and transitive, i.e., if $x \leq y$ and $y \leq z$ implies $x \leq z$ (via composition).

fig:cat-summary

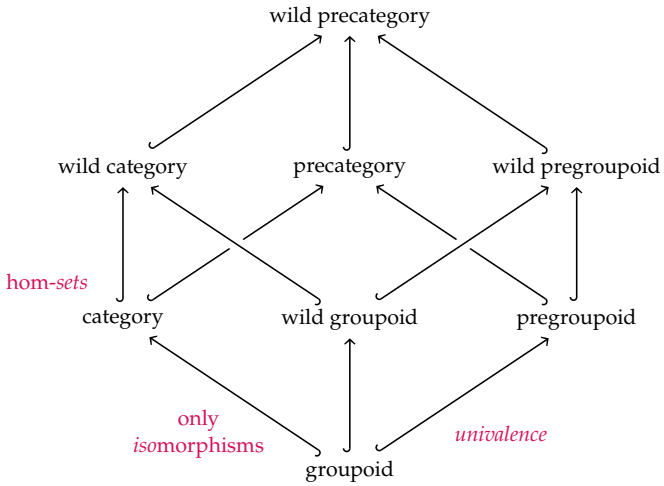


FIGURE 6.1: The various notions of categories arranged in a cube: on the bottom face we loose the adjective "wild" by requiring arrows to form *sets*; on the front left face we loose the prefix "pre" by requiring *univalence*; and on the front right face we restrict to groupoids by requiring all arrows be *invertible*.

A *partial order*, also known as a *poset*, is a univalent preorder. In this case, the type of objects is a set. This happens if and only if the relation is symmetric, i.e., if $x \leq y$ and $y \leq x$ implies $x = y$.

Typical examples are $(\mathbb{N}, \leq)$, $(\mathbb{Z}, \leq)$, (Prop, $\to$), and $(\mathrm{Sub}(S), \subseteq)$ for a set S. A preorder that fails to be a poset is the two-element type $\mathbb{2}$ with the always true relation. This is hence also an example of a precategory that fails to be univalent. ⌟

xca:order-poset

EXERCISE 6.2.10. Show that the type of orders, Order, with the divisibility relation from Definition 3.6.20 forms a poset. ⌟

Another important special case is when every morphism is an isomorphism.

def:wild-pre-groupoid

DEFINITION 6.2.11. A (*wild*) *pregroupoid* is a (wild) precategory in which every arrow is invertible. A (*wild*) *groupoid* is a univalent (wild) pregroupoid. ⌟

ex:path-groupoid

EXAMPLE 6.2.12. Every type X gives rise to a wild groupoid, its (wild) *path groupoid*, having X as its type of objects and $x \xrightarrow{=} y$ as its type of arrows. The arrows are invertible, since paths are always invertible.

If X is a 1-type, then this structure is a groupoid. ⌟

There's no conflict with the terminology introduced in Section 2.15, because this construction gives an equivalence from the type of 1-types (in $\mathcal{U}$) to the type of groupoids (in $\mathcal{U}$), as we shall see below in Theorem 6.7.15.

Definitions 6.2.1, 6.2.3, 6.2.6 and 6.2.11 are summarized as a diagram of inclusions in Figure 6.1.

rem:cat-sizes

REMARK 6.2.13. This is a good moment to remark on the size issues we have so far swept under the rug. The definition of a wild precategory $\mathcal{C}$ can be parametrized by *two* universe levels: The type of objects belong to one, $Ob : \mathcal{U}'$, while the family of arrows belong to another, $\mathrm{hom} : Ob \to Ob \to \mathcal{U}$.

If they coincide, then we call $\mathcal{C}$ a *$\mathcal{U}$-small category*. For instance, a path groupoid for a type $X : \mathcal{U}$ is $\mathcal{U}$-small.

The other common case is where $\mathcal{U} : \mathcal{U}'$, in which case we call $\mathcal{C}$ *locally $\mathcal{U}$-small*. For example, the (wild) categories of types, sets, pointed types, groups, etc., built from types in $\mathcal{U}$ are all locally $\mathcal{U}$-small. This

generalizes Definition 2.19.1 for the case of path groupoids. ⌟

Many notions in category theory work already at the level of wild categories,[12] but a notable exception is the construction of slice categories, (also known as *over categories*).

def:slice-cat

Example 6.2.14. The *slice precategory* of a precategory $\mathcal{C}$ over an object $C : \mathcal{C}$, denoted $\mathcal{C}/C$, has as objects the type $\sum_{A:\mathcal{C}} A \to C$ of pairs (A, f) of an object A and an arrow $f : A \to C$ with codomain C. An arrow from (A, f) to (A', f') is an arrow $g : A \to A'$ such that $f' \circ g = f$.[15] The identities and compositions are inherited from $\mathcal{C}$. If $\mathcal{C}$ is univalent (and hence a category), then so is $\mathcal{C}/C$.

If we try to define the slice $\mathcal{C}/C$ for an arbitrary wild precategory $\mathcal{C}$, using identifications $f' \circ g \xrightarrow{=} f$, we find that we need the pentagon coherence for α for $\mathcal{C}$ in order to define the α for $\mathcal{C}/C$.

Of course, for *particular* wild categories, it may very well happen that $\mathcal{C}/C$ is again a wild categories.[16] ⌟

xca:univ-slice-cat

Exercise 6.2.15. Construct a wild precategory structure on the *slice of the universe* $\mathcal{U}/B$ over a fixed type $B : \mathcal{U}$. ⌟

6.3 *Abstract notions and duality*

sec:duality

Many concepts that we introduced in Chapter 2 for the wild category of types make sense in arbitrary wild precategories.

def:terminal-obj

Definition 6.3.1. A *terminal object* in a wild precategory $\mathcal{C}$ is an object 1 such that for any object $A : \mathcal{C}$, the arrow type $A \to 1$ is contractible. ⌟

xca:terminal-prop

Exercise 6.3.2. Show that if $\mathcal{C}$ is univalent, then the type of terminal objects is a proposition. ⌟

The unit type $\mathbb{1}$ is a terminal object in the wild category of types, and in the category of sets, while the trivial group $\mathbb{1}$ is a terminal object in the category groups.

The is a "dual" notion as well.

def:initial-obj

Definition 6.3.3. An *initial object* in a wild precategory $\mathcal{C}$ is an object 0 such that for any object $A : \mathcal{C}$, the arrow type $0 \to A$ is contractible. ⌟

For example, the empty type $\mathbb{0}$ is initial in the wild category of types, while the trivial group $\mathbb{1}$ is initial in the category of groups.

xca:unit-prd-types

Exercise 6.3.4. Show that the unit type $\mathbb{1}$, pointed at its unique element, is both terminal and initial in the wild category of pointed types. ⌟

The relationship between terminal and initial objects reflects a deep aspect of category theory: Every concept comes with a dual version obtained by "reversing all the arrows". More formally, can introduce for every wild precategory its opposite category that has its arrows reversed.

def:op-cat

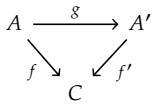
Definition 6.3.5. For any wild precategory $\mathcal{C} \equiv (Ob, \hom, \mathrm{id}, \lambda, \rho, \alpha)$, define the *opposite* $\mathcal{C}^{\mathrm{op}}$ to have the same type of objects, morphisms $\hom_{\mathcal{C}^{\mathrm{op}}}(A, B) :\equiv \hom_{\mathcal{C}}(B, A)$, identities the same, and composition reversed: If $f : A \to_{\mathcal{C}} B$ and $g : B \to_{\mathcal{C}} C$, then $g \circ f : A \to_{\mathcal{C}} C$ works as the composite $f \circ_{\mathcal{C}^{\mathrm{op}}} g$ of $g : C \to_{\mathcal{C}^{\mathrm{op}}} B$ and $f : B \to_{\mathcal{C}^{\mathrm{op}}} A$. We then swap the roles of λ and ρ, and α^{-1} plays the role of α in $\mathcal{C}^{\mathrm{op}}$. ⌟

[12] They *all* work at the level of $(\infty, 1)$-categories. We refer to Lurie[13] and Land[14] for details.

[13] Jacob Lurie. *Higher topos theory*. Vol. 170. Annals of Mathematics Studies. Princeton, NJ: Princeton University Press, 2009, pp. xviii+925. ISBN: 9781400830558. DOI: 10.1515/9781400830558.

[14] Markus Land. *Introduction to Infinity-Categories*. Compact Textbooks in Mathematics. Birkhäuser/Springer, Cham, 2021, pp. ix+296. ISBN: 978-3-030-61524-6. DOI: 10.1007/978-3-030-61524-6.

[15] This is a proposition since $\mathcal{C}$ is a precategory. We illustrate the arrow as a commuting triangle:



[16] This will be the case when $\mathcal{C}$ should be an $(\infty, 1)$-category, carrying the whole hierarchy of coherences, which then carry over to $\mathcal{C}/C$.

lem:op-idem

Lemma 6.3.6. *The operation of taking opposites defines an equivalence from the type of wild precategories to itself, with a trivially defined identification* $(\mathcal{C}^{\mathrm{op}})^{\mathrm{op}} \xrightarrow{=} \mathcal{C}$.

It follows that any construction or theorem about wild precategories has a dual version, obtained by precomposition with $(_)^{\mathrm{op}}$.

For example, the dual of the slice category construction is the *coslice* category $C/\mathcal{C}$ (also known as the *under category*).

As a further example of a pair dual notions, we consider that of monomorphisms and epimorphisms.

def:mono-in-cat

Definition 6.3.7. An arrow $f : A \to B$ in a wild precategory $\mathcal{C}$ is called a *monomorphism*[17] if post-composition with f is an injection

$$f \circ _ : (C \to_{\mathcal{C}} A) \to (C \to_{\mathcal{C}} B)$$

for all objects $C : \mathcal{C}$. ⌟

[17] For short: a mono

def:epi-in-cat

Definition 6.3.8. Dually, f is called an *epimorphism*[18] if pre-composition with f is an injection

$$_ \circ f : (B \to_{\mathcal{C}} C) \to (A \to_{\mathcal{C}} C)$$

for all objects $C : \mathcal{C}$.[19] ⌟

[18] For short: an epi

If C is a precategory, then these conditions reduce to the implications

$$f \circ g = f \circ h \to g = h, \quad \text{and} \quad g \circ f = h \circ f \to g = h,$$

respectively.

We already met the monomorphisms in the category of groups in Definition 5.3.13 using a different definition.

xca:mono-cat-grp

Exercise 6.3.9. Show that the monomorphisms in the category of groups are the same as those of Definition 5.3.13. ⌟

xca:monos-epis-sets-types

Exercise 6.3.10. Show that the monomorphisms in the wild category of types are just the injections, and the epimorphisms in the category of sets are just the surjections. ⌟

The epimorphisms in the wild category of types are always surjections, but are much more restricted. See Buchholtz, de Jong, and Rijke[20] for details.

xca:monos-epis-preorder

Exercise 6.3.11. Show that every morphism in a preorder is both a mono and an epi. ⌟

[19] We can illustrate these in diagrams as saying that f is a mono if a map into the codomain of f factors in at most one way through the domain of f,



and dually, f is an epi if a map out of the domain of f factors in at most one way through the codomain of f:



[20] Ulrik Buchholtz, Tom de Jong, and Egbert Rijke. "On epimorphisms and acyclic types in univalent mathematics". In: *The Journal of Symbolic Logic* (2025), pp. 1–36. DOI: 10.1017/jsl.2024.76. arXiv: 2401.14106.

6.4 *Functors and natural transformations*

sec:naturality

Not only do have arrows *in* (wild pre-)categories, there's also a notion of arrow *between* them. These are called functors.

def:functor

Definition 6.4.1. A *wild functor* $F : \mathcal{C} \to \mathcal{D}$ between wild precategories $\mathcal{C}$ and $\mathcal{D}$ consists of a function $F : \mathrm{Ob}(\mathcal{C}) \to \mathrm{Ob}(\mathcal{D})$, mapping objects to objects, and a family of functions[21]

$$F : \prod_{A,B:\mathcal{C}} (A \to_{\mathcal{C}} B) \to (F(A) \to_{\mathcal{D}} F(B))$$

together with identifications

$$F_{\mathrm{id}} : F(\mathrm{id}_A) \xrightarrow{=} \mathrm{id}_A, \quad \text{and} \quad F_{\circ} : F(g \circ f) \xrightarrow{=} F(g) \circ F(f),$$

[21] In practice, the functions on objects and arrows are named the same as the functor, but they could be disambiguated with subscripts, say, F_0 and F_1, if needed.

for all objects A and composable arrows f and g in $\mathcal{C}$.

If $\mathcal{D}$ is a precategory, then the types of F_{id} and $F_\circ$ are propositions, and in this case we just call F a *functor*. ⌟

xca:wild-functor-isos

Exercise 6.4.2. Show that every wild functor maps isomorphisms to isomorphisms. ⌟

ex:functor-preorders

Example 6.4.3. A functor between preorders $(P, \leq)$ and $(Q, \leq)$ amounts to a monotone map $F : P \to Q$, i.e., $p \leq p'$ implies $F(p) \leq F(p')$ for $p, p' : P$. ⌟

ex:USym-functor

Example 6.4.4. Taking the underlying set of symmetries U gives a functor $\mathrm{U} : \mathrm{Group} \to \mathrm{Set}$. It's easy to check that $\mathrm{U}(\mathrm{id}_G) = \mathrm{id}_{UG}$, and we verified the preservation of composition in Corollary 4.4.16. ⌟

In Chapter 7 we shall define the category of abstract groups, $\mathrm{Group}^{\mathrm{abs}}$, in such a way that the map $\mathrm{abs} : \mathrm{Group} \to \mathrm{Group}^{\mathrm{abs}}$ from Definition 4.3.4 becomes a functor in line with the discussion from Remark 4.4.20.

ex:action-functors

Example 6.4.5. Given a group homomorphism $f : G \to H$, we have three functors

$$G\text{-Set} \underset{f_*}{\overset{f_!}{\leftarrow}} f^* - H\text{-Set}$$

with actions on objects described in Definition 5.5.9 and Remark 5.5.13. The action on arrows of restriction is again given by restriction: If $g : X \to Y$ is a map of H-sets with $X, Y : BH \to \mathrm{Set}$, then $f^*(g) : f^*X \to f^*Y$ maps $z : BG$ to $g_{Bf(z)} : X(Bf(z)) \to Y(Bf(z))$.

The action on arrows of induction along f takes a map of G-sets $g : X \to Y$, for $X, Y : BG \to \mathrm{Set}$, to the functorial action of set truncation, for $w : BH$: ⌟

def:n-trunc-functor

Example 6.4.6. Taking n-truncation gives a wild functor $\|_\|_n : \mathcal{U} \to \mathcal{U}^{\leq n}$, with action on arrows given by the universal property.[22] For $n = 0$, this is a functor from $\mathcal{U}$ to $\mathrm{Set}_{\mathcal{U}}$. ⌟

[22] Here, $\mathcal{U}^{\leq n}$ is the full subcategory (Definition 6.2.7) of $\mathcal{U}$ on the n-truncated types.

ex:add-remove-basepoint

Example 6.4.7. We can extend the operation of adding a default element from Definition 2.21.1 to a wild functor $(_)_+ : \mathcal{U} \to \mathcal{U}_*$. It takes a function $f : A \to B$ to the function $f_+ : A_+ \to B_+$ with

$$f_+(\mathrm{inl}_a) :\equiv \mathrm{inl}_{f(a)}, \quad \text{and} \quad f_+(\mathrm{pt}_{A_+}) :\equiv \mathrm{pt}_{B_+}.$$

The operation of taking underlying types of pointed types likewise extends to a wild functor $(_)_\div : \mathcal{U}_* \to \mathcal{U}$. ⌟

ex:loop-functor

Example 6.4.8. Taking loop types extends to a wild functor $\Omega : \mathcal{U}_* \to \mathcal{U}_*$. We defined the action on maps in Definition 4.4.3, except we didn't equip Ωk with a pointing path, for $k : X \to_* Y$. However, that's easily remedied using the path groupoids laws from back in Exercise 2.5.3:

$$\mathrm{pt}_{\Omega Y} \equiv \mathrm{refl}_{\mathrm{pt}_Y} \xrightarrow{=} k_{\mathrm{pt}}^{-1} \cdot \mathrm{refl}_{k(\mathrm{pt}_X)} \cdot k_{\mathrm{pt}} \equiv k_{\mathrm{pt}}^{-1} \cdot \mathrm{ap}_{k_\div}(\mathrm{pt}_{\Omega X}) \cdot k_{\mathrm{pt}} \equiv \Omega k(\mathrm{pt}_{\Omega X})$$

We leave it to the reader to fill in the remaining data. ⌟

ex:slice-projection

Example 6.4.9. For an object C of a precategory $\mathcal{C}$, recall the slice precategory $\mathcal{C}/C$ of Example 6.2.14. Taking the domain of an object $(A, f : A \to C)$ of the slice extends to a functor $\mathrm{fst} : \mathcal{C}/C \to \mathcal{C}$. ⌟

ex:arrow-cat

Example 6.4.10. Similar to the slice and coslice constructions, we have the formation of the *arrow precategory* $\mathcal{C}^{\to}$ of a precategory $\mathcal{C}$. It has as

objects triples (A, B, f) of two objects $A, B : \mathcal{C}$ and an arrow $f : A \to B$. The arrows from (A, B, f) to (A', B', f') are pairs of arrows $g : A \to A'$ and $h : B \to B'$ making a commutative square:

$$\begin{array}{ccc} A & \xrightarrow{g} & A' \\ {\scriptstyle f}\downarrow & & \downarrow{\scriptstyle f'} \\ B & \xrightarrow[h]{} & B' \end{array}$$

Projecting out the domain and the codomain gives two functors

$$\mathcal{C} \xleftarrow{\text{dom}} \mathcal{C}^{\to} \xrightarrow{\text{cod}} \mathcal{C}$$

Note that $\mathcal{C}^{\to}$ is a category if $\mathcal{C}$ is. ⌟

xca:wildpreCat-of-wildpreCats

Exercise 6.4.11. Define identity wild functors and composition of wild functors, along with identifications $\lambda : \mathrm{id}_{\mathcal{D}} \circ F \xrightarrow{=} F$, $\rho : F \circ \mathrm{id}_{\mathcal{C}} \xrightarrow{=} F$, and $\alpha : H \circ (G \circ F) \xrightarrow{=} (H \circ G) \circ F$. This defines a wild precategory of (small) wild precategories. If you're feeling adventurous, define the fillers for the pentagon diagram (6.2.1) for four composable wild functors

$$\mathcal{C}_0 \xrightarrow{F} \mathcal{C}_1 \xrightarrow{G} \mathcal{C}_2 \xrightarrow{H} \mathcal{C}_3 \xrightarrow{K} \mathcal{C}_4.$$

How much easier is this if C_4 is non-wild? ⌟

xca:wildCat-of-precats

Exercise 6.4.12. Show that the wild full subprecategory PreCat on the precategories of the wild precategory of small wild precategories is univalent. Conclude that the further wild full subcategory Cat on the categories is univalent as well.[23] ⌟

def:full-faithful

Definition 6.4.13. A functor $F : \mathcal{C} \to \mathcal{D}$ is *faithful* if all the maps on morphisms, $F : \hom_{\mathcal{C}}(A, B) \to \hom_{\mathcal{D}}(F(A), F(B))$, are injective, and *full* if they are surjective. We say that F is *fully faithful* if they are both (and hence equivalences). ⌟

Example 6.4.14. core inclusion is faithful, projections can be full, not faithful, full subcategory inclusions are fully faithful. ⌟

There's also a notion of arrow *between functors*. These are called natural transformations.[24]

def:nat-trans

Definition 6.4.15. A *wild natural transformation* $\alpha : F \to G$ between two wild functors $F, G : \mathcal{C} \to \mathcal{D}$ between wild precategories $\mathcal{C}$ and $\mathcal{D}$ consists of a family of arrows

$$\alpha_A : F(A) \to_{\mathcal{D}} G(A)$$

indexed by objects $A : \mathcal{C}$, and a family of fillers for the squares

$$\begin{array}{ccc} F(A) & \xrightarrow{\alpha_A} & G(A) \\ {\scriptstyle F(f)}\downarrow & & \downarrow{\scriptstyle G(f)} \\ F(B) & \xrightarrow{\alpha_B} & G(B) \end{array}$$

in $\mathcal{D}$ for each arrow $f : A \to_{\mathcal{C}} B$.[26]

If $\mathcal{D}$ is a precategory, then the types of the naturality square fillers are propositions. In this case, we just call α a *natural transformation*. ⌟

ex:pt-unpt-unit

Example 6.4.16. There is a wild natural transformation $\eta : \mathrm{id}_{\mathcal{U}} \to ((_)_+)_\div$ from the identity wild functor on the universe $\mathcal{U}$ to the composition

$$\mathcal{U} \xrightarrow{(_)_+} \mathcal{U}_* \xrightarrow{(_)_\div} \mathcal{U}$$

[23] There is in fact also a wild category of all wild precategories, but its notion of arrow is more refined than the wild functors of Definition 6.4.1: We would need to include components corresponding to the unit and associativity laws as well.

[24] Freyd[25] observed that "categories are what one must define in order to define functors, and that functors are what one must define in order to define natural transformations." In this sense, natural transformations are at the heart of category theory.

[25] Peter Freyd. *Abelian categories. An introduction to the theory of functors*. Harper's Series in Modern Mathematics. Harper & Row, Publishers, New York, 1964, pp. xi+164.

[26] These squares are called the "naturality squares" for α.

of the wild functors from Example 6.4.7. Its action on objects is $\mathsf{inl} : A \to (A_+)_\div$, where $(A_+)_\div \equiv A \amalg \mathbb{1}$. The naturality squares commute by reflexivity. ⌟

ex:path-gpd-nat

Example 6.4.17. Every function $f : A \to B$ between types in $\mathcal{U}$ becomes a wild functor between the corresponding wild path groupoids using the action on paths, ap_f from Definition 2.6.1.

Likewise, given two functions $f, g : A \to B$, we get for every family of identifications $h : \prod_{x:A} f(x) \xrightarrow{=} g(x)$ a wild natural transformation between the corresponding wild functors, using the naturality squares of Definition 2.6.5. ⌟

With natural transformations as arrows we can elevate the type of functors to a precategory.

def:functor-cat

Definition 6.4.18. For a wild precategory $\mathcal{C}$ and a precategory $\mathcal{D}$ we have the *functor precategory* $\mathcal{C} \to \mathcal{D}$, also written $[\mathcal{C}, \mathcal{D}]$ or $\mathcal{D}^{\mathcal{C}}$, with functors from $\mathcal{C}$ to $\mathcal{D}$ as objects and natural transformations as arrows. The identity arrow at F is the identity natural transformation $\mathrm{id}_F : F \to F$ that assigns to each object $A : \mathcal{C}$ the identity arrow $\mathrm{id}_{F(A)}$. The composition likewise forms compositions objectwise. ⌟

xca:fibrewise-nat-trans

Exercise 6.4.19. Show that a natural transformation $\alpha : F \to G$ in a functor precategory $\mathcal{C} \to \mathcal{D}$ is invertible if and only if each component $\alpha_A : F(A) \to_{\mathcal{D}} G(A)$ is. ⌟

xca:functor-cat-univalent

Exercise 6.4.20. Show that the functor precategory $\mathcal{C} \to \mathcal{D}$ is univalent if $\mathcal{D}$ is.[27] In this case we call it the *functor category*. ⌟

[27] This is Thm. 9.2.5 in the HoTT book[28].

xca:path-core-adj

Exercise 6.4.21. Let A be a type and $\mathcal{C}$ category. Show that restricting to the action on objects induces an equivalence

$$[A, \mathcal{C}] \to (A \to \mathrm{Ob}(\mathcal{C}))$$

from the functor category whose domain is the wild path category of A to the type of functions from A to the objects of $\mathcal{C}$. ⌟

[28] Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*.

A functor $F : \mathcal{C}^{\mathrm{op}} \to \mathcal{D}$ whose domain is an opposite category is called *contravariant*, because it reverses the directions of arrows: $f : C \to_{\mathcal{C}} C'$ in $\mathcal{C}$ maps to $F(f) : F(C') \to_{\mathcal{D}} F(C)$ in $\mathcal{D}$. If needed for emphasis, we may say that a functor $F : \mathcal{C} \to \mathcal{D}$ is *covariant* by contrast.

ex:repr-functors

Example 6.4.22. For an object C of a *locally $\mathcal{U}$-small* wild precategory $\mathcal{C}$, we can form the co- and contravariant wild functors

$$k_C \coloneqq \mathrm{Hom}_{\mathcal{C}}(C, _) : \mathcal{C} \to \mathcal{U},$$
$$h_C \coloneqq \mathrm{Hom}_{\mathcal{C}}(_, C) : \mathcal{C}^{\mathrm{op}} \to \mathcal{U},$$

whose actions on morphisms are by post- and precomposition, respectively. These are called *representable* functors. ⌟

6.5 *Adjunctions*

sec:adjunctions

We have already seen two examples of an adjunctions in Exercises 5.5.11 and 5.5.15. Given a group homomorphism $f : G \to H$, there are families of bijections

$$\alpha : \mathrm{Hom}_H(f_! X, Y) \xrightarrow{\cong} \mathrm{Hom}_G(X, f^* Y),$$
$$\beta : \mathrm{Hom}_G(f^* Y, X) \xrightarrow{\cong} \mathrm{Hom}_H(Y, f_* X),$$

for G-sets X and H-sets Y that are furthermore *natural*. This just means that if we fix either X or Y, then we get natural transformations of the corresponding functors. For example, fixing X, we can regard α as a natural transformation

$$\alpha : \mathrm{Hom}_H(f_! X, _) \to \mathrm{Hom}_G(X, f^* _)$$

from the representable functor for H-Set at $f_! X$ to the composition of f^* and the representable functor for G-Set at X.

def:adjunction

Definition 6.5.1. A (*wild*) *adjunction* between two (wild) precategories $\mathcal{C}$ and $\mathcal{D}$ consists of:

- a (wild) functor $F : \mathcal{C} \to \mathcal{D}$ (the *left adjoint*),
- a (wild) functor $G : \mathcal{D} \to \mathcal{C}$ (the *right adjoint*),
- a (wild) natural isomorphism $\alpha : \mathrm{Hom}_{\mathcal{D}}(F_, _) \xrightarrow{\simeq} \mathrm{Hom}_{\mathcal{C}}(_, G_)$.

We write $F \dashv G$ to denote this situation. ⌟

The essence of an adjunction is thus the ability to transpose between arrows $F(C) \to_{\mathcal{D}} D$ and $C \to_{\mathcal{C}} G(D)$. There is however another way of packaging this information. We can transpose the identity $\mathrm{id}_{F(C)}$ to get an arrow $\eta_C : C \to_{\mathcal{C}} GF(C)$, and the identity $\mathrm{id}_{G(D)}$ to get an arrow $\varepsilon_D : FG(D) \to_{\mathcal{D}} D$. Naturality of α makes these into natural transformations

eq:adj-unit-counit

$$\eta : \mathrm{id}_{\mathcal{C}} \to GF, \quad \text{and} \quad \varepsilon : FG \to \mathrm{id}_{\mathcal{D}} \tag{6.5.1}$$

called the *unit* and *counit* of the adjunction.

xca:adj-triangles

Exercise 6.5.2. Use naturality of α, along with unit laws to fill the triangle laws in Figure 6.2. ⌟

Conversely, given F, G, η, and ε, along with fillers for the triangle laws, we can recover α by sending $f : F(C) \to_{\mathcal{D}} D$ to the composite

$$C \xrightarrow{\eta_C} GF(C) \xrightarrow{G(f)} G(D).$$

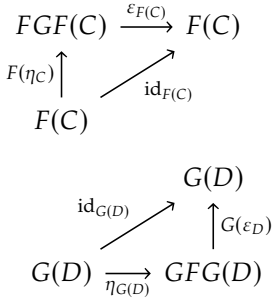


Figure 6.2: Triangle laws for an adjunction $F \dashv G$.

fig:adj-triangles

Exercise 6.5.3. Use the functor and triangle laws to check that α thus defined is a natural isomorphism.[29] ⌟

xca:adj-from-triangles

Example 6.5.4. We have a wild adjunction $\|_\|_n \dashv \iota_n$ with $\|_\|_n : \mathcal{U} \to \mathcal{U}^{\leq n}$ (cf. Example 6.4.6) as the left adjoint and the inclusion $\iota_n : \mathcal{U}^{\leq n} \to \mathcal{U}$ as the right adjoint. Indeed, the constructor $|_|_n$ acts as the unit, precomposition with which gives the equivalence

ex:trunc-adj

$$(\|X\|_n \to Y) \xrightarrow{\simeq} (X \to Y)$$

for $X : \mathcal{U}$ and $Y : \mathcal{U}^{\leq n}$. Naturality follows from the universal property. ⌟

ex:pt-unpt-adj

Example 6.5.5. Also the add/forget base points functors from Example 6.4.7 can be arranged into a wild adjunction $(_)_+ \dashv (_)_\div$. For $A : \mathcal{U}_*$ and $X : \mathcal{U}$ we have

$$\alpha : (A_+ \to_* X) \xrightarrow{\simeq} (A \to X_\div)$$

given by precomposition with $\mathrm{inl} : A \to A_+$. ⌟

[29] We can thus go back and forth between the different bundles of data for a wild adjunction. We shall see later in Corollary 6.6.8 for a functor whose domain is a category, that the data of a right adjoint is a proposition.

ex:Sigma-Pi-adj

Example 6.5.6. Generalizing Exercises 5.5.11 and 5.5.15 slightly, we have for any function $f : A \to B$, a string of adjunctions $f_! \dashv f^* \dashv f_*$,

$$\mathrm{Set}^A \underset{f_*}{\overset{f_!}{\underset{\leftarrow f^* -}{\overset{\bot}{\bot}}}} \mathrm{Set}^B,$$

where $f^*(Y) :\equiv Y \circ f$, and we set

$$f_!(X)(b) :\equiv \left\| \sum_{a:A} (f(a) \xrightarrow{=} b) \times X(a) \right\|_0,$$
$$f_*(X)(b) :\equiv \prod_{a:A} (f(a) \xrightarrow{=} b) \to X(a).$$

Generalizing a bit further, we have a similar string of wild adjunctions

$$\mathcal{U}^A \underset{f_*}{\overset{f_!}{\underset{\leftarrow f^* -}{\overset{\bot}{\bot}}}} \mathcal{U}^B,$$

where we leave out the set truncation in the definition of $f_!$.

For the special case of the terminal projection $! : A \to \mathbb{1}$, these simplify to

$$\mathcal{U}^A \underset{\Pi_A}{\overset{\Sigma_A}{\underset{\longleftarrow \mathrm{cst} \longrightarrow}{\overset{\bot}{\bot}}}} \mathcal{U},$$

where $\Sigma_A(X) :\equiv \sum_{a:A} X(a)$ and $\Pi_A(X) :\equiv \prod_{a:A} X(a)$. ⌟

rem:adj-in-posets

Remark 6.5.7. For a monotone map $F : P \to Q$ of preorders, we have that $G : Q \to P$ is a right adjoint if and only if we have the logical biimplication

$$F(p) \leq q \leftrightarrow p \leq G(q).$$

For example, the inclusion of the integers into the rationals with their usual order, $\iota : \mathbb{Z} \hookrightarrow \mathbb{Q}$ has both adjoints, the left adjoint being rounding up, and the right adjoint rounding down: $\lceil _ \rceil \dashv \iota \dashv \lfloor _ \rfloor$.

We also have the propositional analog of Example 6.5.6, using the operations from Remarks 5.5.12 and 5.5.14:

$$\mathrm{Sub}(A) \underset{\forall_f}{\overset{\exists_f}{\underset{\leftarrow f^* -}{\overset{\bot}{\bot}}}} \mathrm{Sub}(B),$$

for any map $f : A \to B$. ⌟

6.6 *The Yoneda Lemma*

sec:yoneda
def:yoneda

Definition 6.6.1. For a locally small[30] precategory $\mathcal{C}$, taking contravariant representable functors (recall Example 6.4.22) induces a functor

$$よ : \mathcal{C} \to \mathrm{Set}^{\mathcal{C}^{\mathrm{op}}}, \qquad よ(C) :\equiv h_C \equiv \mathrm{Hom}_{\mathcal{C}}(_, C).$$ ⌟

[30] We suppress the universe $\mathcal{U}$, even though any precategory is $\mathcal{U}$-small for some universe $\mathcal{U}$; the point is that we should consider the category of $\mathcal{U}$-small sets for the same universe as the one containing the hom-sets.

thm:yoneda-lemma

Theorem 6.6.2. *For any locally small precategory $\mathcal{C}$, functor $F : \mathrm{Set}^{\mathcal{C}^{\mathrm{op}}}$, and $C : \mathcal{C}$, evaluation at id_C gives an isomorphism*

$$\mathrm{Hom}_{\mathrm{Set}^{\mathcal{C}^{\mathrm{op}}}}(よ(C), F) \xrightarrow{\cong} F(C)$$

that is moreover natural in F and C.

cor:yo-ff

Corollary 6.6.3. *The Yoneda embedding* $\yo : \mathcal{C} \to \mathrm{Set}^{\mathcal{C}^{\mathrm{op}}}$ *is fully faithful.*

cor:yo-emb-obj

Corollary 6.6.4. *If* $\mathcal{C}$ *is a category, then the action on objects of the Yoneda embedding is an injection* $\mathrm{Ob}(\mathcal{C}) \hookrightarrow \mathrm{Ob}(\mathrm{Set}^{\mathcal{C}^{\mathrm{op}}})$.

def:repr

Definition 6.6.5. A functor $F : \mathrm{Set}^{\mathcal{C}^{\mathrm{op}}}$ is called *representable* if there exists $C : \mathcal{C}$ together with an isomorphism $\alpha : \yo(C) \xrightarrow{\cong} F$. ⌟

cor:repr-prop

Corollary 6.6.6. *Representability is a proposition when* $\mathcal{C}$ *is a category, i.e., the type of pairs* (C, α) *in Definition 6.6.5 is a proposition.*

One benefit of the Yoneda lemma is that we can characterize adjoint functors in terms of representability, and then derive the uniqueness of adjoint functors in the case of categories.

lem:adj-via-repr

Lemma 6.6.7. *Suppose* $\mathcal{C}, \mathcal{D}$ *are precategories (*$\mathcal{D}$ *could be wild), and* $F : \mathcal{C} \to \mathcal{D}$ *is a (wild) functor. If we have a map on objects* $G : \mathrm{Ob}(\mathcal{D}) \to \mathrm{Ob}(\mathcal{C})$ *along with natural isomorphisms* $\alpha_D : \mathrm{Hom}_{\mathcal{D}}(F_, D) \xrightarrow{\cong} \yo(G(D))$ *for each* $D : \mathrm{Ob}(\mathcal{D})$*, then there is a unique extension to the data of a wild adjunction, i.e., extensions of* G *to a functor* $G : \mathcal{D} \to \mathcal{C}$*, such that* α_D *is natural in* $D : \mathcal{D}$.

cor:adj-unique

Corollary 6.6.8. *If* $F : \mathcal{C} \to \mathcal{D}$ *is a (wild) functor defined on a category* $\mathcal{C}$*, then the data of a right adjoint* $F \dashv G$ *is a proposition.*

6.7 *Equivalences of categories*

def:cat-equiv

Definition 6.7.1. A functor $F : \mathcal{C} \to \mathcal{D}$ is an *equivalence of (pre)categories* if it is a left adjoint whose unit and counit transformations η and ε from (6.5.1) are isomorphisms. We write $\mathcal{C} \xrightarrow{\simeq} \mathcal{D}$ for the type of equivalences. ⌟

From Corollary 6.6.8 we immediately see the following:

lem:cat-equiv-is-prop

Lemma 6.7.2. *If* $\mathcal{C}$ *is a category, then being an equivalence is a proposition for a functor* $F : \mathcal{C} \to \mathcal{D}$.

It's often useful to show that a functor $F : \mathcal{C} \to \mathcal{D}$ is an equivalence by giving just a functor $G : \mathcal{D} \to \mathcal{C}$ along with natural isomorphisms $\eta : \mathrm{id}_{\mathcal{C}} \xrightarrow{\cong} GF$ and $\varepsilon : FG \xrightarrow{\cong} \mathrm{id}_{\mathcal{D}}$. This is analogous to Construction 2.9.9, and we refer to Ahrens, Kapulkin, and Shulman[31] for the implementation.

[31] Ahrens, Kapulkin, and Shulman, "Univalent categories and the Rezk completion", Lemma 6.2.

lem:cat-equiv-ff

Lemma 6.7.3. *Any equivalence of precategories is fully faithful.*

def:functor-split-eso

Definition 6.7.4. A functor $F : \mathcal{C} \to \mathcal{D}$ is *split essentially surjective on objects* if for all objects $D : \mathcal{D}$ we have a pair of an object $C : \mathcal{C}$ and a isomorphism $h : F(C) \xrightarrow{\cong} D$.

We say that F is *essentially surjective on objects* if for all D, we merely have such a pair. ⌟

lem:equiv-precat-is-ff-split-eso

Lemma 6.7.5. *For precategories* $\mathcal{C}$ *and* $\mathcal{D}$ *and a functor* $F : \mathcal{C} \to \mathcal{D}$*, the map from the type*

(1) *F is an equivalence of precategories*

to the type

(2) *F is fully faithful and split essentially surjective on objects,*

defined using Lemma 6.7.3 and sending $D : \mathcal{D}$ *to the pair* $(G(D), \varepsilon_D)$*, is an equivalence.*

We again refer to Ahrens, Kapulkin, and Shulman[32] for details.

[32] Ahrens, Kapulkin, and Shulman, "Univalent categories and the Rezk completion", Lemma 6.6.

lem:ff-eso

Lemma 6.7.6. *If* $F : \mathcal{C} \to \mathcal{D}$ *is fully faithful and* $\mathcal{C}$ *is a category, then for any* $D : \mathcal{D}$ *the type* $\sum_{C : \mathcal{C}} F(C) \xrightarrow{\cong} D$ *is a proposition.*

From this we conclude that if $\mathcal{C}$ is category, then split essential surjectivity is equivalent to essential surjectivity, and then Lemma 6.7.5 gives one of the fundamental theorems of category theory:

thm:ff-eso-equiv

THEOREM 6.7.7. *If $\mathcal{C}$ is a category, then the projection $(\mathcal{C} \xrightarrow{\simeq} \mathcal{D}) \to (\mathcal{C} \to \mathcal{D})$ is an injection with image given by the fully faithful and essentially surjective functors.*[33]

cor:Cat-ua

COROLLARY 6.7.8. *For categories $\mathcal{C}$ and $\mathcal{D}$ we have that the map $(\mathcal{C} \xrightarrow{=} \mathcal{D}) \to (\mathcal{C} \xrightarrow{\simeq} \mathcal{D})$ that sends $\mathrm{refl}_{\mathcal{C}}$ to the identity equivalence on $\mathcal{C}$ is an equivalence.*[34]

Since the type of functors between categories is 1-truncated, we conclude as well:

cor:cat-2-type

COROLLARY 6.7.9. *The type of categories* Cat *is 2-truncated.*

This result is one of the reasons that the primary objects of interest in category theory are the categories (i.e., univalent precategories), rather than the more general precategories.

However, it's occasionally useful to present a category via a non-univalent precategory. This will not be *equivalent* to the corresponding category, so we need a notion that establishes their connection.

def:we-cat

DEFINITION 6.7.10. A functor $F : \mathcal{C} \to \mathcal{D}$ of precategories is a *weak equivalence* if it is fully faithful and essentially surjective on objects. ⌟

From the point of view of categories, however, weak equivalences are as good as equivalences.

lem:precomp-equiv-cat

LEMMA 6.7.11. *If $H : \mathcal{C} \to \mathcal{D}$ is a weak equivalence of precategories, and $\mathcal{E}$ is a category, then $_ \circ H : \mathcal{E}^{\mathcal{D}} \to \mathcal{E}^{\mathcal{C}}$ is an equivalence of categories.*

Our goal now is to construct for every precategory $\mathcal{C}$ a category $L(\mathcal{C})$ along with a weak equivalence $\eta : \mathcal{C} \to L(\mathcal{C})$. Here, $L(\mathcal{C})$ is called the *Rezk completion* of $\mathcal{C}$.

The idea is a generalization of the Cayley's Theorem 5.6.1, which is also how we'll go from abstract groups to concrete groups in Chapter 7. The reader may want to read that first, and then come back here before reading the comparison between Rezk completion and delooping of abstract groups in Section 7.8.

Note that Set is a category, and hence so is the functor category $\mathrm{Set}^{\mathcal{C}^{\mathrm{op}}}$ by Exercise 6.4.20. From Corollary 6.6.3 we know that the Yoneda embedding $よ : \mathcal{C} \to \mathrm{Set}^{\mathcal{C}^{\mathrm{op}}}$ is fully faithful, motivating the following definition.

def:Rezk-completion

DEFINITION 6.7.12. For any precategory $\mathcal{C}$, we define its Rezk completion as the full subcategory of $\mathrm{Set}^{\mathcal{C}^{\mathrm{op}}}$ on the representable functors, $L(\mathcal{C}) :\equiv (\mathrm{Set}^{\mathcal{C}^{\mathrm{op}}})_{\mathrm{isRepr}}$. We define $\eta : \mathcal{C} \to L(\mathcal{C})$ to be the factorization of $よ$ via the trivial proofs that any $よ(C)$ is representable. ⌟

thm:Rezk-completion

THEOREM 6.7.13. *The inclusion $\eta : \mathcal{C} \to L(\mathcal{C})$ is a weak equivalence. Furthermore, if $\mathcal{C}$ is essentially $\mathcal{U}$-small, then so $L(\mathcal{C})$.*

Proof. We have that η is fully faithful since $よ$ is, and it's essentially surjective by definition. Suppose now that $\mathcal{C}$ is essentially $\mathcal{U}$-small, meaning that the type of objects $\mathrm{Ob}(\mathcal{C})$ and all hom-sets are essentially $\mathcal{U}$-small. Then we have the hom-functor $\mathrm{hom}_{\mathcal{C}} : \mathcal{C}^{\mathrm{op}} \times \mathcal{C} \to \mathrm{Set}_{\mathcal{U}}$, so we get a Yoneda embedding $\mathcal{C} \to \mathrm{Set}_{\mathcal{U}}^{\mathcal{C}^{\mathrm{op}}}$. Since $\mathrm{Ob}(\mathcal{C})$ is essentially $\mathcal{U}$-small, then $\mathrm{Set}_{\mathcal{U}}^{\mathcal{C}^{\mathrm{op}}}$ is locally $\mathcal{U}$-small, and hence so is $L(\mathcal{C})$. And now $\mathrm{Ob}(L(\mathcal{C}))$

[33] Compare Lemma 2.17.4, stating that a function $f : A \to B$ of types is an equivalence if and only if it is injective and surjective.

[34] There is also a version of this statement for precategories, but it concerns the stronger notion of *isomorphism of precategories*, i.e., fully faithful functors that induce an equivalence on the types of objects. The point is that for categories, this is the same as equivalence.

is the image of $\mathrm{Ob}(\lrcorner) : \mathrm{Ob}(\mathcal{C}) \to \mathrm{Ob}(\mathrm{Set}_{\mathcal{U}}^{\mathcal{C}^{\mathrm{op}}})$, which is then essentially $\mathcal{U}$-small by the Replacement **??** 2.19.4. □

xca:Rezk-completion-locally-small

Exercise 6.7.14. Show that if $\mathcal{C}$ is a locally $\mathcal{U}$-small, then so is $L(\mathcal{C})$, even though $\mathrm{Set}_{\mathcal{U}}^{\mathcal{C}^{\mathrm{op}}}$ is only locally $\mathcal{U}$-small when $\mathcal{C}$ is essentially $\mathcal{U}$-small (Freyd and Street[35]). ⌟

[35] Peter Freyd and Ross Street. "On the size of categories". In: *Theory Appl. Categ.* 1 (1995), No. 9, 174–178. ISSN: 1201-561X. URL: http://www.tac.mta.ca/tac/volumes/1995/n9/1-09abs.html.

Another consequence of Corollary 6.7.8 is the following.

thm:types-are-groupoids

Theorem 6.7.15. *For 1-types A and B, the projection from the functor groupoid $[A, B]$, of the corresponding path groupoids, to the function type $(A \to B)$ is an equivalence. Consequently, the path groupoid construction (Example 6.2.12) is an equivalence $\mathcal{U}^{\leq 1} \to \mathrm{Gpd}$, where* Gpd *is the subtype of groupoids in* Cat.

Proof. Let's write out the definition of the type of functors in the case of path groupoids:

$$[A, B] \equiv \sum_{f : A \to B} \sum_{g : \prod_{x,y:A} (x \xrightarrow{=} y) \to (f(x) \xrightarrow{=} f(y))} \left(\prod_{x : A} g(\mathrm{refl}_x) = \mathrm{refl}_{f(x)} \right) \times \left(\prod_{x,y,z : A} \prod_{p : x \xrightarrow{=} y} \prod_{q : y \xrightarrow{=} z} g(qp) = g(q)g(p) \right)$$

Here, the two bracketed components are propositions. By path induction, g is determined by its action on reflexivity, so the first bracketed component nails it down, so only the first component, the function $f : A \to B$, remains up to equivalence.

As a first consequence, the projection from the type of equivalences of groupoids, $A \xrightarrow{\simeq} B$, to the type of equivalences of types, confusingly also denoted $A \xrightarrow{\simeq} B$, is an equivalence. (Hence the confusion is harmless.)

This shows that $\mathcal{U}^{\leq 1} \to \mathrm{Gpd}$ is injective, so by Lemma 2.17.4, it remains to show it is surjective. Observe that for a groupoid, the hom-sets and the identity arrows are determined by the univalence condition. And relative to this, the composition operation is determined (by path induction) by its values at $\mathrm{refl}_y p$, for all $p : x \xrightarrow{=} y$. But these values are forced to be p by the left unit law. This identifies any groupoid with the path groupoid on its objects, showing that $\mathcal{U}^{\leq 1} \to \mathrm{Gpd}$ is surjective. □

xca:Rezk-completion-1trunc

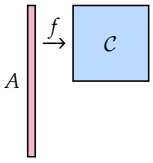
Exercise 6.7.16. Show that for any type A, we have a path pregroupoid with $\hom(x, y) :\equiv \|x \xrightarrow{=} y\|_0$. Identify its Rezk completion with the 1-truncation $\|A\|_1$. ⌟

Finally, let us use the Rezk completion to explore the relationship between precategories and categories more deeply. For example, we emphasize the fundamental importance of categories, but as mentioned, (non-univalent) precategories do appear from time to time. Can we think of them just in terms of categories? It turns out we can. The idea is that any precategory $\mathcal{C}$ presents a category $L(\mathcal{C})$, its Rezk completion, but it has more structure, namely its type of objects, $\mathrm{Ob}(\mathcal{C})$, as well as the relation between these and those of $L(\mathcal{C})$, given by the functor $\eta : \mathcal{C} \to L(\mathcal{C})$. Since η is fully faithful, this structure boils down to the map on objects, $\mathrm{Ob}(\eta) : \mathrm{Ob}(\mathcal{C}) \to \mathrm{Ob}(L(\mathcal{C}))$. This motivates the following definition.

def:flagged-cat

Definition 6.7.17. A *flagged category* is a triple $(\mathcal{C}, A, f)$ of a category $\mathcal{C}$, a type A, and a surjection $f : A \to \mathrm{Ob}(\mathcal{C})$.[36] We define the type of flagged

[36] The terminology is a bit whimsical: it's meant to indicate that we have a fat "flag" object, the category $\mathcal{C}$, equipped with a thinner "flagpole" object, the (higher) groupoid A, connected by the function f:



categories to be

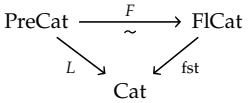
$$\mathrm{FlCat} :\equiv \sum_{\mathcal{C}:\mathrm{Cat}} \sum_{A:\mathcal{U}} \sum_{f:A\to \mathrm{Ob}(\mathcal{C})} \mathrm{isSurj}(f). \qquad \lrcorner$$

The flagged categories describe the fibers of the Rezk completion.

thm:flagged-cat-equiv

THEOREM 6.7.18. *The map* $F : \mathrm{PreCat} \to \mathrm{FlCat}$ *sending a category* $\mathcal{C}$ *to the triple* $(L(\mathcal{C}), \mathrm{Ob}(\mathcal{C}), \mathrm{Ob}(\eta))$ *is an equivalence.*[37]

[37] We conclude that the following triangle commutes definitionally, displaying the fiber of Rezk completion at a category $\mathcal{C}$ as the type $\sum_{A:\mathcal{U}} \sum_{f:A\to\mathrm{Ob}(\mathcal{C})} \mathrm{isSurj}(f)$:



Proof. The inverse at a triple $(\mathcal{C}, A, f)$ "pulls back" the (pre)category structure $\mathcal{C}$ along f to get a precategory $\mathcal{A}$ on the object type A and

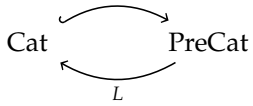
$$\hom(x, y) :\equiv \hom_{\mathcal{C}}(f(x), f(y)),$$

and likewise using f to define identity arrows, composition, as well as unit and associativity laws. By construction, this extends f to a fully faithful functor $f : \mathcal{A} \to \mathcal{C}$. This is also essentially surjective by the surjectivity of f, hence inducing an equivalence $L(\mathcal{A}) \xrightarrow{\simeq} \mathcal{C}$.

For the other round trip, starting with a precategory $\mathcal{C}$, we use the fully faithfulness of $\eta : \mathcal{C} \to L(\mathcal{C})$ to identify the precategory structure on $\mathrm{Ob}(\mathcal{C})$ with that pulled back from $L(\mathcal{C})$. □

rem:chicken-or-egg

REMARK 6.7.19. We have seen that precategories and categories are connected by an injection and retraction L:



From the point of view of precategories, categories are precategories with an extra property (univalence), while from the point of view of categories, precategories are categories equipped with extra (higher) structure (the flagged structure). ⌟

6.8 *Limits and colimits*

sec:limits

We have also seen two instances of limits and colimits, namely terminal and initial objects (Definitions 6.3.1 and 6.3.3, respectively). We now build up to the general notions by first looking at another dual pair of examples: products and coproducts.

def:prod-in-a-cat

DEFINITION 6.8.1. In a wild precategory $\mathcal{C}$, a *product cone* for a pair of objects A and B is a triple (P, p_1, p_2) as in the diagram in Figure 6.3(1), satisfying the *universal property of the product*, namely that for any other cone of the same form (X, x_1, x_2), there is a unique arrow $f : X \to P$ with $x_i \xrightarrow{=} p_i f$, for $i = 1, 2$, i.e., filling the triangles in Figure 6.3(2). ⌟

xca:product-cone-prop

EXERCISE 6.8.2. Show that if $\mathcal{C}$ is univalent, then the type of products cones on given objects A, B is a proposition. ⌟

We immediately recognize the product of types $A \times B$, along with the first and second projections, as a product cone $(A \times B, \mathrm{fst}, snd)$ in the wild category of types. Likewise, in the category of groups, the product $G \times H$ from Example 4.2.26 forms a product cone when equipped with the projection homomorphisms.

We say that a (wild) category *has binary products* if every pair of objects A, B as a product cone. In this case we are justified by Exercise 6.8.2 in writing $(A \times B, \mathrm{fst}, \mathrm{snd})$ for the (unique) product cone on A, B.

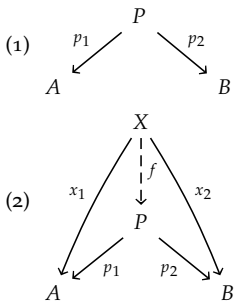


FIGURE 6.3: (1) A product cone, and (2) its universal property.

fig:product-cone

def:product-cones-as-terminal

REMARK 6.8.3. We can recover the notion of product cone from that of terminal objects by considering the wild (pre)*category of cones* given fixed objects A, B. This has a objects the cones of Figure 6.3(1) and as arrows the filled triangles of Figure 6.3(2). A product cone is then simply a terminal cone. ⌟

Like every categorical notion, there is of course a dual notion, obtained by turning all the arrows around.

def:coprod-in-a-cat

DEFINITION 6.8.4. In a wild precategory $\mathcal{C}$, a *coproduct cocone* for a pair of objects A and B is a triple (C, i_1, i_2) as in the diagram in Figure 6.4(1), satisfying the *universal property of the coproduct*, namely that for any other cocone of the same form (X, x_1, x_2), there is a unique arrow $f : C \to X$ with $x_j \xrightarrow{=} f i_j$, for $j = 1, 2$, i.e., filling the triangles in Figure 6.3(2). ⌟

In other words, a coproduct cocone is an initial cocone under the pair of objects A, B.

In the wild category of types, we have the coproduct cocones $(A \amalg B, \mathrm{inl}, \mathrm{inr})$. We haven't yet seen the coproduct of groups; knowing about products and categorical duality, we'd be bound to discover them! We shall study them in Section 8.6.

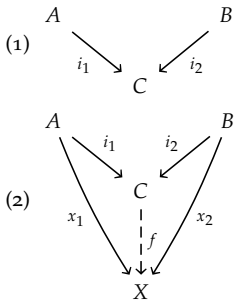


FIGURE 6.4: (1) A coproduct cocone, and (2) its universal property.

fig:coproduct-cone

Let's look at another pair of examples: pullbacks and pushouts.

Given two functions $f : B \to D$ and $g : C \to D$ with common target, the "pullback" which we will now define should be thought about as the type of all pairs of elements $(b, c) : B \times C$ so that $f(b) \xrightarrow{=} g(c)$. This construction is important in many situations also beyond group theory.

def:pullback

DEFINITION 6.8.5. Let B, C, D be types and let $f : B \to D$ and $g : C \to D$ be two maps. The *pullback* of f and g is the type

$$\prod(f, g) :\equiv \sum_{(b,c):B\times C} (f(b) \xrightarrow{=}_D g(c))$$

together with the two projections $\mathrm{pr}_B : \prod(f, g) \to B$ and $\mathrm{pr}_C : \prod(f, g) \to C$ sending $(b, c, p) : \prod(f, g)$ to $b : B$ or $c : C$. If f and g are clear from the context, we may write $B \times_D C$ instead of $\prod(f, g)$ and summarize the situation by the diagram

$$\begin{array}{ccc} B \times_D C & \xrightarrow{\mathrm{pr}_C} & C \\ {\scriptstyle pr j_B}\downarrow & & \downarrow{\scriptstyle g} \\ B & \xrightarrow[f]{} & D. \end{array} \quad ⌟$$

xca:univpropofpullback

EXERCISE 6.8.6. Let $f : B \to D$ and $g : C \to D$ be two maps with common target. If A is a type show that

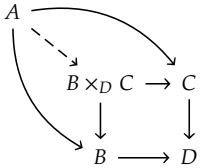
$$\begin{aligned} &(A \to B) \times_{(A\to D)} (A \to C) \to (A \to B \times_D C) \\ &\qquad (\beta, \gamma, p : f\beta \xrightarrow{=} g\gamma) \mapsto (a \mapsto (f(a), g(a), p(a) : f\beta(a) \xrightarrow{=} g\gamma(a))) \end{aligned}$$

is an equivalence.[38] ⌟

In view of Exercise 8.4.2 we will say that we have a *pullback diagram*

$$\begin{array}{ccc} A & \xrightarrow{f'} & C \\ {\scriptstyle g'}\downarrow & \lrcorner & \downarrow{\scriptstyle g} \\ B & \xrightarrow[f]{} & D \end{array}$$

[38] This verifies that our defined pullback is the pullback in the categorical sense: if the solid diagram commutes there is a unique dashed arrow making the resulting diagram commute:



to indicate that we have an element in $(A \to B) \times_{(A \to D)} (A \to C)$ such that the resulting map $A \to B \times_D C$ is an equivalence. This is emphasized with the little ⌟ symbol in the upper left corner.

Example 6.8.7. If $d : \mathbb{1} \to D$ denotes the constant function at $d : D$ and $g : C \to D$ is any map, then $\mathbb{1} \times_D C$ is equivalent to the preimage $g^{-1}(d) \equiv \sum_{b:B} d \xrightarrow{=} g(b)$.[39] ⌟

[39] I.e., the preimage is a pullback:

$$\begin{array}{ccc} g^{-1}(d) & \longrightarrow & C \\ \downarrow & \lrcorner & \downarrow g \\ \mathbb{1} & \xrightarrow[d]{} & D \end{array}$$

6.9 *Monoidal categories*

sec:monoidal-cats

7
Groups, abstractly

ch:absgroup

7.1 *Brief overview of the chapter*

Recall from Section 4.3 the definition of an abstract group and how to obtain an abstract group from a concrete one. In this chapter we will implement an inverse construction, how to obtain a (concrete) group from an abstract one, in Section 7.4. Likewise, in Section 7.5, we show how to obtain a (concrete) homomorphism from an abstract one. Thus we will have shown that, in principle,[1] it doesn't matter whether one develops group theory on the concrete or on the abstract level. More precisely, we give an equivalence of categories between the categories of concrete and of abstract groups.

Before we implement the above constructions, we first introduce in Section 7.2 a simpler structure, called monoid, of which abstract groups are a special case.[2] We then define in Section 7.3 the notion of homomorphism for abstract groups.

After groups and homomorphism, it is natural to continue to group actions, in Section 7.7, and again relate the abstract to the concrete.

In the optional Section 7.9 we look at how general identities types $a \xrightarrow{=}_A a'$ relate to groups.

[1] Of course this is not a reason to stop here, but to continue finding out which parts of group theory benefit from the concrete approach. Just to mention a few we have seen already: the conceptual simplicity of homomorphisms being pointed maps, actions being maps from the classifying type to Set, and the generalizations to ∞-groups indicated in Chapter 5.

[2] One could advocate for the name 'abstract monoid' here, were it not the case that we have no concrete analogue for monoids in our setting. The reason is the symmetry of the identity types.

7.2 *Monoids and abstract groups*

sec:monoids

A monoid is a collection of data consisting only of (1), (2), and (3) from the list in Definition 4.3.1. In other words, the existence of inverses is not assumed. For convenience we reproduce the shortened list here.

def:monoid

Definition 7.2.1. A *monoid* consists of the following data.

struc:monoid-set

(1) A set S, called the *underlying set*.

struc:monoid-unit

(2) An element $e : S$, called the *unit* or the *neutral element*.

struc:monoid-mult

(3) A function $S \to S \to S$, called *multiplication*, taking two elements $g_1, g_2 : S$ to their *product*, denoted by $g_1 \cdot g_2 : S$.

Moreover, the following equations should hold, for all $g, g_1, g_2, g_3 : S$.

monoid:unit-laws

(a) $g \cdot e = g$ and $e \cdot g = g$ (the *unit laws*)

monoid:ass-law

(b) $g_1 \cdot (g_2 \cdot g_3) = (g_1 \cdot g_2) \cdot g_3$ (the *associativity law*)

The property that S is a set, the unit laws, and the associativity law, are together known as the *monoid laws*. ⌟

ex:monoid

EXAMPLE 7.2.2. Let S be a set, and consider the type S^* of lists of elements of S as defined in Definition 2.12.11. Then S^* is a set according to Theorem 2.22.2. We can give S^* the structure of a monoid with the empty list ε as unit, and concatenation from Exercise 2.12.13 as multiplication, denoted $*$. Then the monoid laws can easily be proven to hold and hence $(S^*, \varepsilon, *)$ is a monoid. ⌟

Building on the definition of a monoid, we may encode the type of abstract groups as follows. We let S denote the underlying set, $e : S$ denote the unit, $\mu : S \to S \to S$ denote the multiplication operation $g \mapsto (h \mapsto g \cdot h)$, and $\iota : S \to S$ denote the inverse operation $g \mapsto g^{-1}$. Using that notation, we introduce names for the relevant propositions.

not:GroupLaws

$$\begin{aligned}
\text{UnitLaws}(S, e, \mu) &:\equiv \prod_{g:S} \big((\mu(g)(e) = g) \times (\mu(e)(g) = g)\big) \\
\text{AssocLaw}(S, \mu) &:\equiv \prod_{g_1, g_2, g_3 : S} \big(\mu(g_1)(\mu(g_2)(g_3)) = \mu(\mu(g_1)(g_2))(g_3)\big) \\
\text{MonoidLaws}(S, e, \mu) &:\equiv \text{isSet}(S) \times \text{UnitLaws}(S, e, \mu) \times \text{AssocLaw}(S, \mu) \\
\text{InverseLaw}(S, e, \mu, \iota) &:\equiv \prod_{g:S} \big(\mu(g)(\iota(g)) = e\big) \\
\text{GroupLaws}(S, e, \mu, \iota) &:\equiv \text{MonoidLaws}(S, e, \mu) \times \text{InverseLaw}(S, e, \mu, \iota)
\end{aligned}$$

def:type-absrgp

DEFINITION 7.2.3. Recall the definition of abstract group in Definition 4.3.1. The type of abstract groups is

$$\text{Group}^{\text{abs}} :\equiv \sum_{S:\mathcal{U}} \sum_{e:S} \sum_{\mu : S \to S \to S} \sum_{\iota : S \to S} \text{GroupLaws}(S, e, \mu, \iota). \qquad ⌟$$

Thus, following the convention introduced in Remark 2.8.2, an abstract group G will be a quintuple of the form $G \equiv (S, e, \mu, \iota, !)$. For brevity, we will usually omit the proof of the properties from the display, since it's unique, and write an abstract group as though it were a quadruple $G \equiv (S, e, \mu, \iota)$.

rem:inverses-as-property

REMARK 7.2.4. Instead of including the inverse operation as part (4) of the structure (including the property (4) (c)), some authors assume the existence of inverses by positing the property (4) (c) below.

monoid:inv-op

(4) A function $(_)^{-1} : S \to S$, the *inverse operation*, satisfying:

monoid:inv-law

(c) $g \cdot g^{-1} = e$ for all $g : S$ (the *law of inverses*).

axiom:mere-inverse

(5) For all $g : S$ there exists an element $h : S$ such that $e = g \cdot h$.

We will now compare (5) to (4). Property (5) contains the phrase "there exists", and thus its translation into type theory uses the quantifier $\exists$, as defined in Section 2.16. Under this translation, property (5) does not immediately allow us to speak of "the inverse of g". However, the following lemma shows that we can define an inverse operation as in (4) from a witness of (5) – its proof goes by using the unit laws (3) (a) and the associativity law (3) (b) to prove that inverses are unique. As a consequence, we can speak of "*the* inverse of g". ⌟

lem:group-inv-operation

LEMMA 7.2.5. *Given a set S together with e and $\cdot$ as in Definition 7.2.1 satisfying the unit laws, the associativity law, and property (5), we have a unique "inverse" function $S \to S$ having property (4) (c) of Definition 4.3.1.*

Proof. Consider the function $\mu : S \to (S \to S)$ defined as $g \mapsto (h \mapsto g \cdot h)$. Let $g : S$. We claim that the fiber $\mu(g)^{-1}(e)$ is contractible. Contractibility is a proposition, hence to prove it from (5), one can as well assume the actual existence of h such that $g \cdot h = e$. Then $(h, !)$ is an element of the fiber $\mu(g)^{-1}(e)$. We will now prove that it is a center of contraction. For any other element $(h', !)$, we want to prove $(h, !) = (h', !)$, which is equivalent to the equation $h = h'$. In order to prove the latter, we show that h is also an inverse on the left of g, meaning that $h \cdot g = e$. This equation is also a proposition, so we can assume from (5) that we have an element $k : S$ such that $h \cdot k = e$. Multiplying that equation by g on the left, one obtains

$$k = e \cdot k = (g \cdot h) \cdot k = g \cdot (h \cdot k) = g \cdot e = g,$$

from which we see that $h \cdot g = e$. Now it follows that

$$h = h \cdot e = h \cdot (g \cdot h') = (h \cdot g) \cdot h' = e \cdot h' = h',$$

as required. Hence $\mu(g)^{-1}(e)$ is contractible, and we may define g^{-1} to be the center of the contraction, for any $g : S$. The function $g \mapsto g^{-1}$ satisfies the law of inverses (4) (c), as required.[3] Since the inverse of each $g : S$ is unique, it follows by function extensionality that this 'inverse' function is unique. □

[3] Note that this proof also shows that $(g^{-1})^{-1} = g$ and hence $g^{-1} \cdot g = e$, for any $g : S$.

Remark 7.2.6. That the concept of an abstract group synthesizes the idea of symmetries will be justified in Section 7.4 where we prove that the function abs : Group → Group^{abs} from Definition 4.3.4 is an equivalence. ⌟

rem:abs-iso

Remark 7.2.7. If $\mathcal{G} \equiv (S, e, \mu, \iota)$ and $\mathcal{G}' \equiv (S', e', \mu', \iota')$ are abstract groups, an element of the identity type $\mathcal{G} \xrightarrow{=} \mathcal{G}'$ consists of quite a lot of information, provided we interpret it by repeated application of Lemma 2.10.3. First and foremost, we need an identification $p : S \xrightarrow{=} S'$ of sets, but from there on the information is a proof of a conjunction of propositions.[4] An analysis shows that this conjunction can be shortened to the equations $e' = p(e)$ and $\mu'(p(s), p(t)) = p(\mu(s, t))$. A convenient way of obtaining an identity p that preserves these equations is to apply univalence to an equivalence $f : S \xrightarrow{\simeq} S'$ that preserves them. We call such a function f an *isomorphism of abstract groups*. ⌟

ref:no-abs-for-ooo

[4] Even though we are able to give a concise definition of ∞-groups in Section 4.7, we don't know how to define the type of "abstract ∞-groups" in a way similar to Definition 4.3.1: such a definition would require infinitely many levels of operations producing identifications of instances of operations of lower levels. And an identification would similarly require infinitely many operations identifying the operations at all levels. See also Remark 7.2.11.

Exercise 7.2.8. Perform the abovementioned analysis. ⌟

xca:op-abs-group

Exercise 7.2.9. Let $\mathcal{G} \equiv (S, e, \mu, \iota)$ be an abstract group. Define another structure $\mathcal{G}^{\text{op}} :\equiv (S, e, \mu^{\text{op}}, \iota)$, where $\mu^{\text{op}} : S \to S \to S$ sends $a, b : S$ to $\mu(b, a)$, i.e., μ^{op} swaps the order of the arguments as compared to μ.

Show that $\iota : S \to S$ defines an isomorphism $\mathcal{G} \xrightarrow{\simeq} \mathcal{G}^{\text{op}}$.[5] ⌟

[5] Hint: in down-to-earth terms this boils down to the equations $e^{-1} = e$ and $(a \cdot b)^{-1} = b^{-1} \cdot a^{-1}$.

xca:conj

Exercise 7.2.10. Let $\mathcal{G} \equiv (S, e, \mu, \iota)$ be an abstract group and let $g : S$. For any $s : S$, let $\text{conj}^g(s) :\equiv g \cdot s \cdot g^{-1}$. Show that the resulting function $\text{conj}^g : S \to S$ preserves the group structure (e.g., $g \cdot (s \cdot s') \cdot g^{-1} = (g \cdot s \cdot g^{-1}) \cdot (g \cdot s \cdot g^{-1})$) and is an equivalence. The resulting identification $\text{conj}^g : \mathcal{G} \xrightarrow{=} \mathcal{G}$ is called *conjugation* by g. ⌟

rem:ee=e_coherence

Remark 7.2.11. Without the requirement that the underlying type of an abstract group or monoid is a set, life would be more complicated. For instance, for the case when g is e, the unit laws (3) (a) of Definition 7.2.1

would provide *two* (potentially different) identifications $e \cdot e \xrightarrow{=} e$, and we would have to separately assume that they agree. This problem vanishes in the setup we adopted for ∞-groups in Section 4.7. ⌟

Exercise 7.2.12. Given an element g in an abstract group, prove that $e = g^{-1} \cdot g$ and $g = (g^{-1})^{-1}$. (Hint: study the proof of Lemma 7.2.5.) ⌟

Exercise 7.2.13. Prove that the types of monoids and abstract groups are groupoids. ⌟

Exercise 7.2.14. There is a leaner way of characterizing what an abstract group is: define a *sheargroup* to be a set S together with an element $e : S$, a function $_ * _ : S \to S \to S$, sending $a, b : S$ to $a * b : S$, and the following propositions, where we use the shorthand $\bar{a} :\equiv a * e$:

(1) $e * a = a$,

(2) $a * a = e$, and

(3) $c * (b * a) = \overline{(c * \bar{b})} * a$,

for all $a, b, c : S$. Construct an equivalence from the type of abstract groups to the type of sheargroups.[6] ⌟

Exercise 7.2.15. Another and even leaner way to define abstract groups, highlighting how we can do away with both the inverse and the unit: a *Furstenberg group*[7] is a nonempty set S together with a function $_ \circ _ : S \to S \to S$, sending $a, b : S$ to $a \circ b : S$, with the property that

(1) for all $a, b, c : S$ we have that $(a \circ c) \circ (b \circ c) = a \circ b$, and

(2) for all $a, c : S$ there is a $b : S$ such that $a \circ b = c$.

Construct an equivalence from the type of Furstenberg groups to the type of abstract groups.[9] ⌟

7.3 *Abstract homomorphisms*

In this section we define the notion of homomorphism for abstract groups, which we touched upon in Remark 4.4.20. We start by an exercise that simplifies the requirements for abstract group homomorphisms.

Exercise 7.3.1. Let $\mathcal{G} :\equiv (S, e_{\mathcal{G}}, \cdot_{\mathcal{G}}, \iota_{\mathcal{G}})$ and $\mathcal{H} :\equiv (T, e_{\mathcal{H}}, \cdot_{\mathcal{H}}, \iota_{\mathcal{H}})$ be abstract groups, and $f : S \to T$ a function satisfying $f(s \cdot_{\mathcal{G}} s') =_T f(s) \cdot_{\mathcal{H}} f(s')$ for all $s, s' : S$. Show that $f(e_{\mathcal{G}}) = e_{\mathcal{H}}$ and $f(\iota_{\mathcal{G}}(s)) = \iota_{\mathcal{H}}(f(s))$ for all $s : S$. ⌟

Thus we see that, due to the properties of the abstract groups, if f preserves multiplication, then f also preserves unit and inverses.[10]

Definition 7.3.2. Let $\mathcal{G} :\equiv (S, e_{\mathcal{G}}, \cdot_{\mathcal{G}}, \iota_{\mathcal{G}})$ and $\mathcal{H} :\equiv (T, e_{\mathcal{H}}, \cdot_{\mathcal{H}}, \iota_{\mathcal{H}})$ be two abstract groups,[11] then the set of homomorphisms from $\mathcal{G}$ to $\mathcal{H}$ is

$$\mathrm{Hom}^{\mathrm{abs}}(\mathcal{G}, \mathcal{H}) :\equiv \sum_{f : S \to T} \prod_{s,s' : S} \big(f(s \cdot_{\mathcal{G}} s') =_T f(s) \cdot_{\mathcal{H}} f(s')\big).$$

For groups G and H, the function

$$\mathrm{abs} : \mathrm{Hom}(G, H) \to \mathrm{Hom}^{\mathrm{abs}}(\mathrm{abs}(G), \mathrm{abs}(H))$$

is defined as the function $f \mapsto \mathrm{abs}(f) :\equiv (Uf, !)$ made explicit in Definition 4.4.5 and satisfying the properties by Lemma 4.4.6. ⌟

[6] Hint: setting $a \cdot b :\equiv \bar{b} * a$ gives you an abstract group from a sheargroup and conversely, letting $a * b = b \cdot a^{-1}$ takes you back. On your way you may need at some point to show that $\bar{\bar{a}} = a$: setting $c = \bar{a}$ and $b = a$ in the third formula will do the trick (after you have established that $\bar{e} = e$). This exercise may be good to look back to in the many instances where the inverse inserted when "multiplying from the right by a" is forced by transport considerations.

[7] Named after Hillel Furstenberg who at the age of 20 published a paper doing this exercise.[8]

[8] Harry Furstenberg. "The inverse operation in groups". In: *Proc. Amer. Math. Soc.* 6 (1955), pp. 991–997. DOI: 10.2307/2033124.

[9] Hint: show that the function $a \mapsto a \circ a$ is constant, with value, say, e. Then show that S together with the "unit" e, "multiplication" $a \cdot b :\equiv a \circ (e \circ b)$ and "inverse" $b^{-1} :\equiv e \circ b$ is an abstract group.

[10] For monoids this is not true: Let M be the monoid with two elements, 1 and 0, with ordinary multiplication, so the unit is 1. Consider $\mathbb{1}$ as the trivial monoid. Now define $h : \mathbb{1} \to M$ by $h(0) = 0$. Then h preserves multiplication, but not the unit. Note that M cannot be extended to an abstract group, since giving 0 an inverse would make 0 equal to 1.

[11] Recall from Definition 4.3.1 that the components comprise the underlying set, the unit element, the multiplication, and the inverse operation. We also need the laws to hold, but this notation elides the corresponding witnesses.

In the display, $f(s \cdot_{\mathcal{G}} s') =_T f(s) \cdot_{\mathcal{H}} f(s')$ is a proposition; hence a homomorphism of abstract groups is uniquely determined by its underlying function of sets, and unless there is danger of confusion we write f instead of $(f, !)$.

rem:monoid-hom

Remark 7.3.3. With our definition it is immediate that a homomorphism of abstract groups also defines a homomorphism of the underlying monoids, preserving multiplication and thereby unit. However, for monoids as defined in Definition 7.2.1, it is possible to preserve multiplication but not the unit, as shown in Footnote 10. Hence, for monoids we define the set of homomorphisms from $M \equiv (S, e_M, \cdot_M)$ to $N \equiv (T, e_N, \cdot_N)$ by

$$\sum_{f : S \to T} \Big(\big(f(e_M) =_T e_N \big) \times \prod_{s,s' : S} \big(f(s \cdot_M s') =_T f(s) \cdot_N f(s') \big) \Big). \qquad \lrcorner$$

xca:abshomcomposition

Exercise 7.3.4. Prove that the composition of two composable abstract homomorphisms[12] is again an abstract homomorphism. Prove also that

$$\mathrm{abs}(\mathrm{id}_G) = \mathrm{id}_{\mathrm{abs}(G)} \quad \text{and} \quad \mathrm{abs}(f_1 f_0) = \mathrm{abs}(f_1)\,\mathrm{abs}(f_0)$$

for all $f_0 : \mathrm{Hom}(G_0, G_1)$ and $f_1 : \mathrm{Hom}(G_1, G_2)$.[13] Show that $\mathrm{Hom}(G, G)$ and $\mathrm{Hom}^{\mathrm{abs}}(\mathcal{G}, \mathcal{G})$ are monoids. $\lrcorner$

[12] Composition here means composition of the functions on the underlying sets, and composable means that these functions have types such that they indeed can be composed. The latter is sometimes tacitly assumed.

[13] In other words, for composable homomorphisms f_0, f_1.

ex:conjhom

Example 7.3.5. Let $\mathcal{G} = (S, e, \mu, \iota)$ be an abstract group and let $g : S$. In Exercise 7.2.10 we defined $\mathrm{conj}^g : S \to S$ by setting $\mathrm{conj}^g(s) :\equiv g \cdot s \cdot g^{-1}$ for all $s : S$, and asked you to show that it "preserves the group structure", i.e., it is a homomorphism

$$\mathrm{conj}^g : \mathrm{Hom}^{\mathrm{abs}}(\mathcal{G}, \mathcal{G})$$

called *conjugation by* g. Actually, we asked for more: namely that conjugation by g is an isomorphism, and hence determines an identification (for which we used the same symbol) $\mathrm{conj}^g : \mathcal{G} \xrightarrow{=} \mathcal{G}$.

If $\mathcal{H}$ is some other abstract group, transport along conj^g gives an identification $\mathrm{conj}^g_* : \mathrm{Hom}(\mathcal{H}, \mathcal{G}) \xrightarrow{=} \mathrm{Hom}(\mathcal{H}, \mathcal{G})$ which should be viewed as "postcomposing with conjugation by g". Similarly for elements in $\mathcal{H}$, giving rise to "precomposition with conjugation by h".

The connection with inner automorphisms of a given group G is as follows. Recalling Example 4.4.21 and Definition 4.4.22, we have that $\mathrm{abs}(\mathrm{Binn})(g) = \Omega(\mathrm{id}_{BG}, g^{-1}) = \mathrm{conj}^g$, for every $g : UG$. $\lrcorner$

xca:abs-homgroup

Exercise 7.3.6. Let $\mathcal{G} :\equiv (S, e_{\mathcal{G}}, \cdot_{\mathcal{G}}, \iota_{\mathcal{G}})$ and $\mathcal{H} :\equiv (T, e_{\mathcal{H}}, \cdot_{\mathcal{H}}, \iota_{\mathcal{H}})$ be abstract groups and consider the set $\mathrm{Hom}^{\mathrm{abs}}(\mathcal{H}, \mathcal{G})$ of homomorphisms from $\mathcal{H}$ to $\mathcal{G}$. For any $f, g : \mathrm{Hom}^{\mathrm{abs}}(\mathcal{H}, \mathcal{G})$, define the function $(f \cdot_{\mathcal{G}} g) : T \to S$ by $(f \cdot_{\mathcal{G}} g)(t) :\equiv f(t) \cdot_{\mathcal{G}} g(t)$ for $t : T$. Show that $\mathcal{G}$ is abelian if and only if any $(f \cdot_{\mathcal{G}} g)$ is a homomorphism. $\lrcorner$

7.4 *Groups: from abstract to concrete and back*

sec:GSetforabstract

For constructing a group from an abstract group, we draw our inspiration from Definition 5.5.4 and Theorem 5.5.7, which identify each group G with the group classified by the type of its torsors, pointed by its principal torsor. That is, in total analogy, we define the torsors for an abstract group, and it will then be relatively simple to show that the constructions of

(1) forming the abstract group of a group and

(2) taking the group classified by the torsors of an abstract group

Recall Footnote 4, explaining why we do not consider an "abstract" counterpart of the concept of ∞-group. Consequently, all we do in this section is set-based.

are inverse to each other.

Let G be a group and $X : BG \to \mathrm{Set}$ a G-set. Using the underlying set $X(\mathrm{sh}_G)$, we can restrict the codomain of X to $\mathrm{Set}_{(X(\mathrm{sh}_G))}$, the classifying type of $\Sigma_{X(\mathrm{sh}_G)}$. Then we can view X as the classifying function of a group homomorphism from G to $\Sigma_{X(\mathrm{sh}_G)}$. We already know the abstract versions of all three ingredients, the two groups and the homomorphism. Thus, the abstract version of X can be expected to consist of the set $X(\mathrm{sh}_G)$ and $\mathrm{abs}(X)$, the abstract homomorphism from $\mathrm{abs}(G)$ to $\mathrm{abs}(\Sigma_{X(\mathrm{sh}_G)})$.

A case in point is the principal G-torsor $\mathbb{P}_{\mathrm{sh}_G} \equiv (z \mapsto (\mathrm{sh}_G \xrightarrow{=} z))$. Its underlying set is UG. The abstract version of the corresponding homomorphism, defined by transport, is the function $UG \to (UG \xrightarrow{=} UG)$ mapping g to $(g \cdot _)$, i.e., postcomposition with g.[14] A small generalization now leads to the following definition.

def:choicesshG

def:abstrGtorsors

Definition 7.4.1. Given an abstract group $\mathcal{G} \equiv (S, e, \mu, \iota)$, a *$\mathcal{G}$-set* $\mathcal{X}$ is a pair $(X, \cdot_X)$ where X is a set and $\cdot_X$ an abstract homomorphism from $\mathcal{G}$ to to $\mathrm{abs}(\Sigma_X)$, the abstract permutation group of X.[15] Thus the type of $\mathcal{G}$-sets is defined as

$$\mathcal{G}\text{-Set}^{\mathrm{abs}} :\equiv \sum_{X : \mathrm{Set}} \mathrm{Hom}^{\mathrm{abs}}(\mathcal{G}, \mathrm{abs}(\Sigma_X)).$$

The *principal $\mathcal{G}$-torsor* $\mathbb{P}^{\mathrm{abs}}_{\mathcal{G}}$ is the $\mathcal{G}$-set consisting of the underlying set S together with the homomorphism $\mathcal{G} \to \mathrm{abs}(\Sigma_S)$ with underlying function $S \to (S \xrightarrow{=} S)$ given by sending $g : S$ to $(s \mapsto \mu(g, s))$.

The type of *$\mathcal{G}$-torsors* is

$$\mathrm{Torsor}^{\mathrm{abs}}_{\mathcal{G}} :\equiv \sum_{\mathcal{X} : \mathcal{G}\text{-Set}^{\mathrm{abs}}} \|\mathbb{P}^{\mathrm{abs}}_{\mathcal{G}} \xrightarrow{=} \mathcal{X}\|. \qquad \lrcorner$$

xca:absprtorsor

Exercise 7.4.2. In the setting of the above definition, give an identification of $(S, (s \mapsto \mu(g, s)))$ with $(S, (s \mapsto \mu(s, \iota(g))))$ in the type $\mathcal{G}\text{-Set}^{\mathrm{abs}}$.[16] $\lrcorner$

Example 7.4.3. Given a group G, recall from Lemma 4.3.3 that the abstract group is $\mathrm{abs}(G) \equiv (UG, e_G, \cdot, (_)^{-1})$ with $UG \equiv (\mathrm{sh}_G \xrightarrow{=} \mathrm{sh}_G)$ and $e_G \equiv \mathrm{refl}_{\mathrm{sh}_G}$, and $\cdot$ and $(_)^{-1}$ as usual for paths. Unravelling the definition, and Definition 7.3.2, we see that an $\mathrm{abs}(G)$-set consists of

(1) a set S, and

(2) a function $f : UG \to (S \xrightarrow{=} S)$ such that

(3) for all $p, q : UG$ we have that $f(p\, q) = f(p)\, f(q)$. $\lrcorner$

Clearly, the types $\mathcal{G}\text{-Set}^{\mathrm{abs}}$ and $\mathrm{Torsor}^{\mathrm{abs}}_{\mathcal{G}}$ are groupoids, and the latter is by definition connected. Thus we define:

def:concr

Definition 7.4.4. For any abstract group $\mathcal{G}$, the (concrete) *group* $\mathrm{concr}(\mathcal{G})$ *associated with* $\mathcal{G}$ is the group classified by the pointed connected groupoid $(\mathrm{Torsor}^{\mathrm{abs}}_{\mathcal{G}}, \mathbb{P}^{\mathrm{abs}}_{\mathcal{G}})$. $\lrcorner$

To help reading the coming proofs we introduce some notation that is redundant, but may aid the memory in cluttered situations. Let x, y, z be elements in some type, then define:[17]

$$\mathrm{preinv} : (y \xrightarrow{=} x) \to ((y \xrightarrow{=} z) \xrightarrow{=} (x \xrightarrow{=} z)), \quad \mathrm{preinv}(q)(p) :\equiv \mathbb{P}_q p :\equiv p q^{-1}$$
$$\mathrm{post} : (y \xrightarrow{=} z) \to ((x \xrightarrow{=} y) \xrightarrow{=} (x \xrightarrow{=} z)), \quad \mathrm{post}(p)(q) :\equiv p q$$

[14] A (free) choice has been made to define $\mathbb{P}_{\mathrm{sh}_G}$ using $(\mathrm{sh}_G \xrightarrow{=} z)$ and not $(z \xrightarrow{=} \mathrm{sh}_G)$. In the latter case the abstract homomorphism would map g to $(_ \cdot g^{-1})$, i.e., precomposition with the inverse of g. See also Exercise 7.4.2 and Example 7.4.5.

[15] For any $s : S$, the application of $\cdot_X$ to s is denoted as $s \cdot_X$ of type $X \xrightarrow{=} X$; by univalence $s \cdot_X$ can be applied to $x : X$ which is denoted as $s \cdot_X x : X$.

The abstract homomorphism $\cdot_X$ is also called the *action* of the $\mathcal{G}$-set, similarly to the action of a G-set X in Definition 5.2.1. In particular when $\mathcal{G}$ is $\mathrm{abs}(G)$ for some group G, the notations $\cdot_X$ for group action and abstract group action comply, see Remark 5.2.14.

[16] Every abstract group (S, e, μ, ι) has an isomorphic *opposite* group (S, e, μ', ι), where $\mu'(g, g') = \mu(g', g)$ for all $g, g' : S$. The canonical isomorphism is ι.

[17] We recognize preinv from Lemma 5.5.6 as the induced map of identity types $\mathbb{P}_{_} : (y \xrightarrow{=} x) \to (\mathbb{P}_y \xrightarrow{=} \mathbb{P}_x)$, followed by evaluation at z. Post-composition post is transport in the family $\mathbb{P}_x$, while preinv is precomposition by the inverse of its argument. We will sometimes write preinv_z to stress the variable z in the type of preinv, and likewise write post_x.

ex:BqG

Example 7.4.5. Given a group G and $z : BG$, the set $\mathbb{P}_z(\mathrm{sh}_G) \equiv (z \xrightarrow{=} \mathrm{sh}_G)$ is an abs(G)-set when we equip it with

$$\mathrm{post}_z : UG \to ((z \xrightarrow{=} \mathrm{sh}_G) \xrightarrow{=} (z \xrightarrow{=} \mathrm{sh}_G)),$$

which is the abstract homomorphism mapping g to $(g_)$, that is, left multiplication by g. For $z \equiv \mathrm{sh}_G$, this abs(G)-set is definitionally equal to the principal abs(G)-torsor.[18] We give this construction a short name by defining, for all $z : BG$, the map

fr:shGz

$$Bq_G : BG \to_* (\mathrm{Torsor}^{\mathrm{abs}}_{\mathrm{abs}(G)}, \mathbb{P}^{\mathrm{abs}}_{\mathrm{abs}(G)}), \quad Bq_G(z) :\equiv (\mathbb{P}_z(\mathrm{sh}_G), \mathrm{post}_z, !),$$

pointed by reflexivity. The name Bq_G anticipates its use as classifier of a homomorphism. ⌟

[18] For $z : BG$, the set $\mathbb{P}_{\mathrm{sh}_G}(z) \equiv (\mathrm{sh}_G \xrightarrow{=} z)$ equipped with $\mathrm{preinv}_z : UG \to ((\mathrm{sh}_G \xrightarrow{=} z) \xrightarrow{=} (\mathrm{sh}_G \xrightarrow{=} z))$ is also an abs(G)-set. Indeed, preinv_z is an abstract homomorphism since, for all $p, q : UG$, we have that $\mathrm{preinv}_z(p\,q) = \mathrm{preinv}_z(p)\mathrm{preinv}_z(q)$: for $r : \mathrm{sh}_G \xrightarrow{=} z$ we have that $\mathrm{preinv}_z(p\,q)(r) = r\,(p\,q)^{-1} = (r\,q^{-1})p^{-1} = \mathrm{preinv}_z(p)(\mathrm{preinv}_z(q)(r))$. Note the role which inversion plays here to get the order right. For $z \equiv \mathrm{sh}_G$, also this abs(G)-set can be identified with the principal abs(G)-torsor, but not definitionally, and one has to use Exercise 7.4.2 for the identification. Therefore we prefer the version in the main text.

def:qG-concr-abstr

Definition 7.4.6. Let G be a group. The group homomorphism

$$q_G : \mathrm{Hom}(G, \mathrm{concr}(\mathrm{abs}(G)))$$

is classified by the function Bq_G defined in Example 7.4.5. ⌟

lem:Groupsareidentitytypes

Lemma 7.4.7. *For all groups G, the homomorphism q_G is an isomorphism.*

Proof. To prove that Bq_G is an equivalence it is, by Corollary 2.17.9(3), enough to show that for $x, y : BG$ the induced map

$$Bq_G : (x \xrightarrow{=}_{BG} y) \to (Bq_G(x) \xrightarrow{=} Bq_G(y))$$

is an equivalence. Now, $Bq_G(x) \xrightarrow{=} Bq_G(y)$ can be unfolded to

$$((x \xrightarrow{=} \mathrm{sh}_G), \mathrm{post}_x) \xrightarrow{=}_{\mathcal{G}\text{-}\mathrm{Set}^{\mathrm{abs}}} ((y \xrightarrow{=} \mathrm{sh}_G), \mathrm{post}_y).$$

By Definition 2.7.3 and Lemma 2.10.3, this is equivalent to

$$\sum_{f : (x \xrightarrow{=} \mathrm{sh}_G) \xrightarrow{\simeq} (y \xrightarrow{=} \mathrm{sh}_G)} \prod_{g : UG} \big(f \circ (\mathrm{post}_x(g)) = (\mathrm{post}_y(g)) \circ f\big).$$

The inner equation in the latter type is visualized by commutativity of the diagram in Figure 7.1.

Under the above identifications, $Bq_G(r)$ is given, for any $r : x \xrightarrow{=} y$, by $f :\equiv \mathrm{preinv}_{\mathrm{sh}_G}(r)$ together with a proof by associativity of the proposition

$$P(f) :\equiv \prod_{g : UG} \prod_{p : x \xrightarrow{=} \mathrm{sh}_G} \big(f(gp) = g\,f(p)\big).$$

$$\begin{array}{ccc} (x \xrightarrow{=} \mathrm{sh}_G) & \xrightarrow[\sim]{g\cdot_} & (x \xrightarrow{=} \mathrm{sh}_G) \\ f \big\downarrow \wr & & \wr \big\downarrow f \\ (y \xrightarrow{=} \mathrm{sh}_G) & \xrightarrow[g\cdot_]{\sim} & (y \xrightarrow{=} \mathrm{sh}_G) \end{array}$$

Figure 7.1: Equivalence f commutes with post$_(g)$; commutation for all $g : UG$ is abbreviated by $P(f)$.

Now, given an equivalence $f : (x \xrightarrow{=} \mathrm{sh}_G) \xrightarrow{\simeq} (y \xrightarrow{=} \mathrm{sh}_G)$ such that $P(f)$ holds, the fiber $Bq_G^{-1}(f)$ can be identified with $\sum_{r : x \xrightarrow{=} y}(f = \mathrm{preinv}_{\mathrm{sh}_G}(r))$. For proving that Bq_G is an equivalence, we have to show that the latter fiber is contractible. This goal is a proposition and BG is connected, so we may assume that we have a path $p_0 : x \xrightarrow{=} \mathrm{sh}_G$. Then any $r, s : x \xrightarrow{=} y$ such that $\mathrm{preinv}_{\mathrm{sh}_G}(r) = f = \mathrm{preinv}_{\mathrm{sh}_G}(s)$ satisfy $p_0 r^{-1} = f(p_0) = p_0 s^{-1}$, so that $r = s$. Thus the preimage is a proposition. It remains to find an r such that $f = \mathrm{preinv}_{\mathrm{sh}_G}(r)$. We take $r = f(p_0)^{-1}p_0$ and verify for any $p : x \xrightarrow{=} \mathrm{sh}_G$, using property $P(f)$ of f with $g \equiv (p p_0^{-1})$, that

$$f(p) = f(gp_0) = g\,f(p_0) = p(p_0^{-1} f(p_0)) = pr^{-1} = \mathrm{preinv}_{\mathrm{sh}_G}(r)(p). \quad \square$$

We are now ready to prove the main result of this section.

thm:Groupsareidentitytypes

THEOREM 7.4.8. *The map* $\mathrm{abs} : \mathrm{Group} \to \mathrm{Group}^{\mathrm{abs}}$ *is an equivalence.*

Proof. Applying Construction 2.9.9 with Definition 7.4.4 as candidate inverse, one half of the the work has been done in Lemma 7.4.7. It remains to give, for any abstract group $\mathcal{G}$, an abstract isomorphism of type

$$\mathcal{G} \xrightarrow{\cong}_{\mathrm{Group}^{\mathrm{abs}}} \mathrm{abs}(\mathrm{concr}(\mathcal{G})).$$

Let $\mathcal{G} = (S, e, \mu, \iota)$ be an abstract group. Then the underlying set of $\mathrm{abs}(\mathrm{concr}(\mathcal{G}))$ is $\mathbb{P}^{\mathrm{abs}}_{\mathcal{G}} \xrightarrow{\simeq}_{\mathrm{Torsor}^{\mathrm{abs}}_{\mathcal{G}}} \mathbb{P}^{\mathrm{abs}}_{\mathcal{G}}$. Unraveling the definitions and using Definition 2.7.3, we see that this set is equivalent to

$$\sum_{\pi : S \xrightarrow{\simeq} S} \prod_{s,t : S} \big(\pi(\mu(s,t)) = \mu(s, \pi(t))\big).$$

Setting $t :\equiv e$ in the last equation, we see that $\pi(s) = \mu(s, \pi(e))$, that is, π is simply right multiplication with an element $\pi(e) : S$. In other words, the function

$$r_{\mathcal{G}} : S \to \sum_{\pi : S \xrightarrow{\simeq} S} \prod_{s,t : S} \big(\pi(\mu(s,t)) = \mu(s, \pi(t))\big), \qquad r_{\mathcal{G}}(u) :\equiv (\mu(_, u^{-1}), !)$$

is an equivalence of sets.[19]

We have to promote $r_{\mathcal{G}}$ from an equivalence of sets to an isomorphism of abstract groups, with $\mathcal{G}$ as domain. The codomain of $r_{\mathcal{G}}$ has its abstract group structure induced by the equivalence with $\mathrm{abs}(\mathrm{concr}(\mathcal{G}))$. The abstract group structure of $\mathrm{abs}(\mathrm{concr}(\mathcal{G}))$ is given by the symmetries of $\mathbb{P}^{\mathrm{abs}}_{\mathcal{G}}$; translated to the codomain $\sum_{\pi : S \xrightarrow{\simeq} S} \prod_{s,t : S} \big(\pi(\mu(s,t)) = \mu(s, \pi(t))\big)$ this corresponds via the first projection to a subset of permutations of S, with the abstract group structure given by composition $\circ$. In view of Definition 7.3.2, for $r_{\mathcal{G}}$ to be an abstract isomorphism, it suffices that $r_{\mathcal{G}}$ preserves multiplication: $r_{\mathcal{G}}(\mu(u,v)) = r_{\mathcal{G}}(u) \circ r_{\mathcal{G}}(v)$. This follows directly from function extensionality, associativity of μ, and the equation $\mu(u,v)^{-1} = \mu(v^{-1}, u^{-1})$. Hence the equivalence $r_{\mathcal{G}}$ is indeed an isomorphism of abstract groups.[20] □

[19] Indeed, conversely, $\mu(_, u^{-1})$ satisfies the condition for π. Prove this! The reason for using u^{-1} here, and not u, becomes clear in the next paragraph, cf. Footnote 18.

[20] This amounts to Cayley's Theorem for abstract groups, stating that every abstract group $\mathcal{G}$ is isomorphic to an abstract subgroup of the abstract permutation group of the underlying set S of $\mathcal{G}$. The abstract subgroup is the codomain of $r_{\mathcal{G}}$ with id_S, $\circ$ and $(_)^{-1}$.

ft:abstract-Cayley

7.4.9 *The left and right adjoint of restriction, abstractly*

sec:phi-functors

In Definition 7.4.1 we defined the notion of $\mathcal{G}$-set for any abstract group $\mathcal{G}$. A natural question that comes up is: what are the abstract versions of the string of adjunctions $f_! \dashv f^* \dashv f_*$ defined for concrete groups in Definition 5.5.9 and Remark 5.5.13?[21] We answer this question here; the precise correspondence of the abstract and the concrete versions will come in Section 7.7.

[21] See also Examples 6.4.5 and 6.5.6 for examples of these functors in the categorical setting.

In the following paragraphs, G and H are groups and $f : \mathrm{Hom}(G, H)$ is a homomorphism.

Recall from Definition 5.5.9 the restriction map

$$f^* : H\text{-Set} \to G\text{-Set}, \qquad f^*(Y) :\equiv Y \circ Bf.$$

The abstract version of restriction, i.e., precompostion, is no surprise:

def:phi1*

DEFINITION 7.4.10. Given abstract groups $\mathcal{G}$ and $\mathcal{H}$ and an abstract group homomorphism $\varphi : \mathrm{Hom}^{\mathrm{abs}}(\mathcal{G}, \mathcal{H})$, we define the map $\varphi^* : \mathcal{H}\text{-Set}^{\mathrm{abs}} \to \mathcal{G}\text{-Set}^{\mathrm{abs}}$ sending any $\mathcal{H}$-set (S, ψ) to the $\mathcal{G}$-set $(S, \psi \circ \varphi)$. ⌟

Also recall from Definition 5.5.9 the left adjoint of f^*:

$$f_! : G\text{-Set} \to H\text{-Set}, \qquad f_!X(w) :\equiv \| \sum_{z:BG} \big((Bf(z) \xrightarrow{=} w) \times X(z)\big) \|_0,$$

for all $X : BG \to \text{Set}$, and $w : BH$. Unlike in the case of f^*, the abstract version of $f_!$ is not immediately clear, so we apply some equivalences first.

By Lemma 5.4.5, the set $f_!X(w)$ is equivalent to the set of orbits of the G-set $(z : BG) \mapsto ((Bf(z) \xrightarrow{=} w) \times X(z))$. Using Bf_{pt}, the underlying set of the latter is equivalent to $\mathbb{P}_{\text{sh}_H}(w) \times X(\text{sh}_G)$. By Corollary 5.4.6, quotienting the latter set with the equivalence relation $(p, x) \sim (q, y)$ defined by $\exists_{g:UG}((pUf(g) = q) \times (x = g \cdot_X y))$ gives an equivalent of $f_!X(w)$ in terms that have abstract counterparts.[22] This motivates the following definition.

[22] The dependence on w is in the type $\mathbb{P}_{\text{sh}_H}(w)$ of p and q.

def:phi_!

Definition 7.4.11. Let $\mathcal{G}$ and $\mathcal{H}$ be abstract groups with underlying sets S and T, respectively. Let $\varphi : \text{Hom}^{\text{abs}}(\mathcal{G}, \mathcal{H})$ be an abstract homomorphism. Define (and justify by doing Exercise 7.4.12!)

$$\varphi_! : \mathcal{G}\text{-Set}^{\text{abs}} \to \mathcal{H}\text{-Set}^{\text{abs}}, \quad \varphi_!(\mathcal{X}) :\equiv \mathbb{P}^{\text{abs}}_{\mathcal{H}} \times_{\mathcal{G}} \mathcal{X} \text{ for all } \mathcal{G}\text{-sets } \mathcal{X}.$$

Here $\mathbb{P}^{\text{abs}}_{\mathcal{H}} \times_{\mathcal{G}} \mathcal{X}$, with $\mathcal{X} \equiv (X, \cdot_X)$, is the $\mathcal{H}$-set consisting of the set quotient $T \times X/\sim$ for the equivalence relation on $T \times X$ defined by

$$(t, x) \sim (u, y) :\equiv \exists_{g:S} (t\varphi(g) = u) \times (x = g \cdot_X y),$$

paired with the abstract homomorphism that maps any $h : T$ to the permutation of $T \times X/\sim$ induced by $(t, x) \mapsto (ht, x)$. ⌟

xca:phi_!-0K

Exercise 7.4.12. With conditions as in Definition 7.4.11, show the following:

(1) $\sim$ is an equivalence relation;

(2) $(t, x) \mapsto (ht, x)$ respects $\sim$;

(3) $(t, x) \mapsto (ht, x)$ induces an abstract homomorphism. ⌟

Like maps of G-sets in Definition 5.2.8, we have a notion of map of $\mathcal{G}$-sets:

def:Hom-absG

Definition 7.4.13. Let $\mathcal{G}$ be an abstract group with underlying set S, and let $\mathcal{X} \equiv (X, \cdot_X)$ and $\mathcal{Y} \equiv (Y, \cdot_Y)$ be $\mathcal{G}$-sets. Then the type of maps from $\mathcal{X}$ to $\mathcal{Y}$ is defined as

$$\text{Hom}_{\mathcal{G}}(\mathcal{X}, \mathcal{Y}) :\equiv \sum_{f : X \to Y} \prod_{s:S} \big(f \circ (s \cdot_X _) = (s \cdot_Y _) \circ f\big). \qquad ⌟$$

The following exercise asks you to prove $\varphi_! \dashv \varphi^*$, which is the abstract version of the adjunction $f_! \dashv f^*$ from Example 6.4.5.[23]

[23] See Section 7.6 for how to obtain this result once the equivalence between the categories Group and Group^{abs} has been established.

xca:phi_!-|-phi^*

Exercise 7.4.14. Let $\mathcal{G}$ and $\mathcal{H}$ be abstract groups and $\varphi : \text{Hom}^{\text{abs}}(\mathcal{G}, \mathcal{H})$ an abstract homomorphism. Give an equivalence between $\text{Hom}_{\mathcal{H}}(\varphi_!\mathcal{X}, \mathcal{Y})$ and $\text{Hom}_{\mathcal{G}}(\mathcal{X}, \varphi^*\mathcal{Y})$ for all $\mathcal{G}$-sets $\mathcal{X}$ and $\mathcal{H}$-sets $\mathcal{Y}$. ⌟

For the other adjunction, $\varphi^* \dashv \varphi_*$, we must first find a suitable definition of $\varphi_*\mathcal{X}$. To get an idea, we recall f_*X defined in Remark 5.5.13:

$$f_* : G\text{-Set} \to H\text{-Set}, \quad f_*X(w) \equiv \prod_{z:BG} \big((Bf(z) \xrightarrow{=} w) \to X(z)\big), \text{for } w : BH.$$

Following Remark 5.2.14, we look at the underlying set $f_*X(\mathrm{sh}_H)$ of the H-set f_*X. This underlying set $\prod_{z:BG}\big((Bf(z) \xrightarrow{=} \mathrm{sh}_H) \to X(z)\big)$ is a set of invariant maps for the G-set sending $z : BG$ to $(Bf(z) \xrightarrow{=} \mathrm{sh}_H) \to X(z)$. Exercise 5.4.23 gives an equivalence between $f_*X(\mathrm{sh}_H)$ and the subset of fixed elements of $(Bf(\mathrm{sh}_G) \xrightarrow{=} \mathrm{sh}_H) \to X(\mathrm{sh}_G)$. This provides the clue for the definition of $\varphi_*\mathcal{X}$. However, since φ corresponds to $Uf \equiv \mathrm{abs}(f)$, we must transport $Bf(\mathrm{sh}_G) \xrightarrow{=} \mathrm{sh}_H$ to UH by precomposition with Bf_{pt}. The following diagram[24] shows how to transport a fixed $a : (Bf(\mathrm{sh}_G) \xrightarrow{=} \mathrm{sh}_H) \to X(\mathrm{sh}_G)$ to an element $a(_Bf_{\mathrm{pt}}^{-1}) : UH \to X(\mathrm{sh}_G)$ and why the latter function is also fixed:

$$\begin{array}{ccccc}
UH & \xleftarrow[\sim]{_Bf_{\mathrm{pt}}} & (Bf(\mathrm{sh}_G) = \mathrm{sh}_H) & \xrightarrow{a} & X(\mathrm{sh}_G) \\
{\scriptstyle _Uf(g^{-1})}\downarrow\wr & & \wr\downarrow{\scriptstyle _Bf(g^{-1})} & & \wr\downarrow{\scriptstyle X(g)} \\
UH & \xleftarrow[_Bf_{\mathrm{pt}}]{\sim} & (Bf(\mathrm{sh}_G) = \mathrm{sh}_H) & \xrightarrow[a]{} & X(\mathrm{sh}_G)
\end{array}$$

It remains to understand the action of f_*X under the above equivalences. Taking into account how w occurs in $f_*X(w)$,[25] one sees that the action of h maps any $a : (Bf(\mathrm{sh}_G) \xrightarrow{=} \mathrm{sh}_H) \to X(\mathrm{sh}_G)$ to $a(h^{-1}_)$.[26] The same action works for functions $UH \to X(\mathrm{sh}_G)$, since $(h^{-1}_)Bf_{\mathrm{pt}} = h^{-1}(_Bf_{\mathrm{pt}})$. We are now ready to define φ_*.

def:phi_*

Definition 7.4.15. Let $\mathcal{G}$ and $\mathcal{H}$ be abstract groups with underlying sets S and T, respectively. Let $\varphi : \mathrm{Hom}^{\mathrm{abs}}(\mathcal{G}, \mathcal{H})$ be an abstract homomorphism. Define $\varphi_* : \mathcal{G}\text{-Set}^{\mathrm{abs}} \to \mathcal{H}\text{-Set}^{\mathrm{abs}}$ by mapping every $\mathcal{G}$-set $\mathcal{X} \equiv (X, \cdot_X)$ to

$$\varphi_*(\mathcal{X}) :\equiv \{b : T \to X \mid \prod_{s:S}\big(b \circ (_\varphi(s^{-1})) = (s \cdot_X _) \circ b\big)\},$$

with action $h \cdot_{T\to X} b$ defined as $b(h^{-1} \cdot_X _)$. ⌟

The following exercise asks you to prove $\varphi^* \dashv \varphi_*$, which is the abstract version of the adjunction $f^* \dashv f_*$ from Example 6.4.5.[27]

xca:phi1a*_|phi_*

Exercise 7.4.16. Let $\mathcal{G}$ and $\mathcal{H}$ be abstract groups and $\varphi : \mathrm{Hom}^{\mathrm{abs}}(\mathcal{G}, \mathcal{H})$ an abstract homomorphism. Give an equivalence between $\mathrm{Hom}_{\mathcal{G}}(\varphi^*\mathcal{Y}, \mathcal{X})$ and $\mathrm{Hom}_{\mathcal{H}}(\mathcal{Y}, \varphi_*\mathcal{X})$ for all $\mathcal{G}$-sets $\mathcal{X}$ and $\mathcal{H}$-sets $\mathcal{Y}$. ⌟

7.5 *Homomorphisms, from abstract to concrete and back*

sec:homabsisconcr

Now that we know how to identify the type of groups with the type of abstract groups, it is natural to ask if the respective notions of group homomorphism also coincide.

They do, and we provide two independent and somewhat different arguments. Translating from group homomorphisms to abstract group homomorphisms is easy: if G and H are groups, then we defined

$$\mathrm{abs} : \mathrm{Hom}(G, H) \to \mathrm{Hom}^{\mathrm{abs}}(\mathrm{abs}(G), \mathrm{abs}(H))$$

in Definition 4.4.5 and Definition 7.3.2 as the function which takes a homomorphism, classified by a pointed map $Bf : BG \to_* BH$, to the induced map of identity types

$$Uf \equiv \Omega Bf : UG \to UH$$

together with the proof that this is an abstract group homomorphism from $\mathrm{abs}(G)$ to $\mathrm{abs}(H)$.

[24] Here $g : UG$ is inverted in $Bf(g^{-1})$, $Uf(g^{-1})$ as $Bf(\mathrm{sh}_G)$ occurs on the left in $Bf(\mathrm{sh}_G) \xrightarrow{=} \mathrm{sh}_H$.

[25] Namely, as the endpoint of a path that is the domain of a function type, i.e., a negative or contravariant position.

[26] We don't have to worry about the latter being fixed whenever a is, since everything comes from the H-set f_*X. However, you may check this as an exercise.

[27] Again, see Section 7.6 for a categorical approach.

Going back is somewhat more involved, and it is here we consider two approaches. The first is a compact argument showing directly how to reconstruct a pointed map $Bf : BG \to_* BH$ from an abstract group homomorphism from $\mathrm{abs}(G)$ to $\mathrm{abs}(H)$. The second approach offers a direct equivalence between the categories Group and $\mathrm{Group}^{\mathrm{abs}}$.

"Delooping" a group homomorphism

sec:delooping

We now explore the first approach. It might be helpful to review Lemma 3.4.9 for a simple example of delooping in the special case of the circle. Here we elaborate the general case.

lem:homomabsrrconcr

LEMMA 7.5.1. *If G and H are groups, then*

$$\mathrm{abs} : \mathrm{Hom}(G,H) \to \mathrm{Hom}^{\mathrm{abs}}(\mathrm{abs}(G), \mathrm{abs}(H))$$

is an equivalence.

Proof. Suppose we are given an abstract group homomorphism

$$f : \mathrm{Hom}^{\mathrm{abs}}(\mathrm{abs}(G), \mathrm{abs}(H))$$

and we explain how to build a map $Bg : BG \to BH$ with a path $p : \mathrm{sh}_H \xrightarrow{=} Bg(\mathrm{sh}_G)$ such that $pf(\omega) = Bg(\omega)p$ for all $\omega : \mathrm{sh}_G \xrightarrow{=} \mathrm{sh}_G$ (so that $g : \mathrm{Hom}(G,H)$ is a "delooping" of f, that is, $f = \mathrm{abs}(g)$).[28]

To get an idea of our strategy, let us assume the problem solved. The map $Bg : BG \to BH$ will then send any path $\alpha : \mathrm{sh}_G \xrightarrow{=} x$ to a path $Bg(\alpha) : Bg(\mathrm{sh}_G) \xrightarrow{=} Bg(x)$ and so we get a family of paths $p(\alpha) :\equiv Bg(\alpha)p$ in $\mathrm{sh}_H \xrightarrow{=} Bg(x)$ such that

$$p(\alpha\omega) = Bg(\alpha)Bg(\omega)p = Bg(\alpha)pf(\omega) = p(\alpha)f(\omega)$$

for all $\omega : \mathrm{sh}_G \xrightarrow{=} \mathrm{sh}_G$ and $\alpha : \mathrm{sh}_G \xrightarrow{=} x$.

This suggests to introduce the following family

$$C(x) :\equiv \sum_{y:BH} \sum_{p:(\mathrm{sh}_G \xrightarrow{=} x) \to (\mathrm{sh}_H \xrightarrow{=} y)} \prod_{\omega:\mathrm{sh}_G \xrightarrow{=} \mathrm{sh}_G} \prod_{\alpha:\mathrm{sh}_G \xrightarrow{=} x} p(\alpha\omega) = p(\alpha)f(\omega)$$

An element of $C(x)$ has three components, the last component being a proposition since BH is a groupoid.

The type $C(\mathrm{sh}_G)$ has a simpler description. An element of $C(\mathrm{sh}_G)$ is a pair y, p such that $p(\alpha\omega) = p(\alpha)f(\omega)$ for any α and ω in $\mathrm{sh}_G \xrightarrow{=} \mathrm{sh}_G$. Since f is an abstract group homomorphism, this condition can be simplified to $p(\omega) = p(\mathrm{refl}_{\mathrm{sh}_G})f(\omega)$, and the map p is completely determined by $p(\mathrm{refl}_{\mathrm{sh}_G})$. Thus $C(\mathrm{sh}_G)$ is equal to $\sum_{y:BH} \mathrm{sh}_H \xrightarrow{=} y$ and is contractible. Since BG is connected, we have $\prod_{x:BG} \mathrm{isContr}\ C(x)$ and so, in particular, we have an element of $\prod_{x:BG} C(x)$.

By projecting out the centers we get a map $Bg : BG \to BH$ together with a map $p : (\mathrm{sh}_G \xrightarrow{=} x) \to (\mathrm{sh}_H \xrightarrow{=} Bg(x))$ such that $p(\alpha\omega) = p(\alpha)f(\omega)$ for all α in $\mathrm{sh}_G \xrightarrow{=} x$ and ω in $\mathrm{sh}_G \xrightarrow{=} \mathrm{sh}_G$. We have, for $\alpha : \mathrm{sh}_G \xrightarrow{=} x$

$$\prod_{x':BG} \prod_{\lambda : x \xrightarrow{=} x'} p(\lambda\alpha) = Bg(\lambda)p(\alpha)$$

since this holds for $\lambda = \mathrm{refl}_x$. In particular, $p(\omega) = Bg(\omega)p(\mathrm{refl}_{\mathrm{sh}_G})$.

We also have $p(\omega) = p(\mathrm{refl}_{\mathrm{sh}_G})f(\omega)$, hence $p(\mathrm{refl}_{\mathrm{sh}_G})Bg(\alpha) = f(\alpha)p(\mathrm{refl}_{\mathrm{sh}_G})$ for all $\alpha : \mathrm{sh}_G \xrightarrow{=} \mathrm{sh}_G$ and we have found a delooping of f. □

[28] We will thus have displayed a map $\mathrm{deloop} : \mathrm{Hom}^{\mathrm{abs}}(\mathrm{abs}(G), \mathrm{abs}(H)) \to \mathrm{Hom}(G,H)$ with $(\mathrm{abs} \circ \mathrm{deloop}) = \mathrm{id}$. We leave it to the reader to prove that $\mathrm{deloop} \circ \mathrm{abs} = \mathrm{id}$.

Direct equivalence of categories

We now define a functor

$$\mathrm{concr} : \mathrm{Group}^{\mathrm{abs}} \to \mathrm{Group}$$

which we prove directly is an inverse functor to $\mathrm{abs} : \mathrm{Group} \to \mathrm{Group}^{\mathrm{abs}}$.

On objects, concr is defined via the type of $\mathcal{G}$-torsors from Definition 7.4.1 as $\mathrm{concr}(\mathcal{G}) :\equiv \underline{\Omega}(\mathrm{Torsor}^{\mathrm{abs}}_{\mathcal{G}}, \mathbb{P}^{\mathrm{abs}}_{\mathcal{G}})$. This we proved is the inverse of abs on objects via two identifications $q_G : G \xrightarrow{\cong} \mathrm{concr}(\mathrm{abs}(G))$, from Definition 7.4.6 and Lemma 7.4.7, and $r_{\mathcal{G}} : \mathcal{G} \xrightarrow{\cong} \mathrm{abs}(\mathrm{concr}(\mathcal{G}))$, from Theorem 7.4.8.

To define the action on morphisms, suppose $\mathcal{G}$ and $\mathcal{H}$ are abstract groups with underlying sets S and T, respectively. Then concr sends an abstract group homomorphism $\varphi : \mathrm{Hom}^{\mathrm{abs}}(\mathcal{G}, \mathcal{H})$ to the concrete group homomorphism $\mathrm{concr}(\varphi) : \mathrm{Hom}(\mathrm{concr}(\mathcal{G}), \mathrm{concr}(\mathcal{H}))$ classified by the pointed map

$$\mathrm{Bconcr}(\varphi) : \mathrm{Torsor}^{\mathrm{abs}}_{\mathcal{G}} \to_* \mathrm{Torsor}^{\mathrm{abs}}_{\mathcal{H}}$$

defined as follows. Changed First, recall $\varphi_!$ from Definition 7.4.11 as unpointed map from $\mathcal{G}$-sets to $\mathcal{H}$-sets. Second, do Exercise 7.5.2(1) to justify the restriction of $\varphi_!$ to the components at the respective abstract principal torsors. Define the unpointed $\mathrm{Bconcr}(\varphi)$ as the restriction of $\varphi_!$ to these components. Finally, define $\mathrm{Bconcr}(\varphi)_{\mathrm{pt}}$ as the inverse of the path given in Exercise 7.5.2(1).End changed

xca:Bconcr-OK

i:Bconcr_pt

Exercise 7.5.2. Show the following:

(1) $(t, x) \mapsto t\varphi(x)$ induces a path from $\varphi_! \mathbb{P}^{\mathrm{abs}}_{\mathcal{G}}$ to $\mathbb{P}^{\mathrm{abs}}_{\mathcal{H}}$;

(2) concr is a functor from $\mathrm{Group}^{\mathrm{abs}}$ to Group. ⌟

thm:concr-equiv-cats

Theorem 7.5.3. *Thus defined,* concr *is functorial and gives an equivalence of categories.*

Proof. It remains to show that q and r give natural isomorphisms

$$q : \mathrm{id}_{\mathrm{Group}} \xrightarrow{\cong} \mathrm{concr} \circ \mathrm{abs}, \quad r : \mathrm{id}_{\mathrm{Group}^{\mathrm{abs}}} \xrightarrow{\cong} \mathrm{abs} \circ \mathrm{concr}.$$

For naturality of q, consider a homomorphism $f : \mathrm{Hom}(G, H)$ classified by a pointed map $Bf : BG \to_* BH$. We need to show that the square of pointed maps in Figure 7.2 commutes. For $z : BG$, we have

$$\begin{aligned} \mathrm{Bconcr}(\mathrm{abs}(f))(Bq_G(z)) &\equiv \mathbb{P}^{\mathrm{abs}}_{\mathrm{abs}(H)} \times_{\mathrm{abs}(G)} \mathbb{P}_z(\mathrm{sh}_G) \\ &\equiv (\mathrm{sh}_H \xrightarrow{=} \mathrm{sh}_H) \times_{\mathrm{abs}(G)} (z \xrightarrow{=} \mathrm{sh}_G) \end{aligned}$$

and

$$Bq_H(Bf(z)) \equiv \mathbb{P}_{Bf(z)}(\mathrm{sh}_H) \equiv (Bf(z) \xrightarrow{=} \mathrm{sh}_H).$$

Concatenation $(h, p) \mapsto hBf^{-1}_{\mathrm{pt}}Bf(p)$ descends to the quotient as an isomorphism of $\mathrm{abs}(H)$-torsors:[29]

$$c_z : \big((\mathrm{sh}_H \xrightarrow{=} \mathrm{sh}_H) \times_{\mathrm{abs}(G)} (z \xrightarrow{=} \mathrm{sh}_G)\big) \xrightarrow{\cong} (Bf(z) \xrightarrow{=} \mathrm{sh}_H),$$

commuting with the homomorphisms which are both based on left multiplication. And for $z \equiv \mathrm{sh}_G$, this agrees with our chosen pointing paths, see Figure 7.2.[30]

[29] If $hUf(g) = h'$ and $p = gp'$ for some (unique) $g : UG$, then we have:

$$h'Bf^{-1}_{\mathrm{pt}}Bf(p') = hUf(g)Bf^{-1}_{\mathrm{pt}}Bf(p') = hBf^{-1}_{\mathrm{pt}}Bf(g)Bf(p') = hBf^{-1}_{\mathrm{pt}}Bf(p).$$

[30] Indeed, $Bq(Bf_{\mathrm{pt}})$ in Figure 7.2 is right multiplication by Bf^{-1}_{pt}, which makes up for the difference between Exercise 7.5.2(1) and the function defining c_{sh_G}. The dotted second components, which are the homomorphisms, give rise to propositional data in the paths and can safely be ignored.

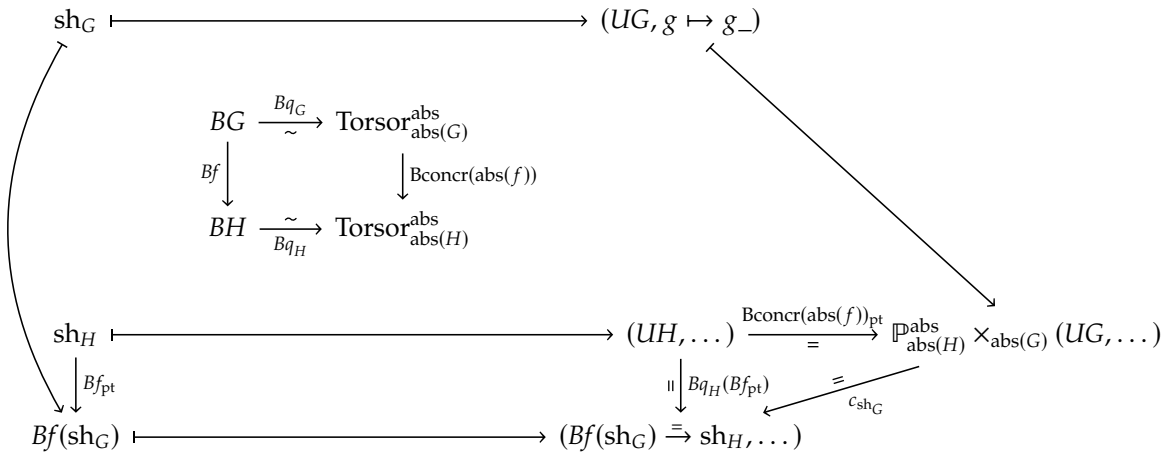


Figure 7.2: Commuting inner square of maps and lower right triangle of pointing paths.

fig:q-natural

For naturality of r, consider an abstract group homomorphism $\varphi : \mathcal{G} \to \mathcal{H}$, where again S and T are the underlying sets of $\mathcal{G}$ and $\mathcal{H}$, respectively. Recall from Theorem 7.4.8 that $r(s)$ is right multiplication by s^{-1}. It suffices to check that the square of underlying sets commutes:

$$\begin{array}{ccc} S & \xrightarrow[\sim]{r_{\mathcal{G}}} & (\mathbb{P}^{\mathrm{abs}}_{\mathcal{G}} \overset{=}{\to} \mathbb{P}^{\mathrm{abs}}_{\mathcal{G}}) \\ {\scriptstyle\varphi}\downarrow & & \downarrow{\scriptstyle \mathrm{abs}(\mathrm{concr}(\varphi))} \\ T & \xrightarrow[\sim]{r_{\mathcal{H}}} & (\mathbb{P}^{\mathrm{abs}}_{\mathcal{H}} \overset{=}{\to} \mathbb{P}^{\mathrm{abs}}_{\mathcal{H}}) \end{array}$$

For $s : S$, this amounts to showing that $\mathrm{abs}(\mathrm{concr}(\varphi))$ maps $\mu_{\mathcal{G}}(_, s^{-1})$ to $\mu_{\mathcal{H}}(_, \varphi(s)^{-1})$.[31] For this we have to show that the following square commutes, where the left isomorphism is $r_{\mathcal{H}}(\varphi(s))$, and the right–down–left detour is $\mathrm{abs}(\mathrm{concr}(\varphi))(r_{\mathcal{G}}(s))$:

$$\begin{array}{ccc} \mathbb{P}^{\mathrm{abs}}_{\mathcal{H}} & \xrightarrow[=]{\mathrm{Bconcr}(\varphi)_{\mathrm{pt}}} & \mathbb{P}^{\mathrm{abs}}_{\mathcal{H}} \times_{\mathcal{G}} \mathbb{P}^{\mathrm{abs}}_{\mathcal{G}} \\ {\scriptstyle \mu_{\mathcal{H}}(_,\varphi(s)^{-1})}\downarrow\wr & & \wr\downarrow{\scriptstyle \mathrm{id}\times_{\mathcal{G}}\mu_{\mathcal{G}}(_,s^{-1})} \\ \mathbb{P}^{\mathrm{abs}}_{\mathcal{H}} & \xrightarrow[\mathrm{Bconcr}(\varphi)_{\mathrm{pt}}]{=} & \mathbb{P}^{\mathrm{abs}}_{\mathcal{H}} \times_{\mathcal{G}} \mathbb{P}^{\mathrm{abs}}_{\mathcal{G}}. \end{array}$$

The pointing path of $\mathrm{Bconcr}(\varphi)$ corresponds with the inverse of the map in Exercise 7.5.2(1), and maps t to $[(t, e_{\mathcal{G}})]$. Hence the square commutes since $(t, s^{-1}) \sim (t\varphi(s)^{-1}, e_{\mathcal{G}})$. □

As a corollary, we get a different proof of Lemma 7.5.1 by applying Lemma 6.7.3 to the above theorem.

[31] The application of $\mathrm{Bconcr}(\varphi)$ on an identification $e : X \overset{=}{\to} X'$ can be analyzed stepwise. First, between the products $\mathbb{P}^{\mathrm{abs}}_{\mathcal{H}} \times X$ and $\mathbb{P}^{\mathrm{abs}}_{\mathcal{H}} \times X'$ we get the equivalence $\mathrm{id}_{\mathbb{P}^{\mathrm{abs}}_{\mathcal{H}}} \times e$.

Second, concerning the quotients modulo $\sim$ and $\sim'$, respectively, we observe that $x = gy$ in X and $e(x) = ge(y)$ in X' are equivalent, as the respective homomorphisms of X and X' correspond via e. All this means that we can completely focus on the sets.

7.6 *Benefits from having the categorical equivalence*

sec:cat-equiv

Exercise 7.6.1. Show that $\mathrm{Iso}(\Sigma_2, C_2)$ is contractible. ⌟

xca:SG2-SG2-contractible

Equivalence of monos in both categories.

Exercises 7.4.14 and 7.4.16 follow from Exercises 5.5.11 and 5.5.15, resp.

7.7 *Actions, from abstract to concrete and back*

sec:Gsetsabstrconcr

Given a group G it should by now come as no surprise that the type of G-sets is equivalent to the type of abs(G)-sets. As explained in the introduction to Section 7.4, just above Definition 7.4.1, G-sets are closely connected to homomorphisms from G to a permutation group. According to Lemma 7.5.1

$$\mathrm{abs} : \mathrm{Hom}(G, \Sigma_S) \to \mathrm{Hom}^{\mathrm{abs}}(\mathrm{abs}(G), \mathrm{abs}(\Sigma_S))$$

is an equivalence, where the group Σ_S is classified by the component of the groupoid Set, pointed at S. The component information is moot by Exercise 5.2.13.

Using Remark 5.2.14, we have the following chain of known equivalences and definitions:

$$\begin{aligned} G\text{-Set} &\xrightarrow{\simeq} \sum_{S:\mathrm{Set}} \mathrm{Hom}(G, \Sigma_S) \\ &\xrightarrow{\simeq} \sum_{S:\mathrm{Set}} \mathrm{Hom}^{\mathrm{abs}}(\mathrm{abs}(G), \mathrm{abs}(\Sigma_S)) \\ &\equiv \mathrm{abs}(G)\text{-Set}^{\mathrm{abs}}. \end{aligned}$$

Backtracking these equivalences we see that we have established

lem:actionsconcr2abstr

Lemma 7.7.1. *Let G be a group. Then the map*

$$\mathrm{ev}_{\mathrm{sh}_G} : G\text{-Set} \to \mathrm{abs}(G)\text{-Set}^{\mathrm{abs}}, \qquad \mathrm{ev}_{\mathrm{sh}_G}(X) :\equiv (X(\mathrm{sh}_G), a_X)$$

is an equivalence, where the abstract homomorphism a_X from $\mathrm{abs}(G) \equiv \mathcal{U}G$ *to* $\mathrm{abs}(\Sigma_{X(\mathrm{sh}_G)}) \equiv (X(\mathrm{sh}_G) \xrightarrow{=} X(\mathrm{sh}_G))$ *is given by the group action of X: $a_X(g) :\equiv X(g) \equiv (g \cdot_X _)$, for all $g : \mathcal{U}G$.*

ex:abstrandconj

Example 7.7.2. Let H and G be groups. Recall from Example 5.2.6 that the set of homomorphisms from H to G is a G-set in a natural way:

$$\mathrm{Hom}(H, G) : BG \to \mathrm{Set}, \quad \mathrm{Hom}(H, G)(z) :\equiv \mathrm{Hom}(H, \underline{\Omega}(BG_{\div}, z))$$

What abstract abs(G)-set does this correspond to? In particular, under the equivalence $\mathrm{abs} : \mathrm{Hom}(H, G) \to \mathrm{Hom}^{\mathrm{abs}}(\mathrm{abs}(H)\, \mathrm{abs}(G))$, what is the corresponding action of abs(G) on the abstract homomorphisms? The answer is that $g : \mathcal{U}G$ acts on $\mathrm{Hom}^{\mathrm{abs}}(\mathrm{abs}(H), \mathrm{abs}(G))$ by postcomposing with conjugation conj^g by g as defined in Example 7.3.5.

Let us spell this out in some detail. Consider a path $p : \mathrm{sh}_G \xrightarrow{=} z$. Transport along p in the family $\mathrm{Hom}(H, G)(z)$ is postcomposing a homomorphism in $\mathrm{Hom}(H, G)$ with the isomorphism $\underline{\Omega}(\mathrm{id}_{BG}, p^{-1}) : \mathrm{Hom}(G, \underline{\Omega}(BG_{\div}, z))$, see Example 4.4.21. Indeed, postcomposition with $\underline{\Omega}(\mathrm{id}_{BG}, g^{-1})$ is an abstract homomorphism from $\mathcal{U}G$ to the abstract permutation group of the set $\mathrm{Hom}(H, G)$. This answers the first question above. As to the second question, recall from Example 4.4.21 that $\mathrm{abs}(\underline{\Omega}(\mathrm{id}_{BG}, g^{-1})) = \mathrm{conj}^g$. Therefore the action of g on $\mathrm{Hom}^{\mathrm{abs}}(\mathrm{abs}(H), \mathrm{abs}(G))$ is postcomposition with conj^g. ⌟

For reference we list the conclusion of this example as a lemma:

lem:abstrandconj

Lemma 7.7.3. *If H and G are groups, then the equivalence of Lemma 7.7.1 sends the G-set* $\mathrm{Hom}(H, G)$ *to the* abs(G)*-set* $\mathrm{Hom}^{\mathrm{abs}}(\mathrm{abs}(H), \mathrm{abs}(G))$ *with action given by postcomposing with conjugation by elements of* abs(G).

Recall from Section 7.4.9 the adjoint triple $\varphi_! \dashv \varphi^* \dashv \varphi_*$ for a given abstract homomorphism φ. Let G and H be groups and let $f : \mathrm{Hom}(G, H)$ be a homomorphism. Also recall the adjoint triple $f_! \dashv f^* \dashv f_*$ from Definition 5.5.9 and Remark 5.5.13. In the following we address how f^* and $\mathrm{abs}(f)^*$ correspond, and how $f_!$ and $\mathrm{abs}(f)_!$ correspond. Recall the equivalences ev_ from Lemma 7.7.1.

Since f^* and $\mathrm{abs}(f)^*$ are both defined by precomposition, the following diagram can easily be filled, which we leave as an exercise to the reader.

$$\begin{array}{ccc} H\text{-Set} & \xrightarrow[\sim]{\mathrm{ev}_{\mathrm{sh}_H}} & \mathrm{abs}(H)\text{-Set}^{\mathrm{abs}} \\ {\scriptstyle f^*}\downarrow & & \downarrow{\scriptstyle \mathrm{abs}(f)^*} \\ G\text{-Set} & \xrightarrow[\mathrm{ev}_{\mathrm{sh}_G}]{\sim} & \mathrm{abs}(G)\text{-Set}^{\mathrm{abs}}. \end{array}$$

Exercise 7.7.4. Fill the diagram above. ⌟

xca:f*-abs(f)*

For $f_!$ and $\mathrm{abs}(f)_!$ we have a similar diagram. Filling this requires the equivalences preceding Definition 7.4.11 motivating the definition of $\varphi_!$ in view of the definition of $f_!$.

$$\begin{array}{ccc} G\text{-Set} & \xrightarrow[\sim]{\mathrm{ev}_{\mathrm{sh}_G}} & \mathrm{abs}(G)\text{-Set}^{\mathrm{abs}} \\ {\scriptstyle f_!}\downarrow & & \downarrow{\scriptstyle \mathrm{abs}(f)_!} \\ H\text{-Set} & \xrightarrow[\mathrm{ev}_{\mathrm{sh}_H}]{\sim} & \mathrm{abs}(H)\text{-Set}^{\mathrm{abs}}. \end{array}$$

Exercise 7.7.5. Fill the diagram above. ⌟

xca:f_!-abs(f)_!

7.8 *Delooping groups and homomorphisms via the Rezk completion*

sec:Rezk-delooping

7.9 *Heaps* (†)

sec:heaps

Recall that we in Remark 4.2.3 wondered about the status of general identity types $a \xrightarrow{=}_A a'$, for a and a' elements of a groupoid A, as opposed to the more special loop types $a \xrightarrow{=}_A a$. Here we describe the resulting algebraic structure and how it relates to groups.

This section has no implications for the rest of the book, and can thus safely be skipped on a first reading.

We proceed in a fashion entirely analogous to that of Section 4.2, but instead of looking a pointed types, we look at *bipointed types*.

Definition 7.9.1. The type of *bipointed, connected groupoids* is the type

$$\mathcal{U}^{=1}_{**} :\equiv \sum_{A:\mathcal{U}^{=1}} (A \times A). \qquad ⌟$$

def:bipt-com-groupoid

Recall that $\mathcal{U}^{=1}$ is the type of connected groupoids A, and that we also write $A : \mathcal{U}$ for the underlying type. We write $(A, a, a') : \mathcal{U}^{=1}_{**}$ to indicate the two endpoints.

Analogous to the loop type of a pointed type, we have a designated identity type of a bipointed type, where we use the two points as the endpoints of the identifications: We set $\mathrm{I}(A, a, a') :\equiv (a \xrightarrow{=}_A a')$.

def:heap

Definition 7.9.2. The type of *heaps*[32] is a wrapped copy (cf. Section 2.12.8) of the type of bipointed, connected groupoids $\mathcal{U}_{**}^{=1}$,

$$\mathrm{Heap} :\equiv \mathrm{Copy}_{\underline{\mathrm{I}}}(\mathcal{U}_{**}^{=1}),$$

with constructor $\underline{\mathrm{I}} : \mathcal{U}_{**}^{=1} \to \mathrm{Heap}$. ⌟

We call the destructor $\mathrm{B} : \mathrm{Heap} \to \mathcal{U}_{**}^{=1}$, and call BH the *classifying type* of the heap $H \equiv \underline{\mathrm{I}}BH$, just as for groups, and we call the first point in BH is *start shape* of H, and the second point the *end shape* of H.

The identity type construction $\mathrm{I} : \mathcal{U}_{**}^{=1} \to \mathrm{Set}$ induces a map $\mathrm{U} : \mathrm{Heap} \to \mathrm{Set}$, mapping $\underline{\mathrm{I}}X$ to $\mathrm{I}X$. These are the *underlying identifications* of the heaps.

These is an obvious map (indeed a functor) from groups to heaps, given by doubling the point. That is, we keep the classifying type and use the designated shape as both start and end shape of the heap. In fact, this map lifts to the type of heaps with a chosen identification.

Exercise 7.9.3. Define *two* equivalences $l, r : \mathrm{Heap} \xrightarrow{\simeq} \sum_{G:\mathrm{Group}} BG$, and one $c : \mathrm{Group} \xrightarrow{\simeq} \sum_{H:\mathrm{Heap}} UH$. ⌟

xca:group+torsor-heap

Recalling the equivalence between BG and the type of G-torsors from Theorem 5.5.7, we can also say that a heap is the same as a group G together with a G-torsor.[34] It also follows that the type of heaps is a (large) groupoid.

In the other direction, there are *two* obvious maps (functors) from heaps to groups, taking either the start or the end shape to be the designated shape.

Here's an *a priori* different map from heaps to groups: For a heap H, consider all the symmetries of the underlying set of identifications UH that arise as $r \mapsto pq^{-1}r$ for $p, q \in UH$.

Note that (p, q) and (p', q') determine the same symmetry if and only if $pq^{-1} = p'q'^{-1}$, and if and only if $p'^{-1}p = q'^{-1}q$.

For the composition, we have $(p, q)(p', q') = (pq^{-1}p', q') = (p, q'p'^{-1}q)$.

Exercise 7.9.4. Complete the argument that this defines a map from heaps to groups. Can you identify the resulting group with the symmetry group of the start or end shape? How would you change the construction to get the other endpoint? ⌟

Exercise 7.9.5. Show that the symmetry groups of the two endpoints of a heap are *merely* isomorphic.

Define the notion of an *abelian heap*, and show that for abelian heaps, the symmetry groups of the endpoints are (*purely*) isomorphic. ⌟

Now we come to the question of describing the algebraic structure of a heap. Whereas for groups we can define the abstract structure in terms of the reflexivity path and the binary operation of path composition, for heaps, we can define the abstract structure in terms of a *ternary operation*, as envisioned by the following exercise.

xca:heap-variety

Exercise 7.9.6. Fix a set S. Show that the fiber $\mathrm{U}^{-1}(S) \equiv \sum_{H:\mathrm{Heap}}(S \xrightarrow{=} UH)$ is a set.

Now fix in addition a ternary operation $t : S \times S \times S \to S$ on S. Show that the fiber of the map $\mathrm{Heap} \to \sum_{S:\mathrm{Set}}(S \times S \times S \to S)$, mapping H to $(UH, (p, q, r) \mapsto pq^{-1}r)$, at (S, t) is a proposition, and describe this proposition in terms of equations. ⌟

[32] The concept of heap (in the abelian case) was first introduced by Prüfer[33] under the German name *Schar* (swarm/flock). In Anton Sushkevich's book Теория Обобщенных Групп (*Theory of Generalized Groups*, 1937), the Russian term груда (heap) is used in contrast to группа (group). For this reason, a heap is sometimes known as a "groud" in English.

[33] Heinz Prüfer. "Theorie der Abelschen Gruppen". In: *Math. Z.* 20.1 (1924), pp. 165–187. DOI: 10.1007/BF01188079.

[34] But be aware that are *two* such descriptions, according to which endpoint is the designated shape, and which is the "twisted" torsor.

8

Constructing groups

ch:congp

8.1 *Brief overview of the chapter*

8.2 *Semidirect products*

sec:Semidirect-products

In this section we describe a generalization of the product of two groups, recall Example 4.2.26, called the *semidirect* product, which gives us a new way of building new groups. Like the product, both its classifying type and its set of symmetries consist of pairs. It takes as input the action of a group on a group.

Recall from Definition 5.2.28 that an action of a group G *in* groups is a function $H : BG \to \mathrm{Group}$. This acts *on* the group $H(\mathrm{sh}_G)$. If the group being acted on is fixed, say $H : \mathrm{Group}$, then an action of G on H is given by a homomorphism from G to $\mathrm{Aut}(H)$.

def:semidirect-product

Definition 8.2.1. Given a group G and an action $H : BG \to \mathrm{Group}$, we define a group called the *semidirect product* as follows:

$$G \ltimes H :\equiv \underline{\Omega} \sum_{t : BG} BH(t)$$

Here the basepoint of the sum is taken to be the point $(\mathrm{sh}_G, \mathrm{sh}_H)$.[1] ⌟

Observe that if the action of G is trivial, then $H(t) \equiv H(\mathrm{sh}_G)$ for all t, so $G \ltimes H \equiv G \times H(\mathrm{sh}_G)$, reproducing the ordinary product of groups.

Before we study the underlying symmetries in $G \ltimes H$, let us pause to look at some examples. In Example 5.2.30 we saw an action of Σ_2 on C_3. Let us generalize this by giving an action of Σ_2 on the cyclic group C_n for any $n : \mathrm{Order}$. Recall from Definition 3.6.21 that we may represent C_n as the automorphism group of the standard n-cycle $(\mathbb{Z}/\sim_n, \mathrm{s})$, where $z \sim_n z'$ if and only for $t^z = t^{z'}$ for any/all cycles (T, t) of order n.[2]

The idea is that a cycle has two "directions" and Σ_2 can act by swapping them. To implement this, let S be a 2-element set, and consider, as in Example 5.2.30, the type $\sum_{X : \mathrm{Set}}(S \to (X \to X))$. Let $X :\equiv S \times \mathbb{Z}/\sim$ be the set quotient of $S \times \mathbb{Z}$ where $(s, z) \sim (s, z')$ if $z \sim_n z'$ and $(s, z) \sim (s', z')$ if $s \neq s'$ and $z \sim_n -z'$,[3] Now we can define $f : S \to X \to X$ by setting:

$$f_s([(s, z)]) :\equiv [(s, z + 1)], \quad \text{and} \quad f_s([(s', z)]) :\equiv [(s', z - 1)] \quad \text{for } s' \neq s.$$

Note that we can construct the element $x_0 :\equiv [(s, 0)]$ of X unambiguously, as we always have $0 \sim_n -0$.

Exercise 8.2.2. Check that f is well defined, using the universal property of the set quotient, Theorem 2.22.13.

Give an identification of our (X, f) with that of Example 5.2.30 when n is 3. ⌟

[1] We deduce from Lemma 2.15.5(4), that $\sum_{t : BG} BH(t)$ is a groupoid. See Exercise 2.16.11 for a proof that $\sum_{t : BG} BH(t)$ is connected.

[2] Recall also Definition 3.6.20. If n is principal, then either n is infinite and $(\mathbb{Z}, \mathrm{s}) \to (\mathbb{Z}/\sim_n, \mathrm{s})$ is an equivalence, or n is finite, then this map factors through $\mathbb{m}$ to give an equivalence $(\mathbb{m}, \mathrm{s}) \to (\mathbb{Z}/\sim_n, \mathrm{s})$. If LPO (**??** 3.6.22) holds, then every order is principal and these are the only possibilities by Lemma 3.6.25.

[3] Check that $\sim$ defines an equivalence relation.

For any $s : S$, we can let s be the "forwards" direction, and get an identification $(X, f_s) \xrightarrow{\simeq} (\mathrm{Z}/{\sim_n}, \mathrm{s})$ by sending $[(s, z)]$ to $[z]$ (and $[(s', z)]$ to $[-z]$ for $s' \neq s$). Thus, we've constructed an action of Σ_2 on C_n.

def:gen-dihedral

Definition 8.2.3. Given any order $n : \mathrm{Order}$, we define the corresponding *dihedral group of degree n* by $\mathrm{D}_n :\equiv \Sigma_2 \ltimes \tilde{\mathrm{C}}_n$, where $\tilde{\mathrm{C}}_n : \mathrm{B}\Sigma_2 \to \mathrm{Group}$ is the above action of Σ_2 on C_n. ⌟

We shall later see that if n is finite, then D_n is a finite group of cardinality $2n$.[4] But first we need to remedy the potential clash with our previous definition of D_∞ from Definition 4.6.3. Rather than just construct a comparison for the infinite order, we'll do it for all orders, thus also constructing bicycles realizing all dihedral groups.

[4] Since "order" is often used to denote the cardinality of a group, it would be confusing to call D_n the dihedral group of *order* n, although it would match our notion of "order".

con:bidirectional-bicycle

Construction 8.2.4. *For each order $n : \mathrm{Order}$, there is a pointed equivalence*

$$\varphi : \mathrm{BD}_n \xrightarrow{\simeq}_* \mathrm{Bicyc}_{((\mathrm{Z}/{\sim_n} \amalg \mathrm{Z}/{\sim_n}, a, b))'}$$

from the classifying type of the dihedral group of degree n to the connected component of Bicyc *at the* standard dihedral bicycle of degree n, *where*:

$$a(\mathrm{inl}_{[z]}) :\equiv \mathrm{inl}_{[z+1]} \quad b(\mathrm{inl}_{[z]}) :\equiv \mathrm{inr}_{[z]}$$
$$a(\mathrm{inr}_{[z]}) :\equiv \mathrm{inr}_{[z-1]} \quad b(\mathrm{inr}_{[z]}) :\equiv \mathrm{inl}_{[z]}$$

Implementation of Construction 8.2.4. The idea is to think of an element of BD_n, which is a subtype of $\sum_{S : \mathrm{B}\Sigma_2} \sum_{X : \mathrm{Set}} (S \to X \to X)$, as a "bidirectional cycle", from which we can construct a bicycle on two copies of X, more precisely on $S \times X$, as depicted in Figure 8.1. That is, we let $\varphi(S, X, f) :\equiv (S \times X, a, b)$, where

$$a(s, x) :\equiv (s, f_s(x)), \quad b(s, x) :\equiv (\mathrm{swap}(s), x).$$

We identify the image of the base point of BD_n, $(\{\pm 1\}, \mathrm{Z}/{\sim_n}, f)$, where $f_s([z]) = [z + s]$, with the standard dihedral bicycle of degree n by letting $(+1, x) \mapsto \mathrm{inl}_x$ and $(-1, x) \mapsto \mathrm{inr}_x$.

To define the inverse, suppose we have a bicycle (Y, a, b) in the component of the standard dihedral bicycle of degree n. Then we set $S :\equiv Y/a$, the set quotient of Y where we equate y and y' if $y' = a^z(y)$ for some $z : \mathrm{Z}$, i.e., if y and y' are connected by the a equivalence. Then S is a 2-element set, because it is so in the standard case. Similarly, we set $X :\equiv Y/b$, which is merely equivalent to the underlying set of the standard n-cycle, $\mathrm{Z}/{\sim_n}$.

The key observation is now that any equivalence classes $[y]_a : Y/a$ and $[y']_b : Y/b$, thought of a subsets of Y, intersect in unique element $y'' : Y$. In the bottom of Figure 8.1, the two classes in Y/a are the inner and outer 5-cycles, and the 5 classes in Y/b are the 5 pairs linked by doubled bluebell lines. Thus, we can define the corresponding bidirectional cycle to be (S, X, f), where $f_{[y]_a}([y']_b) :\equiv [a(y'')]$. We leave it to the reader to verify that these two constructions are indeed inverse. □

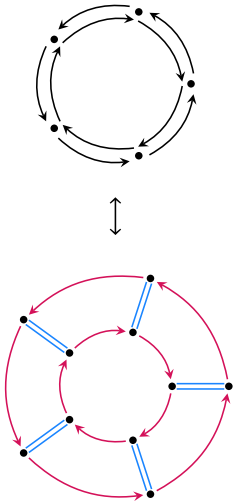


Figure 8.1: How a bidirectional 5-cycle corresponds to a dihedral bicycle of degree 5. The doubled bluebell lines indicate that b swaps the two endpoints.

fig:bidirectional-cycle

Exercise 8.2.5. Complete the verification that two maps in the implementation of Construction 8.2.4 are inverse. ⌟

Thus, since we easily verify that the standard dihedral bicycles are normal (Definition 4.6.5), we see that if n is finite, then D_n has the same cardinality as $n \amalg n$, i.e., $2n$.

EXERCISE 8.2.6. Prove that the two 8-element groups, the quaternion group Q_8 (Definition 4.6.3) and the dihedral group of degree 4, D_4, are not isomorphic.[5] ⌟

[5] Hint: Count elements of order 2.

To better understand the underlying symmetries of a general semidirect product $G \ltimes H$, we note that Lemma 2.10.3 (on paths in Σ-types) takes a simpler form when y and y' are values of a family $x \mapsto f(x)$ of elements of the family $x \mapsto Y(x)$, as the following lemma shows.

lem:pathpairsection

LEMMA 8.2.7. *Suppose we are given a type X and a family of types $Y(x)$ parametrized by the elements x of X. Suppose we are also given a function $f : \prod_{x:X} Y(x)$. For any elements x and x' of X, there is an equivalence of type*

$$\big((x, f(x)) = (x', f(x'))\big) \simeq (x = x') \times (f(x) = f(x)),$$

where the identity type on the left side is between elements of $\sum_{x:X} Y(x)$.

Proof. By Lemma 2.10.3 and by composition of equivalences, it suffices to establish an equivalence of type

$$\left(\sum_{p:x=x'} f(x) \xrightarrow[p]{=} f(x')\right) \simeq (x = x') \times (f(x) = f(x)).$$

Rewriting the right hand side as a sum over a constant family, it suffices to find an equivalence of type

$$\left(\sum_{p:x=x'} f(x) \xrightarrow[p]{=} f(x')\right) \simeq \sum_{p:x=x'} (f(x) = f(x)).$$

By Lemma 2.9.15 it suffices to establish an equivalence of type

$$\left(f(x) \xrightarrow[p]{=} f(x')\right) \simeq (f(x) = f(x))$$

for each $p : x = x'$. By induction on x' and p we reduce to the case where x' is x and p is refl_x, and it suffices to establish an equivalence of type

$$\left(f(x) \xrightarrow[\mathrm{refl}_x]{=} f(x)\right) \simeq (f(x) = f(x)).$$

Now the two sides are equal by definition, so the identity equivalence provides what we need. □

The lemma above shows how to rewrite certain paths between pairs as pairs of paths. Now we wish to establish the formula for composition of paths, rewritten in terms of pairs of paths, but first we introduce a convenient definition for the transport of loops in $Y(x)$ along paths in X.

def:pathsectionaction

DEFINITION 8.2.8. Suppose we are given a type X and a family of types $Y(x)$ parametrized by the elements x of X. Suppose we are also given a function $f : \prod_{x:X} Y(x)$. For any elements x and x' of X and for any identity $p : x = x'$, define a function $(f(x') = f(x')) \to (f(x) = f(x))$, to be denoted by $q' \mapsto q'^p$, by induction on p and x', reducing to the case where x' is x and p is refl_x, allowing us to set $q'^{\mathrm{refl}_x} :\equiv q'$. ⌟

We turn now to associativity for the operation just defined.

def:pathsectionactionassoc

LEMMA 8.2.9. *Suppose we are given a type X and a family of types $Y(x)$ parametrized by the elements x of X. Suppose we are also given a function $f : \prod_{x:X} Y(x)$. For any elements x, x', and x'' of X, for any identities $p : x = x'$ and $p' : x' = x''$, and for any $q : f\,x'' = f\,x''$, there is an identification of type $(q^{p'})^p = q^{(p' \cdot p)}$.*

Proof. By induction on p and p', it suffices to show that $(q^{\mathrm{refl}_y})^{\mathrm{refl}_y} = q^{(\mathrm{refl}_y \cdot \mathrm{refl}_y)}$, in which both sides are equal to q by definition. □

Observe that the operation depends on f, but f is not included as part of the notation.

The next lemma contains the formula we are seeking.

lem:pathpairsectionmult

LEMMA 8.2.10. *Suppose we are given a type X and a family of types $Y(x)$ parametrized by the elements x of X. Suppose we are also given a function $f : \prod_{x:X} Y(x)$. For any elements x, x', and x'' of X, and for any two identities $e : (x, f(x)) = (x', f(x'))$ and $e' : (x', f(x')) = (x'', f(x''))$, if e corresponds to the pair (p, q) with $p : x = x'$ and $q : fx = fx$ under the equivalence of Lemma 8.2.7, and e' corresponds to the pair (p', q') with $p' : x' = x''$ and $q' : fx' = fx'$, then $e' \cdot e$ corresponds to the pair $(p' \cdot p, (q'^{p}) \cdot q)$.*

Proof. By induction on p and p' we reduce to the case where x' and x'' are x and p and p' are refl_x. It now suffices to show that $e' \cdot e$ corresponds to the pair $(\mathrm{refl}_x, q' \cdot q)$. Applying the definition of the map Φ in the proof of Lemma 2.10.3 to our three pairs, we see that it suffices to show that $\left(\mathrm{apap}_g(\mathrm{refl}_x)(q')\right) \cdot \left(\mathrm{apap}_g(\mathrm{refl}_x)(q)\right) = \mathrm{apap}_g(\mathrm{refl}_x)(q' \cdot q)$, with g, as there, being the function $g(x)(y) :\equiv (x, y)$. By Definition 2.7.8 it suffices to show that $\left(\mathrm{ap}_{g(x)}\, q'\right) \cdot \left(\mathrm{ap}_{g(x)}\, q\right) = \mathrm{ap}_{g(x)}\,(q' \cdot q)$, which follows from compatibility of $\mathrm{ap}_{g(x)}$ with composition, as in Construction 2.6.2. □

The lemma above will be applied mostly in the case where x' and x'' are x, but if it had been stated only for that case, we would not have been able to argue by induction on p and p'.

Projection onto the first factor gives a homomorphism $p :\equiv \underline{\Omega}\,\mathrm{fst} : G \ltimes \tilde{H} \to G$. Moreover, there is a homomorphism $s : G \to G \ltimes \tilde{H}$ defined by $s :\equiv \underline{\Omega}\left(t \mapsto (t, \mathrm{sh}_{\tilde{H}(t)})\right)$, for $t : \mathrm{B}G$. The two maps are homomorphisms because they are made from basepoint-preserving maps. The map s is a *section* of p in the sense the $p \circ s = \mathrm{id}_G$. There is also a homomorphism $j : H \to G \ltimes \tilde{H}$ defined by $j :\equiv \underline{\Omega}(u \mapsto (\mathrm{sh}_G, u))$, for $u : \mathrm{B}H$.

LEMMA 8.2.11. *The homomorphism j above is a monomorphism, and it gives the same (normal) subgroup of $G \ltimes \tilde{H}$ as the kernel $\ker p$ of p.*

6

[6] MUST BE MOVED TO THE SUBGROUP CHAPTER

Proof. See 9.3.2 for the definition of kernel. According to Lemma 2.25.1, the map $\mathrm{B}H \to (\mathrm{B}p)^{-1}(\mathrm{sh}_G)$ defined by $u \mapsto ((\mathrm{sh}_G, u), \mathrm{refl}_{\mathrm{sh}_G})$ is an equivalence. This establishes that the fiber $(\mathrm{B}p)^{-1}(\mathrm{sh}_G)$ is connected and thus serves as the classifying type of $\ker p$. Pointing out that the composite map $H \xrightarrow{\cong} \ker p \to G \ltimes \tilde{H}$ is j and using univalence to promote the equivalence to an identity gives the result. □

Our next goal is to present the explicit formula for the multiplication operation in $\mathrm{U}G \ltimes \tilde{H}$. First we apply Lemma 8.2.7 to get a bijection $\mathrm{U}G \ltimes \tilde{H} \simeq UG \times UH$. Now use that to transport the multiplication operation of the group $\mathrm{U}G \ltimes \tilde{H}$ to the set $UG \times UH$. Now Lemma 8.2.10 tells us the formula for that transported operation is given as follows.

$$(p', q') \cdot (p, q) = (p' \cdot p, (q'^{p}) \cdot q)$$

In a traditional algebra course dealing with abstract groups, this formula is used as the definition of the multiplication operation on the set $UG \times UH$,

but then one must prove that the operation satisfies the properties of Definition 4.3.1. The advantage of our approach is that the formula emerges from the underlying logic that governs how composition of paths works.

8.3 *Wreath products*

sec:wreath

A special class of semidirect products are prominent enough to be given special attention: These are the *wreath products*.

Let G and H be groups, and $X : BG \to \text{Set}$ a G-set. Then G acts on the power $H^{X(\text{sh}_G)}$, i.e., the symmetries of the constant function $_ \mapsto \text{sh}_H$ in the function type $X(\text{sh}_G) \to BH$. Indeed, consider the map $H^X : BG \to \text{Group}$ with

$$H^X(z) :\equiv \text{Aut}_{X(z) \to BH}(_ \mapsto \text{sh}_H).$$

Recall that if the underlying set of X is finite, then the classifying space of $H^X(z)$ can be identified with the whole function type $X(z) \to BH$, see Exercise 4.2.29(1).

Definition 8.3.1. With G, H, and X as above, we define the *wreath product of H by G via X* as the semidirect product

$$H \wr_X G :\equiv G \ltimes H^X. \qquad \lrcorner$$

Note that when $X(\text{sh}_G)$ is finite, then the classifying type of $H \wr_X G$ is the type

$$\text{B}(H \wr_X G) \xrightarrow{\simeq} \sum_{z : BG} X(z) \to BH.$$

Example 8.3.2 (The symmetry group of a hypercubes). $\lrcorner$

Example 8.3.3 (Sudoku). $\lrcorner$

Example 8.3.4 (Symmetry groups of trees). $\lrcorner$

8.4 *The pullback*

sec:pullback

Given two functions $f : B \to D$ and $g : C \to D$ with common target, the "pullback" which we will now define should be thought about as the type of all pairs of elements $(b, c) : B \times C$ so that $f(b) = g(c)$. This construction is important in many situations also beyond group theory.

def:pullback

Definition 8.4.1. Let B, C, D be types and let $f : B \to D$ and $g : C \to D$ be two maps. The *pullback* of f and g is the type

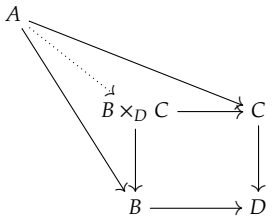
$$\prod(f, g) :\equiv \sum_{(b,c) : B \times C} (f(b) =_D g(c))$$

together with the two projections $\text{pr}_B : \prod(f, g) \to B$ and $\text{pr}_C : \prod(f, g) \to C$ sending $(b, c, p) : \prod(f, g)$ to $b : B$ or $c : C$. If f and g are clear from the context, we may write $B \times_D C$ instead of $\prod(f, g)$ and summarize the situation by the diagram

$$\begin{array}{ccc} B \times_D C & \xrightarrow{\text{pr}_C} & C \\ \downarrow{\scriptstyle \text{pr}_B} & & \downarrow{\scriptstyle g} \\ B & \xrightarrow{f} & D. \end{array}$$

$\lrcorner$

Illustrating the exercise: if the solid diagram commutes there is a unique dotted arrow so that the resulting diagram commutes:



xca:univpropofpullback

EXERCISE 8.4.2. Let $f : B \to D$ and $g : C \to D$ be two maps with common target. If A is a type show that

$$(A \to B) \times_{(A \to D)} (A \to C) \to (A \to B \times_D C)$$
$$(\beta, \gamma, p : f\beta = g\gamma) \mapsto (a \mapsto (f(a), g(a), p(a) : f\beta(a) = g\gamma(a)))$$

is an equivalence. ⌟

In view of Exercise 8.4.2 we will say that we have a *pullback diagram*

$$\begin{array}{ccc} A & \xrightarrow{f'} & C \\ \downarrow{\scriptstyle g'} & & \downarrow{\scriptstyle g} \\ B & \xrightarrow{f} & D \end{array}$$

to indicate that we have an element in $(A \to B) \times_{(A \to D)} (A \to C)$ such that the resulting map $A \to B \times_D C$ is an equivalence.

EXAMPLE 8.4.3. If $g : \mathbb{1} \to D$ has value $d : D$ and $f : B \to D$ is any map, then $\prod(f, g) \equiv B \times_D \mathbb{1}$ is equivalent to the preimage $f^{-1}(d) :\equiv \sum_{b:B} d = f(b)$. ⌟

Preimage as a pullback:

$$\begin{array}{ccc} f^{-1}(d) & \longrightarrow & \mathbb{1} \\ \downarrow & & \downarrow{\scriptstyle d} \\ B & \xrightarrow{f} & D \end{array}$$

ex:pullbackandgcd

EXAMPLE 8.4.4. Much group theory is hidden in the pullback. For instance, the greatest common divisor $\gcd(a, b)$ of $a, b : \mathbb{N}$ is another name for the number of components you get if you pull back the a-fold and the b-fold set bundles of the circle: we have a pullback

$$\begin{array}{ccc} S^1 \times \mathrm{B}C_{\gcd(a,b)} & \longrightarrow & S^1 \\ \downarrow & & \downarrow{\scriptstyle (-)^b} \\ S^1 & \xrightarrow{(-)^a} & S^1 \end{array}$$

(where C_n was the cyclic group of order n). To get a geometric idea, think of the circle as the unit circle in the complex numbers so that the a-fold set bundle is simply taking the a-fold power. With this setup, the pullback should consist of pairs (z_1, z_2) of unit length complex numbers with the property that $z_1^a = z_2^b$. Let $a = a' \gcd(a, b)$ and $b = b' \gcd(a, b)$. Taking an arbitrary unit length complex number z, then the pair $(z^{b'}, z^{a'})$ is in the pull back (since $a'b = ab'$). But so is $(\zeta z^{b'}, z^{a'})$, where ζ is any $\gcd(a, b)^{\text{th}}$ root of unity. Each of the $\gcd(a, b)$-choices of ζ contributes in this way to a component of the pullback. In more detail: identifying the cyclic group $C_{\gcd(a,b)}$ of order $\gcd(a, b)$ with the group of gth roots of unity, the top horizontal map $S^1 \times C_{\gcd(a,b)} \to S^1$ sends (z, ζ) to $z^{a'}$ and the left vertical map sends (z, ζ) to the product $\zeta z^{b'}$.

Also the least common multiple $\mathrm{lcm}(a, b) = a'b$ is hidden in the pullback; in the present example it is demonstrated that the map(s) across the diagram makes each component of the pullback a copy of the $\mathrm{lcm}(a, b)$-fold set bundle. ⌟

def:intersectionand uniomofsets

DEFINITION 8.4.5. Let S be a set and consider two subsets A and B of S given by two families of propositions (for $s : S$) $P(s)$ and $Q(s)$. The *intersection* $A \cap B$ of the two subsets is given by the family of propositions $P(s) \times Q(s)$. The *union* $A \cup B$ is given by the set family of propositions $A(s) + B(s)$. ⌟

xca:intersectionpullbackofsets

EXERCISE 8.4.6. Given two subsets A, B of a set S, prove that

(1) The pullback $A \times_S B$ maps by an equivalence to the intersection $A \cap B$,

(2) If S is finite, then the sum of the cardinalities of A and B is equal to the sum of the cardinalities of $A \cup B$ and $A \cap B$. ⌟

xca:cardinalityintersectionunion

DEFINITION 8.4.7. Let $f : \mathrm{Hom}(H, G)$ and $f' : \mathrm{Hom}(H', G)$ be two homomorphisms with common target. The *pullback* $H \times_G H'$ is the group obtained as the (pointed) component of

def:intersectionofgroups

$$\mathrm{pt}_{H \times_G H'} :\equiv (\mathrm{sh}_H, \mathrm{pt}_{H'}, p_{f'} p_f^{-1})$$

of the pullback $BH \times_{BG} BH'$ (where $p_f : \mathrm{sh}_G = f(\mathrm{sh}_H)$ is the name we chose for the data displaying f as a pointed map, so that $p_{f'} p_f^{-1} : f(\mathrm{sh}_H) = f'(\mathrm{pt}_{H'})$).

If $(H, f, !)$ and $(H', f', !)$ are monomorphisms into G, then the pullback is called the *intersection* and if the context is clear denoted simply $H \cap H'$. ⌟

EXAMPLE 8.4.8. If $a, b : \mathbb{N}$ are natural number with least common multiple L, then $L\mathbb{Z}$ is the intersection $a\mathbb{Z} \cap b\mathbb{Z}$ of the subgroups $a\mathbb{Z}$ and $b\mathbb{Z}$ of $\mathbb{Z}$. ⌟

EXERCISE 8.4.9. Prove that if $f : \mathrm{Hom}(H, G)$ and $f' : \mathrm{Hom}(H', G)$ are homomorphisms, then the pointed version of Exercise 8.4.2 induces an equivalence

$$\mathrm{Hom}(K, H) \times_{\mathrm{Hom}(K,G)} \mathrm{Hom}(K, H') \simeq \mathrm{Hom}(K, H \times_G H')$$

for all groups K and an equivalence

$$UH \times_{UG} UH' \simeq (\mathrm{sh}_{H \times_G H'} = \mathrm{sh}_{H \times_G H'}).$$ [7]

Elevate the last equivalence to a statement about abstract groups. ⌟

REMARK 8.4.10. The pullback is an example of when a construction of types *not* preserving connectivity can be used profitably also for groups. We get the pullback of groups by restricting to a pointed component, but also the other components have group theoretic importance. We will return to this when discussing subgroups. ⌟

8.5 *Pushouts of types*

sec:pushouts

(TBW)

8.6 *Sums of groups*

sec:coprod

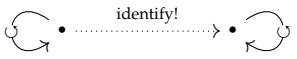
We have seen how the group of integers $\mathbb{Z} = (S^1, \bullet)$ synthesizes the notion of one symmetry with no relations: every symmetry in the circle is of the form $\circlearrowleft^n$ for some unique n. Also, given any group $G = \mathrm{Aut}_A(a)$, the set $a = a$ of symmetries of a corresponds to the set of homomorphisms $\mathbb{Z} \to G$, i.e., to pointed functions $(S^1, \bullet) \to_* (A, a)$ by evaluation at $\circlearrowleft$. What happens if we want to study more than one symmetry at the time?

For instance, is there a group $\mathbb{Z} \vee \mathbb{Z}$ so that for any group $G = \mathrm{Aut}_A(a)$ a homomorphism $\mathbb{Z} \vee \mathbb{Z} \to G$ corresponds to *two* symmetries of a? At the very least, $\mathbb{Z} \vee \mathbb{Z}$ itself would have to have two symmetries and these two can't have any relation, since in a general group $G = \mathrm{Aut}_A(a)$ there is a priori no telling what the relation between the symmetries of a might be. Now, *one* symmetry is given by a pointed function $(S^1, \bullet) \to_* (A, a)$

[7] Hint: set $A :\equiv \mathrm{S}^1$, $B :\equiv BH$, $C :\equiv BH'$ and $D :\equiv BG$.

and so a *pair* of symmetries is given by a function $f : S^1 + S^1 \to A$ with the property that f sends each of the base points of the circles to a. But $S^1 + S^1$ is not connected, and so not a group. To fix this we take the clue from the requirement that both the base points were to be sent to a common base point and *define* $S^1 \vee S^1$ to be what we get from $S^1 + S^1$ when we *insert an identity* between the two basepoints.

$S^1 \vee S^1$ if formed from $S^1 + S^1$ by inserting an identity



The amazing thing is that this works – an enormous simplification of the classical construction of the "free products" or "amalgamated sum" of groups. We need to show that the "wedge" $S^1 \vee S^1$ is indeed a group, and this proof simultaneously unpacks the classical description.

We start by giving a definition of the wedge construction which is important for pointed types in general and then prove that the wedge of two groups is a group whose symmetries are arbitrary "words" in the original symmetries.

def:wedge

Definition 8.6.1. Let (A_1, a_1) and (A_2, a_2) be pointed types. The *wedge* is the pointed type $(A_1 \vee A_2, a_{12})$ given as a higher inductive type by

(1) functions $i_1 : A_1 \to A_1 \vee A_2$ and $i_2 : A_2 \to A_1 \vee A_2$

(2) an identity $g : i_1 a_1 = i_2 a_2$.

We point this type at $a_{12} :\equiv i_1 a_1$. The function

$$i_2^g : (a_2 =_{A_2} a_2) \to (a_{12} =_{A_1 \vee A_2} a_{12})$$

is defined by $i_2^g(p) :\equiv g^{-1} i_2(p) g$, whereas (for notational consistency only) we set $i_1^g :\equiv i_1 : (a_1 =_{A_1} a_1) \to (a_{12} =_{A_1 \vee A_2} a_{12})$. Simplifying by writing $i : A_1 + A_2 \to A_1 \vee A_2$ for the function given by i_1 and i_2 (with basepoints systematically left out of the notation), the induction principle is

$$\prod_{C : (A_1 \vee A_2) \to \mathcal{U}} \sum_{s : \prod_{a : A_1 + A_2} C i(a)} ((s(a_1) = C(g^{-1}) s(a_2)) \to \prod_{x : (A_1 \vee A_2)} C(x)).$$

⌟

Unraveling the induction principle we see that if B is a pointed type, then a pointed function $f : A_1 \vee A_2 \to_* B$ is given by providing pointed functions $f_1 : A_1 \to_* B$ and $f_2 : A_2 \to_* B$ – the identity $f_1(a_1) = f_2(a_2)$ which seems to be missing is provided by the requirement of the functions being pointed. For the record

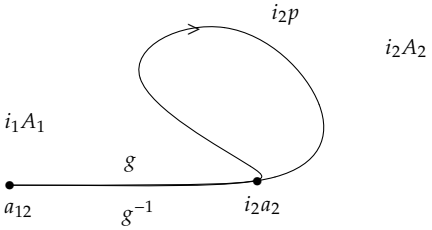
lem:univwee

Lemma 8.6.2. *If B is a pointed type, then the function*

$$i^* : (A_1 \vee A_2 \to_* B) \to (A_1 \to_* B) \times (A_2 \to_* B), \qquad i^*(f) = (f i_1, f i_2)$$

is an equivalence.

To the right you see a picture of $i_2^g(p)$: it is the symmetry of the base point $a_{12} :\equiv i_1 a_1$ you get by *first* moving to $i_2 a_2$ with g, *then* travel around with p ($i_2 p$, really) and finally go home to the basepoint with the inverse of g.



The idea is that an identity in $a_{12} = x$ can be factored into a string of identities, each lying solely in A_1 or in A_2. We define a family of sets consisting of exactly such strings of identities – it is a set since A_1 and A_2 are groupoids – and prove that it is equivalent to the family $P(x) :\equiv (a_{12} =_{A_1 \vee A_2} x)$ which consequently must be a family of sets. We need to be able to determine whether a symmetry is reflexivity or not, but once we know that, the symmetries of the base point in the wedge are then given by "words $p_0 p_1 \dots p_n$" where the p_j alternate between being symmetries in the first or the second group, and none of the p_j for positive j are allowed to be reflexivity. Note that there order of the p_js is not negotiable: if I shuffle them I get a new symmetry.

def:sumofgroup

Definition 8.6.3. If $G_1 = \mathrm{Aut}_{A_1}(a_1)$ and $G_2 = \mathrm{Aut}_{A_2}(a_2)$ are groups, then their *sum* is defined as

$$G_1 \vee G_2 :\equiv \mathrm{Aut}_{A_1 \vee A_2}(a_{12}).$$

The homomorphisms $i_1 : G_1 \to G_1 \vee G_2$ and $i_2 : G_2 \to G_1 \vee G_2$ induced from the structure maps $i_1 : A_1 \to A_1 \vee A_2$ and $i_2 : A_2 \to A_1 \vee A_2$ are also referred to as *structure maps*. ⌟

lem:sumofgroupsIsSum

LEMMA 8.6.4. *If G_1, G_2 and G are groups, then the function*

$$\mathrm{Hom}(G_1 \vee G_2, G) \to \mathrm{Hom}(G_1, G) \times \mathrm{Hom}(G_2, G)$$

given by restriction along the structure maps is an equivalence.

Proof. This is a special case of Lemma 8.6.2. □

Specializing further, we return to our initial motivation and see that mapping out of a wedge of two circles *exactly* captures the information of two independent symmetries:

cor:ZplusZuniv

COROLLARY 8.6.5. *If G is a group, then the functions*

$$\mathrm{Hom}(\mathbb{Z} \vee \mathbb{Z}, G) \to \mathrm{Hom}(\mathbb{Z}, G) \times \mathrm{Hom}(\mathbb{Z}, G) \simeq UG \times UG$$

is an equivalence.

xca:whatAbelianGroups

EXERCISE 8.6.6. This leads to the following characterization of abelian groups formulated purely in terms of pointed connected groupoids (with no direct reference to identity types). A group G is abelian if and only if the canonical map

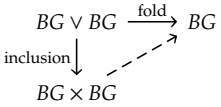
$$\mathrm{fold} : BG \vee BG \to_* BG$$

(given via Lemma 8.6.4 by $\mathrm{id}_G : G \to G$) extends over the inclusion

$$i : BG \vee BG \to_* BG \times BG$$

(given by the inclusions $\mathrm{in}_1, \mathrm{in}_2 : G \to G \times G$).

Do we know at this point that this extension problem is a proposition, i.e., that the wedge inclusion is an epimorphism? Check re pointedness.



As a cute aside, one can see that the required map $BG \times BG \to_* BG$ actually doesn't need to be pointed: factoring $\mathrm{fold} : BG \vee BG \to BG$ over $i : BG \vee BG \to BG \times BG$ – even in an unpointed way – kills all "commutators" $ghg^{-1}h^{-1} : \mathrm{U}(G \vee G)$. (is this still a proposition, or do we need to truncate?) ⌟

We end the section by proving that wedges of decidable groups are decidable groups and that they can be given the classical description in terms of words.

lem:wedgeofgpoiddisgpoid

LEMMA 8.6.7. *Let $G_1 :\equiv \mathrm{Aut}_{A_1}(a_1)$ and $G_2 :\equiv \mathrm{Aut}_{A_2}(a_2)$ be decidable groups, then the wedge sum $G_1 \vee G_2 :\equiv \mathrm{Aut}_{A_1 \vee A_2}(a_{12})$ is a decidable group.*

Let C_1 be the set of strings $(p_0, n, p_1, \ldots, p_n)$ with $n : \mathbb{N}$ and, for $0 \leq j \leq n$

- *$p_j : UG_1$ for even j*
- *$p_j : UG_2$ for odd j and*
- *p_j is not reflexivity for j positive*

(the last requirement makes sense and is a proposition since our groups are decidable).

Then the function given by composition in $UG_{12} :\equiv (a_{12} = a_{12})$

$$\beta : C_1 \to UG_{12}, \qquad \beta(p_0, n, p_1, \ldots p_n) :\equiv i_1^g p_0 i_2^g p_1 i_1^g p_2 \ldots i_?^g p_n$$

(where $i_?^g p_n$ is $i_1^g p_n$ or $i_2^g p_n$ according to whether n is even or odd) is an equivalence.

Proof. That the wedge is connected follows by transitivity of identifications, if necessary passing through the identification $g : i_1 a_1 = i_2 a_2$ in the wedge.

We must prove that the wedge is a groupoid, i.e., that all identity types are sets, which we do by giving an explicit description of the universal set bundle.

We use the notation of Definition 8.6.1 freely, and for ease of notation, let $a_{2k+i} :\equiv a_i$ and $i^g_{2k+i} :\equiv i^g_i$ for $i = 1, 2$, $k : \mathbb{N}$. Define families of sets

$$C_i : A_i \to \mathrm{Set}, \qquad i = 1, 2$$

by

$$C_i(x) :\equiv (a_i =_{A_i} x) \times \sum_{n:\mathbb{N}} \prod_{1 \le k \le n} \sum_{p_k : a_{i+k} = a_{i+k}} (p_k \neq \mathrm{refl}_{a_{i+k}})$$

when $x : A_i$. Note that $p_k \neq \mathrm{refl}_{a_{i+k}}$ is a proposition; we leave it out when naming elements. Hence, an element in $C_1(a)$ is a tuple $(p_0, n, p_1, \dots, p_n)$ where $p_0 : a_1 =_{A_1} a$, $p_1 : a_2 =_{A_2} a_2$, $p_2 : a_1 =_{A_1} a_1$, and so on – alternating between symmetries of a_1 and a_2, and where p_0 is the only identity allowed to be refl. Define $C_{12} : C_1(a_1) \to C_2(a_2)$ by

$$C_{12}(p_0, n, p_1 \dots, p_n) = \begin{cases} (\mathrm{refl}_{a_2} 0,) & \text{if } p_0 = \mathrm{refl}_{a_1}, n = 0, \\ (p_1, n-1, p_2 \dots, p_n) & \text{if } p_0 = \mathrm{refl}_{a_1}, n \neq 0, \\ (\mathrm{refl}_{a_2}, n+1, p_0, \dots, p_n) & \text{if } p_0 \neq \mathrm{refl}_{a_1}. \end{cases}$$

It is perhaps instructive to see a table of the values $C_{12}(p_0, n, p_1, \dots, p_n)$ for $n < 3$:

	$(p_0, 0)$	$(p_0, 1, p_1)$	$(p_0, 2, p_1, p_2)$
$p_0 = \mathrm{refl}_{a_1}$	$(\mathrm{refl}_{a_2}, 0)$	$(p_1, 0)$	$(p_1, 1, p_2)$
$p_0 \neq \mathrm{refl}_{a_1}$	$(\mathrm{refl}_{a_2}, 1, p_0)$	$(\mathrm{refl}_{a_2}, 2, p_0, p_1)$	$(\mathrm{refl}_{a_2}, 3, p_0, p_1, p_2)$

Since C_{12} is an equivalence, the triple (C_1, C_2, C_{12}) defines a family

$$C : A_1 \vee A_2 \to \mathrm{Set}.$$

In particular, $C(a_{12}) :\equiv C_1(a_1)$. For $x : A_1$ we let $i^C_1 : C_1(x) \to C(i_1(x))$ be the induced equivalence, and likewise for i^C_2. We will show that C is equivalent to $P :\equiv \mathbb{P}_{a_{12}}$, where $P(x) :\equiv (a_{12} = x)$, and so that the identity types in the wedge are equal to the sets provided by C.

One direction is by transport in C; more precisely,

$$\alpha : \prod_{x : A_1 \vee A_2} (P(x) \to C(x))$$

is given by transport with $\alpha(a_{12})(\mathrm{refl}_{a_{12}}) :\equiv (\mathrm{refl}_{a_1}, 0) : C(a_{12})$. The other way,

$$\beta : \prod_{x : A_1 \vee A_2} (C(x) \to P(x))$$

is given by composing identities, using the glue g to make their ends meet:

$$\beta(i_1 a)(p_0, n, p_1, \dots, p_n) :\equiv i_1(p_0) i^g_2(p_1) i^g_3(p_2) \dots i^g_{n+1}(p_n)$$

(here the definition $\dots i^g_3 :\equiv i^g_1 :\equiv i_1$ proves handy since we don't need to distinguish the odd and even cases) and likewise

$$\beta(i_2 a)(p_0, n, p_1, \dots, p_n) :\equiv i_2(p_0) g\, i^g_1(p_1) i^g_2(p_2) \dots i^g_n(p_n)$$

and compatibility with the glue C_{12} is clear since the composite $\mathrm{refl}_x p$ is equal to p.

For notational convenience, we hide the x in $\alpha(x)(p)$ and $\beta(x)(p)$ from now on.

That $\beta\alpha(p) = p$ follows by path induction: it is enough to prove it for $x = a_{12}$ and $p :\equiv \mathrm{refl}_{a_{12}}$:

$$\beta\alpha(\mathrm{refl}_{a_{12}}) = \beta(\mathrm{refl}_{a_1}, 0) = i_1^g \mathrm{refl}_{a_1} = \mathrm{refl}_{a_{12}}.$$

That $\alpha\beta(p_0, n, p_1 \ldots, p_n) = (p_0, n, p_1, \ldots, p_n)$ follows by induction on n and p_0. For $n = 0$ it is enough to consider $x = a_{12}$ and $p_0 = \mathrm{refl}_{a_1}$, and then $\alpha\beta(\mathrm{refl}_{a_1}, 0) :\equiv \alpha(\mathrm{refl}_{a_{12}}) :\equiv (\mathrm{refl}_{a_1}, 0)$. In general, (for $n > 0$)

$$\begin{aligned}\alpha\beta(p_0, n, p_1 \ldots, p_n) &= \mathrm{trp}^C_{i_1(p_0) i_2^g(p_1) i_1^g(p_2) \ldots i^g_{n+1}(p_n)}(\mathrm{refl}_{a_1,0}) \\ &= \mathrm{trp}^C_{i_1(p_0)} \ldots \mathrm{trp}^C_{i^g_{n+1}(p_n)}(\mathrm{refl}_{a_1,0}).\end{aligned}$$

The induction step is as follows: let $0 < k \leq n$, then

$$\begin{aligned}&\mathrm{trp}^C_{i_k^g p_{k-1}}\ i^C_{k-1}(p_k, n-k-1, p_{k+1}, \ldots, p_n) \\ =&\mathrm{trp}^C_{i_k^g p_{k-1}}\ i^C_k(\mathrm{refl}_{a_{k-1}}, n-k, p_k, \ldots, p_n) \\ =&i^C_k\ \mathrm{trp}^{C_k}_{p_{k-1}}(\mathrm{refl}_{a_{k-1}}, n-k, p_k, \ldots, p_n) \\ =&(p_{k-1}, n-k, p_k, \ldots, p_n).\end{aligned}$$

□

8.7 *Free groups*

sec:freegroups

We have seen in Example 4.4.17 that the group of integers $\mathbb{Z}$ is the free group on one generator in the sense that the set of homomorphisms from $\mathbb{Z}$ to any group G is equivalent (by evaluation at the loop) to the underlying set of symmetries in G, UG. This set is of course equivalent (by evaluation at the unique element) to the set of maps $(\mathbb{1} \to UG)$.

Likewise, we have seen in Corollary 8.6.5 that the binary sum $\mathbb{Z} \vee \mathbb{Z}$ is the free group on two generators, corresponding to the left and right summands.

In general, a free group on a set of generators S is a group F_S with specified elements $\iota_s : \mathrm{UF}_S$ labeled by $s : S$, such that evaluation gives an equivalence $\mathrm{Hom}(\mathrm{F}_S, G) \xrightarrow{\simeq} (S \to UG)$ for each group G.

We now give a definition of the classifying type of a free group as a higher inductive type that is very much like that of the circle, except that instead of having a single generating loop, it has a loop $\circlearrowleft_s$ for each element $s : S$.

def:bffree

Definition 8.7.1. Fix a set S. The classifying type of the free group on S, BF_S, is a type with a point $\bullet : \mathrm{BF}_S$ and a constructor $\circlearrowleft__ : S \to \bullet \xrightarrow{=} \bullet$.

Let $A(x)$ be a type for every element $x : \mathrm{BF}_S$. The induction principle for BF_S states that, in order to define an element of $A(x)$ for every $x : \mathrm{BF}_S$, it suffices to give an element a of $A(\bullet)$ together with an identification $l_s : a \xrightarrow[\circlearrowleft_s]{=} a$ for every $s : S$. The function f thus defined satisfies $f(\bullet) \equiv a$ and we are provided identifications $\mathrm{apd}_f(\circlearrowleft_s) \xrightarrow{=} l_s$ for each $s : S$.

We define the *free group* on S as $\mathrm{F}_S :\equiv \underline{\Omega}(\mathrm{BF}_S, \bullet)$. ⌟

A priori, F_S is only an ∞-group. Nevertheless, we get immediately from the induction principle that evaluation at the elements of S gives an equivalence $\mathrm{Hom}(\mathrm{F}_S, G) \xrightarrow{\simeq} (S \to UG)$ for each ∞-group G.

In order to see that F_S is a group, we need to know that BF_S is a groupoid. This follows from a general theorem on identifications in pushouts due to Wärn.[8] Here we restrict our discussion to decidable sets S, where we can give a more concrete proof.

[8] David Wärn. *Path spaces of pushouts*. Preprint. 2023. URL: https://dwarn.se/po-paths.pdf.

We can follow that same strategy as in Theorem 3.4.5 and Lemma 8.6.7 and show this by giving a description of F_S as an *abstract* group. To see what this should be, think about what symmetries of $\bullet$ we can write using the constructors $\circlearrowleft_s$ for $s : S$. We can compose these out of $\circlearrowleft_s$ and $\circlearrowleft_s^{-1}$ with various generators s. However, if we at any point have $\circlearrowleft_s \circlearrowleft_s^{-1}$ or $\circlearrowleft_s^{-1} \circlearrowleft_s$, then these cancel. This motivates the following definitions.

Definition 8.7.2. Fix a decidable set S. Let $\tilde{S} :\equiv S + S$ be the (decidable) set of *signed* letters from S. Also, let $\bar{_} : \tilde{S} \to \tilde{S}$ be the equivalence that swaps the two copies of S. This map is an involution called *complementation*. ⌟

If $a : S$, we'll also write $a : \tilde{S}$ for the left inclusion, and we'll write $A :\equiv \bar{a} : \tilde{S}$ for the right inclusion, so that $\bar{a} \equiv A$ and $\bar{A} \equiv a$, i.e., a and A are complementary.

Recall the definition of lists T^* over a set T, Definition 2.12.11, inductively generated by the empty list ε and the recursive constructor that concatenates an element $t : T$ to a list ℓ, forming a new list $t\ell$ with head t and tail ℓ. Instead of "lists" we shall often speak about "words" formed from "letters" taken from the set T, which is thus a kind of "alphabet".

If we take $T :\equiv \tilde{S}$ we get the set of words in the signed letters from S. If we have $a, b : S$, we find among the elements of $\tilde{S}^*$ the following:

$$\varepsilon, a, b, A, B, aa, ab, aA, aB, ba, bb, bA, bB, Aa, Ab, AA, AB, \ldots$$

When we interpret these as symmetries in BF_S, i.e., as elements in UF_S, the words aA and Bb, etc., become trivial.

Definition 8.7.3. A word $w : \tilde{S}^*$ is called *reduced* if it doesn't contain any consecutive pairs of complementary letters. The map $\rho_S : \tilde{S}^* \to \tilde{S}^*$ maps a word to its *reduction*, which is obtained by repeatedly deleting consecutive pairs of complementary letters until none remain. ⌟

Exercise 8.7.4. Complete the definition of ρ_S by nested induction on words.[9] ⌟

[9] Hint: This is precisely the point where we need S to have decidable equality.

Definition 8.7.5. We define $\mathcal{R}_S$ to be the image of ρ_S in $\tilde{S}^*$, whose elements are the *reduced words*. We define $\mathcal{D}_S$ to be the fiber of ρ_S at the empty word, $\rho_S^{-1}(\varepsilon)$, whose elements are called *Dyck words*.[10] ⌟

[10] Considered as a set of words, $\mathcal{D}_S$ is called the 2-*sided Dyck language*. Perhaps the 1-*sided Dyck language* is more familiar in language theory: Here, S is considered as a set of 'opening parentheses', while the complementary elements are 'closing parentheses'. For example, the 1-sided Dyck language for $\tilde{S} = \{(,)\}$ consists of all *balanced* words of opening and closing parentheses, e.g., (), (()), ()(), etc., while our $\mathcal{D}_S$ in this case also has words like)(and))((.

Remark 8.7.6. Like any map, ρ_S induces an equivalence relation $\sim$ on the set $\tilde{S}^*$ where two words u, v are related if and only if they map to the same reduced word, in other words, $u \sim v$ if and only if $\rho_S(u) = \rho_S(v)$. Thus, ρ_S induces an equivalence $\tilde{S}^*/\sim \xrightarrow{\simeq} \mathcal{R}_S$. ⌟

We are now ready to prove that set $\mathcal{R}_S$ of reduced words is equivalent to UF_S. We'll do this be defining an interpretation function from words to elements of the free group.

Definition 8.7.7. We define $[\![_]\!] : \tilde{S}^* \to \mathrm{UF}_S$ by induction on words by

setting

$$\begin{aligned} [\![\varepsilon]\!] &:\equiv \mathrm{refl}_\bullet \\ [\![aw]\!] &:\equiv \underset{a}{\circlearrowleft}\cdot[\![w]\!], && \text{for } a : S, \\ [\![\bar{a}w]\!] \equiv [\![Aw]\!] &:\equiv \overset{-1}{\underset{a}{\circlearrowleft}}\cdot[\![w]\!], && \text{for } a : S. \end{aligned}$$ ⌟

tbm:free-group-elements

THEOREM 8.7.8. *Fix a decidable set S. The interpretation map $[\![_]\!]$ restricts to an equivalence, denoted the same way, $[\![_]\!] : \mathcal{R}_S \to \mathrm{UF}_S$.*

Proof. We extend $\mathcal{R}_S$ to an F_S-set, $\mathcal{R}_S : \mathrm{BF}_S \to \mathrm{Set}$, where we define $\mathcal{R}_S(x)$ by induction on $x : \mathrm{BF}_S$, with

$$\mathcal{R}_S(\bullet) :\equiv \mathcal{R}_S, \quad \text{and} \quad \mathcal{R}_S(\underset{a}{\circlearrowleft}) :\equiv \overline{\mathrm{s}_a}, \quad \text{for } a : S.$$

Here $\mathrm{s}_a : \mathcal{R}_S \xrightarrow{\simeq} \mathcal{R}_S$ is the equivalence sending a word w to $\rho_S(aw)$, whose inverse sends w to $\rho_S(Aw)$. These operations are indeed mutual inverses, since $aAw \sim w \sim Aaw$.[11]

Our goal now is to extend the definition of $[\![_]\!]$ to $[\![_]\!]_x : \mathcal{R}_S(x) \to \mathbb{P}_\bullet$, where $\mathbb{P}_\bullet(x) \equiv (\bullet \xrightarrow{=} x)$, for $x : \mathrm{BF}_S$, so that this is an inverse to the map given by transport of ε, $\tau_x : (\bullet \xrightarrow{=} x) \to \mathcal{R}_S(x)$, with $\tau_x(p) :\equiv \mathrm{trp}^{\mathcal{R}_S}_p(\varepsilon)$. Thinking back to Definition 3.4.4, we define $[\![_]\!]_x$ by induction on x with $[\![_]\!]_\bullet :\equiv [\![_]\!]$ and using $[\![aw]\!] \equiv \circlearrowleft_a \cdot [\![w]\!]$.[12]

We get an identification $[\![_]\!]_x \circ \tau_x \xrightarrow{=} \mathrm{id}$ by path induction, since $[\![\varepsilon]\!] \equiv \mathrm{refl}_\bullet$.

To prove the proposition $\tau_x([\![w]\!]_x) = w$ for all $x : \mathrm{BF}_S$ and $w : \mathcal{R}_S(x)$, it suffices to consider the case $x \equiv \bullet$, since BF_S is connected. We prove that $\tau_\bullet([\![w]\!]) \sim w$ holds for *all* words $w : \tilde{S}^*$ by induction on w, because then it follows that $\tau_\bullet([\![w]\!]) = w$ for *reduced* words w. The case $w \equiv \varepsilon$ is trivial. In the step case for adding $a : S$, we calculate,

$$\tau_\bullet([\![aw]\!]) \equiv \mathrm{trp}^{\mathcal{R}_S}_{\circlearrowleft_a \cdot [\![w]\!]}(\varepsilon) = \mathrm{trp}^{\mathcal{R}_S}_{\circlearrowleft_a}(\tau_\bullet([\![w]\!])) = \mathrm{s}_a(w) = \rho_S(aw) \sim aw,$$

as desired, the complementary case being similar. □

EXERCISE 8.7.9. Construct an equivalence $\mathcal{R}_{\mathbb{1}} \xrightarrow{\simeq} \mathrm{Z}$ sending ε to 0 such that s_* corresponds to s, where $* : \mathbb{1}$ is the unique element. This gives us two more options to add to the list in Footnote 7 on Page 67: $\tilde{\mathbb{1}}^*/{\sim}$ and $\mathcal{R}_{\mathbb{1}}$! ⌟

EXERCISE 8.7.10. Construct an equivalence $\mathrm{F}_{m\mathrm{LITrue}} \xrightarrow{\simeq} \mathrm{F}_m \vee \mathbb{Z}$ for each $n : \mathbb{N}$ using the universal properties. As a result, give identifications

$$\mathrm{F}_m \xrightarrow{=} \big((\mathbb{Z} \vee \mathbb{Z}) \vee \cdots\big) \vee \mathbb{Z},$$

for $n : \mathbb{N}$, where there are n copies of $\mathbb{Z}$ on the right-hand side. ⌟

[11] The set $\mathcal{R}_S$ is very much like Z, but instead of having only one successor equivalence s, it has one for each element of S.

[12] In a picture, the case for $\circlearrowleft_a$ should prove that it does not matter what path you take around the square

$$\begin{array}{ccc} \mathcal{R}_S & \xrightarrow{[\![_]\!]} & (\bullet \xrightarrow{=} \bullet) \\ \| \downarrow \mathrm{s}_a & & \| \downarrow \circlearrowleft_a \cdot _ \\ \mathcal{R}_S & \xrightarrow{[\![_]\!]} & (\bullet \xrightarrow{=} \bullet). \end{array}$$

9
Normal subgroups and quotients

ch:subgroups

sec:subgp-overview

9.1 *Brief overview of the chapter*

TBW (and stolen from the below)

sec:epis

9.2 *Epimorphisms, and an equivalent take on monomorphisms*

In set theory we say that a function $f : B \to C$ of sets is an injection if for all $b, b' : B$ we have that $f(b) = f(b')$ implies that $b = b'$. This conforms with our definition, see Exercise 2.17.3. Furthermore, since giving an element $b : B$ is equivalent to giving a (necessarily constant) function $c_b : \mathbb{1} \to B$, we could alternatively say that a function $f : B \to C$ is an injection if and only if for any two $g, h : \mathbb{1} \to B$ such that $fg = fh$ we have that $g = h$. In fact, by function extensionality we can replace $\mathbb{1}$ by any set A (two functions are identical if and only if they have identical values at every point).

Similarly, a function $f : B \to C$ is surjective if for all $c : C$ the preimage $f^{-1}(c) \equiv \sum_{b:B}(c = f(b))$ is non-empty (Definition 2.17.1). A smart way to say this is to say that the first projection from $\sum_{c:C}\|f^{-1}(c)\|$, the propostitional image of f, to C is an equivalence. Using Exercise 2.17.14, we see that for a surjection $f : B \to C$ and family of propositions $P : C \to$ Prop, the propositions $\prod_{c:C} P(c)$ and $\prod_{b:B} P(f(b))$ are equivalent.[1] In particular, if $g, h : C \to D$ are two functions into a set D the proposition $\prod_{c:C}(g(c) = h(c))$ is equivalent to $\prod_{b:B}(gf(b) = hf(b))$.[2]

From this we condense the following characterizations of injections and surjections of sets. Note that these characterizations are instantiations of the concepts of monomorphism and epimorphism, respectively, in the category of sets, implying that this category is balanced (**??**).

[1] This actually holds for any map $f : B \to C$ with B and C types.

[2] For the converse: take Prop for D and define $g(c) :\equiv \|f^{-1}(c)\|$ and $h(c) :\equiv$ True for all $c : C$, then $gf = hf$, whereas $g = h$ expresses surjectivity of f.

lem:injmonosurjepiSet

it:injmono

Lemma 9.2.1. *Let $f : B \to C$ be a function of sets.*

(1) *The function f is an injection if and only if for any set A and functions $g, h : A \to B$, if*

$$A \underset{h}{\overset{g}{\rightrightarrows}} B \xrightarrow{f} C$$

commutes, i.e., $fg =_{A\to C} fh$, then $g = h$.

it:surjepi

(2) *The function f is a surjection if and only if for any set D and functions $g, h : C \to D$, if*

$$B \xrightarrow{f} C \underset{h}{\overset{g}{\rightrightarrows}} D$$

commutes, i.e., $gf =_{B\to D} hf$, then $g = h$.

By Lemma 9.2.1 there is a pleasing reformulation which highlights that injections/surjections of sets are characterized by injections of sets of functions: a function of sets $f : B \to C$ is

(1) an injection if and only if for any set A postcomposition by f gives an injection from $A \to B$ to $A \to C$

(2) a surjection if and only if for any set D precomposition by f gives an injection from $C \to D$ to $B \to D$.

The above observations about sets translate fruitfully to other categories, in particular to the category of groups.[3] When dealing with groups, the concepts of monomorphism and epimorphism instantiate to particular morphisms in the category of groups, i.e., to group homomorphisms having a particular property.[4]

In fact we did define monomorphisms of groups already in Definition 5.3.13. The two definitions are proved to be equivalent in Lemma 9.2.5(1),(2), and we use the same name for both.

[3] This is of course the reason why the concepts of monomorphism **??** and epimorphism **??** are defined for categories in general.

[4] As opposed to, e.g., their underlying unpointed functions of connected groupoids in their own (wild) category.

def:monomorphism
def:epimorphism

Definition 9.2.2. Given groups G, H, a homomorphism $f : \mathrm{Hom}(G, H)$ is called a

(1) *monomorphism* if for any group F, postcomposition by f is an injection from $\mathrm{Hom}(F, G)$ to $\mathrm{Hom}(F, H)$, and an

(2) *epimorphism* if for any group I, precomposition by f is an injection from $\mathrm{Hom}(H, I)$ to $\mathrm{Hom}(G, I)$.

The corresponding families of propositions are called

$$\mathrm{isMono}, \mathrm{isEpi} : \mathrm{Hom}(G, H) \to \mathrm{Prop}.$$

A monomorphism (epimorphism) is called *proper* if it is not an isomorphism.

The *type of monomorphisms into* H is

$$\mathrm{Mono}(H) :\equiv \sum_{G : \mathrm{Group}} \sum_{i : \mathrm{Hom}(G,H)} \mathrm{isMono}(i).$$

The *type of epimorphisms from* G is

$$\mathrm{Epi}(G) :\equiv \sum_{H : \mathrm{Group}} \sum_{f : \mathrm{Hom}(G,H)} \mathrm{isEpi}(f).$$ ⌟

exa:projections-epi

Example 9.2.3. For groups G_1 and G_2, the two projections from $G_1 \times G_2$ are epimorphisms. ⌟

xca:monoist-epi2nd

Exercise 9.2.4. Show that the composition of monomorphisms is again a monomorphism, and similarly for epimorphisms. Consider a composite $f = f_2 f_1$ of homomorphisms. Show that f_2 is an epimorphism if f is, and f_1 is a monomorphism if f is. ⌟

We've seen in Example 4.4.17 that for any group G, the underlying set $UG :\equiv (\mathrm{sh}_G \xrightarrow{=} \mathrm{sh}_G)$ of $\mathrm{abs}(G)$ is equivalent to the set of homomorphisms $\mathrm{Hom}(\mathbb{Z}, G)$. Moreover, given a group homomorphism $f : \mathrm{Hom}(G, H)$, Lemma 4.4.18 states that the diagram in the margin commutes. Hence, in this situation, saying that Uf is an injection is equivalent to saying that postcomposition by f is an injection $\mathrm{Hom}(\mathbb{Z}, G) \to \mathrm{Hom}(\mathbb{Z}, H)$.

$$\begin{array}{ccc} \mathrm{Hom}(\mathbb{Z}, G) & \xrightarrow{f \circ _} & \mathrm{Hom}(\mathbb{Z}, H) \\ \mathrm{ev} \downarrow \simeq & & \mathrm{ev} \downarrow \simeq \\ UG & \xrightarrow{Uf} & UH \end{array}$$

Figure 9.1: Naturality of ev.

In this observation, the integers $\mathbb{Z}$ plays no more of a role than $\mathbb{1}$ does in Lemma 9.2.1; we can let the source vary over any group F:

fig:Znatural

lem:eq-mono-cover
lem:eq-epi-conn
it:mono
it:injection
it:cover

LEMMA 9.2.5. *Let G and H be groups and $f : \mathrm{Hom}(G, H)$ a homomorphism. The following propositions are equivalent*::

(1) *f is a monomorphism in the sense of Definition 9.2.2;*

(2) *$Uf : UG \to UH$ is an injection;*

(3) *$Bf_\div : BG_\div \to BH_\div$ is a set bundle* (*or*: *is 0-truncated, see Definition 3.9.5*).

Proof. We have already seen that condition (1) implies condition (2) (let F be $\mathbb{Z}$). Conversely, suppose that (2) holds and F is a group. Consider the commutative diagram

$$\begin{array}{ccc} \mathrm{Hom}(F,G) & \xrightarrow{\quad f\circ_ \quad} & \mathrm{Hom}(F,H) \\ {\scriptstyle g\mapsto(g\circ_)}\downarrow & & \downarrow{\scriptstyle h\mapsto(h\circ_)} \\ (\mathrm{Hom}(\mathbb{Z},F)\to\mathrm{Hom}(\mathbb{Z},G)) & \xrightarrow{f\circ_} & (\mathrm{Hom}(\mathbb{Z},F)\to\mathrm{Hom}(\mathbb{Z},H)), \end{array}$$

where the vertical maps are injections by the naturality of Figure 9.1 using that U acts injectively. Since the bottom function is by assumption is an injection, so is the upper one, using **??** 9.2.4.[5]

The equivalence of (3) and (2) follows immediately from Corollary 2.17.9(1), using that $\mathrm{Hom}(G, H)$ is classified by $BG \to_* BH$, BG is connected, and f is pointed. □

[5] Alternatively: and $g, h : \mathrm{Hom}(F, G)$. Then $fg = fh$ implies that for all $p : \mathrm{Hom}(\mathbb{Z}, F)$ we have by associativity that $f(gp) = (fg)p = (fh)p = f(hp)$, and so, by assumption, that $gp = hp$. Again, by function extensionality (of functions $\mathrm{Hom}(\mathbb{Z}, F) \to \mathrm{Hom}(\mathbb{Z}, G)$), this is exactly saying that Ug is identical to Uh.

Similarly, we have:

lem:epi-surj
it:epi
it:surjection
it:connfib

LEMMA 9.2.6. *The following propositions are equivalent*:

(1′) *f is an epimorphism;*

(2′) *$Uf : UG \to UH$ is a surjection;*

(3′) *$Bf_\div : BG_\div \to BH_\div$ has connected fibers* (*or*: *is 0-connected, see Definition 3.9.4*).

Proof. The equivalence of (2′) and (3′) is immediate.

For the rest, the easy direction is that (2′) implies (1′): (TODO) Since by **??** the categories of groups and of abstract groups are equivalent, it suffices to show that Uf is an epimorphism of abstract groups whenever it is surjective. Since abstract homomorphisms are just functions with additional properties, we can just apply Lemma 9.2.1(2).

The harder direction, that (1′) implies (2′), is a consequence of Construction 9.2.7. Indeed, we can factor any $f : \mathrm{Hom}(G, H)$ via the image as a an epi followed by a mono (**??** TBD in 9.3):

$$G \xrightarrow{q} \mathrm{im}(f) \xrightarrow{i} H$$

If f is an epi, then so is i by **??** 9.2.4. But i is an equalizer (Construction 9.2.7), that is, we have φ, ψ such that

$$\mathrm{im}(f) \xrightarrow{i} H \underset{\psi}{\overset{\varphi}{\rightrightarrows}} L \quad \text{commutes.}$$

As an epi, $\varphi i = \psi i$ implies $\varphi = \psi$, so i is an equalizer of already equal homomorphisms, so i is an isomorphism, which implies that f is an epimorphism. □

con:monos-are-equalizers

Construction 9.2.7. *Given groups G and H and monomorphism $i : H \to G$, we have a group W and two homomorphisms $\varphi, \psi : \mathrm{Hom}(G, W)$ such that i is an equalizer of φ, ψ.*[6]

[6] This proof follows an idea by Trimble[7].

[7] Todd Trimble. *Monomorphisms in the category of groups*. https://ncatlab.org/toddtrimble/published/monomorphisms+in+the+category+of+groups. Jan. 2020.

Implementation of Construction 9.2.7. [draft] Consider the projection $\pi : G \to G/H$ to the set of cosets. Let $j : G/H \to A$ be an injection into a group A. (We could for instance let A be the free (abelian) group on G/H. [Add xref to statement that inclusion of generators in an injection.])

Consider the group $W :\equiv \mathrm{Aut}_E(\mathrm{sh}_G, \mathrm{cst}_{\mathrm{sh}_A})$, where

$$E :\equiv \sum_{t : BG} \left((\mathrm{sh}_G \xrightarrow{=} t) \to BA \right).$$

We have two homomorphisms $\varphi, \psi : G \to W$ with the same classifying map, $t \mapsto (t, \mathrm{cst}_{\mathrm{sh}_A})$, but with different pointing paths:[8]

$$\varphi_{\mathrm{pt}} :\equiv \mathrm{refl}_{(\mathrm{sh}_G, \mathrm{cst}_{\mathrm{sh}_A})}, \quad \psi_{\mathrm{pt}} :\equiv (\mathrm{refl}_{\mathrm{sh}_G}, j\pi),$$

where $j\pi$ is short for $\mathrm{ptw}^{-1}(\mathrm{ap}_{j\pi})$. The equalizer of φ and ψ thus consists of all $g : UG$ such that $j\pi(gg') = j\pi(g')$ for all $g' : UG$. Since j is injective, this is equivalent to $\pi(gg') = \pi(g')$ for all $g' : UG$, and this holds if and only if g belongs to H. □

[8] For $t \equiv \mathrm{sh}_G, \mathrm{cst}_{\mathrm{sh}_A} \xrightarrow{=} \mathrm{cst}_{\mathrm{sh}_A}$ can be given by functions $UG \to UA$, so I'd like $\pi : BG \to G/H$ and $j : G/H \to BA$ so that $\mathrm{ap}_{j\pi} : UG \to UA$.

lem:epimonoiso

Lemma 9.2.8. *Given groups G and H, a homomorphism $f : \mathrm{Hom}(G, H)$ is an isomorphism if and only if f is both a monomorphism and an epimorphism.*

Proof. By Lemma 9.2.5(2) Lemma 9.2.6(2′), using that the categories of groups and of abstract groups are equivalent by **??**. □

9.3 *Images, kernels and cokernels*

subsec:ker

The set of subgroups of a group G encodes much information about G, partially because homomorphisms between G and other groups give rise to subgroups.

In Example 4.2.23 we studied a homomorphism from $\mathbb{Z}$ to Σ_m classified by the pointed map $R_m : S^1 \to_* \mathrm{B}\Sigma_m$ sending $\bullet$ to $\mathbb{m}$ and $\circlearrowleft$ to the cyclic sucessor $s_m : \mathrm{U}\Sigma_m \equiv (\mathbb{m} \xrightarrow{=} \mathbb{m})$, generating the iterates of s_m among all permutations. From this we defined the group C_m through a quite general process which we define in this section, namely by taking the *set image* of R_m.[9]

[9] Later: the image of a homomorphism f is the group that is classified by the set image of Bf.

We also noted that the resulting pointed map from S^1 to $\mathrm{B}C_m$ was intimately tied up with the degree m map $\delta_m : S^1 \to_* S^1$ from Definition 3.6.5, pointed by reflexivity. In Section 5.3 we discussed δ_m as classifying a monomorphism in $\mathrm{Hom}(\mathbb{Z}, \mathbb{Z})$ that picks out exactly the iterates of $\circlearrowleft^m$. This process is also a special case of something, namely the *kernel*.

The relations between the cyclic groups in the forms $\mathbb{Z}/m$, C_m and C'_m as in Example 4.2.22 are also special cases of what we do in this section.

In our setup with a group homomorphism $f : \mathrm{Hom}(G, G')$ being given by a pointed function $Bf : BG \to_* BG'$, the above mentioned kernel, cokernel and image are just different aspects of the preimages

$$(Bf)^{-1}(w) \equiv \sum_{z : BG} (w \xrightarrow{=} Bf(z))$$

For those familiar with traditional group theory, the following summary may guide the intuition.

If $\phi : \mathrm{Hom}^{\mathrm{abs}}(\mathcal{G}, \mathcal{G}')$ is an abstract group homomorphism, the preimage $\phi^{-1}(e_{G'})$ is an abstract subgroup which is traditionally called the kernel of ϕ.

On the other hand, the cokernel is the quotient set of $\mathcal{G}'$ by the equivalence relation generated by $g' \sim g' \cdot \phi(g)$ whenever $g : \mathcal{G}$ and $g' : \mathcal{G}'$.

for $w : BG'$. Note that all these preimages are groupoids.

The kernel will correspond to a preferred component of the preimage of $\mathrm{sh}_{G'}$, the cokernel will be the (G'-)set of components and for the image we will choose the monomorphism into G' corresponding to the cokernel. This point of view makes it clear that the image will be a subgroup of G', the kernel will be a subgroup of G. In general the cokernel is a transitive, pointed G'-set (see Remark 9.3.5), but we will see in Definition 9.5.8 that in certain situations the cokernel also gives rise to a group called the quotient group.

9.3.1 *Kernels and cokernels*

The kernel of $f : \mathrm{Hom}(G, G')$ is a component of the fiber of Bf at the shape of G', whereas the cokernel is the G'-set of components of each fiber of Bf. We spell out the details.

sec:kerandcoker
def:kernel

Definition 9.3.2. We define a function

$$\ker : \mathrm{Hom}(G, G') \to \mathrm{Mono}(G)$$

which we call the *kernel*. If $f : \mathrm{Hom}(G, G')$ is a homomorphism we must specify the ingredients in $\ker(f) :\equiv (\mathrm{Ker}(f), \mathrm{in}_{\ker(f)}, !) : \mathrm{Mono}(G)$. The classifying type $\mathrm{BKer}(f)$ of the *kernel group*[10] (often just called the "kernel") is the component of the fiber of Bf pointed by

$$\mathrm{sh}_{\mathrm{Ker}(f)} :\equiv (\mathrm{sh}_G, Bf_{\mathrm{pt}}) : (Bf)^{-1}(\mathrm{sh}_{G'}),$$

where $Bf_{\mathrm{pt}} : \mathrm{sh}_{G'} \xrightarrow{=} Bf(\mathrm{sh}_G)$ is the pointing path of Bf.[11] The first projection $\mathrm{BKer}(f) \to BG$ is a set bundle, since by Corollary 2.9.11 the preimages are equivalent to the sets $\sum_{p : \mathrm{sh}_{G'} \xrightarrow{=} Bf(z)} \|\mathrm{sh}_{\mathrm{Ker}(f)} \xrightarrow{=} (z, p)\|$, giving a monomorphism $\mathrm{in}_{\ker(f)}$ of $\mathrm{Ker}(f)$ into G; together defining $\ker(f) :\equiv (\mathrm{Ker}(f), \mathrm{in}_{\ker(f)}, !) : \mathrm{Mono}(G)$. ⌟

Written out, the classifying type of the kernel, $\mathrm{BKer}(f)$, is[12]

$$\sum_{z : BG} \sum_{p : \mathrm{sh}_{G'} \xrightarrow{=} Bf(z)} \|(\mathrm{sh}_G, Bf_{\mathrm{pt}}) \xrightarrow{=} (z, p)\|$$

pointed at $(\mathrm{sh}_G, Bf_{\mathrm{pt}}, !)$, and $\mathrm{in}_{\ker(f)} : \mathrm{Hom}(\mathrm{Ker}(f), G)$ is given by the first projection. Indeed, the symmetries of the shape $\mathrm{sh}_{\mathrm{Ker}(f)}$ of the kernel of f are those $g : UG$ such that $g \cdot Bf_{\mathrm{pt}} = Bf_{\mathrm{pt}}$, i.e., $Uf(g) = \mathrm{refl}_{\mathrm{sh}_{G'}}$.

In the following definition and in the next subsection on the image of a homomorphism, we use the set truncation $\|_\|_0$ operation defined in Definition 2.22.4.

def:cokernel

Definition 9.3.3. Let $f : \mathrm{Hom}(G, G')$ be a homomorphism. The *cokernel* of f is[13] the G'-set

$$\mathrm{coker}(f) : BG' \to \mathrm{Set}, \qquad \mathrm{coker}(f)(w) :\equiv \|(Bf)^{-1}(w)\|_0,$$

defining a function of sets

$$\mathrm{coker} : \mathrm{Hom}(G, G') \to G'\text{-Set}.$$ ⌟

If a monomorphism i from G to G' is clear from the context, then $(G, i, !) : \mathrm{Mono}(G')$ is a subgroup of G', or, using the convention of Remark 5.3.24, $G \subseteq G'$. In that case we may write G'/G for the cokernel of i, generalizing the notation of Definition 9.5.8.

[10] There is an inherent ambiguity in our notation: is the kernel of f a group or a monomorphism into G? This is common usage and is only resolved by a type check.

[11] That is:

$$\mathrm{Ker}(f) :\equiv \mathrm{Aut}_{(Bf)^{-1}(\mathrm{sh}_{G'})}(\mathrm{sh}_G, Bf_{\mathrm{pt}})$$

[12] In the vocabulary of Chapter 5, $\mathrm{BKer}(f)$ is the component of the action type (Definition 5.4.1) of the restriction $f^*\mathbb{P}_{\mathrm{sh}_{G'}}$ of $\mathbb{P}_{\mathrm{sh}_G}$ to G (Definition 5.5.9) at $(\mathrm{sh}_G, Bf_{\mathrm{pt}})$. In this view we get $\mathrm{Ker}(f)$ as the stabilizer group (Definition 5.4.10) of $f^*\mathbb{P}_{\mathrm{sh}_{G'}}$ at $Bf_{\mathrm{pt}} : f^*\mathbb{P}_{\mathrm{sh}_{G'}}(\mathrm{sh}_G)$.

[13] In the vocabulary of Chapter 5, $\mathrm{coker}(f)$ is the induced G'-set $f_!(\mathrm{triv}_G \mathbb{1})$ (see Definition 5.4.1) of the constant G-set.

The associated $\mathrm{abs}(G')$-set $\mathrm{coker}(f)(\mathrm{sh}_{G'})$ is (also) referred to as the (abstract) cokernel of f.

lem:1sttrans(coker)

LEMMA 9.3.4. *For any* $f : \mathrm{Hom}(G, G')$, *the cokernel* $\mathrm{coker}(f)$ *is a transitive* G'*-set.*

Proof. We use many times that the goal is a proposition. First, since BG' is connected, it is enough to show that for all $u : \mathrm{coker}(\mathrm{sh}_{G'})$ there exists a $g : UG'$ such that $g \cdot |(\mathrm{sh}_G, Bf_{\mathrm{pt}})|_0 = u$. Second, it suffices to prove the latter for all $z : BG$ and $p : \mathrm{sh}_{G'} \xrightarrow{=} Bf(z)$, taking $u \equiv |(z, p)|_0$. Third, since BG is connected, it is enough to do this taking sh_G for z, and then $g \equiv p^{-1} Bf_{\mathrm{pt}}$ will do.[14] □

rem:imageandcokernel

REMARK 9.3.5. Since the cokernel is a transitive G'-set, we need just to provide $\mathrm{coker}(f)(\mathrm{sh}_{G'}) \equiv \|Bf^{-1}(\mathrm{sh}_{G'})\|_0$ with a point to say that the cokernel defines a subgroup of G'. The obvious point to choose is $|(\mathrm{sh}_G, Bf_{\mathrm{pt}})|_0$ and we get $(\mathrm{coker}(f), |(\mathrm{sh}_G, Bf_{\mathrm{pt}})|_0, !)) : \mathrm{Sub}(G')$, with $\mathrm{Sub}(G')$ the set of subgroups of G' as in Definition 5.3.2. In the next section we will consider this subgroup in more detail and call it the image of f.

Another proof of $\mathrm{coker}(f)$ being a transitive G'-set would be to use Lemma 5.2.22: since BG is connected and equivalent to $\sum_{w : BG'} Bf^{-1}(w)$, which maps surjectively onto $\sum_{w : BG'} \|Bf^{-1}(w)\|_0$, the latter type is connected.[15] ⌟

EXERCISE 9.3.6. Given a homomorphism $f : \mathrm{Hom}(G, G')$, prove that

(1) f is a monomorphism if and only if the kernel is trivial;

(2) f is an epimorphism if and only if the cokernel is contractible;

(3) if $h : \mathrm{Hom}(L, G)$ is a homomorphism such that $fh : \mathrm{Hom}(L, G')$ is the trivial homomorphism (equivalently, fh factors through the trivial group $\mathbb{1}$), then there is a unique $k : \mathrm{Hom}(L, \ker(f))$ such that $h \xrightarrow{=} \mathrm{in}_{\ker(f)}\, k$.[16] ⌟

The kernel, cokernel and image constructions satisfy a lot of important relations which we will review in a moment, but in our setup many of them are just complicated ways of interpreting the following fact about preimages (see Figure 9.2 in the margin). Here, for clarity, we have made the equivalence $\overline{(_, _)}$ from Definition 2.10.1, mapping pairs of paths to paths between pairs, explicit.

lem:fibersofcomposites

LEMMA 9.3.7. *Consider pointed functions* $(f_1, p_1) : (X_0, x_0) \to_* (X_1, x_1)$ *and* $(f_2, p_2) : (X_1, x_1) \to_* (X_2, x_2)$ *and the following functions*:

$$\begin{aligned} F_1 &: (f_2 f_1)^{-1}(x_2) \to f_2^{-1}(x_2), & F_1(x, q) &\equiv (f_1 x, q) \\ F_2 &: f_1^{-1}(x_1) \to (f_2 f_1)^{-1}(x_2), & F_2(x, p) &\equiv (x, f_2(p) p_2) \\ H &: F_1^{-1}(x_1, p_2) \to f_1^{-1}(x_1), & H(x, q, \overline{(p, r)}) &\equiv (x, p) \end{aligned}$$

Then we have the following:[17]

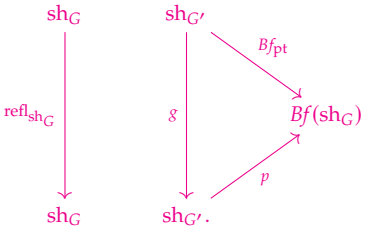
(1) *H is an equivalence with inverse*

$$H^{-1}(x, p) \equiv ((x, f_2(p) p_2), \overline{(p, \mathrm{refl}_{f_2(p) p_2})});$$

(2) *The lower square and its adjacent triangles commute by definition*;

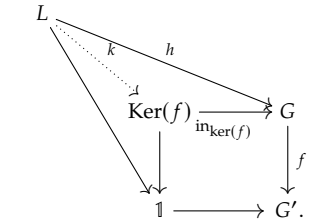
(3) *The left upper triangle, representing the identity* $F_2 H \xrightarrow{=} \mathrm{fst}$, *can be filled using function extensionality as follows*: *if* $(x, q, \overline{(p, r)}) : F_1^{-1}(x_1, p_2)$, *then* $\mathrm{fst}(x, q, \overline{(p, r)}) \equiv (x, q)$, *whereas* $F_2 H(x, q, \overline{(p, r)}) \equiv (x, f_2(p) p_2)$, *and the latter two pairs can be identified using* $r : f_2(p) p_2 \xrightarrow{=} q$.

[14] The third step could alternatively be done by induction on p. Note that the action of g does not affect sh_G. The situation is illustrated by the following diagram:



[15] In the vocabulary of Definition 3.9.3, this type is called the set image of Bf and the subgroup of G' associated with the cokernel is the "image" of f defined in the next section.

[16] Hint: consider the corresponding property of the preimage of Bf.



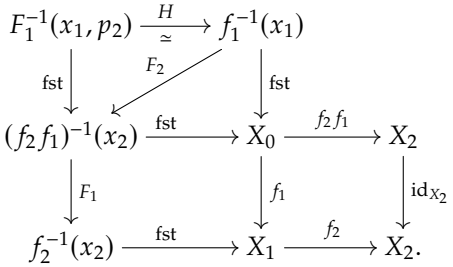


FIGURE 9.2: Crossing horizontal and vertical fiber sequences.

[17] Note that the codomain $f_1^{-1}(x_1)$ of H does not depend on f_2.

Proof. That H is an equivalence is seen by noting that $F_1^{-1}(x_1, p_2)$ is equivalent to $\sum_{x:X_0} \sum_{q:x_2 \xrightarrow{=} (f_2 f_1)(x)} \sum_{p:x_1 \xrightarrow{=} f_1(x)} (q \xrightarrow{=} f_2(p) p_2)$ and that H contracts away q as in Lemma 2.9.10, resulting in values in the (equivalent) codomain $f_1^{-1}(x_1) \equiv \sum_{x:X_0} (x_1 \xrightarrow{=} f_1(x))$. □

Hence, through univalence, H provides an identification

$$\bar{H} : (F_1^{-1}(x_1, p_2), \mathrm{fst}) \xrightarrow{=} (f_1^{-1}(x_1), F_2)$$

in the type $\sum_{X:\mathcal{U}} (X \to (f_2 f_1)^{-1}(x_2))$.

From the universal property of the pullback (Definition 8.4.1),[18] it furthermore follows that F_2 is the unique map such that $(\mathrm{fst}\, F_2) \xrightarrow{=}_{f_1^{-1}(x_1) \to X_0} \mathrm{fst}$ and H^{-1} is similarly unique with respect to $\mathrm{fst}\, H^{-1} \xrightarrow{=} F_2$.

[18] The fiber $(f_2 f_2)^{-1}$ is the pullback of $X_0 \xrightarrow{f_2 f_1} X_2 \xleftarrow{x_2} \mathbb{1}$.

Exercise 9.3.8. Fill the diagram in Figure 9.2 with all maps pointed. ⌟

The following corollary list a number of useful facts about the kernel and cokernel of the composition of two homomorphisms, illustrated by Figure 9.3. Figure 9.3 is an instantiation for groups of Figure 9.2, enhanced with pointing data as in Exercise 9.3.8, with all maps being induced maps on components.[19]

Corollary 9.3.9. *Consider two composable homomorphisms* $f_1 : \mathrm{Hom}(G_0, G_1)$ *and* $f_2 : \mathrm{Hom}(G_1, G_2)$.

(1) *The equivalence H from Lemma 9.3.7, pointed by reflexivity, induces an isomorphism* $\mathrm{Ker}(F_1) \xrightarrow{\simeq} \mathrm{Ker}(f_1)$ *which we also denote by H.*

(2) *There is a unique homomorphism F_1 from* $\mathrm{Ker}(f_2 f_1)$ *to* $\mathrm{Ker}(f_2)$ *such that* $f_1 \,\mathrm{in}_{\mathrm{ker}(f_2 f_1)} = \mathrm{in}_{\mathrm{ker}(f_2)} F_1$.

(3) *There is a unique monomorphism F_2 from* $\mathrm{Ker}(f_1)$ *to* $\mathrm{Ker}(f_2 f_1)$ *such that* $\mathrm{in}_{\mathrm{ker}(f_1)} = \mathrm{in}_{\mathrm{ker}(f_2 f_1)} F_2$.[20]

(4) *We have* $(\mathrm{Ker}(f_1), F_2) =_{\mathrm{Mono}(\mathrm{Ker}(f_2 f_1))} (\mathrm{Ker}(F_1), \mathrm{in}_{\mathrm{ker}(F_1)})$, *i.e., they are equal as subgroups of* $\mathrm{Ker}(f_2 f_1)$.

(5) *The equivalence H from Lemma 9.3.7 induces an identification $\bar{H}'$ of the restriction of the cokernel of f_1 along the kernel map of f_2 with the cokernel of F_1, that is,* $\bar{H}' : \mathrm{coker}(F_1) \xrightarrow{=}_{\mathrm{BKer}(f_2) \to \mathrm{Set}} \mathrm{coker}(f_1) \mathrm{Bin}_{\mathrm{ker}(f_2)}$.

(6) *If f_2 is a monomorphism, then* $F_2 : \mathrm{Hom}(\mathrm{Ker}(f_1), \mathrm{Ker}(f_2 f_1))$ *is an isomorphism. So, postcomposing with a monomorphism doesn't change the kernel.*

[19] Recall that any $f : A \to B$, given an element $a : A$, induces a map $A_{(a)} \to B_{(f(a))}$ of the respective components. If B has a given point $b : B$ and $f : (A, a) \to_* (B, b)$ is pointed, then f induces a pointed map on the components in the obvious way. The operation $(A, a) \mapsto (A_{(a)}, a)$ is a wild functor from $\mathcal{U}_*$ to itself.

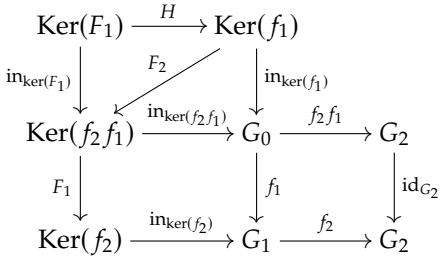


Figure 9.3: Kernels of composite homomorphisms.

[20] This nails the fact that $\mathrm{Ker}(f_1)$ is a subgroup of $\mathrm{Ker}(f_2 f_1)$.

TBsplit off Furthermore, the map F_1 defined in Lemma 9.3.7 can be generalized to $F_1'(w) : (f_2 f_1)^{-1}(w) \to f_2^{-1}(w)$, for $w : BG_2$, by abstracting from the points.[21] *By set-truncation (Example 6.4.6) followed by totalization (Definition 2.9.14), we get a map of G_2-sets:*[22]

$$\mathrm{tot}(\|F_1'(_)\|_0) : \mathrm{Hom}_{G_2}(\mathrm{coker}(f_2 f_1), \mathrm{coker}(f_2)).$$

Then we have the following:

(7) *If f_1 is an epimorphism, then* $\mathrm{tot}(\|F_1'(_)\|_0)$ *is an equivalence. So, precomposing with an epimorphism doesn't change the cokernel.*

[21] The (untyped) code of $F_1'(w)$ is the same as for F_1.

[22] Recall the notion of a map of G-sets from Definition 5.2.8 and the notation $\mathrm{Hom}_G(X, Y) \equiv \prod_{z:BG} (X(z) \to Y(z))$ for the type of such maps. Recall also that $\phi : \mathrm{Hom}_G(X, Y)$ is called an equivalence if $\prod_{z:BG} \mathrm{isEquiv}\, \phi(z)$, see Lemma 2.9.15.

xca:prd-fibersofcomposites
cor:cokermaps
it:H-equiv
it:F1
it:F2-mono
it:Ker-f1-F1
it:coker-f1-F1
it:monoF2->isoF2
it:epif1->equivF'1
ft:cmpnt:hom->grp
fig:kernelsofcomposites

Proof. The commutativity of Figure 9.3 follows from that of Figure 9.2 using Footnote 19. Points (1)-(4) follow immediately from Lemma 9.3.7, with (4) represented by the left upper commuting triangle. Unicity of F_1, F_2 follows by Definition 9.2.2 since injection into a kernel is mono.

Point (5) requires a construction, abstracting from x_1 and p_2 in the type of H from Lemma 9.3.7. For any $u : \mathrm{BKer}(f_2)$, define $H'(u) : BF_1^{-1}(u) \to Bf_1^{-1}(\mathrm{fst}(u))$ by the same (untyped) code as for H. For $u \equiv \mathrm{sh}_{\mathrm{Ker}(f_2)}$ we have $H'(u) \equiv H$ and $\mathrm{coker}(F_1)(u) \equiv \|BF_1^{-1}(\mathrm{sh}_{\mathrm{Ker}(f_2)})\|_0$ and $(\mathrm{coker}(f_1)\mathrm{Bin}_{\mathrm{ker}(f_2)})(u) \equiv \|Bf_1^{-1}(\mathrm{sh}_{G_1})\|_0$. Since $\mathrm{BKer}(f_2)$ is connected and H is an equivalence, it follows that $H'(u)$ is an equivalence for all u, from which the path $\bar{H}'$ is obtained.

For Point (6) we use notations as in Figure 9.2, i.e., x_i for sh_{G_i} ($i = 0, 1, 2$) and p_j for the pointing path of Bf_j ($j = 1, 2$). Assume f_2 is a monomorphism, then Bf_2 is a set bundle, so an injection on paths. The fiber of BF_2 at the shape $(x_0, Bf_2(p_1)p_2)$ of $\mathrm{BKer}(f_2 f_1)$ is

$$\sum_{(x,p)\,:\,\mathrm{BKer}(f_1)} \left((x_0, Bf_2(p_1)p_2) \xrightarrow{=} (x, Bf_2(p)p_2)\right).$$

By contracting away x this fiber can be identified with the type

$$\sum_{p\,:\,(x_1 \xrightarrow{=} Bf_1(x_0))} \left(\|p_1 = p\| \times (Bf_2(p_1)p_2 = Bf_2(p)p_2)\right),$$

which is contractible since Bf_2 is an injection on paths. Since $\mathrm{BKer}(f_2 f_1)$ is connected, all fibers of BF_2 are contractible.

For Point (7), assume that f_1 is an epimorphism. We have to show that $\mathrm{tot}(\|F_1'(_)\|_0)$ is an equivalence. Since BG_2 is connected, it suffices to show for $x_2 \equiv \mathrm{sh}_{G_2}$ that $\|F_1'(x_2)\|_0 : \|B(f_2 f_1)^{-1}(x_2)\|_0 \to \|Bf_2^{-1}(x_2)\|_0$ is an equivalence. The argument we use now is similar to that in (5), but we generalize H not to $u : \mathrm{BKer}(f_2)$, i.e., u in the component of the shape of $\mathrm{BKer}(f_2)$, but to $v : Bf_2^{-1}(x_2)$, i.e., v in the entire fiber. Define $H''(v) : F_1'(x_2)^{-1}(v) \to Bf_1^{-1}(\mathrm{fst}(v))$ by the same (untyped) code as H. Then $H''(v)$ is an equivalence for all $v : Bf_2^{-1}(x_2)$.[23]

Since $H''(v)$ is an equivalence, the assumption that f_1 an epimorphism implies by Lemma 9.2.6(3′) that the fiber $F_1'(x_2)^{-1}(b)$ is connected, for all $b : Bf_2^{-1}(x_2)$. However, we need that the fibers of $\|F_1'(x_2)\|_0$ are contractible. This will be proved in the following sequence of propositions, each implied by the next proposition in the sequence:

(a) $\|F_1'(x_2)\|_0^{-1}(a)$ is contractible, for all $a : \|Bf_2^{-1}(x_2)\|_0$

(b) $\|F_1'(x_2)\|_0^{-1}(|b|_0)$ is contractible, for all $b : Bf_2^{-1}(x_2)$[24]

(c) $\sum_{c\,:\,\|B(f_2 f_1)^{-1}(x_2)\|_0}(|b|_0 = \|F_1'(x_2)\|_0(c))$ is contractible[25]

(d) $\sum_{d\,:\,B(f_2 f_1)^{-1}(x_2)}(|b|_0 = \|F_1'(x_2)\|_0(|d|_0))$ is connected[26]

(e) $\sum_{d\,:\,B(f_2 f_1)^{-1}(x_2)}\|b \xrightarrow{=} F_1'(x_2)(d)\|$ is connected[27]

(f) $\sum_{d\,:\,B(f_2 f_1)^{-1}(x_2)}(b \xrightarrow{=} F_1'(x_2)(d))$ is connected

The last point follows by the assumption that f_1 is an epimorphism, as argued above. □

Example 9.3.10. New: A simple case of Lemma 9.3.7 is when the types X_0, X_1, X_2 are sets. Then all fibers involved are subsets and $F_2(x, !) =$

[23] In (5) we used that $\mathrm{BKer}(f_2)$ is connected, but this is not necessarily the case for the fiber $Bf_2^{-1}(x_2)$. Therefore we resort to contracting away such as for H in the proof of Lemma 9.3.7.

[24] All subsequent points are also for all $b : Bf_2^{-1}(x_2)$.

[25] By next step, using Exercises 2.22.5 and 2.22.6.

[26] By next step, using Remark 2.22.18.

[27] By next step, using Exercise 2.16.12.

exa:fibersofcomposites

$(x, !), F_1(x, !) = (f_1(x), !)$, and H is an equivalence since $x_1 = f_1(x)$ implies $x_2 = (f_2 f_1)(x)$ for any $x : X_0$.

A more interesting example is obtained by considering

$$\mathrm{B}\mathbb{Z} \xrightarrow[f_1]{\mathrm{Bmod}_4} \mathrm{BC}_4 \xrightarrow[f_2]{\mathrm{Bsgn\,pr}} \mathrm{B}\Sigma_2.$$

Here Bmod_4 is defined in Example 4.4.13(4) Note that $\mathrm{s} : 4 \xrightarrow{\simeq} 4$ in the definition of Bmod_4 is the odd cyclic permutation $(32)(21)(10) = (0123)$ (Definition 4.5.9). The cycle structure is forgotten by $\mathrm{pr} : \mathrm{Cyc}_4 \to_* \mathrm{FinSet}_4$, $(X, t) \mapsto X$, from Example 4.2.23, after which Bsgn from Definition 4.5.6, the classifying map of the sign homomorphim, is applied. All maps involved are pointed by reflexivity.

The fibers involved are, with $x_1 \equiv (4, \mathrm{s})$ and $x_2 \equiv 2$,

$$\begin{aligned}
f_1^{-1}(x_1) &\equiv \sum_{z:\mathrm{S}^1} ((4, \mathrm{s}) \xrightarrow{=} \mathrm{Bmod}_4(z)) \text{ pointed at } (\bullet, \mathrm{refl}_{(4,\mathrm{s})}),\\
f_2^{-1}(x_2) &\equiv \sum_{(X,t):\mathrm{Cyc}_4} (2 \xrightarrow{=} \mathrm{Bsgn}(X)) \text{ pointed at } (4, \mathrm{s}, \mathrm{refl}_2),\\
(f_2 f_1)^{-1}(x_2) &\equiv \sum_{z:\mathrm{S}^1} (2 \xrightarrow{=} (\mathrm{Bsgn\,pr\,Bmod}_4)(z)) \text{ pointed at } (\bullet, \mathrm{refl}_2).
\end{aligned}$$

These fibers are action types (Definition 5.4.1) of G-sets (Definition 5.2.1), so they are groupoids. They happen to be connected (since the maps classify epimorphisms) and with canonical points given above. Hence they classify subgroups, see Definition 5.3.2, which we elaborate now.

Consider the $\mathbb{Z}$-set associated with $f_1 \equiv \mathrm{Bmod}_4$, sending $z : \mathrm{S}^1$ to $(4, \mathrm{s}) \xrightarrow{=} \mathrm{Bmod}_4(z)$, with underlying set pointed at $\mathrm{refl}_{(4,\mathrm{s})}$. The stabilizer group (Definition 5.4.10) of this point picks out the symmetries $\circlearrowleft^{4k} : (\bullet \xrightarrow{=} \bullet)$ for all $k : \mathrm{Z}$.

In a similar way we find that for the C_4-set associated with $f_2 \equiv \mathrm{Bsgn\,pr}$, sending $(X, t) : \mathrm{BC}_4$ to $(2 \xrightarrow{=} \mathrm{Bsgn}(X)$, we get the symmetries s^k of $(4, \mathrm{s})$ with $k = 0, 2$, that is, the two even permutations of 4.

Finally, for the $\mathbb{Z}$-set associated with $f_2 f_1 \equiv \mathrm{Bsgn\,pr\,Bmod}_4$, sending $z : \mathrm{S}^1$ to $2 \xrightarrow{=} (\mathrm{Bsgn\,pr\,Bmod}_4)$, we get the symmetries $\circlearrowleft^{2k} : (\bullet \xrightarrow{=} \bullet)$ for all $k : \mathrm{Z}$. These are the symmetries in the circle that, via an even permutation of 4, map to the identity permutation of 2, under the application of f_1 and f_2 to paths.

Furthermore, $F_1 : (f_2 f_1)^{-1}(x_2) \to_* f_2^{-1}(x_2)$ (pointed by reflexivity) is a homomorphism, and F_1^{abs} maps $\circlearrowleft^{2k}$ to $(0123)^{2k} = ((02)(13))^k$. The lemma now states that contracting away q in

$$F_1^{-1}(4, \mathrm{s}, \mathrm{refl}_2) \equiv \sum_{(z,q):(f_2 f_1)^{-1}(x_2)} (4, \mathrm{s}, \mathrm{refl}_2) \xrightarrow{=} (\mathrm{Bmod}_4(z), q)$$

identifies the latter type with the action type $\sum_{z:\mathrm{S}^1}((4, \mathrm{s}) \xrightarrow{=} \mathrm{Bmod}_4(z))$. ⌟

The example above is not as general as possible since all fibers involved are connected. See the following exercise for a more general case.

Exercise 9.3.11. Elaborate the details of Example 9.3.10 in the following case, with R_4 as in Example 4.2.23:

$$\mathrm{B}\mathbb{Z} \xrightarrow{R_4} \mathrm{B}\Sigma_4 \xrightarrow{\mathrm{Bsgn}} \mathrm{B}\Sigma_2. \qquad ⌟$$

xca:fibersofcomposites

xca:abstract-kernel

EXERCISE 9.3.12. Let $f : \mathrm{Hom}(G,H)$. Let $X(f) : \mathrm{Sub}(G)$ be the transitive G-set sending $z : BG$ to

$$\sum_{p : \mathrm{sh}_H \xrightarrow{=} Bf(z)} \exists_{g : \mathrm{sh}_G \xrightarrow{=} z} (p \xrightarrow{=} Bf(g) Bf_{\mathrm{pt}}),$$

with the chosen point $Bf_{\mathrm{pt}} : X(f)(\mathrm{sh}_G)$. Give an identification of $X(f)$ with the subgroup $E(\ker(f)) : \mathrm{Sub}(G)$ associated with the kernel, see Lemma 5.3.18. Also show that, if f is an epimorphism, $X(f)$ can be simplified to

$$z \mapsto (\mathrm{sh}_H \xrightarrow{=} Bf(z)). \qquad \lrcorner$$

9.3.13 *The image of a homomorphism of groups*

sec:image

Given types A, B and a function $f : A \to B$, we defined in Definition 2.17.11 the (propositional) image $\mathrm{im}(f)$ of f as a subtype of B, as well as the unique factorization of f into a surjection $A \to \mathrm{im}(f)$ followed by an injection $\mathrm{im}(f) \to B$. Here we will study these concepts for groups.

Given groups G, H and a homomorphism $f : \mathrm{Hom}(G,H)$, as the classifying types of groups are connected, $\mathrm{im}(Bf)$ is uninformative from the group perspective.[28] Therefore we use the set image, or 0-image from Section 3.9:

$$\mathrm{im}_0(Bf) \equiv \sum_{w : BH} \| \sum_{z : BG} (w \xrightarrow{=} Bf(z)) \|_0.$$

[28] The map $\mathrm{fst} : \mathrm{im}(Bf) \to BH$ is an equivalence, as $\|(Bf)^{-1}(w)\|$ is contractible for all $w : BH$.

def:im(grphom)

DEFINITION 9.3.14. Let G and H be groups and $f : \mathrm{Hom}(G,H)$ a homomorphism. Define the *image of* f to be the group[29]

$$\mathrm{Im}(f) :\equiv \underline{\Omega}(\mathrm{im}_0(Bf), (\mathrm{sh}_H, |(\mathrm{sh}_G, Bf_{\mathrm{pt}})|_0)). \qquad \lrcorner$$

[29] The groupoid $\mathrm{im}_0(Bf)$ is also the action type (Definition 5.4.1) of the cokernel of f (Definition 9.3.3), and is connected since the cokernel is transitive (Lemma 9.3.4).

In other words, the image is the subgroup of H associated with the cokernel as discussed in Remark 9.3.5, including the choice of the shape.

We will show that every $f : \mathrm{Hom}(G,H)$ has a unique factorization as an epimorphism $p(f) : \mathrm{Hom}(G, \mathrm{Im}(f))$ followed by a monomorphism $i(f) : \mathrm{Hom}(\mathrm{Im}(f), H)$. For the underlying unpointed maps of $Bf, Bp(f), Bi(f)$ (denoted in the same way) we use the factorization of Theorem 3.9.11, point $Bi(f)$ by reflexivity, and leave it as an exercise to point $Bp(f)$:

$$\begin{aligned} &Bp(f) : BG \to \mathrm{im}_0(Bf) && Bp(f)(z) :\equiv (Bf(z), |(z, \mathrm{refl}_{Bf(z)})|_0), \\ &Bi(f) : \mathrm{im}_0(Bf) \to BH && Bi(f) :\equiv \mathrm{fst}, \text{ pointed by reflexivity.} \end{aligned}$$

Under these definitions we have $Bf \equiv Bi(f) \circ Bp(f)$ for the unpointed maps.

xca:p-epi-i-mono

EXERCISE 9.3.15. Give the pointing path of $Bp(f)$ such that $Bf = Bi(f) \circ Bp(f)$ for the pointed maps. Hint: use induction on Bf_{pt}. Show that $Bp(f)$ and $Bi(f)$ are an epimorphism and a monomorphism, respectively. Hint: for the epimorphism, use Lemma 3.9.6. $\lrcorner$

That the image gives a *unique factorization* is elegantly expressed using the pullback construction from Definition 8.4.1 to express the type of epi/mono factorizations of homomorphisms from G to H as $\mathrm{Epi}(G) \times_{\mathrm{Group}} \mathrm{Mono}(H)$ where the maps to Group are understood

to be the first projections (so that the epimorphisms and monomorphisms in question can, indeed, be composed, using a mediating path/isomorphism).[30] Recall that the pullback type in question is defined by

$$\sum_{(Z,p):\mathrm{Epi}(G)} \sum_{(Z',i):\mathrm{Mono}(H)} (Z \xrightarrow{=} Z').$$

[30] This is the price one has to pay for using the pullback. Alternatively, one could use the simpler type obtained by contracting away Z' in the displayed pullback type.

After all these preparations we can now define the factorization map and state the property that it is an equivalence.

def:im-fact

Definition 9.3.16. For all groups G and H the map

$$\mathrm{fact} : \mathrm{Hom}(G,H) \to (\mathrm{Epi}(G) \times_{\mathrm{Group}} \mathrm{Mono}(H))$$

is defined by $\mathrm{fact}(f) :\equiv ((\mathrm{Im}(f), p(f)), (\mathrm{Im}(f), i(f)), \mathrm{refl}_{\mathrm{Im}(f)})$. ⌟

lem:im-fact-unique

Lemma 9.3.17. *For all groups G and H the map* $\mathrm{fact}(f)$ *is an equivalence.*

We prove this lemma by applying Theorem 3.9.11 in the special case $n = 0$, but for pointed maps instead of unpointed ones. We use the occasion to extend Theorem 3.9.11 to pointed maps for all $n \geq -2$, reusing several results from Section 3.9.

rem:n-im-prd-map

Remark 9.3.18. We extend the type $\mathrm{Fact}(f)$ from Definition 3.9.10 to pointed maps f. Let X and Z be pointed types. For any pointed map $f : X \to_* Z$ we define the type $\mathrm{Fact}_*(f)$ of factorizations of f as follows:

$$\mathrm{Fact}_*(f) :\equiv \sum_{Y:\mathcal{U}_*} \sum_{p:X\to_* Y} \sum_{i:Y\to_* Z} f \xrightarrow{=} ip.$$

Then the following type $\mathrm{Fact}_*^n(f)$ of *factorizations of f through its n-image* is contractible for any $n \geq -2$:

$$\mathrm{Fact}_*^n(f) :\equiv \sum_{(I,p,i,h):\,\mathrm{Fact}_*(f)} (\mathrm{is}n\mathrm{Conn}(p) \times \mathrm{is}n\mathrm{Trunc}(i)).$$

ft:(i,p)-equiv

Consider the factorization of $f : X \to Z$ from Theorem 3.9.11:

$$X \xrightarrow{x\mapsto (f(x),|(x,\mathrm{refl}_{f(x)})|_n)} \textstyle\sum_{z:Z} \|f^{-1}(z)\|_n \xrightarrow{\mathrm{fst}} Z.$$

By the same argument as used for Equation (3.9.2), the left map is n-connected, and fst is n-truncated. By giving a pointing path for the left map as in Exercise 9.3.15, generalized for $n > 0$, and with fst pointed by reflexivity, we already have a candidate center of $\mathrm{Fact}_*^n(f)$.

Given an n-connected $p : X \to Y$ (i.e., for all $y : Y$ the n-truncation of $p^{-1}(y)$ is contractible), an n-truncated $i : Y \to Z$ (i.e., for all $z : Z$ the fiber $i^{-1}(z)$ is n-truncated), and $h : \prod_{x:X}(f(x) \xrightarrow{=} i(p(x)))$ (coming from the identification $f \xrightarrow{=} ip$ in $\mathrm{Fact}_*(f)$), we have for each $z : Z$ an equivalence[31]

$$(i,p)_z^h : \|f^{-1}(z)\|_n \simeq i^{-1}(z), \quad |(x,q)|_n \mapsto (p(x), h(x)q).$$

Hence $(i,p)_z^h$ induces an equivalence from $\sum_{z:Z}\|f^{-1}(z)\|_n$ to $\sum_{z:Z} i^{-1}(z)$, and further to Y by using Lemma 2.25.2. This equivalence maps $(z, |(x,q)|_n)$ via $(z,(x,q))$ to $p(x)$, and is denoted by p^h in Figure 9.4. For the unpointed maps, the left triangle can easily be filled by reflexivity, and the upper right triangle as well since the first leg of p^h is fiberwise. The lower right triangle is filled mapping $(z,(y,r)) : \sum_{z:Z} i^{-1}(z)$ to $r : z \xrightarrow{=} i(y)$.[32] This identifies the two unpointed factorizations of f.

[31] To construct this equivalence, use h and known equivalences to get

$$\begin{aligned} f^{-1}(z) \simeq (ip)^{-1}(z) &\equiv \sum_{x:X} (z \xrightarrow{=} i(p(x))) \\ &\simeq \sum_{x:X}\sum_{y:Y} ((z \xrightarrow{=} i(y)) \times (y \xrightarrow{=} p(x))) \\ &\simeq \sum_{y:Y} ((z \xrightarrow{=} i(y)) \times p^{-1}(y)). \end{aligned}$$

Now n-truncate left and right, then use Exercise 3.9.9 to also n-truncate $p^{-1}(y)$. By the assumption on p, the latter type is contractible, and the right hand side simplifies to $i^{-1}(z)$ by the assumption on i.

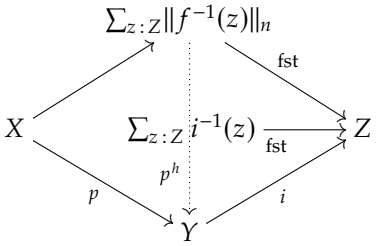


Figure 9.4: Two n-image factorizations of f.

[32] Alternatively, we can use the inverse of p^h (Exercise 2.21.11) to fill the right (outer) triangle and identify $\mathrm{fst} \circ (p^h)^{-1}$ with i by reflexivity.

The are two remaining proof obligations before we can conclude that $\mathrm{Fact}^n_*(f)$ is contractible: (1) we have to point p^h, and (2) we have to fill the diagram in Figure 9.4 for pointed maps. For both we need the point data, which we give next.

The pointing paths of f, p and i are $f_{\mathrm{pt}} : \mathrm{pt}_Z \xrightarrow{=} f(\mathrm{pt}_X)$, $p_{\mathrm{pt}} : \mathrm{pt}_Y \xrightarrow{=} p(\mathrm{pt}_X)$ and $i_{\mathrm{pt}} : \mathrm{pt}_Z \xrightarrow{=} i(\mathrm{pt}_Y)$, respectively. We point $\sum_{z:Z} \|f^{-1}(z)\|_n$ at $(\mathrm{pt}_Z, |(\mathrm{pt}_X, f_{\mathrm{pt}})|_n)$, using $(\mathrm{pt}_X, f_{\mathrm{pt}})$ as the natural point of the fiber $f^{-1}(\mathrm{pt}_Z)$.[33] The pointing path of the equivalence p^h is p_{pt}. The first projections are pointed by reflexivity. Furthermore, from the identification $f \xrightarrow{=} ip$ in $\mathrm{Fact}_*(f)$ we not only get $h : \prod_{x:X}(f(x) \xrightarrow{=} i(p(x)))$ as mentioned above, but also a path

$$h_{\mathrm{pt}} : (h(\mathrm{pt}_X) f_{\mathrm{pt}}) \xrightarrow{=} (ip)_{\mathrm{pt}}.$$

[33] Cf. in Definition 9.3.14, when f is a group homomorphism, the shape of $\mathrm{Im}(f)$.

We are now ready to fill the triangles Figure 9.4 for the pointed maps. The right upper triangle is filled by reflexivity, since the first leg of p_h is fiberwise. For the right lower triangle, recall that the filler for the unpointed maps sends the point $(\mathrm{pt}_Z, (\mathrm{pt}_Y, i_{\mathrm{pt}}))$ to i_{pt}.

Recall that the filler for the unpointed maps in this triangle sends $(\mathrm{pt}_Z, |(\mathrm{pt}_X, f_{\mathrm{pt}})|_n)$ to $h(\mathrm{pt}_X) f_{\mathrm{pt}}$. Now h_{pt} fills the triangle for the pointing paths $(ip)_{\mathrm{pt}}$ and $\mathrm{fst}_{\mathrm{pt}} \equiv \mathrm{refl}_{\mathrm{pt}_Z}$.

For the upper left triangle, recall that the left vertical map $x \mapsto (f(x), |(x, \mathrm{refl}_{f(x)})|_n)$ is pointed by the path $(\mathrm{pt}_Z, |(\mathrm{pt}_X, f_{\mathrm{pt}})|_n) \xrightarrow{=} (f(\mathrm{pt}_X), |(\mathrm{pt}_X, \mathrm{refl}_{f(\mathrm{pt}_X)})|_n)$ obtained by path induction on f_{pt} and reflexivity. Hence the left upper triangle can also be filled by path induction on f_{pt} and reflexivity, given that both p and p^h are pointed by p_{pt}.

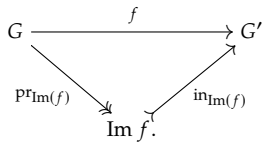
We can now conclude that the type $\mathrm{Fact}^n_*(f)$ is contractible since we have proved that all its elements are equal and we had already given an element. ⌟

CURSOR

Proof. given by composition,[34]

$$\circ((Z, p, !), (Z', j, !), \alpha) :\equiv j\tilde{\alpha}p$$

is an equivalence with inverse given by the image factorization.



□

[34] Here p is an epimorphism from G to the group Z, j a monomorphism from the group Z' to H, α is an identification of Z and Z' and $\tilde{\alpha}$ is the isomorphism corresponding to the identification $\alpha : Z \xrightarrow{=} Z'$, as in Definition 2.13.1, so that the composite looks like

$$G \xrightarrow{p} Z \xrightarrow[\sim]{\tilde{\alpha}} Z' \xrightarrow{j} H$$

def: image

Definition 9.3.19. Explicitly, the image factorization for groups is the function

$$\circ^{-1} : \mathrm{Hom}(G, G') \to \mathrm{Epi}(G) \times_{\mathrm{Group}} \mathrm{Mono}(G')$$
$$\circ^{-1}(f) :\equiv ((\mathrm{Im}(f), \mathrm{pr}_{\mathrm{im}(f)}, !), (\mathrm{Im}(f), \mathrm{in}_{\mathrm{im}(f)}, !), \mathrm{refl}_{\mathrm{Im}(f)}),$$

where as before the *image group* is the group

$$\mathrm{Im}(f) :\equiv \mathrm{Aut}_{\sum_{z:BG'} \mathrm{coker}(f)(z)}(\mathrm{sh}_{G'}, |(\mathrm{sh}_G, Bf_{\mathrm{pt}})|_0),$$

the monomorphism $\mathrm{in}_{\mathrm{im}(f)}$ is obtained from the wrapping of the first projection

$$\mathrm{Bin}_{\mathrm{im}(f)} :\equiv \mathrm{fst} : B\,\mathrm{Im}(f) \to BG'$$

and the epimorphism $\mathrm{pr}_{\mathrm{im}(f)}$ is given on the level of classifying types by sending $x : BG$ to

$$B\mathrm{pr}_{\mathrm{Im}(f)} f(x) :\equiv (Bf(x), |(x, \mathrm{refl}_{Bf(x)})|_0) : \mathrm{B\,Im}\, f.$$

Occasionally we may refer to the two projections of the image factorization

$$\begin{aligned} \mathrm{im} &: \mathrm{Hom}(G,G') \to \mathrm{Mono}(G'), \qquad \mathrm{im}(f) :\equiv (\mathrm{Im}(f), \mathrm{in}_{\mathrm{im}(f)}, !) \\ \mathrm{pr}^{\mathrm{im}} &: \mathrm{Hom}(G,G') \to \mathrm{Epi}(G), \qquad \mathrm{pr}^{\mathrm{im}} f :\equiv (\mathrm{Im}(f), \mathrm{pr}_{\mathrm{im}(f)}, !) \end{aligned}$$

as the *image* and the *projection to the image*. ⌟

In view of Exercise 9.3.20 below, the families

$$\mathrm{isepi}, \mathrm{ismono} : \mathrm{Hom}(G,G') \to \mathrm{Prop}$$

of propositions that a given homomorphism is an epimorphism or monomorphism have several useful interpretations (parts of the exercise have already been done).

ex.charsurinj

Exercise 9.3.20. Let $f : \mathrm{Hom}(G,G')$ Prove that

(1) the following are equivalent
   a) f is an epimorphism,
   b) Uf is a surjection
   c) the cokernel of f is contractible,
   d) the inclusion of the image $\mathrm{in}_{\mathrm{im}(f)} : \mathrm{Hom}(\mathrm{Im}\, f, G')$ is an isomorphism,

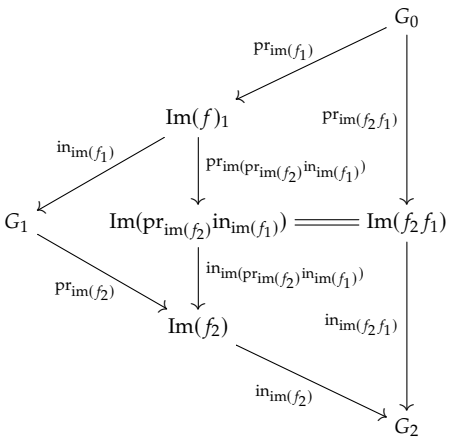
(2) the following are equivalent
   a) f is a monomorphism,
   b) Uf is an injection
   c) the kernel of f is trivial
   d) $Bf : BG \to BG'$ is a set bundle.
   e) the projection onto the image $\mathrm{pr}_{\mathrm{im}(f)} : \mathrm{Hom}(G, \mathrm{Im}(f))$ is an isomorphism.

⌟

We need to understand how the image factorization handles composition of homomorphisms. This is forced by the uniqueness as follows.

Lemma 9.3.21. *Given composable homomorphisms* $f_1 : \mathrm{Hom}(G_0, G_1)$ *and* $f_2 : \mathrm{Hom}(G_1, G_2)$*, unique factorization induces identifications*

$$\begin{aligned} \mathrm{im}(f_2 f_1) &\xrightarrow{=}_{\mathrm{Mono}(G_2)} (\mathrm{Im}(\mathrm{pr}_{\mathrm{im}(f_2)}\mathrm{in}_{\mathrm{im}(f_1)}), \mathrm{in}_{\mathrm{im}(f_2)}\mathrm{in}_{\mathrm{im}(\mathrm{pr}_{\mathrm{im}(f_2)}\mathrm{in}_{\mathrm{im}(f_1)})}, !) \\ \mathrm{pr}^{\mathrm{im}}(f_2 f_1) &\xrightarrow{=}_{\mathrm{Epi}(G_0)} (\mathrm{Im}(\mathrm{pr}_{\mathrm{im}(f_2)}\mathrm{in}_{\mathrm{im}(f_1)}), \mathrm{pr}_{\mathrm{im}(\mathrm{pr}_{\mathrm{im}(f_2)}\mathrm{in}_{\mathrm{im}(f_1)})}\mathrm{pr}_{\mathrm{im}(f_1)}, !) \end{aligned}$$



Proof. Since composition preserves monomorphisms and epimorphisms – in particular $\mathrm{in}_{\mathrm{im}(f_2)}\mathrm{in}_{\mathrm{im}(\mathrm{pr}_{\mathrm{im}(f_2)}\mathrm{in}_{\mathrm{im}(f_1)})} : \mathrm{Hom}(\mathrm{Im}(\mathrm{pr}_{\mathrm{im}(f_2)}\mathrm{in}_{\mathrm{im}(f_1)}), G_2)$ is a monomorphism and $\mathrm{pr}_{\mathrm{im}(\mathrm{pr}_{\mathrm{im}(f_2)}\mathrm{in}_{\mathrm{im}(f_1)})}\mathrm{pr}_{\mathrm{im}(f_1)} : \mathrm{Hom}(G_0, \mathrm{Im}(\mathrm{pr}_{\mathrm{im}(f_2)}\mathrm{in}_{\mathrm{im}(f_1)}))$ is an epimorphism – this is just uniqueness of the image factorization of the composite $f_2 f_1$. □

lem:kerandcoker

Lemma 9.3.22. *Let* $f : \mathrm{Hom}(G,G')$ *be a group homomorphism. The induced map* $(B\mathrm{pr}_{\mathrm{im}(f)})^{-1}(\mathrm{sh}_{\mathrm{Im}(f)}) \to (Bf)^{-1}(\mathrm{sh}_{G'})$ *gives an identification*

$$\ker \mathrm{pr}_{\mathrm{im}(f)} \xrightarrow{=}_{\mathrm{Mono}(G)} \ker(f).$$

Proof. Using univalence, this is a special case of Corollary 9.3.9 with $f_2 :\equiv \mathrm{in}_{\mathrm{im}(f)}$ and $f_1 :\equiv \mathrm{pr}_{\mathrm{im}(f)}$. [35] □

[35] See also counting results for finite groups.

Exercise 9.3.23. (1) If $f : \mathrm{Mono}(G')$, then $\mathrm{ua}(\mathrm{pr}_{\mathrm{im}(f)}) : f \xrightarrow{=}_{\mathrm{Mono}(G')} \mathrm{in}_{\mathrm{im}(f)}$.

(2) If $f : \mathrm{Epi}(G)$, then $\mathrm{ua}(\mathrm{in}_{\mathrm{im}(f)}) : f \xrightarrow{=}_{\mathrm{Epi}(G)} \mathrm{pr}_{\mathrm{im}(f)}$.

(True propositions suppressed). ⌟

Example 9.3.24. An example from linear algebra: let A be any $n \times n$-matrix with nonzero determinant and with integer entries, considered as a homomorphism $A : \mathrm{Hom}(\mathbb{Z}^n, \mathbb{Z}^n)$. "Nonzero determinant" corresponds to "monomorphism". Then the cokernel of A is a finite set with cardinality the absolute value of the determinant of A. You should picture A as a $|\det(A)|$-fold set bundle of the n-fold torus $(S^1)^{\times n}$ by itself.

In general, for an $m \times n$-matrix A, then the "nullspace" is given by the kernel and the "rowspace" is given by the image. ⌟

9.4 *The action on the set of subgroups*

sec:actiononsub

Not only is the type of subgroups of G a set, it is in a natural way also a G-set, which we denote by the same name.[36] We first do the monomorphism interpretation based on $\mathrm{Mono}(G)$ from Definition 5.3.13.

[36] Just as we did for $\mathrm{Hom}(H, G)$ as a G-set in Example 5.2.6.

Definition 9.4.1. Let G be a group. The G-set $\mathrm{Mono}(G)$ *of monomorphisms into* G is defined as the family of sets $\mathrm{Mono}(\underline{\Omega}(BG_{\div}, _))$:

$$\mathrm{Mono}(G) : BG \to \mathrm{Set}, \qquad z \mapsto \mathrm{Mono}(\underline{\Omega}(BG_{\div}, z)).$$

The group action of the G-set $\mathrm{Mono}(G)$, elaborated below in Remark 9.4.2, is called *conjugation*. ⌟

Note that the underlying set $\mathrm{Mono}(G)(\mathrm{sh}_G)$ of the G-set $\mathrm{Mono}(G)$ is definitionally equal to the set $\mathrm{Mono}(G)$. Further, $\mathrm{Mono}(G)(z)$ can be unpacked as $\sum_{H : \mathrm{Group}} \sum_{f : \mathrm{Hom}(H, \underline{\Omega}(BG_{\div}, z))} \mathrm{isMono}(f)$.

Remark 9.4.2. As $z : BG$ varies in $\mathrm{Mono}(\underline{\Omega}(BG_{\div}, z))$, the only thing that changes is the shape z. This has been analyzed in Example 4.4.21 for the group $\underline{\Omega}(BG_{\div}, z)$, and in Example 7.7.2 for $\mathrm{Hom}(H, \underline{\Omega}(BG_{\div}, z))$. Here we must do this for

rem:action-Mono(G)

$$\mathrm{Mono}(\underline{\Omega}(BG_{\div}, z)) \equiv \sum_{H : \mathrm{Group}} \sum_{f : \mathrm{Hom}(H, \underline{\Omega}(BG_{\div}, z))} \mathrm{isMono}(f),$$

in which H and $\mathrm{isMono}(f)$ do not depend on z.[37]

Hence Example 7.7.2 applies and the application of $\mathrm{Mono}(G)$ to a path $p : (z \xrightarrow{=} z')$ maps $(H, f) : \mathrm{Mono}(G)(z)$ to $(H, \underline{\Omega}(\mathrm{id}_{BG}, p^{-1})f) : \mathrm{Mono}(G)(z')$.[38] In other words, the action of the G-set $\mathrm{Mono}(G)$ is postcomposing the monomorphism f in (H, f) with the isomorphism $\underline{\Omega}(\mathrm{id}_{BG}, p^{-1})$. We call this action *conjugation*, and we call $(H, \underline{\Omega}(\mathrm{id}_{BG}, p^{-1})f)$ the *conjugate of* (H, f) *by* p.[39] ⌟

As a consequence of Remark 9.4.2, taking the equivalence of Theorem 7.5.3 between the categories Group and $\mathrm{Group}^{\mathrm{abs}}$ into account, we get:

[37] Recall that $\mathrm{isMono}(f)$ is equivalent to $Bf_{\div}$ being a set bundle.

[38] Note that this transports the pointing path $Bf_{\mathrm{pt}} : z \xrightarrow{=} Bf(\mathrm{sh}_H)$ to $Bf_{\mathrm{pt}} p^{-1} : z' \xrightarrow{=} Bf(\mathrm{sh}_H)$.

[39] Example 4.4.21 observes that $\Omega(\mathrm{id}_{BG}, p^{-1})$ and p_p^{-1} are equal maps of type $(z \xrightarrow{=} z) \to (z' \xrightarrow{=} z')$, explaining this terminology. BTW, how do you move the shape in the case of strictly pointed maps?

lem:conj-abstract

Lemma 9.4.3. *Under the equivalence of Lemma 7.7.1 between G-sets and* $\mathrm{abs}(G)$*-sets, the G-set* $\mathrm{Mono}(G)$ *corresponds to the* $\mathrm{abs}(G)$*-set*

$$\sum_{\mathcal{H}:\mathrm{Group}^{\mathrm{abs}}} \sum_{\varphi:\mathrm{Hom}^{\mathrm{abs}}(\mathcal{H},\mathrm{abs}(G))} \mathrm{isMono}(\varphi)$$

of abstract monomorphisms into $\mathrm{abs}(G)$*, with action*

$$g \cdot (\mathcal{H}, \varphi) :\equiv (\mathcal{H}, \mathrm{conj}^g\, \varphi), \quad \textit{for all } g : \mathrm{abs}(G).$$

Here $\mathrm{conj}^g : \mathrm{abs}(G) \xrightarrow{=} \mathrm{abs}(G)$ *is conjugation as defined in Example 7.3.5.*

xca:conj-abstract

Exercise 9.4.4. Prove Lemma 9.4.3. ⌟

rem:typeofsubgpstrivfab

Remark 9.4.5. As defined in Definition 4.2.31, a group G is abelian if all symmetries of its shape commute. This condition is equivalent to $\mathrm{conj}^g = \mathrm{id}$ for all $g : UG$. It follows that the $\mathrm{abs}(G)$-set from Lemma 9.4.3, and hence the G-set $\mathrm{Mono}(G)$, is trivial if and only if G is abelian. ⌟

The subgroup analog of $\mathrm{Mono}(G)$ as a G-set is based on $\mathrm{Sub}(G)$ from Definition 5.3.2. Again we use the notation $\mathrm{Sub}(G)$ for both the set and the G-set, and $\mathrm{Sub}(G)(\mathrm{sh}_G) \equiv \mathrm{Sub}(G)$.

Definition 9.4.6. Let G be a group. The G-set of *subgroups of* G is defined as the family of sets $\mathrm{Sub}(\underline{\Omega}(BG_{\div}, _))$:

$$\mathrm{Sub}(G) : BG \to \mathrm{Set}, \qquad z \mapsto \mathrm{Sub}(\underline{\Omega}(BG_{\div}, _)). \qquad ⌟$$

$\mathrm{Sub}(G)(z)$ can be unpacked as $\sum_{X:BG\to\mathrm{Set}}(X(z)\times\mathrm{isTrans}(X))$. The only thing depending on z in $\mathrm{Sub}(G)(z)$ is where the chosen point is residing, namely in $X(z)$ rather than in $X(\mathrm{sh}_G)$. Hence the action of the G-set $\mathrm{Sub}(G)$ applied to (X, x) does nothing with X and applies the action of X to x.

Definition 9.4.7. Extending the equivalence of $\mathrm{Mono}(G)$ and $\mathrm{Sub}(G)$ as sets from Lemma 5.3.18, we get an equivalence $E : \mathrm{Hom}_G(\mathrm{Mono}(G), \mathrm{Sub}(G))$ of G-sets via[40]

$$E_z : \mathrm{Mono}(G)(z) \to \mathrm{Sub}(G)(z), \qquad E_z(H, f) :\equiv (Bf^{-1}, (\mathrm{sh}_H, Bf_{\mathrm{pt}}))$$

for $z : BG$, H a group, and $f : \mathrm{Hom}(H, \underline{\Omega}(BG_{\div}, z))$ a monomorphism. ⌟

[40] Note that $(\mathrm{sh}_H, Bf_{\mathrm{pt}}) : Bf^{-1}(z) \equiv (\sum_{w:BH}(z \xrightarrow{=} Bf(w)))$, the variable z occurs on the right only in the types.

Note that $E_{\mathrm{sh}_G} \equiv E$. Since the G-sets are equivalent (via E) we use $\mathrm{Mono}(G)$ and $\mathrm{Sub}(G)$ interchangeably.

xca:Sub(S1gma3)

Exercise 9.4.8. Check: Let G be a group. Prove that $\mathrm{Sub}(G)$ is a transitive G-set if and only if G is trivial. Elaborate the orbit structure of $\mathrm{Sub}(G)(\mathrm{sh}_G)$ for $G \equiv \Sigma_3$. ⌟

9.5 *Normal subgroups*

sec:normal

In the study of groups, the notion of a "normal subgroup" is of vital importance and, as for any important concept, it comes in many guises revealing different aspects. For now we just state the definition in the form that it is a subgroup "fixed under the action of G" on the G-set of subgroups.

def:normalsubgroup

Definition 9.5.1. The set of *normal subgroups* is

$$\mathrm{Nor}(G) :\equiv \prod_{y:BG} \mathrm{Sub}(G)(y)$$

considered as a subset of $\mathrm{Sub}(G)$ via the injection

$$i : \mathrm{Nor}(G) \to \mathrm{Sub}(G), \qquad i(N) :\equiv N(\mathrm{sh}_G).$$

⌟

Remark 9.5.2. The function i taking a fixed point of the action $Sub(G)z$ to its actual subgroups is indeed an injection. Given two normal subgroups $N, N' : \prod_{y:BG} Sub(G)(y)$ and a shape $y : BG$, the identity type $N(y) \xrightarrow{=} N'(y)$ is a proposition as $Sub(G)(y)$ is a set. Hence, by connectedness of BG, we construct an element $N \xrightarrow{=} N'$ as soon as we have one of $N(\mathrm{sh}_G) \xrightarrow{=} N'(\mathrm{sh}_G)$. This is exactly the statement of i being an injection.

In particular, $\mathrm{Nor}(G)$ being a subset of $Sub(G)$ allows us to make the same abuse as we did for other subtype: a subgroup H of G is said to be normal whenever the fiber $i^{-1}(H)$ has an (necessarily propositionally unique) element. ⌟

The corresponding set of fixed point in the G-set of monomorphisms

$$\prod_{y:BG} \mathrm{Mono}(G)(y)$$

Restricting the equivalence $E : \mathrm{Mono}(G) \to \mathrm{Sub}(G)$ to the fixed sets, we get an equivalence from $\prod_{y:BG} \mathrm{Mono}(G)(y)$ to $\mathrm{Nor}(G)$

will not figure as prominently, since the focus shifts naturally to an equivalent set which we have already defined, namely the kernels.

def:setofkernels

Definition 9.5.3. If G is a group, let

$$\mathrm{Epi}(G) \xrightarrow{\ker} \twoheadrightarrow \mathrm{Ker}_G \rightarrowtail \xrightarrow{i} \mathrm{Mono}(G)$$

be the surjection/injection factorization of the kernel function restricted to the epimorphisms from G. We call the subset Ker_G the *set of kernels*. ⌟

Our aim is twofold:

(1) we want to show that $\ker : \mathrm{Epi}(G) \to \mathrm{Ker}_G$ is an equivalence, so that knowing that a monomorphism is *a* kernel is equivalent to knowing an epimorphism it is *the* kernel of.

(2) we want to show that the kernels correspond via the equivalence E to the fixed points under the G action on the G-set of subgroups.

We will achieve these goals by defining a function $\mathrm{nor} : \mathrm{Epi}(G) \to \mathrm{Nor}(G)$ which we show is an equivalence and, furthermore, that the two functions $i\mathrm{nor}, Ei\ker : \mathrm{Epi}(G) \to \mathrm{Sub}(G)$ are identical. Since $i\mathrm{nor}$ is an injection, this forces the surjection ker to be injective too and we are done.

For $x', y' : BG'$, recall the notation $\mathbb{P}_{y'}(x') :\equiv (y' \xrightarrow{=} x')$.

def:ker2

Definition 9.5.4. Define

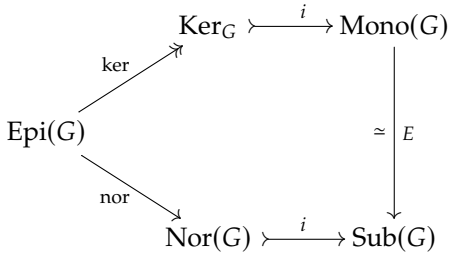
$$\mathrm{nor} : \mathrm{Epi}(G) \to \mathrm{Nor}(G)$$

by $\mathrm{nor}(G', f, !)(y) :\equiv (\mathbb{P}_{f(y)} f, \mathrm{refl}_{f(y)}, !)$ for $y : BG$. ⌟

Remark 9.5.5. The G-set $\mathbb{P}_{f(y)} f$ is not a G-torsor (except if f is an isomorphism). ⌟

lem:diagfornormal

Lemma 9.5.6. *The diagram*



commutes, where the top composite is the image factorization of the kernel and the bottom inclusion is the inclusion of fixed points.

Proof. Following $(G', f, !) : \mathrm{Epi}(G)$ around the top to $\mathrm{Sub}(G)$ yields the transitive G-set sending $y : BG$ to the set $\mathrm{sh}_{G'} \xrightarrow{=} f(y)$ together with the point $Bf_{\mathrm{pt}} : \mathrm{sh}_{G'} \xrightarrow{=} f(\mathrm{sh}_G)$ while around the bottom we get the transitive G-set sending $y : BG$ to the set $f(\mathrm{sh}_G) \xrightarrow{=} f(y)$ together with the point $\mathrm{refl}_{f(\mathrm{sh}_G)} : f(\mathrm{sh}_G) \xrightarrow{=} f(\mathrm{sh}_G)$. Hence, precomposition by Bf_{pt} gives the identity proving that the diagram commutes. □

We will prove that both ker and nor in the diagram of Lemma 9.5.6 are equivalences, leading to the desired conclusion that the equivalence $E : \mathrm{Mono}(G) \xrightarrow{\simeq} \mathrm{Sub}(G)$ takes the subset Ker_G identically to $\mathrm{Nor}(G)$. Actually, since $\ker : \mathrm{Epi}(G) \to \mathrm{Ker}_G$ is a surjection, we only need to know it is an injection, and for this it is enough to show that nor is an equivalence; we'll spell out the details.

Since it has independent interest, we take a detour via a quotient group construction of Definition 9.5.8 which gives us the desired inverse of nor.

We start with a small, but crucial observation.

lem:evaliseqhenmoma1

LEMMA 9.5.7. *Let $N : \mathrm{Nor}(G)$ be a normal subgroup with $N(y) \equiv (X_y, \mathrm{pt}_y, !)$ for $y : BG$. Then for any $y, z : BG$*

(1) *the evaluation map*

$$\mathrm{ev}_{yz} : (X_y \xrightarrow{=} X_z) \to X_z(y), \qquad \mathrm{ev}_{yz}(f) \xrightarrow{=} f_y(\mathrm{pt}_y)$$

is an equivalence and

(2) *the map $X : (y \xrightarrow{=} z) \to (X_y \xrightarrow{=} X_z)$ (given by induction via $X_{\mathrm{refl}_y} :\equiv \mathrm{refl}_{X_y}$) is surjective.*

Proof. To establish the first fact we need to do induction independently on $y : BG$ and $z : BG$ in $X_y(z)$ at the same time as we observe that it suffices (since BG is connected) to show that ev_{yy} is an equivalence.

The composite

$$\mathrm{ev}_{yy} X : (y \xrightarrow{=} y) \to X_y y$$

is determined by $\mathrm{ev}_{yy} X(\mathrm{refl}_y) \equiv \mathrm{pt}_y$. By transitivity of X_y this composite is surjective, hence ev_{yy} is surjective too.

On the other hand, in Lemma 5.2.23 we used the transitivity of X_y to deduce that ev_{yy} was injective. Consequently ev_{yy} is an equivalence. But since ev_{yy} is an equivalence and $\mathrm{ev}_{yy} X$ is surjective we conclude that X is surjective □

def:normalquotient

DEFINITION 9.5.8. Let $N : \mathrm{Nor}(G)$ be a normal subgroup with $N(y) \equiv (X_y, \mathrm{pt}_y, !)$ for $y : BG$. The *quotient group* is

$$G/N :\equiv \mathrm{Aut}_{G\text{-Set}}(X_{\mathrm{sh}_G}).$$

The *quotient homomorphism* is the homomorphism $q_N : \mathrm{Hom}(G, G/N)$ defined by $Bq_N(z) \xrightarrow{=} X_z$ (strictly pointed). By Lemma 9.5.7, q_N is an epimorphism and we have defined a map

$$q : \mathrm{Nor}(G) \to \mathrm{Epi}(G), \qquad q(N) :\equiv (G/N, q_N, !).$$

⌟

Remark 9.5.9. It is instructive to see how the quotient homomorphism $Bq_N : BG \to BG/N$ is defined in the torsor interpretation of BG. If $Y : BG \to \mathcal{U}$ is a G-type we can define the quotient as

$$Y/N : BG \to \mathcal{U}, \qquad Y/N(y) :\equiv \sum_{z:BG} Y(z) \times X_z(y).$$

We note that in the case $\mathbb{P}_{\mathrm{sh}_G}(y) :\equiv (\mathrm{sh}_G \xrightarrow{=} y)$ we get that $\mathbb{P}_{\mathrm{sh}_G}/N(y) :\equiv \sum_{z:BG}(\mathrm{sh}_G \xrightarrow{=} z) \times X_z(y)$ is equivalent to X_{sh_G}. Consequently, if Y is a G-torsor, then Y/N is in the component of X_{sh_G} and we have

$$-/N : \mathrm{Torsor}_G :\equiv (G\text{-set})_{(\mathbb{P}_{\mathrm{sh}_G})} \to (G\text{-set})_{(X_{\mathrm{sh}_G})}.$$

Our quotient homomorphism $q_N : \mathrm{Hom}(G, G/N)$ is the composite of the equivalence $\mathbb{P}^G : BG \xrightarrow{\simeq} \mathrm{Torsor}_G$ of Theorem 5.5.7 and the quotient map $-/N$. ⌟

lem:qeq

Lemma 9.5.10. *The map* $\mathrm{nor} : \mathrm{Epi}(G) \to \mathrm{Nor}(G)$ *is an equivalence with inverse* $q : \mathrm{Nor}(G) \to \mathrm{Epi}(G)$.

Proof. Assume $N : \mathrm{Nor}(G)$ with $N(y) :\equiv (X_y, \mathrm{pt}_y, !)$ for $y : BG$. Then $\mathrm{nor}\, q(N) : BG \to \mathrm{Set}$ takes $y : BG$ to $(\mathrm{nor}\, q(N))(y) \equiv (Y_y, \mathrm{refl}_{X_y}, !)$, where $Y_y(z) :\equiv (X_y \xrightarrow{=} X_z)$. Noting that the equivalence $\mathrm{ev}_{yz} : (X_y \xrightarrow{=} X_z) \xrightarrow{\simeq} X_z(y)$ of Lemma 9.5.7 has $\mathrm{ev}_{yy}(\mathrm{refl}_{X_y}) :\equiv \mathrm{pt}_y$ we see that univalence gives us the desired identity $\mathrm{nor}\, q(N) \xrightarrow{=} N$.[41]

[41] fix so that it adhers to dogmatic language and naturality in N is clear

Conversely, let $f : \mathrm{Hom}(G, G')$ be an epimorphism. Recall that the quotient group is $G/\mathrm{nor}(f) :\equiv \mathrm{Aut}_{G\text{-Set}}(\mathbb{P}_{f(\mathrm{sh}_G)} f)$ and the quotient homomorphism $q_{\mathrm{nor}\, f} : \mathrm{Hom}(G, G/\mathrm{nor}\, f)$ is given by sending $y : BG$ to $\mathbb{P}_{f(y)} f : BG \to \mathrm{Set}$ (strictly pointed – i.e., by $\mathrm{refl}_{\mathbb{P}_{f(\mathrm{sh}_G)} f}$). We define a homomorphism $Q : \mathrm{Hom}(G', G/\mathrm{nor}\, f)$ by sending $z : BG'$ to $\mathbb{P}_z f$ and using the identification $\mathbb{P}_{\mathrm{sh}_{G'}} f \xrightarrow{=} \mathbb{P}_{f(\mathrm{sh}_G)} f$ induced by $p f : \mathrm{sh}_{G'} \xrightarrow{=} f(\mathrm{sh}_G)$ and notice the equality by definition:

$$Q\, f \equiv q_{\mathrm{nor}\, f} : \mathrm{Hom}(G, G/\mathrm{nor}\, f).$$

We are done if we can show that Q is an isomorphism. The preimage of the base point $\mathbb{P}_{f(\mathrm{sh}_G)} f$ is

$$\sum_{z:BG'} \prod_{y:BG} (z \xrightarrow{=} f(y)) \xrightarrow{=} (f(\mathrm{sh}_G) \xrightarrow{=} f(y))$$

which by Lemma 5.5.17 is equivalent to

$$\sum_{z:BG'} \prod_{v:BG'} (z \xrightarrow{=} v) \xrightarrow{=} (f(\mathrm{sh}_G) \xrightarrow{=} v)$$

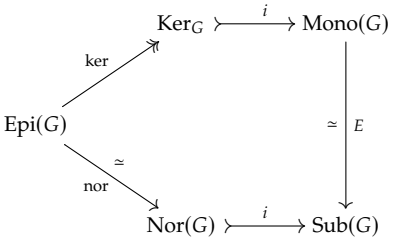
which by Lemma 5.5.6 is equivalent to the contractible type $\sum_{z:BG'} z \xrightarrow{=} f(\mathrm{sh}_G)$. □

cor:normal:is:normal

Corollary 9.5.11. *The kernel* $\ker : \mathrm{Epi}(G) \to \mathrm{Ker}_G$ *is an equivalence of sets.*

Proof. Since $\mathrm{nor} : \mathrm{Epi}(G) \to \mathrm{Nor}(G)$ and $E : \mathrm{Mono}(G) \to \mathrm{Sub}(G)$ are equivalences, the inclusion of fixed points $i : \mathrm{Nor} \to \mathrm{Sub}$ is an injection and the diagram in Lemma 9.5.6 commutes, the surjection $\ker : \mathrm{Epi}(G) \to \mathrm{Ker}_G$ is also an injection. □

the diagram in Lemma 9.5.6



Summing up, using the various interpretations of subgroups, we get the following list of equivalent sets all interpreting what a normal subgroup is.

lem:characterizations of normal

LEMMA 9.5.12. *Let G be a group, then the following sets are equivalent*

(1) *The set* $\mathrm{Epi}(G)$ *of epimorphisms from* G,

(2) *the set* Ker_G *of kernels of epimorphisms from* G,

(3) *the set* $\mathrm{Nor}(G)$ *of fixed points of the* G*-set* $\mathrm{Sub}(G)$ (*aka. normal subgroups*),

(4) *the set of fixed points of the* G*-set* $\mathrm{Mono}(G)$,

(5) *the set of fixed points of the* G*-set of abstract subgroups of* $\mathrm{abs}(G)$ *of Lemma 9.4.3.*

sec:assker

9.5.13 *The associated kernel*

With this much effort in proving that different perspectives on the concept of "normal subgroups" (in particular, kernels and fixed points) are the same, it can be worthwhile to make the composite equivalence

$$\ker\, q : \mathrm{Nor}(G) \xrightarrow{\simeq} \mathrm{Ker}_G$$

explicit – where the quotient group function $q : \mathrm{Nor}(G) \to \mathrm{Epi}(G)$ is the inverse of nor constructed in Definition 9.5.8 – and even write out a simplification.

Let $N : \mathrm{Nor}(G)$ be a normal subgroup with $N(y) \equiv (X_y, \mathrm{pt}_y, !)$ for $y : BG$ with $X_y : BG \to \mathrm{Set}$, $\mathrm{pt}_y : X_y(y)$ and $! : \mathrm{isTrans}(X_y)$. Then

$$\mathrm{Ker}\, q(N) :\equiv \mathrm{Aut}_{\sum_{x:BG}(X_x \xrightarrow{=} X_{\mathrm{sh}_G})}(\mathrm{sh}_G, \mathrm{refl}_{X_{\mathrm{sh}_G}})$$

and with the monomorphism $\mathrm{in}_{\ker q(N)} : \mathrm{Hom}(\mathrm{Ker}\, q(N), G)$ given by the first projection from $\sum_{x:BG}(X_x \xrightarrow{=} X_{\mathrm{sh}_G})$ to BG.

However, going the other way around the pentagon of Lemma 9.5.6, we see that $\mathrm{ass}(N) :\equiv E^{-1} i(N) : \mathrm{Mono}(G)$ consists of the group

$$\mathrm{Ass}(N) :\equiv \mathrm{Aut}_{\sum_{x:BG} X_{\mathrm{sh}_G}(x)}(\mathrm{sh}_G, \mathrm{pt}_{\mathrm{sh}_G})$$

and the monomorphism into G given by the first projection (monomorphism because X_{sh_G} has values in sets). Since the pentagon commutes we know that $\mathrm{ass}(N)$ is the kernel of $q(N) : \mathrm{Epi}(G)$, and the identification $\mathrm{ev} : i \ker q(N) \xrightarrow{=}_{\mathrm{Mono}(G)} \mathrm{ass}(N)$ is given via Lemma 9.5.7 and univalence by the equivalence

$$\mathrm{ev}_{x\,\mathrm{sh}_G} : (X_x \xrightarrow{=} X_{\mathrm{sh}_G}) \to X_{\mathrm{sh}_G}(x).$$

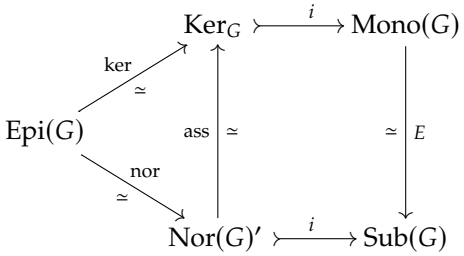
Letting the proposition that $\mathrm{ass}(N)$ is a kernel be invisible in the notation we may summarize the above as follows:

def:associatednormal

DEFINITION 9.5.14. If $N : \mathrm{Nor}(G)$ is a normal subgroup we call the kernel $\mathrm{ass}(N) : \mathrm{Ker}_G$ the *kernel associated to* N. ⌟

lem:normalsarekernels

LEMMA 9.5.16. *The diagram of equivalences*



commutes.

REMARK 9.5.15. In forming the kernel associated to N, where did we use that N was a fixed point of the G-set $\mathrm{Sub}(G)$? If $Y : BG \to \mathrm{Set}$ is a transitive G-set and $\mathrm{pt} : Y(\mathrm{sh}_G)$, then surely we could consider the group

$$W :\equiv \mathrm{Aut}_{X:BG \to \mathrm{Set}}(Y)$$

as a substitute for the quotient group (see Section 9.8). One problem is that we wouldn't know how to construct a homomorphism from G to W which we then could consider the kernel of. And even if we tried our hand inventing formulas for the outcomes, ignoring all subscripts, we'd be stuck at the very end where we used Lemma 9.5.7 to show that the evaluation map is an equivalence; if we only had transitivity we could try to use a variant of Lemma 5.2.23 to pin down injectivity, but surjectivity needs the extra induction freedom. ⌟

9.6 *Intersecting with normal subgroups*

In Section 8.4 we defined the intersection of two monomorphisms and by extension, of two subgroups. This is particularly interesting when one of them is represented by a normal subgroup.

EXERCISE 9.6.1. If $\mathcal{G}$ is an abstract group and $\mathcal{H}$ and $\mathcal{K}$ are abstract subgroups. Give a definition of the intersection $\mathcal{H} \cap \mathcal{K}$ is the abstract subgroup of $\mathcal{G}$ agreeing with our definition for monomorphisms as in Definition 8.4.7. ⌟

lem:whatSylowzneeds

LEMMA 9.6.2. *Let* $(G', f, !) : \mathrm{Epi}(G)$ *be an epimorphism, let* N *be the kernel of* f *and let* $(H, i, !) : \mathrm{Mono}(G)$. *Then* $N \cap H$ *is the kernel of* $fi : \mathrm{Hom}(H, G')$. *and the induced homomorphism in* $\mathrm{Hom}(H/(N \cap H), G')$ *is a monomorphism.*

Proof. Now, N is the kernel of the epimorphism f, giving an equivalence between $BN_{\div}$ and the preimage

$$(Bf)^{-1}(\mathrm{sh}_{G'}) :\equiv \sum_{y : BG} (\mathrm{sh}_{G'} \xrightarrow{=} Bf(y)).$$

Writing out the definition of the pullback (and using that for each $x : BH$ the type $\sum_{y : BG} y \xrightarrow{=} Bi(x)$ is contractible), we get an equivalence between $BN \times_{BG} BH$ and

$$B(fi)^{-1}(\mathrm{sh}_{G'}) :\equiv \sum_{x : BH} \mathrm{sh}_{G'} \xrightarrow{=} B(fi)x,$$

the preimage of $\mathrm{sh}_{G'}$ of the composite $B(fi) : BH \to BG'$. By definition, the intersection $B(N \cap H)$ is the pointed component of the pullback containing $(\mathrm{pt}_N, \mathrm{sh}_H)$. Under the equivalence with $B(fi)^{-1}(\mathrm{sh}_{G'})$ the intersection corresponds to the component of $(\mathrm{sh}_H, Bf(p_i)\, Bf_{\mathrm{pt}})$. Since (by definition of the composite of pointed maps) $p_{fi} :\equiv Bf(p_i)\, Bf_{\mathrm{pt}}$ we get that the intersection $N \cap H$ is identified with the kernel of the composite $fi : \mathrm{Hom}(H, G')$.

Finally, since $N \cap H$ is the kernel of the composite $fi : \mathrm{Hom}(H, G')$, under the equivalence of Lemma 9.3.22, $N \cap H$ is equivalent to the kernel of the epimorphism $\mathrm{pr}_{\mathrm{im}(fi)} : \mathrm{Hom}(H, \mathrm{Im}(fi))$. Otherwise said, the quotient group $H/(N \cap H)$ is another name for the image $\mathrm{Im}(fi)$, and $\mathrm{in}_{\mathrm{im}(fi)}$ is indeed a monomorphism into G'. □

EXERCISE 9.6.3. Write out all the above in terms of the set $\mathrm{Sub}(G)$ of subgroups of G instead of in terms of the set $\mathrm{Mono}(G)$ of monomorphism into G. ⌟

Is the below misplaced?

Recall that if $X : BG \to \mathrm{Set}$ is a G-set, then the set of fixed points is the set $\prod_{v : BG} X(v)$, which is a subset of $X(\mathrm{sh}_G)$ via the evaluation map. If a homomorphism from a group H to G is given by $F : BH_{\div} \to BG_{\div}$ and $Bf_{\mathrm{pt}} : \mathrm{sh}_G \xrightarrow{=} F(\mathrm{sh}_H)$, then precomposition ("restriction of scalars") by F gives an H-set

$$F^*X :\equiv X\, F : BH \to \mathrm{Set}.$$

In the case of inclusions of subgroups (or other situations where the homomorphism is clear from the context) it is not uncommon to talk about "the H-set X" rather than "F^*X". This can be somewhat confusing when it comes to fixed points: the fixed points of F^*X are given by $\prod_{v : BH} XF(v)$

which evaluates nicely to $XF(\mathrm{sh}_H)$, but in order to considered these as elements in $X(\mathrm{sh}_G)$ we need to apply $X(Bf_{\mathrm{pt}}^{-1}) : X(F(\mathrm{sh}_H)) \xrightarrow{=} X(\mathrm{sh}_G)$.

Consequently, we'll say that $x : X(\mathrm{sh}_G)$ is an *H-fixed point* if there is an $f : \prod_{v:BH} XF(v)$ such that $x \xrightarrow{=} X(Bf_{\mathrm{pt}}^{-1})f(\mathrm{sh}_H)$.

lem:thereisaconjugate

Lemma 9.6.4. *Let G be a group, $X : BG \to Set$ a G-set, $x : X(\mathrm{sh}_G)$, $g : UG$ and $H \xrightarrow{=} (H, F, p, !) : \mathrm{Sub}(G)$ a subgroup of G ($F : BH_{\div} \to BG_{\div}$ and $p : \mathrm{sh}_G \xrightarrow{=} F(\mathrm{sh}_H)$).*

Then $g\,x$ is a fixed point for the H-action on X if and only if x is a fixed point for the action of the conjugate subgroup $g\,H :\equiv (H, F, g^{-1}Bf_{\mathrm{pt}}, !)$ on X.

Proof. Consider an $f : \prod_{v:BH} XF(v)$. Then $g \cdot x \xrightarrow{=} X(Bf_{\mathrm{pt}}^{-1})(f(\mathrm{sh}_H))$ if and only if $x \xrightarrow{=} g^{-1} \cdot X(Bf_{\mathrm{pt}}^{-1})(f(\mathrm{sh}_H)) \equiv X((g^{-1}Bf_{\mathrm{pt}})^{-1}(f(\mathrm{sh}_H))$. □

9.7 *Automorphisms of groups*

sec:aut-group

Most of this intro including Rem. 5.7.1 has been copied to the end of Section 4.4, but a short recap is of course never wrong. This section explores the relation between the symmetries *in* a group G, and the symmetry *of* the group G. More formally, recall that Group is a groupoid, hence $\mathrm{Aut}_{\mathrm{Group}}(G)$ is defining a group, that we will simply denote $\mathrm{Aut}(G)$ in the rest of this section. Recall in particular that $\mathrm{BAut}(G)$ is the connected component of G in type of groups (pointed at G), which is equivalent to the connected component of BG in $\mathcal{U}_*$ (pointed at BG). Let us now use this equivalence to define an homomorphism $\mathrm{inn} : G \to \mathrm{Aut}(G)$ by setting

$$\mathrm{Binn} : BG \to_* \mathrm{BAut}(G), \quad y \mapsto \underline{\Omega}(BG_{\div}, y)$$

where the path pointing Binn is $p_{\mathrm{inn}} :\equiv \mathrm{refl}_G : G \xrightarrow{=} \mathrm{Binn}(\mathrm{sh}_G)$. Notice that for this map Binn to be defined properly, we need to show that, for all $y : BG$, the proposition $\|G \xrightarrow{=} \underline{\Omega}((BG_{\div}, y))\|$ holds. Easy with Example 4.4.21 We are targeting a family of propositions from the connected type BG, so it is enough to prove the proposition at $y \equiv \mathrm{sh}_G$, for which it is obvious: take $|\mathrm{refl}_G|$ as an element of $\|G \xrightarrow{=} \underline{\Omega}(BG_{\div}, \mathrm{sh}_G)\|$.

Remark 9.7.1. For pedagogical purposes, we will now make explicit the map

$$\mathrm{U\,inn} : UG \to \mathrm{U}(\mathrm{Aut}(G)).$$

More precisely, for each symmetry $g : UG$, the element $\mathrm{U\,inn}(g)$ is a symmetry of $\mathrm{Aut}(G)$, that is, through univalence, a isomorphism of groups from G to itself. We want to describe the automorphism $\mathrm{U\,inn}(g)$. By definition, (Easier with Example 4.4.21) $\mathrm{U\,inn} \equiv \mathrm{refl}_G^{-1} \cdot \mathrm{ap}_{\mathrm{Binn}}(_) \cdot \mathrm{refl}_G$. So it remains to determine $\mathrm{ap}_{\mathrm{Binn}}$. We proceed by induction on $p : \mathrm{sh}_G \xrightarrow{=} y$ to prove that $\mathrm{B}(\mathrm{ap}_{\mathrm{Binn}}(p))$ is equal to the path in $BG \xrightarrow{=} (BG_{\div}, y)$ given by the pair of paths $(\mathrm{refl}_{BG_{\div}}, p)$: indeed, this is trivial for $p \equiv \mathrm{refl}_{\mathrm{sh}_G}$. Then, through univalence, $\overset{\mathrm{B}(}{}_{\mathrm{Binn}(p))}$ is the equivalence $\mathrm{id}_{BG_{\div}}$ pointed by the path p. In particular, when $p : UG$ is a symmetry in G, then $\mathrm{B}(\mathrm{U\,inn}(g))$ is the equivalence in $BG \xrightarrow{\simeq} BG$ given by $\mathrm{id}_{BG_{\div}}$ pointed by g. Or in terms of abstract groups:

$$\mathrm{U}(\mathrm{U\,inn}(g)) : UG \xrightarrow{\cong} UG, \quad h \mapsto g^{-1}hg$$

In that form, it is easier for the reader that is used to group theory in set-theoretic foundations to see that the homomorphism inn is taking each elements of the group to the inner automorphism associated to it. ⌟

After the interlude in the remark, it should come as no surprise that we can identify the kernel of inn with the center of G. Indeed, there is a composition of identifications from the fiber at $G' : \mathrm{BAut}(G)$ of Binn as follows:

$$\begin{aligned}(\mathrm{Binn})^{-1}(G') &\equiv \left(\sum_{y:BG} G' \xrightarrow{=} \underline{\Omega}((BG_\div, y))\right) \\ &\xrightarrow{\simeq} \left(\sum_{y:BG}\sum_{p:BG'_\div \xrightarrow{=} BG_\div} y \xrightarrow{=} \mathrm{trp}_p(\mathrm{sh}_{G'})\right) \\ &\xrightarrow{\simeq} (BG'_\div \xrightarrow{=} BG_\div)\end{aligned}$$

In particular, we can consider the equivalence from the fiber at $\mathrm{sh}_{\mathrm{Aut}(G)} \equiv G$ to $BG_\div \xrightarrow{=} BG_\div$. Through this equivalence, the point $(\mathrm{sh}_G, \mathrm{Binn}_*)$ is transported to $\mathrm{refl}_{BG_\div}$. Hence, we have an identification in

$$\mathrm{Ker}(\mathrm{inn}) :\equiv \mathrm{Aut}_{(\mathrm{Binn})^{-1}(G)}(\mathrm{sh}_G, \mathrm{refl}_{BG}) \xrightarrow{\simeq} \mathrm{Aut}_{(BG_\div \xrightarrow{=} BG_\div)}(\mathrm{refl}_{BG_\div}) \xrightarrow{\simeq} \mathrm{Z}(G).$$

Under this equivalence, the associated map $\mathrm{in}_{\mathrm{ker(inn)}}$ becomes the homomorphism z_G described in Section 12.2.

Definition 9.7.2. The $\mathrm{Aut}(G)$-set of *outer automorphism*, denoted $\mathrm{out}(G)$, is the cokernel of inn. ⌟

Lemma 9.7.3. *The* $\mathrm{Aut}(G)$*-set* $\mathrm{out}(G)$ *can be identified with*

$$\mathrm{Aut}(G) \to \mathrm{Set}, \quad G' \mapsto \|BG'_\div \xrightarrow{=} BG_\div\|_0$$

lemma:coker-out-action

Proof. Simply recall the computation of the fibers of Binn above. Then, for each $G' : \mathrm{BAut}(G)$, we have an element of

$$\mathrm{out}(G)(G') \equiv \|(\mathrm{Binn})^{-1}(G')\|_0 \xrightarrow{=} \|BG'_\div \xrightarrow{=} BG_\div\|_0$$

□

Definition 9.7.4. The group $\mathrm{Inn}(G)$ of inner morphisms of G is the image $\mathrm{Im(inn)}$ of inn. ⌟

Notice that, the classifying type $\mathrm{B\,Im(inn)}$ being the total type of the cokernel of inn, the above identification of $\mathrm{out}(G)$ provides us with an equivalence in

$$\mathrm{B\,Inn}(G) \xrightarrow{\simeq} \left(\sum_{G' : \mathrm{Group}} \|BG'_\div \xrightarrow{=} BG_\div\|_0\right)$$

Lemma 9.7.5. *The group* $\mathrm{Inn}(G)$ *is normal when seen as a subgroup of* $\mathrm{Aut}(G)$.

Proof. The precise meaning of the statement is that there exists a dependent function $N : \prod_{G' : \mathrm{Aut}(G)} \mathrm{Sub}_{\mathrm{Aut}(G)}(G')$ and a path in $N(\mathrm{sh}_{\mathrm{Aut}(G)}) \xrightarrow{=} E(\mathrm{im(inn)})$. Expanding the definition of $\mathrm{Sub}_{\mathrm{Aut}(G)}$, our task in defining N is to find for every G' a transitive $\mathrm{Aut}(G)$-set X together with a point of $X(G')$. We suggest to define $N(G')$ to be the transitive $\mathrm{Aut}(G)$-set

$$X : \mathrm{Aut}(G) \to \mathrm{Set}, \quad H \mapsto \|BG'_\div \xrightarrow{=} BH_\div\|_0$$

together with the point $|\mathrm{refl}_{BG'}|_0 : X(G')$.

Let us prove that $N(\mathrm{sh}_{\mathrm{Aut}(G)})$ can be identified with the subgroup $E(\mathrm{im}(\mathrm{inn}))$. First notice that $\mathrm{sh}_{\mathrm{Aut}(G)} \equiv G$ and that $E(\mathrm{im}(\mathrm{inn})) \equiv (\mathrm{out}(G), |(G, \mathrm{refl}_{BG})|_0)$. For simplicity, write X_G for the first component $N(G)$ and $x_G : X_G(G)$ for its second component. Lemma 9.7.3 provides us with a path $p : \mathrm{out}(G) \xrightarrow{=} X_G$. Checking that $\mathrm{trp}_p(|(G, \mathrm{refl}_{BG})|_0)$ can be identified with $|\mathrm{refl}_{BG}|_0$ is just a matter of looking at the equivalence exhibited in Lemma 9.7.3.

To be thorough, we actually need to prove that the first component of each $N(G')$ (denoted X above) is transitive: being transitive is a proposition and by connectedness of $\mathrm{Aut}(G)$, it suffices to prove it when $G' \equiv G$, for which the first component of $N(G')$ has been identified with $\mathrm{out}(G)$; however, $\mathrm{out}(G)$, as a cokernel, is known to be transitive. □

We make the abuse of denoting $\mathrm{Inn}(G)$ for the normal subgroup of $\mathrm{Aut}(G)$ defined by $\mathrm{Inn}(G)$ as specified above.

Definition 9.7.6. The group of *outer automorphisms* of G, denoted $\mathrm{Out}(G)$, is the group

$$\mathrm{Out}(G) :\equiv \mathrm{Aut}(G)/\mathrm{Inn}(G) \equiv \mathrm{Aut}_{\mathrm{Aut}(G)\text{-}\mathrm{Set}}(\mathrm{out}(G))$$

⌟

Construction 9.7.7. *There is an identification of groups*

$$\Phi : \mathrm{Aut}_{\|\mathcal{U}\|_1}(|BG_\div|_1) \xrightarrow{=} \mathrm{Out}(G)$$

cons:simpler-version-Out

Before going through the construction of Φ, let us describe its domain in more details. The goal of this construction is to have a alternative version of $\mathrm{Out}(G)$ with a more tractable classifying type. Because $\mathrm{out}(G)$ is a transitive $\mathrm{Aut}(G)$-set, and because the associated subgroup is normal, then its type of symmetries should be equivalent to $\mathrm{out}(G)(G)$, which we know can be identified with $\|BG_\div \xrightarrow{=} BG_\div\|_0$. The idea is then to find a pointed groupoid for which the loop space is readily $\|BG_\div \xrightarrow{=} BG_\div\|_0$. However, $\|a \xrightarrow{=}_A b\|_0$ is equivalent to $|a|_1 \xrightarrow{=}_{\|A\|_1} |b|_1$ for any element a and b of type A. Hence it becomes natural to try to establish an equivalence between $\mathrm{Out}(G)$ and the group of symmetries of $|BG_\div|_1$ in the groupoid $\|\mathcal{U}\|_1$.

Implementation of Construction 9.7.7. Notice that the function $|_|_1 : \mathcal{U} \to \|\mathcal{U}\|_1$ induces an isomorphism on connected components: indeed, $|X|_1 = |Y|_1$ if and only if $\|\|X \xrightarrow{=} Y\|_0\|$ if and only if $X = Y$. In other words, $\mathrm{BAut}_{\|\mathcal{U}\|_1}(|BG_\div|_1)$ identifies with the 1-truncation of $\mathcal{U}_{(BG_\div)}$.

As $\mathrm{BOut}(G)$ is a groupoid, every map $\mathrm{BAut}_{\|\mathcal{U}\|_1}(|BG_\div|_1) \to_* \mathrm{BOut}(G)$ is indud by a map $\mathcal{U}_{(BG_\div)} \to_* \mathrm{BOut}(G)$. Thus we define Φ by setting the pointed map $\mathrm{B}\Phi$ to be the map induced by:

$$\varphi : \left(\sum_{X:\mathcal{U}} BG_\div = X\right) \to_* \mathrm{BOut}(G)$$
$$(X, \omega) \mapsto \{\mathrm{BAut}(G) \to \mathrm{Set},\ G' \mapsto \|BG'_\div \xrightarrow{=} X\|_0\}$$

This map is well defined: given (X, ω) is the domain, we are trying to prove the proposition $\mathrm{out}(G) \xrightarrow{=} \varphi(X, \omega)$, so we can lift the propositional truncation of ω and assume that we have $w : BG_\div \xrightarrow{=} X$. Then, we craft

Here, $|w|_0$ is not the element represented by w in $\|BG_\div \xrightarrow{=} X\|_0$, but in fact the equivalence $\|BG_\div\|_0 \xrightarrow{=} \|X\|_0$ induced by w.

an identification of type $\mathrm{out}(G) \xrightarrow{=} \phi(X, \omega)$ by noticing that we have for every $G' : \mathrm{BAut}(G)$ an identification

$$|w|_0 \circ _ : \|BG'_\div \xrightarrow{=} BG_\div\|_0 \xrightarrow{=} \|BG'_\div \xrightarrow{=} X\|_0$$

We no proceed to prove that $\mathrm{B}\Phi$ is an equivalence, to conclude that Φ is an isomorphism of groups. As both the domain and codomain of $\mathrm{B}\Phi$ are connected, to prove that it is an equivalence, it is enough to show that $\mathrm{ap}_{\mathrm{B}\Phi} : (a \xrightarrow{=} a) \to (\mathrm{B}\Phi a \xrightarrow{=} \mathrm{B}\Phi a)$ for a chosen a in the domain. We consider of course $a \equiv (|BG_\div|_1, \mathrm{refl}_{|BG_\div|_1})$. Then,

$$\mathrm{B}\Phi(a) \equiv \phi(BG_\div, \mathrm{refl}_{BG_\div}) \equiv (G' \mapsto \|BG'_\div \xrightarrow{=} BG_\div\|_0)$$

By path induction, one can show that for each $p : |BG_\div|_1 \xrightarrow{=} |BG_\div|_1$, we get paths of type

$$\mathrm{ap}_{\mathrm{B}\Phi}(p) \xrightarrow{=} \{G' \mapsto \hat{p} \circ _\}$$

where $\hat{_}$ is the equivalence $(|x|_1 \xrightarrow{=} |y|_1) \xrightarrow{\simeq} \|x \xrightarrow{=} y\|_0$.

Because the subgroup associated with $\mathrm{out}(G)$ is normal, Lemma 9.5.7 povides us with an equivalence $\mathrm{ev} : (\mathrm{out}(G) \xrightarrow{=} \mathrm{out}(G)) \to \mathrm{out}(G)(G)$. Write ψ for the path $\mathrm{out}(G) \xrightarrow{=} G' \mapsto \|BG'_\div \xrightarrow{=} BG_\div\|_0$ of Lemma 9.7.3. Then, for every $p : |BG_\div|_1 \xrightarrow{=} |BG_\div|_1$, one get an identification

$$\psi_G \left(\mathrm{ev}\left(\psi^{-1} \cdot \mathrm{ap}_{\mathrm{B}\Phi}(p) \cdot \psi\right)\right) \xrightarrow{=} \hat{p}$$

Hence, composition of $\mathrm{ap}_{\mathrm{B}\Phi}$ with equivalences is an equivalence, proving that $\mathrm{ap}_{B\Phi}$ itself is an equivalence. □

9.8 *The Weyl group*

sec:Weyl

In Definition 9.5.8 defined the quotient group of a normal subgroup. As commented in Definition 9.5.14, the definition itself never used that the subgroup was normal (but the quotient homomorphism did) and is important in this more general context.

Recall the equivalence E between the set $\mathrm{Mono}(G)$ of monomorphisms and the set $\mathrm{Sub}(G)$ of of subgroups of G (pointed transitive G-sets): The subgroup $(X, \mathrm{pt}_X, !) : \mathrm{Sub}(G)$ where $X : BG \to \mathrm{Set}$ is a transitive G-set and $\mathrm{pt}_X : X(\mathrm{sh}_G)$ corresponds to $(H, i_H, !) : \mathrm{Mono}(G)$ defined by

$$H :\equiv \mathrm{Aut}_{\sum_{y:BG} X(y)}(\mathrm{sh}_G, \mathrm{pt}_X)$$

together with the first projection from $\sum_{y:BG} X(y)$ to BG. Conversely, if $(H, i_H, !) : \mathrm{Mono}(G)$, then the corresponding transitive G-set is $G/H :\equiv \mathrm{coker}\, i_H$ pointed at $|\mathrm{sh}_H, p_{i_H}| : \mathrm{coker}\, i_H(\mathrm{sh}_G) :\equiv \|\sum_{x:BH} \mathrm{sh}_G \xrightarrow{=} Bi_H(x)\|_0$.

For the remainder of the section we'll consider a fixed group G, monomorphism $i_H : \mathrm{Hom}(H, G)$ and $(X, \mathrm{pt}_X, !)$ will be the associated pointed transitive G-set.

Definition 9.8.1. The *Weyl group*

def:Weyl

$$W_G H :\equiv \mathrm{Aut}_{G\text{-set}}(X)$$

is defined by the component $BW_G H$ of the groupoid of G-sets pointed at X.

The *normalizer subgroup*

def:normalizer

$$N_G H :\equiv \mathrm{Aut}_{\sum_{y:BG} \mathrm{Sub}(G)(y)}(\mathrm{sh}_G, X, \mathrm{pt}_X)$$

is defined by the component BN_GH of the groupoid $\sum_{y:BG} \mathrm{Sub}(G)(y)$ pointed at $(\mathrm{sh}_G, X, \mathrm{pt}_X)$. ⌟

Unpacking, we find that

$$BN_GH_{\div} \equiv \sum_{y:BG} \sum_{Y:BG\to\mathrm{Set}} \sum_{\mathrm{pt}_Y^y : Y(y)} \|(\mathrm{sh}_G, X, \mathrm{pt}_X) \xrightarrow{=} (y, Y, \mathrm{pt}_Y^y)\|.$$

While the projection $((\mathrm{sh}_G, X, \mathrm{pt}_X) \xrightarrow{=} (y, Y, \mathrm{pt}_Y^y)) \to (X \xrightarrow{=} Y)$ may not be an equivalence, the transitivity of X tells us that for any $\beta : X \xrightarrow{=} Y$ there is a $g : \mathrm{sh}_G \xrightarrow{=} y$ such that $X(g)\, p_Y^y \xrightarrow{=} \beta_y^{-1}\mathrm{pt}_X$, and so the propositional truncation $\|(\mathrm{sh}_G, X, \mathrm{pt}_X) \xrightarrow{=} (y, Y, \mathrm{pt}_Y^y)\| \to \|X \xrightarrow{=} Y\|$ is an equivalence. Consequently, the projection

$$BN_GH_{\div} \to \sum_{y:BG} \sum_{Y:BG\to\mathrm{Set}} Y(y) \times \|X \xrightarrow{=} Y\|$$

is an equivalence. With an innocent rewriting, we see that we have provided an equivalence

$$e : BN_GH_{\div} \xrightarrow{\simeq} \sum_{(y\times Y):BG\times BW_GH} Y(y) \qquad e(y, Y, \mathrm{pt}_Y^y, !) :\equiv (y, Y, \mathrm{pt}_Y^y, !).$$

This formulation has the benefit of simplifying the analysis of the monomorphism

$$i_{N_GH} : \mathrm{Hom}(N_GH, G)$$

given by $Bi_{N_GH}(y, Y, \mathrm{pt}_Y^y, !) :\equiv y$, the "projection"

$$p_G^H : \mathrm{Hom}(N_GH, W_GH)$$

$Bp_G^H(y, Y, \mathrm{pt}_Y^y, !) :\equiv (Y, !)$ and the monomorphism

$$j_H : \mathrm{Hom}(H, N_GH)$$

given by $Bj_H(y, v) :\equiv (y, X, v, !))$.

Lemma 9.8.2. *The monomorphism $i_G^H : \mathrm{Hom}(N_GH, G)$ displays the normalizer as a subgroup of G and the projection $p_G^H : \mathrm{Hom}(N_GH, W_GH)$ is an epimorphism.*

The homomorphism $j_H : \mathrm{Hom}(H, N_GH)$ defines H as a normal subgroup of the normalizer,

$$\ker p_G^H \xrightarrow{=}_{\mathrm{Mono}(N_GH)?for?} (H, i_H, !)$$

and $i_H \xrightarrow{=}_{\mathrm{Hom}(H,G)} i_G^H\, j_H$.

Proof. Immediate from (our rewriting of) the definitions. □

The Weyl group W_GH has an important interpretation. It is defined as symmetries of the transitive G-set X, and so $\mathrm{pt}_{W_GH} \xrightarrow{=} \mathrm{pt}_{W_GH}$ is nothing but $(X \xrightarrow{=}_{G\text{-set}} X) \xrightarrow{=} \prod_{y:BG}(X(y) \xrightarrow{=} X(y))$. On the other hand, $BH_{\div}$ is equivalent to $\sum_{y:BG} X(y)$ and

$$\prod_{y:BG} (X(y) \xrightarrow{=} X(y)) \simeq \prod_{\sum_{y:BG} X(y)} X(y),$$

so $\mathrm{pt}_{W_GH} \xrightarrow{=} \mathrm{pt}_{W_GH}$ is equivalent to the set $\prod_{x:BH} X\, Bi_H x$ of fixed points of $X \xrightarrow{=} G/H$ (regarded as an H-set through i_H).

Summing up

Lemma 9.8.3. *The map $e : (X \xrightarrow{=} X) \to \prod_{x:BH} X\, Bi_H x$ with $e(f)(y, v) \xrightarrow{=} f(y)$ defines an equivalence*

$$e : (\mathrm{pt}_{W_GH} \xrightarrow{=} \mathrm{pt}_{W_GH}) \xrightarrow{\simeq} (G/H)^H.$$

1em:WGH:stf:xo:fG/H

9.9 *The isomorphism theorems*

sec:noether-theorems

Cf. Section 2.27

Group homomorphisms provide examples of forgetting stuff and structure. For example, the map from cyclically ordered sets with cardinality n to the type of sets with cardinality n forgets structure, and represents an injective group homomorphism from the cyclic group of order n to the symmetric group Σ_n.

And the map from pairs of n-element sets to n-element sets that projects onto the first factor clearly forgets stuff, namely, the other component. It represents a surjective group homomorphism.

More formally, fix two groups G and H, and consider a homomorphism φ from G to H, considered as a pointed map $\mathrm{B}\varphi : BG \to_{\mathrm{pt}} BH$. Then $\mathrm{B}\varphi$ factors as

$$\begin{aligned} BG &= \sum_{w:BH} \sum_{z:BG} (\mathrm{B}\varphi(z) = w) \\ &\to_{\mathrm{pt}} \sum_{w:BH} \left\| \sum_{z:BG} (\mathrm{B}\varphi(z) = w) \right\|_0 \\ &\to_{\mathrm{pt}} \sum_{w:BH} \left\| \sum_{z:BG} (\mathrm{B}\varphi(z) = w) \right\|_{-1} = BH. \end{aligned}$$

The pointed, connected type in the middle represents a group that is called the *image* of φ, $\mathrm{Im}(\varphi)$.

(FIXME: Quotient groups as automorphism groups, normal subgroups/normalizer, subgroup lattice)

lem:aut-orbit

Lemma 9.9.1. *The automorphism group of the G-set G/H is isomorphic to $\mathrm{N}_G(H)/H$.*

thm:fund-thm-homs

Theorem 9.9.2 (Fundamental Theorem of Homomorphisms). *For any homomorphism $f : \mathrm{Hom}(G, G')$ the map TODO defines an isomorphism $G/\ker f \simeq \mathrm{im}\, f$.*[42]

[42] TODO: Fix and move to Ch. 5

9.10 *More about automorphisms*

sec:automorphisms

For every group G (which for the purposes of the discussion in this section we allow to be a higher group) we have the automorphism group $\mathrm{Aut}(G)$. This is of course the group of self-identifications $G = G$ in the type of groups, Group. If we represent G by the pointed connected classifying type BG, then $\mathrm{Aut}(G)$ is the type of pointed self-equivalences of BG.

We have a natural forgetful map from groups to the type of connected groupoids. Define the type Bunch to be the type of all connected groupoid. If X : Bunch, then all the elements of X are merely isomorphic, that is, they all look alike, so it makes sense to say that X consists of a *bunch* of alike objects.

For every group G we have a corresponding bunch, $BG_{\div}$, i.e., the collection of G-torsors, and if we remember the basepoint $\mathrm{sh}_G : BG_{\div}$, then we recover the group G. Thus, the type of groups equivalent to the type $\sum_{X:\mathrm{Bunch}} X$ of pairs of a bunch together with a chosen element. (This is essentially our definition of the type Group.)

Sometimes we want to emphasize that we $BG_{\div}$ is a bunch, so we define $\mathrm{bunch}(G) :\equiv BG_{\div} : \mathrm{Bunch}$.

def:center

DEFINITION 9.10.1 (The center as an abelian group). Let

$$Z(G) :\equiv \prod_{z:BG}(z = z)$$

denote the type of fixed points of the adjoint action of G on itself. This type is equivalent to the automorphism group of the identity on bunch(G), and hence the loop type of

$$\mathrm{B}Z(G) :\equiv \sum_{f:BG\to BG} \|f \sim \mathrm{id}\|_{-1}.$$

This type is itself the loop type of the pointed, connected type

$$\mathrm{B}^2Z(G) :\equiv \sum_{X:\mathrm{Bunch}} \|\mathrm{bunch}(G) = X\|_0,$$

and we use this to give $Z(G)$ the structure of an *abelian* group, called the *center* of G. ⌟

There is a canonical homomorphism from $Z(G)$ to G given by the pointed map from $\mathrm{B}Z(G)$ to BG that evaluates at the point sh_G. The fiber of the evaluation map $e : \mathrm{B}Z(G) \to_{\mathrm{pt}} BG$ is

$$\begin{aligned}\mathrm{fiber}_e(\mathrm{sh}_G) &\equiv \sum_{f:BG\to BG} \|f \sim \mathrm{id}\|_{-1} \times (f\,\mathrm{sh}_G = \mathrm{sh}_G)\\ &\simeq \sum_{f:BG\to_{\mathrm{pt}}BG} \|f \sim \mathrm{id}\|_{-1},\end{aligned}$$

and this type is the loop type of the pointed, connected type

$$\mathrm{B}\,\mathrm{Inn}(G) :\equiv \sum_{H:\mathrm{Group}} \|\mathrm{bunch}(G) = \mathrm{bunch}(H)\|_0,$$

thus giving the homomorphism $Z(G)$ to G a normal structure with quotient group Inn(G), called the *inner automorphism group*.

Note that there is a canonical homomorphism from Inn(G) to Aut(G) given by the pointed map $i : \mathrm{B}\,\mathrm{Inn}(G) \to \mathrm{B}\,\mathrm{Aut}(G)$ that forgets the component. On loops, i gives the inclusion into Aut(G) of the subtype of automorphisms of G that become merely equal to the identity automorphism of bunch(G). The fiber of i is

$$\begin{aligned}\mathrm{fiber}_i(\mathrm{sh}_G) &\equiv \sum_{H:\mathrm{Group}} \|\mathrm{bunch}(G) = \mathrm{bunch}(H)\|_0 \times (H = G)\\ &\simeq \|\mathrm{bunch}(G) = \mathrm{bunch}(G)\|_0.\end{aligned}$$

This is evidently the type of loops in the pointed, connected groupoid

$$\mathrm{B}\,\mathrm{Out}(G) :\equiv \left\| \sum_{X:\mathrm{Bunch}} \|\mathrm{bunch}(G) = X\|_{-1} \right\|_1,$$

thus giving the homomorphism Inn(G) to Aut(G) a normal structure with quotient group Out(G), called the *outer automorphism group*. Note that Out(G) is always a 1-group, and that it is the decategorification of Aut(bunch(G)).

thm:hom-mod-conj

THEOREM 9.10.2. *Let two groups G and H be given. There is a canonical action of* Inn(H) *on the set of homomorphisms from G to H, $\|BG \to_{\mathrm{pt}} BH\|_0$. This gives rise to an equivalence*

$$\|BG_{\div} \to BH_{\div}\|_0 \simeq \left\| \left(\|BG \to_{\mathrm{pt}} BH\|_0\right)_{h\,\mathrm{Inn}(H)} \right\|_0$$

between the set of maps from bunch(G) *to* bunch(H) *and the set of components of the orbit type of this action.*

Proof. We give the action by defining a type family $X : \mathrm{B}\,\mathrm{Inn}(H) \to \mathcal{U}$ as follows

$$X\,\langle K, \phi\rangle :\equiv \|\mathrm{Hom}(G, K)\|_0 \equiv \|BG \to_{\mathrm{pt}} BK\|_0,$$

for $\langle K, \phi\rangle : \mathrm{B}\,\mathrm{Inn}(H) \equiv \sum_{K:\mathrm{Group}} \|\mathrm{bunch}(H) = \mathrm{bunch}(K)\|_0$. Now we can calculate

$$\begin{aligned}
\|X_{\mathrm{Inn}(H)}\|_0 &\equiv \left\| \sum_{K:\mathrm{Group}} \|\mathrm{bunch}(H) = \mathrm{bunch}(K)\|_0 \times \|\mathrm{Hom}(G, K)\| \right\|_0 \\
&\simeq \left\| \sum_{K:\mathrm{Group}} (\mathrm{bunch}(H) = \mathrm{bunch}(K)) \times \mathrm{Hom}(G, K) \right\|_0 \\
&\simeq \left\| \sum_{K:\mathrm{Bunch}} \sum_{k:K} (\mathrm{bunch}(H) = K) \times \sum_{f\,:\,\mathrm{bunch}(G)\to K)} f\ \mathrm{pt} = k \right\|_0 \\
&\simeq \left\| \sum_{K:\mathrm{Bunch}} (\mathrm{bunch}(H) = K) \times (\mathrm{bunch}(G) \to K) \right\|_0 \\
&\simeq \|\mathrm{bunch}(G) \to \mathrm{bunch}(H)\|_0 \equiv \|BG_{\div} \to BH_{\div}\|_0.
\end{aligned}$$

□

10

Finite groups

sec:fingp

Objects having only a finite number of symmetries can be analyzed through counting arguments. The strength of this approach is stunning.

The orbit-stabilizer theorem Construction 5.4.28 is at the basis of this analysis: if G is a group and $X : BG \to \mathrm{Set}$ is a G-set, then

$$X(\mathrm{sh}_G) \simeq \amalg_{x : X/G} \mathcal{O}_x$$

and each orbit set $\mathcal{O}_x$ is equivalent to the cokernel of the inclusion $G_x \subseteq G$ of the stabilizer subgroup of x. Consequently, if $X(\mathrm{sh}_G)$ is a finite set, then its cardinality is the sum of the cardinality of these cokernels. If also the set UG is finite much more can be said and simple arithmetical considerations often allow us to deduce deep statements like the size of a certain subset of $X(\mathrm{sh}_G)$ and in particular whether or not there are any fixed points.

Example 10.0.1. A typical application could go like this. If $X(\mathrm{sh}_G)$ is a finite set with 13 elements and for some reason we know that all the orbits have cardinalities dividing 8 – which we'll see happens if UG has 8 elements – then we must have that some orbits are singletons (for a sum of positive integers dividing 8 to add up to 13, some of them must be 1). That is, X has fixed points. ⌟

The classical theory of finite groups is all about symmetries coupled with simple counting arguments. Lagrange's Exercise 5.3.29 gives the first example: if H is a subgroup of G, then the cardinality "$|G|$" of UG is divisible by $|H|$, putting severe restrictions on the possible subgroups. For instance, if $|G|$ is a prime number, then G has no nontrivial proper subgroups! (actually, G is necessarily a cyclic group). To prove this result we interpret G as an H-set.

Further examples come from considering the G-set Sub_G of subgroups of G from Section 5.3. Knowledge about the G-set of subgroups is of vital importance for many applications and Sylow's theorems in Section 10.4 give the first restriction on what subgroups are possible and how they can interact. The first step is Cauchy's Theorem 10.3.2 which says that if $|G|$ is divisible by a prime p, then G contains a cyclic subgroup of order p. Sylow's theorems goes further, analyzing subgroups that have cardinality powers of p, culminating in very detailed and useful information about the structure of the subgroups with cardinality the maximal possible power of p.

Example 10.0.2. For instance, for the permutation group Σ_3, Sylow's theorems will deduce from the simple fact $|\Sigma_3| = 6$ that Σ_3 contains a unique subgroup $|H|$ with $|H| = 3$. Since it is unique, H must be a normal subgroup.

On the other hand, for Σ_4 the information $|\Sigma_4| = 24$ only suffices to tell us that there are either 1 or 4 subgroups K with $|K| = 3$, but that all of them are conjugate. However, the inclusion of Σ_3 in Σ_4 shows that the $H \subseteq \Sigma_3$ above (which is given by the cyclic permutations of three letters) can be viewed as a subgroup of Σ_4, and elementary inspection gives that this subgroup is not normal. Hence there must be more than one subgroup K with $|K| = 3$, pinning the number of such subgroups down to 4.

Indeed, Σ_n has $n(n-1)(n-2)/6$ subgroups of order 3 (for $n > 2$), but when $n > 5$ something like a phase transformation happens: the subgroups of order 3 are no longer all conjugate. This can either be seen as a manifestation of the fact that $3^2 = 9$ divides $n! = |\Sigma_n|$ for $n > 5$ or more concretely by observing that there is room for "disjoint" cyclic permutations. For instance the subgroup of cyclic permutations of $\{1, 2, 3\}$ will not be conjugate to the subgroup of cyclic permutations of $\{4, 5, 6\}$. Together these two cyclic subgroups give a subgroup K with $|K| = 9$ and there are 10 of these (one for each subset of $\{1, 2, 3, 4, 5, 6\}$ of cardinality 3). ⌟

rem:hoofsubgps

Remark 10.0.3. One should observe that the number of subgroups is often very large and the structure is often quite involved, even for groups with a fairly manageable size and transparent structure (for instance, the number of subgroups of the group you get by taking the product of the cyclic group C_2 with itself n times grows approximately as $7 \cdot 2^{n^2/4}$ – e.g., $C_2^{\times 18}$ has 177417531717496268409526 85 subgroups, see https://oeis.org/A006116). ⌟

10.1 *Brief overview of the chapter*

sec:fingp-overview

We start by giving the above-mentioned counting version Lemma 10.2.3 of Lagrange's theorem Exercise 5.3.29. We then moves on to prove Cauchy's Theorem 10.3.2 stating that any finite group whose cardinality is divisible by a prime p has a cyclic subgroup of cardinality p. Cauchchy's theorem has many applications, and we use it already in Section 10.4 in the proof of Sylow's Theorems which give detailed information about the subgroups of a given finite group G. Sylow's theorems is basically a study of the G-set of subgroups of G from a counting perspective. In particular, if p^n divides the cardinality of G, but p^{n+1} does not, then Sylow's Third Theorem 10.4.5 gives valuable information about the cardinality of the G-set of subgroups of G of cardinality p^n.

10.2 *Lagrange's theorem, counting version*

sec:lagrangecounting

We start our investigation by giving the version of Lagrange's theorem which has to do with counting, but first we pin down some language.

def:finitegrd

Definition 10.2.1. A *finite group* is a group such that the set UG is finite. If G is a finite group, then the *cardinality* $|G|$ is the cardinality of the finite set UG (i.e., $UG : \mathrm{FinSet}_{(|G|)}$). ⌟

Example 10.2.2. The trivial group has cardinality 1, the cyclic group C_n of order n has cardinality n and the permutation group Σ_n has cardinality $n!$. ⌟

In the literature, "order" and "cardinality" are used interchangeably for groups.

For finite groups, Lagrange's Exercise 5.3.29 takes on the form of a counting argument

thm:Lagrangeascounting

LEMMA 10.2.3 (Lagrange's theorem: counting version). *Let $i : \mathrm{Hom}(H, G)$ be a subgroup of a finite group G. Then*

$$|G| = |G/H| \cdot |H|.$$

If $|H| = |G|$, then $H = G$ (as subgroups of G).

Proof. Consider the H action of H on G, i.e., the H-set $i^*G : BH \to \mathrm{Set}$ with $i^*G(x) :\equiv (\mathrm{sh}_G = Bi(x))$, so that G/H is just another name for the orbits $i^*G/H :\equiv \sum_{x:BH} i^*G(x)$. Note that composing with the structure identity $p_i : \mathrm{sh}_G = Bi(\mathrm{sh}_H)$ gives an equivalence $i^*G(\mathrm{sh}_H) \simeq UG$, so that $|i^*G(\mathrm{sh}_H)| = |G|$.

Lagrange's Exercise 5.3.29 says that i^*G is a free H-set [1] and so all orbits $\mathcal{O}_x$ are equivalent to the H-set $\tilde{H}(x) = (\mathrm{sh}_H = x)$. Consequently, the equivalence

$$i^*G(\mathrm{sh}_H) \simeq \sum_{x : i^*G/H} \mathcal{O}_x$$

of Section 5.4.24 gives that G/H and H are finite and that $|G| = |G/H| \cdot |H|$.[2]

Finally, since we are considering a subgroup, the preimage $Bi^{-1}(\mathrm{pt})$ is equivalent to the set G/H. If $|H| = |G|$, then $|G/H| = 1$ and so the set G/H is contractible. □

[1] Exercise 5.3.29 doesn't say this at present: fix it

[2] somewhere: prove that if A is a finite set and $B(a)$ is a family of finite sets indexed over $a : A$, then $\sum_{a:A} B(a)$ is a finite set of cardinality $\sum_{i:\mathbb{n}} |B(f(i))|$ for any $f : \mathbb{n} = A$, hence if $m = |B(a)|$ for all a then $|\sum_A B(a)| = n \cdot m$.

cor:cyclicgroupsaresimple

COROLLARY 10.2.4. *If p is a prime, then the cyclic group C_p has no non-trivial proper subgroups.*

Proof. By Lagrange's counting Lemma 10.2.3 a subgroup of C_p has cardinality dividing $p = |C_p|$, i.e., either 1 or p. □

cor:whatSylow2needs

COROLLARY 10.2.5. *Let $f : \mathrm{Hom}(G, G')$ be a surjective homomorphism with kernel N and let H be a subgroup of G. If H and G' are finite with coprime cardinalities, then H is a subgroup of N.*

Proof. Let $i : \mathrm{Hom}(H, G)$ be the inclusion. By Lemma 9.6.2 the intersection $N \cap H$ is the kernel of the composite $fi : \mathrm{Hom}(H, G')$. Let H' be the image of fi. Now, Lagrange's counting Lemma 10.2.3 gives that $|H| = |H'| \cdot |N \cap H|$ and $|G'| = |G'/H'| \cdot |H'|$. This means that $|H'|$ divides both $|H|$ and $|G'|$, but since these numbers are coprime we must have that $|H'| = 1$, and finally that $|H| = |N \cap H|$. This implies that $N \cap H = H$, or in other words, that H is a subgroup of N ((elaborate)). □

COROLLARY 10.2.6. *If G and G' are finite groups, then the cardinality $|G \times G'|$ of the product is the product $|G| \cdot |G'|$ of the cardinalities.*

REMARK 10.2.7. Hence the cardinality of the n-fold product of Remark 10.0.3 of C_2 with itself is (2^n and so grows quickly, but is still) dwarfed by the number of subgroups as n grows. ⌟

10.3 *Cauchy's theorem*

lem:fixedptsize

LEMMA 10.3.1. *Let p be a prime and G a group of cardinality p^n for some positive $n : \mathbb{N}$. If $X : BG \to \mathrm{Set}$ is a non-empty finite G-set such that the cardinality of $X(\mathrm{sh}_G)$ is divisible by p, then the cardinality of the set of fixed points $X^G :\equiv \prod_{z:BG} X(z)$ is divisible by p.*

Proof. Recall that the evaluation at sh_G gives an injection of sets $X^G \to X(\mathrm{sh}_G)$ through which we identify X^G with the subset "$X(\mathrm{sh}_G)^G$" of all trivial orbits of $X(\mathrm{sh}_G)$. The orbits of $X(\mathrm{sh}_G)$[3] all have cardinalities that divide the cardinality p^n of G. This means that all the the cardinalities of the non-trivial orbits (as well as of $X(\mathrm{sh}_G)$) are positive integers divisible by p.

[3] or of X? Reference for identification of orbits with quotients by stabilizers

Burnside's Lemma Section 5.7 states that $X(\mathrm{sh}_G)$ is the sum of its orbits. Hence the cardinality of the set of all trivial orbits, i.e., of X^G, is the difference of two numbers both divisible by p. □

thm:cauchys

THEOREM 10.3.2. *Let p be a prime and let G be a finite group of cardinality divisible by p. Then G has a subgroup which is cyclic of cardinality p.*

Proof. Recall the cyclic group $\mathrm{C}_p :\equiv \mathrm{Aut}_{\mathrm{Cyc}}\, \mathrm{Z}/p$ of cardinality p where $\mathrm{Z}/p :\equiv (\mathbb{p}, s)$ is the standard p-cycle. In other words, there is an identification of pointed groupoids

$$\mathrm{B}\,\mathrm{C}_p \xrightarrow{\simeq} (\sum_{S:\mathrm{Set}} \sum_{j : S \xrightarrow{\simeq} S} \|(S, j) = \mathrm{Z}/p\|, (\mathrm{Z}/p, !)).$$

Informally, $\mathrm{B}\,\mathrm{C}_p$ consists of pairs (S, j), where S is a set of cardinality p and $j : S \xrightarrow{\simeq} S$ is a cyclic permutation in the sense that for $0 < k < p$ we have that j^k is not refl while $j^p = \mathrm{refl}$. Given a set A, a function $a : \mathbb{p} \to A$ is an ordered p-tuple of elements of A: it suffices to write a_i for $a(i)$ to retrieve the usual notations for tuples. Given $(S, j) : \mathrm{B}\,\mathrm{C}_p$ however, functions $S \to A$ cannot really be thought the same because S is not explicitely enumerated. But as soon as we are given $q : \mathrm{Z}/p \xrightarrow{\simeq} (S, j)$, then functions $S \to A$ are just as good to model ordered p-tuples of A (just by precomposing with the first projection of q). With this in mind, define $\mu_p : (\mathbb{p} \to UG) \to UG$ to be the p-ary multiplication, meaning $\mu_p(g) :\equiv g_0 g_1 \dots g_{p-1}$. Then, one can define $\mu : \prod_{(S,j):\mathrm{B}\,\mathrm{C}_p} (\mathrm{Z}/p \xrightarrow{\simeq} (S, j)) \to (S \to UG) \to UG$ by $\mu_{(S,j)}(q)(g) :\equiv (gq)_0 \cdot \dots \cdot (gq)_{p-1}$ (where we use gq abusively to denote the composition of g with the equivalence given by applying the first projection to the identification q). We can now define the C_p-set $X : \mathrm{B}\,\mathrm{C}_p \to \mathrm{Set}$ as:

$$X(S, j) :\equiv \sum_{g : S \to UG} \prod_{q : \mathrm{Z}/p \xrightarrow{\simeq} (S,j)} \mu_{(S,j)}(q)(g) = e_G.$$

In particular, an element of $X(\mathrm{Z}/p)$ is a tuple $(g_0, \dots, g_{p-1})$ satisfying that $g_{\sigma 0} \dots g_{\sigma(p-1)} = e_G$ for every $\sigma : \mathrm{U}\,\mathrm{C}_p$. Note that this is equivalent to the set of tuples $(g_0, \dots, g_{p-1})$ satisfying that $g_0 \dots g_{(p-1)} = e_G$. So, the map $X(\mathrm{Z}/p) \to UG^{p-1}$ that send an element $(g_0, \dots, g_{p-1})$ to $(g_1, \dots, g_{p-1})$ is an equivalence (the condition $g_0 \dots g_{(p-1)} = e_G$ says exactly that we can reconstruct g_0 from $(g_1, \dots, g_{p-1})$). In particular, p divides the cardinality of $X(\mathrm{Z}/p)$.

Now, a C_p-fixed point of X, that is an element $f : \prod_{(S,j):\mathrm{B}\,\mathrm{C}_p} X(S, j)$, will have $f_{\mathrm{Z}/p}$ being an element $(g_0, \dots, g_{p-1})$ of $X(\mathrm{Z}/p)$ that satisfies (in

particular) $(g_0, \dots, g_{p-1}) = (g_1, \dots, g_{p-1}, g_0)$, i.e., such that $g_0 = g_1 = g_2 = \cdots = g_{p-1}$. In other words, a fixed point f is such that $f_{\mathbb{Z}/p} : X(\mathbb{Z}/p)$ is of the form $(g, \dots, g)$ where g satisfies $g^p = e_G$. So, there is a map $\mathrm{ev} : X^{\mathrm{C}_p} \to \sum_{g:UG} g^p = e_G$ simply given by evaluation at $\mathbb{Z}/p$. This map is an equivalence. Indeed, each fiber of ev is already a proposition, and we only need to show that each is inhabited. Given any $g : UG$ such that $g^p = e_G$, and given $(S, j) : \mathrm{B}\,\mathrm{C}_p$, one can consider the constant function $\hat{g} : S \to UG$ given by $\hat{g}(s) = g$ for all $s : S$. Then, for all $q : \mathbb{Z}/p \xrightarrow{=} (S, j)$, $\hat{g}q$ is the tuple $(g, \dots, g)$, so that we have $(\hat{g}, !) : X(S, j)$. In other words, we just constructed a fixed point of X whose image through ev is g, that is an element of the fiber of ev at g. In particular, X^{C_p} is not empty as it is equivalent to $\sum_{g:UG} g^p = e_G$, which contains at least e_G.

Now, Lemma 10.3.1 claims that p divides the cardinality of X^{C_p}, and since there *are* fixed points, there must be at least p fixed points. One of them is the trivial one (given by $g :\equiv e_G$ above), but the others are nontrivial.

[4] □

[4] Two slight variations commented away. Have to choose one. The first needs some background essentially boiling down to $B\mathrm{C}_n$ being the truncation of the nth Moore space.

lem:nontrivcenter

Lemma 10.3.3. *Let be G be a finite subgroup of cardinality p^n, where p is prime and n a positive integer. Then the center $Z(G)$ of G is nontrivial.* (*point to center in the symmetry chapter*)

Proof. Recall the G-set $\mathrm{Ad}_G : BG \to \mathrm{Set}$ given by $\mathrm{Ad}_G(z) = (z = z)$. Then the map

$$\mathrm{ev}_{\mathrm{sh}_G} : \prod_{z:BG} (z = z) \to UG, \quad \mathrm{ev}_G(f) = f(\mathrm{sh}_G)$$

has the structure of a (n abstract) inclusion of a subgroup; namely the inclusion of the center $Z(G)$ in G. The center thus represents the fixed points of the G-set Ad_G. Since G has cardinality a power of p, all orbits but the fixed points have cardinality divisible by p. Consequently, Burnside's lemma states that the number of fixed points, i.e., the cardinality of $Z(G)$, must be divisible by p. □

cor:orderpsquaredgroups

Corollary 10.3.4. *If G is a noncyclic group of cardinality p^2, then G of the form $C_p \times C_p$.*

Proof. The center $Z(G)$ is by Lemma 10.3.3 of cardinality p or p^2. Since G is not cyclic we have that $g^p = e_G$ for all $g : UG$. [5] □

[5] ((To be continued: the classical proof involves choosing nontrivial elements – see what can be done about that. At present this corollary is not used anywhere))

10.4 *Sylow's Theorems*

sec:sylow
thm:sylow1

Theorem 10.4.1. *If p is a prime, $n : \mathbb{N}$ and G a finite group whose cardinality is divisible by p^n, then G has a subgroup of cardinality p^n.*

Proof. We prove the result by induction on n. If $n = 0$ we need to have a subgroup of cardinality 1, which is witnessed by the trivial subgroup. If $n > 0$, assume by induction that G contains a subgroup K of cardinality p^{n-1}. Now, K acts on the set G/K. The cardinality of G/K is divisible by p (since p^n divides the cardinality of G), and so by Lemma 10.3.1 the fixed point set $(G/K)^K$ has cardinality divisible by p.

Recall the Weyl group $W_G K$. By Lemma 9.8.3,

$$|W_G K| = |(G/K)^K|,$$

and so $W_G K$ has cardinality divisible by p.

Recall the normalizer subgroup $N_G(K)$ of G from Definition 9.8.1 and Section 9.9 and the surjective homomorphism p_G^H from $N_G H$ to $W_G H$, whose kernel may be identified with H so that $|N_G H| = |W_G H| \cdot |H|$ by Lagrange's theorem.

By Cauchy's Theorem 10.3.2 there is a subgroup L of $W_G K$ of cardinality p. Taking the preimage of L under the projection $p_G^H : \mathrm{Hom}(N_G H, W_G H)$, or, equivalently, the pullback

$$BH :\equiv BL \times_{BW_G K} BN_G K,$$

we obtain a subgroup H of $N_G(K)$ of cardinality p^n (H is a free K-set with p orbits). The theorem is proven by considering H as a subgroup of G. □

def:sylowsubgroup

DEFINITION 10.4.2. Let p^n be the largest power of p which divides the cardinality of G. A subgroup of G of cardinality p^n is called a *p-Sylow subgroup* of G and Syl_G^p is the G-subset of Sub_G of p-Sylow subgroups of G. ⌟

lem:numberofconjofSylow

LEMMA 10.4.3. *Let G be a finite group and P a p-Sylow subgroup. Then the number of conjugates of P is not divisible by p.*

Proof. Let X be the G-set of conjugates of P. Being a G-orbit, X is equivalent G/Stab_P, where P is the stabilizer subgroup of P. Now, P is contained in the stabilizer so the highest power of p dividing the cardinality of G also divides the cardinality of Stab_P. □

lem:sylowsareconjugate

THEOREM 10.4.4. [6] *Let G be a finite group. Then any two p-Sylow subgroups are conjugate, or in other words, the G-set Syl_G^p is transitive.*

Furthermore, if H a subgroup of G of cardinality p^s and P a p-Sylow subgroup of G. Then H is conjugate to a subgroup of P.

[6] ((the approach below is on the abstract G-sets which may be ok given that this is what we're counting, but consider whether there is a more typie approach))

Proof. We prove the last claim first. Consider the set $\mathcal{O}_P$ of conjugates of P as an H-set. Since the cardinality of $\mathcal{O}_P \simeq G/Stab_P$ is prime to p there must be an H-fixed point Q. In other words, $H \subseteq Stab_Q$. By Lemma 9.6.4 there is a conjugate H' of H with $H' \subseteq Stab_P$. Now, $P \subseteq Stab_P$ (ref) is a normal subgroup and so by. [7]

[7] the end of the sentence appears to be missing

The first claim now follows, since if both H and P are p-Sylow subgroup, then a conjugate of H is a subgroup of P, but since these have the same cardinalities they must be equal. □

thm:syl0w3

THEOREM 10.4.5. *Let G be a finite group and let P be a p-Sylow subgroup of G. Then the cardinality of Syl_G^p*

(1) *divides $|G|/|P|$ and*

(2) *is 1 modulo p.*

Proof. Theorem 10.4.4 claims that Syl_G^p is transitive, so as a G-set it is equivalent to $G/N_G P$ ($N_G P$ is the stabilizer of P in Sub_G. Since P is a subgroup of $N_G P$ we get that $|P|$ divides $N_G P$ and so $|\mathrm{Syl}_G^p| = |G|/|N_G P|$ divides $|G|/|P|$.

Let i be the inclusion of P in G and consider the P-set $i^*\mathrm{Syl}_G^p$ obtained by restricting to P. Since the cardinality only depends on the underlying

set we have that $|i^*\mathrm{Syl}_G^p| = |\mathrm{Syl}_G^p|$ and we analyze the decomposition into P-orbits to arrive at our conclusion.

Let $Q : i^*\mathrm{Syl}_G^p$ be a fixed point, i.e., $P \subseteq N_G Q$. Now, since $N_G Q$ is a subgroup of G, we get that $|N_G Q|$ divides $|G$, so this proves that P is a p-Sylow subgroup of $N_G Q$. However, the facts that Q is normal in $N_G Q$ and that all Sylow subgroups being conjugates together conspire to show that $P = Q$. That is, the number of fixed points in $i^*\mathrm{Syl}_G^p$ is one. Since P is a p-group, all the other orbits have cardinalities divisible by p, and so

$$|\mathrm{Syl}_G^p| = |i^*\mathrm{Syl}_G^p| \equiv 1 \mod p.$$

□

((Should we include standard examples, or is this not really wanted in this book?))

11

Group presentations

ch:fggroups

sec:fggroups-overview

11.1 *Brief overview of the chapter*

TODO:

- Make a separate chapter on combinatorics? Actions and Burnside and counting colorings?
- Cayley actions: G acts on $\Gamma(G,S)$: Action on vertices is the left action of G on itself: $t \mapsto (t =_{BG} \mathrm{pt})$, on vertices, for $s:S$, have edge $t = \mathrm{pt}$ to $t = \mathrm{pt}$
- Recall universal property of free groups: If we have a map $\varphi : S \to H$, then we get a homomorphism $\bar{\varphi} : F(S) \to H$, represented by $BF(S) \to_{\mathrm{pt}} BH$ defined by induction, sending pt to pt and s to $\varphi(s)$.
- define different types of graphs (S-digraphs, $\tilde{S}$-graphs, (partial) functional graphs, graph homomorphisms, quotients of graphs)
- define (left/right) Cayley graphs of f.g. groups – $\mathrm{Aut}(\Gamma_G) = G$ (include $\alpha : F(S) \to G$ in notation?) – Cayley graphs are vertex transitive
- Cayley graphs and products, semi-direct products, homomorphisms
- Some isomorphisms involving semi-direct products – Exceptional automorphism of Σ_6: – Exotic map $\Sigma_5 \to \Sigma_6$. (Conjugation action of Σ_5 on 6 5-Sylow subgroups.) A set bundle $X : B\Sigma_6 \to B\Sigma_6$.
- https://math.ucr.edu/home/baez/six.html Relating Σ_6 to the icosahedron. The icosahedron has 6 axes. Two axes determines a golden rectangle (also known as a *duad*,[1] so there are 15 such. A symmetry of the icosahedron can be described by knowing where a fixed rectangle goes, and a symmetry of that rectangle. Picking three rectangles not sharing a diagonal gives a *syntheme*: three golden rectangles whose vertices make up the icosahedron. Some synthemes (known as *true crosses* have the rectangles orthogonal to each other, as in Figure 11.1. Fact: The symmetries of the icosahedron form the alternating symmetries of the 5 true crosses. Of course, we get an action on the 6 axes, thus a homomorphism $A_5 \to \Sigma_6$. Every golden rectangle lies in one true cross and two skew crosses. The combinatorics of duads, synthemes, and synthematic totals are illustrated in the Cremona-Richardson configuration and the resulting Tutte-Coxeter graph. The automorphism group of the latter is in fact $\mathrm{Aut}(\Sigma_6)$. If we color the vertices according to duad/syntheme, we get Σ_6 itself.
- define (left/right) presentation complex of group presentation

[1] These names come from Sylvester.

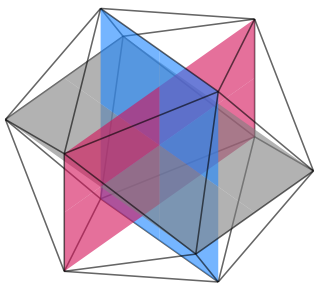


Figure 11.1: Icosahedron with an inscribed true cross

fig:true-cross

- define Stallings folding
- deduce Nielsen–Schreier and Nielsen basis
- deduce algorithms for generalized word problem, conjugation, etc.
- deduce Howson's theorem
- think about 2-cell replacement for folding; better proofs in HoTT?
- move decidability results to main flow
- include undecidability of word problem in general – doesn't depend on presentation (for classes closed under inverse images of monoid homomorphisms)
- describe $F(S)/H$ in the case where H has infinite index
- describe normal closure of R in $F(S)$ – still f.g.? – get Cayley graph of $F(S)/\langle R\rangle$. – Todd-Coxeter algorithm?
- in good cases we can recognize $\mathcal{S}(R)$ as a "fundamental domain" in Cayley graph of $\langle S \mid R\rangle$.

Remark 11.1.1. In this chapter, we use letters from the beginning of the alphabet $a, b, c, \ldots$ to denote generators, and we use the corresponding capital letters A, B, C to denote their inverses, so, e.g., $aA = Aa = 1$. This cleans up the notational clutter significantly. ⌟

Do we fix S, a finite set $S = \{a, b, \ldots\}$? Mostly F will denote the free group on S. And for almost all examples, we take $S = \{a, b\}$.

11.2 *Graphs and Cayley graphs*

sec:cayley-graphs

We have seen in the previous chapter how cyclic groups (those generated by a single generator) have neatly described types of torsors. Indeed, $\mathrm{BC}_n \equiv \mathrm{Cyc}_n$, where Cyc_n is the type of n-cycles, and the classifying type of the integers, $\mathrm{B}\mathbb{Z} \equiv \mathrm{S}^1$, i.e., the circle, is equivalent to the type of infinite cycles, Cyc_0. In Chapter 3, we defined the types of (finite or infinite) cycles as certain components of $\sum_{X:\mathcal{U}}(X \xrightarrow{=} X)$, but we can equivalently consider components of $\sum_{X:\mathcal{U}}(X \to X)$, since the former is a subtype of the latter. By thinking of functions in terms of their graphs, we might as well look at components of $\sum_{X:\mathcal{U}}(X \to X \to \mathcal{U})$.

In this section we shall generalize this story to groups G generated by a (finite or just decidable) set of generators S.

First recall from Cayley's Theorem 5.6.1 that any group G can be realized as a subgroup of the permutation group on the underlying set of symmetries in G, UG. In this description, a G-shape is a set X equipped a G-action that defines a G-torsor, which in turn can be expressed as the structure of a map $\alpha : UG \to X \to X$ satisfying certain properties.

It may happen that already α restricted to a subset S of UG suffices to specify the action. In that case we say that S generates G, though we'll take the following as the official definition.

def:gens-gp

Definition 11.2.1. Let G be a group and S be a subset of UG, given by an inclusion $\iota : S \to UG$. We say that S *generates* G if the induced homomorphism from the free group on S,

$$\mathrm{F}_S \to G,$$

fig:cayley-s3

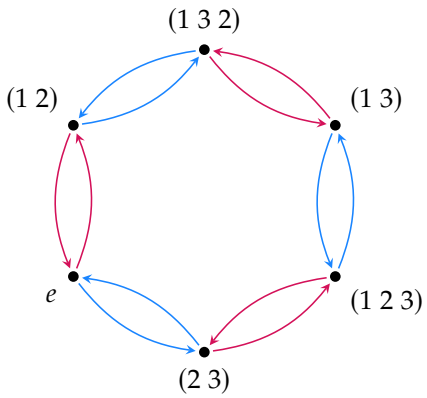


FIGURE 11.2: Cayley graph for Σ_3 with respect to $S = \{(1\ 2), (2\ 3)\}$.

is an epimorphism. ⌟

lem:gens-gp-iff

LEMMA 11.2.2. *Let G be a group and $\iota : S \to UG$ an inclusion of a subset of the elements of G. Then S generates G if and only if the map*

$$\rho_S : BG \to \sum_{X:\mathcal{U}} (S \to X \to X), \quad \rho_S(t) :\equiv \left(t \xrightarrow{=} \mathrm{sh}_G, s \mapsto \iota(s) \cdot _\right)$$

is an embedding.[2]

[2] We use $t \xrightarrow{=} \mathrm{sh}_G$ rather than the equivalent $\mathrm{sh}_G \xrightarrow{=} t$ in order to conform to the representation from Cayley's theorem.

In this case, then, G can be identified with the automorphism group of $\rho_S(\mathrm{sh}_G)$ in the type $\sum_{X:\mathcal{U}}(S \to X \to X)$, or even in the larger type (of which it's a subtype), $\sum_{X:\mathcal{U}}(S \to X \to X \to \mathcal{U})$.

Also note that S generates G if and only if the map on elements $\mathrm{UF}_S \to UG$ is surjective, meaning every element of G can be expressed as a product of the letters in a (reduced) word from $\mathcal{R}_S$, interpreted according to the inclusion of S into UG. This is the case for example for S consisting of the transpositions (1 2), (2 3) in Σ_3, as illustrated in Figure 11.2, where the blue color represents (1 2) and the red color represents (2 3).

Before we give the proof of Lemma 11.2.2, let us study these types more closely.

DEFINITION 11.2.3. An S-labeled graph is an element (V, E) of the type $\sum_{V:\mathcal{U}}(S \to V \to V \to \mathcal{U})$. The first component V is called the type of *vertices* of the graph, and the type $E(s, x, y)$ is called the type of s-colored *edges* from x (the source) to y (the target). ⌟

If for every vertex $x : V$ and every color $s : S$ there is unique s-colored edge out of x, i.e., the type $\sum_{y:V} E(s, x, y)$ is contractible, then we say that the graph is *functional*. This means that the graph lives in the subtype $\sum_{V:\mathcal{U}}(S \to V \to V)$, as is the case for the graph $\rho_S(\mathrm{sh}_G)$ for a group G. This graph is called the Cayley graph of G with respect to the set S:

def:cayley-graph

DEFINITION 11.2.4. The *Cayley graph* of a group G with respect to a generating subset S is the graph $\mathrm{Cay}(G; S)$ is the S-colored graph with vertices UG and edges $S \times UG$ where the edge (s, g) has source g, target sg, and color s. ⌟

Convince yourself that this is really an equivalent description of $\rho_S(\mathrm{sh}_G)$ considered as an S-colored graph.

If S is contractible (so there's only one color), then we just say *graph*, and then we simplify the type of edges to $V \to V \to \mathcal{U}$. Of course, every S-labeled graph (V, E) gives rise to such an unlabeled label by

summing over the colors, i.e., the type of edges from x to y in this graph is $\sum_{s:S} E(s,x,y)$.

Another way to represent a graph is to sum over all the sources and targets (and colors), via Lemma 2.25.3, i.e., as a tuple (V,E,s,t,c), where $V:\mathcal{U}$ is the type of vertices, E is the (total) type of edges, $s,t:E\to V$ give the source and target of an edge, while $c:E\to S$ gives the color (if we're talking about S-colored graphs). In this description, to get the unlabeled graph we simply drop the last component.

Every graph (V,E) (and thus every labeled graph) gives rise to a type by "gluing the edges to the vertices" defined as follows.

Definition 11.2.5. Fix an unlabeled graph (V,E). The *graph quotient*[3] V/E is the higher inductive type with constructors:

(1) For every vertex $x:V$ a point $[x]:V/E$.

(2) For every edge $e:E(x,y)$ an identification $\curvearrowright_e : [x] \xrightarrow{=} [y]$.

[3] If the graph is represented by source and target maps $s,t:E\rightrightarrows V$, then the graph quotient is also called the *coequalizer* of s and t.

Let $A(z)$ be a type for every element $z:V/E$. The induction principle for V/E states that, in order to define an element of $A(z)$ for every $z:V/E$, it suffices to give elements $a_x:A([x])$ for every vertex $x:V$ together with identifications $q_e:a_x \xrightarrow[\curvearrowright_e]{=} a_y$ for every $e:E(x,y)$. The function f thus defined satisfies $f([x])\equiv a_x$ for $x:V$ and we are provided identifications $\mathrm{apd}_f(\curvearrowright_e) \xrightarrow{=} q_e$ for each $e:E(x,y)$. ⌟

Remark 11.2.6. Note the similarity with the classifying type of a free group, cf. Definition 8.7.1. Indeed, if we form the (unlabeled!) graph $(\mathbb{1},S)$ on one vertex with S edges, then $\mathbb{1}/S$ is essentially the same as BF_S. ⌟

Exercise 11.2.7. An equivalence relation $R:A\to A\to \mathrm{Prop}$ on a set A can be regarded as a graph (A,R). Construct an equivalence between set truncation of the graph quotient $\|A/R\|_0$ and the set quotient A/R from Definition 2.22.11 in this case. (So in the world of sets, the two notations agree.) ⌟

While we're building up to the proof of Lemma 11.2.2 we need a description of a sum type over a graph quotient. By the above remark, this applies also to sum types over BF_S.

def:graph-quotient-flattening

Construction 11.2.8. *Given a graph (V,E) and a family of types $X:V/E\to\mathcal{U}$. Define $V' :\equiv \sum_{v:V} X([v])$ and $E'((v,x),(w,y)) :\equiv \sum_{e:E(v,w)} x \xrightarrow[\curvearrowright_e]{=} y$. Then we have an equivalence*[4]

$$\mathrm{flt}:\Big(\sum_{z:V/E} X(z)\Big) \xrightarrow{\simeq} V'/E'$$

[4] This is often called the *flattening construction* (or flattening lemma), as it "flattens" a sum over a graph quotient into a single graph quotient.

Implementation of Construction 11.2.8. We define functions $\varphi:V'/E'\to\sum_{z:V/E}X(z)$ and $\psi:\prod_{z:V/E}\big(X(z)\to V'/E'\big)$ using the induction principles:

$$\varphi([(v,x)]) :\equiv ([v],x) \qquad \tilde\psi([v]) :\equiv (x\mapsto [(v,x)])$$
$$\mathrm{ap}_\varphi(\underset{(e,q)}{\curvearrowright}) :\equiv \overline{(\underset{e}{\curvearrowright},q)} \quad \mathrm{apd}_{\tilde\psi}(\underset{e}{\curvearrowright}) :\equiv h,$$

where we need to construct $h:(x\mapsto[(v,x)]) \xrightarrow[\curvearrowright_e]{=} (y\mapsto[(w,y)])$ for all $e:E(v,w)$. By transporting in families of functions, it suffices to give an

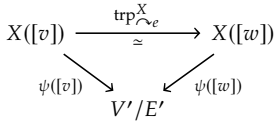


fig:cayley-a5

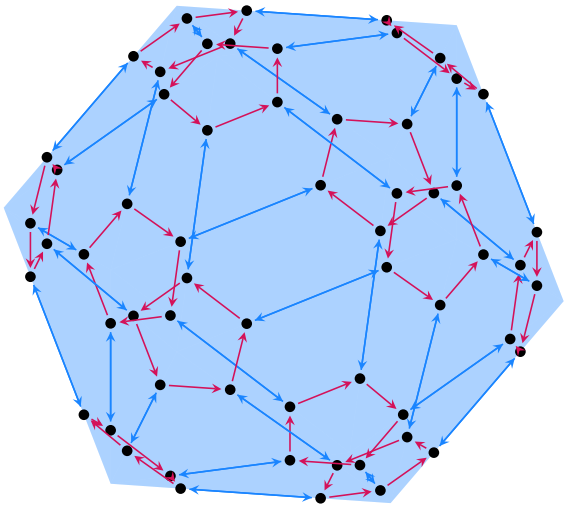


FIGURE 11.3: Cayley graph for A_5 with respect to $S = \{a, b\}$, where a is a 1/5-rotation about a vertex and b is a 1/2-rotation about an edge in an icosahedron.

identification $[(v, x)] \xrightarrow{=} [(w, \mathrm{trp}^X_{\curvearrowright_e}(x))]$ for all $x : X([v])$. We get this as the identification constructor $\curvearrowright_{(e,q)}$ for V'/E', where $q : x \xrightarrow[\curvearrowright_e]{=} \mathrm{trp}^X_{\curvearrowright_e}(x)$ is the identification over $\curvearrowright_e$ corresponding to the reflexivity identification at $\mathrm{trp}^X_{\curvearrowright_e}(x)$ via Definition 2.7.3. □

EXERCISE 11.2.9. Complete the implementation by giving identifications $\psi \circ \phi \xrightarrow{=} \mathrm{id}$ and $\phi \circ \psi \xrightarrow{=} \mathrm{id}$, where $\psi : \left(\sum_{z:V/E} X(z)\right) \to V'/E'$ is defined by $\psi((z, x)) :\equiv \tilde{\psi}(z)(x)$. ⌟

Later on we'll need also need the following results about graph quotients.

xca:graph-quotient-in-steps

EXERCISE 11.2.10. Suppose the edges E of a graph (V, E) are expressed as a binary sum $E_0 \amalg E_1$. (Here, it doesn't matter whether E is expressed as a type family $E : V \to V \to \mathcal{U}$, in which case we have a family of equivalences $E(v, w) \xrightarrow{\simeq} E_0(v, w) \amalg E_1(v, w)$, or E is the total type of edges.)

Then we can obtain the graph quotient V/E by first gluing in the edges from E_0, and then gluing in the edges from E_1 to the resulting type V/E_0. Using the description of graphs with a total type of edges $E \xrightarrow{\simeq} E_0 \amalg E_1$, we have corresponding source and target maps expressed as compositions:

$$E_1 \hookrightarrow E_0 \amalg E_1 \xrightarrow{\simeq} E \rightrightarrows V \to V/E_0.$$

Construct an equivalence $V/E \xrightarrow{\simeq} V/(E_0 \amalg E_1) \xrightarrow{\simeq} (V/E_0)/E_1$. ⌟

xca:graph-quotient-whisker

EXERCISE 11.2.11. Suppose we have any type X with an element $x : X$. We can form a graph $(X \amalg \mathbb{1}, \mathbb{1})$ with vertex type $X \amalg \mathbb{1}$ and a single edge from inl_x to inr_0. Construct an equivalence $X \xrightarrow{\simeq} (X \amalg \mathbb{1})/\mathbb{1}$.[5] ⌟

11.3 *Examples*

sec:fg-examples

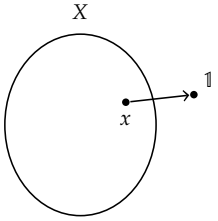
Proof of Lemma 11.2.2. TBD (perhaps put in graph quotients first) □

11.4 *Subgroups of free groups*

sec:subgroups-free

We now study subgroups of free groups. We'll eventually prove the

[5] This equivalence can be visualized as follows, where X "grows a whisker" along the single edge.



Our discussion follows the work of Swan[6].

[6] Andrew W. Swan. "On the Nielsen–Schreier Theorem in Homotopy Type Theory". In: *Log. Methods Comput. Sci.* 18.1 (2022). DOI: 10.46298/lmcs-18(1:18)2022.

Nielsen–Schreier theorem, which states that a finite index subgroup H of a free group F_S is itself a free group. Furthermore, when S is finite, the set of free generators of H is itself finite.

Recall from Definition 5.3.2 that a subgroup is (or can be represented by) a transitive G-set $X : BG \to \mathrm{Set}$ along with an element of $X(\mathrm{sh}_G)$.

def:finite-index

Definition 11.4.1. A subgroup of a group G has *finite index* m if the underlying transitive G-set, $X : BG \to \mathrm{Set}$ is a family of finite sets of cardinality m. ⌟

The is the case, of course, if and only if the set acted on, $X(\mathrm{sh}_G)$, is finite of cardinality m. Notice that the definition doesn't depend on the chosen element of $X(\mathrm{sh}_G)$, so applies equally to all conjugacy classes of the subgroup.

Recall also that the classifying type of the subgroup is the total type $\sum_{t:BG} X(t)$ (which is pointed via the chosen point of $X(\mathrm{sh}_G)$). We'll use the Flattening Construction 11.2.8 to analyze this in case where G is the free group on a set S, F_S, so we need to show that the quotient of the resulting graph is equivalent to $\mathbb{1}/T$ for some set T.

We do this by finding a "spanning tree" in the graph.

Definition 11.4.2. A graph (V, E) is *connected* if V/E is a connected type and it's a *tree* if V/E is contractible. ⌟

Definition 11.4.3. A *subgraph* of a graph (V, E) consists of a subtype $h : U \hookrightarrow V$ of the vertices along with, for every pair of vertices v, w in U, a subtype $D(v, w)$ of the edges $E(v, w)$. ⌟

If we represent graphs by source and target maps, then this amounts to embeddings $h : U \hookrightarrow V$ and $k : D \hookrightarrow E$ along with witnesses that the following squares commute:

$$\begin{array}{ccc} D & \xhookrightarrow{k} & E \\ {\scriptstyle s}\downarrow & & \downarrow{\scriptstyle s} \\ U & \xhookrightarrow[h]{} & V \end{array} \qquad \begin{array}{ccc} D & \xhookrightarrow{k} & E \\ {\scriptstyle t}\downarrow & & \downarrow{\scriptstyle t} \\ U & \xhookrightarrow[h]{} & V \end{array}$$

Definition 11.4.4. A *spanning tree* in a graph (V, E) is a subgraph (U, D) such that (U, D) is a tree, and the embedding of the vertices $U \hookrightarrow V$ is an equivalence. ⌟

Equivalently, it's given by subtypes of the edges (leaving the vertices alone) such that the underlying graph is a tree. Very often we'll require that the edge embeddings are decidable, i.e., we can decide whether a given edge $e : E(v, w)$ is part of the tree.

lem:crossing-edge

Lemma 11.4.5. *Suppose we have a connected graph (V, E) whose type of vertices decomposes as a binary sum $V \xrightarrow{\simeq} V_0 \amalg V_1$ and we have $v_0 : V_0$ and $v_1 : V_1$. Then there merely exists an edge e either with source in V_0 and target in V_1 or the other way round.*

The situation is illustrated in Figure 11.4, where we assume there is an edge relation on the binary sum that gives a connected graph, and hence there must be a "crossing edge" e, going either from V_0 to V_1 or the other way.

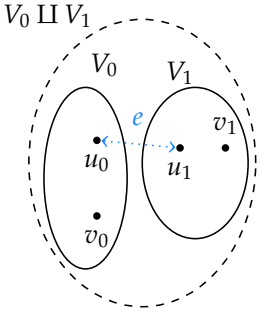


Figure 11.4: A connected graph with a crossing edge

fig:crossing-edge

Proof. We may assume $V \equiv V_0 \amalg V_1$ by path induction. The idea is then to define a family of propositions $P : V/E \to \mathrm{Prop}$ that, on one hand is

trivially true over V_0, and on the other hand expresses our desired goal, the existence of a "crossing edge", over V_1.

We now define $P(z)$, for $z : V/E$, by the induction principle for the graph quotient V/E. We set $P([\mathrm{inl}_v]) :\equiv \mathrm{True}$ for $v : V_0$ and

$$P([\mathrm{inr}_v]) :\equiv \left\| \sum_{u_0 : V_0} \sum_{u_1 : V_1} \big(E(u_0, u_1) \amalg E(u_1, u_0)\big) \right\|$$

for $v : V_1$. We must then prove that the propositions $P([v])$ and $P([v'])$ are equivalent whenever there's an edge from v to v'. This is the case by definition when v, v' lie in the same summand, and it's also the case when they lie in different summands, since then we get a witness for the truth over V_1.

Since V/E is connected, P must have a constant truth value, and since $P([\mathrm{inl}_{v_0}]) \equiv \mathrm{True}$, every $P(z)$ is true. Hence also $P([\mathrm{inr}_{v_1}])$ is true, which is exactly what we wanted. □

lem:spanning-tree-step

Lemma 11.4.6. *Fix a connected graph (V, E) where V has decidable equality and E is a family of sets. For any subgraph (U, D), where the embedding $U \hookrightarrow V$ is decidable, and with vertices $u \in U$ and $v \in V \setminus U$, there merely exists[7] a larger subgraph with exactly one more vertex and one more edge, $(U \amalg \mathbb{1}, D \amalg \mathbb{1})$ such that the induced map on graph quotients $U/D \to (U \amalg \mathbb{1})/(D \amalg \mathbb{1})$ is an equivalence.*

Proof. Since the embedding $U \hookrightarrow V$ is decidable, we can write V as the binary sum $U \amalg (V \setminus U)$. Apply Lemma 11.4.5 to find a "crossing edge" e, and form the new subgraph $(U \amalg \mathbb{1}, D \amalg \mathbb{1})$ by adding the incident vertex not in U as well as the edge e itself. The embedding $U \amalg \mathbb{1} \to V$ is still decidable, since V has decidable equality. Finally, we have

$$(U \amalg \mathbb{1})/(D \amalg \mathbb{1}) \xrightarrow{\simeq} \big((U \amalg \mathbb{1})/\mathbb{1}\big)/D \xrightarrow{\simeq} U/D,$$

using Exercises 11.2.10 and 11.2.11, as desired. □

[7] Keep in mind that subgraphs consist not only of the vertices and edges, but also of the corresponding embeddings into the supergraph. It's for the sake of these that we only prove mere existence.

$$\begin{array}{ccccc} D & \hookrightarrow & D \amalg \mathbb{1} & \hookrightarrow & E \\ \downarrow\downarrow & & \downarrow\downarrow & & \downarrow\downarrow \\ U & \hookrightarrow & U \amalg \mathbb{1} & \hookrightarrow & V \\ \downarrow & & \downarrow & & \downarrow \\ U/D & \xrightarrow{\simeq} & (U \amalg \mathbb{1})/(D \amalg \mathbb{1}) & \to & V/E \end{array}$$

lem:spanning-tree

Lemma 11.4.7. *Let (V, E) be a connected graph where V is an n-element set, and E is a family of decidable sets. Then the graph merely has a spanning tree with exactly $n - 1$ edges.*

Proof. We show by induction on k, with $1 \leq k \leq n$, that there merely exists a subgraph (U, D) with k vertices, $k-1$ edges, and U/D contractible, i.e., the graph (U, D) is a tree.

For $k \equiv 1$, we use that V/E is connected to get that V merely has a vertex v. This then defines the desired subgraph on one vertex with no edges, and this is clearly a tree.

Suppose we have such a desired subgraph (U, D) with k vertices and $k - 1$ edges and $k < n$. Since V is finite, there exists vertices $u \in U$ and $v \in V \setminus U$. Now apply Lemma 11.4.6 to get the next subgraph.

Finally, the subgraph (U, D) with n vertices and $n - 1$ edges gives the desired spanning tree, and any embedding of an n-element set in another n-element set is an equivalence.[8] □

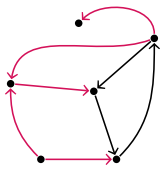


Figure 11.5: A connected graph on 6 vertices with a spanning tree indicated in red.

fig:spanning-tree-exampl

[8] Assuming the Axiom of Choice, we can show the mere existence of a spanning tree in any graph (V, E) with a sets of vertices and edges. See the above work by Swan.

Theorem 11.4.8 (Nielsen–Schreier Theorem). *Suppose that S is a set with decidable equality and $X : \mathrm{BF}_S \to \mathrm{Set}$ defines a (conjugacy class of a) finite*

index subgroup of F_S. *Then* $\sum_{z:\mathrm{BF}_S} X(z)$ *is merely equivalent to* BF_T *for some set* T.

Moreover, if S *is a finite set of cardinality* n *and the subgroup has index* m, *then* T *can be taken to be a finite set of cardinality* $m(n-1)+1$.

Proof. By the Flattening Construction 11.2.8, we have an equivalence $\mathrm{flt} : \left(\sum_{z:\mathrm{BF}_S} X(z)\right) \xrightarrow{\simeq} V/E$, with $V :\equiv X(\bullet)$ and $E(x,y) :\equiv \sum_{s:S}(x \xrightarrow[\circlearrowright_s]{=} y)$. By the finite index assumption, V is a finite set, say, of cardinality $m > 0$, and since both S and $X(\bullet)$ are decidable, so is E.

By Lemma 11.4.7, the graph (V,E) merely contains a spanning tree with $m-1$ edges E_0, and complementary edge set E_1. Hence, using Exercise 11.2.10, we have a chain of equivalences:

$$V/E \xrightarrow{\simeq} (V/E_0)/E_1 \xrightarrow{\simeq} \mathbb{1}/E_1 \xrightarrow{\simeq} \mathrm{BF}_{E_1}$$

This establishes the first claim with $T :\equiv E_1$.

If, furthermore, S has cardinality n, then the graph (V,E) has mn edges, as there are precisely n outgoing edges from each vertex. Since E_0 has $m-1$ edges, that leaves $mn-(m-1) = mn-m+1 = m(n-1)+1$ edges in E_1, as desired. □

(This also has an automata theoretic proof, see below.)

11.5 *Intersecting subgroups*

sec:intersecting-subgroups

Stallings folding[9].

[9] John R. Stallings. "Foldings of G-trees". In: *Arboreal group theory (Berkeley, CA, 1988)*. Vol. 19. Math. Sci. Res. Inst. Publ. Springer, New York, 1991, pp. 355–368. DOI: 10.1007/978-1-4612-3142-4_14.

Theorem 11.5.1. *Let* H *be a finitely generated subgroup of* $F(S)$ *and let* $u \in \tilde{S}^*$ *be a reduced word. Then* u *represents an element of* H *if and only if* u *is recognized by the Stallings automaton* $\mathcal{S}(H)$.

Theorem 11.5.2. *Let* H *be a finitely generated subgroup of* $F(S)$. *Then* H *has finite index if and only if* $\mathcal{S}(H)$ *is total.*

Furthermore, in this case the index equals the number of vertices of $\mathcal{S}(H)$.

Corollary 11.5.3. *If* H *has index* n *in* $F(S)$, *then* $\mathrm{rk}\, H = 1 + n(\mathrm{card}\, S - 1)$.

thm:howson-neumann

Theorem 11.5.4. *Suppose* H_1, H_2 *are two subgroups of* F *with finite indices* h_1, h_2. *Then the intersection* $H_1 \cap H_2$ *has finite index at most* $h_1 h_2$.

The qualitative part of Theorem 11.5.4 is known as *Howson's theorem*, while the inequality is known as *Hanna Neumann's inequality*. Hanna's son, Walter Neumann, conjectured that the 2 could be removed, and this was later proved independently by Joel Friedman and Igor Mineyev.

11.6 *Connections with automata* (*)

(S is still a fixed finite set.)

Let $\iota : F(S) \to \tilde{S}^*$ map an element of the free group to the corresponding reduced word. The kernel of ι is the 2*-sided Dyck language* $\mathcal{D}_S$.

The following theorem is due to Benois.

Theorem 11.6.1. *A subset* X *of* $F(S)$ *is rational if and only if* $\iota(X) \subseteq \tilde{S}^*$ *is a regular language.*

Lemma 11.6.2. *Let* $\rho : \tilde{S}^* \to \tilde{S}^*$ *map a word to its reduction. Then* ρ *maps regular languages to regular languages.*

The following is due to Sénizergues:

Theorem 11.6.3. *A rational subset of* $F(S)$ *is either disjunctive or recognizable.*

Given a surjective monoid homomorphism $\alpha : S^* \to G$, we define the corresponding *matched homomorphism* $\tilde{\alpha} : \tilde{S}^* \to G$ by $(\tilde{\alpha}(a^{-1}) :\equiv \alpha(a)^{-1}$.

THEOREM 11.6.4 (?). *Consider a f.g. group G with a surjective homomorphism $\alpha : F(S) \to G$. A subset X of G is recognisable by a finite G-action if and only if $\tilde{\alpha}^{-1}(X) \subseteq \tilde{S}^*$ is rational (i.e., regular).*

THEOREM 11.6.5 (Chomsky–Schützenberger). *A language $L \subseteq T^*$ is context-free if and only if $L = h(R \cap D_S)$ for some finite S, where $h : T^* \to \tilde{S}^*$ is a homomorphism, $R \subseteq \tilde{S}^*$ is a regular language, and D_S is the Dyck language for S.*[10]

[10] References TODO. The theorem is also true if we replace D_S by its one-sided variant, but in this case it reduces to the well-known equivalence between context-free languages and languages recognizable by push-down automata.

THEOREM 11.6.6 (Muller–Schupp, ?). *Suppose $\tilde{\alpha} : \tilde{S}^* \to G$ is a surjective matched homomorphism onto a group G. Then G is virtually free (i.e., G has a normal free subgroup of finite index) if and only if* $\ker(\tilde{\alpha})$ *is a context-free language.*

THEOREM 11.6.7.

The Stallings automaton is an *inverse automaton*: it's deterministic, and there's an edge (p, a, q) if and only if there's one (q, A, p). We can always think of the latter as the *reverse* edge. (It's then also deterministic in the reverse direction.)

Two vertices p, q get identified in the Stallings graph/automaton if and only if there is a run from p to q with a word w whose reduction is 1. (So a word like $aAAaBBbb$.)

THEOREM 11.6.8. *Let $X \subseteq F(S)$. Then Y is a coset Hw with H a finitely generated subgroup, if and only if there is a finite state inverse automaton whose language (after reduction) is Y.*

COROLLARY 11.6.9. *The generalized word problem in $F(S)$ is solvable: Given a finitely generated subgroup H, and a word $u : \tilde{S}^*$, we can decide whether u represents an element of H.*

The Stallings automaton for H can be constructed in time $O(n \log^* n)$, where n is the sum of the lengths of the generators for H. [Cite: Touikan: A fast algorithm for Stallings' folding process.] Once this has been constructed, we can solve membership in H in linear time.

As above, we get a basis for H as a free group from a spanning tree in $\mathcal{S}(H)$.

THEOREM 11.6.10. *We can decide whether two f.g. subgroups of $F(S)$ are conjugate. Moreover, a f.g. subgroup H is normal if and only if $\mathcal{S}(H)$ is vertex-transitive.*

Proof. G, H are conjugate of and only if their cores are equal. □

There are other connections between group theory and language theory:

THEOREM 11.6.11 (Anisimov and Seifert). *A subgroup H of G is rational if and only if H is finitely generated.*

THEOREM 11.6.12. *A subgroup H of G is recognizable if and only if it has finite index.*

12

Abelian Groups

ch:abelian

12.1 *Brief overview of the chapter*

12.2 *Abelian groups*

sec:abelian-groups

Recall that given a pointed type X, we coerce it silently to its underlying unpointed type $X_\div$ whenever this coercion can be inferred from context. For example, given a group G, the type $BG \simeq BG$ can not possibly mean anything but $BG_\div \simeq BG_\div$ as the operator "$\simeq$" acts on bare types. To refer to the type of pointed equivalences (that is the pointed functions whose underlying functions are equivalences), we shall use the notation $BG \xrightarrow{\simeq}_* BG$.

12.2.1 *Center of a group*

sec:center-group

Definition 12.2.2. Let G be a group. The *center* of G, denoted $\mathrm{Z}(G)$, is the group $\mathrm{Aut}_{(BG_\div \xrightarrow{=} BG_\div)}(\mathrm{refl}_{BG_\div})$. ⌟

We work transparently through the equivalence

$$(BG_\div = BG_\div) \simeq (BG \simeq BG)$$

so that $\mathrm{id}_{BG_\div}$ is freely used in place of $\mathrm{refl}_{BG_\div}$ when convenient.

There is a natural map $\mathrm{ev}_{\mathrm{sh}_G} : (BG_\div \xrightarrow{=} BG_\div) \to BG_\div$ defined by $\mathrm{ev}_{\mathrm{sh}_G}(\varphi) :\equiv \varphi(\mathrm{sh}_G)$, where the path φ is coerced to a function through univalence. In particular, $\mathrm{ev}_{\mathrm{sh}_G}(\mathrm{refl}_{BG_\div}) \equiv \mathrm{sh}_G$. It makes the restriction of this map to the connected component of $\mathrm{refl}_{BG_\div}$ a pointed map. In other words, it defines a group homomorphism

$$\mathrm{z}_G : \mathrm{Hom}(\mathrm{Z}(G), G).$$

such that $\mathrm{Bz}_G :\equiv \mathrm{ev}_{\mathrm{sh}_G}$. We will now justify the name *center* for $\mathrm{Z}(G)$, and connect it to the notion of center for abstract groups in ordinary mathematics. The homomorphism z_G induces a homomorphism of abstract groups from $\mathrm{abs}(\mathrm{Z}(G))$ to $\mathrm{abs}(G)$. By induction on $p : \mathrm{refl}_{BG_\div} \xrightarrow{=} \varphi$ for $\varphi : BG_\div \xrightarrow{=} BG_\div$, one proves that $\mathrm{ap}_{\mathrm{Bz}_G}(p) = p(\mathrm{sh}_G)$: indeed, this is true when $p \equiv \mathrm{refl}_{\mathrm{refl}_{BG_\div}}$. One proves furthermore, again by induction on $p : \mathrm{refl}_{BG_\div} \xrightarrow{=} \varphi$, that $\mathrm{ap}_\varphi = (q \mapsto p(\mathrm{sh}_G)^{-1} q p(\mathrm{sh}_G))$.

In particular, when $\varphi \equiv \mathrm{refl}_{BG_\div}$, it shows that for every $p : \mathrm{refl}_{BG_\div} \xrightarrow{=} \mathrm{refl}_{BG_\div}$, the following proposition holds:

$$\prod_{g : \mathit{UG}} p(\mathrm{sh}_G) g = g p(\mathrm{sh}_G)$$

$$\begin{array}{ccc} \mathrm{sh}_G & \xrightarrow[=]{q} & \mathrm{sh}_G \\ {\scriptstyle p(\mathrm{sh}_G)}\downarrow & & \downarrow{\scriptstyle p(\mathrm{sh}_G)} \\ \varphi(\mathrm{sh}_G) & \xrightarrow[\mathrm{ap}_\varphi(q)]{=} & \varphi(\mathrm{sh}_G). \end{array}$$

Figure 12.1: Check in text

In other words, $\mathrm{abs}(\mathrm{z}_G)$ maps elements of $\mathrm{abs}(Z(G))$ to elements of $\mathrm{abs}(G)$ that commute with every other elements. (The set of these elements is usually called the center of the group $\mathrm{abs}(G)$ in ordinary group theory.)

Lemma 12.2.3. *The map* Bz_G *is a set bundle over* BG.

lemma:center-is-subgroup

Proof. One wants to prove the proposition $\mathrm{isSet}((\mathrm{Bz}_G)^{-1}(x))$ for each $x : BG$. By connectedness of BG, it reduces to showing the proposition only at $x \equiv \mathrm{sh}_G$. However,

$$(\mathrm{Bz}_G)^{-1}(\mathrm{sh}_G) :\equiv \sum_{\varphi : \mathrm{B}\,\mathrm{Z}(G)} \mathrm{sh}_G \xrightarrow{=} \varphi(\mathrm{sh}_G)$$

Recall that $\mathrm{B}\,\mathrm{Z}(G)$ is the connected component of $\mathrm{refl}_{BG_\div}$ in $BG_\div \xrightarrow{=} BG_\div$. In particular, if (φ, p) and (ψ, q) are two elements of the type on the right hand-side above, the characterization of identity types in sum types gives an equivalence:

$$((\varphi, p) \xrightarrow{=} (\psi, q)) \xrightarrow{\simeq} \sum_{\pi : \varphi \xrightarrow{=} \psi} \pi(\mathrm{sh}_G)p = q.$$

We shall prove that the type on the right is a proposition, and it goes as follows:

(1) for $\pi : \varphi \xrightarrow{=} \psi$, the type $\pi(\mathrm{sh}_G)p = q$ is a proposition; hence $\sum_{\pi : \varphi \xrightarrow{=} \psi} \pi(\mathrm{sh}_G)p = q$ is a subset of the set $\varphi \xrightarrow{=} \psi$, so for elements $(\pi, !)$ and $(\pi', !)$ of the subset, we have to prove $\pi = \pi'$,

(2) because $\pi = \pi'$ is a proposition, by connectedness of BG, it is enough to prove $\pi(\mathrm{sh}_G) = \pi'(\mathrm{sh}_G)$,

(3) finally the propositional condition on π and π' allows us to conclude that $\pi(\mathrm{sh}_G) = qp^{-1} = \pi'(\mathrm{sh}_G)$.

□

Corollary 12.2.4. *The induced map* $\mathrm{abs}\,(\mathrm{z}_G) : \mathrm{abs}(\mathrm{Z}(G)) \to \mathrm{abs}(G)$ *is injective.*

The following result explains how every element of the "abstract center" of G is picked out by $\mathrm{abs}\,(\mathrm{z}_G)$.

Lemma:center-inc-inj-on-paths

Lemma:center-inc-surj-on-paths

Lemma 12.2.5. *Let* $g : UG$ *and suppose that* $gh = hg$ *for every* $h : UG$*. The fiber* $(\mathrm{ap}_{\mathrm{Bz}_G})^{-1}(g)$ *contains an element.*

Proof. One must construct an element $\hat{g} : \mathrm{refl}_{BG_\div} = \mathrm{refl}_{BG_\div}$ such that $g = \hat{g}(\mathrm{sh}_G)$. We shall use function extensionality and produce an element $\hat{g}(x) : x \xrightarrow{=} x$ for all $x : BG$ instead. Note that $x \xrightarrow{=} x$ is a set, and that connectedness of BG is not directly applicable here. We will use a technique that has already proven useful in many situations in the book, along the lines of the following sketch:

(1) for a given $x : BG$, if such a $\hat{g}(x) : x \xrightarrow{=} x$ existed, it would produce an element of the type $T(\hat{g}(x))$ for a carefully chosen type family T,

(2) aim to prove $\mathrm{isContr}(\sum_{u : x \xrightarrow{=} x} T(u))$ for any $x : BG$,

(3) this is a proposition, so connectedness of BG can be applied and only $\mathrm{isContr}(\sum_{u : UG} T(u))$ needs to be proven,

(4) hopefully, $\sum_{u : UG} T(u)$ reduces to an obvious singleton type.

Here, for any $x : BG$, we define the type family $T : (x \xrightarrow{=} x) \to \mathcal{U}$ by

$$T(q) :\equiv \prod_{p : \mathrm{sh}_G \xrightarrow{=} x} (pg = qp).$$

And we claim that $\sum_{q:x \xrightarrow{=} x} T(q)$ is contractible for any $x : BG$. Because this is a proposition, one only need to check that it holds on one point of the connected type BG, say $x \equiv \mathrm{sh}_G$. We consider the following composition of equivalences:

$$\begin{aligned}
\sum_{q:UG} T(q) &\equiv \sum_{q:UG} \prod_{p:UG} (pg = qp) \\
&\xrightarrow{\simeq} \sum_{q:UG} \prod_{p:UG} (g = q) \\
&\xrightarrow{\simeq} \sum_{q:UG} UG \to (g = q) \\
&\xrightarrow{\simeq} \sum_{q:UG} \|UG\| \to (g = q) \\
&\xrightarrow{\simeq} \sum_{q:UG} (g = q) \\
&\xrightarrow{\simeq} 1
\end{aligned}$$

In that composition, the first equivalence is using that g commutes with every other element $p : UG$, so that $pgp^{-1} = g$. The second equivalence acknowledges the fact that the codomain $(g = q)$ does not depend on p anymore, so that the dependent function type inside the sum is a simple function type. The third equivalence uses the universal property of propositional truncation under the sum. The fourth equivalence is the evaluation at $|\mathrm{refl}_{\mathrm{sh}_G}|$ under the sum. The last equivalence is the contractibility of singleton types.

We have just shown that for all $x : BG$, the type $\sum_{q:x \xrightarrow{=} x} T(q)$ is contractible. We define now $\hat{g}(x) : x \xrightarrow{=} x$ as the chosen center of contraction of that type. More precisely, by connectedness of BG, the inverse φ^{-1} of the exhibited equivalence $\varphi : \sum_{q:UG} T(q) \xrightarrow{\simeq} 1$ produces a dependent function of type $\prod_{x:BG} 1 \xrightarrow{\simeq} \sum_{q:x \xrightarrow{=} x} T(q)$, and $\hat{g}$ is the pointwise evaluation at the unique element triv of 1. In particular, $\hat{g}(\mathrm{sh}_G) = \varphi^{-1}(\mathrm{triv}) = g$ as wanted. □

Together, Corollary 12.2.4 and Lemma 12.2.5 show that $\mathrm{abs}(\mathrm{z}_G)$ establishes an equivalence

$$\mathrm{U}\,\mathrm{Z}(G) \xrightarrow{\simeq} \sum_{g:UG} \prod_{h:UG} gh = hg \tag{12.2.1}$$

In yet other words, $\mathrm{B}\,\mathrm{Z}(G) :\equiv (BG_{\div} \xrightarrow{=} BG_{\div})_{(\mathrm{refl}_{BG_{\div}})}$ is (equivalent to) the classifying type of a group whose abstract group is the "abstract center" of $\mathrm{abs}(G)$.

The following lemma is then immediate:

Lemma 12.2.6. *A group G is* abelian *if and only if z_G is an isomorphism of groups.*

def:abelian-groups

Remark 12.2.7. In the style of this book, we could have used Lemma 12.2.6 directly as the definition of abelian groups. However, the definition of z_G would have been too intricate to give properly as early as Definition 4.2.31. ⌟

12.2.8 *Universal set bundle and simple connectedness*

sec:univ-cover-simple

Let us say that a pointed type (A, a) is *simply connected* when both A and $a \xrightarrow{=} a$ are connected types.

The definition of the universal set bundle is reminiscent of the notion of connected component: instead of selecting elements that are merely equal to a fixed element a, the universal set bundle selects elements together with mere witnesses of the equality with a.

Definition 12.2.9. Let A be a type and $a : A$ an element. The *universal set bundle* of A at a is the type

$$A_{(a)}\langle 1\rangle :\equiv \sum_{x:A} \|a \xrightarrow{=} x\|_0$$

together with the first projection $A_{(a)}\langle 1\rangle \to A$.[1] ⌟

[1] The number 1 indicates that $A_{(a)}\langle 1\rangle$ is the universal 1-connected cover of A.

When needed, we will consider $A_{(a)}\langle 1\rangle$ as a pointed type, with distinguished point $(a, |\mathrm{refl}_a|_0)$. Note that when A is a groupoid, then the set truncation is redundant and the universal set bundle of A at a is then the singleton at a. In particular, groupoids have contractible universal set bundles.

The identity types in $A_{(a)}\langle 1\rangle$ can be understood easily once we introduce the following function for elements $x, y, z : A$:

$$_ \cdot _ : \|y \xrightarrow{=} z\|_0 \times \|x \xrightarrow{=} y\|_0 \to \|x \xrightarrow{=} z\|_0.$$

It is defined as follows: given $\chi : \|y \xrightarrow{=} z\|_0$, we want to define $\chi \cdot _$ in the set $\|x \xrightarrow{=} y\|_0 \to \|x \xrightarrow{=} z\|_0$, hence we can suppose $\chi \equiv |q|_0$ for some $q : y \xrightarrow{=} z$; now given $\pi : \|x \xrightarrow{=} y\|_0$, one want to define $|q|_0 \cdot \pi$ in the set $\|x \xrightarrow{=} z\|_0$, hence one can suppose $\pi \equiv |p|_0$ for some $p : x \xrightarrow{=} y$; finally, we define

$$|q|_0 \cdot |p|_0 :\equiv |q \cdot p|_0.$$

Then one proves, by induction on $p : x \xrightarrow{=} y$, that $\mathrm{trp}_p^{\|a \xrightarrow{=} _\|_0}$ is equal to the function $\alpha \mapsto |p|_0 \cdot \alpha$. In particular, there exists an equivalence from the type of path between two points (x, α) and (y, β) of the universal set bundle $A_{(a)}\langle 1\rangle$ to sum type, analagous to the identification of paths in sum types:

$$\big((x, \alpha) \xrightarrow{=} (y, \beta)\big) \xrightarrow{\simeq} \sum_{p : x \xrightarrow{=} y} |p|_0 \cdot \alpha = \beta. \tag{12.2.2}$$

This description allows us to prove the following lemma.

Lemma 12.2.10. *Let A be a type and $a : A$ an element. The universal set bundle $A_{(a)}\langle 1\rangle$ is simply connected.*

Proof. First, we prove that $A_{(a)}\langle 1\rangle$ is connected. It has a point $(a, |\mathrm{refl}_a|_0)$ and, for every $(x, \alpha) : A_{(a)}\langle 1\rangle$, one wants $\|(a, |\mathrm{refl}_a|_0) \xrightarrow{=} (x, \alpha)\|$. This is proposition, hence a set, so that one can suppose $\alpha \equiv |p|_0$ for a path $p : a \xrightarrow{=} x$. Now, the proposition $|p|_0 \cdot |\mathrm{refl}_a|_0 = |p|_0$ holds. So one can use the inverse of the equivalence of Equation (12.2.2) to produce a path $(a, |\mathrm{refl}_a|_0) \to (x, \alpha)$.

Next, we prove that $(a, |\mathrm{refl}_a|_0) \xrightarrow{=} (a, |\mathrm{refl}_a|_0)$ is connected. One uses again the equivalence of Equation (12.2.2) to produce a composition of equivalences:

$$\begin{aligned}((a, |\mathrm{refl}_a|_0) \xrightarrow{=} (a, |\mathrm{refl}_a|_0)) &\xrightarrow{\simeq} \sum_{p : a = a} \big(|p|_0 = |\mathrm{refl}_a|_0\big) \\ &\xrightarrow{\simeq} \sum_{p : a = a} \big(\|p \xrightarrow{=} \mathrm{refl}_a\|\big)\end{aligned}$$

In other words, $(a, |\mathrm{refl}_a|_0) \xrightarrow{=} (a, |\mathrm{refl}_a|_0)$ is equivalent to the connected component of refl_a in $a \xrightarrow{=} a$. In particular, it is connected. □

Lemma 12.2.11. *Let A be a type pointed at $a : A$. The projection* $\mathrm{fst} : A_{(a)}\langle 1\rangle \to_* A$ *is a universal set bundle in the sense of Definition 3.3.10.*

eq:id-types-universal-cover
lemma:universal-cover-simply-connected
lemma:universal-cover-is-universal

Proof. Let $f : B \to_* A$ be a pointed set bundle. We need to show that the type of pointed functions $\varphi : A_{(a)}\langle 1\rangle \to_* B$ together with an identification $q : \mathrm{fst} \xrightarrow{=} f\varphi$ is contractible. However, such a φ is uniquely determined by the family of functions $\varphi_x : \|a \xrightarrow{=} x\|_0 \to f^{-1}(x)$ for $x : A$. For each $x : A$, $f^{-1}(x)$ is a set, so φ_x is uniquely determined by $\varphi_x \circ |_|_0 : a \xrightarrow{=} x \to f^{-1}(x)$. By induction on $p : a \xrightarrow{=} x$, we prove that $\varphi_x(|p|_0) = \mathrm{trp}_p^{f^{-1}}(b, f_0)$ where b is the element pointing B and f_0 the path pointing f. Indeed, for $p \equiv \mathrm{refl}_a$, we get $\varphi_a(|\mathrm{refl}_a|_0) \equiv (\varphi(|\mathrm{refl}_a|_0), q_{|\mathrm{refl}_a|_0}) = (b, f_0)$ because q is an identification $\mathrm{fst} \xrightarrow{=} f\varphi$ of pointed functions. □

12.2.12 *Abelian groups and simply connected 2-types*

sec:abel-groups-simply

We will now give an alternative characterization of the type of abelian groups, more in line with the geometrical intuition we are trying to build in this chapter. Recall that a type A is called a 2-*truncated type*, or 2-*type* for short, when the identity type $x \xrightarrow{=} y$ is a groupoid for every $x, y : A$.

thm:abelian-groups-weq-sc2types

Theorem 12.2.13. *The type* AbGroup *of abelian groups is equivalent to the type of pointed simply connected* 2*-types. Give equivalences here.*

Proof. Define the map $\mathrm{B}^2 : \mathrm{AbGroup} \to \mathcal{U}_*$ by $\mathrm{B}^2 G :\equiv \mathcal{U}_{(BG_\div)}\langle 1\rangle$.[2] Proving that $\mathrm{B}^2 G$ is a 2-type is equivalent to proving the proposition $\mathrm{isSet}(p \xrightarrow{=} q)$ for all $p, q : x \xrightarrow{=} y$ and all $x, y : \mathrm{B}^2 G$. One can then use connectedness of $\mathrm{B}^2 G$ and restrict to only show that $p \xrightarrow{=} q$ is a set for all path $p, q : (BG_\div, |\mathrm{id}_{BG_\div}|_0) \xrightarrow{=} (\mathrm{B}G_\div, |\mathrm{id}_{BG_\div}|_0)$. Recall that there is a canonical equivalence of type:

eq:

(12.2.3)

$$((BG_\div, |\mathrm{id}_{BG_\div}|_0) \xrightarrow{=} (BG_\div, |\mathrm{id}_{BG_\div}|_0)) \xrightarrow{\simeq} \sum_{r : BG_\div \xrightarrow{=} BG_\div} \mathrm{trp}_r(|\mathrm{id}_{BG_\div}|_0) \xrightarrow{=} |\mathrm{id}_{BG_\div}|_0$$

Under that equivalence, p and q can be rewritten as $(p_0, !)$ and $(q_0, !)$ with $p_0, q_0 : BG_\div \xrightarrow{=} BG_\div$ and the elements ! are proofs of the proposition $\mathrm{trp}_{p_0}(|\mathrm{id}_{BG_\div}|_0) = |\mathrm{id}_{BG_\div}|_0$ and $\mathrm{trp}_{q_0}(|\mathrm{id}_{BG_\div}|_0) = |\mathrm{id}_{BG_\div}|_0$ respectively. As a consequence, the proposition $\mathrm{isSet}(p \xrightarrow{=} q)$ is equivalent to the proposition $\mathrm{isSet}(p_0 \xrightarrow{=} q_0)$. As part of the definition of the group G, the type $BG_\div$ is a 1-type, hence $BG_\div \xrightarrow{=} BG_\div$ is also a 1-type through univalence. This means that $\mathrm{isSet}(p_0 \xrightarrow{=} q_0)$ holds.

So one gets a map, denoted again B^2 abusively,

$$\mathrm{B}^2 : \mathrm{AbGroup} \to \mathcal{U}_*^{=2}$$

where the codomain $\mathcal{U}_*^{=2}$ is the type of pointed simply connected 2-types, that is

$$\mathcal{U}_*^{=2} :\equiv \sum_{(A,a) : \mathcal{U}_*} \left(\mathrm{isConn}(A) \times \mathrm{isConn}(a \xrightarrow{=} a) \times \mathrm{isGrpd}(a \xrightarrow{=} a)\right)$$

We shall now provide an inverse for this map. Given a pointed simply connected 2-type (A, a), one can construct a group, denoted $\mathrm{Aut}^2(A, a)$, with classifying type:

$$\mathrm{BAut}^2(A, a) :\equiv (a \xrightarrow{=} a, \mathrm{refl}_a).$$

Indeed, this pointed type is connected because (A, a) is simply connected, and it is a 1-type because A is a 2-type. Moreover, $\mathrm{Aut}^2(A, a)$ is abelian.

[2] This is slightly misleading: If G is an abelian group in universe $\mathcal{U}$, then this definition makes $\mathrm{B}^2 G$ a pointed type in a successor universe, which is not what we want. The solution is to note that $\mathrm{B}^2 G$ is a locally $\mathcal{U}$-small type, which as a connected type is the image of the base point map $\mathrm{pt} : \mathbb{1} \to \mathrm{B}^2 G$, so it's an essentially $\mathcal{U}$-small type by the Replacement **??** 2.19.4. So really, $\mathrm{B}^2 G$ should be the $\mathcal{U}$-small type equivalent to $\mathcal{U}_{(BG_\div)}\langle 1\rangle$.

MB: It seems that a large part of the proof doesn't use that G is abelian. Remark?

To see it, let us use the bare definition of abelian groups (cf. Definition 4.2.31). We shall then prove that for all elements $g, h : \mathrm{refl}_a \xrightarrow{=} \mathrm{refl}_a$, the proposition $gh = hg$ holds. This property holds in even more generality and is usually called "Eckmann-Hilton's argument". It goes as follows: for $x, y, z : A$, for $p, q : x \xrightarrow{=} y$ and $r, s : y \xrightarrow{=} z$ and for $g : p \xrightarrow{=} q$ and $h : r \xrightarrow{=} s$, one prove

eq:horizontal-comp

$$\mathrm{ap}_{_\cdot q}(h) \cdot \mathrm{ap}_{r\cdot_}(g) = \mathrm{ap}_{s\cdot_}(g) \cdot \mathrm{ap}_{_\cdot p}(h). \tag{12.2.4}$$

This equality takes place in $r \cdot p \xrightarrow{=} s \cdot q$ and is better represented by the diagram in Figure 12.2. One prove such a result by induction on

fig:horizontal-comp

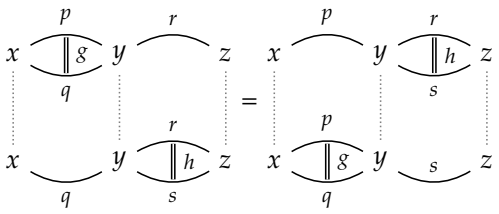


FIGURE 12.2: Visual representation of Equation (12.2.4). The vertical dotted lines denotes composition.

h. Indeed, when $h \equiv \mathrm{refl}_r$, then both sides of the equation reduces through path algebra to $\mathrm{ap}_{r\cdot_}(g)$. Now we are interested in this result when x, y, z are all equal to a by definition, and p, q, r, s are all equal to refl_a by definition. In that case, one has that $\mathrm{ap}_{\mathrm{refl}_a\cdot_}$ and $\mathrm{ap}_{_\cdot\mathrm{refl}_a}$ both act trivially, and the equation becomes: $h \cdot g = g \cdot h$.

One still has to prove that the function Aut^2 is an inverse for B^2. Given an abelian group G, the proof of Lemma 12.2.10 gives an equivalence between $\mathrm{BAut}^2(\mathrm{B}^2 G)$ and the connected component of $\mathrm{refl}_{BG_\div}$ in $BG_\div \xrightarrow{=} BG_\div$. By definition, this is the classifying type of $\mathrm{Z}(G)$. Being abelian, G is isomorphic to its center (Lemma 12.2.6), and so it yields an element of $\mathrm{Aut}^2(\mathrm{B}^2 G) \xrightarrow{=}_{\mathrm{Group}} G$. Conversely, take a pointed simply connected 2-type (A, a). We want to produce a pointed equivalence $\Phi : (A, a) \xrightarrow{\simeq} \mathrm{B}^2(\mathrm{Aut}^2(A, a))$. One should first notice that the function

If $X \xrightarrow{\simeq}_* Y$ denote the type of pointed equivalences between pointed types $X, Y : \mathcal{U}_*$, then the univalence axiom implies that there is an equivalence

$$(X = Y) \simeq (X \xrightarrow{\simeq}_* Y).$$

eq:loopspace-A-abelian

$$\mathrm{ev}^a_{\mathrm{refl}_a}\, \mathrm{BAut}^2\left(\mathrm{B}^2(\mathrm{Aut}^2(A, a))\right) \equiv ((a \xrightarrow{=} a) \xrightarrow{=} (a \xrightarrow{=} a))_{(\mathrm{refl}_{a \xrightarrow{=} a})} \to (a \xrightarrow{=} a, \mathrm{refl}_a). \tag{12.2.5}$$

that maps a path

$$(p, !) : (a \xrightarrow{=} a, |\mathrm{refl}_{a \xrightarrow{=} a}|_0) \xrightarrow{=} (a \xrightarrow{=} a, |\mathrm{refl}_{a \xrightarrow{=} a}|_0)$$

to the evaluation $p(\mathrm{refl}_a) : a \xrightarrow{=} a$ is an equivalence, because $\mathrm{Aut}^2(A, a)$ is an abelian group.

We will now define a pointed map $\Phi : (A, a) \to_* \mathrm{B}^2(\mathrm{Aut}^2(A, a))$, and prove subsequently that this is an equivalence. Let $T : A \to \mathcal{U}$ be the type family (of sets) define by

$$T(a') :\equiv \sum_{\alpha : \|(a \xrightarrow{=} a) \simeq (a \xrightarrow{=} a')\|_0} \prod_{p : a \xrightarrow{=} a'} \alpha = |p \cdot _|_0$$

We claim that $T(a')$ is contractible for all $a' : A$. By connectedness of A, it

is equivalent to show that $T(a)$ is contractible. However,

$$\begin{aligned} T(a) &\equiv \sum_{\alpha : \|(a =\to a) \simeq (a =\to a)\|_0} \prod_{p : a =\to a} \alpha = |p \cdot _|_0 \\ &\simeq \sum_{\alpha : \|(a =\to a) \simeq (a =\to a)\|_0} \alpha = |\mathrm{id}_{a=a}|_0 \\ &\simeq 1 \end{aligned}$$

Then, we define $\Phi(a')$ to be the element $(a =\to a', \kappa_{a'}) : \mathcal{U}_{(a =\to a)}\langle 1 \rangle$ where $\kappa_{a'}$ is the first projection of the center of contraction of $T(a')$. In particular, following the chain of equivalences above, $\Phi(a)$ is defined as $(a =\to a, |\mathrm{refl}_{a =\to a}|_0)$, hence $\Phi(a)$ is trivially pointed by a reflexivity path. To verify that Φ, thus defined, is an equivalence, one can use connectedness of $\mathrm{B}^2(\mathrm{Aut}^2(A, a))$ and only check that $\Phi^{-1}(a =\to a, |\mathrm{refl}_{a =\to a}|_0)$ is contractible. However, there is a canonical equivalence of type:

$$\Phi^{-1}(a =\to a, |\mathrm{refl}_{a =\to a}|_0) \simeq \sum_{a' : A} \sum_{\varphi : (a =\to a) \simeq (a =\to a')} |\varphi|_0 = \kappa_{a'}.$$

So we will show that the type on the right hand-side is contractible. For an element $a' : A$ together with $\varphi : (a =\to a) \simeq (a =\to a')$ such that the proposition $|\varphi|_0 = \kappa_{a'}$ holds, a path between $(a, \mathrm{id}_{a =\to a}, !)$ and $(a', \varphi, !)$ consists of a path $p : a =\to a'$ and a path $q : (x \mapsto px) =\to \varphi$. We have a good candidate for p, namely $p :\equiv \varphi(\mathrm{refl}_a) : a =\to a'$. However we don't have quite q yet. Consider, for any $a' : A$, the function

$$\mathrm{ev}^{a'}_{\mathrm{refl}_a} : ((a =\to a, |\mathrm{refl}_{a =\to a}|_0) = (a =\to a', \kappa_{a'})) \to (a =\to a')$$

defined as $(\psi, !) \mapsto \psi(\mathrm{refl}_a)$. Note that $\mathrm{ev}^{a}_{\mathrm{refl}_a}$ is precisely the equivalence $\mathrm{BAut}^2(\mathrm{B}^2\mathrm{Aut}^2(A, a))_\div \simeq (a = a)$ described in Equation (12.2.5). Hence, by connectedness of A, one gets that the proposition $\mathrm{isEquiv}(\mathrm{ev}^{a'}_{\mathrm{refl}_a})$ holds for all $a' : A$. In particular, because the propositions $|\varphi|_0 = \kappa_{a'}$ and $|p \cdot _|_0 = \kappa_{a'}$ holds, one gets elements $(\varphi, !)$ and $(x \mapsto px, !)$ in the domain of $\mathrm{ev}^{a'}_{\mathrm{refl}_a}$. Their images $\mathrm{ev}^{a'}_{\mathrm{refl}_a}(\varphi, !)$ and $\mathrm{ev}^{a'}_{\mathrm{refl}_a}(x \mapsto px, !)$ are both identifiable with p. By composition, we obtain a path $(x \mapsto px, !) =\to (\varphi, !)$ in the domain. The first component provide the path $q : (x \mapsto px) =\to \varphi$ that we wanted. □

12.2.14 *Higher deloopings*

The function B^2 defined in the proof of Theorem 12.2.13 provides a delooping of BG whenever G is abelian. That is, there is an identification $\Omega\mathrm{B}^2 G =\to BG$. A systematic way of obtaining such deloopings has been developed by David Wärn[3], that can be applied here to give an alternative definition of $\mathrm{B}^2 G$, and to obtain further deloopings of it.

[3] David Wärn. *Eilenberg-MacLane spaces and stabilisation in homotopy type theory*. 2023. arXiv: 2301.03685 [math.AT].

Definition 12.2.15 (Wärn). Given a pointed type X, the type of *X-torsors* is

$$TX :\equiv \sum_{Y : \mathcal{U}} \|Y\| \times \left(\prod_{y : Y} (Y, y) \simeq_* X \right).$$

The type of *pointed X-torsors* is $TX_* :\equiv \sum_{t : TX} \mathrm{fst}\, t$. ⌟

The usefulness of these definitions in the context of deloopings comes from the following proposition.

LEMMA 12.2.16 (Wärn). *Let X be a pointed type. If TX_* is contractible, then for any pointed X-torsors (t, y), the pointed type (TX, t) is a delooping of X.*

Proof. Suppose (t, y) is a center of contraction for TX_*. By contracting away (Lemma 2.9.10) in two different ways, we obtained a composition of equivalences:

$$(t =\!\!\to t) \xrightarrow{\simeq} \sum_{u : TX} \operatorname{fst} u \times (t =\!\!\to u) \xrightarrow{\simeq} \operatorname{fst} t$$

that maps refl_t to y. In other words, this equivalence, trivially pointed, presents (TX, t) as a delooping of $(\operatorname{fst} t, y)$. Moreover, the X-torsor t comes by definition with an identification $(\operatorname{fst} t, y) \xrightarrow{\simeq}_* X$. So in the end, we have an equivalence $(TX, t) \xrightarrow{\simeq}_* X$. □

EXERCISE 12.2.17. Recall that a *section* (see Definition 2.17.1 and its accompanying footnote) of a function $f : A \to B$ is a function $s : B \to A$ together with an identification $f \circ s =\!\!\to \mathrm{id}_B$. Construct an equivalence from the type $\operatorname{sec} f$ of sections of f to the type $\prod_{b : B} \sum_{a : A} b =\!\!\to f(a)$. ⌟

xca:sections-as-dependent-functions

Consider the evaluation function $\mathrm{ev}_{X_+, Y} : (X_+ =\!\!\to Y) \to Y$ (defined by path-induction, sending refl_X to the distinguished point of X). In other words, the function $\mathrm{ev}_{X_+, Y}$ takes an identification of X_+ with Y and returns the point in Y corresponding to the distinguished point of X under this identification. Applying Exercise 12.2.17 to $\mathrm{ev}_{X_+, Y}$ we get an equivalence of type

$$TX \xrightarrow{\simeq} \sum_{Y : \mathcal{U}} \|Y\| \times \operatorname{sec}(\mathrm{ev}_{X_+, Y}).$$

This alternative description of the type of X-torsors is the key ingredient to compare Wärn's delooping of the classifying type of an abelian group with our.

LEMMA 12.2.18. *For any abelian group G, the type $T(BG)$ can be identified with $\mathrm{B}^2 G$.*

lem:warn-abelian-group

Proof. Let G be an abelian group. We first construct, for each type Y, a function $f_Y : \|Y\| \times \operatorname{sec}(\mathrm{ev}_{BG_+, Y}) \to \|BG_+ =\!\!\to Y\|_0$, and then prove that f_Y is an equivalence. Given a type Y and an element $(!, s) : \|Y\| \times \operatorname{sec}(\mathrm{ev}_{X_+, Y})$, we can easily prove that Y is connected: being connected is a proposition, so we can assume that we have an actual $y : Y$ and then $s(y) : BG_+ =\!\!\to Y$ proves that Y is as connected as BG_+ is. Consequently s must send Y into one of the connected component of $BG_+ =\!\!\to Y$, that we choose to be $f_Y(!, s)$. With this definition, the fiber of f_Y at any given $c : \|BG_+ =\!\!\to Y\|_0$ can be identified with the type of sections s of $\mathrm{ev}_{BG_+, Y}$ with values in c. However, for any Z and $p : BG_+ =\!\!\to Z$ the restriction of the evaluation $\mathrm{ev}_{BG_+, Z} \restriction_p : (BG_+ =\!\!\to Z)_{(p)} \to Z$ is an equivalence: indeed, by induction, we only have to show it for $p \equiv \operatorname{refl}_{BG_+}$, in which case $\mathrm{ev}_{BG_+, BG_+} \restriction_{\operatorname{refl}_{BG_+}}$ is exactly the map Bz_G defined in Section 12.2.1, which is an equivalence since G is abelian by Lemma 12.2.6. Thus, given any p, the fiber of f_Y at $|p|_0$ is contractible. Being contractible is a proposition, hence a set, so it follows that the fiber of f_Y at any $c : \|BG_+ =\!\!\to Y\|_0$ is contractible. In other words, f_Y is an equivalence, as announced. We have thus a chain of equivalences:[4].

[4] Notice that the construction of an equivalence $TX \xrightarrow{\simeq} \mathcal{U}_{(X_+)}\langle 1\rangle$ that we carried for $X \equiv BG$ relies only on X_+ being connected and $\mathrm{ev}_{X_+, X_+} \restriction_{\operatorname{refl}_{X_+}}$ being an equivalence. Such types X are called *central* and are studied in details by Buchholtz et al.[5].

[5] Ulrik Buchholtz et al. "Central H-spaces and banded types". 2023. arXiv: 2301.02636.

$$T(BG) \xrightarrow{\simeq} \sum_{Y:\mathcal{U}} \|Y\| \times \mathrm{sec}(\mathrm{ev}_{X_\div, Y}) \xrightarrow{\simeq} \sum_{Y:\mathcal{U}} \|BG_\div \xrightarrow{=} Y\|_0 \equiv \mathrm{B}^2 G$$

□

Notice that where Wärn's method shines, compared to our, is in producing further delooping $\mathrm{B}^n G$ for $n \geq 3$.

12.3 *Homomorphisms of abelian groups*

sec:ab-hom

For concrete groups G and H, we have defined the set $\mathrm{Hom}(G, H)$ of group homomorphisms from G to H as the set $BG \to_* BH$. When the codomain H is abelian, it turns out that $\mathrm{Hom}(G, H)$ is an abstract abelian group. In this section, we define a concrete abelian group $\underline{\mathrm{Hom}}(G, H)$ with an identification $\mathrm{Hom}(G, H) = \mathrm{U}\underline{\mathrm{Hom}}(G, H)$.

Given types X, Y and a point $y : Y$, recall that cst_y is the function that maps all $x : X$ to y. If X and Y are pointed, then the function $\mathrm{cst}_{\mathrm{pt}_Y}$ is trivially pointed by $\mathrm{refl}_{\mathrm{pt}_Y}$.

Definition 12.3.1. Let G and H be concrete groups. Assume H is abelian. Define

$$\underline{\mathrm{Hom}}(G, H) :\equiv \mathrm{Aut}_{BG \to_* \mathrm{B}^2 H}(\mathrm{cst}_{\mathrm{pt}_{\mathrm{B}^2 H}})$$

⌟

Notice that, since $\mathrm{B}^2 H$ is a 2-type, the type $BG \to_* \mathrm{B}^2 H$ a is 1-type and thus $\underline{\mathrm{Hom}}(G, H)$ is well-defined.

Construction 12.3.2. *Let G and H be abelian groups. There is a bijection of sets of type*

$$\mathrm{U}\underline{\mathrm{Hom}}(G, H) \xrightarrow{\simeq} \mathrm{Hom}^{\mathrm{abs}}(G, H).$$

Proof. We simply follow the chain of equivalences:

$$\begin{aligned}
\mathrm{U}\underline{\mathrm{Hom}}(G, H) &:\equiv \mathrm{cst}_{\mathrm{pt}_{\mathrm{B}^2 H}} \xrightarrow{=} \mathrm{cst}_{\mathrm{pt}_{\mathrm{B}^2 H}} \\
&\xrightarrow{\simeq} \sum_{e \,:\, \prod_{x:BG} \mathrm{pt}_{\mathrm{B}^2 H} \xrightarrow{=} \mathrm{pt}_{\mathrm{B}^2 H}} \mathrm{refl}_{\mathrm{pt}_{\mathrm{B}^2 H}} = e(\mathrm{pt}_{BG}) \\
&\xrightarrow{\simeq} \sum_{e \,:\, BG \to BH} \mathrm{sh}_H = e(\mathrm{sh}_G) \\
&\xrightarrow{\simeq} \mathrm{Hom}(G, H) \\
&\xrightarrow{\simeq} \mathrm{Hom}^{\mathrm{abs}}(G, H)
\end{aligned}$$

□

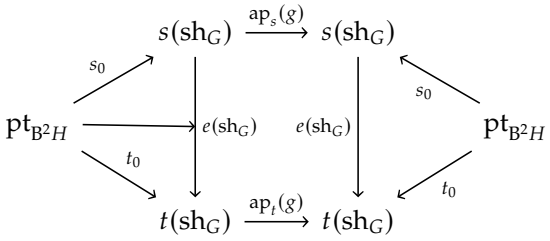
Lemma 12.3.3. *The previous equivalence is a homomorphism of abstract group from* $\mathrm{abs}\,\underline{\mathrm{Hom}}(G, H)$ *to the pointwise abstract group structure on* $\mathrm{Hom}^{\mathrm{abs}}(G, H)$.

Proof. We have to verify that any concatenation of symmetries in $\mathrm{U}\underline{\mathrm{Hom}}(G, H)$ is sent by the previous equivalence to the pointwise multiplication of $\mathrm{Hom}^{\mathrm{abs}}(G, H)$. Recall that the pointwise multiplication $\varphi \cdot_{\mathrm{ptw}} \psi : UG \to UH$ of $\varphi, \psi : \mathrm{Hom}^{\mathrm{abs}}(G, H)$ is defined as $\varphi \cdot_{\mathrm{ptw}} \psi(g) :\equiv \varphi(g) \cdot \psi(g)$.

We now generalize the goal. Consider two pointed functions $s, t : BG \to_* \mathrm{B}^2 H$, pointed by s_0 and t_0 repectively. Then consider the following composition:

$$(s \xrightarrow{=} t) \xrightarrow{\simeq} \sum_{e \,:\, \prod_{x:BG} s(x) \xrightarrow{=} t(x)} t_0 = e(\mathrm{pt}_{BG}) \cdot s_0$$
$$\xrightarrow{\Omega_g} \prod_{g \,:\, UG} \Omega(s)(g) \xrightarrow{=} \Omega(t)(g)$$

where $\Omega_g(e, e_0)(g) :\equiv e_0 \triangleleft \mathrm{apd}_e(g) \triangleright e_0$ is the pasting:



[TODO: defn of pasting triangles on both sides of a square]

By induction, we prove easily that the composition above identifies with ap_Ω. Since ap_Ω preserves concatenation, we only need to see that, when both s and t are $\mathrm{cst}_{\mathrm{pt}_{\mathrm{B}^2H}}$, the above composition identifies with the (forward direction of the) previous equivalence. This is immediate when noticing that Ω_g identifies to Ω by path algebra when both s and t are $\mathrm{cst}_{\mathrm{pt}_{\mathrm{B}^2H}}$. □

Lemma 12.3.4. *Let G and H be abelian groups. There is a bijection of sets of type*

$$\mathrm{U}\underline{\mathrm{Hom}}(G, H) \xrightarrow{\simeq} (\mathrm{B}^2 G \to_* \mathrm{B}^2 H).$$

12.4 *The category of abelian groups*

12.5 *Direct sums and reduced wreath products*

sec:ab-mon-closed-cat
sec:direct-sums

Sketch: We saw in Section 8.6 how to produce sums of groups, and noticed that a sum of abelian groups is rarely abelian. Indeed, the free group on two generators F_2 is the sum of two copies of $\mathbb{Z}$.

But a very similar construction works to produce sums of *abelian groups*.

Example 12.5.1 (Lamplighter group). $\mathrm{C}_2 \wr \mathbb{Z}$ Wait: how do we do infinite direct sums in general? ⌟

12.6 *Stabilization*

13
Rings, fields and vector spaces

ch:fields

In this chapter we will extend the hierarchy of algebraic structures from monoids (Definition 7.2.1) and groups (Definition 4.2.8) to rings (Definition 13.1.2), fields (**??**), and vector spaces (**??**). Of all these structures there are several varieties, satisfying additional properties, such as abelian groups (Section 12.2), non-trivial rings (**??**), commutative rings (**??**), ...

Quotients; subspaces (= ?). Bases and so. Dual space; orthogonality. (all of this depends on good implementations of subobjects). Eigen-stuff. Characteristic polynomials; Hamilton-Cayley.

13.1 *Rings, abstract and concrete*

sec:rings

A ring is an algebraic structure that consists of a group and a monoid that share the same underlying set. The interaction between the respective operations is governed by laws that are called the distributivity laws. The standard example of a (commutative) ring is the ring with set of integers as underlying set, with addition as group operation and multiplication as monoid operation. Note that multiplication in a ring need not be commutative.[1] We start by defining rings abstractly.

[1] In contrast, in Exercise 13.1.4 you are asked to prove that the group of a ring is always abelian, as a consequence of the extra structure and properties.

13.1.1 *Abstract rings*

sec:abstrings

We follow the convention that the group data of an abstract group are denoted by 0, $+$, $-$ and the monoid data by 1, $\cdot$.

def:abstractring

Definition 13.1.2. An *abstract ring* $\mathcal{R}$ consists of an abstract group $(R, 0, +, -)$ and a monoid $(R, 1, \cdot)$ with the same underlying set R. Moreover, the following equations should hold for all $a, b, c : R$:

ring:ldistr_law

(1) $a \cdot (b + c) = a \cdot b + a \cdot c$ (the *left distributive law*)

ring:rdistr_law

(2) $(a + b) \cdot c = a \cdot c + b \cdot c$ (the *right distributive law*)

The latter two properties are together denoted by $\mathrm{DistrLaws}(R, \cdot, +)$.

The abstract ring $\mathcal{R}$ is called *non-trivial* if $0 \neq 1$ and *commutative* if its multiplication $\cdot$ is commutative, that is, if $a \cdot b = b \cdot a$ for all $a, b : R$. ⌟

The abstract group $(R, 0, +, -)$ is called the (*additive*) *group* of $\mathcal{R}$, and the monoid $(R, 1, \cdot)$ the (*multiplicative*) *monoid* of $\mathcal{R}$.

def:typering

Definition 13.1.3. The type of abstract rings is defined as[2]

$$\mathrm{Ring} :\equiv \sum_{(R,0,+,-):\mathrm{Group}^{\mathrm{abs}}} \sum_{e:R} \sum_{\mu:R\to R\to R} \mathrm{MonoidLaws}(R, e, \mu) \times \mathrm{DistrLaws}(R, \mu, +).$$

[2] See Section 7.2 for the monoid laws.

The type CRing of commutative rings is similar to the type of rings with the additional property $\prod_{a,b:R} \mu(a,b) = \mu(b,a)$. ⌟

xca:ring-group-abelian

EXERCISE 13.1.4. Let $\mathcal{R}$ be an abstract ring. Show that the additive group of $\mathcal{R}$ is abelian. Hint: elaborate $(a+1)\cdot(b+1)$. ⌟

def:ringhom

DEFINITION 13.1.5. Let $\mathcal{R}, \mathcal{S} : \mathrm{Ring}$ be abstract rings, with $\mathcal{R}$ consisting of an abstract group $\mathcal{R}$ with underlying set R and a monoid $(R, 1_R, \cdot_R)$, and $\mathcal{S}$ consisting of an abstract group $\mathcal{S}$ with underlying set S and a monoid $(S, 1_S, \cdot_S)$. An *abstract ring homomorphism* from $\mathcal{R}$ to $\mathcal{S}$ is an abstract homomorphism $f : \mathrm{Hom}^{\mathrm{abs}}(\mathcal{R}, \mathcal{S})$ that is a monoid homomorphism from $(R, 1_R, \cdot_R)$ to $(S, 1_S, \cdot_S)$. ⌟

exa:ring-Z-polynomials

EXAMPLE 13.1.6. We elaborate the abstract ring of polynomials with integer coefficients. TBD ⌟

13.1.7 *Mixed rings*

sec:mixring

Here we explore a definition of a ring that is based on a concrete group G and left and right multiplications that are still half abstract.

We first note that, for any abstract ring $\mathcal{R}$ and elements $a, b : R$, the left multiplication function $(a \cdot _)$ and the right multiplication function $(_ \cdot b)$ are abstract homomorphisms of the additive group $(R, 0, +, -)$ of $\mathcal{R}$ to itself.[3] There are two ways to compose them: $(a \cdot (_ \cdot b))$ and $((a \cdot _) \cdot b)$. Equality of the latter two functions is an elegant way of expressing associativity. These observations lead to the following alternative definition of a ring.

[3] These functions provide two ways to write the product $a \cdot b$, see the coherence law in Definition 13.1.8(2).

def:mixring

DEFINITION 13.1.8. An *mixed ring* R consists of a group[4] also denoted R together with a symmetry $1_R : UR$ and two maps $\ell, r : UR \to \mathrm{Hom}(R, R)$ from the set of symmetries in R to the set of homomorphisms from R to R.[5] Given $g : UR$, we write ℓ_g for the homomorphism $\ell(g)$ and r_g for $r(g)$. Moreover, the following equations should hold.

[4] It will follow as in Exercise 13.1.4 that the group R is abelian.

[5] We call these rings "mixed" since they are based on a concrete group R and data referring to $\mathrm{abs}(R)$.

mixring:unit-laws

(1) $\ell_{1_R} = \mathrm{id}_G = r_{1_R}$ (the *multiplicative unit laws*)

mixring:lr-coherence-law

(2) $(\mathrm{U}\ell_g)(h) = (Ur_h)(g)$, for all $g, h : UR$ (the *coherence law*)

mixring:assoc-law

(3) $\ell \circ r = r \circ \ell$ (the *associativity law*)

The ring R is called *commutative* if $\ell = r$, and *non-trivial* if $1_R \neq \mathrm{refl}_R$. ⌟

The coherence law (2) allows us to abbreviate both $(\mathrm{U}\ell_g)(h)$ and $(Ur_h)(g)$ by $g \cdot h$. We will do this when no confusion can occur. Then, $\ell = r$ amounts to $g \cdot h = h \cdot g$, for all $g, h : UG$, as could be expected from the abstract case.

We proceed by giving the standard example of the integers as a ring in the sense of Definition 13.1.8.

EXAMPLE 13.1.9. Consider the group $\mathbb{Z}$ classified by the circle. Using the same notation $\mathbb{Z}$ also for the ring, take $1_{\mathbb{Z}} :\equiv \circlearrowleft$ and $\ell : (\bullet \xrightarrow{=} \bullet) \to \mathrm{Hom}(\mathbb{Z}, \mathbb{Z})$ defined as follows. For every $g : \bullet \xrightarrow{=} \bullet$, let ℓ_g be the homomorphism classified by the map $\mathrm{B}\ell_g(\bullet) :\equiv \bullet$, $\mathrm{B}\ell_g(\circlearrowleft) := g$, and pointed by reflexivity.[6] Take $r :\equiv \ell$. Now the unit laws, the coherence law and the associativity law can easily be verified. It follows that $(\mathbb{Z}, 1_{\mathbb{Z}}, \ell, !)$ is a non-trivial commutative ring. ⌟

[6] The reader may recognize the degree m map from Definition 3.6.5 as a special case.

def:typemixring

DEFINITION 13.1.10. The type of rings is defined as

$$\mathrm{Ring} :\equiv \sum_{R:\mathrm{Group}} \sum_{1_R:UR} \sum_{\ell,r:UR\to\mathrm{Hom}(R,R)} \mathrm{RingProperties}(R,1_R,\ell,r).$$

The type CRing of commutative rings is similar to the type of rings but with $\mathrm{RingProperties}(R,1_R,\ell,r)\times(\ell=r)$. ⌟

xca:RmixringtoURabstring

EXERCISE 13.1.11. Let $(R,1_r,\ell,r)$ be an mixed ring. Show that UR is an abstract ring with additive group $\mathrm{abs}(R)$ and multiplicative monoid $(UR,1_R,\cdot)$. TBD ⌟

13.1.12 *Move to a better place* (*Ch.* 11 *or* 2)

def:ptd-homotopy-compo

DEFINITION 13.1.13. Let X and Y be pointed types and $f,g:X\to_* Y$ pointed maps from X to Y. Recall from Construction 2.21.8 the equivalence ptw_* of type $(f \xrightarrow{=} g) \xrightarrow{\simeq} H(f,g)$, where

$$H(f,g) :\equiv \sum_{k:\prod_{x:X}(f_\div(x)\xrightarrow{=}g_\div(x))} ((k(\mathrm{pt}_X)\cdot f_{\mathrm{pt}}) \xrightarrow{=} g_{\mathrm{pt}}).$$

Assume also $h:X\to_* Y$ and let $k:H(f,g)$ and $k':H(g,h)$. In line with the notation for pointed maps, we denote the pair k by $(k_\div,k_{\mathrm{pt}})$, an likewise for k'. Define the *pointwise composition* $(k'\cdot_{\mathrm{ptw}}k)$ of k' and k by[7]

$$(k'\cdot_{\mathrm{ptw}}k) :\equiv (k'_\div\cdot_{\mathrm{ptw}}k_\div\ ,\ k'_{\mathrm{pt}}\cdot \mathrm{ap}_{(k'_\div(\mathrm{pt}_X)\cdot_)}(k_{\mathrm{pt}})), \quad \text{where}$$
$$(k'_\div\cdot_{\mathrm{ptw}}k_\div) :\equiv (x\mapsto k'_\div(x)\cdot k_\div(x)).$$

In Figure 13.1, the upper-right triangle represents the type of k_{pt}, the upper-left triangle is a reflexivity triangle, the lower triangle represents the type of k'_{pt}, and the outer diagram represents the type $k'(\mathrm{pt}_X)\cdot k(\mathrm{pt}_X)\cdot f_{\mathrm{pt}} \xrightarrow{=} h_{\mathrm{pt}}$ of $k'_{\mathrm{pt}}\cdot \mathrm{ap}_{(k'_\div(\mathrm{pt}_X)\cdot_)}(k_{\mathrm{pt}})$. Thus we see that $(k'\cdot_{\mathrm{ptw}}k)$ is an element of $H(f,h)$. ⌟

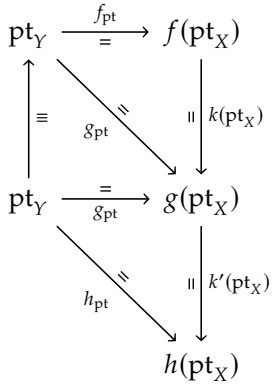


FIGURE 13.1: Path for $(k'\cdot_{\mathrm{ptw}}k)$.

[7] We use $k_\div$ to denote the first component of k, as we do for non-dependent pointed maps, but we will often drop this subscript "$\div$". We use the notation "$\cdot_{\mathrm{ptw}}$" for pointwise composition of k and k', as well as of $k_\div$ and $k'_\div$.

con:ptd-homotopy-compo

CONSTRUCTION 13.1.14. *Let conditions be as in Definition 13.1.13. Let $p:(f\xrightarrow{=}g)$ and $q:(g\xrightarrow{=}h)$. Then we have an identification of $\mathrm{ptw}_*(qp)$ with $\mathrm{ptw}_*(q)\cdot_{\mathrm{ptw}}\mathrm{ptw}_*(p)$.*

Implementation of Construction 13.1.14. By path induction on q, it suffices to construct an identification of $\mathrm{ptw}_*(p)$ and $\mathrm{ptw}_*(\mathrm{refl}_g)\cdot_{\mathrm{ptw}}\mathrm{ptw}_*(p)$. Using Construction 2.21.8 and **??** 2.9.18 we can identify $\mathrm{ptw}_*(\mathrm{refl}_g)$ with the pair $((x\mapsto \mathrm{refl}_{g_\div(x)}),\mathrm{refl}_{g_{\mathrm{pt}}})$. For use in Figure 13.1 we write the latter pair as $(k'_\div,k'_{\mathrm{pt}})$, noting that $h\equiv g$ in this case. Writing also $(k_\div,k_{\mathrm{pt}})$ for $\mathrm{ptw}_*(p)$, the goal is to identify $(k'\cdot_{\mathrm{ptw}}k)$ with k. This identification is easily obtained by using that $\mathrm{refl}_{g(x)}\cdot r$ is definitionally equal to r, for all $x:X$ and $r:f(x)\xrightarrow{=}g(x)$. □

def:cst-ptd

DEFINITION 13.1.15. Let A and B be pointed types. For any $b:B$ and $p:\mathrm{pt}_B\xrightarrow{=}b$, define the *pointed constant map* $\mathrm{cst}^A_*(b,p):A\to_* B$ *at* (b,p) by setting $\mathrm{cst}^A_*(b,p) :\equiv (\mathrm{cst}^A_b,p)$.[8] Thus cst^A_* is a function from $\sum_{x:B}(\mathrm{pt}_B\xrightarrow{=}x)$ to $A\to_* B$.[9] ⌟

rem:loops-at-ptd-cst

REMARK 13.1.16. In case f and g in Construction 2.21.8 are both the point of $X\to_* Y$, i.e., $f\equiv g\equiv \mathrm{pt}_{X\to_* Y}\equiv(\mathrm{cst}_{\mathrm{pt}_Y},\mathrm{refl}_{\mathrm{pt}_Y})$, it is convenient to work with a minor variant of ptw_* of type $\Omega(X\to_* Y)\xrightarrow{\simeq}(X\to_*\Omega Y)$. The latter type is obtained by definitional simplifications and replacing

[8] Here cst^A_b is from Definition 2.2.1. We may omit superscripts A if A is clear from the context.

[9] Of course, $\sum_{x:B}(\mathrm{pt}_B\xrightarrow{=}x)$ is contractible. In Definition 13.1.21 we will see why cst^A_* is useful.

[10] By laws of symmetry and right unit.

$(h(\mathrm{pt}_X) \cdot \mathrm{refl}_{\mathrm{pt}_Y}) \xrightarrow{=} \mathrm{refl}_{\mathrm{pt}_Y}$ in $H(\mathrm{pt}_{X\to_* Y}, \mathrm{pt}_{X\to_* Y})$ from Definition 13.1.13 by an equivalent type:[10]

$$H(\mathrm{pt}_{X\to_* Y}, \mathrm{pt}_{X\to_* Y}) \xrightarrow{\simeq} \left(\sum_{h : X\to \Omega Y} (\mathrm{refl}_{\mathrm{pt}_Y} \xrightarrow{=} h(\mathrm{pt}_X))\right) \equiv (X \to_* \Omega Y).$$

Abusing notations, we denote this variant also by ptw_*. ⌟

The following construction is useful since it will allow us to simplify identifying two pointed maps to identifying their underlying unpointed maps in some important cases. The construction is based on BCFR which in turn uses a result by Cavallo.

con:id-(B->*A)

Construction 13.1.17. *Let A be a pointed type and let* $\mathrm{ev} : (\mathrm{id}_A \xrightarrow{=} \mathrm{id}_A) \to (\mathrm{pt}_A \xrightarrow{=} \mathrm{pt}_A)$ *be the evaluation map that sends identifications* $i : (\mathrm{id}_A \xrightarrow{=} \mathrm{id}_A)$ *to paths* $\mathrm{ptw}(i)(\mathrm{pt}_A) : (\mathrm{pt}_A \xrightarrow{=} \mathrm{pt}_A)$. *Furthermore, let* $s : (\mathrm{pt}_A \xrightarrow{=} \mathrm{pt}_A) \to (\mathrm{id}_A \xrightarrow{=} \mathrm{id}_A)$ *be a section of* ev, *that is, we are given identifications* $\mathrm{ev}(s(p)) \xrightarrow{=} p$ *for all* $p : (\mathrm{pt}_A \xrightarrow{=} \mathrm{pt}_A)$. *Let also B be a pointed type and consider pointed maps* $f, f' : B \to_* A$ *with underlying unpointed maps* $f_\div, f'_\div : B \to A$.

Then we have a map $(f_\div \xrightarrow{=} f'_\div) \to (f \xrightarrow{=} f')$.

Implementation of Construction 13.1.17. By path induction on $f_\div \xrightarrow{=} f'_\div$ we may take $f_\div \equiv f'_\div$, so that the goal is to identify[11] (f, f_{pt}) with (f, f'_{pt}), for two paths $f_{\mathrm{pt}}, f'_{\mathrm{pt}} : (\mathrm{pt}_A \xrightarrow{=} f(\mathrm{pt}_B))$.

Define $r :\equiv (f'_{\mathrm{pt}} \cdot f_{\mathrm{pt}}^{-1}) : (f(\mathrm{pt}_B) \xrightarrow{=} f(\mathrm{pt}_B))$. By Construction 2.21.8, it suffices to give an element $h : \prod_{b:B}(f(b) \xrightarrow{=} f(b))$ and an identification of $h(\mathrm{pt}_B)$ with r. By path induction on f_{pt} we may take $\mathrm{pt}_A \equiv f(\mathrm{pt}_B)$, so that the domain of s is $f(\mathrm{pt}_B) \xrightarrow{=} f(\mathrm{pt}_B)$, and so r is an element of this domain. Now take $h(b) :\equiv \mathrm{ptw}(s(r))(f(b))$ for any $b : B$. Then indeed $h(b) : (f(b) \xrightarrow{=} f(b))$, and we can identify $h(\mathrm{pt}_B) \equiv \mathrm{ptw}(s(r))(f(\mathrm{pt}_B)) \equiv \mathrm{ev}(s(r))$ with r since s is a section of ev. □

[11] Henceforth we simply write f for $f_\div$.

con:ev-section-loopsA

Construction 13.1.18. *Let A be a pointed type and ΩA its pointed loop type. We use* pt *for* pt_A *and* refl *for* $\mathrm{refl}_{\mathrm{pt}_A}$. *Let* $\mathrm{ev} : (\mathrm{id}_{\Omega A} \xrightarrow{=} \mathrm{id}_{\Omega A}) \to (\mathrm{refl} \xrightarrow{=} \mathrm{refl})$ *be the evaluation map that sends* $i : (\mathrm{id}_{\Omega A} \xrightarrow{=} \mathrm{id}_{\Omega A})$ *to* $\mathrm{ptw}(i)(\mathrm{refl})$. *Then* ev *has a section, that is, a map* $s : (\mathrm{refl} \xrightarrow{=} \mathrm{refl}) \to (\mathrm{id}_{\Omega A} \xrightarrow{=} \mathrm{id}_{\Omega A})$ *with identifications* $\mathrm{ev}(s(\alpha)) \xrightarrow{=} \alpha$ *for all* $\alpha : (\mathrm{refl} \xrightarrow{=} \mathrm{refl})$.

Implementation of Construction 13.1.18. Recall from **??** 2.9.18 the equivalence ptw identifying $\mathrm{id}_{\Omega A} \xrightarrow{=} \mathrm{id}_{\Omega A}$ with $\prod_{p:\Omega A}(p \xrightarrow{=} p)$. Since $(\mathrm{refl} \cdot p)$ and p are definitionally equal for any $p : \Omega A$, any $\alpha : (\mathrm{refl} \xrightarrow{=} \mathrm{refl})$ gives a path $\mathrm{ap}_{_\cdot p}(\alpha) : (p \xrightarrow{=} p)$.[12] Taking refl for p, $\mathrm{ap}_{_\cdot \mathrm{refl}}(\alpha)$ can be identified with α.[13] For any $\alpha : (\mathrm{refl} \xrightarrow{=} \mathrm{refl})$ and $p : \Omega A$, define s_α by $s_\alpha(p) :\equiv \mathrm{ap}_{_\cdot p}(\alpha)$. Then $\mathrm{ptw}^{-1}(s_\alpha) : (\mathrm{id}_{\Omega A} \xrightarrow{=} \mathrm{id}_{\Omega A})$. Hence

$$s :\equiv (\alpha \mapsto \mathrm{ptw}^{-1}(s_\alpha)) : (\mathrm{refl} \xrightarrow{=} \mathrm{refl}) \to (\mathrm{id}_{\Omega A} \xrightarrow{=} \mathrm{id}_{\Omega A}),$$

and we have $\mathrm{ev}(s(\alpha)) \equiv \mathrm{ptw}(\mathrm{ptw}^{-1}(s_\alpha))(\mathrm{refl}) \xrightarrow{=} \alpha$ by **??** 2.9.18 and **??** 13.1.19. □

[12] In a picture:

$$\begin{array}{ccc} \mathrm{refl} & \xmapsto{_\cdot p} & (\mathrm{refl}\cdot p) \equiv p \\ {\scriptstyle\alpha}\downarrow & & \parallel \downarrow{\scriptstyle \mathrm{ap}_{_\cdot p}(\alpha)} \\ \mathrm{refl} & \xmapsto{_\cdot p} & (\mathrm{refl}\cdot p) \equiv p \end{array}$$

[13] As obvious as this may seem, it requires a generalization of the type of α to enable path induction, and we delegate this to **??** 13.1.19.

xca:ev-section-loopsA

Exercise 13.1.19. Given a type A with elements $a, x : A$ and a path $q : a \xrightarrow{=} x$, define $\rho_q : (q \cdot \mathrm{refl}_a) \xrightarrow{=} q$ by induction on q. For any $p : a \xrightarrow{=} a$ and $\beta : (p \xrightarrow{=} \mathrm{refl}_a)$, define $i(\beta) : \mathrm{ap}_{_\cdot \mathrm{refl}_a}(\beta) \xrightarrow{=} \beta \cdot \rho_p$ by induction on β. Now, give an identification of $\mathrm{ap}_{_\cdot \mathrm{refl}_a}(\alpha)$ and α for any $\alpha : (\mathrm{refl}_a \xrightarrow{=} \mathrm{refl}_a)$. ⌟

cor:1d-(B→*loops A)

Corollary 13.1.20. *The combination of Construction 13.1.17 and Construction 13.1.18 yields a function from* $(f_\div \xrightarrow{=} f'_\div)$ *to* $(f \xrightarrow{=} f')$ *for all pointed maps* $f, f' : B \to_* \Omega A$.

Recall Ω from Definition 4.2.10 and Definition 4.4.3, which together form a wild functor $\mathcal{U}_* \to \mathcal{U}_*$, cf. Section 6.4. In the following we will define a closely related wild functor that is sometimes easier to use.

def:O-functor

Definition 13.1.21. For any pointed type A, define $\mathrm{O}\,A :\equiv (\mathrm{S}^1 \to_* A)$. Let A and B be pointed types and let $f : A \to_* B$ be a pointed map. Define $\mathrm{O}(f) : \mathrm{O}\,A \to_* \mathrm{O}\,B$ to be composition with f, that is, for $g : (\mathrm{S}^1 \to_* A)$, $\mathrm{O}(f)(g) :\equiv (f \circ g) : (\mathrm{S}^1 \to_* B)$.[14] We point $\mathrm{O}(f)$ as follows. First, observe that $\mathrm{pt}_{\mathrm{O}\,B} \equiv \mathrm{pt}_{\mathrm{S}^1 \to_* B} \equiv (\mathrm{cst}_{\mathrm{pt}_B}, \mathrm{refl}_{\mathrm{pt}_B})$ and $\mathrm{O}(\mathrm{pt}_{\mathrm{O}\,A}) \equiv f \circ \mathrm{pt}_{\mathrm{O}\,A} \equiv (\mathrm{cst}_{f(\mathrm{pt}_A)}, f_{\mathrm{pt}})$. So both $\mathrm{pt}_{\mathrm{O}\,B}$ and $\mathrm{O}(\mathrm{pt}_{\mathrm{O}\,A})$ are images of $\mathrm{cst}_*^{\mathrm{S}^1}$, and we can obtain a path between them by applying $\mathrm{cst}_*^{\mathrm{S}^1}$ to the unique path $(f_{\mathrm{pt}}, \rho_{f_{\mathrm{pt}}})$ between $(\mathrm{pt}_B, \mathrm{refl}_{\mathrm{pt}_B})$ and $(f(\mathrm{pt}_A), f_{\mathrm{pt}})$ in the contractible type $\sum_{x:B}(\mathrm{pt}_B \xrightarrow{=} x)$.[15] The situation is illustrated in the diagram below and we define $\mathrm{O}(f)_{\mathrm{pt}} :\equiv \mathrm{ap}_{\mathrm{cst}_*}(f_{\mathrm{pt}}, \rho_{f_{\mathrm{pt}}})$. ⌟

$$\begin{array}{ccc} (\mathrm{pt}_B, \mathrm{refl}_{\mathrm{pt}_B}) & \overset{\mathrm{cst}_*}{\longmapsto} & (\mathrm{cst}_{\mathrm{pt}_B}, \mathrm{refl}_{\mathrm{pt}_B}) \equiv \mathrm{pt}_{\mathrm{O}\,B} \\ {\scriptstyle (f_{\mathrm{pt}}, \rho_{f_{\mathrm{pt}}})}\downarrow \| & & \| \downarrow {\scriptstyle \mathrm{ap}_{\mathrm{cst}_*}(f_{\mathrm{pt}}, \rho_{f_{\mathrm{pt}}})} \\ (f(\mathrm{pt}_A), f_{\mathrm{pt}}) & \overset{\mathrm{cst}_*}{\longmapsto} & (\mathrm{cst}_{f(\mathrm{pt}_A)}, f_{\mathrm{pt}}) \equiv \mathrm{O}(\mathrm{pt}_{\mathrm{O}\,A}) \end{array}$$

Note that $\mathrm{O}(f)_{\mathrm{pt}}$ is a reflexivity path if $f_{\mathrm{pt}} \equiv \mathrm{refl}_{\mathrm{pt}_B}$.

xca:O-functor

Exercise 13.1.22. Complete the structure of O as a wild functor, cf. Section 6.4. Identify $\mathrm{O}(f)_{\mathrm{pt}}$ with $\mathrm{ptw}_*^{-1}(\mathrm{cst}_{f_{\mathrm{pt}}}, \rho_{f_{\mathrm{pt}}})$. ⌟

rem:pointing-ev

Remark 13.1.23. Given a pointed type A, recall from Corollary 3.1.3 the equivalence ev_A from $\mathrm{S}^1 \to_* A$ to ΩA. This equivalence sends $f : \mathrm{S}^1 \to_* A$ to $\Omega(f)(\circlearrowleft) \equiv f_{\mathrm{pt}}^{-1} \cdot f(\circlearrowleft) \cdot f_{\mathrm{pt}}$, and the inverse ev_A^{-1} sends $p : \Omega A$ to the pointed map $f : \mathrm{S}^1 \to_* A$ defined by $f(\bullet) :\equiv \mathrm{pt}_A$ and $f(\circlearrowleft) := p$, pointed by reflexivity.[16]

The equivalence ev_A can itself be pointed as follows. The point of $\mathrm{S}^1 \to_* A$ is the constant map $\mathrm{cst}_{\mathrm{pt}_A}$, pointed by reflexivity, which is sent by ev_A to $\mathrm{cst}_{\mathrm{pt}_A}(\circlearrowleft) \cdot \mathrm{refl}_{\mathrm{pt}_A}$.[17] Define $\varepsilon_A(p) : \mathrm{refl}_{\mathrm{pt}_A} \xrightarrow{=} (\mathrm{cst}_{\mathrm{pt}_A}(p) \cdot \mathrm{refl}_{\mathrm{pt}_A})$, for any $z : \mathrm{S}^1$, $p : \bullet \xrightarrow{=} z$, by path induction, setting $\varepsilon_A(\mathrm{refl}_\bullet) :\equiv \mathrm{refl}_{\mathrm{refl}_{\mathrm{pt}_A}}$.[18] Now we define $(\mathrm{ev}_A)_{\mathrm{pt}} :\equiv \varepsilon_A(\circlearrowleft)$. ⌟

The following construction shows that Ω corresponds to O from Definition 13.1.21 under the equivalences ev, as illustrated in Figure 13.2.[19]

con:Omega-O

Construction 13.1.24. *Let A and B be pointed types and let $f : A \to_* B$ be a pointed map. Then we have an identification of $\Omega(f) \circ \mathrm{ev}_A$ and $\mathrm{ev}_B \circ \mathrm{O}(f)$, as represented by Figure 13.2. Consequently, we have that $e :\equiv (\mathrm{ev}_B^{-1} \circ _ \circ \mathrm{ev}_A)$ is an equivalence of type $(\Omega A \to_* \Omega B) \xrightarrow{\simeq} (\mathrm{O}\,A \to_* \mathrm{O}\,B)$, and $\mathrm{O} \xrightarrow{=} (e \circ \Omega)$.*

Implementation of Construction 13.1.24. We will apply Construction 2.21.8. Elaborating the situation in Figure 13.3, we have to identify (1) $\equiv f_{\mathrm{pt}}^{-1} \cdot f(p_{\mathrm{pt}}^{-1} \cdot p(\circlearrowleft) \cdot p_{\mathrm{pt}}) \cdot f_{\mathrm{pt}}$ and (2) $\equiv (f(p_{\mathrm{pt}}) \cdot f_{\mathrm{pt}})^{-1} \cdot (f \circ p)(\circlearrowleft) \cdot (f(p_{\mathrm{pt}}) \cdot f_{\mathrm{pt}})$ by a path $i(f, f_{\mathrm{pt}})(p, p_{\mathrm{pt}})$, for all (p, p_{pt}).[20] Moreover, we must fill the

fig:Omega-O

[14] Here we mean composition as pointed maps, so that the pointing path $\mathrm{O}(f)(g)_{\mathrm{pt}}$ is defined in Definition 2.21.1 as $f(g_{\mathrm{pt}}) \cdot f_{\mathrm{pt}}$.

[15] Here $\rho_{f_{\mathrm{pt}}} : f_{\mathrm{pt}} \cdot \mathrm{refl}_{\mathrm{pt}_B} \xrightarrow{=} f_{\mathrm{pt}}$ is defined by induction on f_{pt}, setting $\rho_{\mathrm{refl}_{\mathrm{pt}_B}} :\equiv \mathrm{refl}_{\mathrm{refl}_{\mathrm{pt}_B}}$.

[16] When A is clear from the context we may simply write ev. Similarly for ε_A defined next.

[17] Recall that reflexivity paths cancel definitionally on the left.

[18] Note that $\mathrm{cst}_{\mathrm{pt}_A}(p) : \Omega A$ for any p.

[19] ev_ is an example of a wild natural equivalence between the wild functors $\Omega, \mathrm{S}^1_ : \mathcal{U}_* \to_* \mathcal{U}_*$, cf. Chapter 6.

$$\begin{array}{ccc} \mathrm{O}\,A & \xrightarrow{\mathrm{O}(f)} & \mathrm{O}\,B \\ {\scriptstyle \mathrm{ev}_A}\downarrow \wr & & \wr \downarrow {\scriptstyle \mathrm{ev}_B} \\ \Omega A & \xrightarrow{\Omega(f)} & \Omega B \end{array}$$

Figure 13.2: $\Omega(f)$ and $\mathrm{O}(f)$ correspond.

[20] We often leave out the "ap_'s".

$$(\mathrm{S}^1 \to_* A) \xrightarrow{\mathrm{ev}_A} \Omega A \xrightarrow{\Omega(f)} \Omega B$$

$$(p, p_{\mathrm{pt}}) \longmapsto p_{\mathrm{pt}}^{-1} \cdot p(\circlearrowleft) \cdot p_{\mathrm{pt}} \longmapsto f_{\mathrm{pt}}^{-1} \cdot f(p_{\mathrm{pt}}^{-1} \cdot p(\circlearrowleft) \cdot p_{\mathrm{pt}}) \cdot f_{\mathrm{pt}} \quad (1)$$

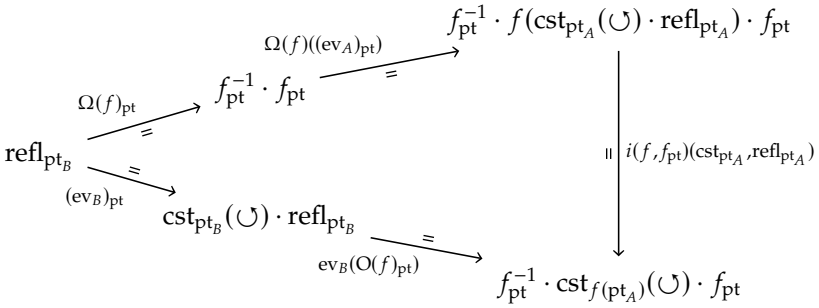
$$(\mathrm{S}^1 \to_* A) \xrightarrow{\mathrm{O}(f)} (\mathrm{S}^1 \to_* B) \xrightarrow{\mathrm{ev}_B} \Omega B$$

$$(p, p_{\mathrm{pt}}) \longmapsto (f \circ p, f(p_{\mathrm{pt}}) \cdot f_{\mathrm{pt}}) \longmapsto (f(p_{\mathrm{pt}}) \cdot f_{\mathrm{pt}})^{-1} \cdot (f \circ p)(\circlearrowleft) \cdot (f(p_{\mathrm{pt}}) \cdot f_{\mathrm{pt}}) \quad (2)$$

Figure 13.3: Elaborating the two composites $(\mathrm{S}^1 \to_* A) \to \Omega B$.

fig:Omega-O-elab

following triangle:



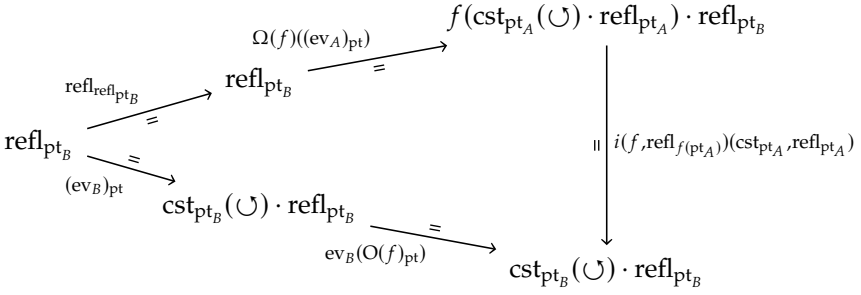
For defining $i(f)(p, p_{\mathrm{pt}})$ we apply path induction on p_{pt} and on f_{pt}, setting $\mathrm{pt}_A \equiv p(\bullet)$ and $p_{\mathrm{pt}} \equiv \mathrm{refl}_{p(\bullet)}$, as well as $\mathrm{pt}_B \equiv f(\mathrm{pt}_A) \equiv f(p(\bullet))$ and $f_{\mathrm{pt}} \equiv \mathrm{refl}_{f(p(\bullet))}$. Then identifying (1) and (2) reduces to the task of identifying $f(p(\circlearrowleft) \cdot \mathrm{refl}_{p(\bullet)}) \cdot \mathrm{refl}_{f(p(\bullet))}$ and $(f \circ p)(\circlearrowleft) \cdot \mathrm{refl}_{f(p(\bullet))}$.

The latter identity type stays well typed when we replace $\circlearrowleft$ by an arbitrary $g : \bullet \xrightarrow{=} z$, $z : \mathrm{S}^1$. By induction on g, we define an element

$$\iota(f, p, g) : \big((f(p(g) \cdot \mathrm{refl}_{p(\bullet)}) \cdot \mathrm{refl}_{f(p(\bullet))}) \xrightarrow{=} ((f \circ p)(g) \cdot \mathrm{refl}_{f(p(\bullet))})\big),$$

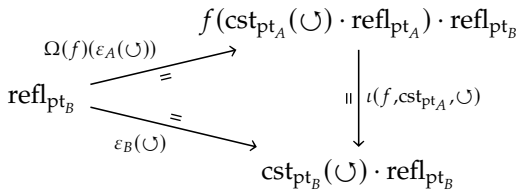
setting $\iota(f, p, \mathrm{refl}_\bullet) :\equiv \mathrm{refl}_{\mathrm{refl}_{f(p(\bullet))}}$. We complete the definition of $i(f, f_{\mathrm{pt}})(p, p_{\mathrm{pt}})$ by setting $i(f, \mathrm{refl}_{f(p(\bullet))})(p, \mathrm{refl}_{p(\bullet)}) :\equiv \iota(f, p, \circlearrowleft)$.

Again applying path induction on f_{pt}, assuming that $\mathrm{pt}_B \equiv f(\mathrm{pt}_A)$ and $f_{\mathrm{pt}} \equiv \mathrm{refl}_{\mathrm{pt}_B}$, the triangle above reduces to the following triangle:



Note that $i(f, \mathrm{refl}_{f(\mathrm{pt}_A)})(\mathrm{cst}_{\mathrm{pt}_A}, \mathrm{refl}_{\mathrm{pt}_A}) \equiv \iota(f, \mathrm{cst}_{\mathrm{pt}_A}, \circlearrowleft)$. By Definition 13.1.21, as f_{pt} is a reflexivity path, we get that $\mathrm{O}(f)_{\mathrm{pt}}$ and $\mathrm{ev}_B(\mathrm{O}(f)_{\mathrm{pt}})$ are also reflexivity paths. Hence, recalling Remark 13.1.23 for the pointing paths

$(\mathrm{ev}_A)_{\mathrm{pt}}$, $(\mathrm{ev}_B)_{\mathrm{pt}}$, we have to fill the following triangle:



This last triangle stays well typed when we replace $\circlearrowleft$ by an arbitrary $g : \bullet \xrightarrow{=} z$, $z : \mathrm{S}^1$.[21] Then apply induction on g, setting $g \equiv \mathrm{refl}_{\bullet}$, which boils down to the same triangle with $\circlearrowleft$ replaced by $\mathrm{refl}_{\bullet}$. The whole diagram has now become a reflexivity diagram, as also $\iota(f, \mathrm{cst}_{\mathrm{pt}_A}, \mathrm{refl}_{\bullet})$ is reflexivity by definition, and we are done. □

[21] $\varepsilon_A(g) : \mathrm{refl}_{\mathrm{pt}_A} \xrightarrow{=} \mathrm{cst}_{\mathrm{pt}_A}(g) \cdot \mathrm{refl}_{\mathrm{pt}_A}$, so $\Omega(f)(\varepsilon_A(g))$ is a path from $\Omega(f)(\mathrm{refl}_{\mathrm{pt}_A}) \equiv \mathrm{refl}_{\mathrm{pt}_B}$ to $\Omega(f)(\mathrm{cst}_{\mathrm{pt}_A}(g) \cdot \mathrm{refl}_{\mathrm{pt}_A}) \equiv f(\mathrm{cst}_{\mathrm{pt}_A}(g) \cdot \mathrm{refl}_{\mathrm{pt}_A}) \cdot \mathrm{refl}_{\mathrm{pt}_B}$, by the induction on f_{pt}. Also, $\iota(f, \mathrm{cst}_{\mathrm{pt}_A}, g)$ has the right type.

Composition with O, i.e., $(\mathrm{O} \circ _) :\equiv (q \mapsto \mathrm{O} \circ q)$, gives the map[22]

$$(\mathrm{O} \circ _) : (\mathrm{S}^1 \to_* (A \to_* B)) \to (\mathrm{S}^1 \to ((\mathrm{S}^1 \to_* A) \to_* (\mathrm{S}^1 \to_* B)).$$

[22] This map corresponds to a map of type $\Omega(X \to_* Y) \to \Omega(\Omega X \to_* \Omega Y)$.

con:swap-prtd-doms

Construction 13.1.25. *Let X, Y and Z be pointed types. We use $T_\div$ to denote the underlying type of a pointed type T. Then we have a pointed equivalence*

$$\mathrm{swap} : (X \to_* (Y \to_* Z)) \to_* (Y \to_* (X \to_* Z))$$

such that the totally unpointed map $\mathrm{swap}_{\div\div}$*, defined by*

$$\begin{aligned} \mathrm{swap}_{\div\div} &:\equiv (f \mapsto (y \mapsto (x \mapsto f(x)(y)))) \\ &: (X_\div \to (Y_\div \to Z_\div)) \to (Y_\div \to (X_\div \to Z_\div)) \end{aligned}$$

can be identified with the map swapping the two arguments of any input map.

Implementation of Construction 13.1.25. TBD (Do first the equivalence of $(X \to_* (Y \to_* Z))$ with the sum type of totally unpointed maps with additional structure, including coherence.) □

xca:allprd-S1-contractible
exa:allS1=-swap

Exercise 13.1.26. Let X, Y, Z be pointed types and let X, Y be connected and Z a 1-type. Show that $X \to_* (Y \to_* Z)$ is contractible. ⌟

Example 13.1.27. In this example we will explore the type

$$M :\equiv (\mathrm{S}^1 \to_* (\mathrm{S}^1 \to_* \mathrm{B}^2\mathbb{Z})),$$

where $\mathrm{B}^2\mathbb{Z}$ is the simply connected 2-type $\sum_{X:\mathcal{U}} \|\mathrm{S}^1 \xrightarrow{=} X\|_0$ from Section 12.2, pointed at $\mathrm{pt} :\equiv (\mathrm{S}^1, |\mathrm{refl}_{\mathrm{S}^1}|_0)$.

There are several ways to see that $\Omega(\mathrm{B}^2\mathbb{Z}) \equiv (\mathrm{pt} = \mathrm{pt}, \mathrm{refl}_{\mathrm{pt}})$ is equivalent to $\mathrm{B}\mathbb{Z} \equiv \mathrm{S}^1$. One is to use Theorem 12.2.13, as $\mathbb{Z}$ is abelian. Another is to recall from the proof of Lemma 12.2.10 the equivalence between $\mathrm{pt} \xrightarrow{=} \mathrm{pt}$ and the component of $\mathrm{S}^1 \xrightarrow{=} \mathrm{S}^1$ at $\mathrm{id}_{\mathrm{S}^1}$.[23] The latter component is equivalent to S^1 by Exercise 3.4.12. Using ev from Remark 13.1.23, it follows that M is equivalent to $\Omega\Omega(\mathrm{B}^2\mathbb{Z})$ and hence to Z.

[23] We use $\mathrm{refl}_{\mathrm{pt}}$ and $\mathrm{id}_{\mathrm{S}^1}$ interchangeably.

Via the above equivalence between $\Omega(\mathrm{B}^2\mathbb{Z})$ and S^1, combined with the equivalence ptw_* from Construction 2.21.8, we get an equivalence between $\Omega(\mathrm{S}^1 \to_* \mathrm{B}^2\mathbb{Z})$ and $\mathrm{S}^1 \to_* \mathrm{S}^1$. This means that we have maps corresponding to the degree k map δ_k in $\Omega(\mathrm{S}^1 \to_* \mathrm{B}^2\mathbb{Z})$, for any $k > 0$. We will need such maps for all $k : \mathrm{Z}$ and define them explicitly in the following paragraph.

For any $k : \mathbb{Z}$, define $\delta'_k : S^1 \to_* \Omega(B^2\mathbb{Z})$ by $\delta'_k(\bullet) :\equiv \mathrm{id}_{S^1}$, taking for $\delta'_k(\circlearrowleft)$ the k-loop around id_{S^1} given by function extensionality and $\circlearrowleft^k : \bullet \xrightarrow{=} \bullet$ (the loop case is a true proposition). We point δ'_k by reflexivity.

Now let $k : \mathbb{Z}$ and define $m_k : M$ as follows:

$$\begin{aligned} m_k(\bullet) &:\equiv \mathrm{pt}_{S^1 \to_* B^2\mathbb{Z}} \equiv (\mathrm{cst}_{\mathrm{pt}}, \mathrm{refl}_{\mathrm{pt}}) \\ m_k(\circlearrowleft) &:= \mathrm{ptw}_*^{-1}(\delta'_k) : \Omega(S^1 \to_* B^2\mathbb{Z}) \\ (m_k)_{\mathrm{pt}} &:\equiv \mathrm{refl}_{\mathrm{pt}_{S^1 \to_* B^2\mathbb{Z}}} \end{aligned}$$

We see that applying $\mathrm{ptw}_*(m_k(\circlearrowleft)) = \delta'_k$ to $\circlearrowleft^j$ gives the jk-loop around id_{S^1}. Also, using Construction 13.1.14, applying $\mathrm{ptw}_*(m_k(\circlearrowleft^i))$ to $\circlearrowleft^j$ gives the ijk-loop around id_{S^1}.

TBD1: clarify the relation between m_k and multiplication of its arguments by k (m_1 is useful for the concrete ring of the integers).

TBD2: Find out if $\mathrm{swap}(m_k)$ is equal to m_k or to m_{-k}. ⌟

Swapping the arguments of an untyped (curried) function of two arguments has the following effect on the application on paths.

con:swap-on-paths

CONSTRUCTION 13.1.28. *Let A, B, C be types and $f : A \to (B \to C)$ a map. Define the map $g : B \to (A \to C)$ by swapping the arguments of f, i.e., $g(b)(a) :\equiv f(a)(b)$ for all $a : A$ and $b : B$. Let $q : b \xrightarrow{=} b'$ be a path in B. Then we have an identification of the dependent functions $f(_)(q)$ and $\mathrm{ptw}(g(q))$.*[24]

Implementation of Construction 13.1.28. We use induction of q, taking $b' \equiv b$ and $q \equiv \mathrm{refl}_b$. Using function extensionality, let $a : A$. Then $f(a)(\mathrm{refl}_b) \equiv \mathrm{refl}_{f(a)(b)} \equiv \mathrm{refl}_{g(b)(a)} \equiv \mathrm{ptw}(\mathrm{refl}_{g(b)})(a) \equiv \mathrm{ptw}(g(\mathrm{refl}_b))(a)$. □

[24] $f(_)(q)$ is short for the function mapping $a : A$ to the path $\mathrm{ap}_{f(a)}(q) : (f(a)(b) \xrightarrow{=} f(a)(b'))$. Also, $\mathrm{ptw}(g(q))$ is the function mapping $a : A$ to the path $\mathrm{ptw}(\mathrm{ap}_g(q))(a) : (g(b)(a) \xrightarrow{=} g(b')(a))$. By the definition of g, the types $f(a)(b) \xrightarrow{=} f(a)(b')$ and $g(b)(a) \xrightarrow{=} g(b')(a)$ are definitionally equal.

rem:groupoids

REMARK 13.1.29. In Figure 13.4, X and Y are pointed types, and ptw_* is from Remark 13.1.16. The three occurrences of Ω in the labels of the downward arrows are all instances of Definition 4.4.3. In Figure 13.4, we have colored occurrences of Ω that come from the Ω in the left upper corner. Note that Ω shifts position from first to second along the arrow labelled $(i \circ _)$, where $i :\equiv (q : \Omega^2 Y \mapsto q^{-1})$.

In order to formally define $\Omega(\Omega)$, we need to define the pointing path Ω_{pt} of Ω. Note that $\mathrm{pt}_{X \to_* Y} \equiv (\mathrm{cst}_{\mathrm{pt}_Y}, \mathrm{refl}_{\mathrm{pt}_Y})$, i.e., the point of $X \to_* Y$ is the constant map $x \mapsto \mathrm{pt}_Y$ pointed by reflexivity. Likewise, the point of $\Omega X \to_* \Omega Y$ is the pointed constant map $(\mathrm{cst}_{\mathrm{refl}_{\mathrm{pt}_Y}}, \mathrm{refl}_{\mathrm{refl}_{\mathrm{pt}_Y}})$. We want a path Ω_{pt} of type $(\mathrm{cst}_{\mathrm{refl}_{\mathrm{pt}_Y}}, \mathrm{refl}_{\mathrm{refl}_{\mathrm{pt}_Y}}) \xrightarrow{=} \Omega(\mathrm{cst}_{\mathrm{pt}_Y}, \mathrm{refl}_{\mathrm{pt}_Y})$, where[25]

$$\Omega(\mathrm{cst}_{\mathrm{pt}_Y}, \mathrm{refl}_{\mathrm{pt}_Y}) \equiv (p \mapsto \mathrm{ap}_{\mathrm{cst}_{\mathrm{pt}_Y}}(p) \cdot \mathrm{refl}_{\mathrm{pt}_Y}, \mathrm{refl}_{\mathrm{refl}_{\mathrm{pt}_Y}}).$$

By induction on $p : (\mathrm{pt}_X \xrightarrow{=} x)$, define $h(p) : \mathrm{refl}_{\mathrm{pt}_Y} \xrightarrow{=} (\mathrm{ap}_{\mathrm{cst}_{\mathrm{pt}_Y}}(p) \cdot \mathrm{refl}_{\mathrm{pt}_Y})$ setting $h(\mathrm{refl}_{\mathrm{pt}_X}) :\equiv \mathrm{refl}_{\mathrm{refl}_{\mathrm{pt}_Y}}$. Applying ptw_* we can now define

$$\Omega_{\mathrm{pt}} :\equiv \mathrm{ptw}_*^{-1}(h, \mathrm{refl}_{\mathrm{refl}_{\mathrm{refl}_{\mathrm{pt}_Y}}}) : \big((\mathrm{cst}_{\mathrm{refl}_{\mathrm{pt}_Y}}, \mathrm{refl}_{\mathrm{refl}_{\mathrm{pt}_Y}}) \xrightarrow{=} \Omega(\mathrm{cst}_{\mathrm{pt}_Y}, \mathrm{refl}_{\mathrm{pt}_Y})\big).$$

Now we can state the definition of $\Omega(\Omega)$:

fig:ulrik

$$\Omega(\Omega)(q) \equiv \Omega_{\mathrm{pt}}^{-1} \cdot \mathrm{ap}_\Omega(q) \cdot \Omega_{\mathrm{pt}} \quad \text{for all } q : \Omega(X \to_* Y).$$

We want to fill the diagram in Figure 13.4 in full generality, even though we will only need it for X a pointed 1-type and Y a pointed 2-type.[26] TODO: Elaborate the composites, and identify their first components, then using Corollary 13.1.20. ⌟

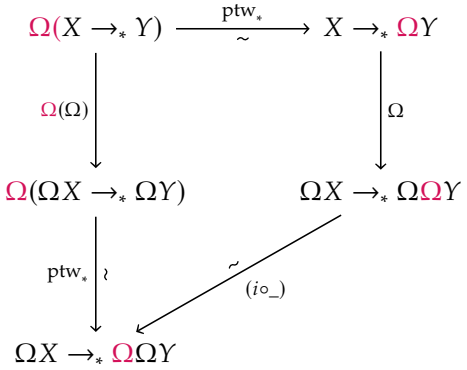


FIGURE 13.4: Fill (i is path reversal)!

[25] Recall that $\Omega(f)$ is pointed by path algebra identifying $\mathrm{refl}_{\mathrm{pt}_Y}$ with $f_{\mathrm{pt}}^{-1} \cdot \mathrm{refl}_{\mathrm{pt}_Y} \cdot f_{\mathrm{pt}}$, by induction on f_{pt}.

[26] Then $p =_{\Omega X} p'$ and $q =_{\Omega^2 Y} q'$ are proof-irrelevant.

def:O'

Definition 13.1.30. Let A and B be pointed types. Define the map map $O'_{A,B} : ((A \to_* B) \to_* ((S^1 \to_* A) \to_* (S^1 \to_* B))$ by $O'_{A,B} :\equiv (O_{A,B} \circ \mathrm{swap}^{-1} \circ _)$.[27] ⌟

[27] Again, we often write O' for $O'_{A,B}$.

con:ptw-swap-prtd-doms

Construction 13.1.31. *Let X and Y be pointed types and consider the equivalences* $\mathrm{ptw}_* : \Omega(X \to_* Y) \to (X \to_* \Omega Y)$ *from Remark 13.1.16,* swap *from Construction 13.1.25, and* ev *from Remark 13.1.23. Then we have an identification of* $\mathrm{ev} \circ \mathrm{swap}(_)$ *and* $(\mathrm{ptw}_* \circ \mathrm{ev})$*, as represented by Figure 13.5.*

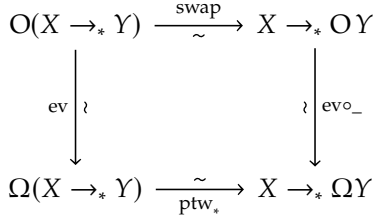


Figure 13.5: swap and ptw_* correspond.

Implementation of Construction 13.1.31. Using function extensionality, it suffices to identify $\mathrm{ev} \circ \mathrm{swap}(f)$ and $(\mathrm{ptw}_* \circ \mathrm{ev})(f)$ for every $f : S^1 \to_* (X \to_* Y)$. The latter identification is in the type $X \to_* \Omega Y$, which means that we only have to identify the underlying functions, due to Corollary 13.1.20. This greatly simplifies our task: given $f : S^1 \to_* (X \to_* Y)$, the pointing path of $\mathrm{swap}(f)$ plays no role in the underlying function of $\mathrm{ev} \circ \mathrm{swap}(f)$. In contrast, the pointing path $f_{\mathrm{pt}} : \mathrm{pt}_{X \to_* Y} \xrightarrow{=} f(\bullet)$ is important, but only in so far it applies to the underlying functions of $\mathrm{pt}_{X \to_* Y}$ and $f(\bullet)$. Therefore we abbreviate $f'_{\mathrm{pt}} :\equiv \mathrm{ptw}(\mathrm{fst}(f_{\mathrm{pt}}))$, so that $f'_{\mathrm{pt}}(x) : (\mathrm{pt}_Y \xrightarrow{=} f(\bullet)(x))$ for $x : X$.

The underlying function of $\mathrm{swap}(f)$ maps any $x : X$ to the function $(z : S^1) \mapsto f(z)(x)$, pointed by $f'_{\mathrm{pt}}(x)$. Evaluating this pointed map at $\circlearrowleft$ gives $(\mathrm{ev} \circ \mathrm{swap}(f))(x) \equiv f'_{\mathrm{pt}}(x)^{-1} \cdot f(\circlearrowleft)(x) \cdot f'_{\mathrm{pt}}(x)$.[28] This is the result of going first right and then down in Figure 13.5, applied to $x : X$.

[28] Here $f(\circlearrowleft)(x)$ is short for $\mathrm{ap}_{z \mapsto \mathrm{fst}(f_\div(z))(x)}(\circlearrowleft)$.

Now we go first down and then right in Figure 13.5. Evaluating f as above at $\circlearrowleft$ gives $\mathrm{ev}(f) \equiv f_{\mathrm{pt}}^{-1} \cdot f(\circlearrowleft) \cdot f_{\mathrm{pt}}$. Applying ptw_* and taking the underlying function gives $\mathrm{ptw}(\mathrm{fst}(f_{\mathrm{pt}}^{-1} \cdot f(\circlearrowleft) \cdot f_{\mathrm{pt}}))$. Applying the latter function to $x : X$ gives a result that is easily identified with $f'_{\mathrm{pt}}(x)^{-1} \cdot \mathrm{ptw}(\mathrm{fst}(f(\circlearrowleft)))(x) \cdot f'_{\mathrm{pt}}(x)$, as both fst and ptw preserve composition.[29]

[29] Here $\mathrm{ptw}(\mathrm{fst}(f(\circlearrowleft)))(x)$ is in fact $\mathrm{ptw}(\mathrm{fst}(\mathrm{ap}_{f_\div}(\circlearrowleft)))(x)$.

Finally, we complete the construction by identifying the results of the last two paragraphs, for which it suffices to identify the elements as given in the footnotes. We generalize them from $\circlearrowleft$ to an arbitrary $p : \bullet \xrightarrow{=} z$, $z : S^1$, and note that both $\mathrm{ap}_{z \mapsto \mathrm{fst}(f_\div(z))(x)}(p)$ and $\mathrm{ptw}(\mathrm{fst}(\mathrm{ap}_{f_\div}(p)))(x)$ have type $\mathrm{fst}(f_\div(\bullet))(x) \xrightarrow{=} \mathrm{fst}(f_\div(z))(x)$, and are readily identified by induction on p. □

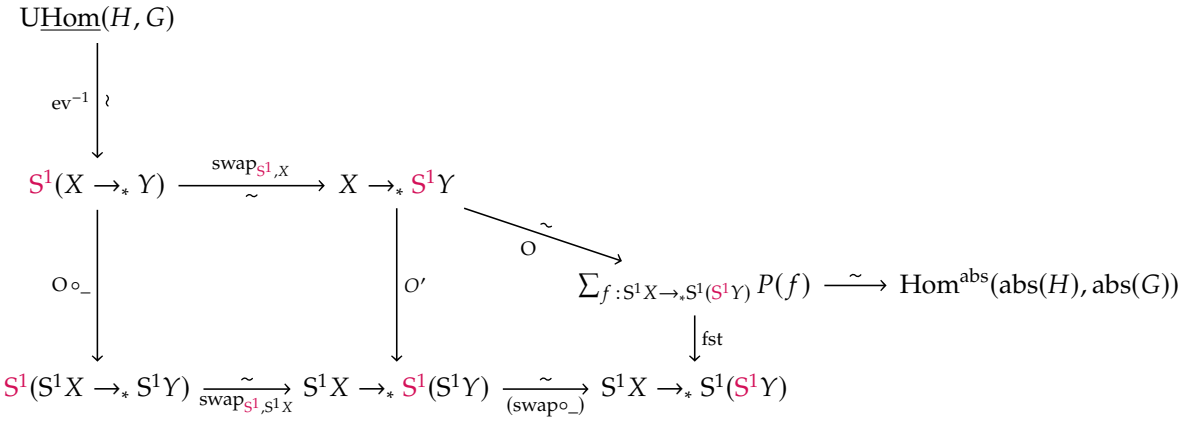


Figure 13.6: Legenda: $X :\equiv BH$; $Y :\equiv B^2G$; ev is from Corollary 3.1.3; swap is from Construction 13.1.31; O is from Definition 13.1.21; O' is from Definition 13.1.30; $P(f)$ expresses that $\mathrm{ev} \circ f \circ \mathrm{ev}^{-1}$ classifies a homomorphism. Moreover, the colors track related occurrences of S^1.

fig:bjkyrn

Recall from Theorem 12.2.13 the equivalence B^2 from the type of abelian groups to the type of pointed simply connected 2-types. Let $H : \mathrm{Group}$ be a group and let $G : \mathrm{AbGroup}$ be an abelian group. Then $\mathrm{B}^2 G$ and hence also $BH \to_* \mathrm{B}^2 G$ is a 2-type, pointed at the constant map that sends any $w : BH$ to the point $\mathrm{pt}_{\mathrm{B}^2 G} :\equiv (BG_\div, |\mathrm{id}_{BG_\div}|_0)$ of $\mathrm{B}^2 G$.[30] In fact, the type $BG \to_* \mathrm{B}^2 G$ is a 1-type, since the maps are pointed.

[30] Itself pointed by reflexivity.

def:Abhomgroup

Definition 13.1.32. Let $H : \mathrm{Group}$ be a group and let $G : \mathrm{AbGroup}$ be an abelian group. Define the group $\underline{\mathrm{Hom}}(H, G)$ of homomorphisms from H to G by

$$\underline{\mathrm{Hom}}(H, G) :\equiv \mathrm{Aut}_{BH \to_* \mathrm{B}^2 G}(\mathrm{cst}_{\mathrm{pt}_{\mathrm{B}^2 G}}, \mathrm{refl}_{\mathrm{pt}_{\mathrm{B}^2 G}}). \qquad \lrcorner$$

The following lemma identifies the group $\underline{\mathrm{Hom}}(H, G)$ as the delooping of $\mathrm{Hom}^{\mathrm{abs}}_{\mathrm{ptw}}(\mathrm{abs}(H), \mathrm{abs}(G))$, the abelian abstract group of abstract homomorphisms with pointwise operations, as given by Exercise 7.3.6 and Exercise 4.3.5. Consequently, $\underline{\mathrm{Hom}}(H, G)$ is an abelian group.

lem:grphomabk

Lemma 13.1.33. *Let conditions be as in Definition 13.1.32. Abbreviate the shape* $(\mathrm{cst}_{\mathrm{pt}_{\mathrm{B}^2 G}}, \mathrm{refl}_{\mathrm{pt}_{\mathrm{B}^2 G}})$ *of* $\underline{\mathrm{Hom}}(H, G)$ *by* sh. *The composite of the following chain of equivalences defines an abstract isomorphism from* $\mathrm{abs}(\underline{\mathrm{Hom}}(H, G))$ *to the abstract group* $\mathrm{Hom}^{\mathrm{abs}}_{\mathrm{ptw}}(\mathrm{abs}(H), \mathrm{abs}(G))$.

$$\begin{aligned}
(\mathrm{sh} \xrightarrow{=} \mathrm{sh}) &\xrightarrow{\simeq} \sum_{h : BH_\div \to (\mathrm{pt}_{\mathrm{B}^2 G} \xrightarrow{=} \mathrm{pt}_{\mathrm{B}^2 G})} (\mathrm{refl}_{\mathrm{pt}_{\mathrm{B}^2 G}} \xrightarrow{=} h(\mathrm{sh}_H)) \quad (\textit{by } \mathrm{ptw}_*) \\
&\equiv \quad \left(BH \to_* ((\mathrm{pt}_{\mathrm{B}^2 G} \xrightarrow{=} \mathrm{pt}_{\mathrm{B}^2 G}), \mathrm{refl}_{\mathrm{pt}_{\mathrm{B}^2 G}})\right) \\
&\xrightarrow{\simeq} \quad \left(BH \to_* BG\right) \quad (\textit{by } \mathrm{ev} \circ _) \\
&\xrightarrow{\simeq} \quad \mathrm{Hom}^{\mathrm{abs}}(\mathrm{abs}(H), \mathrm{abs}(G)) \quad (\textit{by } \mathrm{abs}).
\end{aligned}$$

Proof. Given $p : (\mathrm{sh} \xrightarrow{=} \mathrm{sh})$, the equivalence ptw_* from Construction 2.21.8 gives a pair $\mathrm{ptw}_*(p)$ which we denote by $(\tilde{p}, p_{\mathrm{pt}})$. Here $\tilde{p} : BH_\div \to (\mathrm{pt}_{\mathrm{B}^2 G} \xrightarrow{=} \mathrm{pt}_{\mathrm{B}^2 G})$ maps any $w : BH$ to $\tilde{p}(w)$ corresponding[31] to a pair consisting of an equivalence (by the same name) $\tilde{p}(w) : (BG_\div \xrightarrow{\simeq} BG_\div)$ and a proof of $\|\tilde{p}(w) \xrightarrow{=} \mathrm{id}_{BG_\div}\|$, that is, all $\tilde{p}(w)$ are merely equal to the identity. The second component p_{pt} of $(\tilde{p}, p_{\mathrm{pt}})$ is an identification of $\mathrm{refl}_{\mathrm{pt}_{\mathrm{B}^2 G}}$ with $\tilde{p}(\mathrm{sh}_H)$, which gives us an identification (by the same name) p_{pt} of $\mathrm{id}_{BG_\div}$ with $\tilde{p}(\mathrm{sh}_H)$.[32]

[31] This uses Equation (12.2.2) and univalence.

[32] Technically, we should write $\mathrm{fst}(p_{\mathrm{pt}})$, but $\mathrm{snd}(p_{\mathrm{pt}})$ is true propositional data and plays no role.

In the following, until the end of the proof, we abbreviate sh_G by $\bullet$. The goal is to prove, for any $p, q : (\mathrm{sh} \xrightarrow{=} \mathrm{sh})$ and $g : UH$, the proposition

$$\left(\Omega(\mathrm{ev}_\bullet \circ \mathrm{ptw}_*(pq))\right)(g) = \left(\Omega(\mathrm{ev}_\bullet \circ \mathrm{ptw}_*(p))\right)(g) \cdot \left(\Omega(\mathrm{ev}_\bullet \circ \mathrm{ptw}_*(q))\right)(g).$$

We first elaborate the right hand side. Note that $\mathrm{ev}_\bullet(\tilde{p}(\mathrm{sh}_H)) \equiv \tilde{p}(\mathrm{sh}_H)(\bullet)$. The pointing path of $\mathrm{ev}_\bullet \circ (\tilde{p}, p_{\mathrm{pt}})$ is $\mathrm{ev}_\bullet(p_{\mathrm{pt}})$, which is equal to $\mathrm{ptw}(p_{\mathrm{pt}})(\bullet)$, and we have $\mathrm{ev}_\bullet(\tilde{p}(g)) = \mathrm{ptw}(p(g))(\bullet)$.[33]

[33] By induction on $r : s \xrightarrow{=}_{A \xrightarrow{=} A} t$ one easily constructs for all types A and elements $a : A$ an identification $\mathrm{ev}_a(r) \xrightarrow{=} \mathrm{ptw}(r)(a)$.

Using the definition of Ω from Definition 4.4.3, we calculate:

$$\begin{aligned}
\Omega(\mathrm{ev}_\bullet \circ (\tilde{p}, p_{\mathrm{pt}}))(g) &= \mathrm{ptw}(p_{\mathrm{pt}})(\bullet)^{-1} \cdot \mathrm{ev}_\bullet(\tilde{p}(g)) \cdot \mathrm{ptw}(p_{\mathrm{pt}})(\bullet) \\
&= \mathrm{ptw}(p_{\mathrm{pt}}^{-1} \cdot \tilde{p}(g) \cdot p_{\mathrm{pt}})(\bullet) \text{ in type } \bullet \xrightarrow{=} \bullet.
\end{aligned}$$

Likewise we obtain for $\tilde{q}$ and $(\widetilde{pq})$ the following equalities:

$$\begin{aligned}
\Omega(\mathrm{ev}_\bullet \circ (\tilde{q}, q_{\mathrm{pt}}))(g) &= \mathrm{ptw}(q_{\mathrm{pt}}^{-1} \cdot \tilde{q}(g) \cdot q_{\mathrm{pt}})(\bullet) \\
\Omega(\mathrm{ev}_\bullet \circ (\widetilde{pq}, (pq)_{\mathrm{pt}}))(g) &= \mathrm{ptw}((pq)_{\mathrm{pt}}^{-1} \cdot (\widetilde{pq})(g) \cdot (pq)_{\mathrm{pt}})(\bullet)
\end{aligned}$$

We continue with elaboration the latter equality in three steps.

First, by Construction 13.1.14, we have an identification $\mathrm{ptw}_*(pq) \xrightarrow{=} \mathrm{ptw}_*(p) \cdot_{\mathrm{ptw}} \mathrm{ptw}_*(q)$, with $\cdot_{\mathrm{ptw}}$ as given in Definition 13.1.13. This gives an identification of type $(\tilde{pq}) \xrightarrow{=} (w \mapsto \tilde{p}(w) \cdot \tilde{q}(w))$, which we (abusively) take to be definitional. Hence, $(\tilde{pq})(\mathrm{sh}_H) \equiv (\tilde{p}(\mathrm{sh}_H) \cdot \tilde{q}(\mathrm{sh}_H))$, which as function $BG_\div \xrightarrow{\simeq} BG_\div$ corresponds to the composition $\tilde{p}(\mathrm{sh}_H) \circ \tilde{q}(\mathrm{sh}_H)$.

Second, also need a path $(pq)_{\mathrm{pt}}$ from $\mathrm{id}_{BG_\div}$ to $(\tilde{pq})(\mathrm{sh}_H)$. For this we take $(pq)_{\mathrm{pt}} \equiv \mathrm{ap}_{(\tilde{p}(\mathrm{sh}_H)\cdot_)}(q_{\mathrm{pt}}) \cdot p_{\mathrm{pt}}$, which follows from Definition 13.1.13.[34]

[34] The order is reversed wrt 13.1.13 because the paths are reversed: $q_{\mathrm{pt}} : \mathrm{refl}_{BG_\div} \xrightarrow{=} \tilde{q}(\mathrm{sh}_H)$, so $\mathrm{ap}_{(\tilde{p}(\mathrm{sh}_H)\cdot_)}(q_{\mathrm{pt}}) : \tilde{p}(\mathrm{sh}_H) \xrightarrow{=} \tilde{p}(\mathrm{sh}_H) \cdot \tilde{q}(\mathrm{sh}_H)$, so p_{pt} must be precomposed.

Third, we elaborate $(\tilde{pq})(g)$ which is shorthand for $\mathrm{ap}_{w\mapsto\tilde{p}(w)\tilde{q}(w)}(g)$. In order to be able to do induction on g, we loosen up the endpoint of g. Let $w : BH_\div$, $g : \mathrm{sh}_H \xrightarrow{=} w$, $z : BG_\div$ and consider the following diagram. The task is to find a path from $\tilde{p}(\mathrm{sh}_H)(\tilde{q}(\mathrm{sh}_H)$ to $\tilde{p}(w)(\tilde{q}(w)$ that can be identified with $\mathrm{ap}_{w\mapsto\tilde{p}(w)\tilde{q}(w)}(g)$.

$$
\begin{array}{ccccccc}
 & & BG_\div & \xrightarrow{\tilde{q}(_)} & BG_\div & \xrightarrow{\qquad\tilde{p}(_)\qquad} & BG_\div \\
\mathrm{sh}_H & & z & \xmapsto{\tilde{q}(\mathrm{sh}_H)} & \tilde{q}(\mathrm{sh}_H)(z) & \xmapsto{\tilde{p}(\mathrm{sh}_H)} & \tilde{p}(\mathrm{sh}_H)(\tilde{q}(\mathrm{sh}_H)(z)) \\
 & & & & & & \Vert \downarrow r_1(g,z) \\
\Vert & & & & & & \tilde{p}(\mathrm{sh}_H)(\tilde{q}(w)(z)) \\
 & & & & & & \Vert \downarrow r_2(g,z) \\
\downarrow g & & z & \xmapsto{\tilde{q}(w)} & \tilde{q}(w)(z) & \xmapsto{\tilde{p}(w)} & \tilde{p}(w)(\tilde{q}(w)(z)) \\
w & &
\end{array}
$$

There are two ways to proceed. One is based on the composite of the paths $r_1(g,z) :\equiv \tilde{p}(\mathrm{sh}_H)(\mathrm{ptw}(\tilde{q}(g))(z))$ and $r_2(g,z) :\equiv \mathrm{ptw}(\tilde{p}(g))(\tilde{q}(w)(z))$ of types as shown above.[35]

[35] The other way is analogous, based on the intermediate point $\tilde{p}(w)(\tilde{q}(\mathrm{sh}_H)(z))$ instead of $\tilde{p}(\mathrm{sh}_H)(\tilde{q}(w)(z))$, with $r_1(g,z) :\equiv (\mathrm{ptw}(\tilde{p}(g)))(\tilde{q}(\mathrm{sh}_H)(z))$ and $r_2(g,z) :\equiv \tilde{p}(w)((\mathrm{ptw}(\tilde{q}(g)))(z))$.

We proceed by induction on g and take $g \equiv \mathrm{refl}_{\mathrm{sh}_H}$, so $w \equiv \mathrm{sh}_H$. Then we have $\mathrm{ap}_{w\mapsto\tilde{p}(w)\tilde{q}(w)}(g) \equiv \mathrm{refl}_{\tilde{p}(\mathrm{sh}_H)(\tilde{q}(\mathrm{sh}_H)}$. Also in this case, both $r_1(g,z)$ and $r_2(g,z)$ are definitionally equal to $\mathrm{refl}_{\tilde{p}(\mathrm{sh}_H)(\tilde{q}(\mathrm{sh}_H)(z))}$. It follows that $\mathrm{ap}_{w\mapsto\tilde{p}(w)\tilde{q}(w)}(g)$ is equal to $\mathrm{ptw}^{-1}(z \mapsto r_2(g,z) \cdot r_1(g,z))$ for any $g : \mathrm{sh}_H \xrightarrow{=} w$.

We can now reformulate the goal as the equality of

$$\mathrm{ptw}\left(p_{\mathrm{pt}}^{-1} \cdot \tilde{p}(g) \cdot p_{\mathrm{pt}} \cdot q_{\mathrm{pt}}^{-1} \cdot \tilde{q}(g) \cdot q_{\mathrm{pt}}\right)(\bullet)$$

and the composition of the following four paths:

$$
\begin{aligned}
\mathrm{ptw}\left(p_{\mathrm{pt}}^{-1} \cdot \mathrm{ap}_{(\tilde{p}(\mathrm{sh}_H)\cdot_)}(q_{\mathrm{pt}}^{-1})\right)(\bullet) &\quad \text{coming from } (pq)_{\mathrm{pt}}^{-1} \\
\mathrm{ptw}\left(\tilde{p}(g)\right)(\tilde{q}(\mathrm{sh}_H)(\bullet)) &\quad \text{coming from } r_2 \\
\tilde{p}(\mathrm{sh}_H)(\mathrm{ptw}\left(\tilde{q}(g)\right)(\bullet)) &\quad \text{coming from } r_1 \\
\mathrm{ptw}\left(\mathrm{ap}_{(\tilde{p}(\mathrm{sh}_H)\cdot_)}(q_{\mathrm{pt}}) \cdot p_{\mathrm{pt}}\right)(\bullet) &\quad \text{coming from } (pq)_{\mathrm{pt}}.
\end{aligned}
$$

Plan: Transport the goal along the paths $p_{\mathrm{pt}} : (\mathrm{id}_{BG_\div} \xrightarrow{=} \tilde{p}(\mathrm{sh}_H))$ and $q_{\mathrm{pt}} : (\mathrm{id}_{BG_\div} \xrightarrow{=} \tilde{q}(\mathrm{sh}_H))$.

W.i.p.

□

13.1.34 *Concrete rings*

sec:concrings

We will now elaborate an approach to rings that is even more concrete than mixed rings. For the latter rings we took the obvious first step

to replace the abstract additive group by a (concrete) group. Since monoids have no concrete counterpart in our set up, we replaced in Definition 13.1.8 the multiplicative monoid by the half abstract $\ell, r : UR \to \mathrm{Hom}(R, R)$.

The use of ℓ, r was based on the observation that, for any abstract ring $\mathcal{R}$, left and right multiplication by a fixed but arbitrary element of R are abstract homomorphisms from the additive group $(R, 0, +, -)$ of $\mathcal{R}$ to itself. Even more so, the map $a \mapsto (a \cdot _)$ is an abstract homomorphism from $(R, 0, +, -)$ to the abstract group $\mathrm{Hom}^{\mathrm{abs}}_{\mathrm{ptw}}(R, R)$ of abstract homomorphisms from $(R, 0, +, -)$ to itself, with pointwise operations induced by $(R, 0, +, -)$.[36]

Given that we have replaced $(R, 0, +, -)$ by an abelian group $G : \mathrm{Group}$, the plan is to deloop $\mathrm{Hom}^{\mathrm{abs}}_{\mathrm{ptw}}(\mathrm{abs}(G), \mathrm{abs}(G))$. Denoting the result of the delooping by $\underline{\mathrm{Hom}}(G, G)$,[37] we can then define the multiplication as a homomorphism $\mu : \mathrm{Hom}(G, \underline{\mathrm{Hom}}(G, G))$.

One way of delooping $\mathrm{Hom}^{\mathrm{abs}}_{\mathrm{ptw}}(\mathrm{abs}(G), \mathrm{abs}(G))$ would be to use the inverse of abs in Lemma 7.5.1 which involves torsors. We prefer to use $\underline{\mathrm{Hom}}(G, G)$ from Definition 13.1.32, making direct use of the assumption that G is abelian.

[36] $\mathrm{Hom}^{\mathrm{abs}}_{\mathrm{ptw}}(R, R)$ is an abelian abstract group by Exercise 7.3.6 and Exercise 4.3.5.

[37] This notation presupposes that G is abelian and distinguishes the *set* of homomorphisms from G to G from the *group* with this set of homomorphisms as underlying set.

def:ring

Definition 13.1.35. A *ring* R consists of the following data:

(1) An abelian group also denoted R;

(2) A homomorphism $1_R : \mathrm{Hom}(\mathbb{Z}, R)$;

(3) A homomorphism $\mu : \mathrm{Hom}(R, \underline{\mathrm{Hom}}(R, R))$, with $\underline{\mathrm{Hom}}(R, R)$ the group defined in Definition 13.1.32.

Moreover, the following equations should hold:

ring:unit-laws

(1) $\mathrm{ev} \circ (\mathrm{U}(\mu \circ 1_R)(\circlearrowleft)) = \mathrm{Bid}_R \approx TBD$ (the *multiplicative unit laws*)[38]

ring:assoc-law

(2) TBD (the *associative law*).

The properties (1)-(2) are together denoted by $\mathrm{RingProperties}(R, 1_R, \mu)$. The ring R is called *commutative* if TBD, and *non-trivial* if 1_R is not trivial.[39] ⌟

[38] Not great: $\mathrm{U}(\mu \circ 1_R)$ is an abstract homomorphism from $\mathrm{U}\mathbb{Z}$ to $\mathrm{U}\,\mathrm{Hom}(R, R)$ and the latter type is equivalent to $(BR \to_* \Omega \mathrm{B}^2 R)$. Finally by postcomposition with ev, we get equivalence with $(BR \to_* BR)$. The other unit law is probably worse.

[39] A homomorphism is trivial if it classified by the constant function at the shape to the target group. Or, equivalently, if it factors through the trivial group.

We proceed by giving the standard example of the integers as a ring in the sense of Definition 13.1.35.

Example 13.1.36. We take the group $\mathbb{Z}$ of the integers classified by the circle as the abelian group for the ring of the integers. We take $1_{\mathbb{Z}} :\equiv \mathrm{id}_{\mathbb{Z}}$, the identity homomorphism. For defining μ we first elaborate $\mathrm{Hom}(\mathbb{Z}, \mathbb{Z})$ as a group. Unfolding the definition we get (leaving the points implicit) $\mathrm{B}\,\mathrm{Hom}(\mathbb{Z}, \mathbb{Z}) \equiv (\mathrm{S}^1 \to_* \sum_{X:\mathcal{U}} \|\mathrm{S}^1 \xrightarrow{=} X\|_0)$. The shape of $\mathrm{Hom}(\mathbb{Z}, \mathbb{Z})$ is the constant map that sends any $z : \mathrm{S}^1$ to $(\mathrm{S}^1, |\mathrm{id}_{\mathrm{S}^1}|_0)$, pointed by reflexivity.

Recall that $\mathrm{B}^2\mathbb{Z} \equiv \sum_{X:\mathcal{U}} \|\mathrm{S}^1 \xrightarrow{=} X\|_0)$, pointed at $\mathrm{sh}_{\mathrm{B}^2\mathbb{Z}} \equiv (\mathrm{S}^1, |\mathrm{id}_{\mathrm{S}^1}|_0)$. For $\mu : \mathrm{Hom}(\mathbb{Z}, \mathrm{Hom}(\mathbb{Z}, \mathbb{Z}))$ we take,[40] with ve from Theorem 3.1.2,

$$\mathrm{B}\mu :\equiv (z : \mathrm{S}^1) \mapsto \mathrm{ve}_{\mathrm{B}^2\mathbb{Z}}(\mathrm{sh}_{\mathrm{B}^2\mathbb{Z}}, (e_z, !)).$$

In this succint definition, $\mathrm{ve}_{\mathrm{B}^2\mathbb{Z}}(\mathrm{sh}_{\mathrm{B}^2\mathbb{Z}}, |e_z|_0)$ can be identified as the function from S^1 to $\mathrm{B}^2\mathbb{Z}$ that sends $\bullet$ to S^1 and $\circlearrowleft$ to $(e_z, !)$ where $e_z : (\mathrm{S}^1 \xrightarrow{=} \mathrm{S}^1)$, $! : \|e_z \xrightarrow{=} \mathrm{id}_{\mathrm{S}^1}\|$. In the following we focus on first components, that is, on S^1 and e_z, analyzing how $\mathrm{B}\mu$ applies to paths.

[40] Exercise material? Define $s : \mathrm{id}_{\mathrm{S}^1} \xrightarrow{=} \mathrm{id}_{\mathrm{S}^1}$ by function extensionality, setting $s(\bullet) :\equiv \circlearrowleft$, $s(\circlearrowleft) := !$. Now define $e_z : \mathrm{S}^1 \xrightarrow{=} \mathrm{S}^1$ by $e_z(\bullet) :\equiv z$, $e_z(\circlearrowleft) := s(z) : (z \xrightarrow{=} z)$. Indeed, $e_\bullet = \mathrm{id}_{\mathrm{S}^1}$ and, by path induction $e_p(\bullet) = p$ for all $p : \bullet \xrightarrow{=} z$, so $e_{\circlearrowleft} = s$.

For any $z : S^1$ and $k : Z$ we have that $B\mu(z, \circlearrowleft^k) = e_z^k : (S^1 \xrightarrow{=} S^1)$. Hence for any $j : Z$ we have that $B\mu(\circlearrowleft^j, \circlearrowleft^k) = e^k_{\circlearrowleft^j} = s^{jk} : (\mathrm{id}_{S^1} \xrightarrow{=} \mathrm{id}_{S^1})$. Almost there! Use ev to get to U$\mathbb{Z}$?

It follows that $(\mathbb{Z}, 1_{\mathbb{Z}}, \mu)$ is a non-trivial commutative ring. ⌟

Exercise 13.1.37. Let $(R, 1_r, \mu)$ be a ring. Show that UR is an abstract ring with additive group abs(R) and multiplicative monoid $(UR, U1_R(\circlearrowleft), U\mu$. TBD ⌟

xca:Rconcring->URabstring

TBD define type of (abstract) rings, prove equivalence, define ring homomorphisms, delooping etc. No interesting difficulties expected before we come to fields.

Definition 13.1.38. Given a commutative ring R, an element $e : R$ is **invertible** if there exists an element $a : R$ such that $e \cdot a = 1$ and $a \cdot e = 1$:

$$\mathrm{isInvertible}(e) :\equiv \left\| \sum_{a:R} (e \cdot a = 1) \times (a \cdot e = 1) \right\|$$

⌟

Theorem 13.1.39. *In any nontrivial commutative ring R, 0 is always a non-invertible element.*

$$\mathrm{isNonTrivialCRing}(R) \to \neg\mathrm{isInvertible}(0)$$

Proof. Suppose that 0 is invertible. Then there exists an element $a : R$ such that $a \cdot 0 = 1$. However, due to the absorption properties of 0 and the fact that R is a set, $a \cdot 0 = 0$. This implies that $0 = 1$, which contradicts the fact that $0 \neq 1$ in a nontrivial commutative ring. Thus, 0 is a non-invertible element in any nontrivial commutative ring R. □

Definition 13.1.40. A nontrivial commutative ring R is a **field** if and only if the type of all non-invertible elements in R is contractible:

$$\mathrm{isField}(R) :\equiv \mathrm{isNonTrivialCRing}(R) \times \mathrm{isContr}\left(\sum_{x:R} \neg\mathrm{isInvertible}(x) \right)$$

Equivalently, R is a field if and only if every non-invertible element is equal to zero. ⌟

Remark 13.1.41. In other parts of the constructive mathematics literature, such as in Peter Johnstone's *Rings, Fields, and Spectra*, this is called a "residue field". However, in this book we shall refrain from using the term "residue field" for our definition, since that contradicts the usage of "residue field" in other parts of mathematics, such as in algebraic geometry. ⌟

Definition 13.1.42. A field is **discrete** if every element is either invertible or equal to zero.

$$\mathrm{isDiscreteField}(R) :\equiv \mathrm{isField}(R) \times \prod_{a:R} \|(a = 0) \sqcup \mathrm{isInvertible}(a)\|$$

⌟

Definition 13.1.43. A nontrivial commutative ring R is a **local ring** if for every element $a : R$ and $b : R$, if the sum $a + b$ is invertible, then either a is invertible or b is invertible.

$$\mathrm{isLocalRing}(R) :\equiv \mathrm{isNonTrivialCRing}(R) \times \prod_{a:R} \prod_{b:R} \mathrm{isInvertible}(a+b) \to \|\mathrm{isInvertible}(a) \sqcup \mathrm{isInvertible}(b)\|$$

⌟

DEFINITION 13.1.44. A field R is **Heyting** if it is also a local ring.

$$\text{isHeytingField}(R) := \text{isField}(R) \times \text{isLocalRing}(R)$$

⌟

References used in this section:

- Emmy Noether, *Ideal Theory in Rings*, Mathematische Annalen 83 (1921)
- Henri Lombardi, Claude Quitté, *Commutative algebra: Constructive methods (Finite projective modules)*
- Peter Johnstone, *Rings, Fields, and Spectra*, Journal of Algebra 49 (1977) 238-260

13.2 *vector spaces*

DEFINITION 13.2.1. Given a field K, a K-**vector space** is an abelian group V with a bilinear function $(-)(-) : K \times V \to V$ called **scalar multiplication** such that $1v = v$ and for all elements $a : K$, $b : K$, and $v : V$, $(a \cdot b)v = a(bv)$. ⌟

DEFINITION 13.2.2. A K-**linear map** between two K-vector spaces V and W is a group homomorphism $h : V \to W$ which also preserves scalar multiplication: for all elements $a : K$ and $v : V$, $f(av) = af(v)$. ⌟

DEFINITION 13.2.3. Given a field K and a set S, the **free** K-**vector space** on S is the homotopy initial K-vector space V with a function $i : S \to V$: for every other K-vector space W with a function $j : S \to W$, the type of linear maps $h : V \to W$ such that for all elements $s : S$, $h(i(s)) = j(s)$ is contractible. ⌟

DEFINITION 13.2.4. Given a field K and a natural number n, an n-**dimensional** K-**vector space** is a free K-vector space on the finite type $\text{Fin}(n)$. ⌟

13.3 *the general linear group as automorphism group*

13.4 *determinants* (†)

13.5 *examples*: *rationals, polynomials, adding a root, field extensions*

13.6 *ordered fields, real-closed fields, pythagorean fields, euclidean fields*

13.7 *complex fields, quadratically closed fields, algebraically closed fields*

14
Geometry and groups

ch:euclidean

In this chapter we study Euclidean geometry. We assume some standard linear algebra over real numbers, including the notion of finite dimensional vector space over the real numbers and the notion of inner product. In our context, the field of real numbers, $\mathbb{R}$, is a set, and so are vector spaces over it. Moreover, a vector space V has an underlying additive abstract group, and we will feel free to pass from it to the corresponding group.

14.1 *Inner product spaces*

def:InnerProductSpace

Definition 14.1.1. An *inner product space* V is a real vector space of finite dimension equipped with an inner product $H : V \times V \to \mathbb{R}$. ⌟

Let $\tilde{\mathbb{V}}$ denote the type of inner product spaces. It is a type of pairs whose elements are of the form (V, H). For $n : \mathbb{N}$, let $\tilde{\mathbb{V}}_n$ denote the type of inner product spaces of dimension n.

For each natural number n, we may construct the *standard* inner product space $\mathbb{V}^n :\equiv (V, H)$ of dimension n as follows. For V we take the vector space $\mathbb{R}^n$, and we equip it with the standard inner product given by the dot product

$$H(x, y) :\equiv x \cdot y,$$

where the dot product is defined as usual as

$$x \cdot y :\equiv \sum_i x_i y_i.$$

thm:GramSchmidt

Theorem 14.1.2. *Any inner product space V is merely equal to $\mathbb{V}^n$, where n is* $\dim V$.

For the definition of the adverb "merely", refer to Definition 2.16.13.

Proof. Since any finite dimensional vector space merely has a basis, we may assume we have a basis for V. Now use Gram-Schmidt orthonormalization to show that $V = \mathbb{V}^n$. □

lem:InnerProductSpaceIType

Lemma 14.1.3. *The type $\tilde{\mathbb{V}}$ is a* 1*-type.*

Proof. Given two inner product spaces V and V', we must show that the type $V = V'$ is a set. By univalence, its elements correspond to the linear isomorphisms $f : V \xrightarrow{\simeq} V'$ that are compatible with the inner products. Those form a set. □

def:OrthogonalGroup

DEFINITION 14.1.4. Given a natural number n, we define the *orthogonal group* $\mathrm{O}(n)$ as follows.

$$\mathrm{O}(n) :\equiv \underline{\Omega}\tilde{\mathbb{V}}_n$$

Here $\tilde{\mathbb{V}}_n$ is equipped with the basepoint provided by $\mathrm{sh}_{\mathrm{O}(n)} :\equiv \mathbb{V}^n$, and with the proof that it is a connected groupoid provided by Theorem 14.1.2 and Lemma 14.1.3. ⌟

The standard action (in the sense of Definition 5.2.29) of $\mathrm{O}(n)$ is an action of it on its designated shape $\mathbb{V}^n$. Letting $\mathrm{Vect}_{\mathbb{R}}$ denote the type of finite dimensional real vector spaces, we may compose the standard action with the projection map $\mathrm{BO}(n) \to \mathrm{Vect}_{\mathbb{R}}$ that forgets the inner product to get an action of $\mathrm{O}(n)$ on the vector space $\mathbb{R}^n$.

14.2 *Euclidean spaces*

In high school geometry courses, one encounters the Euclidean plane (of dimension 2) and the Euclidean space of dimension 3. The vectors and the points of Euclidean geometry are the basic ingredients, from which the other concepts are derived. Those concepts include such things as lines, line segments, triangles, tetrahedra, spheres, and so on. Symmetries of those objects are also studied: for example, an isosceles non-equilateral triangle has a total of 2 symmetries: the identity and the reflection through the midline.

So, a Euclidean space will come with two sets: a set of points and a set of vectors. The structure on the two sets includes the following items.

(1) If v and w are vectors, then there is a vector $v + w$ called its *sum*.

(2) If v is a vector and r is a real number, then there is a vector rv called the *scalar multiple* of v by r.

(3) If v is a vector, then there is a real nonnegative number called its *length*.

(4) If P and Q are points, then there is a unique vector v which can be "positioned" so its tail is "at" P and its head is "at" Q. It is called the vector *from P to Q*. The *distance* from P to Q is the length of v.

(5) If P is a point and v is a vector, then there is a unique point Q so that v which can be positioned so its tail is at P and its head is at Q. It is called the point obtained from P by *translation along v*.

We introduce the (new) notation $v + P$ for the point Q obtained from P by translation along v. Another fact from high school geometry is that if w is a vector, too, then the associative rule $v + (w + P) = (v + w) + P$ holds. This suggests that the essential features of high school geometry can be captured by describing the set of points as a torsor for the group of vectors.

We use that idea now to give a precise definition of *Euclidean space of dimension n*, together with its points and vectors. More complicated geometric objects will be introduced in subsequent sections.

def:EuclideanSpace

DEFINITION 14.2.1. A *Euclidean space* E is an torsor A for the additive group underlying an inner product space V. (For the definition of torsor, see Definition 7.4.1.) ⌟

We will write V also for the additive group underlying V. Thus an expression such as $\mathrm{B}V$ or Torsor_V will be understood as applying to the underlying additive group[1] of V.

[1] We are careful not to refer to the group as an Abelian group at this point, even though it is one, because the operator B may be used in some contexts to denote a different construction on Abelian groups.

DEFINITION 14.2.2. We denote the type of all Euclidean spaces of dimension n by $\tilde{\mathbb{E}}_n :\equiv \sum_{V:\tilde{\mathbb{V}}_n} \mathrm{Torsor}_V$. The elements of $\mathrm{Pts}\, E$ will be the *points* in the geometry of E, and the elements of $\mathrm{Vec}\, E$ will be the *vectors* in the geometry of E. We let $\tilde{\mathbb{E}}$ denote the type of all Euclidean spaces; it is equivalent to the sum $\sum_{n:\mathbb{N}} \tilde{\mathbb{E}}_n$. ⌟

The torsor $\mathrm{Pts}\, E$ is a nonempty set upon which V acts. Since V is an additive group, we prefer to write the action additively, too: given $v : V$ and $P : \mathrm{Pts}\, E$ the action provides an element $v + P : \mathrm{Pts}\, E$. Moreover, given $P, Q : \mathrm{Pts}\, E$, there is a unique $v : V$ such $v + P = Q$; for it we introduce the notation $Q - P :\equiv v$, in terms of which we have the identity $(Q - P) + P = Q$.

For each natural number n, we may construct the *standard* Euclidean space $\mathbb{E}^n : \tilde{\mathbb{E}}_n$ of dimension n as follows. For $\mathrm{Vec}\, E$ we take the standard inner product space $\mathbb{V}^n$, and for $\mathrm{Pts}\, E$ we take the corresponding principal torsor $\mathbb{P}_{\mathrm{sh}_{\mathbb{R}^n}}$.

THEOREM 14.2.3. *Any Euclidean space E is merely equal to $\mathbb{E}^n$, where n is* $\dim E$.

thm:EuclideanNormalization

Proof. Since we are proving a proposition and any torsor is merely trivial, by Theorem 14.1.2 we may assume $\mathrm{Vec}\, E$ is $\mathbb{V}^n$. Similarly, we may assume that $\mathrm{Pts}\, E$ is the trivial torsor. □

LEMMA 14.2.4. *The type $\tilde{\mathbb{E}}_n$ is a* 1*-type.*

lem:EuclideanSpace1Type

Proof. Observe using Theorem 5.5.7 that $\tilde{\mathbb{E}}_n \simeq s \sum_{V:\mathrm{BO}(n)} \mathrm{B}V$. The types $\mathrm{BO}(n)$ and $\mathrm{B}V$ are 1-types, so the result follows from Item (4). □

DEFINITION 14.2.5. Given a natural number n, we define the *Euclidean group* by

def:EuclideanGroup

$$\mathrm{E}(n) :\equiv \underline{\Omega}\tilde{\mathbb{E}}_n.$$

Here we take the basepoint of $\tilde{\mathbb{E}}_n$ to be $\mathbb{E}^n$, and we equip $\tilde{\mathbb{E}}_n$ with the proof that it is a connected groupoid provided by Theorem 14.2.3 and Lemma 14.2.4. ⌟

The *standard action* of $\mathrm{E}(n)$ (in the sense of Definition 5.2.29) is an action of it on the Euclidean space $\mathbb{E}^n$.

THEOREM 14.2.6. *For each n, the Euclidean group $\mathrm{E}(n)$ is equivalent to a semidirect product $\mathrm{O}(n) \ltimes \mathbb{R}^n$.*

thm:EuclideanGroupSemidirect

Proof. Recall Definition 8.2.1 and apply it to the standard action $\tilde{H} : \mathrm{BO}(n) \to$ Group of $\mathrm{O}(n)$ on the additive group underlying $\mathbb{R}^n$, as defined in Definition 14.1.4. The semidirect product $\mathrm{O}(n) \ltimes \mathbb{R}^n$ has $\sum_{V:\mathrm{BO}(n)} \mathrm{B}V$ as its underlying pointed type. Finally, observe that $\mathrm{E}(n) \simeq \sum_{V:\mathrm{BO}(n)} \mathrm{B}V$, again using Theorem 5.5.7. □

14.3 *Geometric objects*

In this section, we discuss the notion of "object" in Euclidean space, but much of what we say is more general and applies equally well to other sorts of geometry, such as projective geometry or hyperbolic geometry.

Let E be a Euclidean space, as defined in Definition 14.2.1. The points of E are the elements of $\mathrm{Pts}\,E$, and intuitively, a geometric object in E ought to come with a way to tell which points of E are inside the object.

For example, in the standard Euclidean plane with coordinates labelled x and y, the x-axis is described by the equation $y = 0$. In other words, we have a function of type $g : \mathrm{Pts}\,E \to \mathrm{Prop}$ defined by $(x, y) \mapsto y = 0$. It's the predicate that defines the line as a subset of the plane. More complicated objects can also be specified as sets of points of E by other functions $\mathrm{Pts}\,E \to \mathrm{Prop}$. Now consider a typical Euclidean symmetry of the line, for example, the symmetry given by the function $t : (x, y) \mapsto (x + 3, y)$. It is compatible with the action of $\mathrm{Vec}\,E$ on $\mathrm{Pts}\,E$, and it sends the line to itself. If we consider the pair (E, g) as an element of the type $\sum_{E:\tilde{\mathbb{E}}}(\mathrm{Pts}\,E \to \mathrm{Prop})$, then, by univalence, we see that the translation t gives rise to an identification of type $(E, g) = (E, g)$.

Now suppose the object to be described is a car, as an object in a 3-dimensional Euclidean space. Then presumably we would like to give more information than just whether a point is inside the car: we may wish to distinguish points of the car by the type of material found there. For example, to distinguish the windshield (made of glass) from the hood (made of steel). Thus, letting M denote the set of materials found in the car, with one extra element for the points not in the car, we may choose to model the car as a function of type $\mathrm{Pts}\,E \to M$.

In order to unify the two examples above into a general framework, one may observe that Prop is a set (with 2 distinguished elements, True and False). That motivates the following definition.

Definition 14.3.1. Let M be a set. A *geometric object* is a pair (E, g) of type $\mathrm{EucObj} :\equiv \sum_{E:\tilde{\mathbb{E}}}(\mathrm{Pts}\,E \to M)$. If one wishes to emphasize the role played by the set M, we may refer to (E, g) as a geometric object *with materials drawn from the set* M.[2] We may also say that (E, g) is a geometric object *in* E. When M is Prop, we will think of the object as the subset of $\mathrm{Pts}\,E$ consisting of those points P such that $g(P)$ holds. ⌟

[2] It would be a mistake to regard a geometric object as a triple (E, M, g), for then symmetries would be allowed to permute the materials.

Exercise 14.3.2. Show that EucObj is a groupoid. ⌟

The exercise above allows us to speak of the symmetry group of a geometric object.

Exercise 14.3.3. Show that the symmetry group of a geometric object in $\mathbb{E}^n$ is a subgroup of $\mathrm{E}(n)$. ⌟

Exercise 14.3.4. Let E be a Euclidean space of dimension n, and let P be a point of E. The subset of $\mathrm{Pts}\,E$ containing just the point P is defined by the predicate $Q \mapsto (Q = P)$. Show that its symmetry group is isomorphic to $\mathrm{O}(n)$. ⌟

One often considers situations in geometry with multiple objects in the same space. For example, one may wish to consider two lines in the plane, or a point and a plane in space. This prompts the following definitions.

DEFINITION 14.3.5. Suppose we are given an parameter type I and a set M_i for each $i \in I$. A *configuration* of geometric objects relative to that data is a Euclidean space E together with a function $p_i : \mathrm{Pts}\, E \to M_i$ for each $i \in I$. Its *consituents* are the geometric objects of the form (E, p_i), for each $i \in I$. If n is a natural number, and we let I be the finite type with n elements, then we may refer to the configuration as a configuration of n objects. ⌟

DEFINITION 14.3.6. Given an type I and a family of geometric objects T_i parametrized by the elements of I, an *arrangement* of the objects is a configuration, also parametrized by the elements of I, whose i-th consituent is merely equal to T_i. ⌟

For example, suppose we consider arrangements consisting of a point and a line in the plane. The arrangements where the point is at a distance d from the line, where $d \geq 0$, are all merely equal to each other, because there is a Euclidean motion that relates any two of them. Hence, in some sense, the arrangements are classified by the set of nonnegative real numbers d. This motivates the following definition.

DEFINITION 14.3.7. Given an parameter type I and a collection of geometric objects T_i parametrized by the elements of I, then an *incidence type* between them is a connected component of the type of all arrangements of the objects. ⌟

14.4 *The icosahedron*

DEFINITION 14.4.1. The *icosahedron* (with side length 2) is the regular solid in standard euclidean three-space $\mathbb{E}^3$ with vertices at cyclic permutations of $(0, \pm 1, \pm\varphi)$, where $\varphi = (1 + \sqrt{5})/2$ is the golden ratio. ⌟

REMARK 14.4.2. The four vertices $(0, \pm 1, \pm\varphi)$ make up a *golden rectangle* with short side length equal to 2. To check that the above vertices really form a regular polyhedron, we just need to calculate the length between to adjacent corners of golden rectangles:

$$\|(0, 1, \varphi) - (1, \varphi, 0)\| = \sqrt{1 + (\varphi - 1)^2 + \varphi^2} = \sqrt{4} = 2 \qquad ⌟$$

14.5 *Frieze patterns*

sec:friezes

See Figures 14.2 and 14.3

14.6 *Incidence geometries and the Levi graph*

14.7 *Affine geometry*

Barycentric calculus. Affine transformations. Euclidean / Hermitian geometry (isometries, conformity...)

fig:icosahedron

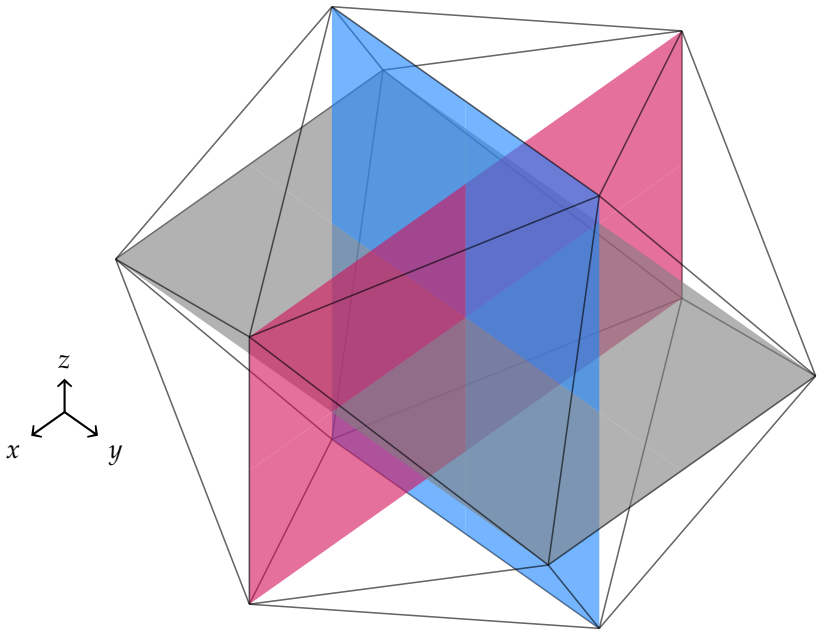


FIGURE 14.1: Icosahedron with its golden rectangles.

fig:friezes

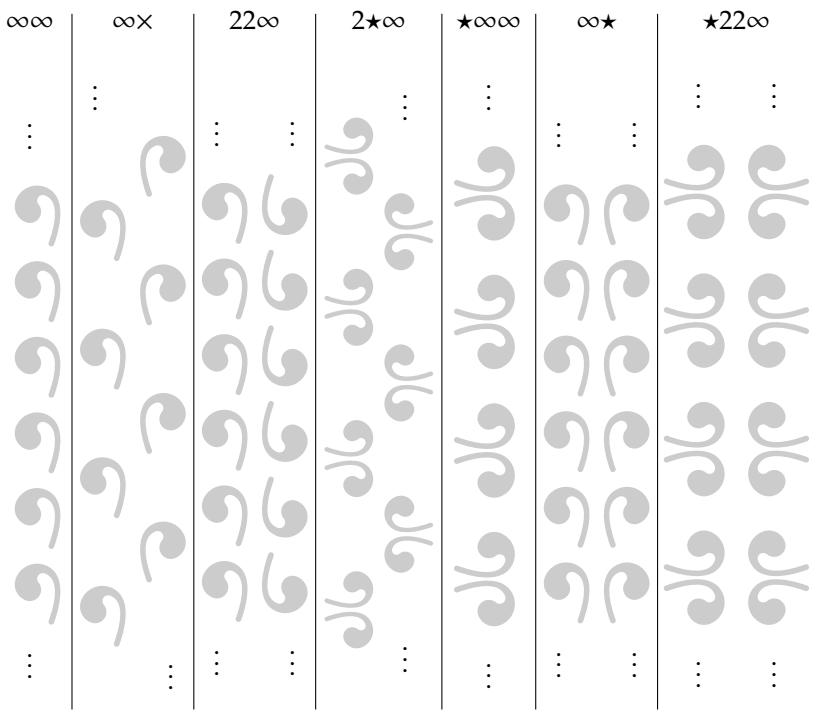


FIGURE 14.2: The seven frieze patterns up to isometry, with their orbifold symbols.

fig:friezes-gen

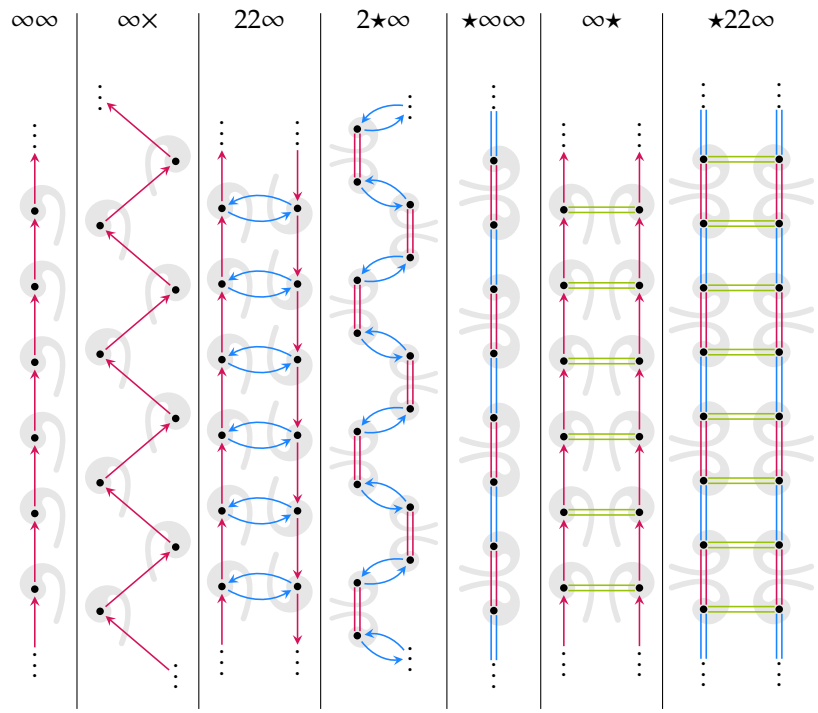


FIGURE 14.3: The seven frieze patterns up to isometry, with their orbifold symbols and superimposed generators.

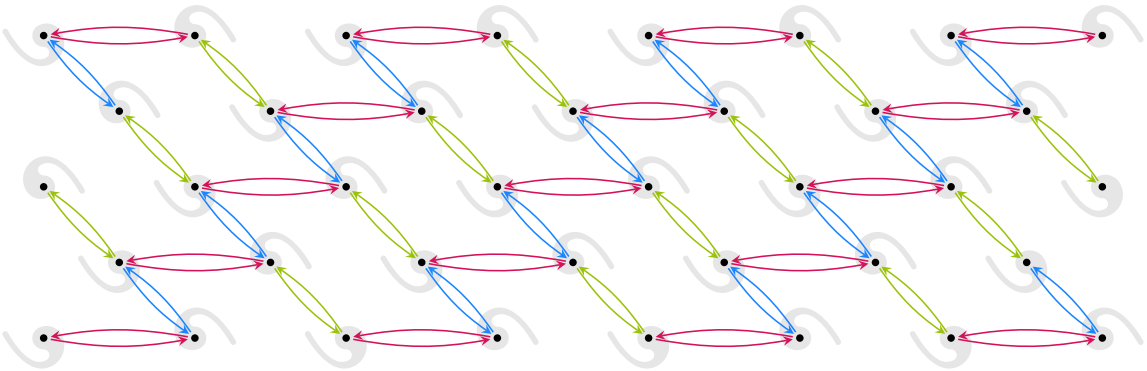


Figure 14.4: Tricycle on carpet.

fig:carpet

14.7.1 *affine planes and Pappus' law*

14.7.2 *affine frames, affine planes*

14.7.3 *the affine group as an automorphism group*

14.7.4 *the affine group as a semidirect product*

14.7.5 *affine properties (parallelism, length ratios)*

14.8 *Inversive geometry (Möbius)*

14.8.1 *residue at a point is affine*

14.8.2 *Miquel's theorem*

14.9 *Projective geometry*

Projective spaces (projective invariance, cross ratio, harmonic range...).
Conics/quadrics. (Classification in low dimensions?)

complex algebraic plane projective curves (tangent complexes, singular points, polar, hessian, ...).

14.9.1 *projective planes*

14.9.2 *projective frames*

14.9.3 *the projective group and projectivities*

14.9.4 *projective properties* (*cross-ratio*)

14.9.5 *fundamental theorem of projective geometry*

15
Galois theory

chap:galois-theory

The goal of Galois theory is to study how the roots of a given polynomial can be distinguished from one another. Take for example $X^2 + 1$ as a polynomial with real coefficients. It has two distincts roots in $\mathbb{C}$, namely i and $-i$. However, an observer, who is limited to the realm of $\mathbb{R}$, can not distinguish between the two. Morally speaking, from the point of view of this observer, the two roots i and $-i$ are pretty much the same. Formally speaking, for any polynomial $Q : \mathbb{R}[X, Y]$, the equation $Q(i, -i) = 0$ is satisfied if and only if $Q(-i, i) = 0$ also. This property is easily understood by noticing that there is a automorphism of fields $\sigma : \mathbb{C} \to \mathbb{C}$ such that $\sigma(i) = -i$ and $\sigma(-i) = i$ which also fixes $\mathbb{R}$. The goal of this chapter is to provide the rigourous framework in which this statement holds. TODO: complete/rewrite the introduction

15.1 *Covering spaces and field extensions*

sec:cover-spac-fields

Recall that a field extension is simply a morphism of fields $i : k \to K$ from a field k to a field K. Given a fixed field k, the type of fields extensions of k is defined as

$$k\backslash\mathbf{Fields} :\equiv \sum_{K : \mathbf{Fields}} \hom_{\mathbf{Fields}}(k, K)$$

def:galois-group

Definition 15.1.1. The Galois group of an extension (K, i) of a field K, denoted $\mathrm{Gal}(K, i)$ or $\mathrm{Gal}(K/k)$ when i is clear from context, is the group $\mathrm{Aut}_{k\backslash\mathbf{Fields}}(K, i)$. ⌟

rem:sip-univalence

Remark 15.1.2. The Structure Identity Principle holds for fields, which means that for $K, L : \mathbf{Fields}$, one has

$$(K = L) \simeq \mathrm{Iso}(K, L)$$

where $\mathrm{Iso}(K, L)$ denotes the type of these equivalences that are homomorphisms of fields. Indeed, if one uses K and L also for the carrier types of the fields, one gets:

$$\begin{aligned}(K = L) \simeq \sum_{p : K =_{\mathcal{U}} L} &(\mathrm{trp}_p(+_K) = +_L) \times (\mathrm{trp}_p(\cdot_K) = \cdot_L) \\ &\times (\mathrm{trp}_p(0_K) = 0_L) \times (\mathrm{trp}_p(1_K) = 1_L)\end{aligned}$$

Any $p : K =_{\mathcal{U}} L$ is the image under univalence of an equivalence $\phi : K \simeq L$,

and then:

$$\begin{aligned}
\mathrm{trp}_p(+_K) &= (x, y) \mapsto \phi(\phi^{-1}(x) +_K \phi^{-1}(y)) \\
\mathrm{trp}_p(\cdot_K) &= (x, y) \mapsto \phi(\phi^{-1}(x) \cdot_K \phi^{-1}(y)) \\
\mathrm{trp}_p(0_K) &= \phi(0_K) \\
\mathrm{trp}_p(1_K) &= \phi(1_K)
\end{aligned}$$

It follows that:

$$\begin{aligned}
(K = L) \simeq \sum_{\phi : K \simeq L} & (\phi(x +_K y) = \phi(x) +_L \phi(y)) \\
& \times (\phi(x \cdot_K y) = \phi(x) \cdot_L \phi(y)) \\
& \times (\phi(0_K) = 0_L) \times (\phi(1_K) = 1_L)
\end{aligned}$$

The type on the right hand side is the same as $\mathrm{Iso}(K, L)$ by definition.

In particular, given an extension (K, i) of K:

$$\mathrm{UGal}(K, i) \simeq \sum_{p : K = K} \mathrm{trp}_p\, i = i \simeq \sum_{\sigma : \mathrm{Iso}(K,K)} \sigma \circ i = i$$

This is how the Galois group of the extension (K, i) is defined in ordinary mathematics. ⌟

Given an extension (K, i) of field k, there is a map of interest:

$$i^* : K\backslash\mathbf{Fields} \to k\backslash\mathbf{Fields}, \quad (L, j) \mapsto (L, ji)$$

Lemma 15.1.3. *The map i^* is a set-bundle.*

iem-field-ext-restriction-set-bundle

Proof. Given a field extension (K', i') in $k\backslash\mathbf{Fields}$, one wants to prove that the fiber over (K', i') is a set. Suppose (L, j) and (L', j') are extensions of K, together with paths $p : (K', i') = (L, ji)$ and $p' : (K', i') = (L', j'i)$. Recall that p and p' are respectively given by equivalences $\pi : K' = L$ and $\pi' : K' = L'$ such that $\pi i' = ji$ and $\pi' i' = j' i$. A path from $((L, j), p)$ to $((L', j'), p')$ in the fiber over (K', i') is given a path $q : (L, j) = (L', j')$ in $K\backslash\mathbf{Fields}$ such that $\mathrm{trp}_q\, p = p'$. However, such a path q is the data of an equivalence $\varphi : L = L'$ such that $\varphi j = j'$, and then the condition $\mathrm{trp}_q\, p = p'$ translates as $\varphi\pi = \pi'$. So it shows that φ is necessarily equal to $\pi'\pi^{-1}$, hence is unique. □

The fiber of this map at a given extension (L, j) of k is:

$$\begin{aligned}
(i^*)^{-1}(L, j) &\simeq \sum_{L' : \mathbf{Fields}} \sum_{j' : K \to L'} (L, j) = (L', j'i) \\
&\simeq \sum_{L' : \mathbf{Fields}} \sum_{j' : K \to L'} \sum_{p : L = L'} pj = j'i \\
&\simeq \sum_{j' : K \to L} j = j'i \\
&\simeq \hom_k(K, L)
\end{aligned}$$

where the last type denotes the type of homomorphisms of k-algebra (the structure of K and L being given by i and j respectively).

In particular, the map $t : \mathrm{UGal}(K, i) \to (i^*)^{-1}(K, i)$ mapping g to $\mathrm{trp}_g(\mathrm{id}_K)$ identifies with the inclusion of the k-automorphisms of K into the k-endomorphisms of K.

TODO: write a section on polynomials in chapter 12

DEFINITION 15.1.4. Given an extension $i : k \to K$, an element $\alpha : K$ is algebraic if α is merely a root of a polynomial with coefficients in k. That is if the following proposition holds:

$$\| \sum_{n:\mathbb{N}} \sum_{a:n+1\to k} i(a(0)) + i(a(1))\alpha + \cdots + i(a(n))\alpha^n = 0 \|$$

⌟

DEFINITION 15.1.5. A field extension (K, i) is said to be algebraic when each $a : K$ is algebraic. ⌟

REMARK 15.1.6. Note that when the extension (K, i) is algebraic, then t is an equivalence. However, the converse is false, as shown by the non-algebraic extension $\mathbb{Q} \hookrightarrow \mathbb{R}$. We will prove that every $\mathbb{Q}$-endomorphism of $\mathbb{R}$ is the identity function. Indeed, any $\mathbb{Q}$-endormorphism $\varphi : \mathbb{R} \to \mathbb{R}$ is linear and sends squares to squares, hence is non-decreasing. Let us now take an irrational number $\alpha : \mathbb{R}$. For any rational $p, q : \mathbb{Q}$ such that $p < \alpha < q$, then $p = \varphi(p) < \varphi(\alpha) < \varphi(q) = q$. Hence $\varphi(\alpha)$ is in any rational interval that α is. One deduces $\varphi(\alpha) = \alpha$. ⌟

DEFINITION 15.1.7. A field extension $i : k \to K$ is said finite when K as a k-vector space, the structure of which is given by i, is of finite dimension. In that case, the dimension is called the degree of i, denoted $[(K, i)]$ or $[K : k]$ when i is clear from context. ⌟

15.2 *Intermediate extensions and subgroups*

Given two extensions $i : k \to K$ and $j : K \to L$, the map i^* can be seen as a pointed map

$$i^* : \mathrm{BGal}(L, j) \to \mathrm{BGal}(L, ji), \quad x \mapsto x \circ i.$$

Then, through Lemma 15.1.3, i^* presents $\mathrm{Gal}(L, j)$ as a subgroup of $\mathrm{Gal}(L, ji)$. One goal of Galois theory is to characterize those extensions $i' : k \to L$ for which all subgroups of $\mathrm{Gal}(L, i')$ arise in this way.

Given any extension $i : k \to L$, there is an obivous $\mathrm{Gal}(L, i)$-set X given by

$$(L', i') \mapsto L'.$$

For a pointed connected set-bundle $g : B \to \mathrm{BGal}(L, i)$, one can consider the type of fixed points of the $\underline{\Omega}B$-set Xf:

$$K :\equiv (Xg)^{\underline{\Omega}B} \equiv \prod_{x:B} X(g(x))$$

It is a set, which can be equipped with a field structure, defined pointwise. Morevover, if one denotes b for the distinguished point of B, and (L'', j'') for $g(b)$, then, because g is pointed, one has a path $p : L = L''$ such that $pi' = j''$. There are fields extensions $i' : k \to K$ and $j' : K \to L$ given by:

$$i'(a) :\equiv x \mapsto \mathrm{snd}(g(x))(a), \quad j'(f) :\equiv p^{-1} f(b)$$

In particular, for all $a : k$, $j'i'(a) = p^{-1}\,\mathrm{snd}(g(b))(a) = p^{-1}j''(a) = i'(a)$.

Galois theory is interested in the settings when these two contructions are inverse from each other.

defn:algebraic-element
defn:algebraic-extension
aic-endomorphisms-are-automorphisms
defn:degree-field-extension

15.3 *separable/normal/etc.*

15.4 *fundamental theorem*

sec:cover-spac-fields-1

sec:fundamental-theorem

A
Historical remarks

ch:groupHistory

Here we briefly sketch some of the history of groups. See the book by Wussing[1] for a detailed account, as well as the shorter survey by Kleiner[2]. There's also the book by Yaglom[3].

Some waypoints we might mention include:

- Early nineteenth century geometry, the rise of projective geometry, Möbius and Plücker
- Early group theory in number theory, forms, power residues, Euler and Gauss.
- Permutation groups, Lagrange and Cauchy, leading (via Ruffini) to Abel and Galois.
- Liouville and Jordan[4] ruminating on Galois.
- Cayley, Klein and the Erlangen Program[5].
- Lie and differentiation.
- von Dyck and Hölder.
- J.H.C. Whitehead and crossed modules.
- Artin and Schreier theory.
- Algebraic groups (Borel and Chevalley et al.)
- Feit-Thompson and the classification of finite simple groups.
- Grothendieck and the homotopy hypothesis.
- Voevodsky and univalence.

[1] Hans Wussing. *The genesis of the abstract group concept*. A contribution to the history of the origin of abstract group theory, Translated from the German by Abe Shenitzer and Hardy Grant. MIT Press, Cambridge, MA, 1984, p. 331.

[2] Israel Kleiner. "The evolution of group theory: a brief survey". In: *Math. Mag.* 59.4 (1986), pp. 195–215. DOI: 10.2307/2690312.

[3] I. M. Yaglom. *Felix Klein and Sophus Lie. Evolution of the idea of symmetry in the nineteenth century.* Transl. from the Russian by Sergei Sossinsky. Ed. by Hardy Grant and Abe Shenitzer. Birkhäuser Boston, Inc., Boston, MA, 1988, pp. xii+237.

[4] Camille Jordan. *Traité des substitutions et des équations algébriques*. Les Grands Classiques Gauthier-Villars. Reprint of the 1870 original. Éditions Jacques Gabay, Sceaux, 1989, pp. xvi+670.

[5] Felix Klein. "Vergleichende Betrachtungen über neuere geometrische Forschungen". In: *Math. Ann.* 43.1 (1893), pp. 63–100. DOI: 10.1007/BF01446615.

B

Metamathematical remarks

app:metamath

Metamathematics is the study of mathematical theories as mathematical objects in themselves. This book is primarily a mathematical theory of symmetries. Occasionally, however, we have made statements like "the law of the excluded middle is not provable in our theory". This is a statement *about*, and not *in*, the type theory of this book. As such it is a metamathematical statement.

Sometimes it is possible to encode statements about a theory in the language of the theory itself. Even if true, the encoded metamathematical statement can be unprovable in the theory itself. The most famous example is Gödel's second incompleteness theorem.[1]. Gödel encoded, for any theory T extending Peano Arithmetic and satisfying some general assumptions, the statement that T is consistent as a statement $\mathrm{Con}(T)$ in Peano Arithmetic. Then he showed that $\mathrm{Con}(T)$ is not provable in T.

We say that a metamathematical statement about a theory T is *internally* provable if its encoding is provable in T. For example, the metamathematical statement "if P is unprovable in T, then T is consistent" is internally provable in T, for any T that satisfies the assumptions of Gödel's second incompleteness theorem.

The type theory in this book satisfies the assumptions of Gödel's second incompleteness theorem, which include, of course, the assumption that T is consistent. Thus there is no hope that we can prove the consistency of our type theory internally. Moreover, by the previous paragraph, we must be prepared that no unprovability statement can be proved internally.

[TODO For consistency of UA, LEM, etc, refer to simplicial set model[6]. For unprovability of LEM, refer to cubical set model[7].]

One property of type theory that we will use is *canonicity*. We call an expression *closed* if it does not contain free variables. One example of canonicity is that every closed expression of type $\mathbb{N}$ is a *numeral*, that is, either 0 or $S(n)$ for some numeral n. Another example of canonicity is that every closed expression of type $L \amalg R$ is either of the form inl_l for some $l : L$ or of the form inr_r for some $r : R$.

Both examples of canonicity above are clearly related to the inductive definitions of the types involved: they are expressed in terms of the constructors of the respective types. One may ask what canonicity then means for the empty type False, defined in Section 2.12.1 as the inductive type with no constructors at all. The answer is that canonicity for False means that there cannot be a closed expression of type False. But this actually means that our type theory is consistent! Therefore we cannot prove general canonicity internally.

We leave aside that this sometimes can be done in different ways. Historically, the first way was by "Gödel-numbering": encoding all bits of syntax, including statements, as natural numbers, so that the constructions and deductions of the theory correspond to definable operations on the encoding numbers. In type theory, there are usually much more perspicacious ways of encoding mathematical theories using types and type families.

[1] The original reference is Gödel[2], translated into English in van Heijenoort[3]. For an accessible introduction, see for instance Franzén[4] or Smullyan[5].

[2] Kurt Gödel. "Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I". in: *Monatsh. Math. Phys.* 38.1 (1931), pp. 173–198. DOI: 10.1007/BF01700692.

[3] Jean van Heijenoort. *From Frege to Gödel: A Source Book in Mathematical Logic, 1879–1931*. Source Books in the History of the Sciences. Harvard University Press, 2002, pp. xii+661.

[4] Torkel Franzén. *Gödel's Theorem: An Incomplete Guide to Its Use and Abuse*. A. K. Peters, 2005, pp. x+172.

[5] Raymond M. Smullyan. *Gödel's incompleteness theorems*. Vol. 19. Oxford Logic Guides. The Clarendon Press, Oxford University Press, New York, 1992, pp. xvi+139.

[6] Krzysztof Kapulkin and Peter LeFanu Lumsdaine. "The simplicial model of Univalent Foundations (after Voevodsky)". In: *Journal of the European Mathematical Society* 23.6 (Mar. 2021), pp. 2071–2126. DOI: 10.4171/jems/1050.

[7] Marc Bezem, Thierry Coquand, and Simon Huber. "A model of type theory in cubical sets". In: *19th International Conference on Types for Proofs and Programs*. Vol. 26. LIPIcs. Leibniz Int. Proc. Inform. Schloss Dagstuhl. Leibniz-Zent. Inform., Wadern, 2014, pp. 107–128. DOI: 10.4230/LIPIcs.TYPES.2013.107.

[TODO no canonical forms: $x : \mathbb{N}$, $\mathrm{trp}^{P}_{\mathrm{ua(id)}}(0) : \mathbb{N}$, with $P :\equiv (p : \mathrm{True} \mapsto \mathbb{N})$ and (problematic) $\mathrm{trp}^{Q}_{\circlearrowleft}(0) : \mathbb{N}$ with $Q :\equiv (z : \mathrm{S}^1 \mapsto \mathbb{N})$.]

[TODO A second important property of our theory is that one can compute canonical forms.]

B.1 *Equality by definition*

sec:defeq

B.1.1 *Basics*

sec:defeq-basics

The concept of definition was introduced in Section 2.2, together with what it means to be *the same by definition*. Being the same by definition (NB appears for the first time on p. 26!) is a relationship between syntactic expressions. In this section we provide more details about this relationship.

There are four basic forms of equality by definition:

it:exp-defeq

(1) Resulting from making an explicit definition, e.g., $1 :\equiv \mathrm{succ}(0)$, after which we have $1 \equiv \mathrm{succ}(0)$;[8]

it:imp-defeq

(2) Resulting from making an implicit definition, like we do in inductive definitions, e.g., $n + 0 :\equiv n$ and $n + \mathrm{succ}(m) :\equiv \mathrm{succ}(n + m)$, after which we have $n + 0 \equiv n$ and $n + \mathrm{succ}(m) \equiv \mathrm{succ}(n + m)$;

it:beta

(3) Simplifying the application of an explicitly defined function to an argument, e.g., $(x \mapsto e_x)(a) \equiv e_a$;

it:eta

(4) Simplifying $(x \mapsto e_x)$ to f when e_x is the application of the function f to the variable x, e.g., $(x \mapsto S(x)) \equiv S$.

[8] The notation $:\equiv$ tells the reader that we make a definition (or reminds the reader that this definition has been made).

Equality by definition is the *congruence closure* of these four basic forms, that is, the smallest reflexive, symmetric, transitive and congruent relation that contains all instances of the four basic forms. Here a congruent relation is a relation that is closed under all syntactic operations of type theory. One such operation is substitution, so that we get from the examples above that, e.g., $1 + 0 \equiv 1$ and $n + \mathrm{succ}(\mathrm{succ}(m)) \equiv \mathrm{succ}(n + \mathrm{succ}(m))$. Another important operation is application. For example, we can apply succ to each of the sides of $n + \mathrm{succ}(m) \equiv \mathrm{succ}(n + m)$ and get $\mathrm{succ}(n + \mathrm{succ}(m)) \equiv \mathrm{succ}(\mathrm{succ}(n + m))$, and also $n + \mathrm{succ}(\mathrm{succ}(m)) \equiv \mathrm{succ}(\mathrm{succ}(n + m))$ by transitivity.

Let's elaborate $\mathrm{id} \circ f \equiv f$ claimed on page 12. The definitions used on the left hand side are $\mathrm{id} :\equiv (y \mapsto y)$ and $g \circ f :\equiv (x \mapsto g(f(x)))$. In the latter definition we substitute id for g and get $\mathrm{id} \circ f \equiv (x \mapsto \mathrm{id}(f(x)))$. Unfolding id we get $(x \mapsto \mathrm{id}(f(x))) \equiv (x \mapsto (y \mapsto y)(f(x)))$. Applying (3) we can substitute $f(x)$ for $(y \mapsto y)(f(x)))$ and get $(x \mapsto (y \mapsto y)(f(x))) \equiv (x \mapsto f(x))$. By (4) the right hand side is equal to f by definition. Indeed $\mathrm{id} \circ f \equiv f$ by transitivity.

Equality by definition is also relevant for typing. For example, let $A : \mathcal{U}$ and $P : A \to \mathcal{U}$. If $B \equiv A$, then $(B \to \mathcal{U}) \equiv (A \to \mathcal{U})$ by congruence, and also $P : B \to \mathcal{U}$, and even $\prod_{x:B} P(x) \equiv \prod_{x:A} P(x)$.

B.1.2 *Deciding equality by definition* (*not updated yet*)

sec:defeq-computation

By a *decision procedure* we mean a terminating algorithmic procedure that answers a yes/no question. Although it is possible to enumerate

all true equalities by definition, this does not give a test that answers whether or not a given instance $e \equiv e'$ holds. In particular when $e \equiv e'$ does not hold, such an enumeration will not terminate. A test of equality by definition is important for type checking, as the examples in the last paragraph of the previous section show.

A better approach to a test of equality by definition is the following. First direct the four basic forms of equality by definition from left to right as they are given.[9] For the first two forms this can be viewed as unfolding definitions, and for the last two forms as simplifying function application and (unnecessary) abstraction, respectively. This defines a basic reduction relation, and we write $e \to e'$ if e' can be obtained by a basic reduction of a subexpression in e. The reflexive transitive closure of $\to$ is denoted by $\to^*$. The symmetric closure of $\to^*$ coincides with $\equiv$.

[9] TODO: think about the last, η.

We mention a few important properties of the relations $\to$, $\to^*$ and $\equiv$. The first is called the Church–Rosser property, and states that, if $e \equiv e'$, then there is an expression c such that $e \to^* c$ and $e' \to^* c$. The second is called type safety and states that, if $e : T$ and $e \to e'$, then also $e' : T$. The third is called termination and states that for well-typed expressions e there is no infinite reduction sequence starting with e. The proofs of Church–Rosser and type safety are long and tedious, but pose no essential difficulties. For a non-trivial type theory such as in this book the last property, termination, is extremely difficult and has not been carried out in full detail. The closest come results on the Coq[10] (TODO: find good reference).

[10] The Coq Development Team. *The Coq Proof Assistant*. Available at https://coq.inria.fr/.

Testing whether or not two given well-typed terms e and e' are equal by definition can now be done by reducing them with $\to$ until one reaches irreducible expressions n and n' such that $e \to^* n$ and $e' \to^* n'$, and then comparing n and n'. Now we have: $e \equiv e'$ iff $n \equiv n'$ iff (by Church–Rosser) there exists a c such that $n \to^* c$ and $n' \to^* c$. Since n and n' are irreducible the latter is equivalent to n and n' being identical syntactic expressions.

B.2 *The Limited Principle of Omniscience*

sec:LPO

rem:LPO-solves-halting problem

Remark B.2.1. Recall the Limited Principle of Omniscience (LPO), **??** 3.6.22: for any function $P : \mathbb{N} \to 2$, either there is a smallest number $n_0 : \mathbb{N}$ such that $P(n_0) = 1$, or P is a constant function with value 0. We will show that LPO is not provable in our theory.

The argument is based on the halting problem: given a Turing machine M and an input n, determine whether M halts on n. It is known that the halting problem cannot be solved by an algorithm that can be implemented on a Turing machine.[11]

[11] It's commonly accepted that every algorithm *can* be thus implemented.

We use a few more facts from computability theory. First, Turing machines can be enumerated. We denote the nth Turing machine M_n, so we can list the Turing machines in order: $M_0, M_1, \ldots$. Secondly, there exists a function $T(e, n, k)$ such that $T(e, n, k) = 1$ if M_e halts on input n in at most k steps, and $T(e, n, k) = 0$ otherwise. This function T can be implemented in our theory.

Towards a contradiction, assume we have a closed proof t of LPO in our theory. We assume as well that t does not depend on any axiom.[12] It is clear that $k \mapsto T(e, n, k)$ is a constant function with value 0 if and only

[12] It is possible to weaken the notion of canonicity so that the argument still works even if the proof t uses the Univalence Axiom. Of course, the argument must fail if we allow t to use LEM!

if M_e does not halt on input n. Now consider $t(k \mapsto T(e,n,k))$, which is an element of a type of the form $L \amalg R$.

We now explain how to solve the halting problem. Let e and n be arbitrary numerals. Then $t(k \mapsto T(e,n,k))$ is a closed element of $L \amalg R$. Hence we can compute its canonical form. If $t(k \mapsto T(e,n,k)) \equiv \mathrm{inr}_r$ for some $r : R$, then $k \mapsto T(e,n,k)$ is a constant function with value 0, and M_e does not halt on input n. If $t(k \mapsto T(e,n,k)) \equiv \mathrm{inl}_l$ for some $l : L$, then M_e does halt on input n. Thus we have an algorithm to solve the halting problem for all e and n. Since this is impossible, we have refuted the assumption that there is a closed proof t of LPO in our theory. ⌟

B.3 *Topology*

sec:topology

In this section we will explain how our intuition about types relates to our intuition about topological spaces.

INSERT AN INTRODUCTORY PARAGRAPH HERE. [Intuitively, the types of type theory can be modeled by topological spaces, and elements as points thereof. However, this is not so easy to achieve, and the first model of homotopy theory theory was in simplicial sets. Topological spaces and simplicial sets are both *models* of homotopy types. And by a lucky coincidence, it makes sense to say that homotopy type theory is a theory of homotopy types.] Some references include: Hatcher, May, and May and Ponto[13]

injectionsurjectionisnotwhatyouthink

Remark B.3.1. Our definitions of injections and surjections are lifted directly from the intuition about sets. However, types need not be sets, and thinking of types as spaces may at this point lead to a slight confusion.

The real line is contractible and the inclusion of the discrete subspace $\{0,1\}$ is, well, an inclusion (of sets, which is the same thing as an inclusion of spaces). However, $\{0,1\}$ is not connected, seemingly contradicting the next result.

This apparent contradiction is resolved once one recalls the myopic nature of our setup: the contractibility of the real line means that "all real numbers are identical", and *our* "preimage of 3.25" is not a proposition: it contains *both* 0 and 1. Hence "$\{0,1\} \subseteq \mathbb{R}$" would not count as an injection in our sense.

sec:cohesive

We should actually have been more precise above: we were referring to the *homotopy type* of the real line, rather than the real line itself.[14] We shall later (in the chapters on geometry) make plenty of use of the latter, which is as usual a set with uncountably many elements. ⌟

B.4 *Choice for finite sets* (†)

sec:choicefin

This section is a short overview of how group theory is involved in relating different choice principles for families of finite sets. A paradigmatic case is that if we have choice for all families of 2-element sets, then we have choice for all families of 4-element sets.[16]

The axiom of choice is a principle that we may add to our type theory (it holds in the standard model), but there are many models where it doesn't hold.

[13] Allen Hatcher. *Algebraic Topology*. Cambridge University Press, 2001, pp. xii+551. ISBN: 978-0-521-79540-1. URL: https://pi.math.cornell.edu/~hatcher/AT/AT.pdf; J. P. May. *A concise course in algebraic topology*. Chicago Lectures in Mathematics. University of Chicago Press, Chicago, IL, 1999; J. P. May and K. Ponto. *More concise algebraic topology*. Chicago Lectures in Mathematics. Localization, completion, and model categories. University of Chicago Press, Chicago, IL, 2012.

[14] We don't define this formally here, see Shulman[15] for a synthetic account. The idea is that the homotopy type $\mathrm{h}(X)$ of a type X has a map from X, $\iota : X \to \mathrm{h}(X)$, and any continuous function $f : [0,1] \to X$ gives rise to a path $\iota(f(0)) = \iota(f(1))$ in $\mathrm{h}(X)$.

[15] Michael Shulman. "Brouwer's fixed-point theorem in real-cohesive homotopy type theory". In: *Mathematical Structures in Computer Science* 28.6 (2018), pp. 856–941. DOI: 10.1017/S0960129517000147. arXiv: 1509.07584.

[16] This is due to Tarski, see Jech[17], p. 107.

[17] Thomas J. Jech. *The axiom of choice*. Studies in Logic and the Foundations of Mathematics, Vol. 75. North-Holland Publishing Co., Amsterdam, 1973, pp. xi+202.

pri:ac

Principle B.4.1 (The Axiom of Choice). For every set X and every family of *non-empty* sets $P : X \to \mathrm{Set}_{\neq\emptyset}$, there exists an dependent function of type $\prod_{x:X} P(x)$. In other terms, for any set X and any family of sets $P : X \to \mathrm{Set}$, we have

eq:ac-impl

$$\prod_{x:X} \|P(x)\| \to \left\| \prod_{x:X} P(x) \right\|. \tag{B.4.1}$$

⌟

Remark B.4.2. We have an equivalence between the Pi-type $\prod_{x:X} P(x)$ and the type of sections of the projection map $\mathrm{pr}_1 : \sum_{x:X} P(x) \to X$, under which families of non-empty sets correspond to surjections between sets (using that X is a set). Thus, the axiom of choice equivalently says that any surjection between sets admits a section.

Because of this equivalence, we'll sometimes also call elements of the Pi-type *sections*. ⌟

The following is usually called Diaconescu's theorem[18] or the Goodman–Myhill theorem[19], but it was first observed in a problem in Bishop's book on constructive analysis[20].

Theorem B.4.3. *The axiom of choice implies the law of the excluded middle,* **??** 2.18.2.

Proof. Let P be a proposition, and consider the quotient map $q : 2 \to 2/\!\sim$, where $\sim$ is the equivalence relation on 2 satisfying $(0 \sim 1) = P$. Like any quotient map, q is surjective, so by the axiom of choice, and because our goal is a proposition, it has a section $s : 2/\!\sim \to 2$. That is, we also have $q \circ s = \mathrm{id}$.

Using decidable equality in 2, check whether $s([0])$ and $s([1])$ are equal or not.

If they are, then we get the chain of identifications $[0] = q(s([0])) = q(s([1])) = [1]$, so P holds.

If they aren't, then assuming P leads to a contradiction, meaning $\neg P$ holds. □

We'll now define some restricted variants of the axiom of choice, that however are not always true, and our goal is to see how they relate to each other and to other principles.

Definition B.4.4. Let AC denote the full axiom of choice, as in **??** B.4.1. If we fix the set X, and consider (B.4.1) for arbitrary families $P : X \to \mathrm{Set}$, we call this the *X-local axiom of choice*, denoted X-AC.

If we restrict P to take values in n-element sets, for some $n : \mathbb{N}$, we denote the resulting principle $\mathrm{AC}(n)$. (That is, here we consider families $P : X \to \mathrm{B}\Sigma_n$.)

If we both fix X and restrict to families of n-element sets, we denote the resulting principle X-$\mathrm{AC}(n)$. ⌟

Exercise B.4.5. Show that X-AC is always true whenever X is a finite set. ⌟

lem:ac-impl-triv-coh-sets

Lemma B.4.6. *If X-AC holds for a set X, then $\|X \to BG\|_0$ is contractible for any group G.*

Proof. Suppose we have a map $f : X \to BG$. We need to show that f is merely equal to the constant map. Consider the corresponding family of sets consisting of the underlying sets of the G-torsors represented by

[18] Radu Diaconescu. "Axiom of choice and complementation". In: *Proc. Amer. Math. Soc.* 51 (1975), pp. 176–178.

[19] N. Goodman and J. Myhill. "Choice implies excluded middle". In: *Z. Math. Logik Grundlagen Math.* 24.5 (1978), p. 461. DOI: 10.1002/malq.19780242514.

[20] Errett Bishop. *Foundations of constructive analysis*. McGraw-Hill Book Co., New York, 1967, pp. xiii+370.

In fancier language, this says that the axiom of choice implies that all cohomology sets $\mathrm{H}^1(X, G)$ are trivial.

$f(x) : BG$, for $x : X$. That is, define $P : X \to \text{Set}$ by setting $P(x) :\equiv (\text{sh}_G = f(x))$. Since BG is connected, this is a family of non-empty sets, so by the axiom of choice for families over X, there exists a section. Since we're proving a proposition, let $s : \prod_{x:X}(\text{sh}_G = f(x))$ be a section. Then s identifies f with the constant map, as desired. □

We might wonder what happens if we consider general ∞-groups G in Lemma B.4.6. Then the underlying type of a G-torsor is no longer a set, but can be any type. Correspondingly, we need an even stronger version of the axiom of choice, where the family P is allowed to be arbitrary. Let AC_∞ denote this untruncated axiom of choice, and let $X\text{-AC}_\infty$ denote the local version, fixing a set X. This is connected to another principle, which is much more constructive, yet still not true in all models.

pri:sc

Principle B.4.7 (Sets Cover). For any type A, there exists a set X together with a surjection $X \to A$. ⌟

We abbreviate this as SC.

Exercise B.4.8. Prove that the untruncated axiom of choice, AC_∞, is equivalent to the conjunction of the standard axiom of choice, AC, and the principle that sets cover, SC. ⌟

Exercise B.4.9. Prove that we cannot relax the requirement that X is a set in the axiom of choice. Specifically, prove that $\text{S}^1\text{-AC}(2)$ is false ⌟

We now come to the analogue of Lemma B.4.6 for arbitrary ∞-groups.

Exercise B.4.10. Prove that if the untruncated X-local axiom of choice, $X\text{-AC}_\infty$, holds for a set X, then $\|X \to BG\|_0$ is contractible for all ∞-groups G. ⌟

We now discuss two partial converses to Lemma B.4.6, both due to Blass[21].

thm:Blass-1

Theorem B.4.11 (Blass). *Let X be a set such that $\|X \to BG\|_0$ is contractible for all groups G. Then every family of non-empty sets over X, $P : X \to \text{Set}$, that factors through a connected component of* Set*, merely admits a section.*

Proof. We suppose $P : X \to \text{Set}$ is such that all the sets $P(x)$ have the same size, i.e., the function P factors through $\text{BAut}(S)$ for some non-empty set S. This in turn means that we have a function $h : X \to BG$, where $G :\equiv \text{Aut}(S)$, with $P = \text{pr}_1 \circ h$, where $\text{pr}_1 : \text{BAut}(S) = \sum_{A:\text{Set}}\|S \simeq A\| \to \text{Set}$ is the projection.

By assumption, h is merely equal to the constant family. But since we are proving a proposition, we may assume that h *is* constant, so P is the constant family at S. And this has a section since S is non-empty. □

Obviously, the same argument works if we consider all ∞-groups G and families of types that are all equivalent. For the second partial converse, we look at decidable sets.

thm:Blass-2

Theorem B.4.12 (Blass). *Let X be a decidable set such that $\|X \to BG\|_0$ is contractible for all groups G. Then every family of non-empty decidable sets over X merely admits a section.*[22]

Proof. Equivalently, consider a surjection $p : Y \to X$, where X and Y are decidable sets, and let C be the higher inductive type with constructors $c : C$, $f : X \to C$, and $k : \prod_{y:Y}(c = f(p(y))$.[23] Using the same kind of

[21] Andreas Blass. "Cohomology detects failures of the axiom of choice". In: *Trans. Amer. Math. Soc.* 279.1 (1983), pp. 257–269. DOI: 10.2307/1999384.

[22] We might call this conclusion $X\text{-AC}^{\text{dec}}$.

[23] This kind of higher inductive type is also known as a pushout, and its constructors fit together to give a commutative square:

$$\begin{array}{ccc} Y & \xrightarrow{p} & X \\ \downarrow & & \downarrow f \\ \mathbb{1} & \xrightarrow[c]{} & C \end{array}$$

argument as in Lemma 8.6.7 and Theorem 8.7.8, we can show, using decidability of equality in X and Y, that the identity type $c =_C f(x)$ is equivalent to a type of reduced words over $Y \amalg Y$. In particular, C is a groupoid, and it's easy to check that it's connected. Hence we can form the group $G :\equiv \underline{\Omega}(C, c)$.

By assumption, the map f is merely equal to the constant map, so since we're proving a proposition, we may assume we have a family of elements $h(x) : c = f(x)$, for $x : X$. Taking for each x the last y in the corresponding reduced word, we get a family of elements $s(x) : Y$ such that $p(s(x)) = x$, but this is precisely the section we wanted. □

It seems to be an open problem, whether we can do without the decidability assumption, i.e., whether the converse of Lemma B.4.6 holds generally.

Now we turn as promised to the connections between the various local choice principles X-AC(n). The simplest example is the following.

thm:1AC-2-3-4

THEOREM B.4.13. *Let X be any set. Then X-AC(4) follows from X-AC(2) and X-AC(3).*

Proof. Let $P : X \to \mathrm{B}\Sigma_4$ be a family of 4-element sets over X. Consider the map $Bf : \mathrm{B}\Sigma_4 \to \mathrm{B}\Sigma_3$ that maps a 4-element set to the 3-element set of its $2 + 2$ partitions. Choose a section of $Bf \circ P$ by X-AC(3). Now use X-AC(2) twice to choose for each chosen partition first one of the 2-element parts, and secondly one of the 2 elements in each chosen part. □

We now look a bit more closely at what happened in this proof, so as to better understand the general theorem. The key idea is the concept of "reduction of the structure group".

[TODO, Elaborate: For a family of n-element sets over a base type X, $P : X \to \mathrm{B}\Sigma_n$, there is a section if and only if there is a " to a subgroup of Σ_n, whose action on the standard n-element set, $\mathbb{m}$, has a fixed point.]

Now we return to the local case, and we give the general sufficient condition that ensures that X-AC(n) follows from X-AC(z) for each $z : Z$, where Z is a finite subset of $\mathbb{N}$.

DEFINITION B.4.14. The condition $L(Z, n)$ is that for every finite subgroup G of Σ_n that acts on $\mathbb{m}$ without fixed points, there exists finitely many proper, finite subgroups $K_1, \cdots, K_r$ of G such that the sum of the indices,

$$|G : H_1| + \cdots + |G : H_r|,$$

lies in Z. ⌟

We now turn to the global case, where we can change the base set. Here the basic case is Tarski's result alluded to above, which shows that we don't need choice for 3-element sets, in contrast to the local case, Theorem B.4.13.

THEOREM B.4.15. AC(2) *implies* AC(4).

Proof. Let $P : X \to \mathrm{B}\Sigma_4$ be a family of 4-element sets indexed by a set X. Consider the new set Y consisting of all 2-element subsets of $P(x)$, as x runs over X,

$$Y :\equiv \sum_{x:X} [P(x)]^2.$$

The set Y carries a canonical family of 2-element sets, so we may choose an element of each. In other words, we have chosen an element of each of the 6 different 2-element subsets of each of the 4-element sets $P(x)$.

For every $a : P(x)$, let $q_x(a)$ be the number of 2-element subsets $\{a, b\}$ of $P(x)$ with $b \neq a$ for which a is the chosen element.

Define the sets $B(x) :\equiv \{\, a : P(x) \mid q_x(a) \text{ is a minimum of } q_x \,\}$, and remember that they are subsets of $P(x)$. This determines a decomposition of X into three parts $X = X_1 + X_2 + X_3$, where

$$X_i :\equiv \sum_{x:X} (B(x) \text{ has cardinality } i), \quad i = 1, 2, 3.$$

Note that $B(x)$ can't be all of $P(x)$, since that would mean that q_x is constant, and that is impossible, since the sum of q_x over the 4-element $P(x)$ is 6.

Over X_1, we get a section of P by picking the unique element in $B(x)$.

Over X_3, we get a section of P by picking the unique element *not* in $B(x)$.

Over X_2, we get a section of P by picking the already chosen element of the 2-element set $B(x)$. □

The following appears as Theorem 6 in Blass[24].

[24] Blass, "Cohomology detects failures of the axiom of choice".

THEOREM B.4.16. *Assume* $\|X \to \mathrm{BC}_n\|_0$ *is contractible for all sets* X *and positive integers* n. *Then* $\mathrm{AC}(n)$ *holds for all* n.

Proof. We use well-founded induction on n, the case $n \equiv 1$ being trivial.

Let $P : X \to \mathrm{B}\Sigma_n$ be a family of n-element sets, and let $Y :\equiv \sum_{x:X} P(x)$ be the domain set of this set bundle. Consider the family $Q : Y \to \mathrm{B}\Sigma_{n-1}$ defined by

$$Q((x, y)) :\equiv \{\, y' : P(x) \mid y \neq y' \,\} = P(x) \setminus \{y\},$$

where we use the fact that $P(x)$ is an n-element set and thus has decidable equality, so we can form the $(n-1)$-element complement $P(x) \setminus \{y\}$.

By induction hypothesis, we get a section of Q, which we can express as a family of functions

$$f : \prod_{x:X} \big(P(x) \to P(x)\big)$$

where $f_x(y) \neq y$ for all x, y. Since $P(x)$ is an n-element set, we can decide whether f_x is a permutation or not, and if so, whether it is a cyclic permutation. We have thus obtained a partition $X = X_1 + X_2 + X_3$, where

$$X_1 :\equiv \{\, x : X \mid f_x \text{ is not a permutation} \,\},$$
$$X_2 :\equiv \{\, x : X \mid f_x \text{ is a non-cyclic permutation} \,\},$$
$$X_3 :\equiv \{\, x : X \mid f_x \text{ is a cyclic permutation} \,\}.$$

We get a section of P over X_1 by induction hypothesis by considering the family of the images of f_x.

We get a section of P over X_2 by first choosing a cycle of f_x (there are fewer then n cycles because there are no 1-cycles), and then choosing an element of the chosen cycle.

We get a section of P over X_3 by the assumption applied to the map $X_3 \to \mathrm{BC}_n$ induced by equipping each $P(x)$ with the cyclic order determined by the cyclic permutation f_x. □

[25] Andrzej Mostowski. "Axiom of choice for finite sets". In: *Fund. Math.* 33 (1945), pp. 137–168. DOI: 10.4064/fm-33-1-137-168.

[TODO: State the general positive result due to Mostowski[25], maybe as an exercise and give references to the negative results, due to Gauntt (unpublished).]

Bibliography

AKS2015 Ahrens, Benedikt, Krzysztof Kapulkin, and Michael Shulman. "Univalent categories and the Rezk completion". In: *Math. Structures Comput. Sci.* 25.5 (2015), pp. 1010–1039. ISSN: 0960-1295. DOI: 10.1017/S0960129514000486 (pages 150, 162).

Brouwer-1908 Atten, Mark van and Göran Sundholm. "L.E.J. Brouwer's 'Unreliability of the Logical Principles A New Translation, with an Introduction". In: *History and Philosophy of Logic* 38.1 (2017), pp. 24–47. DOI: 10.1080/01445340.2016.1210986. arXiv: 1511.01113 (page 45).

AwodeyCat Awodey, Steve. *Category theory*. Second. Vol. 52. Oxford Logic Guides. Oxford University Press, Oxford, 2010, pp. xvi+311. ISBN: 978-0-19-923718-0 (page 150).

Baez-Shulman Baez, John C. and Michael Shulman. "Lectures on n-categories and cohomology". In: *Towards higher categories*. Vol. 152. IMA Vol. Math. Appl. Springer, New York, 2010, pp. 1–68. DOI: 10.1007/978-1-4419-1524-5_1. arXiv: math/0608420 (page 62).

BezCoqHub Bezem, Marc, Thierry Coquand, and Simon Huber. "A model of type theory in cubical sets". In: *19th International Conference on Types for Proofs and Programs*. Vol. 26. LIPIcs. Leibniz Int. Proc. Inform. Schloss Dagstuhl. Leibniz-Zent. Inform., Wadern, 2014, pp. 107–128. DOI: 10.4230/LIPIcs.TYPES.2013.107 (page 278).

Bishop Bishop, Errett. *Foundations of constructive analysis*. McGraw-Hill Book Co., New York, 1967, pp. xiii+370 (page 282).

Blass-Finite-Choice Blass, Andreas. "Cohomology detects failures of the axiom of choice". In: *Trans. Amer. Math. Soc.* 279.1 (1983), pp. 257–269. DOI: 10.2307/1999384 (pages 283, 285).

BdJR2025 Buchholtz, Ulrik, Tom de Jong, and Egbert Rijke. "On epimorphisms and acyclic types in univalent mathematics". In: *The Journal of Symbolic Logic* (2025), pp. 1–36. DOI: 10.1017/jsl.2024.76. arXiv: 2401.14106 (page 156).

BCFR Buchholtz, Ulrik et al. "Central H-spaces and banded types". 2023. arXiv: 2301.02636 (page 248).

Connes1983 Connes, Alain. "Cohomologie cyclique et foncteurs Ext^n". In: *C. R. Acad. Sci. Paris Sér. I Math.* 296.23 (1983), pp. 953–958 (page 80).

Coq Coq Development Team, The. *The Coq Proof Assistant*. Available at https://coq.inria.fr/ (page 280).

sep-type-theory Coquand, Thierry. "Type Theory". In: *The Stanford Encyclopedia of Philosophy*. Ed. by Edward N. Zalta. Metaphysics Research Lab, Stanford University, 2018. URL: https://plato.stanford.edu/archives/fall2018/entries/type-theory/ (page 13).

Diaconescu Diaconescu, Radu. "Axiom of choice and complementation". In: *Proc. Amer. Math. Soc.* 51 (1975), pp. 176–178 (page 282).

Douglas1951I Douglas, Jesse. "On finite groups with two independent generators. I–IV". In: *Proc. Nat. Acad. Sci. U.S.A.* 37 (1951), pp. 604–610, 677–691, 749–760, 808–813. DOI: 10.1073/pnas.37.9.604 (page 117). DOI: 10.1073/pnas.37.10.677. DOI: 10.1073/pnas.37.11.749. DOI: 10.1073/pnas.37.12.808.

Douglas1961 — "On the supersolvability of bicyclic groups". In: *Proc. Nat. Acad. Sci. U.S.A.* 47 (1961), pp. 1493–1495. DOI: 10.1073/pnas.47.9.1493 (page 117).

EscardoFactorial Escardó, Martín. *UF-Factorial*. Agda formalization. 2019. URL: https://www.cs.bham.ac.uk/~mhe/TypeTopology/UF-Factorial.html (page 88).

Franzen-Goedel Franzén, Torkel. *Gödel's Theorem*: *An Incomplete Guide to Its Use and Abuse*. A. K. Peters, 2005, pp. x+172 (page 278).

Freyd1964

Freyd, Peter. *Abelian categories. An introduction to the theory of functors*. Harper's Series in Modern Mathematics. Harper & Row, Publishers, New York, 1964, pp. xi+164 (page 158).

FreydStreet1995

Freyd, Peter and Ross Street. "On the size of categories". In: *Theory Appl. Categ.* 1 (1995), No. 9, 174–178. ISSN: 1201-561X. URL: http://www.tac.mta.ca/tac/volumes/1995/n9/1-09abs.html (page 164).

Furstenberg

Furstenberg, Harry. "The inverse operation in groups". In: *Proc. Amer. Math. Soc.* 6 (1955), pp. 991–997. DOI: 10.2307/2033124 (page 171).

giraud1971

Giraud, Jean. *Cohomologie non abélienne*. Die Grundlehren der mathematischen Wissenschaften, Band 179. Springer-Verlag, Berlin-New York, 1971, pp. ix+467 (page 142).

Goedel2nd

Gödel, Kurt. "Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I". In: *Monatsh. Math. Phys.* 38.1 (1931), pp. 173–198. DOI: 10.1007/BF01700692 (page 278).

Goodman-Myhill

Goodman, N. and J. Myhill. "Choice implies excluded middle". In: *Z. Math. Logik Grundlagen Math.* 24.5 (1978), p. 461. DOI: 10.1002/malq.19780242514 (page 282).

Hatcher-AlgTop

Hatcher, Allen. *Algebraic Topology*. Cambridge University Press, 2001, pp. xii+551. ISBN: 978-0-521-79540-1. URL: https://pi.math.cornell.edu/~hatcher/AT/AT.pdf (page 281).

Heijenoort-source

Heijenoort, Jean van. *From Frege to Gödel*: *A Source Book in Mathematical Logic, 1879–1931*. Source Books in the History of the Sciences. Harvard University Press, 2002, pp. xii+661 (page 278).

Jech-AC

Jech, Thomas J. *The axiom of choice*. Studies in Logic and the Foundations of Mathematics, Vol. 75. North-Holland Publishing Co., Amsterdam, 1973, pp. xi+202 (page 281).

Jordan

Jordan, Camille. *Traité des substitutions et des équations algébriques*. Les Grands Classiques Gauthier-Villars. Reprint of the 1870 original. Éditions Jacques Gabay, Sceaux, 1989, pp. xvi+670 (page 277).

KapLum

Kapulkin, Krzysztof and Peter LeFanu Lumsdaine. "The simplicial model of Univalent Foundations (after Voevodsky)". In: *Journal of the European Mathematical Society* 23.6 (Mar. 2021), pp. 2071–2126. DOI: 10.4171/jems/1050 (page 278).

Klein-EP-de

Klein, Felix. "Vergleichende Betrachtungen über neuere geometrische Forschungen". In: *Math. Ann.* 43.1 (1893), pp. 63–100. DOI: 10.1007/BF01446615 (page 277).

Kleiner-group-survey

Kleiner, Israel. "The evolution of group theory: a brief survey". In: *Math. Mag.* 59.4 (1986), pp. 195–215. DOI: 10.2307/2690312 (page 277).

Kuperberg1996

Kuperberg, Greg. "Noninvolutory Hopf algebras and 3-manifold invariants". In: *Duke Math. J.* 84.1 (1996), pp. 83–129. DOI: 10.1215/S0012-7094-96-08403-3 (page 115).

LandInftyCat

Land, Markus. *Introduction to Infinity-Categories*. Compact Textbooks in Mathematics. Birkhäuser/Springer, Cham, 2021, pp. ix+296. ISBN: 978-3-030-61524-6. DOI: 10.1007/978-3-030-61524-6 (page 155).

LurieHTT

Lurie, Jacob. *Higher topos theory*. Vol. 170. Annals of Mathematics Studies. Princeton, NJ: Princeton University Press, 2009, pp. xviii+925. ISBN: 9781400830558. DOI: 10.1515/9781400830558 (page 155).

MacLaneWorking

Mac Lane, Saunders. *Categories for the working mathematician*. Second. Vol. 5. Graduate Texts in Mathematics. Springer-Verlag, New York, 1998, pp. xii+314. ISBN: 0-387-98403-8 (page 150).

MangelRijke2023

Mangel, Éléonore and Egbert Rijke. *Delooping the sign homomorphism in univalent mathematics*. 2023. arXiv: 2301.10011 [math.GR] (page 113).

May-Concise

May, J. P. *A concise course in algebraic topology*. Chicago Lectures in Mathematics. University of Chicago Press, Chicago, IL, 1999 (page 281).

MayPonto-MoreConcise

May, J. P. and K. Ponto. *More concise algebraic topology*. Chicago Lectures in Mathematics. Localization, completion, and model categories. University of Chicago Press, Chicago, IL, 2012 (page 281).

Mostowski-Finite-Choice

Mostowski, Andrzej. "Axiom of choice for finite sets". In: *Fund. Math.* 33 (1945), pp. 137–168. DOI: 10.4064/fm-33-1-137-168 (page 285).

peano-principia

Peano, Giuseppe. *Arithmetices principia*: *nova methodo*. See also https://github.com/mdnahas/Peano_Book/ for a parallel translation by Vincent Verheyen. Fratres Bocca, 1889. URL: https://books.google.com/books?id=z80GAAAAYAAJ (page 13).

Pruefer-AG

Prüfer, Heinz. "Theorie der Abelschen Gruppen". In: *Math. Z.* 20.1 (1924), pp. 165–187. DOI: 10.1007/BF01188079 (page 183).

WhetstoneOfWitte

Recorde, Robert and John Kingston. *The whetstone of witte*: *whiche is the seconde parte of Arithmetike, containyng thextraction of rootes, the cossike practise, with the rule of equation, and the woorkes of surde nombers*. Imprynted

at London: By Ihon Kyngstone, 1557. URL: https://archive.org/details/TheWhetstoneOfWitte (page 35).

Riehl, Emily. *Category Theory in Context*. Aurora: Modern Math Originals. Dover Publications, 2016. URL: https://math.jhu.edu/~eriehl/context/ (pages 78, 150).

Rijke, Egbert. *Introduction to Homotopy Type Theory*. Forthcoming book with CUP. Version from 06/02/22. 2022 (page 62).

— *The join construction*. 2017. arXiv: 1701.07538 (pages 45, 62).

Russell, Bertrand. *Introduction to mathematical philosophy*. 2nd Ed. Dover Publications, Inc., New York, 1993, pp. viii+208 (page 54).

Shulman, Michael. "Brouwer's fixed-point theorem in real-cohesive homotopy type theory". In: *Mathematical Structures in Computer Science* 28.6 (2018), pp. 856–941. DOI: 10.1017/S0960129517000147. arXiv: 1509.07584 (page 281).

Smullyan, Raymond M. *Gödel's incompleteness theorems*. Vol. 19. Oxford Logic Guides. The Clarendon Press, Oxford University Press, New York, 1992, pp. xvi+139 (page 278).

Stallings, John R. "Foldings of G-trees". In: *Arboreal group theory (Berkeley, CA, 1988)*. Vol. 19. Math. Sci. Res. Inst. Publ. Springer, New York, 1991, pp. 355–368. DOI: 10.1007/978-1-4612-3142-4_14 (page 239).

Swan, Andrew W. "On the Nielsen–Schreier Theorem in Homotopy Type Theory". In: *Log. Methods Comput. Sci.* 18.1 (2022). DOI: 10.46298/lmcs-18(1:18)2022 (pages 236, 238).

Trimble, Todd. *Monomorphisms in the category of groups*. https://ncatlab.org/toddtrimble/published/monomorphisms+in+the+category+of+groups. Jan. 2020 (page 200).

Univalent Foundations Program, The. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study: https://homotopytypetheory.org/book, 2013 (pages 24, 26, 48, 50, 51, 65, 150, 159).

Wärn, David. *Eilenberg-MacLane spaces and stabilisation in homotopy type theory*. 2023. arXiv: 2301.03685 [math.AT] (page 247).

— *Path spaces of pushouts*. Preprint. 2023. URL: https://dwarn.se/po-paths.pdf (page 195).

Wussing, Hans. *The genesis of the abstract group concept*. A contribution to the history of the origin of abstract group theory, Translated from the German by Abe Shenitzer and Hardy Grant. MIT Press, Cambridge, MA, 1984, p. 331 (page 277).

Yaglom, I. M. *Felix Klein and Sophus Lie. Evolution of the idea of symmetry in the nineteenth century.* Transl. from the Russian by Sergei Sossinsky. Ed. by Hardy Grant and Abe Shenitzer. Birkhäuser Boston, Inc., Boston, MA, 1988, pp. xii+237 (page 277).

RiehlContext
Rijke-Intro
Rijke-Join
russell-intro-mp
Shulman-Real-Cohesive
Smullyan-Goedel
Stallings1991
Swan2022
TrimbleEpiSurjective
hottbook
Warn-EM
Warn2023
Wussing-genesis
Yaglom1988

Glossary

! · placeholder for an element (proof) of a proposition, 76
$\emptyset$ · the empty type, Section 2.12.1, 29
p^{-1} · reverse identification, path inverse, Definition 2.5.1, 17
X^* · list of elements of X, 31
$\bar{f}$ · path obtained by univalence from f, 33
$A_\div$ · underlying type of a pointed type A, 48
$\neg P$ · negation of a proposition P, 35
$-z$ · negation of an integer z, 67
$[x]$ · orbit through $x : X(\mathrm{sh}_G)$, 136
$[u]_0$ · orbit through $u : X_{hG}$, 135
A_+ · A together with a disjoint base point, 48
$\sqrt[m]{t}$ · m^{th} root function on cycles, 89
$\tilde{S}$ · signed version of the set S, 195
$\mathbb{1}$ · trivial group, Example 4.2.20(1), 103
$\tilde{p}$ · transport between types along a path, 33
$|t|$ · constructor for the propositional truncation $\|T\|$ applied to $t : T$, 40
$\|T\|$ · propositional truncation of a type T, 39
: · element judgment, 9
:= · identification in a definition, 65
:≡ · definition, 12
= · equality, 35
$\xrightarrow{=}$ · identity type, Item (E1), 15
≡ · equality by definition, 12
$y \xrightarrow[p]{=} y'$ · path-over type, Definition 2.7.1, 20
LL · binary sum, 30
∘ · function composition, 12
$p * q, q \cdot p, qp, q \circ p$ · path concatenation or composition, 17
⊣ · adjunction, 160
$\xrightarrow{\simeq}$ · type of equivalences, 24
$\exists_{x:X} P(x)$ · proposition expressing existential quantification, 40
→ · function type, 10
↦ · "maps to", function definition, 12
$\xrightarrow{\cong}$ · type of isomorphisms, 152
$\{ t : T \mid P(t) \}$ · set comprehension, 48
− · subtraction of integers, 67
× · cartesian product of two types, 28
∨ · disjunction of propositions, 40
$G \ltimes H$ · semidirect product group, Definition 8.2.1, 184
ε · the empty list, Definition 2.12.11, 31
η · η-rule, 12
ι_+ · embedding of $\mathbb{N}$ into Z, 67
ι_- · embedding of $\mathbb{N}^-$ into Z, 67

$\prod_{x:X} T(x)$ · product type of dependent functions, 10
ρ_m · formal m^{th} root function, Definition 3.8.3, 89
$\sum_{x:X} Y(x)$ · sum type, of dependent pairs (x, y), 22
Σ_n · symmetric group of degree n, Example 4.2.20(2), 103
Σ_S · permutation group on a set S, Example 4.2.20(3), 103
Ωk · loop map of pointed map, Definition 4.4.3, 109
ΩX · type of symmetries (loops) in pointed type, Definition 4.2.10, 101
$\underline{\Omega}$ · group constructor, Definition 4.2.8, 101
$\underline{\Omega}$ · homomorphism constructor, Definition 4.4.2, 108
0 · the natural number zero, Peano's rules, Item (P2), 13
1 · the natural number 1, 14
2 · the natural number 2, 14
3 · the natural number 3, 14
abs(G) · the abstract group of symmetries in a group G, Definition 4.3.4, 107
A_n · alternating group of degree n, Definition 4.5.7, 115
ap_f · application of f to a path, Definition 2.6.1, 19
$f(p)$ · application of f to the path p, Definition 2.6.1, 19
apd_f · application of a dependent function to a path, Definition 2.7.6, 21
$f(p)$ · application of dependent f to the path p Definition 2.7.6, 21
$\mathrm{Aut}_A(a)$ · automorphism group of the element a in the type A, Definition 4.2.15, 102
Bicyc · the type of bicycles, Definition 4.6.1, 117
C_m · cyclic group of degree m, Example 4.2.22, 103
Cay(G; S) · Cayley graph of a group G with respect to S, 234
coker(f) · cokernel of a homomorphism f, 201
conj^g · conjugation by g, 170
cst_t^X · constant function at $t : T$, 11
Cyc · the type of cycles, Definition 3.6.3, 80
Cyc_0 · the type of infinite cycles, Definition 3.8.1, 88
Cyc_m · the type of cycles of order $m > 0$, Definition 3.8.1, 88
D_n · dihedral group of degree n, 185
D_∞ · infinite dihedral group, Definition 4.6.3, 117
E · equivalence from Mono(G) to Sub(G), 131
Epi(G) · type of epimorphisms from the group G, 198
False · the empty type, Section 2.12.1, 29
FinSet · the groupoid of finite sets, 58
FinSet_n · the groupoid of sets of cardinality n, 58
F_S · free group on a decidable set of generators, 194
Group · type of groups, 101
$\mathrm{Group}^{\mathrm{abs}}$ · type of abstract groups, 169
$\mathcal{G}$-$\mathrm{Set}^{\mathrm{abs}}$ · type of $\mathcal{G}$-sets, 173
G-Set · type of G-sets, 123
X_P · underlying G-set of P, 125
Hom(G, H) · type of group homomorphisms, 108
$\mathrm{Hom}^{\mathrm{abs}}(\mathcal{G}, \mathcal{H})$ · type of abstract homomorphisms, 171
$\mathrm{Hom}_G(X, Y)$ · type of maps of G-sets, 124
$\mathrm{Hom}_{\mathcal{G}}(\mathcal{X}, \mathcal{Y})$ · type of maps of $\mathcal{G}$-sets, 176
id · identity function, 12
im(f) · the (propositional) image of f, 43
Im(f) · image group of homomorphism f, 206
$\mathrm{im}_n(f)$ · the n-image of f, 93
in · inclusion into wrapped copy, 31

InfCyc · the type of infinite cycles, Definition 3.5.3, 77
Inj(T) · type of injections into T, 47
inl · inclusion of left summand, 30
inn · homomorphism from G to its inner automorphisms, Definition 4.4.22, 113
inr · inclusion of right summand, 30
isMono(i) · proposition stating that i is a monomorphism of groups, 131
Ker(f) · the kernel group of the homomorphism f, 201
ker(f) · the inclusion of the kernel group of f into its codomain, 201
$L(\mathcal{C})$ · Rezk completion, Definition 6.7.12, 163
Mono(G) · G-set of monomorphisms into G, 210
Mono(G) · type of monomorphismsinto the group G, 131
Mono(H) · type of monomorphismsinto the group H, 198
$\mathbb{N}$ · the type of natural numbers, Peano's rules, Item (P1), 13
$\mathbb{N}^-$ · the type of negated natural numbers, Example 2.12.9, 31
no · the denying boolean, Section 2.12.1, 29
ns · naturality square, Definition 2.6.5, 20
po_p · convert path over path, Definition 2.7.3, 21
pt_X · base point of a pointed type X, 48
$\mathrm{ptw}_{f,g}$ · equivalence from $f \xrightarrow{=} g$ to pointwise equality, Definition 2.6.4, 20
ptw_* · extension of ptw to pointed maps, Construction 2.21.8, 49
Q_8 · quaternion group, Definition 4.6.3, 117
refl_a · reflexivity, identity type, Item (E2), 15
s · successor function on Z, 67
sgn · sign homomorphism, Definition 4.5.6, 115
Sub(G) · type of subgroups of G, 129
$\mathrm{Sub}_G(X)$ · set of G-subsets of X, 124
succ · the successor function on $\mathbb{N}$, Peano's rules, Item (P3), 13
swap · interchange the elements of Bool, Exercise 2.13.3, 33
Torsor_G · the type of G-torsors, 141
tot(f) · totalization of f, 25
Tot(Y) · the total type $\sum_{x:X} Y(x)$, 22
triv · the element of the unit type, Section 2.12.1, 29
trp_e^T · transport function, Definition 2.5.4, 18
True · the unit type, Section 2.12.1, 29
$\mathrm{ua}_{X,Y}$ · postulated element of the univalence axiom, 33
$\mathbb{P}_{b_0}$ · the type family $b \mapsto (b_0 \xrightarrow{=} b)$, 71
$\mathcal{U}$ · universe, 13
$\mathcal{U}_*^{=1}$ · pointed, connected groupoids, Definition 4.2.5, 101
$\mathcal{U}_*$ · universe of pointed types, Definition 2.21.1, 48
wdg · winding number function, 75
yes · the affirmative boolean, Section 2.12.1, 29
Z · the set of integers, Definition 3.2.1, 67
$\mathbb{Z}$ · group of integers, Example 4.2.18, 102

Index

cardinality, 226

abstract group, 107
action
    coinduced, 143
    free, 138
    induced, 143
    of a group in a type, 127
    of a group on a set, 125
    of a group on an element, 127
    restricted, 143
action type, 134
adjunction, 160
    counit of, 160
    unit of, 160
alternating group, 115
automorphism
    inner, 113
automorphism group, 102, 106

base point, 48
bicycle, 117
binary sum type, 30
binomial coefficient, 88
bound variable, 11

cardinality
    of finite G-set, 125
    of finite group, 102
category, 153
    flagged, 164
    full subcategory, 153
    locally small, 154
    of G-sets, 153
    of groups, 153
    of sets, 153
    opposite, 155
    precategory, 152
    small, 154
    wild category, 153
    wild precategory, 151
classifying map, 108
classifying type, 101

cokernel, 201
composition
    of functions, 12
    of group homomorphisms, 109
    of paths, 17
concatenation
    of paths, 17
congruence, 16
conjugate
    by g, 210
conjugation, 113, 170, 172, 210
connected
    graph, 237
connected set bundle, 68
currying, 26
cycle, 80
    infinite, 77, 88
    of order $m > 0$, 88
cyclic group, 103

decidable
    subgroup, 130
decidable set bundle, 68
decidable proposition, 44
decidable set, 48
definition, 12
degree
    function, 81
designated shape, 101
diagram, 38
    commutative, 38
    commutative by definition, 38
    subtype, 70
    to fill a, 39
dihedral group, 185
disjunction, 40
dummy variable, 11

element, 9
empty type, 29
epimorphism
    of groups, 198
equation, 35

equivalence
of categories, 162
equivalence relation
induced by map, 54
even, 115
exists, 40

factorial function, 15
family
of elements, 10
of types, 9
fiber, 23
fiber sequence, 23
filler
of a diagram, 39
finite G-set, 125
finite set bundle, 68
finite group, 102, 226
fixed, 138
flagged category, 164
flattening construction, 235
forget, 62
higher structure, 62
properties, 62
structure, 62
free, 138
free group, 194
function, 10
n-connected, 93
n-truncated, 93
constant, 11
degree m, 81
factorial, 15
identity, 12
function extensionality, 20
principle of, 26
functor, 156
representable, 162
functor category, 159
functor precategory, *see* functor category

graph
Cayley graph, 234
labeled, 234
group, 101
abstract, 107
acting on a set, 122
alternating group, 115
binary product, 105
cyclic group, 103
dihedral group, 185
finite, 102
infinite dihedral group, 117, 185
Klein four-group, 105
of automorphisms, 102, 106
of integers, 102
permutation group, 103
quaternion group, 117
semidirect product, 184
stabilizer, 137
symmetric group, 103
group action
of G-set, 122
groupoid, 35
G-set (of group), 123
$\mathcal{G}$-set (of abstract group), 173
G-subset, 124
$\mathcal{G}$-subset, 176

Hedberg's theorem, 48
homomorphism, 108
of abstract group, 171
of groups, 108

identification, 15
identity type, 15
image, 63
function, 209
projection to, 209
image group, 208
induced action, 143
induction principle, 14
infinite dihedral group, 117, 185
initial object, 155
injection, 42
into a type, 47
injective, 42
intersection
of monomorphisms, 190
of sets, 189
invariant map type, 135
isomorphism
in a (wild pre-)category, 152
of abstract groups, 170
of groups, 109
iteration, 67

kernel, 201
associated to, 215
group, 201
Klein four-group, 105

Law of Excluded Middle, 44

LEM, *see* Law of Excluded Middle
Limited Principle of Omniscience, 85
list
  head, 31
  tail, 31
list type, 31
loop type constructor, 101
LPO, *see* Limited Principle of Omniscience

map, 10
  of G-sets, 124
  of $\mathcal{G}$-sets, 176
mathematics
  univalent, 9
merely, 41
monoid, 168
monomorphism, 131
  of groups, 198

natural transformation, 158
naturality square, 20
negation
  of integer, 67
neutral element, 107, 168
normal subgroup, 211
normalizer, 220

obelus, 48
odd, 115
orbit set, 135
ordering
  local, 115
  sign, 115

parameter type, 10, 19
partial order, 154
path over, 20
permutation
  even, 115
permutation group, 103
Pi type, 19
pointed type, 48
pointing path, 48
poset, 154
predicate, 46
preimage, 23
preorder, 153
principle
  induction, 14
  recursion, 14
product type, 10, 19
proof
  of a proposition, 34
proper
  epimorphism, 198
  monomorphism, 198
proper monomorphism, 131
proper subgroup, 133
proposition, 34
Propositional resizing, 45
propositional truncation, 39
pullback, 188
  in types, 166
  of groups, 190
pullback diagram, 166, 189

quaternion group, 117
quotient group, 213
quotient homomorphism, 213

recursion principle, 14
Replacement principle, 45
restriction, 143
Rezk completion, 163

semidirect product, 184
set, 35
  of kernels, 212
  comprehension, 48
  of orbits, 135
shape, 101
sigma type, 22
sign, 115
sign ordering, 115
signed set, 195
sink
  of a diagram, 38
source
  of a diagram, 38
stabilizer, 137
substitution, 18
subtype, 46, 47
successor, 13
sum of groups, 191
sum type, 22
Sylow subgroup, 230
symmetric group, 103
symmetries in a group G, 101
symmetry
  in a type, 64
  of an element, 16
symmetry type constructor, 101

terminal object, 155

torsor, 141
total type, 22
totalization, 25
transitive G-set, 126
transport, 18
triple, 22
trivial group, 103
trivial monomorphism, 131
trivial subgroup, 132
truncation
  propositional, 39
tuple, 23
type, 8
  n-connected, 93
  n-truncated, 36
  of epimorphisms from a groups, 198
  of monomorphisms into a group, 131, 198
  of normal subgroups, 211
  binary product, 28
  binary sum, 30
  cartesian product, 28
  empty, 29
  of abstract groups, 169
  of abstract homomorphisms, 171
  of booleans, 29
  of groups, 101
  of lists, 31
  of subgroups of a group, 129
  propositional truncation, 39
  Sigma, 22
  sum, 22
  unary sum, 31
  unit, 29

unary sum type, 31
underscore, 11
union of sets, 189
unit type, 29
univalence axiom, 33
univalent, 153
universe, 13

Vierergruppe, 105

wedge of pointed types, 191
Weyl group, 220
wrapped copy type, 31

zero, 13